
The RBioc Book

SEAN DAVIS

University of ColoradoAnschutz School of Medicine

2026-06-01

Table of contents

Preface	1
What are the goals?	2
Adult learners	2
AI is part of the solution	4
I. Introduction	6
1. About R	7
1.1. What you'll learn	7
1.2. What is R?	7
1.3. Why use R?	9
1.4. Why not use R?	10
1.5. R license and the open source ideal	10
1.6. Where this is going	11
1.7. Summary	11
1.8. Reflection	11
2. RStudio	13
2.1. Getting started with RStudio	13
2.2. The RStudio Interface	13
2.3. Alternatives to RStudio	16
3. R mechanics	19
3.1. Starting R	19
3.2. <i>RStudio</i> : A Quick Tour	19
3.3. Interacting with R	19
3.3.1. Expressions	20
3.3.2. Assignment	21
3.4. Rules for Names in R	23
3.5. About R functions	23
3.6. Resources for Getting Help	24
3.7. Reflection	24

Table of contents

4. Up and Running with R	25
4.1. The R User Interface	25
4.1.1. An exercise	29
4.2. Objects	29
4.3. Functions	37
4.3.1. Sample with Replacement	41
4.4. Writing Your Own Functions	43
4.4.1. The Function Constructor	44
4.5. Arguments	46
4.6. Scripts	49
4.7. Summary	50
5. Packages and more dice	51
5.1. Packages	51
5.1.1. Installing R packages	52
5.1.2. Installing vs loading (library) R packages	52
5.1.3. Finding R packages	53
5.1.4. Creating a package	54
5.2. Are our dice fair?	54
5.3. Bonus exercise	57
6. Reading and writing data files	58
6.1. Introduction	58
6.2. RStudio Projects: Organizing Your Work	58
6.2.1. What are RStudio Projects?	58
6.2.2. Creating an RStudio Project	59
6.2.3. Project Structure Best Practices	59
6.2.4. Working Directories and File Paths	60
6.2.5. The <code>here</code> Package for Robust File Paths [optional]	60
6.2.6. Projects and Reproducibility	60
6.3. CSV files	61
6.3.1. Writing a CSV file	61
6.3.2. Reading a CSV file	62
6.4. Excel files	64
6.4.1. Reading an Excel file	64
6.4.2. Writing an Excel file	68
6.5. Additional options	69
II. R Data Structures	70
Chapter overview	71

Table of contents

7. Vectors	74
7.1. What you'll learn	74
7.2. A vector is a column of measurements	74
7.3. Creating vectors	76
7.4. Vector operations	78
7.5. Logical vectors	79
7.5.1. Logical operators	80
7.6. Indexing vectors	81
7.7. Named vectors	83
7.8. Character vectors, a.k.a. strings	84
7.9. Missing values, a.k.a. NA	86
7.10. Exercises	87
7.11. Summary	90
8. Matrices	91
8.1. What you'll learn	92
8.2. Creating a matrix	92
8.3. Accessing elements of a matrix	95
8.4. Changing values in a matrix	97
8.5. Calculations on matrix rows and columns	99
8.6. Exercises	101
8.7. Summary	110
9. Lists: a catch-all container	111
9.1. What you'll learn	112
9.2. Creating a list	112
9.3. Inspecting your list: what's inside?	113
9.3.1. <code>str()</code> : the structure function	113
9.3.2. <code>length()</code> , <code>names()</code> , and <code>class()</code>	114
9.4. Accessing list elements: getting things out	115
9.4.1. The mighty <code>[[]]</code> and <code>\$</code> for single items	115
9.4.2. The subsetting <code>[]</code> for new lists	116
9.5. Modifying lists	117
9.5.1. Adding and updating elements	117
9.5.2. Removing elements	118
9.6. A biological example: a self-contained gene record	119
9.7. Exercises	120
9.8. Summary	121
10. Data Frames	123
10.1. Learning goals	123

Table of contents

10.2. Learning objectives	123
10.3. Dataset	123
10.4. Reading in data	124
10.5. Inspecting data.frames	125
10.6. Accessing variables (columns) and subsetting	128
10.6.1. Some data exploration	130
10.6.2. More advanced indexing and subsetting	131
10.7. Aggregating data	134
10.8. Creating a data.frame from scratch	135
10.9. Saving a data.frame	136
11. Factors	137
11.1. What you'll learn	137
11.2. A vector with a built-in dictionary	137
11.3. Counting categories with <code>table()</code>	139
11.4. Converting a factor back	139
11.5. Controlling the levels	140
11.6. Exercises	141
11.7. Summary	143
III. Exploratory data analysis	144
12. Introduction to dplyr: mammal sleep dataset	146
12.1. Learning goals	146
12.2. Learning objectives	146
12.3. What is dplyr?	147
12.4. Why Is dplyr useful?	147
12.5. Data: Mammals Sleep	147
12.6. dplyr verbs	148
12.7. Using the dplyr verbs	148
12.7.1. Selecting columns: <code>select()</code>	149
12.7.2. Selecting rows: <code>filter()</code>	151
12.8. "Piping" with <code> ></code>	153
12.8.1. Arrange Or Re-order Rows Using <code>arrange()</code>	154
12.9. Create New Columns Using <code>mutate()</code>	156
12.9.1. Create summaries: <code>summarise()</code>	157
12.10 Grouping data: <code>group_by()</code>	158
13. Case Study: Behavioral Risk Factor Surveillance System	159
13.1. What you'll learn	159

Table of contents

13.2. Loading the dataset	160
13.3. Inspecting the data	160
13.4. Summary statistics	161
13.5. Visualizing single variables	162
13.6. Analyzing relationships between variables	164
13.7. Comparing two groups: a t-test	166
13.8. Fitting a model: weight and height	168
13.9. Exercises	174
13.10. Summary	177
14. Exploring data with ggplot2	179
14.1. ggplot()	180
14.2. geom_*()	181
14.3. Grouping	184
14.4. Scales	185
14.5. Facets	187
14.6. Labels and Titles	188
14.7. Theming	190
14.8. Conclusion	191
15. A self-guided tour of data visualization in R	193
15.1. What you'll learn	193
15.2. Two principles that make any plot better	193
15.2.1. Maximize the data-ink ratio	194
15.2.2. Label everything	197
15.2.3. Use color deliberately	198
15.3. The grammar of graphics, layer by layer	202
15.3.1. Step 1: data and aesthetics	203
15.3.2. Step 2: add a geom	204
15.3.3. Step 3: map a third variable to color	205
15.3.4. Step 4: layer on a second geom	206
15.3.5. Step 5: control scales	207
15.3.6. Step 6: facet into small multiples	208
15.3.7. Step 7: labels and theme	209
15.4. Specialized plots for biology	210
15.4.1. Sets and intersections: UpSet plots	210
15.4.2. Heatmaps for data matrices	211
15.4.3. Visualizing the genome (further exploration)	214
15.5. Exercises	214
15.6. Summary	217

IV. statistics	219
16. Working with distribution functions	220
16.1. What you'll learn	220
16.2. pnorm	221
16.3. dnorm	226
16.4. qnorm	226
16.5. rnorm	230
16.6. Worked example: IQ scores	236
16.7. Exercises	236
16.8. Summary	237
17. The t-statistic and t-distribution	239
17.1. What you'll learn	239
17.2. Background	239
17.3. The Z-score and probability	240
17.3.1. Small diversion: a two-sided pnorm	242
17.4. The t-distribution	243
17.4.1. p-values based on Z vs t	246
17.4.2. Experiment: does the choice of distribution actually matter?	247
17.5. Checkpoint: t vs normal	251
17.6. t.test	251
17.6.1. One-sample	252
17.6.2. Two-sample	253
17.6.3. From a data frame	253
17.6.4. Equivalence to a linear model	255
17.7. Power calculations	256
17.8. Exercises	259
17.9. Summary	261
17.10Resources	262
18. K-means clustering	263
18.1. What you'll learn	263
18.2. What k-means does	263
18.3. The k-means algorithm	264
18.4. Pros and cons of k-means clustering	265
18.5. A worked example: the diauxic shift	266
18.5.1. The data and experimental background	266
18.5.2. Getting the data	266
18.5.3. Preprocessing: keep only the informative genes	267
18.5.4. Clustering	270

Table of contents

18.5.5. Reading the clusters as biology	271
18.6. Exercises	272
18.7. Summary	275
V. Machine Learning	276
19. Introduction	277
19.1. Types of Machine Learning	277
19.2. The Machine Learning Workflow	280
19.3. Understanding Overfitting and Underfitting	283
19.4. Cross-Validation and Model Selection	285
20. Supervised Learning	287
20.1. What you'll learn	287
20.2. A framework for understanding supervised algorithms	288
20.3. Our shared dataset: Palmer penguins	288
20.4. Linear regression	290
20.5. K-nearest neighbours	291
20.6. Penalized regression	292
20.6.1. Ridge regression	292
20.6.2. LASSO regression	293
20.6.3. Elastic net	294
20.7. Classification and regression trees (CART)	295
20.8. Random forests	298
20.9. Summary	299
20.10 Exercises	300
21. The mlr3verse	303
21.1. The Philosophy of mlr3verse	303
21.2. The mlr3verse Ecosystem	304
21.3. Core mlr3 Concepts and Objects	305
22. Examples	309
22.1. Overview	309
22.1.1. Key features of mlr3	309
22.2. The mlr3 workflow	311
22.2.1. The machine learning Task	313
22.2.2. The “Learner” in Machine Learning	315
22.3. Setup	319
22.4. Example: Cancer types	319
22.4.1. Understanding the Problem	319

Table of contents

22.4.2. Data Preparation	320
22.4.3. Feature selection and data cleaning	320
22.4.4. Creating the “task”	321
22.4.5. Splitting the data	322
22.4.6. Example learners	322
22.5. Example Predicting age from DNA methylation	332
22.5.1. Example learners	334
22.6. Example: Expression prediction from histone modification data	343
22.6.1. The Data	343
22.6.2. Create task	348
22.6.3. Example learners	349
VI. Bioconductor	358
23. Accessing and working with public omics data	359
23.1. Background	359
23.2. GEOquery to PCA	360
24. Storing experiments with SummarizedExperiment	364
24.1. What you’ll learn	364
24.2. Anatomy of a SummarizedExperiment	365
24.2.1. Assays	367
24.2.2. ‘Row’ (regions-of-interest) data	368
24.2.3. ‘Column’ (sample) data	369
24.2.4. Experiment-wide metadata	370
24.3. Common operations on SummarizedExperiment	371
24.3.1. Subsetting	371
24.3.2. Getters and setters	372
24.3.3. Range-based operations	374
24.4. Constructing a SummarizedExperiment	374
24.4.1. rowData, or feature information	375
24.4.2. colData, or sample information	376
24.4.3. assays, or the data	376
24.4.4. Putting it all together	378
24.4.5. Getting logRatios	378
24.5. Summary	380
24.6. Exercises	380
25. Microbiome analysis	382
25.1. Getting started	382

Table of contents

25.2. Accessing microbiome datasets	383
25.3. Data containers	384
25.4. Loading a microbiome dataset	384
25.4.1. Experimental measurements: <code>assays</code>	387
25.4.2. Sample information: <code>colData</code>	388
25.4.3. Measured feature information: <code>rowData</code>	391
25.4.4. <code>rowTree</code>	392
25.5. Wrangling and subsetting	395
25.5.1. Subsetting samples	395
25.5.2. Agglomerating data	397
25.6. Community indices	398
25.6.1. Alpha diversity	398
25.6.2. Beta diversity	403
25.7. Microbial composition	405
25.7.1. Abundance	405
25.7.2. Prevalence	408
25.8. Heatmaps	409
25.9. Further fun	409
26. Genomic Ranges Introduction	411
26.1. Introduction	411
26.2. The dataset	411
26.3. The BED File Format	411
26.3.1. BED Format Structure	412
26.3.2. Key Concepts	412
26.4. Loading Required Libraries	412
26.5. Loading CTCF ChIP-seq Data	413
26.6. Understanding GRanges Objects	415
26.7. Exploring Peak Characteristics	418
26.7.1. Basic Peak Statistics	418
26.7.2. Peaks Per Chromosome	420
26.8. Accessing Peak Coordinates	421
26.8.1. Finding Starts and Ends	421
26.9. Manipulating Peak Ranges	421
26.9.1. Shifting Peaks	421
26.9.2. Setting Peak Widths	422
26.9.3. Creating Flanking Regions	424
26.10 Coverage	425
26.11 Key Takeaways	427
26.11.1. Common Use Cases	428
26.11.2. Best Practices	428

Table of contents

27. Genomic ranges and features	429
27.1. What you'll learn	429
27.2. Bioconductor and GenomicRanges	430
27.2.1. Subsetting GRanges objects	439
27.2.2. Interval operations on one GRanges object	440
27.2.3. Set operations for GRanges objects	445
27.3. GRangesList	447
27.3.1. Basic <i>GRangesList</i> accessors	449
27.4. Relationships between region sets	451
27.4.1. Overlaps	451
27.4.2. Nearest feature	455
27.5. Gene models	457
27.6. Summary	461
27.7. Exercises	462
28. Ranges Exercises	465
28.1. Exercise 1	465
28.2. Exercise 2	466
28.3. Exercise 3	467
28.4. Exercise 4	469
29. Genomic ranges and ATAC-Seq	472
<i>R</i> / <i>Bioconductor</i> packages used	472
29.1. Background	472
29.2. Informatics overview	475
29.3. Working with sequencing data in Bioconductor	476
30. Data import and quality control	477
30.1. Coverage	479
30.2. Fragment Lengths	482
30.3. Viewing data in IGV	487
30.4. Additional work	487
MACS2	488
31. Single Cell RNA-seq Analysis	489
31.1. Introduction	489
31.2. Setup and Data Loading	489
31.2.1. Loading Essential Libraries	489
31.2.2. Exploring Available Datasets	490
31.3. Dimensionality reduction	494
31.3.1. Feature Selection: Identifying Highly Variable Genes	495

Table of contents

31.3.2. Non-linear Dimensionality Reduction with t-SNE	497
31.4. Why t-SNE after PCA	497
31.4.1. Visualizing Cell Types and Clusters	498
31.5. Finding Marker Genes	501
31.6. Interactive Visualization with iSEE	503
31.7. Bonus: Working with 10x Genomics Data	504
31.7.1. Understanding 10x Genomics Output Formats	504
31.7.2. Loading 10x Data from Matrix Market Files	504
31.7.3. Loading from HDF5 Files	505
31.7.4. Essential Quality Control After Loading Raw Data	506
31.7.5. Filtering Low-Quality Cells and Genes	506
31.8. Further Reading and Resources	507
References	509
Appendices	512
A. Appendix	512
A.1. Data Sets	512
A.2. Swirl	512
B. Git and GitHub	513
B.1. install Git and GitHub CLI	513
B.2. Configure Git	514
B.3. Create a GitHub account	514
B.4. Login to GitHub CLI	515
B.5. Introduction to Version Control with Git	515
B.5.1. Key Git Commands We'll Learn Today:	515
B.6. The Toy Example: An R Script	516
B.7. Let's Get Started with Git!	516
B.7.1. Step 1: Initialize Your Git Repository	516
B.7.2. Step 2: Your First Commit	518
B.7.3. Step 3: Making and Undoing a Change	518
B.7.4. Step 4: Branching Out	518
B.7.5. Step 5: Seeing Branches in Action	519
B.7.6. Step 6: Merging Your Work	520
C. Additional resources	521
C.1. AI	521

Table of contents

D. AI Tools for Enhanced Learning and Productivity in Bioinformatics	522
D.1. Introduction: AI as Your Bioinformatics Learning Companion	522
D.2. AI for Understanding Statistical Concepts and R Functions	523
D.3. AI for Data Exploration and Visualization Strategies	524
D.4. AI for Boosting Productivity and Troubleshooting	525
D.5. Best Practices for Prompt Engineering	525
D.6. Ethical Considerations and Limitations	526
D.7. The Future: AI in Bioinformatics and Data Science	528
D.8. Practical Exercises and Activities	528
E. Data Visualization with ggplot2	530
F. Matrix Exercises	531
F.1. Data preparation	531
F.2. Exercises	531

List of Figures

1.	Schema theory of learning. While probably not entirely applicable to how adults learn, it does highlight the importance of building on prior knowledge. When new knowledge is introduced, it is more quickly stored into long-term memory if strongly-related concepts are already available for connection. . . .	1
2.	Why do adults choose to learn something?	3
3.	How to stay stuck in data science (or anything). The “Read-Do” loop tends to deliver the best results. Too much reading between doing can be somewhat effective. Reading and simply copy-paste is probably the least effective. When working through material, experiment. Try to break things. Incorporate your own experience or applications whenever possible.	4
1.1.	Worldwide search interest in R and Python over time, from Google Trends. Search interest is a rough proxy for general popularity — it does not capture how heavily specific fields lean on each language. Bioinformatics, in particular, remains deeply invested in R.	8
2.1.	The RStudio interface. In this layout, the source pane is in the upper left, the console is in the lower left, the environment panel is in the top right and the viewer/help/files panel is in the bottom right.	14
2.2.	Dealing with limited screen real estate can be a challenge, particularly when you want to open another window to, for example, view a web page. You can resize the panes by sliding the center divider (red arrows) or by clicking on the minimize/maximize buttons (see blue arrow).	15
2.3.	Jupyter Notebook interface. This is an interactive environment for writing and executing R code, along with rich text support for documentation. . . .	16
2.4.	Visual Studio Code (VSCode) with the R extension. This is a lightweight alternative to RStudio that provides syntax highlighting, code completion, and integrated terminal support.	17
2.5.	Positron Workbench interface. This IDE supports R and Python, providing a similar interface to RStudio with additional features for data science workflows.	18

List of Figures

4.1.	Your computer does your bidding when you type R commands at the prompt in the bottom line of the console pane. Don't forget to hit the Enter key. When you first open RStudio, the console appears in the pane on your left, but you can change this with File > Tools > Global Options in the menu bar.	26
4.2.	Assignment creates an object in the environment pane.	32
4.3.	"When R performs element-wise execution, it matches up vectors and then manipulates each pair of elements independently."	35
4.4.	"R will repeat a short vector to do element-wise operations with two vectors of uneven lengths."	36
4.5.	"When you link functions together, R will resolve them from the innermost operation to the outermost. Here R first looks up die, then calculates the mean of one through six, then rounds the mean."	38
4.6.	"Every function in R has the same parts, and you can use function to create these parts. Assign the result to a name, so you can call the function later."	48
4.7.	"When you open an R Script (File > New File > R Script in the menu bar), RStudio creates a fourth pane (or puts a new tab in the existing pane) above the console where you can write and edit your code."	49
5.1.	Installing vs loading R packages.	53
5.2.	In an ideal world, a histogram of the results would look like this	54
5.3.	Histogram of the sums from 100 rolls of our fair dice	56
5.4.	Histogram with 100000 rolls much more closely approximates the pyramidal shape we anticipated	57
6.1.	A pictorial representation of R's most common data structures are vectors, matrices, arrays, lists, and dataframes. Figure from Hands-on Programming with R.	72
7.1.	"Pictorial representation of three vector examples. The first vector is a numeric vector. The second is a 'logical' vector. The third is a character vector. Vectors also have indices and, optionally, names."	75
8.1.	A matrix is a collection of column vectors, all the same length and the same type.	91
13.1.	Distribution of respondent ages in the BRFSS subset.	163
13.2.	Weight distribution compared between the sexes.	164
13.3.	Scatterplot of age versus weight.	165
13.4.	Female respondent weight in 1990 versus 2010.	167
13.5.	Weight distribution for 2010 male respondents.	169
13.6.	Height distribution for 2010 male respondents.	170

List of Figures

13.7. Weight versus height for 2010 male respondents.	171
13.8. The fitted regression line through the weight-versus-height data, with one point marked in red.	173
13.9. Residuals-versus-fitted diagnostic plot for the weight-versus-height model. .	174
14.1. Initialize the plot with <code>ggplot()</code>	181
14.2. Add points to the plot with <code>geom_point()</code>	182
14.3. Modify point color, size, and transparency.	183
14.4. Add a line of best fit with <code>geom_smooth()</code>	184
14.5. Group points by smoker status.	185
14.6. Modify scales for x and y axes, and colors.	186
14.7. Create facets based on the obese variable.	188
14.8. Add labels and title to the plot.	189
14.9. Customize the plot’s appearance with themes.	191
15.1. Edward Tufte’s landmark book, <i>The Visual Display of Quantitative Infor-</i> <i>mation</i> , popularized the data-ink ratio.	195
15.2. A cluttered plot with a low data-ink ratio. The gray background and heavy gridlines are decoration, not information.	196
15.3. The same data with a high data-ink ratio. The eye goes straight to the points.	197
15.4. RColorBrewer palettes, grouped into sequential (top), qualitative (middle), and diverging (bottom) families.	199
15.5. The subset of RColorBrewer palettes that remain distinguishable for viewers with common color-vision deficiencies.	201
15.6. Data plus aesthetics, but no geom: <code>ggplot()</code> sets up the axes but draws nothing yet.	203
15.7. Adding <code>geom_point()</code> draws one point per penguin.	204
15.8. Mapping species to the color aesthetic reveals three clusters.	205
15.9. A second geom (a linear trend line) layered over the points, one line per species.	206
15.10A Brewer qualitative palette via <code>scale_color_brewer()</code> and tidy axis breaks.	207
15.11Faceting by species puts each species in its own panel for side-by-side com- parison.	208
15.12The finished plot: data, aesthetics, two geoms, a scale, facets, labels, and a clean theme.	209
15.13An UpSet plot of movie genres. Each bar is the size of one intersection; the dots below say which genres it combines. Read it as a scalable Venn diagram.	211
15.14A clustered heatmap of scaled car measurements. Rows (cars) and columns (features) are reordered so similar profiles sit together; the color shows how far above (red) or below (blue) average each value is.	213
15.15Bill length vs. depth, colored by species.	215

List of Figures

15.16	Body-mass distribution by species.	216
15.17	The boxplot, now properly labeled.	217
16.1.	The <code>pnorm</code> function takes a quantile (value on the x-axis) and returns the area under the curve to the left of that value.	222
16.2.	The <code>pnorm</code> function takes a quantile (value on the x-axis) and returns the area under the curve to the left of that value.	223
16.3.	The <code>pnorm</code> function takes a quantile (value on the x-axis) and returns the area under the curve to the left of that value.	224
16.4.	The <code>pnorm</code> function takes a quantile (value on the x-axis) and returns the area under the curve to the left of that value.	225
16.5.	The <code>dnorm</code> function returns the height of the normal distribution at a given point.	227
16.6.	The <code>dnorm</code> function returns the height of the normal distribution at a given point.	228
16.7.	The <code>dnorm</code> function returns the height of the normal distribution at a given point.	229
16.8.	The <code>qnorm</code> function is the inverse of the <code>pnorm</code> function in that it takes a probability and gives the quantile.	230
16.9.	The <code>qnorm</code> function is the inverse of the <code>pnorm</code> function in that it takes a probability and gives the quantile.	231
16.10	The <code>qnorm</code> function is the inverse of the <code>pnorm</code> function in that it takes a probability and gives the quantile.	232
16.11	The <code>qnorm</code> function is the inverse of the <code>pnorm</code> function in that it takes a probability and gives the quantile.	233
16.12	The <code>qnorm</code> function is the inverse of the <code>pnorm</code> function in that it takes a probability and gives the quantile.	234
16.13	The <code>rnorm</code> function takes a number of samples and returns a vector of random numbers from the normal distribution (with mean=0, sd=1 as defaults)	235
17.1.	t-distributions for various degrees of freedom. The tails are fatter for smaller degrees of freedom — the visible cost of estimating the standard deviation from the data.	245
18.1.	K-means clustering takes a dataset and divides it into k clusters.	264
18.2.	Histogram of per-gene standard deviations across the time course. Most genes barely vary (the tall bars on the left); a long right tail marks the genes that change the most.	269

List of Figures

18.3. Expression profiles for the four clusters found by k-means. Each line is one gene; the x-axis runs left to right across the time course, and the y-axis is the (standardized) expression level. Genes within a cluster share a trend, which suggests they may be co-regulated.	271
19.1. A simple view of machine learning according the sklearn.	279
19.2. Data splitting and train/validate/test paradigm. For models without “hyperparameters”, only the training/testing sets are necessary. For models that require learning model <i>structure</i> (such as the k in k-nearest-neighbor) as well as model parameters (like betas in linear regression), a separate validation set is essential.	282
19.3. Data simulated according to the function $f(x) = \sin(2\pi x) + N(0, 0.25)$ fitted with four different models. A) A simple linear model demonstrates <i>underfitting</i> . B) A linear model with a sin function ($y = \sin(2\pi x)$) and C) a loess model with a wide span (0.5) demonstrate <i>good fits</i> . D) A loess model with a narrow span (0.05) is a good example of <i>overfitting</i>	284
20.1. A classification tree for a breast-cancer screening example. Each internal (blue) node tests one feature; following the branch that matches a sample leads, eventually, to a leaf (green/red) that gives the predicted class.	297
20.2. Graphical representation of a random forest: many decision trees, each grown on a slightly different view of the data, combine their votes into one prediction.	298
22.1. The mlr3 ecosystem.	310
22.2. The simplified workflow of a machine learning pipeline using mlr3.	312
22.3. Two stages of a learner. Top: data (features and a target) are passed to an (untrained) learner. Bottom: new data are passed to the trained model which makes predictions for the ‘missing’ target column.	318
22.4. Regression diagnostic plots. The top left plot shows the residuals vs. fitted values. The top right plot shows the normal Q-Q plot. The bottom left plot shows the scale-location plot. The bottom right plot shows the residuals vs. leverage.	335
22.5. What is the combined effect of histone marks on gene expression?	344
22.6. Boxplots of original and scaled data.	346
22.7. Heatmap of 500 randomly sampled rows of the data. Columns are histone marks and there is a row for each gene.	347
24.1. Summarized Experiment. There are three main components, the <code>colData()</code> , the <code>rowData()</code> and the <code>assays()</code> . The accessors for the various parts of a complete SummarizedExperiment object match the names.	366

List of Figures

25.1. Anatomy of a <code>TreeSummarizedExperiment</code> object. Compared to the <code>SingleCellExperiment</code> objects, <code>TreeSummarizedExperiment</code> has five additional slots. rowTree : the hierarchical structure on the rows of the assays. rowLinks : the link information between rows of the assays and the <code>rowTree</code> . colTree : the hierarchical structure on the columns of the assays. colLinks : the link information between columns of the assays and the <code>colTree</code> . referenceSeq (optional): the reference sequence data per feature (row).	386
26.1. Histogram of CTCF Peak Widths. Why is there a large peak at around 200 bp?	416
27.1. The structure of a <code>GRanges</code> object, which behaves a bit like a vector of ranges, although the analogy is not perfect. A <code>GRanges</code> object is composed of the “Ranges” part the lefthand box, the “metadata” columns (the righthand box), and a “seqinfo” part that describes the names and lengths of associated sequences. Only the “Ranges” part is required. The figure also shows a few of the “accessors” and approaches to subsetting a <code>GRanges</code> object.	435
27.2. The structure of a <code>GRangesList</code> , which is a <code>list</code> of <code>GRanges</code> objects. While the analogy is not perfect, a <code>GRangesList</code> behaves a bit like a list. Each element in the <code>GRangesList</code> is a <code>GRanges</code> object. A common use case for a <code>GRangesList</code> is to store a list of transcripts, each of which have exons as the regions in the <code>GRanges</code>	447
27.3. A graphical representation of range operations demonstrated on a gene model.	458
29.1. Chromatin accessibility methods, compared. Representative DNA fragments generated by each assay are shown, with end locations within chromatin defined by colored arrows. Bar diagrams represent data signal obtained from each assay across the entire region. The footprint created by a transcription factor (TF) is shown for ATAC-seq and DNase-seq experiments.	473
29.2. Multimodal chromatin comparisons. From (Buenrostro et al. 2013), Figure 4. (a) CTCF footprints observed in ATAC-seq and DNase-seq data, at a specific locus on chr1. (b) Aggregate ATAC-seq footprint for CTCF (motif shown) generated over binding sites within the genome (c) CTCF predicted binding probability inferred from ATAC-seq data, position weight matrix (PWM) scores for the CTCF motif, and evolutionary conservation (PhyloP). Right-most column is the CTCF ChIP-seq data (ENCODE) for this GM12878 cell line, demonstrating high concordance with predicted binding probability.	474

List of Figures

29.3. A BAM file in text form. The output of <code>samtools view</code> is the text format of the BAM file (called SAM format). Bioconductor and many other tools use BAM files for input. Note that BAM files also often include an index <code>.bai</code> file that enables random access into the file; one can read just a genomic region without having to read the entire file.	475
30.1. Reads per chromosome. In our example data, we are using only chromosomes 21 and 22.	478
30.2. Read counts normalized by chromosome length. This is not a particularly important plot, but it can be useful to see the relative contribution of each chromosome given its length.	479
30.3. Relationship between fragment length and nucleosome number.	483
30.4. Fragment length histogram.	484
30.5. Enrichment of nucleosome-free reads and depletion of mononucleosome reads around the TSS.	486
30.6. Comparison of signals at TSS. Mononucleosome data on the left, nucleosome-free on the right. Both plots are scaled to the same range.	487
31.1. The <code>SingleCellExperiment</code> object.	493
31.2. A plot of the mean vs variance of log-expression for all the genes in the dataset, with the highly variable genes highlighted in red. Here, the x-axis represents the mean log-expression of each gene, and the y-axis represents the variance of log-expression. The blue curve represents the trend of variance as a function of mean expression, while the red points represent the highly variable genes.	496
31.3.	499
31.4.	500
31.5.	501
B.1. This is an overview of how git works along with the commands that make it tick. See this video	517

List of Tables

7.1. Atomic (simplest) data types in R.	76
16.1. Functions for the normal distribution	221
27.1. Classes within the GenomicRanges package. Each class has a slightly different use case.	431
27.2. Methods for accessing, manipulating single objects	432
27.3. Methods for comparing and combining multiple GenomicRanges-class objects	433
29.1. Commonly used Bioconductor and their high-level use cases.	476
D.1. Table AI Tools for Bioinformatics Learning	522

Preface

The goal of the material is NOT to teach you R or Bioconductor, but rather to provide enough groundwork to enable you to learn R and Bioconductor on your own. The idea is that we learn faster and better by developing “schemata” that allow us to organize and understand new information. When a new concept is introduced, it is easier to understand if it can be related to something we already know (see Figure 1).

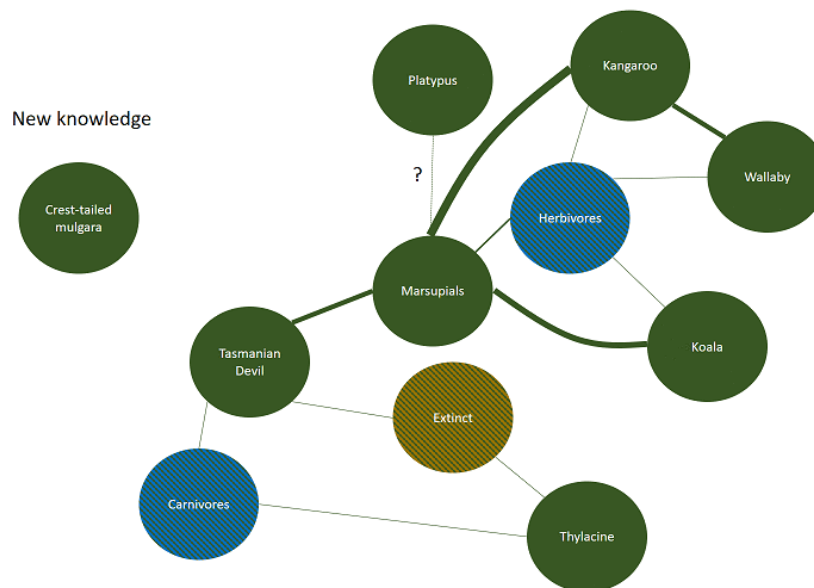


Figure 1.: Schema theory of learning. While probably not entirely applicable to how adults learn, it does highlight the importance of building on prior knowledge. When new knowledge is introduced, it is more quickly stored into long-term memory if strongly-related concepts are already available for connection.

This book is a collection of resources meant to help build your data science, statistical, and computational schemata. It is meant to be largely self-directed, but for those looking to teach data science, it can also be used as a guide for structuring a course. Material is a bit variable in terms of difficulty, prerequisites, and format which is a reflection of the

What are the goals?

organic creation of the material. See below for additional thoughts on adult learning and how it relates to this material.

What are the goals?

We can often get lost in the weeds of technical details of lessons, focusing only on the syntax and semantics of R and Bioconductor. Let's take a step back and consider the big picture. The goals are to:

1. Have a foundation for reading and writing R code to solve problems and understand data.
2. Be able to find and use online resources including AI and tutorials to solve problems and learn new concepts.
3. Be able to effectively communicate with others about R code and data science concepts.
4. Most importantly, to develop the confidence to become a self-directed learner, experimenting with concepts and practice of data science to address real-world problems.

Adult learners

Adult Learning Theory, also known as Andragogy, is the concept and practice of designing, developing, and delivering instructional experiences for adult learners. It is based on the belief that adults learn differently than children, and thus, require distinct approaches to engage, motivate, and retain information (Center 2016). The term was first introduced by Malcolm Knowles, an American educator who is known for his work in adult education (Knowles et al. 2005).

One of the fundamental principles of Adult Learning Theory is that adults are self-directed learners. This means that we prefer to take control of our own learning process and set personal goals for themselves. We are motivated by our desire to solve problems or gain knowledge to improve our lives (see Figure 2). As a result, educational content for adults should be relevant and applicable to real-life situations. Furthermore, adult learners should be given opportunities to actively engage in the learning process by making choices, setting goals, and evaluating their progress.

Another key aspect of Adult Learning Theory is the role of experience. We bring a wealth of experience to the learning process, which serves as a resource for new learning. We often have well-established beliefs, values, and mental models that can influence our willingness to accept new ideas and concepts. Therefore, it is essential to acknowledge and respect our

Adult learners

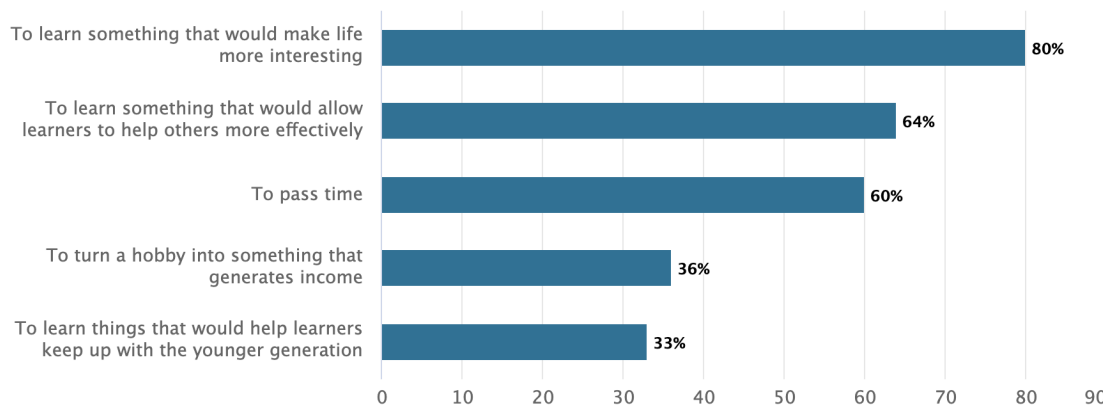


Figure 2.: Why do adults choose to learn something?

shared and unique past experiences and create an environment where we all feel comfortable sharing our perspectives.

To effectively learn as a group of adult learners, it is crucial to establish a collaborative learning environment that promotes open communication and fosters trust among participants. We all appreciate and strive for a respectful and supportive atmosphere where we can express our opinions without fear of judgment. Instructors should help facilitate discussions, encourage peer-to-peer interactions, and incorporate group activities and collaboration to capitalize on the collective knowledge of participants.

Additionally, adult learners often have multiple responsibilities outside of the learning environment, such as work and family commitments. As a result, we require flexible learning opportunities that accommodate busy schedules. Offering a variety of instructional formats, such as online modules, self-paced learning, or evening classes, can help ensure that adult learners have access to education despite any time constraints.

Adult learners benefit from a learner-centered approach that focuses on the individual needs, preferences, and interests of each participant can greatly enhance the overall learning experience. In addition, we tend to be more intrinsically motivated to learn when we have a sense of autonomy and can practice and experiment (see Figure 3) with new concepts in a safe environment.

Understanding Adult Learning Theory and its principles can significantly enhance the effectiveness of teaching and learning as adults. By respecting our autonomy, acknowledging our experiences, creating a supportive learning environment, offering flexible learning opportunities, and utilizing diverse teaching methods, we can better cater to the unique needs and preferences of adult learners.

AI is part of the solution

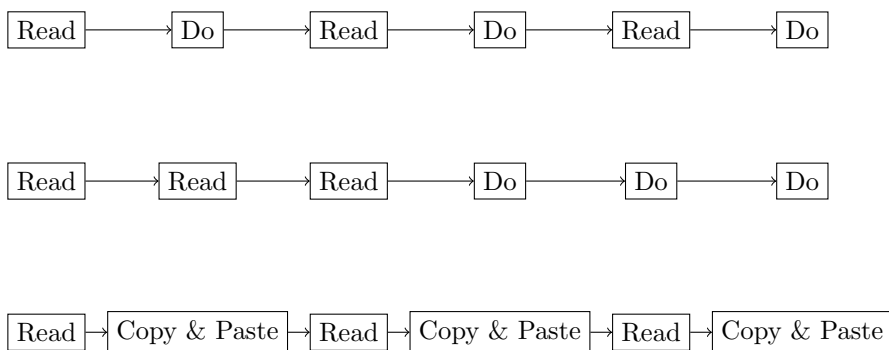


Figure 3.: How to stay stuck in data science (or anything). The “Read-Do” loop tends to deliver the best results. Too much reading between doing can be somewhat effective. Reading and simply copy-paste is probably the least effective. When working through material, experiment. Try to break things. Incorporate your own experience or applications whenever possible.

In practice, that means that we will not be prescriptive in our approach to teaching data science. We will not tell you what to do, but rather we will provide you with a variety of options and you can choose what works best for you. We will also provide you with a variety of resources and you can choose where to focus your time. Given that we cannot possibly cover everything, we will provide you with a framework for learning and you can fill in the gaps as you see fit. A key component of our success as adult learners is to gain the confidence to ask questions and problem-solve on our own.

AI is part of the solution

Artificial Intelligence (AI) is becoming an increasingly important tool in education, and it can be a powerful ally in our quest to learn data science. As it turns out, AI is excellent at helping us learn. It can provide tailored explanations, generate practice problems, and even simulate real-world scenarios for us to work through. AI can also help us find resources, summarize complex topics, and even provide feedback on our work. However, it is important to remember that AI is a tool, not a replacement for our own learning. We still need to engage with the material, ask questions, and seek out additional resources when needed. AI can help us learn more efficiently, but it cannot do the learning for us.

I encourage you to use AI tools to help you learn and to even do work for you when appropriate; AI is an excellent partner in analysis and programming. However, I also encourage you to be critical of the information that AI provides. AI is not perfect and can make mistakes. Throughout this course and book, use AI to dive deeper into the material,

AI is part of the solution

to provide additional explanations, and to generate additional depth and breadth to the material.

Part I.

Introduction

1. About R

Imagine you’ve just gotten back a spreadsheet with expression values for 20,000 genes across 100 samples, and your advisor wants to know which genes differ between treated and untreated cells — by Friday. Clicking through that by hand is hopeless. This is exactly the kind of problem R was built for: load the data, filter and reshape it, run the statistics, and draw a publication-quality figure, all in a handful of lines you can save, rerun, and share.

This chapter is the gentlest possible start. We won’t write much code yet. Instead, we’ll answer two questions that matter before you type a single command: *what is R*, and *why is it worth your time as a biologist?* Once you’re convinced it’s worth learning, the next chapter gets you set up with RStudio and your first commands.

1.1. What you’ll learn

- Describe what R is and where it came from, in plain language.
- Explain why R is so widely used in genomics and bioinformatics.
- Recognize the trade-offs — the things R is *not* good at — so you know when to reach for another tool.
- Understand what “open source” means and why it matters for reproducible science.

1.2. What is R?

R is a programming language and free software environment built for working with data: cleaning it, analyzing it, modeling it, and turning it into graphics. You write short instructions, R carries them out, and you see the results immediately. That’s the whole loop.

R didn’t appear out of nowhere. It’s a dialect of an older language called S, developed in the 1970s at Bell Laboratories. The first version of R was written by Robert Gentleman and Ross Ihaka and released in 1995 (Ihaka’s own [slide deck](#) tells the story well). Since then a volunteer group called the R Core Team — statisticians and computer scientists —

1. About R

has maintained and improved it, releasing a new version roughly every year. So when you use R, you’re standing on three decades of careful work, and you’re not paying a cent for it.

R has become especially popular in genomics and bioinformatics, and that popularity has held up over time even as other languages have risen alongside it (Figure 1.1).

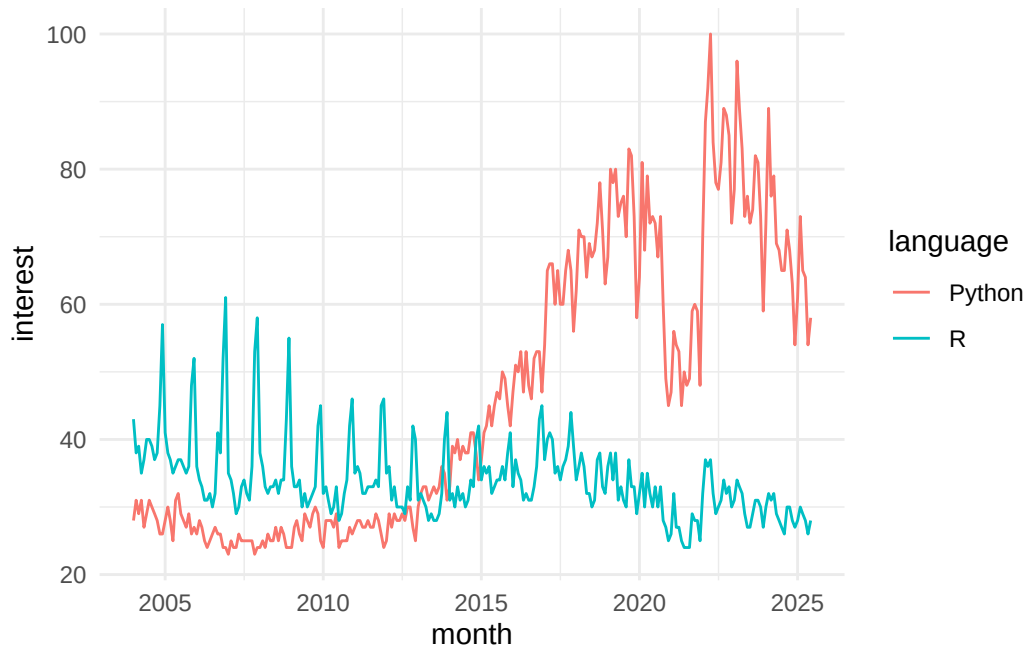


Figure 1.1.: Worldwide search interest in R and Python over time, from [Google Trends](#). Search interest is a rough proxy for general popularity — it does not capture how heavily specific fields lean on each language. Bioinformatics, in particular, remains deeply invested in R.

The figure plots search interest for R and Python over the years. Python is the more general-purpose language and gets more total searches, but don’t read too much into that: a search-popularity chart can’t see *which* communities depend on *which* tool. In bioinformatics specifically — think Bioconductor, the ecosystem this book leans on heavily — R remains a first-class citizen, and that’s a big part of why we teach it here.

1.3. Why use R?

So why has R earned such a devoted following among biologists and data scientists? A few reasons stand out.

It's free and open source. R costs nothing to download, use, modify, or share, and a large worldwide community keeps building it. We'll come back to what "open source" really means in the section on R's license below.

There's a package for almost everything. Thousands of add-on packages extend R's reach, including a deep catalogue of tools written specifically for genomics and bioinformatics. Many have been tested by thousands of users — and, like R itself, they're free.

It's built for wrangling messy data. Real biological data arrives messy. R gives you powerful, well-worn tools to clean, reshape, and summarize even large datasets — not always *easy*, but reliably *possible*.

The graphics are publication quality. R's plotting tools produce figures you can drop straight into a paper or poster, and you can tune nearly every detail to say exactly what you mean.

Your work is reproducible. Because an R analysis is a *script* — a written record of every step — you (or a colleague, or a reviewer) can rerun it and get the same answer. Point-and-click tools rarely leave that kind of trail.

It runs everywhere and scales. R works on Windows, Mac, and Linux, and it can talk to other languages like FORTRAN and C when you need extra speed. The same code serves a quick afternoon analysis or a major project.

That last point is worth a concrete example. I can write and test an analysis on my Mac laptop, then install the *exact same* code on our 20,000-core compute cluster and run it in parallel across a hundred samples, watch the jobs as they go, and have R write the results into a database when it finishes. One language carried me from a back-of-the-envelope idea to a full-scale genomics pipeline without rewriting anything.

i Open source, in one sentence

"Open source" means the program's underlying code is published for anyone to read, change, and redistribute. For science that's not a nicety — it's what lets others inspect exactly how a result was produced and verify it for themselves. R, and the Bioconductor packages you'll meet later, are open source top to bottom.

1.4. Why not use R?

No tool is perfect, and it's healthier to know R's rough edges going in than to be blindsided by them later. R is not always the best tool for every job. It can be slower than languages like C for certain heavy number-crunching, and its documentation can be terse or assume more than a beginner knows. Some community-contributed packages are excellent; others are unmaintained or buggy, so a little judgment helps when you pick one. R also has no single, polished point-and-click interface — that's a feature for reproducibility, but it does mean you commit to writing code. And other languages, notably Python, Julia, and Rust, are increasingly used in biological data science, so R is one strong choice among several rather than the only one.

💡 R will not hold your hand — and that's okay

You'll hear people joke that R doesn't hold your hand; sometimes it *slaps* it instead, with a cryptic error message. Here's the reassuring truth: every R user, including experienced ones, sees errors constantly. An error is not a verdict on you. It's just R saying "I didn't understand that," and the fix is usually a typo, a missing comma, or a quick web search away. We'll return to reading errors calmly in later chapters — for now, just know that getting stuck is normal and temporary.

1.5. R license and the open source ideal

R is free — genuinely, completely free — and distributed under the GNU General Public License. Among other things, that license lets anyone:

- Download the source code.
- Modify it however they like.
- Redistribute the modified code (even for money), as long as it stays under the same GNU license.

That last condition is the clever part: improvements have to flow back to the community on the same open terms. The practical upshot for you is reassuring — R will always be available, will always be open, and can keep growing without being locked behind a paywall or controlled by a single company.

1.6. Where this is going

R is a *programming language*, which means you get things done by writing code rather than clicking buttons. That can feel intimidating at first, but it’s also the source of everything we just praised: scripts you can rerun, functions you can reuse, and analyses you can share. You can work with R two ways — *interactively*, typing one command at a time and seeing each result right away, or as a *script*, saving a whole sequence of commands to run together. You’ll use both.

In the next chapter, on RStudio, we’ll install RStudio — a friendly home base for writing and running R — and take a tour of its panels. The chapter after that gets you up and running with your first real R code, where you’ll build a pair of virtual dice and roll them.

Try it

You don’t need anything installed to look ahead: open the [Google Trends page](#) behind Figure 1.1 and add a third language to the comparison. It’s a gentle reminder that the tools you choose are part of a living, shifting community.

1.7. Summary

You now have the lay of the land. R is a free, open-source language with three decades of history, a vast library of packages, strong graphics, and a deep commitment to reproducibility — which is exactly why it’s a mainstay of genomics and bioinformatics. You also know its limits: it can be slow for some tasks, its error messages can sting, and it’s one good option among several. With the “why” settled, you’re ready to set up the tools and start writing code.

1.8. Reflection

Take a moment to check your understanding before moving on. You don’t need to write anything down — just see whether you can answer each in a sentence or two.

1. In your own words, what does it mean that R is “open source,” and why does that matter for a published scientific result?
2. Name two reasons a biologist might choose R over a point-and-click spreadsheet program for analyzing experimental data.

1. About R

3. The Google Trends figure shows Python with more overall search interest than R. Why is that *not* a good reason to conclude R is unimportant in bioinformatics?
4. What is one situation where R might *not* be the best tool, and what would you consider reaching for instead?

2. RStudio

RStudio is an integrated development environment (IDE) for R. It provides a graphical user interface (GUI) for R, making it easier to write and execute R code. RStudio also provides several other useful features, including a built-in console, syntax-highlighting editor, and tools for plotting, history, debugging, workspace management, and workspace viewing. RStudio is available in both free and commercial editions; the commercial edition provides some additional features, including support for multiple sessions and enhanced debugging.

2.1. Getting started with RStudio

To get started with RStudio, you first need to install both R and RStudio on your computer. Follow these steps:

1. Download and install R from the [official R website](#).
2. Download and install RStudio from the [official RStudio website](#).
3. Launch RStudio. You should see the RStudio interface with four panels.

i R versions

RStudio works with all versions of R, but it is recommended to use the latest version of R to take advantage of the latest features and improvements. You can check your R version by running `version` (no parentheses) in the R console. You can check the latest version of R on the [R-project website](#).

2.2. The RStudio Interface

RStudio's interface consists of four panels (see Figure 2.1):

- **Console** This panel displays the R console, where you can enter and execute R commands directly. The console also shows the output of your code, error messages, and other information.

2. RStudio

- **Source** This panel is where you write and edit your R scripts. You can create new scripts, open existing ones, and run your code from this panel.
- **Environment** This panel displays your current workspace, including all variables, data objects, and functions that you have created or loaded in your R session.
- **Plots, Packages, Help, and Viewer** These panels display plots, installed packages, help files, and web content, respectively.

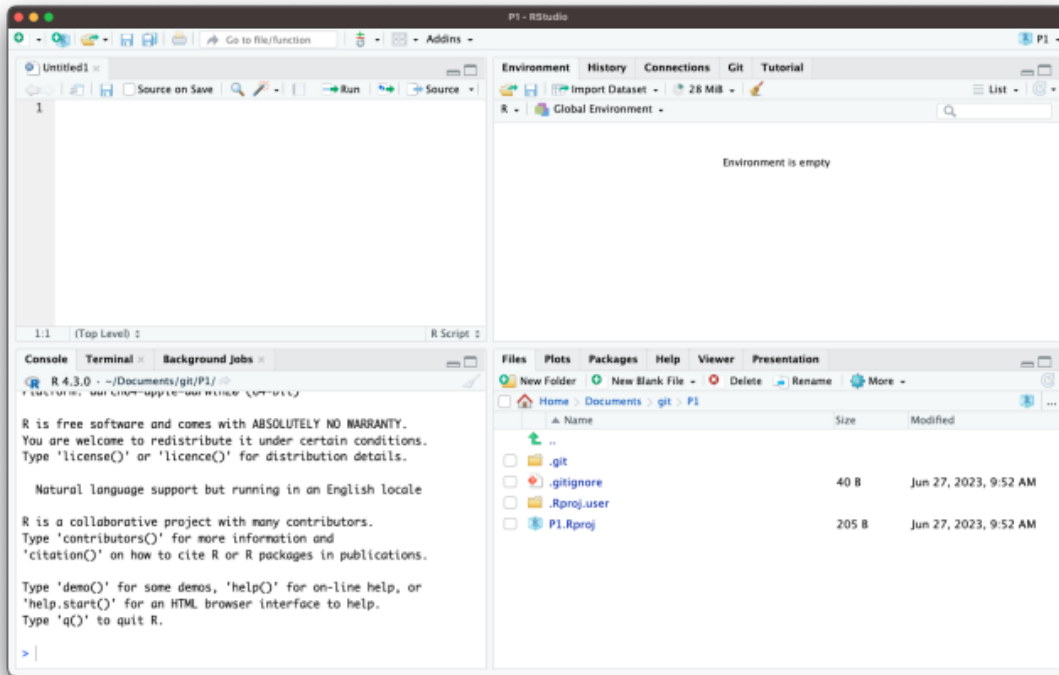


Figure 2.1.: The RStudio interface. In this layout, the **source** pane is in the upper left, the **console** is in the lower left, the **environment** panel is in the top right and the **viewer/help/files** panel is in the bottom right.

i Do I need to use RStudio?

No. You can use R without RStudio. However, RStudio makes it easier to write and execute R code, and it provides several useful features that are not available in the basic R console. Note that the only part of RStudio that is actually interacting with R directly is the console. The other panels are simply providing a GUI that enhances

2. RStudio

the user experience.

💡 Customizing the RStudio Interface

You can customize the layout of RStudio to suit your preferences. To do so, go to **Tools > Global Options > Appearance**. Here, you can change the theme, font size, and panel layout. You can also resize the panels as needed to gain screen real estate (see Figure 2.2).

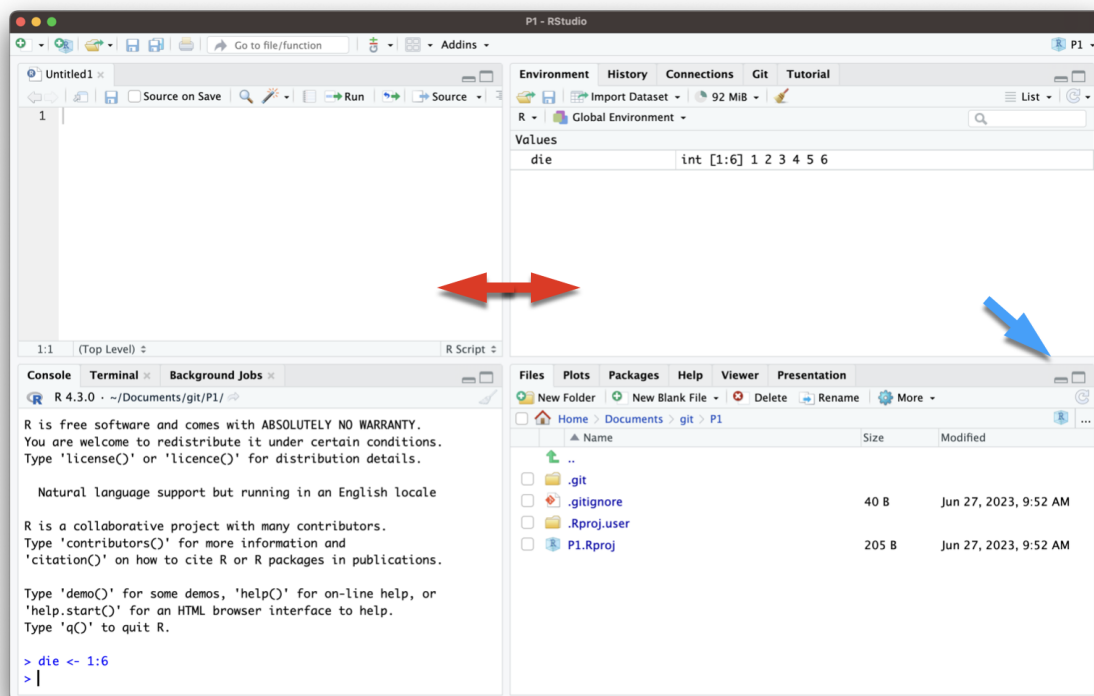


Figure 2.2.: Dealing with limited screen real estate can be a challenge, particularly when you want to open another window to, for example, view a web page. You can resize the panes by sliding the center divider (red arrows) or by clicking on the minimize/maximize buttons (see blue arrow).

In summary, R and RStudio are powerful tools for genomics data analysis. By understanding the advantages of using R and RStudio and familiarizing yourself with the RStudio interface, you can efficiently analyze and visualize your data. In the following chapters, we will delve deeper into the functionality of R, Bioconductor, and various statistical methods to help you gain a comprehensive understanding of genomics data analysis.

2.3. Alternatives to RStudio

While RStudio is a popular choice for R development, there are several alternatives you can consider:

1. **Jupyter Notebooks:** Jupyter Notebooks provide an interactive environment for writing and executing R code, along with rich text support for documentation. You can use the IRKernel to run R code in Jupyter.

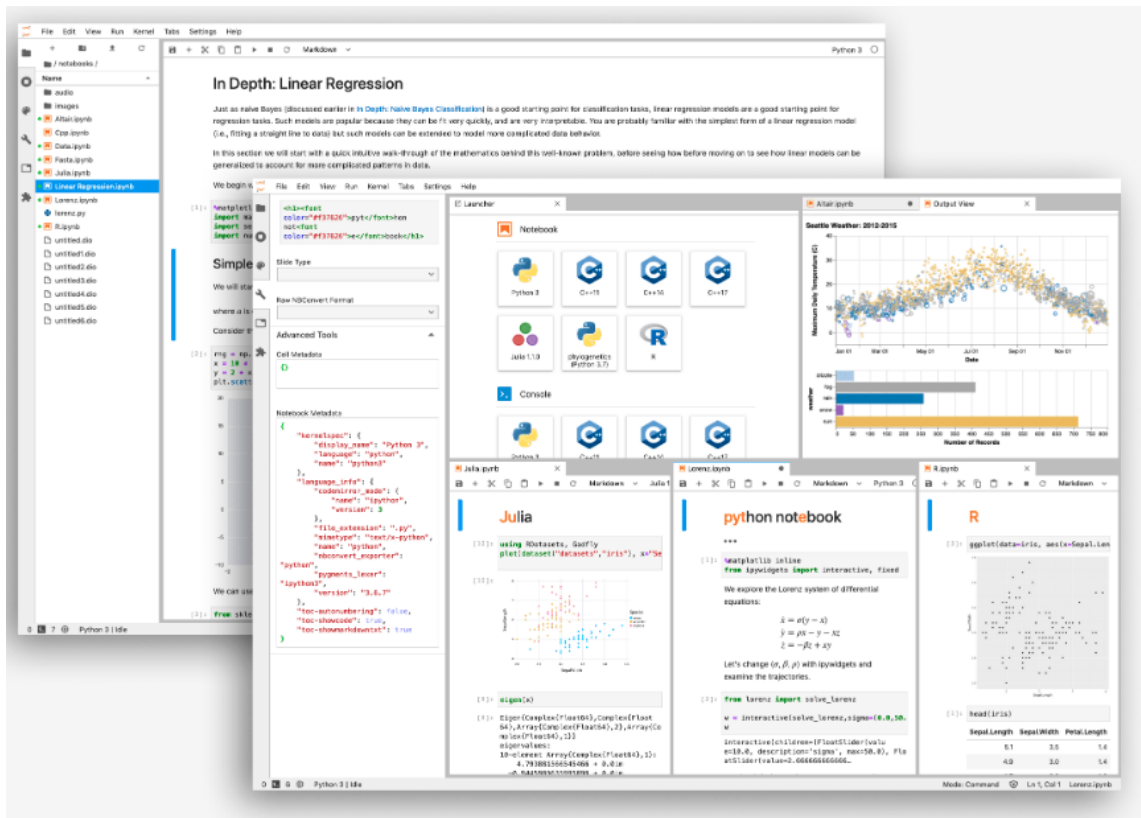


Figure 2.3.: Jupyter Notebook interface. This is an interactive environment for writing and executing R code, along with rich text support for documentation.

2. **Visual Studio Code:** With the R extension for Visual Studio Code, you can write and execute R code in a lightweight editor. This setup provides features like syntax highlighting, code completion, and integrated terminal support.

2. RStudio

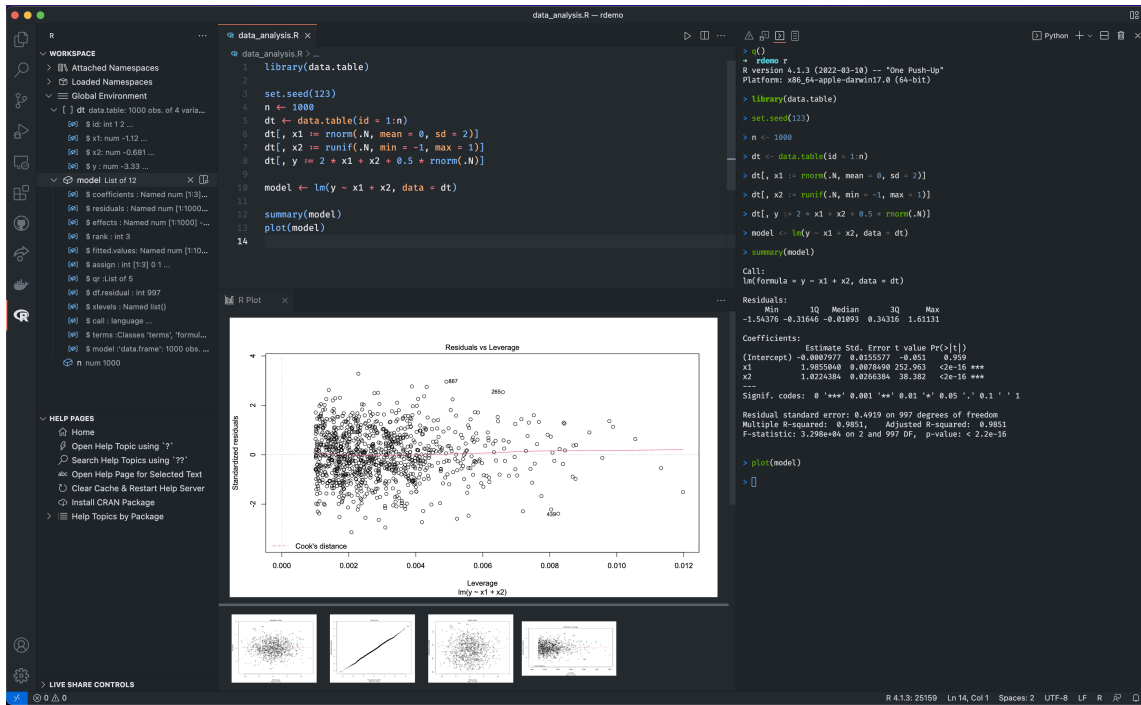


Figure 2.4.: Visual Studio Code (VSCode) with the R extension. This is a lightweight alternative to RStudio that provides syntax highlighting, code completion, and integrated terminal support.

3. **Positron Workbench**: This is a commercial IDE that supports R and Python. It provides a similar interface to RStudio but with additional features for data science workflows, including support for multiple languages and cloud integration.

2. RStudio

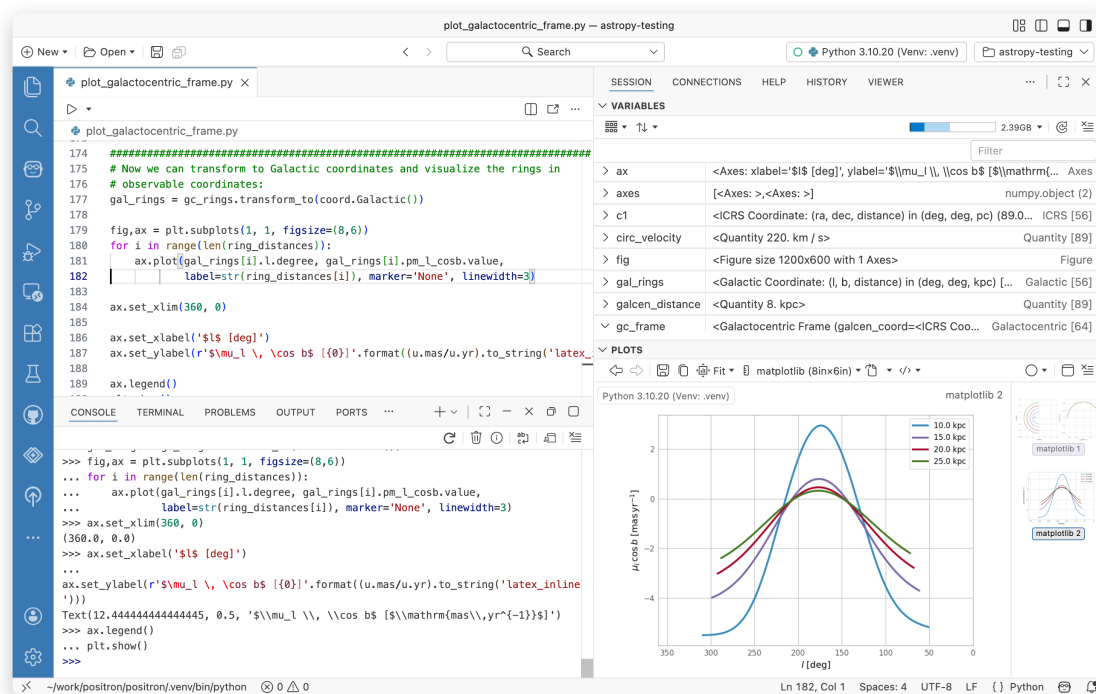


Figure 2.5.: Positron Workbench interface. This IDE supports R and Python, providing a similar interface to RStudio with additional features for data science workflows.

4. **Command Line R:** For those who prefer a minimalistic approach, you can use R directly from the command line. This method lacks the GUI features of RStudio but can be efficient for quick tasks, scripting, automation, or when working on remote servers.

Each of these alternatives has its own strengths and weaknesses, so you may want to try a few to see which one best fits your workflow. All are available for free, and you can install them alongside RStudio if you wish to use multiple environments. Each can be installed in Windows, Mac, and Linux.

3. R mechanics

3.1. Starting R

We've installed R and RStudio. Now, let's start R and get going. How to start R depends a bit on the operating system (Mac, Windows, Linux) and interface. In this course, we will largely be using an Integrated Development Environment (IDE) called *RStudio*, but there is nothing to prohibit using R at the command line or in some other interface (and there are a few).

3.2. *RStudio*: A Quick Tour

The RStudio interface has multiple panes. All of these panes are simply for convenience except the “Console” panel, typically in the lower left corner (by default). The console pane contains the running R interface. If you choose to run R outside RStudio, the interaction will be *identical* to working in the console pane. This is useful to keep in mind as some environments, such as a computer cluster, encourage using R without RStudio.

- Panes
- Options
- Help
- Environment, History, and Files

3.3. Interacting with R

The only meaningful way of interacting with R is by typing into the R console. At the most basic level, anything that we type at the command line will fall into one of two categories:

1. Assignments

```
x = 1  
y <- 2
```

3. R mechanics

2. Expressions

```
1 + pi + sin(42)
```

```
[1] 3.225071
```

The assignment type is obvious because either the `<-` or `=` are used. Note that when we type expressions, R will return a result. In this case, the result of R evaluating `1 + pi + sin(42)` is 3.2250711.

The standard R prompt is a “>” sign. When present, R is waiting for the next expression or assignment. If a line is not a complete R command, R will continue the next line with a “+”. For example, typing the following with a “Return” after the second “+” will result in R giving back a “+” on the next line, a prompt to keep typing.

```
1 + pi +  
sin(3.7)
```

```
[1] 3.611757
```

R can be used as a glorified calculator by using R expressions. Mathematical operations include:

- Addition: `+`
- Subtraction: `-`
- Multiplication: `*`
- Division: `/`
- Exponentiation: `^`
- Modulo: `%%`

The `^` operator raises the number to its left to the power of the number to its right: for example `3^2` is 9. The modulo returns the remainder of the division of the number to the left by the number on its right, for example 5 modulo 3 or `5 %% 3` is 2.

3.3.1. Expressions

```
5 + 2  
28 %% 3  
3^2  
5 + 4 * 4 + 4 ^ 4 / 10
```

3. R mechanics

Note that R follows order-of-operations and groupings based on parentheses.

```
5 + 4 / 9
(5 + 4) / 9
```

3.3.2. Assignment

While using R as a calculator is interesting, to do useful and interesting things, we need to assign *values* to *objects*. To create objects, we need to give it a name followed by the assignment operator `<-` (or, entirely equivalently, `=`) and the value we want to give it:

```
weight_kg <- 55
```

`<-` is the assignment operator. Assigns values on the right to objects on the left, it is like an arrow that points from the value to the object. Using an `=` is equivalent (in nearly all cases). Learn to use `<-` as it is good programming practice.

i What about `<-` and `=` for assignment?

The `<-` and `=` both work fine for assignment. You'll see both used and it is up to you to choose a standard for yourself. However, some programming communities, such as Bioconductor, will strongly suggest using the `<-` as it is clearer that it represents an *assignment* operation.

Objects can be given any name such as `x`, `current_temperature`, or `subject_id` (see below). You want your object names to be explicit and not too long. They cannot start with a number (`2x` is not valid but `x2` is). R is case sensitive (e.g., `weight_kg` is different from `Weight_kg`). There are some names that cannot be used because they represent the names of fundamental functions in R (e.g., `if`, `else`, `for`, see [here](#) for a complete list). In general, even if it's allowed, it's best to not use other function names, which we'll get into shortly (e.g., `c`, `T`, `mean`, `data`, `df`, `weights`). When in doubt, check the help to see if the name is already in use. It's also best to avoid dots (`.`) within a variable name as in `my.dataset`. It is also recommended to use nouns for variable names, and verbs for function names.

When assigning a value to an object, R does not print anything. You can force to print the value by typing the name:

```
weight_kg
```

3. R mechanics

```
[1] 55
```

Now that R has `weight_kg` in memory, which R refers to as the “global environment”, we can do arithmetic with it. For instance, we may want to convert this weight in pounds (weight in pounds is 2.2 times the weight in kg).

```
2.2 * weight_kg
```

```
[1] 121
```

We can also change a variable’s value by assigning it a new one:

```
weight_kg <- 57.5  
2.2 * weight_kg
```

```
[1] 126.5
```

This means that assigning a value to one variable does not change the values of other variables. For example, let’s store the animal’s weight in pounds in a variable.

```
weight_lb <- 2.2 * weight_kg
```

and then change `weight_kg` to 100.

```
weight_kg <- 100
```

What do you think is the current content of the object `weight_lb`, 126.5 or 220?

You can see what objects (variables) are stored by viewing the Environment tab in Rstudio. You can also use the `ls()` function. You can remove objects (variables) with the `rm()` function. You can do this one at a time or remove several objects at once. You can also use the little broom button in your environment pane to remove everything from your environment.

```
ls()  
rm(weight_lb, weight_kg)  
ls()
```

What happens when you type the following, now?

```
weight_lb # oops! you should get an error because weight_lb no longer exists!
```

3.4. Rules for Names in R

R allows users to assign names to objects such as variables, functions, and even dimensions of data. However, these names must follow a few rules.

- Names may contain any combination of letters, numbers, underscore, and “.”
- Names may not start with numbers, underscore.
- R names are case-sensitive.

Examples of valid R names include:

```
pi
x
camelCaps
my_stuff
MY_Stuff
this.is.the.name.of.the.man
ABC123
abc1234asdf
.hi
```

3.5. About R functions

When you see a name followed by parentheses (), you are likely looking a name that represents an R function (or method, but we’ll sidestep that distinction for now). Examples of R functions include `print()`, `help()`, and `ls()`. We haven’t seen examples yet, but when a name is followed by [], that name represents a variable of some kind and the [] are used for “subsetting” the variable. So:

- Name followed by () is a function.
- Name with [] means a variable that is being subset.

In many cases, when you see a new function used, you may not know what it does. The R `help()` function takes the name of another function and gives back the R help document for that function if there is one. The next section reviews that technique.

3.6. Resources for Getting Help

There is extensive built-in help and documentation within R. A separate page contains a collection of [additional resources](#).

If the name of the function or object on which help is sought is known, the following approaches with the name of the function or object will be helpful. For a concrete example, examine the help for the `print` method.

```
help(print)
help('print')
?print
```

There are also tons of online resources that Google will include in searches if online searching feels more appropriate.

I strongly recommend using `help("newfunction")` for all functions that are new or unfamiliar to you.

There are also many open and free resources and reference guides for R.

- [Quick-R](#): a quick online reference for data input, basic statistics and plots
- R reference card [PDF](#) by Tom Short
- Rstudio [cheatsheets](#)

3.7. Reflection

- Can you recognize the difference between *assignment* and *expressions* when interacting with R?
- Can you demonstrate an assignment to a variable?
- Do you know the rules for “names” in R?
- Are you able to get help using the R `help()` function?
- Do you know that functions are recognizable as names followed by `()`?

4. Up and Running with R

In this chapter, we’re going to get an introduction to the R language, so we can dive right into programming. We’re going to create a pair of virtual dice that can generate random numbers. No need to worry if you’re new to programming. We’ll return to many of the concepts here in more detail later.

To simulate a pair of dice, we need to break down each die into its essential features. A die can only show one of six numbers: 1, 2, 3, 4, 5, and 6. We can capture the die’s essential characteristics by saving these numbers as a group of values in the computer. Let’s save these numbers first and then figure out a way to “roll” our virtual die.

4.1. The R User Interface

The RStudio interface is simple. You type R code into the bottom line of the RStudio console pane and then click Enter to run it. The code you type is called a *command*, because it will command your computer to do something for you. The line you type it into is called the *command line*.

When you type a command at the prompt and hit Enter, your computer executes the command and shows you the results. Then RStudio displays a fresh prompt for your next command. For example, if you type `1 + 1` and hit Enter, RStudio will display:

```
> 1 + 1
[1] 2
>
```

You’ll notice that a `[1]` appears next to your result. R is just letting you know that this line begins with the first value in your result. Some commands return more than one value, and their results may fill up multiple lines. For example, the command `100:130` returns 31 values; it creates a sequence of integers from 100 to 130. Notice that new bracketed numbers appear at the start of the second and third lines of output. These numbers just mean that the second line begins with the 14th value in the result, and the third line begins with the 25th value. You can mostly ignore the numbers that appear in brackets:

4. Up and Running with R

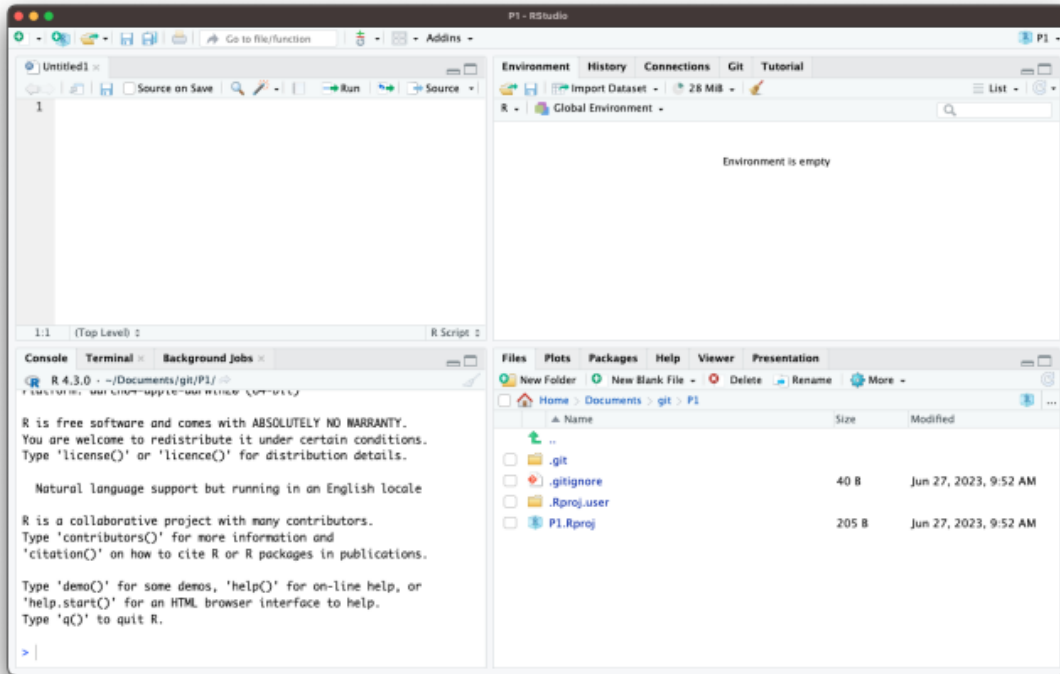


Figure 4.1.: Your computer does your bidding when you type R commands at the prompt in the bottom line of the console pane. Don't forget to hit the Enter key. When you first open RStudio, the console appears in the pane on your left, but you can change this with **File > Tools > Global Options** in the menu bar.

4. Up and Running with R

```
> 100:130
[1] 100 101 102 103 104 105 106 107 108 109 110 111 112
[14] 113 114 115 116 117 118 119 120 121 122 123 124 125
[25] 126 127 128 129 130
```

Tip

The colon operator (:) returns every integer between two integers. It is an easy way to create a sequence of numbers.

When do we compile?

In some languages, like C, Java, and FORTRAN, you have to compile your human-readable code into machine-readable code (often 1s and 0s) before you can run it. If you've programmed in such a language before, you may wonder whether you have to compile your R code before you can use it. The answer is no. R is a dynamic programming language, which means R automatically interprets your code as you run it.

If you type an incomplete command and press Enter, R will display a + prompt, which means R is waiting for you to type the rest of your command. Either finish the command or hit Escape to start over:

```
> 5 -
+
+ 1
[1] 4
```

If you type a command that R doesn't recognize, R will return an error message. If you ever see an error message, don't panic. R is just telling you that your computer couldn't understand or do what you asked it to do. You can then try a different command at the next prompt:

```
> 3 % 5
Error: unexpected input in "3 % 5"
>
```

4. Up and Running with R

Tip

Whenever you get an error message in R, consider googling the error message. You'll often find that someone else has had the same problem and has posted a solution online. Simply cutting-and-pasting the error message into a search engine will often work

Once you get the hang of the command line, you can easily do anything in R that you would do with a calculator. For example, you could do some basic arithmetic:

```
2 * 3
```

```
[1] 6
```

```
4 - 1
```

```
[1] 3
```

```
# this obeys order-of-operations  
6 / (4 - 1)
```

```
[1] 2
```

Tip

R treats the hashtag character, #, in a special way; R will not run anything that follows a hashtag on a line. This makes hashtags very useful for adding comments and annotations to your code. Humans will be able to read the comments, but your computer will pass over them. The hashtag is known as the *commenting symbol* in R.

Cancelling commands

Some R commands may take a long time to run. You can cancel a command once it has begun by pressing ctrl + c or by clicking the “stop sign” if it is available in Rstudio. Note that it may also take R a long time to cancel the command.

4. Up and Running with R

4.1.1. An exercise

That's the basic interface for executing R code in RStudio. Think you have it? If so, try doing these simple tasks. If you execute everything correctly, you should end up with the same number that you started with:

1. Choose any number and add 2 to it.
2. Multiply the result by 3.
3. Subtract 6 from the answer.
4. Divide what you get by 3.

```
10 + 2
```

```
[1] 12
```

```
12 * 3
```

```
[1] 36
```

```
36 - 6
```

```
[1] 30
```

```
30 / 3
```

```
[1] 10
```

4.2. Objects

Now that you know how to use R, let's use it to make a virtual die. The `:` operator from a couple of pages ago gives you a nice way to create a group of numbers from one to six. The `:` operator returns its results as a **vector** (we are going to work with vectors in more detail), a one-dimensional set of numbers:

```
1:6  
## 1 2 3 4 5 6
```

4. Up and Running with R

That’s all there is to how a virtual die looks! But you are not done yet. Running `1:6` generated a vector of numbers for you to see, but it didn’t save that vector anywhere for later use. If we want to use those numbers again, we’ll have to ask your computer to save them somewhere. You can do that by creating an R *object*.

R lets you save data by storing it inside an R object. What is an object? Just a name that you can use to call up stored data. For example, you can save data into an object like `a` or `b`. Wherever R encounters the object, it will replace it with the data saved inside, like so:

```
a <- 1  
a
```

```
[1] 1
```

```
a + 2
```

```
[1] 3
```

i What just happened?

1. To create an R object, choose a name and then use the less-than symbol, `<`, followed by a minus sign, `-`, to save data into it. This combination looks like an arrow, `<-`. R will make an object, give it your name, and store in it whatever follows the arrow. So `a <- 1` stores 1 in an object named `a`.
2. When you ask R what’s in `a`, R tells you on the next line.
3. You can use your object in new R commands, too. Since `a` previously stored the value of 1, you’re now adding 1 to 2.

! Assignment vs expressions

Everything that you type into the R console can be assigned to one of two categories:

- Assignments
- Expressions

An expression is a command that tells R to do something. For example, `1 + 2` is an expression that tells R to add 1 and 2. When you type an expression into the R console, R will evaluate the expression and return the result. For example, if you type `1 + 2` into the R console, R will return 3. Expressions can have “side effects”

4. Up and Running with R

but they don't explicitly result in anything being added to R memory.

```
5 + 2
```

```
[1] 7
```

```
28 %% 3
```

```
[1] 1
```

```
3^2
```

```
[1] 9
```

```
5 + 4 * 4 + 4 ^ 4 / 10
```

```
[1] 46.6
```

While using R as a calculator is interesting, to do useful and interesting things, we need to assign values to objects. To create objects, we need to give it a name followed by the assignment operator `<-` (or, entirely equivalently, `=`) and the value we want to give it:

```
weight_kg <- 55
```

So, for another example, the following code would create an object named `die` that contains the numbers one through six. To see what is stored in an object, just type the object's name by itself:

```
die <- 1:6  
die
```

```
[1] 1 2 3 4 5 6
```

When you create an object, the object will appear in the environment pane of RStudio, as shown in Figure 4.2. This pane will show you all of the objects you've created since opening RStudio.

4. Up and Running with R

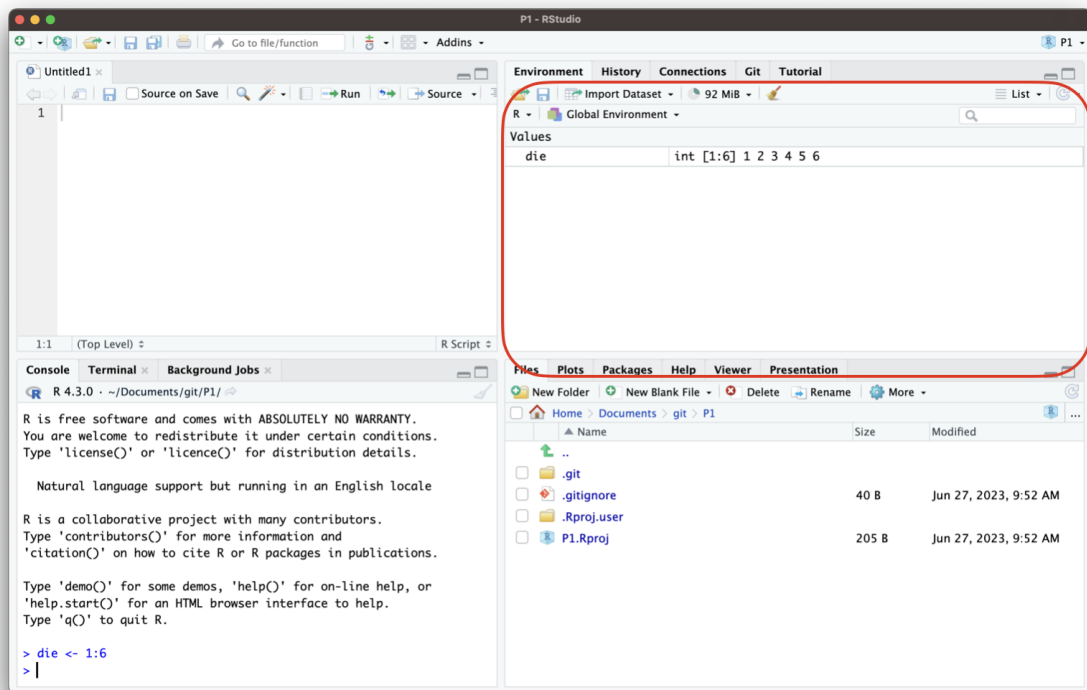


Figure 4.2.: Assignment creates an object in the environment pane.

4. Up and Running with R

You can name an object in R almost anything you want, but there are a few rules. First, a name cannot start with a number. Second, a name cannot use some special symbols, like `^`, `!`, `$`, `@`, `+`, `-`, `/`, or `*`:

Good names	Names that cause errors
a	1trial
b	\$
FOO	$\wedge$ mean
my_var	2nd
.day	!bad

Capitalization matters

R is case-sensitive, so `name` and `Name` will refer to different objects:

```
> Name = 0
> Name + 1
[1] 1
> name + 1
Error: object 'name' not found
```

The error above is a common one!

Finally, R will overwrite any previous information stored in an object without asking you for permission. So, it is a good idea to *not* use names that are already taken:

```
my_number <- 1
my_number
```

```
[1] 1
```

```
my_number <- 999
my_number
```

```
[1] 999
```

You can see which object names you have already used with the function `ls`:

4. Up and Running with R

```
ls()
```

Your environment will contain different names than mine, because you have probably created different objects.

You can also see which names you have used by examining RStudio's environment pane.

We now have a virtual die that is stored in the computer's memory and which has a name that we can use to refer to it. You can access it whenever you like by typing the word *die*.

So what can you do with this die? Quite a lot. R will replace an object with its contents whenever the object's name appears in a command. So, for example, you can do all sorts of math with the die. Math isn't so helpful for rolling dice, but manipulating sets of numbers will be your stock and trade as a data scientist. So let's take a look at how to do that:

```
die - 1
```

```
[1] 0 1 2 3 4 5
```

```
die / 2
```

```
[1] 0.5 1.0 1.5 2.0 2.5 3.0
```

```
die * die
```

```
[1] 1 4 9 16 25 36
```

R uses *element-wise execution* when working with a *vector* like *die*. When you manipulate a set of numbers, R will apply the same operation to each element in the set. So for example, when you run *die - 1*, R subtracts one from each element of *die*.

When you use two or more vectors in an operation, R will line up the vectors and perform a sequence of individual operations. For example, when you run *die * die*, R lines up the two *die* vectors and then multiplies the first element of vector 1 by the first element of vector 2. R then multiplies the second element of vector 1 by the second element of vector 2, and so on, until every element has been multiplied. The result will be a new vector the same length as the first two {Figure 4.3}.

If you give R two vectors of unequal lengths, R will repeat the shorter vector until it is as long as the longer vector, and then do the math, as shown in Figure 4.4. This isn't a

4. Up and Running with R

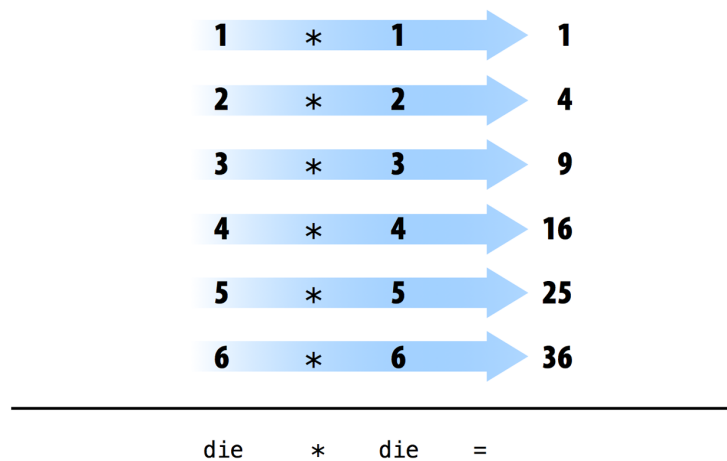


Figure 4.3.: “When R performs element-wise execution, it matches up vectors and then manipulates each pair of elements independently.”

permanent change—the shorter vector will be its original size after R does the math. If the length of the short vector does not divide evenly into the length of the long vector, R will return a warning message. This behavior is known as *vector recycling*, and it helps R do element-wise operations:

```
1:2
```

```
[1] 1 2
```

```
1:4
```

```
[1] 1 2 3 4
```

```
die
```

```
[1] 1 2 3 4 5 6
```

```
die + 1:2
```

```
[1] 2 4 4 6 6 8
```

4. Up and Running with R

```
die + 1:4
```

Warning in `die + 1:4`: longer object length is not a multiple of shorter object length

```
[1] 2 4 6 8 6 8
```

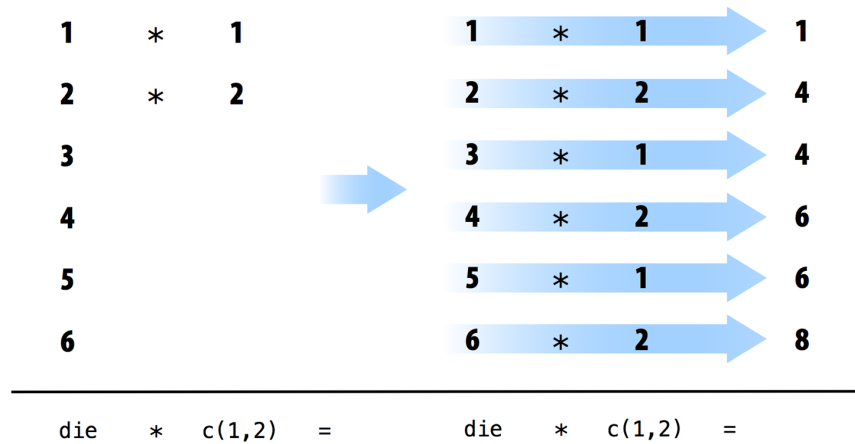


Figure 4.4.: “R will repeat a short vector to do element-wise operations with two vectors of uneven lengths.”

Element-wise operations are a very useful feature in R because they manipulate groups of values in an orderly way. When you start working with data sets, element-wise operations will ensure that values from one observation or case are only paired with values from the same observation or case. Element-wise operations also make it easier to write your own programs and functions in R.

! Element-wise operations are not matrix operations

It is important to know that operations with vectors are not the same that you might expect if you are expecting R to perform “matrix” operations. R can do inner multiplication with the `%*%` operator and outer multiplication with the `%o%` operator:

```
# Inner product (1*1 + 2*2 + 3*3 + 4*4 + 5*5 + 6*6)
die %*% die
# Outer product
die %o% die
```

4. Up and Running with R

Now that you can do math with your `die` object, let's look at how you could “roll” it. Rolling your die will require something more sophisticated than basic arithmetic; you'll need to randomly select one of the die's values. And for that, you will need a *function*.

4.3. Functions

R has many functions and puts them all at our disposal. We can use functions to do simple and sophisticated tasks. For example, we can round a number with the `round` function, or calculate its factorial with the `factorial` function. Using a function is pretty simple. Just write the name of the function and then the data you want the function to operate on in parentheses:

```
round(3.1415)
```

```
[1] 3
```

```
factorial(3)
```

```
[1] 6
```

The data that you pass into the function is called the function's *argument*. The argument can be raw data, an R object, or even the results of another R function. In this last case, R will work from the innermost function to the outermost [Figure 4.5](#).

```
mean(1:6)
```

```
[1] 3.5
```

```
mean(die)
```

```
[1] 3.5
```

```
round(mean(die))
```

```
[1] 4
```

4. Up and Running with R

```
round(mean(die))  
  ↓  
round(mean(1:6))  
  ↓  
round(3.5)  
  ↓  
4
```

Figure 4.5.: “When you link functions together, R will resolve them from the innermost operation to the outermost. Here R first looks up `die`, then calculates the mean of one through six, then rounds the mean.”

Returning to our die, we can use the `sample` function to randomly select one of the die’s values; in other words, the `sample` function can simulate rolling the `die`.

The `sample` function takes *two* arguments: a vector named `x` and a number named `size`. `sample` will return `size` elements from the vector:

```
sample(x = 1:4, size = 2)
```

```
[1] 2 1
```

To roll your die and get a number back, set `x` to `die` and sample one element from it. You’ll get a new (maybe different) number each time you roll it:

```
sample(x = die, size = 1)
```

```
[1] 3
```

```
sample(x = die, size = 1)
```

```
[1] 3
```

```
sample(x = die, size = 1)
```

```
[1] 5
```

4. Up and Running with R

Many R functions take multiple arguments that help them do their job. You can give a function as many arguments as you like as long as you separate each argument with a comma.

You may have noticed that I set `die` and `1` equal to the names of the arguments in `sample`, `x` and `size`. Every argument in every R function has a name. You can specify which data should be assigned to which argument by setting a name equal to data, as in the preceding code. This becomes important as you begin to pass multiple arguments to the same function; names help you avoid passing the wrong data to the wrong argument. However, using names is optional. You will notice that R users do not often use the name of the first argument in a function. So you might see the previous code written as:

```
sample(die, size = 1)
```

```
[1] 3
```

Often, the name of the first argument is not very descriptive, and it is usually obvious what the first piece of data refers to anyways.

But how do you know which argument names to use? If you try to use a name that a function does not expect, you will likely get an error:

```
round(3.1415, corners = 2)
## Error in round(3.1415, corners = 2) : unused argument(s) (corners = 2)
```

If you're not sure which names to use with a function, you can look up the function's arguments with `args`. To do this, place the name of the function in the parentheses behind `args`. For example, you can see that the `round` function takes two arguments, one named `x` and one named `digits`:

```
args(round)
```

```
function (x, digits = 0, ...)
NULL
```

Did you notice that `args` shows that the `digits` argument of `round` is already set to 0? Frequently, an R function will take optional arguments like `digits`. These arguments are considered optional because they come with a default value. You can pass a new value to an optional argument if you want, and R will use the default value if you do not. For example, `round` will round your number to 0 digits past the decimal point by default. To override the default, supply your own value for `digits`:

4. Up and Running with R

```
round(3.1415)
```

```
[1] 3
```

```
round(3.1415, digits = 2)
```

```
[1] 3.14
```

```
# pi happens to be a built-in value in R  
pi
```

```
[1] 3.141593
```

```
round(pi)
```

```
[1] 3
```

You should write out the names of each argument after the first one or two when you call a function with multiple arguments. Why? First, this will help you and others understand your code. It is usually obvious which argument your first input refers to (and sometimes the second input as well). However, you'd need a large memory to remember the third and fourth arguments of every R function. Second, and more importantly, writing out argument names prevents errors.

If you do not write out the names of your arguments, R will match your values to the arguments in your function by order. For example, in the following code, the first value, `die`, will be matched to the first argument of `sample`, which is named `x`. The next value, `1`, will be matched to the next argument, `size`:

```
sample(die, 1)
```

```
[1] 3
```

As you provide more arguments, it becomes more likely that your order and R's order may not align. As a result, values may get passed to the wrong argument. Argument names prevent this. R will always match a value to its argument name, no matter where it appears in the order of arguments:

4. Up and Running with R

```
sample(size = 1, x = die)
```

```
[1] 1
```

4.3.1. Sample with Replacement

If you set `size = 2`, you can *almost* simulate a pair of dice. Before we run that code, think for a minute why that might be the case. `sample` will return two numbers, one for each die:

```
sample(die, size = 2)
```

```
[1] 3 6
```

I said this “almost” works because this method does something funny. If you use it many times, you’ll notice that the second die never has the same value as the first die, which means you’ll never roll something like a pair of threes or snake eyes. What is going on?

By default, `sample` builds a sample *without replacement*. To see what this means, imagine that `sample` places all of the values of `die` in a jar or urn. Then imagine that `sample` reaches into the jar and pulls out values one by one to build its sample. Once a value has been drawn from the jar, `sample` sets it aside. The value doesn’t go back into the jar, so it cannot be drawn again. So if `sample` selects a six on its first draw, it will not be able to select a six on the second draw; six is no longer in the jar to be selected. Although `sample` creates its sample electronically, it follows this seemingly physical behavior.

One side effect of this behavior is that each draw depends on the draws that come before it. In the real world, however, when you roll a pair of dice, each die is independent of the other. If the first die comes up six, it does not prevent the second die from coming up six. In fact, it doesn’t influence the second die in any way whatsoever. You can recreate this behavior in `sample` by adding the argument `replace = TRUE`:

```
sample(die, size = 2, replace = TRUE)
```

```
[1] 1 6
```

4. Up and Running with R

The argument `replace = TRUE` causes `sample` to sample *with replacement*. Our jar example provides a good way to understand the difference between sampling with replacement and without. When `sample` uses replacement, it draws a value from the jar and records the value. Then it puts the value back into the jar. In other words, `sample` *replaces* each value after each draw. As a result, `sample` may select the same value on the second draw. Each value has a chance of being selected each time. It is as if every draw were the first draw.

Sampling with replacement is an easy way to create *independent random samples*. Each value in your sample will be a sample of size one that is independent of the other values. This is the correct way to simulate a pair of dice:

```
sample(die, size = 2, replace = TRUE)
```

```
[1] 5 3
```

Congratulate yourself; you’ve just run your first simulation in R! You now have a method for simulating the result of rolling a pair of dice. If you want to add up the dice, you can feed your result straight into the `sum` function:

```
dice <- sample(die, size = 2, replace = TRUE)
dice
```

```
[1] 5 1
```

```
sum(dice)
```

```
[1] 6
```

What would happen if you call `dice` multiple times? Would R generate a new pair of dice values each time? Let’s give it a try:

```
dice
```

```
[1] 5 1
```

4. Up and Running with R

```
dice
```

```
[1] 5 1
```

```
dice
```

```
[1] 5 1
```

The name `dice` refers to a *vector* of two numbers. Calling more than once does not change the value. Each time you call `dice`, R will show you the result of that one time you called `sample` and saved the output to `dice`. R won't rerun `sample(die, 2, replace = TRUE)` to create a new roll of the dice. Once you save a set of results to an R object, those results do not change.

However, it *would* be convenient to have an object that can re-roll the dice whenever you call it. You can make such an object by writing your own R function.

4.4. Writing Your Own Functions

To recap, you already have working R code that simulates rolling a pair of dice:

```
die <- 1:6  
dice <- sample(die, size = 2, replace = TRUE)  
sum(dice)
```

```
[1] 8
```

You can retype this code into the console anytime you want to re-roll your dice. However, this is an awkward way to work with the code. It would be easier to use your code if you wrapped it into its own function, which is exactly what we'll do now. We're going to write a function named `roll` that you can use to roll your virtual dice. When you're finished, the function will work like this: each time you call `roll()`, R will return the sum of rolling two dice:

4. Up and Running with R

```
roll()
## 8

roll()
## 3

roll()
## 7
```

Functions may seem mysterious or fancy, but they are *just another type of R object*. Instead of containing data, they contain code. This code is stored in a special format that makes it easy to reuse the code in new situations. You can write your own functions by recreating this format.

4.4.1. The Function Constructor

Every function in R has three basic parts: a name, a body of code, and a set of arguments. To make your own function, you need to replicate these parts and store them in an R object, which you can do with the `function` function. To do this, call `function()` and follow it with a pair of braces, `{}`:

```
my_function <- function() {}
```

This function, as written, doesn't do anything (yet). However, it is a valid function. You can call it by typing its name followed by an open and closed parenthesis:

```
my_function()
```

NULL

`function` will build a function out of whatever R code you place between the braces. For example, you can turn your dice code into a function by calling:

```
roll <- function() {
  die <- 1:6
  dice <- sample(die, size = 2, replace = TRUE)
  sum(dice)
}
```

4. Up and Running with R

i Indentation and readability

Notice each line of code between the braces is indented. This makes the code easier to read but has no impact on how the code runs. R ignores spaces and line breaks and executes one complete expression at a time. Note that in other languages like python, spacing is extremely important and part of the language.

Just hit the Enter key between each line after the first brace, {. R will wait for you to type the last brace, }, before it responds.

Don't forget to save the output of `function` to an R object. This object will become your new function. To use it, write the object's name followed by an open and closed parenthesis:

```
roll()
```

```
[1] 7
```

You can think of the parentheses as the “trigger” that causes R to run the function. If you type in a function's name *without* the parentheses, R will show you the code that is stored inside the function. If you type in the name *with* the parentheses, R will run that code:

```
roll
```

```
function ()
{
  die <- 1:6
  dice <- sample(die, size = 2, replace = TRUE)
  sum(dice)
}
```

```
roll()
```

```
[1] 7
```

The code that you place inside your function is known as the *body* of the function. When you run a function in R, R will execute all of the code in the body and then return the result of the last line of code. If the last line of code doesn't return a value, neither will your function, so you want to ensure that your final line of code returns a value. One way

4. Up and Running with R

to check this is to think about what would happen if you ran the body of code line by line in the command line. Would R display a result after the last line, or would it not?

Here's some code that would display a result:

```
dice
1 + 1
sqrt(2)
```

And here's some code that would not:

```
dice <- sample(die, size = 2, replace = TRUE)
two <- 1 + 1
a <- sqrt(2)
```

Again, this is just showing the distinction between expressions and assignments.

4.5. Arguments

What if we removed one line of code from our function and changed the name `die` to `bones` (just a name—don't think of it as important), like this?

```
roll2 <- function() {
  dice <- sample(bones, size = 2, replace = TRUE)
  sum(dice)
}
```

Now I'll get an error when I run the function. The function **needs** the object `bones` to do its job, but there is no object named `bones` to be found (you can check by typing `ls()` which will show you the names in the environment, or memory).

```
roll2()
## Error in sample(bones, size = 2, replace = TRUE) :
##   object 'bones' not found
```

You can supply `bones` when you call `roll2` if you make `bones` an argument of the function. To do this, put the name `bones` in the parentheses that follow `function` when you define `roll2`:

4. Up and Running with R

```
roll2 <- function(bones) {  
  dice <- sample(bones, size = 2, replace = TRUE)  
  sum(dice)  
}
```

Now `roll2` will work as long as you supply `bones` when you call the function. You can take advantage of this to roll different types of dice each time you call `roll2`.

Remember, we're rolling pairs of dice:

```
roll2(bones = 1:4)
```

```
[1] 5
```

```
roll2(bones = 1:6)
```

```
[1] 8
```

```
roll2(1:20)
```

```
[1] 20
```

Notice that `roll2` will still give an error if you do not supply a value for the `bones` argument when you call `roll2`:

```
roll2()  
## Error in sample(bones, size = 2, replace = TRUE) :  
##   argument "bones" is missing, with no default
```

You can prevent this error by giving the `bones` argument a default value. To do this, set `bones` equal to a value when you define `roll2`:

```
roll2 <- function(bones = 1:6) {  
  dice <- sample(bones, size = 2, replace = TRUE)  
  sum(dice)  
}
```


4. Up and Running with R

Now you can supply a new value for `bones` if you like, and `roll2` will use the default if you do not:

```
roll2()
```

```
[1] 7
```

You can give your functions as many arguments as you like. Just list their names, separated by commas, in the parentheses that follow `function`. When the function is run, R will replace each argument name in the function body with the value that the user supplies for the argument. If the user does not supply a value, R will replace the argument name with the argument's default value (if you defined one).

To summarize, `function` helps you construct your own R functions. You create a body of code for your function to run by writing code between the braces that follow `function`. You create arguments for your function to use by supplying their names in the parentheses that follow `function`. Finally, you give your function a name by saving its output to an R object, as shown in Figure 4.6.

Once you've created your function, R will treat it like every other function in R. Think about how useful this is. Have you ever tried to create a new Excel option and add it to Microsoft's menu bar? Or a new slide animation and add it to Powerpoint's options? When you work with a programming language, you can do these types of things. As you learn to program in R, you will be able to create new, customized, reproducible tools for yourself whenever you like.

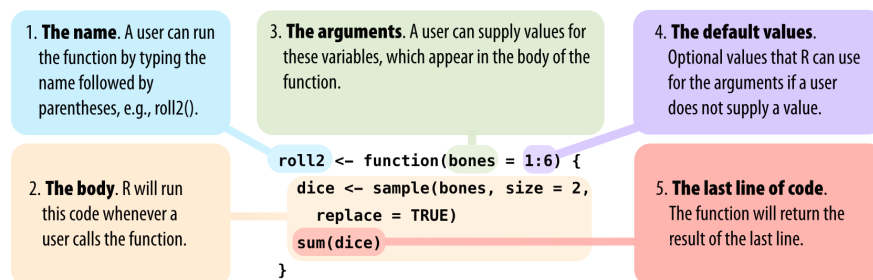


Figure 4.6.: “Every function in R has the same parts, and you can use `function` to create these parts. Assign the result to a name, so you can call the function later.”

4.6. Scripts

Scripts are code that are saved for later reuse or editing. An R script is just a plain text file that you save R code in. You can open an R script in RStudio by going to **File > New File > R script** in the menu bar. RStudio will then open a fresh script above your console pane, as shown in Figure 4.7.

I strongly encourage you to write and edit all of your R code in a script before you run it in the console. Why? This habit creates a reproducible record of your work. When you're finished for the day, you can save your script and then use it to rerun your entire analysis the next day. Scripts are also very handy for editing and proofreading your code, and they make a nice copy of your work to share with others. To save a script, click the scripts pane, and then go to **File > Save As** in the menu bar.

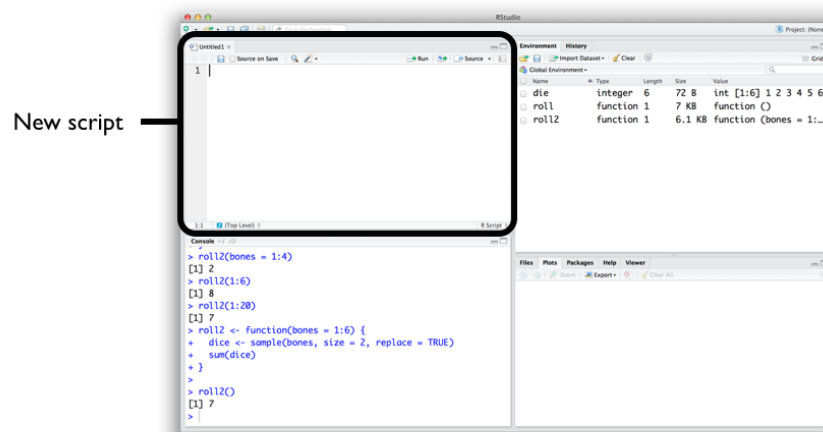


Figure 4.7.: “When you open an R Script (File > New File > R Script in the menu bar), RStudio creates a fourth pane (or puts a new tab in the existing pane) above the console where you can write and edit your code.”

RStudio comes with many built-in features that make it easy to work with scripts. First, you can automatically execute a line of code in a script by clicking the Run button at the top of the editor panel.

R will run whichever line of code your cursor is on. If you have a whole section highlighted, R will run the highlighted code. Alternatively, you can run the entire script by clicking the Source button. Don't like clicking buttons? You can use Control + Return as a shortcut for the Run button. On Macs, that would be Command + Return.

4. Up and Running with R

If you're not convinced about scripts, you soon will be. It becomes a pain to write multi-line code in the console's single-line command line. Let's avoid that headache and open your first script now before we move to the next chapter.

Tip

Extract function

RStudio comes with a tool that can help you build functions. To use it, highlight the lines of code in your R script that you want to turn into a function. Then click **Code > Extract Function** in the menu bar. RStudio will ask you for a function name to use and then wrap your code in a **function** call. It will scan the code for undefined variables and use these as arguments.

You may want to double-check RStudio's work. It assumes that your code is correct, so if it does something surprising, you may have a problem in your code.

4.7. Summary

We've covered a lot of ground already. You now have a virtual die stored in your computer's memory, as well as your own R function that rolls a pair of dice. You've also begun speaking the R language.

The two most important components of the R language are objects, which store data, and functions, which manipulate data. R also uses a host of operators like `+`, `-`, `*`, `/`, and `<-` to do basic tasks. As a data scientist, you will use R objects to store data in your computer's memory, and you will use functions to automate tasks and do complicated calculations.

5. Packages and more dice

We now have code that allows us to roll two dice and add the results together. To keep things interesting, let's aim to weight the dice so that we can fool our friends into thinking we are lucky.

First, though, we should prove to ourselves that our dice are fair. We can investigate the behavior of our dice using two powerful and general tools;

- Simulation (or repetition or repeated sampling)
- Visualization

For the repetition part of things, we will use a built-in R function, `replicate`. For visualization, we are going to use a convenient plotting function, `qplot`. However, `qplot` does not come built into R. We must install a *package* to gain access to it.

5.1. Packages

R is a powerful language for data science and programming, allowing beginners and experts alike to manipulate, analyze, and visualize data effectively. One of the most appealing features of R is its extensive library of packages, which are essential tools for expanding its capabilities and streamlining the coding process.

An R package is a collection of reusable functions, datasets, and compiled code created by other users and developers to extend the functionality of the base R language. These packages cover a wide range of applications, such as data manipulation, statistical analysis, machine learning, and data visualization. By utilizing existing R packages, you can leverage the expertise of others and save time by avoiding the need to create custom functions from scratch.

Using others' R packages is incredibly beneficial as it allows you to take advantage of the collective knowledge of the R community. Developers often create packages to address specific challenges, optimize performance, or implement popular algorithms or methodologies. By incorporating these packages into your projects, you can enhance your productivity, reduce development time, and ensure that you are using well-tested and reliable code.

5.1.1. Installing R packages

To install an R package, you can use the `install.packages()` function in the R console or script. For example, to install the popular data manipulation package “dplyr,” simply type `install.packages("dplyr")`. This command will download the package from the Comprehensive R Archive Network (CRAN) and install it on your local machine. Keep in mind that you only need to install a package once, unless you want to update it to a newer version.

For those who are going to be using R for bioinformatics or biological data science, you will also want to install packages from Bioconductor, which is a repository of R packages specifically designed for bioinformatics and computational biology. To install Bioconductor packages, you can use the `BiocManager::install()` function.

To use this recommended approach, you first need to install the `BiocManager` package, which is the package manager for Bioconductor.

```
install.packages('BiocManager')
```

This is a one-time installation. After that, you can install any R, Bioconductor, rOpenSci, or even GitHub package using the `BiocManager::install()` function. For example, to install the `ggplot2` package, which is widely used for data visualization, you would run:

```
BiocManager::install("ggplot2")
```

5.1.2. Installing vs loading (library) R packages

After installing an R package, you will need to load it into your R session before using its functions. To load a package, use the `library()` function followed by the package name, such as `library(dplyr)`. Loading a package makes its functions and datasets available for use in your current R session. Note that you need to load a package every time you start a new R session.

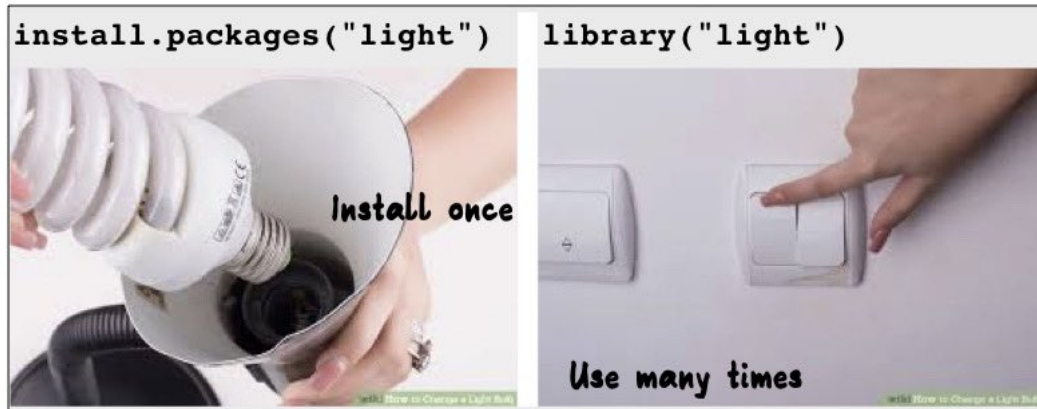
```
library(ggplot2)
```

Now, the functionality of the *ggplot2* package is available in our R session.

5. Packages and more dice

💡 Installing vs loading packages

The main thing to remember is that you only need to install a package once, but you need to load it with `library` each time you wish to use it in a new R session. R will unload all of its packages each time you close RStudio.



Images sourced from <https://www.wikihow.com/Change-a-Light-Bulb>

Figure 5.1.: Installing vs loading R packages.

As in {Figure 5.1}, screw in the lightbulb (eg., `BiocManager::install`) only once and then to use it, you need to turn on the switch each time you want to use it (`library`).

5.1.3. Finding R packages

Finding useful R packages can be done in several ways. First, browsing CRAN (<https://cran.r-project.org/>) and Bioconductor (more later, <https://bioconductor.org>) are an excellent starting points, as they host thousands of packages categorized by topic. Additionally, online forums like Stack Overflow and R-bloggers can provide valuable recommendations based on user experiences. Social media platforms such as Twitter, where developers and data scientists often share new packages and updates, can also be a helpful resource. Finally, don't forget to ask your colleagues or fellow R users for their favorite packages, as they may have insights on which ones best suit your specific needs.

5.1.4. Creating a package

While it may seem overwhelming, creating a package can be fairly simple with the assistance of R packages that provide tips and templates. If interested in exploring package creation, some good starting points are:

- [devtools](#)
- [usethis](#)
- [biocthis](#)
- [roxygen2](#)

5.2. Are our dice fair?

Well, let's review our code.

```
roll2 <- function(bones = 1:6) {
  dice = sample(bones, size = 2, replace = TRUE)
  sum(dice)
}
```

If our dice are fair, then each number should show up equally. What does the sum look like with our two dice?

combinations of die 1 and die 2 that add up to sum						(6,1)					
					(5,1)	(5,2)	(6,2)				
				(4,1)	(4,2)	(4,3)	(5,3)	(6,3)			
			(3,1)	(3,2)	(3,3)	(3,4)	(4,4)	(5,4)	(6,4)		
		(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(3,5)	(4,5)	(5,5)	(6,5)	
	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)	(2,6)	(3,6)	(4,6)	(5,6)	(6,6)
	2	3	4	5	6	7	8	9	10	11	12
	sum										

Figure 5.2.: In an ideal world, a histogram of the results would look like this

5. Packages and more dice

Read the help page for `replicate` (i.e., `help("replicate")`). In short, it suggests that we can repeat our dice rolling as many times as we like and `replicate` will return a *vector* of the sums for each roll.

```
rolls = replicate(n = 100, roll2())
```

What does `rolls` look like?

```
head(rolls)
```

```
[1]  7 12  8  5  5  6
```

```
length(rolls)
```

```
[1] 100
```

```
mean(rolls)
```

```
[1] 6.99
```

```
summary(rolls)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
2.00	5.00	7.00	6.99	8.00	12.00

This looks like it roughly agrees with our sketched out ideal histogram in Figure 5.2. However, now that we've loaded the `qplot` function from the *ggplot2* package, we can make a histogram of the data themselves.

```
qplot(rolls, binwidth=1)
```

Warning: ``qplot()`` was deprecated in *ggplot2* 3.4.0.

5. Packages and more dice

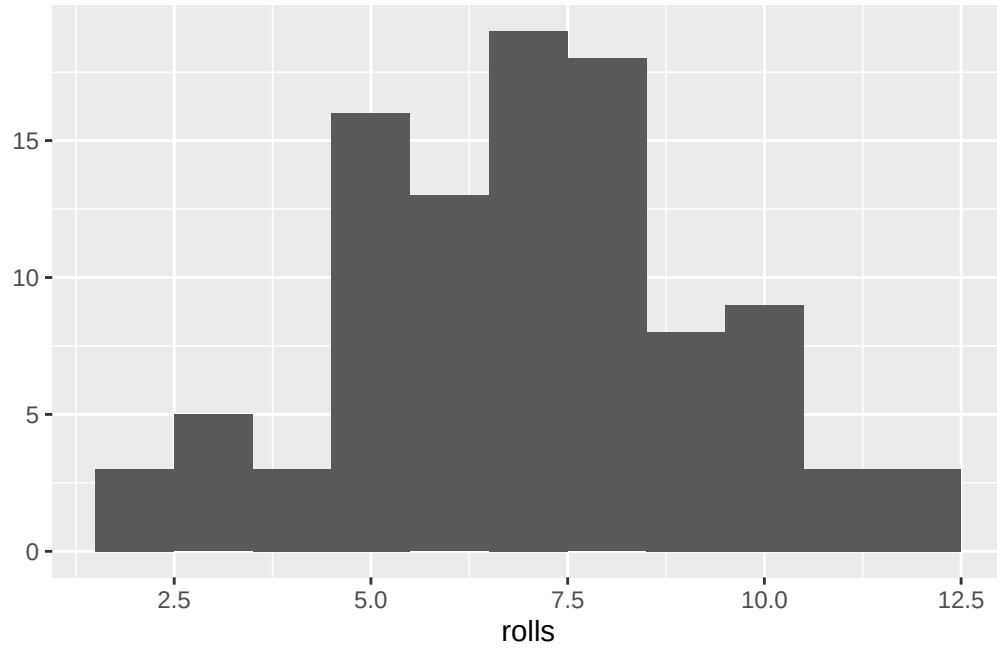


Figure 5.3.: Histogram of the sums from 100 rolls of our fair dice

How does your histogram look (and yours will be different from mine since we are sampling random values)? Is it what you expect?

What happens to our histogram as we increase the number of replicates?

```
rolls = replicate(n = 100000, roll2())  
qplot(rolls, binwidth=1)
```

5. Packages and more dice

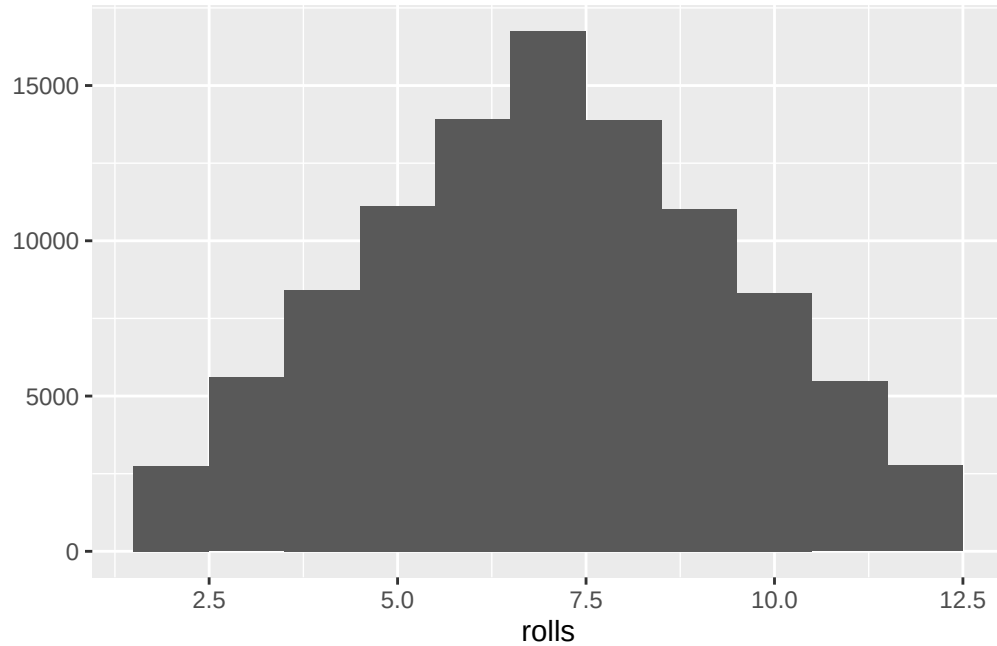


Figure 5.4.: Histogram with 100000 rolls much more closely approximates the pyramidal shape we anticipated

5.3. Bonus exercise

How would you change the `roll2` function to weight the dice?

i Hint

Read the help page for `sample` (i.e., `help("sample")`).

6. Reading and writing data files

6.1. Introduction

In this chapter, we will discuss how to read and write data files in R. Data files are essential for storing and sharing data across different platforms and applications. R provides a variety of functions and packages to read and write data files in different formats, such as text files, CSV files, Excel files. By mastering these functions, you can efficiently import and export data in R, enabling you to perform data analysis and visualization tasks effectively.

6.2. RStudio Projects: Organizing Your Work

Before diving into reading and writing files, it's essential to understand how to organize your work effectively. RStudio Projects provide a powerful way to keep your files, scripts, and data organized in a self-contained workspace.

6.2.1. What are RStudio Projects?

An RStudio Project is a special folder that contains all the files associated with a particular analysis or research project. When you create a project, RStudio creates a `.Rproj` file that serves as the anchor for your project workspace. This approach offers several key benefits:

- **Consistent working directory:** The project folder automatically becomes your working directory
- **File organization:** All related files (scripts, data, outputs) are kept together
- **Reproducibility:** Others can easily run your code without worrying about file paths
- **Version control integration:** Projects work seamlessly with Git and GitHub

6.2.2. Creating an RStudio Project

You can create a new RStudio Project in several ways:

1. **File menu:** Go to **File > New Project...**
2. **Project dropdown:** Click the project dropdown in the top-right corner and select “New Project”
3. **Choose New Directory:** Create a project in a new folder.

When creating a project, you have three main options:

- **New Directory:** Create a fresh project folder
- **Existing Directory:** Turn an existing folder into a project
- **Version Control:** Clone a repository from GitHub or other version control systems

6.2.3. Project Structure Best Practices

A well-organized project typically follows a consistent structure (that YOU define). Here’s a common structure that you might consider:

```
my_analysis_project/  
  my_analysis_project.Rproj  
  data/  
    raw/  
    processed/  
  scripts/  
  notebooks/  
  outputs/  
    figures/  
    tables/  
  README.md  
  .gitignore
```

This structure separates raw data from processed data, keeps scripts organized, and provides clear locations for outputs.

6.2.4. Working Directories and File Paths

One of the most significant advantages of using RStudio Projects is that they solve the common problem of file path management. When you open a project, RStudio automatically sets the working directory to the project folder. This means:

```
# Instead of using absolute paths like this:  
df <- read.csv("/Users/username/Documents/my_analysis/data/dataset.csv")  
  
# You can use relative paths like this:  
df <- read.csv("data/dataset.csv")
```

Relative paths make your code portable—anyone who opens your project will be able to run your scripts without modifying file paths.

6.2.5. The `here` Package for Robust File Paths [optional]

For even more robust file path management, especially in complex projects or when using R Markdown documents, the `here` package is invaluable:

```
install.packages("here")  
library(here)  
  
# The here() function always refers to the project root  
df <- read.csv(here("data", "dataset.csv"))
```

The `here` package works by finding the `.Rproj` file and treating that location as the project root, regardless of where your current script is located within the project hierarchy.

6.2.6. Projects and Reproducibility

RStudio Projects can play a key (but optional) role in creating reproducible analyses. When you share a project folder (or push it to GitHub), collaborators can:

1. Download/clone the entire project
2. Open the `.Rproj` file
3. Run your scripts without any setup or path modifications

6. Reading and writing data files

This seamless workflow is essential for collaborative research and makes your work more credible and verifiable.

Now that we understand how to organize our work with RStudio Projects, let's explore how to read and write the data files that will live within these organized project structures.

6.3. CSV files

Comma-Separated Values (CSV) files are a common file format for storing tabular data. They consist of rows and columns, with each row representing a record and each column representing a variable or attribute. CSV files are widely used for data storage and exchange due to their simplicity and compatibility with various software applications. In R, you can read and write CSV files using the `read.csv()` and `write.csv()` functions, respectively. A commonly used alternative is to use the `readr` package, which provides faster and more user-friendly functions for reading and writing CSV files.

6.3.1. Writing a CSV file

Since we are going to use the `readr` package, we need to install it first. You can install the `readr` package using the following command:

```
install.packages("readr")
```

Once the package is installed, you can load it into your R session using the `library()` function:

```
library(readr)
```

Since we don't have a CSV file sitting around, let's create a simple data frame to write to a CSV file. Here's an example data frame:

```
df <- data.frame(  
  id = c(1, 2, 3, 4, 5),  
  name = c("Alice", "Bob", "Charlie", "David", "Eve"),  
  age = c(25, 30, 35, 40, 45)  
)
```

6. Reading and writing data files

Now, you can write this data frame to a CSV file using the `write_csv()` function from the `readr` package. Here's how you can do it:

```
write_csv(df, "data.csv")
```

You can check the current working directory to see if the CSV file was created successfully. If you want to specify a different directory or file path, you can provide the full path in the `write_csv()` function. If you have created an RStudio Project, the file will be saved in the project directory by default.

```
# see what the current working directory is
getwd()
```

```
[1] "/Users/davsean/Documents/git/RBiocBook"
```

```
# and check to see that the file was created
dir(pattern = "data.csv")
```

```
[1] "data.csv"
```

6.3.2. Reading a CSV file

Now that we have a CSV file, let's read it back into R using the `read_csv()` function from the `readr` package. Here's how you can do it:

```
df2 <- read_csv("data.csv")
```

```
Rows: 5 Columns: 3
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (1): name
```

```
dbl (2): id, age
```

```
i Use `spec()` to retrieve the full column specification for this data.
```

```
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

6. Reading and writing data files

You can check the structure of the data frame `df2` to verify that the data was read correctly:

```
df2
```

```
# A tibble: 5 x 3
  id name    age
  <dbl> <chr> <dbl>
1     1 Alice    25
2     2 Bob     30
3     3 Charlie  35
4     4 David   40
5     5 Eve     45
```

The `readr` package can read CSV files with various delimiters, headers, and data types, making it a versatile tool for handling tabular data in R. It can also read CSV files *directly from web locations* like so:

```
df3 <- read_csv("https://data.cdc.gov/resource/pwn4-m3yp.csv")
```

```
Rows: 1000 Columns: 10
-- Column specification -----
Delimiter: ","
chr  (1): state
dbl  (6): tot_cases, new_cases, tot_deaths, new_deaths, new_historic_cases, ...
dtm  (3): date_updated, start_date, end_date

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

The dataset that you just downloaded is described here: [Covid-19 data from CDC](#). As you can see, the `read_csv()` function automatically detects the structure of the CSV file and imports it into a data frame in R. Using this approach, you can easily read CSV files from various sources, including local files and online datasets. Particularly for analysis of public datasets, this method is very useful as it allows you to access and analyze data without needing to download it manually and provides a high degree of reproducibility in your analyses.

6.4. Excel files

Microsoft Excel files are another common file format for storing tabular data. Excel files can contain multiple sheets, formulas, and formatting options, making them a popular choice for data storage and analysis. In R, you can read and write Excel files using the `readxl` package. This package provides functions to import and export data from Excel files, enabling you to work with Excel data in R.

6.4.1. Reading an Excel file

To read an Excel file in R, you need to install and load the `readxl` package. You can install the `readxl` package using the following command:

```
install.packages("readxl")
```

Once the package is installed, you can load it into your R session using the `library()` function:

```
library(readxl)
```

Now, you can read an Excel file using the `read_excel()` function from the `readxl` package. We don't have an excel file available, so let's download one from the internet. Unlike with CSV files, R needs to download the file first before reading it. Here's an example of how to use R to download an Excel file:

```
download.file('https://www.w3resource.com/python-exercises/pandas/excel/SaleData.xlsx', 'Sa
```

You can check the current working directory to see if the Excel file was downloaded successfully:

```
dir('.')
```

```
[1] "_book"
[2] "_extensions"
[3] "_freeze"
[4] "_quarto.yml"
[5] "310_microbiome.qmd"
[6] "additional_resources.qmd"
```

6. Reading and writing data files

```
[7] "ai_tools.qmd"
[8] "appendix.qmd"
[9] "atac-seq"
[10] "atac-seq-talk.qmd"
[11] "bibliography.bib"
[12] "bioc_ranges.qmd"
[13] "bioc-summarizedexperiment.qmd"
[14] "BRFSS-subset.csv"
[15] "CSHLDataCourse.qmd"
[16] "data"
[17] "data_structures_overview.qmd"
[18] "data_vis_hadley.qmd"
[19] "data.csv"
[20] "data.xlsx"
[21] "dataframes_intro_cache"
[22] "dataframes_intro_files"
[23] "dataframes_intro.qmd"
[24] "dataviz.qmd"
[25] "DESCRIPTION"
[26] "dplyr_intro_msleep.qmd"
[27] "dplyr.qmd"
[28] "drawings"
[29] "eda_and_univariate_brfss_cache"
[30] "eda_and_univariate_brfss_files"
[31] "eda_and_univariate_brfss.qmd"
[32] "eda_overview.qmd"
[33] "eda_with_pca.qmd"
[34] "factors.qmd"
[35] "genomic_ranges_tutorial.qmd"
[36] "GenomicRanges Bioconductor educational examples_.md"
[37] "geoquery_cache"
[38] "geoquery_files"
[39] "geoquery.qmd"
[40] "ggplot2"
[41] "git_and_github.qmd"
[42] "greenleaf.bw"
[43] "GSE2553_series_matrix.txt.gz"
[44] "GSE74089_series_matrix.txt.gz"
[45] "images"
[46] "index_files"
[47] "index.aux"
```

6. Reading and writing data files

```
[48] "index.html"
[49] "index.idx"
[50] "index.lof"
[51] "index.log"
[52] "index.lot"
[53] "index.pdf"
[54] "index.qmd"
[55] "index.tex"
[56] "index.toc"
[57] "intro_files"
[58] "intro_to_rstudio.html"
[59] "intro_to_rstudio.qmd"
[60] "intro.html"
[61] "intro.qmd"
[62] "kmeans.qmd"
[63] "license.qmd"
[64] "lists.qmd"
[65] "machine_learning"
[66] "machine_learning_mlr3_cache"
[67] "machine_learning_mlr3_files"
[68] "matrices.qmd"
[69] "matrix_exercises.qmd"
[70] "ml_practical.qmd"
[71] "NAMESPACE"
[72] "norm.qmd"
[73] "packages_and_dice_files"
[74] "packages_and_dice.html"
[75] "packages_and_dice.qmd"
[76] "protein_simulation_basics.qmd"
[77] "QTDublinIrish.otf"
[78] "r_basics.html"
[79] "r_basics.qmd"
[80] "r_intro_mechanics_cache"
[81] "r_intro_mechanics.html"
[82] "r_intro_mechanics.qmd"
[83] "ranges_and_signals_cache"
[84] "ranges_and_signals_files"
[85] "ranges_and_signals.qmd"
[86] "ranges_exercises_cache"
[87] "ranges_exercises.qmd"
[88] "RBiocBook.Rproj"
```

6. Reading and writing data files

```
[89] "reading_and_writing_files"
[90] "reading_and_writing.html"
[91] "reading_and_writing.qmd"
[92] "reading_and_writing.rmarkdown"
[93] "references.qmd"
[94] "SaleData.xlsx"
[95] "SE.svg"
[96] "simulation_basics_files"
[97] "simulation_basics.qmd"
[98] "single_cell"
[99] "site_libs"
[100] "t-statistic-simulation-exercises.qmd"
[101] "t-stats-and-tests.qmd"
[102] "talks"
[103] "tikz2d0f3bd7aa39.log"
[104] "tikz2e8e2666c9c0.log"
[105] "tikz3ed045a6259e.log"
[106] "tikzdc60f2f7d7b.log"
[107] "tikzf28a6369b1c3.log"
[108] "tikzf696458ac6c2.log"
[109] "tikzf83c5476af83.log"
[110] "transcriptdb.qmd"
[111] "vectors.qmd"
[112] "visualization_guide.qmd"
[113] "working_with_distributions.qmd"
```

Now, you can read the Excel file into R using the `read_excel()` function:

```
df_excel <- read_excel("SaleData.xlsx")
```

You can check the structure of the data frame `df_excel` to verify that the data was read correctly:

```
df_excel
```

```
# A tibble: 45 x 8
```

	OrderDate	Region	Manager	SalesMan	Item	Units	Unit_price	Sale_amt
	<dtm>	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>
1	2018-01-06 00:00:00	East	Martha	Alexander	Tele~	95	1198	113810
2	2018-01-23 00:00:00	Central	Hermann	Shell	Home~	50	500	25000

6. Reading and writing data files

```
3 2018-02-09 00:00:00 Central Hermann Luis      Tele~    36      1198    43128
4 2018-02-26 00:00:00 Central Timothy David    Cell~    27       225     6075
5 2018-03-15 00:00:00 West    Timothy Stephen Tele~    56      1198    67088
6 2018-04-01 00:00:00 East    Martha Alexander Home~    60       500    30000
7 2018-04-18 00:00:00 Central Martha Steven    Tele~    75      1198    89850
8 2018-05-05 00:00:00 Central Hermann Luis      Tele~    90      1198   107820
9 2018-05-22 00:00:00 West    Douglas Michael Tele~    32      1198    38336
10 2018-06-08 00:00:00 East    Martha Alexander Home~    60       500    30000
# i 35 more rows
```

The `readxl` package provides various options to read Excel files with multiple sheets, specific ranges, and data types, making it a versatile tool for handling Excel data in R.

6.4.2. Writing an Excel file

To write an Excel file in R, you can use the `write_xlsx()` function from the `writexl` package. You can install the `writexl` package using the following command:

```
install.packages("writexl")
```

Once the package is installed, you can load it into your R session using the `library()` function:

```
library(writexl)
```

The `write_xlsx()` function allows you to write a data frame to an Excel file. Here's an example:

```
write_xlsx(df, "data.xlsx")
```

You can check the current working directory to see if the Excel file was created successfully. If you want to specify a different directory or file path, you can provide the full path in the `write_xlsx()` function.

```
# see what the current working directory is
getwd()
```

```
[1] "/Users/davsean/Documents/git/RBiocBook"
```

6. Reading and writing data files

```
# and check to see that the file was created
dir(pattern = "data.xlsx")
```

```
[1] "data.xlsx"
```

And, of course, if you find the file on your computer, you can open it in Excel to verify that the data was written correctly.

6.5. Additional options

- Google Sheets: You can read and write data from Google Sheets using the `googlesheets4` package. This package provides functions to interact with Google Sheets, enabling you to import and export data from Google Sheets to R.
- JSON files: You can read and write JSON files using the `jsonlite` package. This package provides functions to convert R objects to JSON format and vice versa, enabling you to work with JSON data in R.
- Database files: You can read and write data from database files using the DBI and `RSQLite` packages. These packages provide functions to interact with various database systems, enabling you to import and export data from databases to R.

Part II.

R Data Structures

Chapter overview

Welcome to the section on R data structures! As you begin your journey in learning R, it is essential to understand the fundamental building blocks of this powerful programming language. R offers a variety of data structures to store and manipulate data, each with its unique properties and capabilities. In this section, we will cover the core data structures in R, including:

- Vectors
- Matrices
- Lists
- Data.frames

By the end of this section, you will have a solid understanding of these data structures, and you will be able to choose and utilize the appropriate data structure for your specific data manipulation and analysis tasks.

In each chapter, we will delve into the properties and usage of each data structure, starting with their definitions and moving on to their practical applications. We will provide examples, exercises, and active learning approaches to help you better understand and apply these concepts in your work.

Chapter overview

- **Vectors** In this chapter, we will introduce you to the simplest data structure in R, the vector. We will cover how to create, access, and manipulate vectors, as well as discuss their unique properties and limitations.
- **Matrices** Next, we will explore matrices, which are two-dimensional data structures that extend vectors. You will learn how to create, access, and manipulate matrices, and understand their usefulness in mathematical operations and data organization.
- **Lists** The third chapter will focus on lists, a versatile data structure that can store elements of different types and sizes. We will discuss how to create, access, and modify lists, and demonstrate their flexibility in handling complex data structures.
- **Data.frames** Finally, we will examine data.frames, a widely-used data structure for organizing and manipulating tabular data. You will learn how to create, access, and manipulate data.frames, and understand their advantages over other data structures for data analysis tasks.

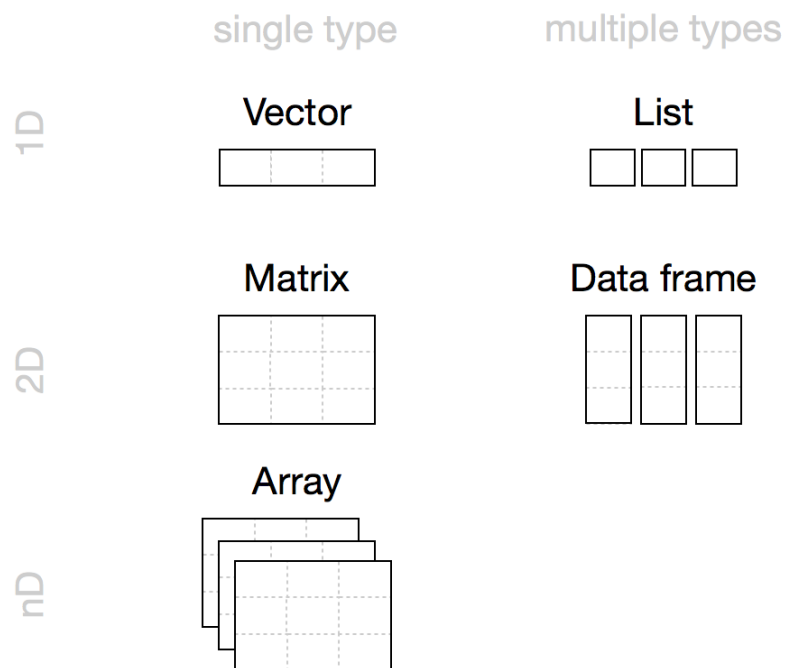


Figure 6.1.: A pictorial representation of R's most common data structures are vectors, matrices, arrays, lists, and dataframes. Figure from [Hands-on Programming with R](#).

Chapter overview

- **Arrays** While we will not focus directly on the `array` data type, which are multidimensional data structures that extend matrices, they are very similar to matrices, but with a third dimension.

As you progress through these chapters, practice the examples and exercises provided, engage in discussion, and collaborate with your peers to deepen your understanding of R data structures. This solid foundation will serve as the basis for more advanced data manipulation, analysis, and visualization techniques in R.

7. Vectors

Suppose you’ve just measured the expression of one gene across a dozen samples, or jotted down the ages of every patient in a study, or listed the gene names you want to pull out of a results table. Each of those is the same shape: a single, ordered run of values of one kind. R has a container built exactly for that shape, and it is the workhorse you’ll reach for more than any other. It’s called a **vector**.

Almost everything in R is built on vectors. A column of a data frame is a vector. The output of most calculations is a vector. Even a single number, as you’ll see in a moment, is just a vector of length one. So getting comfortable here pays off everywhere downstream. Let’s build that comfort.

7.1. What you’ll learn

- Create vectors with `c()`, the colon operator `:`, and `seq()`.
- Do arithmetic on whole vectors at once, and recognize the **recycling** rule.
- Build logical vectors with comparisons like `>` and `==`.
- Pull elements out of a vector by position, by name, and by a logical test.
- Work with character vectors using `paste()`, `nchar()`, `substr()`, and `grep()`.
- Spot and handle missing values (`NA`).

7.2. A vector is a column of measurements

A vector is a one-dimensional, ordered collection of elements that are all the same type. That last part matters: every value in a vector is numeric, *or* every value is text, *or* every value is logical — never a mix. Figure 7.1 shows three vectors side by side: a numeric one, a logical one, and a character one. Notice that each element has a position (an *index*) and can optionally carry a *name*.

Here’s the first surprise. In R, even a lone value is a vector — one of length 1:

7. Vectors

Index	1	2	3	4	5	6	7
Vector	3	7	10	NA	932	127	-3
Vector	TRUE	FALSE	FALSE	TRUE	TRUE	FALSE	NA
Vector	"Cat"	"Dog"	"A"	"C"	"T"	NA	"G"
Names (Optional)	"H"	"I"	"L"	"Z"	"This"	"That"	"Other"

Figure 7.1.: “Pictorial representation of three vector examples. The first vector is a numeric vector. The second is a ‘logical’ vector. The third is a character vector. Vectors also have indices and, optionally, names.”

```
gene_count <- 1
gene_count
```

```
[1] 1
```

```
length(gene_count)
```

```
[1] 1
```

We *assigned* the value 1 to a variable named `gene_count`. Typing `gene_count` by itself is an *expression*: it returns whatever is stored there. The `length()` function reports how many elements an object holds — here, just one. R gives you many ways to interrogate an object, and `length()` is one of the friendliest.

A vector can hold numbers, text (character data), or logical values (`TRUE` and `FALSE`), or other “atomic” types from Table 7.1. What it *cannot* do is hold a mix of types. When you need a grab-bag of different types in one container, R offers a different structure, the `list`, which you’ll meet in a later chapter.

7. Vectors

Table 7.1.: Atomic (simplest) data types in R.

Data type	Stores
numeric	floating point numbers
integer	integers
complex	complex numbers
factor	categorical data
character	strings
logical	TRUE or FALSE
NA	missing
NULL	empty
function	function type

! One type per vector

If you try to mix types, R doesn't error — it quietly *coerces* everything to a single type that can hold all the values. Watch what happens when a logical meets a string: the TRUE becomes the text "TRUE", quotes and all. Keep this in the back of your mind; a column that “looks numeric” but secretly contains one stray word will arrive as character, and your math will fail in confusing ways.

7.3. Creating vectors

The most direct way to build a vector is `c()`, short for *combine*. Character values go in matching quotes — single or double, your choice — and R always displays them back with double quotes.

```
# a few vectors of different types
c("BRCA1", "TP53")
```

```
[1] "BRCA1" "TP53"
```

```
c(1, 3, 4, 5, 1, 2)
```

```
[1] 1 3 4 5 1 2
```

7. Vectors

```
c(1.12341e7, 78234.126)
```

```
[1] 11234100.00    78234.13
```

```
c(TRUE, FALSE, TRUE, TRUE)
```

```
[1] TRUE FALSE TRUE TRUE
```

```
# mixing a logical and a string forces BOTH to character:  
c(TRUE, "hello")
```

```
[1] "TRUE" "hello"
```

That last line is the coercion we just warned about: `TRUE` came back as `"TRUE"`.

When you want a regular run of integers, the colon operator `:` is the quickest tool. It counts from the first number to the second, in either direction.

```
# the integers 1 through 10  
sample_ids <- 1:10  
sample_ids
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

```
# ...and counting back down  
countdown <- 10:1  
countdown
```

```
[1] 10 9 8 7 6 5 4 3 2 1
```

For sequences with a custom step size, `seq()` is more flexible. Here we step by 0.3 instead of 1:

```
# from 1 to 4 in steps of 0.3  
grid <- seq(1, 4, by = 0.3)  
grid
```

7. Vectors

```
[1] 1.0 1.3 1.6 1.9 2.2 2.5 2.8 3.1 3.4 3.7 4.0
```

And you can build a new vector by gluing existing ones together with `c()`:

```
# concatenate two vectors into one
combined <- c(grid, sample_ids)
combined
```

```
[1] 1.0 1.3 1.6 1.9 2.2 2.5 2.8 3.1 3.4 3.7 4.0 1.0 2.0 3.0 4.0
[16] 5.0 6.0 7.0 8.0 9.0 10.0
```

7.4. Vector operations

Here is where vectors earn their keep. Arithmetic on a vector happens **element-by-element**. Add 2 to a vector and R adds 2 to every element, handing back a new vector of the same length. You never have to write a loop.

```
expression <- 1:10
expression + 2
```

```
[1] 3 4 5 6 7 8 9 10 11 12
```

Think of how often that's what you want: shift every measurement, scale every value, take the log of a whole column. One short line does it.

When two vectors are involved, the rules depend on their lengths. If they're the **same length**, R simply pairs up matching elements and operates on each pair:

```
expression + expression
```

```
[1] 2 4 6 8 10 12 14 16 18 20
```

If the lengths differ but the longer is a **multiple** of the shorter, R reuses the shorter vector from the start as many times as needed. Here the two-element vector `c(1, 2)` is repeated five times to line up with the ten-element vector:

7. Vectors

```
expression <- 1:10  
pair <- c(1, 2)  
expression * pair
```

```
[1] 1 4 3 8 5 12 7 16 9 20
```

If the longer length is **not** a clean multiple of the shorter, R still recycles the short vector — but warns you, because this is usually a sign of a mistake:

```
expression <- 1:10  
triple <- c(2, 3, 4)  
expression * triple
```

Warning in expression * triple: longer object length is not a multiple of shorter object length

```
[1] 2 6 12 8 15 24 14 24 36 20
```

The usual arithmetic is all here — multiplication (*), addition (+), subtraction (-), division (/), exponentiation (^) — and because each acts on whole vectors, we call these operations *vectorized*.

The recycling rule can bite

When you combine vectors of different lengths, R silently stretches the shorter one to match. If that wasn't your intent, you'll get plausible-looking but wrong numbers and no error. The “not a multiple” case at least prints a warning; the “clean multiple” case prints nothing at all. When two vectors should be the same length, check that they are — `length(x)` is your friend.

7.5. Logical vectors

A logical vector holds only **TRUE** and **FALSE** (all uppercase, no quotes). These turn up constantly, because asking a yes/no question of a whole vector returns one.

7. Vectors

```
keep <- c(TRUE, FALSE, TRUE)

# a logical vector can also be built from numbers:
# 0 becomes FALSE, anything else becomes TRUE
counts <- c(1, 0, 217)
as_flags <- as.logical(counts)
as_flags
```

```
[1] TRUE FALSE TRUE
```

```
# do keep and as_flags match element-by-element?
all.equal(keep, as_flags)
```

```
[1] TRUE
```

```
# and logical converts back to numeric: TRUE -> 1, FALSE -> 0
as.numeric(keep)
```

```
[1] 1 0 1
```

7.5.1. Logical operators

The comparison operators `<`, `>`, `==`, `>=`, `<=`, and `!=` each take a vector and return a logical vector — one TRUE/FALSE per element. This is the engine behind filtering data.

```
expression <- 1:10
# which elements are greater than 5?
expression > 5
```

```
[1] FALSE FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE
```

```
expression <= 5
```

```
[1] TRUE TRUE TRUE TRUE TRUE FALSE FALSE FALSE FALSE FALSE
```

7. Vectors

```
expression != 5
```

```
[1] TRUE TRUE TRUE TRUE FALSE TRUE TRUE TRUE TRUE TRUE
```

```
expression == 5
```

```
[1] FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE
```

You can save that logical result in a variable to reuse later:

```
is_five <- (expression == 5)
is_five
```

```
[1] FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE
```

 = assigns, == compares

A single = is assignment; a double == is a test for equality. Writing `x = 5` *changes* `x`, while `x == 5` *asks whether* `x` equals 5. Mixing them up is one of the most common beginner errors. In this book we always assign with `<-`, which keeps the two jobs visually distinct.

7.6. Indexing vectors

An *index* lets you reach into a vector and pull out one element or a handful. R uses square brackets, [and], for this.

 R counts from 1

Unlike many programming languages that start counting at 0, R starts at **1**. The first element of `x` is `x[1]`, the second is `x[2]`, and so on. If you've programmed before, this is a frequent source of off-by-one surprises.

```
x <- seq(0, 1, by = 0.1)
# the 4th element
x[4]
```

7. Vectors

```
[1] 0.3
```

You can index with a *vector* of positions to grab several elements at once:

```
x[c(3, 5, 6)]
```

```
[1] 0.2 0.4 0.5
```

```
positions <- 3:6  
x[positions]
```

```
[1] 0.2 0.3 0.4 0.5
```

Now combine indexing with logical vectors, and you have one of the most powerful patterns in all of R. Here we generate ten random values, flag the ones above 0.25, and keep only those:

```
# rnorm() draws random numbers; see help('rnorm')  
measurements <- rnorm(10)  
  
# a logical vector: TRUE where measurements exceed 0.25  
above <- (measurements > 0.25)  
# sum() of a logical vector counts the TRUEs  
sum(above)
```

```
[1] 5
```

```
# keep only the values where 'above' is TRUE  
measurements[above]
```

```
[1] 1.5952808 0.3295078 0.4874291 0.7383247 0.5757814
```

```
# or the complement, using the logical NOT operator "!"  
measurements[!above]
```

```
[1] -0.6264538 0.1836433 -0.8356286 -0.8204684 -0.3053884
```

7. Vectors

```
# the same idea in one compact line
measurements[measurements > 0.25]
```

```
[1] 1.5952808 0.3295078 0.4874291 0.7383247 0.5757814
```

That last line — *subset a vector by a condition on itself* — is something you’ll write hundreds of times. Read it as “the measurements where measurements exceed 0.25.”

7.7. Named vectors

Elements can carry names as well as positions, which often makes code far more readable. Imagine a small set of patient ages keyed by patient ID. You attach names with `names()`:

```
patient_ages <- c(54, 39, 71)
names(patient_ages) <- c("p01", "p02", "p03")
patient_ages
```

```
p01 p02 p03
 54  39  71
```

Once a vector is named, you can access elements by name instead of remembering their position:

```
patient_ages["p02"]
```

```
p02
 39
```

```
# and update by name, too
patient_ages["p01"] <- 55
patient_ages
```

```
p01 p02 p03
 55  39  71
```

Names are just labels riding alongside the values; the vector is still a plain numeric vector underneath.

7.8. Character vectors, a.k.a. strings

Text data — gene names, sample labels, file paths — lives in character vectors, and R has a toolkit for manipulating them. To glue strings together, use `paste()`:

```
paste("BRCA", "1")
```

```
[1] "BRCA 1"
```

```
paste("BRCA", "1", sep = "_")
```

```
[1] "BRCA_1"
```

```
paste0("BRCA", "1")          # paste0 uses no separator
```

```
[1] "BRCA1"
```

```
paste(c("sample", "control"), 1:10)
```

```
[1] "sample 1"  "control 2" "sample 3"  "control 4" "sample 5"
[6] "control 6" "sample 7"  "control 8" "sample 9"  "control 10"
```

```
paste(c("sample", "control"), 1:10, sep = "_")
```

```
[1] "sample_1"  "control_2" "sample_3"  "control_4" "sample_5"
[6] "control_6" "sample_7"  "control_8" "sample_9"  "control_10"
```

Notice `paste()` is vectorized too: in the last two lines it recycles the two-element vector across the ten numbers.

To count characters in each string, use `nchar()`:

```
nchar("BRCA1")
```

```
[1] 5
```

7. Vectors

```
nchar(c("BRCA1", "TP53", "EGFR"))
```

```
[1] 5 4 4
```

To pull a piece out of a string by position, use `substr()`:

```
substr("chr7:55019021", start = 1, stop = 4)
```

```
[1] "chr7"
```

To find-and-replace within a string, use `sub()`:

```
sub("chr", "Chromosome ", "chr7:55019021")
```

```
[1] "Chromosome 7:55019021"
```

And to find *which* strings in a vector contain a pattern, use `grep()` — the name comes from “globally search a regular expression and print.” By default it returns the matching *positions*; add `value = TRUE` to get the matching strings themselves:

```
genes <- c("BRCA1", "BRCA2", "TP53", "EGFR", "BARD1")
# positions of genes containing "BR"
grep("BR", genes)
```

```
[1] 1 2
```

```
# the matching gene names themselves
grep("BR", genes, value = TRUE)
```

```
[1] "BRCA1" "BRCA2"
```

A close relative, `grep1()`, answers the yes/no version of that question — you’ll meet it in the exercises.

7.9. Missing values, a.k.a. NA

Real data has holes: an assay that failed, a measurement never taken. R marks such gaps with the special value `NA` (*Not Available*). Crucially, `NA` is still an element — it occupies a position and counts toward `length()`.

```
x <- 1:5
x
```

```
[1] 1 2 3 4 5
```

```
length(x)
```

```
[1] 5
```

The function `is.na()` tests each element and returns a logical vector. Let's poke a hole in `x` by setting its second element to `NA`:

```
is.na(x)
```

```
[1] FALSE FALSE FALSE FALSE FALSE
```

```
x[2] <- NA
x
```

```
[1] 1 NA 3 4 5
```

The length hasn't changed — `x` still has five slots — but one of them is now flagged as missing:

```
length(x)
```

```
[1] 5
```

7. Vectors

```
is.na(x)
```

```
[1] FALSE TRUE FALSE FALSE FALSE
```

To drop the missing values, lean on the indexing pattern from earlier. `is.na(x)` gives a logical vector that is `TRUE` at the gaps; the `!` operator flips it to `TRUE` everywhere there's a real value; indexing with that keeps only the real values:

```
x[!is.na(x)]
```

```
[1] 1 3 4 5
```

7.10. Exercises

Each solution names the key idea and a one-line *why*, because the reasoning transfers further than the answer.

1. Create a numeric vector called `temperatures` containing the values 72, 75, 78, 81, 76, and 73.

i Solution

```
temperatures <- c(72, 75, 78, 81, 76, 73)
temperatures
```

```
[1] 72 75 78 81 76 73
```

`c()` combines individual values into one numeric vector — the most basic way to build a vector by hand.

2. Create a character vector called `days` containing “Monday”, “Tuesday”, “Wednesday”, “Thursday”, “Friday”, “Saturday”, and “Sunday”.

i Solution

```
days <- c("Monday", "Tuesday", "Wednesday", "Thursday",
          "Friday", "Saturday", "Sunday")
days
```


7. Vectors

```
[1] "Monday"    "Tuesday"    "Wednesday" "Thursday"   "Friday"     "Saturday"
[7] "Sunday"
```

The same `c()`, now with quoted strings, so R makes a character vector instead of a numeric one.

3. Calculate the average of `temperatures` and store it in `average_temperature`.

i Solution

```
average_temperature <- mean(temperatures)
average_temperature
```

```
[1] 75.83333
```

`mean()` reduces a whole numeric vector to a single summary value.

4. Build a named vector `weekly_temperatures` whose names are the (first six) days and whose values are the six temperatures.

i Solution

```
weekly_temperatures <- temperatures
names(weekly_temperatures) <- days[1:6]
weekly_temperatures
```

Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
72	75	78	81	76	73

`names()` attaches labels to existing elements; we use `days[1:6]` so the six names line up with the six temperatures.

5. Create a numeric vector `ages` with the values 25, 30, 35, 40, 45, 50, 55, and 60.

i Solution

```
ages <- c(25, 30, 35, 40, 45, 50, 55, 60)
ages
```

```
[1] 25 30 35 40 45 50 55 60
```

Again `c()` — the same tool builds every vector regardless of the numbers inside.

6. Create a logical vector `is_adult` that is `TRUE` where `ages` is at least 18.

7. Vectors

i Solution

```
is_adult <- ages >= 18
is_adult
```

```
[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
```

A comparison applied to a vector returns a logical vector, one TRUE/FALSE per element.

7. Calculate both the sum and the product of `ages`.

i Solution

```
sum_ages <- sum(ages)
product_ages <- prod(ages)
sum_ages
```

```
[1] 340
```

```
product_ages
```

```
[1] 7.79625e+12
```

`sum()` and `prod()` are reductions: each collapses the whole vector to one number.

8. Extract the ages greater than or equal to 40 into a vector called `older_ages`.

i Solution

```
older_ages <- ages[ages >= 40]
older_ages
```

```
[1] 40 45 50 55 60
```

This is logical indexing: `ages >= 40` builds a TRUE/FALSE mask, and `ages[...]` keeps only the elements where it is TRUE.

9. The `grepl()` function (read `?grepl`) returns a logical vector — TRUE where a pattern is found, FALSE where it isn't. Using the vector below, return a logical vector that is TRUE for each entry containing the letter "a".

```
words <- c("abcdef", "abcd", "bcde", "cdef", "defg")
```

i Solution

```
grepl("a", words)
```

```
[1] TRUE TRUE FALSE FALSE FALSE
```

Where `grep()` returns *positions*, `grepl()` returns a logical vector you can feed straight into indexing — for example, `words[grepl("a", words)]` keeps only the matching entries.

10. From `measurements <- rnorm(20)`, count how many values are negative without writing a loop.

i Solution

```
set.seed(1)
measurements <- rnorm(20)
sum(measurements < 0)
```

```
[1] 8
```

`measurements < 0` is a logical vector, and `sum()` counts its `TRUE`s because `TRUE` equals 1 — the same count-by-summing trick from the indexing section.

7.11. Summary

You can now create and wield the container R uses for almost everything:

- **Create** vectors with `c()`, the colon operator `:`, and `seq()` — and recall that even a single value is a length-1 vector, all of one type.
- **Operate** on whole vectors at once; arithmetic is element-by-element, and the **recycling rule** stretches a shorter vector to match a longer one (with a warning when the lengths don't divide evenly).
- **Compare** with `>`, `==`, `!=`, and friends to produce logical vectors.
- **Index** with positions, with names, or with a logical test — and remember R counts from 1.
- **Manipulate strings** with `paste()`, `nchar()`, `substr()`, `sub()`, and `grep()/grepl()`.
- **Handle gaps** with `NA` and `is.na()`, dropping them via `x[!is.na(x)]`.

The logical-indexing pattern — ask a yes/no question of a vector, then use the answer to subset it — is the single most useful idea here. It reappears every time you filter data, so the practice you put in now pays off in every chapter to come.

8. Matrices

Picture the result of a genomics experiment: you measured the activity of twenty thousand genes across a dozen samples. The natural way to hold that is a grid — one row per gene, one column per sample, a number in every cell. That grid is a **matrix**, and it is the central object you’ll meet again and again in genomics. The gene-expression matrix you build here is the same shape that later powers a **SummarizedExperiment**, so the moves you learn now pay off for the rest of the book.

A matrix is just a rectangle of values that are **all the same type** — all numbers, all characters, or all logicals (see Figure 8.1). You can read it as a stack of columns (each column a vector of the same length and type) or as a stack of rows. If you’ve already met vectors, a matrix is the natural next step: take several vectors of equal length and the same type, and line them up side by side.

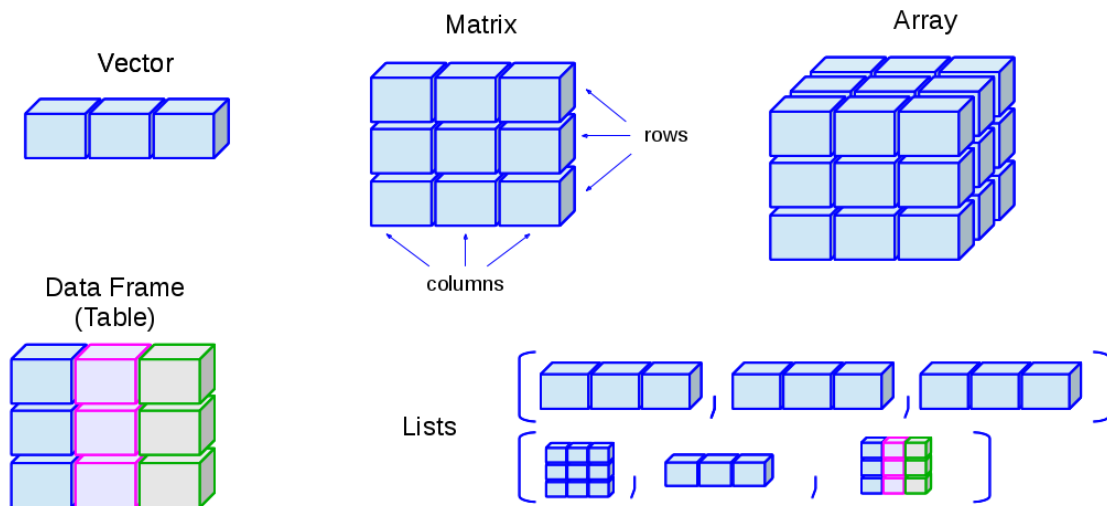


Figure 8.1.: A matrix is a collection of column vectors, all the same length and the same type.

i Matrix versus data frame

A *data frame* is also a rectangle, and it's easy to confuse the two. The difference is one rule: a matrix needs every value to be the **same type**, while a data frame lets each **column** have its own type — numbers in one, text in another. Use a matrix when your whole grid is numeric (like expression values); reach for a data frame when columns mix types (a sample table with names, ages, and treatment labels). The two even name their rows and columns differently under the hood, which is a classic source of confusion — but that's a problem for the data-frame chapter, not this one.

8.1. What you'll learn

- Build a matrix with `matrix()`, `rbind()`, and `cbind()`.
- Inspect a matrix's shape and labels with `dim()`, `nrow()`, `ncol()`, `rownames()`, and `colnames()`.
- Pull out elements, rows, and columns with `[row, col]` indexing.
- Change values in part of a matrix by combining subsetting with assignment.
- Summarize across rows and columns with `rowMeans()`, `colMeans()`, and `apply()` — the per-gene and per-sample summaries you'll use constantly in genomics.

8.2. Creating a matrix

There are many ways to build a matrix in R. The most direct is the `matrix()` function. Below we pour the numbers 1:16 into a four-row grid.

```
mat1 <- matrix(1:16, nrow = 4)
mat1
```

```
      [,1] [,2] [,3] [,4]
[1,]    1    5    9   13
[2,]    2    6   10   14
[3,]    3    7   11   15
[4,]    4    8   12   16
```

By default R fills the matrix **column by column** — walk down the first column, then the second, and so on. If you'd rather fill it row by row, set `byrow = TRUE`.

8. Matrices

```
mat1 <- matrix(1:16, nrow = 4, byrow = TRUE)
mat1
```

```
      [,1] [,2] [,3] [,4]
[1,]     1     2     3     4
[2,]     5     6     7     8
[3,]     9    10    11    12
[4,]    13    14    15    16
```

Look closely at where the numbers land. Same sixteen values, same shape, but a completely different arrangement. Whenever you create a matrix from a flat vector, ask yourself which direction it filled.

We can also assemble a matrix from parts by *binding* vectors together. Let's make two numeric vectors of length 10.

```
gene_a <- 1:10
gene_b <- rnorm(10)
```

Both are numeric and both have length 10, so they can sit together in a matrix. `rbind()` stacks them as rows (think “r for row”):

```
mat <- rbind(gene_a, gene_b)
mat
```

```
      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
gene_a 1.0000000 2.0000000 3.0000000 4.0000000 5.0000000 6.0000000 7.0000000
gene_b -0.6264538 0.1836433 -0.8356286 1.595281 0.3295078 -0.8204684 0.4874291
      [,8] [,9] [,10]
gene_a 8.0000000 9.0000000 10.0000000
gene_b 0.7383247 0.5757814 -0.3053884
```

The companion function `cbind()` glues vectors together as columns:

```
mat <- cbind(gene_a, gene_b)
mat
```

8. Matrices

```
      gene_a    gene_b
[1,]      1 -0.6264538
[2,]      2  0.1836433
[3,]      3 -0.8356286
[4,]      4  1.5952808
[5,]      5  0.3295078
[6,]      6 -0.8204684
[7,]      7  0.4874291
[8,]      8  0.7383247
[9,]      9  0.5757814
[10,]     10 -0.3053884
```

Notice that R kept the vector names — `gene_a` and `gene_b` — as labels. Row and column names are worth their weight in gold once they carry real meaning, like gene identifiers or sample names. You can read them back at any time:

```
rownames(mat)
```

```
NULL
```

```
colnames(mat)
```

```
[1] "gene_a" "gene_b"
```

And you can set them by assigning a vector of valid names:

```
colnames(mat) <- c("control", "treated")
colnames(mat)
```

```
[1] "control" "treated"
```

```
mat
```

```
      control    treated
[1,]      1 -0.6264538
[2,]      2  0.1836433
[3,]      3 -0.8356286
[4,]      4  1.5952808
```

8. Matrices

```
[5,]      5  0.3295078
[6,]      6 -0.8204684
[7,]      7  0.4874291
[8,]      8  0.7383247
[9,]      9  0.5757814
[10,]     10 -0.3053884
```

Every matrix also knows its own shape. `dim()` reports rows then columns; `nrow()` and `ncol()` pull out one each.

```
dim(mat)
```

```
[1] 10  2
```

```
nrow(mat)
```

```
[1] 10
```

```
ncol(mat)
```

```
[1] 2
```

8.3. Accessing elements of a matrix

Indexing a matrix works like indexing a vector, except now you give **two** coordinates: the row first, then the column. The pattern is `mat[r, c]`, where `r` and `c` say which rows and columns you want.

! R counts from one

In R, the first element, row, or column is number **1** — not 0. If you’ve written Python, C, or Java, this will trip you up, because those languages count from 0. Reaching for the wrong starting index is one of the most common beginner mistakes, and the worst kind: it often doesn’t error, it just quietly hands you the wrong value.

Let’s pull various pieces out of `mat`:

8. Matrices

```
# the value in row 1, column 2
mat[1, 2]
```

```
      treated
-0.6264538
```

```
# the entire first ROW (leave the column slot empty)
mat[1, ]
```

```
      control      treated
1.0000000 -0.6264538
```

```
# the entire first COLUMN (leave the row slot empty)
mat[, 1]
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Leaving a slot blank means “give me all of them.” So `mat[1, ]` is the whole first row and `mat[, 1]` is the whole first column. There’s one more form worth knowing — a single index with **no comma**:

```
# every value greater than 4; note: no comma
mat[mat > 4]
```

```
[1] 5 6 7 8 9 10
```

 A matrix is a vector wearing a costume

When you index with a single subscript and no comma, R forgets the rectangle and treats the matrix as one long vector, returning a flat result. That’s handy for “give me all the big values,” but it’s also a trap: code that *looks* like it should respect rows and columns may silently flatten your matrix instead. Whenever you mean a row or a column, include the comma.

You can also *exclude* rows or columns by prefixing the index with a minus sign:

8. Matrices

```
mat[, -1]          # drop the first column
```

```
[1] -0.6264538  0.1836433 -0.8356286  1.5952808  0.3295078 -0.8204684
[7]  0.4874291  0.7383247  0.5757814 -0.3053884
```

```
mat[-c(1:5), ]     # drop the first five rows
```

	control	treated
[1,]	6	-0.8204684
[2,]	7	0.4874291
[3,]	8	0.7383247
[4,]	9	0.5757814
[5,]	10	-0.3053884

8.4. Changing values in a matrix

Let's make a fresh matrix of random values to experiment on — pretend it's a small expression matrix with measurements drawn from a normal distribution.

```
expr <- matrix(rnorm(20), nrow = 10)
summary(expr)
```

V1	V2
Min. : -2.21470	Min. : -1.989352
1st Qu.: -0.03775	1st Qu.: -0.397561
Median : 0.49187	Median : 0.009218
Mean : 0.24884	Mean : -0.133673
3rd Qu.: 0.91318	3rd Qu.: 0.569355
Max. : 1.51178	Max. : 0.918977

Arithmetic on a matrix behaves much like arithmetic on a vector. Multiplying by a single number scales every cell at once:

```
# multiply every value by 20
expr2 <- expr * 20
summary(expr2)
```

8. Matrices

V1	V2
Min. : -44.294	Min. : -39.7870
1st Qu.: -0.755	1st Qu.: -7.9512
Median : 9.837	Median : 0.1844
Mean : 4.977	Mean : -2.6735
3rd Qu.: 18.264	3rd Qu.: 11.3871
Max. : 30.236	Max. : 18.3795

By combining subsetting with assignment, you can rewrite just *part* of a matrix. Here we bump up only the first column:

```
# add 100 to the first column of expr2
expr2[, 1] <- expr2[, 1] + 100
summary(expr2)
```

V1	V2
Min. : 55.71	Min. : -39.7870
1st Qu.: 99.25	1st Qu.: -7.9512
Median :109.84	Median : 0.1844
Mean :104.98	Mean : -2.6735
3rd Qu.:118.26	3rd Qu.: 11.3871
Max. :130.24	Max. : 18.3795

A common reshaping move is to **transpose** — flip rows into columns and vice versa with `t()`. You'll need this surprisingly often: a lab instrument might write samples in rows and genes in columns, while Bioconductor expects genes in rows and samples in columns. One call sets it right.

```
t(expr2)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]
[1,]	130.23562	107.79686	87.57519	55.70600	122.49862	99.101328	99.67619
[2,]	18.37955	15.64273	1.49130	-39.78703	12.39651	-1.122575	-3.11591
	[,8]	[,9]	[,10]				
[1,]	118.87672	116.424424	111.878026				
[2,]	-29.41505	-9.563001	8.358831				

8.5. Calculations on matrix rows and columns

For this section we need a matrix to play with. We'll use `rnorm()` again, but ask for values centered at 5 with a standard deviation of 2 — the two extra arguments are the mean and the spread.

```
expr3 <- matrix(rnorm(100, 5, 2), ncol = 10)
```

Because these values come from a normal distribution centered at 5, any single column (or row) should have a mean near 5 and a standard deviation near 2. Let's check the first column, then the first row:

```
mean(expr3[, 1])
```

```
[1] 5.24146
```

```
sd(expr3[, 1])
```

```
[1] 1.617129
```

```
# now a row
mean(expr3[1, ])
```

```
[1] 5.319228
```

```
sd(expr3[1, ])
```

```
[1] 2.047589
```

Computing one summary at a time gets tedious fast. R gives you convenience functions that sweep across **all** the rows or **all** the columns at once. In a gene-expression matrix, `rowMeans()` is the average expression *per gene* and `colMeans()` is the average *per sample* — two of the most common summaries in genomics.

8. Matrices

```
colMeans(expr3)
```

```
[1] 5.241460 5.268273 5.286914 5.902420 4.504528 5.254721 5.224683 5.694960  
[9] 4.760354 4.987658
```

```
rowMeans(expr3)
```

```
[1] 5.319228 5.055767 6.074298 4.394364 4.717785 5.863346 4.492896 5.169545  
[9] 5.225423 5.813319
```

```
rowSums(expr3)
```

```
[1] 53.19228 50.55767 60.74298 43.94364 47.17785 58.63346 44.92896 51.69545  
[9] 52.25423 58.13319
```

```
colSums(expr3)
```

```
[1] 52.41460 52.68273 52.86914 59.02420 45.04528 52.54721 52.24683 56.94960  
[9] 47.60354 49.87658
```

We can save the column means and look at how they're spread:

```
cmeans <- colMeans(expr3)  
summary(cmeans)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
4.505	5.047	5.248	5.213	5.282	5.902

Notice the column means cluster tightly around 5 — exactly the center we asked for when we built the matrix. The averaging smooths out the noise in the individual values.

What about the standard deviation of each column? There's no built-in `colSd()`, but you don't need one. The `apply()` function takes any function that works on a vector — like `sd()` — and *applies* it across either dimension: 1 means rows, 2 means columns.

8. Matrices

```
csds <- apply(expr3, 2, sd)
summary(csds)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
1.192	1.261	1.610	1.672	1.990	2.577

These per-column standard deviations sit close to 2, the spread we built in. Keep `apply()` in your back pocket: any time R lacks a ready-made `rowX()` or `colX()`, `apply()` lets you roll your own.

💡 Remember the second argument

The number in `apply(x, MARGIN, FUN)` is the dimension you keep: 1 for rows (one result per row), 2 for columns (one result per column). A quick way to remember: it matches the order in `[row, column]` — 1 is rows, 2 is columns.

8.6. Exercises

For these exercises we'll use a dataset that ships with R: monthly sunspot counts from 1749 to 1983. It arrives as a `ts` (time series) object, so the code below reshapes it into a matrix with one **row per year** and one **column per month**. Just run it as written and move on to the questions.

```
data(sunspots)
sunspot_mat <- matrix(as.vector(sunspots), ncol = 12, byrow = TRUE)
colnames(sunspot_mat) <- as.character(1:12)
rownames(sunspot_mat) <- as.character(1749:1983)
```

1. **Get to know the matrix.** What does `sunspot_mat` look like? Find its number of rows, its number of columns, and some basic summary statistics.

i Solution

```
nrow(sunspot_mat)
```

```
[1] 235
```

8. Matrices

```
ncol(sunspot_mat)
```

```
[1] 12
```

```
dim(sunspot_mat)
```

```
[1] 235 12
```

```
summary(sunspot_mat)
```

1		2		3		4	
Min.	: 0.00	Min.	: 0.00	Min.	: 0.00	Min.	: 0.00
1st Qu.:	14.60	1st Qu.:	15.65	1st Qu.:	14.80	1st Qu.:	16.55
Median :	40.60	Median :	44.00	Median :	45.60	Median :	41.00
Mean :	49.11	Mean :	51.17	Mean :	50.04	Mean :	51.00
3rd Qu.:	70.00	3rd Qu.:	74.50	3rd Qu.:	72.25	3rd Qu.:	75.30
Max.	:217.40	Max.	:182.30	Max.	:190.70	Max.	:196.00

5		6		7		8	
Min.	: 0.00	Min.	: 0.00	Min.	: 0.00	Min.	: 0.00
1st Qu.:	19.65	1st Qu.:	15.55	1st Qu.:	16.30	1st Qu.:	16.55
Median :	41.30	Median :	40.50	Median :	41.90	Median :	40.70
Mean :	52.48	Mean :	51.55	Mean :	51.47	Mean :	52.07
3rd Qu.:	76.80	3rd Qu.:	80.95	3rd Qu.:	76.85	3rd Qu.:	73.70
Max.	:238.90	Max.	:200.70	Max.	:191.40	Max.	:200.20

9		10		11		12	
Min.	: 0.00	Min.	: 0.00	Min.	: 0.00	Min.	: 0.00
1st Qu.:	14.60	1st Qu.:	16.60	1st Qu.:	14.75	1st Qu.:	17.25
Median :	42.70	Median :	43.80	Median :	40.50	Median :	41.20
Mean :	52.28	Mean :	51.86	Mean :	50.64	Mean :	51.51
3rd Qu.:	77.00	3rd Qu.:	71.50	3rd Qu.:	68.45	3rd Qu.:	74.70
Max.	:235.80	Max.	:253.80	Max.	:210.90	Max.	:239.40

```
head(sunspot_mat)
```

	1	2	3	4	5	6	7	8	9	10	11	12
1749	58.0	62.6	70.0	55.7	85.0	83.5	94.8	66.3	75.9	75.5	158.6	85.2
1750	73.3	75.9	89.2	88.3	90.0	100.0	85.4	103.0	91.2	65.7	63.3	75.4
1751	70.0	43.5	45.3	56.4	60.7	50.7	66.3	59.8	23.5	23.2	28.5	44.0
1752	35.0	50.0	71.0	59.3	59.7	39.6	78.4	29.3	27.1	46.6	37.6	40.0
1753	44.0	32.0	45.7	38.0	36.0	31.7	22.2	39.0	28.0	25.0	20.0	6.7
1754	0.0	3.0	1.7	13.7	20.7	26.7	18.8	12.3	8.2	24.1	13.2	4.2

dim() confirms 235 years by 12 months, and head() shows the first few years

— always eyeball the shape before you start computing.

2. **Practice subsetting** by pulling out: the first 10 years (rows), the month of July (the 7th column), and the single value for July 1979 using the row name to select it.

i Solution

```
sunspot_mat[1:10, ]
```

	1	2	3	4	5	6	7	8	9	10	11	12
1749	58.0	62.6	70.0	55.7	85.0	83.5	94.8	66.3	75.9	75.5	158.6	85.2
1750	73.3	75.9	89.2	88.3	90.0	100.0	85.4	103.0	91.2	65.7	63.3	75.4
1751	70.0	43.5	45.3	56.4	60.7	50.7	66.3	59.8	23.5	23.2	28.5	44.0
1752	35.0	50.0	71.0	59.3	59.7	39.6	78.4	29.3	27.1	46.6	37.6	40.0
1753	44.0	32.0	45.7	38.0	36.0	31.7	22.2	39.0	28.0	25.0	20.0	6.7
1754	0.0	3.0	1.7	13.7	20.7	26.7	18.8	12.3	8.2	24.1	13.2	4.2
1755	10.2	11.2	6.8	6.5	0.0	0.0	8.6	3.2	17.8	23.7	6.8	20.0
1756	12.5	7.1	5.4	9.4	12.5	12.9	3.6	6.4	11.8	14.3	17.0	9.4
1757	14.1	21.2	26.2	30.0	38.1	12.8	25.0	51.3	39.7	32.5	64.7	33.5
1758	37.6	52.0	49.0	72.3	46.4	45.0	44.0	38.7	62.5	37.7	43.0	43.0

```
sunspot_mat[, 7]
```

1749	1750	1751	1752	1753	1754	1755	1756	1757	1758	1759	1760	1761
94.8	85.4	66.3	78.4	22.2	18.8	8.6	3.6	25.0	44.0	51.0	66.0	94.1
1762	1763	1764	1765	1766	1767	1768	1769	1770	1771	1772	1773	1774
33.8	54.2	30.0	27.0	3.3	21.9	52.6	118.6	109.8	67.7	77.3	27.7	17.5
1775	1776	1777	1778	1779	1780	1781	1782	1783	1784	1785	1786	1787
1.0	1.0	95.0	153.0	143.0	86.0	73.5	37.0	32.2	6.0	36.3	83.0	128.0
1788	1789	1790	1791	1792	1793	1794	1795	1796	1797	1798	1799	1800
141.5	117.0	69.3	71.0	45.8	50.0	41.0	12.9	26.9	4.3	0.0	2.1	21.0
1801	1802	1803	1804	1805	1806	1807	1808	1809	1810	1811	1812	1813
35.0	48.0	48.3	29.8	37.6	30.0	12.7	6.7	0.3	0.0	6.6	0.5	18.3
1814	1815	1816	1817	1818	1819	1820	1821	1822	1823	1824	1825	1826
18.5	35.5	38.8	50.0	28.0	31.4	20.6	2.5	7.9	0.5	0.0	30.9	52.5
1827	1828	1829	1830	1831	1832	1833	1834	1835	1836	1837	1838	1839
42.9	54.3	90.8	43.9	45.2	13.9	7.0	8.7	59.8	116.7	162.8	108.2	84.7
1840	1841	1842	1843	1844	1845	1846	1847	1848	1849	1850	1851	1852
60.7	30.8	12.6	9.5	21.2	30.6	46.5	52.2	139.2	78.0	39.1	36.1	42.0
1853	1854	1855	1856	1857	1858	1859	1860	1861	1862	1863	1864	1865
45.9	18.7	0.4	4.6	22.2	56.7	95.2	116.7	78.0	73.4	32.7	54.7	26.8

8. Matrices

1866	1867	1868	1869	1870	1871	1872	1873	1874	1875	1876	1877	1878
9.3	5.0	28.6	59.2	132.4	103.0	105.5	66.9	67.8	12.5	15.2	5.9	0.1
1879	1880	1881	1882	1883	1884	1885	1886	1887	1888	1889	1890	1891
7.5	21.9	76.9	45.4	80.6	53.1	66.5	30.3	23.3	3.1	9.7	11.6	58.8
1892	1893	1894	1895	1896	1897	1898	1899	1900	1901	1902	1903	1904
76.8	88.8	106.0	47.8	45.0	27.6	9.0	13.5	8.3	0.7	0.9	27.9	50.6
1905	1906	1907	1908	1909	1910	1911	1912	1913	1914	1915	1916	1917
73.0	103.6	49.7	39.5	35.8	14.1	3.5	3.0	1.7	5.4	71.6	53.5	119.8
1918	1919	1920	1921	1922	1923	1924	1925	1926	1927	1928	1929	1930
107.6	64.7	27.5	41.9	10.9	3.5	28.1	38.5	52.3	54.9	98.0	70.2	21.9
1931	1932	1933	1934	1935	1936	1937	1938	1939	1940	1941	1942	1943
17.4	9.6	2.8	9.3	33.9	52.3	145.1	165.3	97.6	67.5	66.9	17.7	13.2
1944	1945	1946	1947	1948	1949	1950	1951	1952	1953	1954	1955	1956
5.0	42.6	116.2	157.9	142.2	125.8	91.0	61.5	39.3	8.6	4.8	26.7	129.1
1957	1958	1959	1960	1961	1962	1963	1964	1965	1966	1967	1968	1969
187.2	191.4	149.6	121.7	70.2	21.8	19.6	3.1	11.9	56.7	91.5	96.1	96.8
1970	1971	1972	1973	1974	1975	1976	1977	1978	1979	1980	1981	1982
112.5	81.0	76.5	23.1	55.8	28.2	1.9	21.4	70.4	159.4	136.3	143.8	106.1
1983												
82.2												

```
sunspot_mat["1979", 7]
```

```
[1] 159.4
```

Because we set row names, you can index by the label "1979" instead of counting rows — far less error-prone than figuring out which row number that is.

3. **Whole-matrix summaries.** A function that expects a vector — like `max()` — will happily treat the whole matrix as one long vector. What is the highest monthly sunspot count in the data?

i Solution

```
max(sunspot_mat)
```

```
[1] 253.8
```

No comma, no row or column — `max()` flattens the matrix and scans every value at once.

4. **And the minimum?**

i Solution

```
min(sunspot_mat)
```

```
[1] 0
```

Same idea as `max()`, in the other direction.

5. **Center of the data.** What are the overall mean and median?

i Solution

```
mean(sunspot_mat)
```

```
[1] 51.26596
```

```
median(sunspot_mat)
```

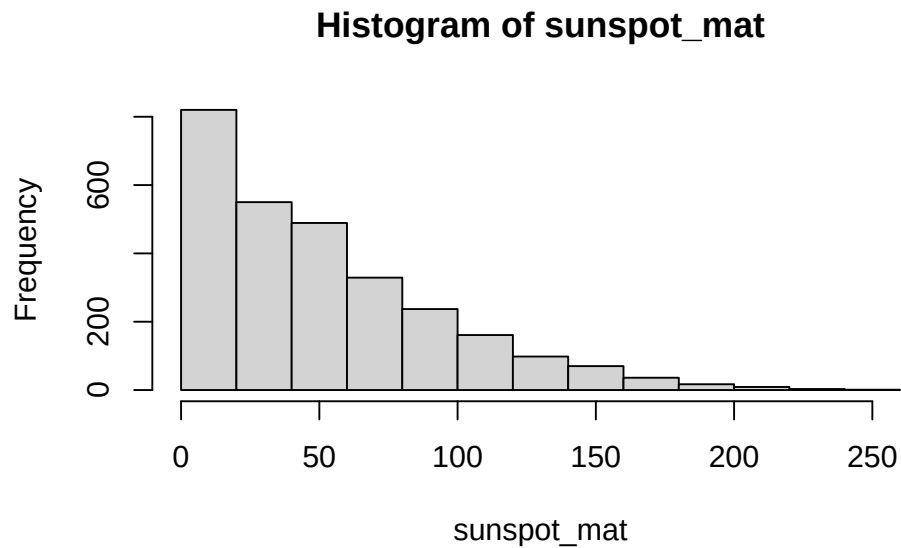
```
[1] 42
```

The mean sits well above the median, a hint that the distribution is skewed — a few very active months pull the average up.

6. **See the shape of the data.** Use `hist()` to look at the distribution of all the monthly counts.

i Solution

```
hist(sunspot_mat)
```



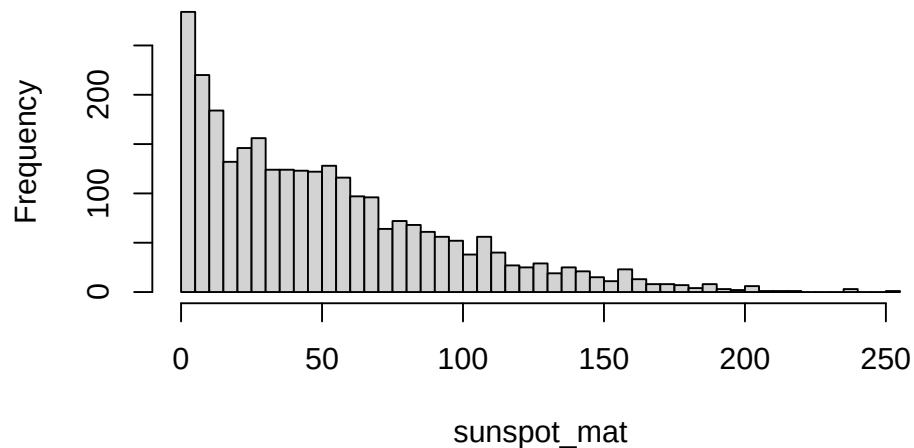
The long right tail confirms the skew we suspected from the mean and median: most months are quiet, a handful are very busy.

7. **Sharpen the histogram.** Read about the `breaks` argument to `hist()` and use it to increase the number of bars for more resolution.

i Solution

```
hist(sunspot_mat, breaks = 40)
```

Histogram of sunspot_mat



More breaks means narrower bars, which reveals finer structure in the distribution.

8. **Average by month.** We'd like the mean number of sunspots for each month of the year. Which summary function gives a result *per column*?

i Solution

```
colMeans(sunspot_mat)
```

```
      1      2      3      4      5      6      7      8
49.11191 51.17489 50.04085 51.00298 52.47830 51.54936 51.46553 52.06511
      9     10     11     12
52.28170 51.86468 50.64170 51.51447
```

```
# apply() does the same thing:
```

```
apply(sunspot_mat, 2, mean)
```

```
      1      2      3      4      5      6      7      8
49.11191 51.17489 50.04085 51.00298 52.47830 51.54936 51.46553 52.06511
      9     10     11     12
52.28170 51.86468 50.64170 51.51447
```

Months are the columns, so `colMeans()` (or `apply(..., 2, mean)`) gives one

8. Matrices

average per month.

9. **Summarize the monthly means.** Save the per-month averages to a variable and summarize it to see the spread across months.

i Solution

```
month_means <- colMeans(sunspot_mat)
summary(month_means)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
49.11	50.91	51.49	51.27	51.91	52.48

The monthly averages are quite close to each other — sunspot activity doesn't depend much on the time of year.

10. **Average by year.** Now play the same game across rows to get one mean per year.

i Solution

```
year_means <- rowMeans(sunspot_mat)
summary(year_means)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.00	16.15	43.05	51.27	70.41	189.85

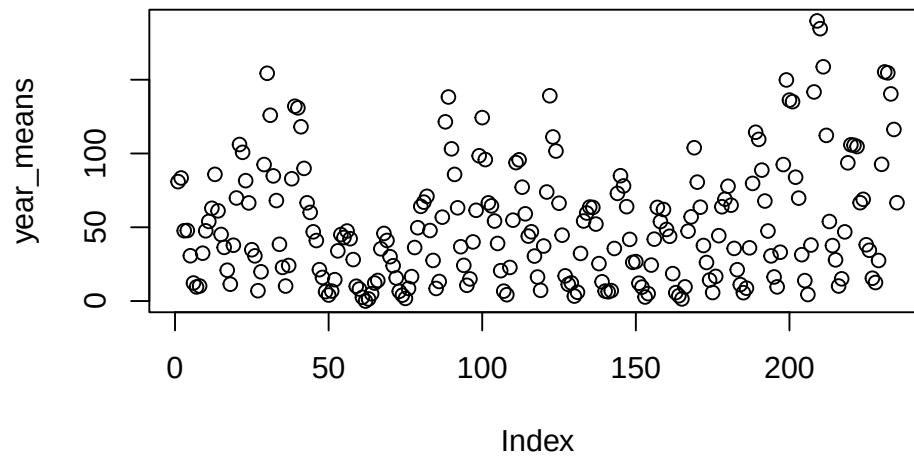
Years are the rows, so `rowMeans()` collapses each year to a single average.

11. **Look for a pattern.** Plot the yearly means. Do you see anything?

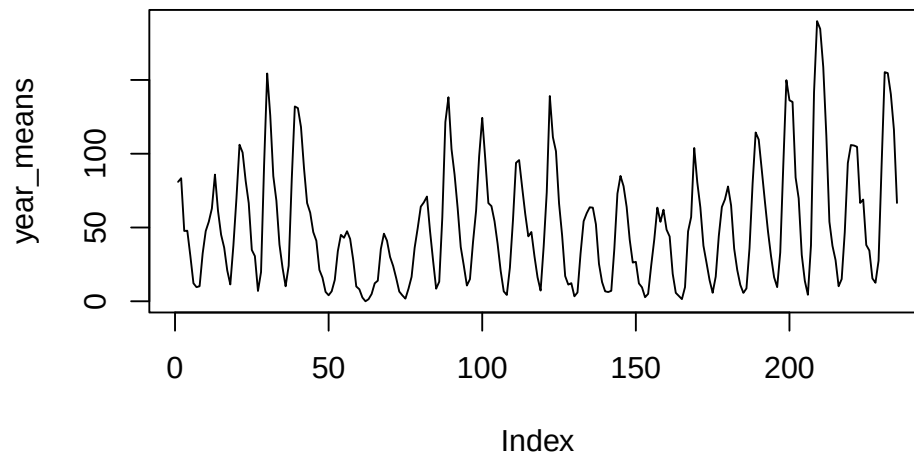
i Solution

```
plot(year_means)
```

8. Matrices



```
# a line makes the cycle clearer  
plot(year_means, type = "l")
```



The famous ~11-year sunspot cycle jumps out as a regular series of peaks and

troughs — a pattern that’s far easier to see across years (rows) than across months (columns).

8.7. Summary

You can now treat a matrix as the workhorse rectangle it is:

- **Build** one with `matrix()`, or assemble it from vectors with `rbind()` and `cbind()`.
- **Inspect** its shape and labels with `dim()`, `nrow()`, `ncol()`, `rownames()`, and `colnames()`.
- **Index** with `mat[row, col]` — remembering R counts from 1, that a blank slot means “all,” and that a single index with no comma flattens the matrix to a vector (Section 8.3).
- **Edit** part of a matrix by combining subsetting with assignment, and flip its orientation with `t()`.
- **Summarize** across rows and columns with `rowMeans()`, `colMeans()`, and the more general `apply()` — the per-gene and per-sample summaries at the heart of genomics.

That last idea — genes in rows, samples in columns, summaries running in either direction — is exactly the mental model you’ll carry into the `SummarizedExperiment` chapter, where a matrix like this becomes the beating heart of a real genomics container.

9. Lists: a catch-all container

So far in our journey through R's data structures, we've dealt with vectors and matrices. These are fantastic tools, but they have one strict rule: all their elements must be of the *same data type*. You can have a vector of numbers or a matrix of characters, but you can't mix and match.

But what about real-world biological data? A single experiment can generate a dizzying variety of information. Imagine you're studying a particular gene. You might have:

- The gene's name (text).
- Its expression level across several samples (a set of numbers).
- A record of whether it's a known cancer-related gene (a simple TRUE/FALSE).
- The raw fluorescence values from your qPCR machine (a matrix of numbers).
- Some personal notes about the experiment (a paragraph of text).

How could you possibly store all of this related, yet different, information together? You could create many separate variables, but that would be clunky and hard to manage. This is exactly the problem that **lists** are designed to solve.

A list in R is like a flexible, multi-compartment container. It's a single object that can hold a collection of *other* R objects, and those objects can be of any type, length, or dimension. You can put vectors, matrices, logical values, and even other lists inside a single list. This makes them one of the most fundamental and powerful data structures for bioinformatics analysis. In fact, almost every rich result R hands you back — a fitted model, the output of a statistical test, a whole data frame — is a list under the hood. Learn lists and you've learned how to take those results apart.

The key features of lists are:

- **Flexibility:** They can contain a mix of any data type.
- **Organization:** You can and should *name* the elements of a list, making your data self-describing.
- **Hierarchy:** Because lists can contain other lists, you can create complex, nested data structures to represent sophisticated relationships in your data.

9.1. What you'll learn

- Create a list with `list()` and give its elements descriptive names.
- Inspect a list's contents with `str()`, `length()`, `names()`, and `class()`.
- Pull a single element *out* of a list with `[[ ]]` or `$`.
- Take a smaller *sub-list* out with single `[ ]`, and explain why that is different.
- Add, update, and remove list elements.
- Use a list to build a self-contained record for a gene.

9.2. Creating a list

You create a list with the `list()` function. The best practice is to name the elements as you create them. This makes your code infinitely more readable and your data easier to work with.

Let's create a list to store the information for our hypothetical gene study.

```
# An experiment tracking list for the gene TP53
experiment_data <- list(
  experiment_id = "EXP042",
  gene_name = "TP53",
  read_counts = c(120, 155, 98, 210),
  is_control = FALSE,
  sample_matrix = matrix(1:4, nrow = 2, dimnames = list(c("Treated", "Untreated"), c("Repli
)

print(experiment_data)
```

```
$experiment_id
[1] "EXP042"
```

```
$gene_name
[1] "TP53"
```

```
$read_counts
[1] 120 155 98 210
```

```
$is_control
[1] FALSE
```

9. Lists: a catch-all container

```
$sample_matrix
      Replicate1 Replicate2
Treated          1          3
Untreated        2          4
```

Notice how the printout walks through the list one named compartment at a time: `$experiment_id` holds a single string, `$read_counts` holds a four-number vector, and `$sample_matrix` holds an entire 2×2 matrix — all living happily inside one object.

i What does `print()` do?

The `print()` function displays the contents of an R object in the console. For a list, it shows each element and its contents. It's the default action when you just type the variable's name and hit Enter, so `print(experiment_data)` and simply typing `experiment_data` do the same thing.

9.3. Inspecting your list: what's inside?

When someone hands you a tube in the lab, the first thing you do is look at the label. When R gives you a complex object like a list, you need to do the same. R provides several “introspection” functions to help you understand the contents and structure of your lists.

9.3.1. `str()`: the structure function

This is arguably the most useful function for inspecting any R object, especially lists.

```
str(experiment_data)
```

```
List of 5
 $ experiment_id: chr "EXP042"
 $ gene_name    : chr "TP53"
 $ read_counts  : num [1:4] 120 155 98 210
 $ is_control   : logi FALSE
 $ sample_matrix: int [1:2, 1:2] 1 2 3 4
 .. attr(*, "dimnames")=List of 2
 .. ..$ : chr [1:2] "Treated" "Untreated"
 .. ..$ : chr [1:2] "Replicate1" "Replicate2"
```

9. Lists: a catch-all container

The output of `str()` tells us everything we need to know: it's a "List of 5", and for each of the 5 elements, it shows the name (e.g., `experiment_id`), the data type (e.g., `chr` for character, `num` for numeric), and a preview of the content.

💡 `str()` is your best friend for any new object

`str()` provides a compact, human-readable summary of an object's internal **structure**. Whenever a function hands you back something you don't recognize — a list, a model, a complicated result — run `str()` on it first. It's the fastest way to see what's inside without printing pages of output.

9.3.2. `length()`, `names()`, and `class()`

These functions give you more specific information about the list itself.

```
# How many top-level elements does the list contain?  
length(experiment_data)
```

```
[1] 5
```

```
# What are the elements called?  
names(experiment_data)
```

```
[1] "experiment_id" "gene_name"      "read_counts"    "is_control"  
[5] "sample_matrix"
```

```
# What kind of object is this?  
class(experiment_data)
```

```
[1] "list"
```

Read together, these tell a story: the list has 5 elements (`length()`), they are named `experiment_id`, `gene_name`, and so on (`names()`), and the object as a whole is a `list` (`class()`). The `names()` output is especially handy — it's a menu of everything you're allowed to ask for.

9.4. Accessing list elements: getting things out

Okay, you've packed your experimental data into a list. Now, how do you get specific items out? This is a critical concept, and R has a few ways to do it, each with a distinct purpose.

9.4.1. The mighty `[[ ]]` and `$` for single items

To pull out a *single element* from a list in its original form, you use either double square brackets `[[ ]]` or the dollar sign `$` (for named lists). Think of this as carefully reaching into a specific compartment of your container and taking out the item itself.

Let's use our `experiment_data` list.

```
# Get the gene name using [[ ]]
gene <- experiment_data[["gene_name"]]
print(gene)
```

```
[1] "TP53"
```

```
class(gene) # It's a character vector, just as it was when we put it in.
```

```
[1] "character"
```

```
# Get the read counts using the $ shortcut. This is often easier to read.
reads <- experiment_data$read_counts
print(reads)
```

```
[1] 120 155 98 210
```

```
class(reads) # It's a numeric vector.
```

```
[1] "numeric"
```

```
# The [[ ]] has a neat trick: you can use a variable to specify the name.
element_to_get <- "read_counts"
experiment_data[[element_to_get]]
```

9. Lists: a catch-all container

```
[1] 120 155 98 210
```

The key takeaway is that `[[ ]]` and `$` **extract the element**. The result is the object that was stored inside the list — a plain character vector, a plain numeric vector — ready to be used in calculations. Notice that `$` needs the name typed out literally, while `[[ ]]` can take a name stored in a variable, which is invaluable when you write loops over a list's elements.

9.4.2. The subsetting `[ ]` for new lists

The single square bracket `[ ]` behaves differently. It always returns a *new, smaller list* that is a subset of the original list. It's like taking a whole compartment, label and all, out of your larger container — the item stays wrapped inside its list.

```
# Get the gene name using [ ]
gene_sublist <- experiment_data["gene_name"]

print(gene_sublist)
```

```
$gene_name
[1] "TP53"
```

```
# Note the class!
class(gene_sublist)
```

```
[1] "list"
```

See the difference? `experiment_data[["gene_name"]]` gave us a `"character"` vector, but `experiment_data["gene_name"]` gives us back a `"list"` that still has `gene_name` inside it.

! `[[ ]]` extracts; `[ ]` subsets. This trips up everyone.

This single-versus-double-bracket distinction is the most common source of confusion for R beginners — so it's worth burning in:

- `[[ ]]` and `$` **reach in and pull the item out**. You get the vector, matrix, or value itself.

9. Lists: a catch-all container

- `[ ]` returns a smaller *list*. The item is still wrapped inside a list.

This matters the moment you try to compute. To take the `mean()` of the read counts you must *extract* them:

```
mean(experiment_data[["read_counts"]]) # works: operating on a numeric vector
```

```
[1] 145.75
```

But `mean(experiment_data["read_counts"])` would fail, because you can't take the mean of a *list*. When in doubt, ask yourself: do I want the thing itself (`[ ]`), or a smaller list (`[ ]`)?

9.5. Modifying lists

Your data is rarely static. You can easily add, remove, or update elements in a list after you've created it.

9.5.1. Adding and updating elements

You can add a new element or change an existing one by using the `$` or `[ ]` assignment syntax. If the name already exists, R updates it; if it's new, R adds it.

```
# Add the date of the experiment
experiment_data$date <- "2024-06-05"

# Add some notes using the [ ] syntax
experiment_data[["notes"]] <- "Initial pilot experiment. High variance in read counts."

# Let's update the control status
experiment_data$is_control <- TRUE

# Let's look at the structure now
str(experiment_data)
```

List of 7

```
$ experiment_id: chr "EXP042"
```

9. Lists: a catch-all container

```
$ gene_name      : chr "TP53"
$ read_counts    : num [1:4] 120 155 98 210
$ is_control     : logi TRUE
$ sample_matrix: int [1:2, 1:2] 1 2 3 4
..- attr(*, "dimnames")=List of 2
.. ..$ : chr [1:2] "Treated" "Untreated"
.. ..$ : chr [1:2] "Replicate1" "Replicate2"
$ date           : chr "2024-06-05"
$ notes          : chr "Initial pilot experiment. High variance in read counts."
```

The list has grown from 5 elements to 7 — `date` and `notes` are new — and `is_control` has flipped from `FALSE` to `TRUE`. Same syntax, two different jobs: a name that already exists gets overwritten, a name that doesn't gets created.

9.5.2. Removing elements

To remove an element from a list, you simply assign `NULL` to it. `NULL` is R's special object representing nothingness.

```
# We've decided the matrix isn't needed for this summary object.
experiment_data$sample_matrix <- NULL

# See the final structure of our list
str(experiment_data)
```

List of 6

```
$ experiment_id: chr "EXP042"
$ gene_name     : chr "TP53"
$ read_counts   : num [1:4] 120 155 98 210
$ is_control    : logi TRUE
$ date          : chr "2024-06-05"
$ notes         : chr "Initial pilot experiment. High variance in read counts."
```

The `sample_matrix` element is gone, and the list is back down to 6 elements. Assigning `NULL` is R's idiom for “delete this compartment entirely.”

9.6. A biological example: a self-contained gene record

Let's put this all together. Lists are perfect for creating self-contained records that you can easily pass to functions or combine into larger lists.

```
brca1_gene <- list(  
  gene_symbol = "BRCA1",  
  full_name = "BRCA1 DNA repair associated",  
  chromosome = "17",  
  expression_log2 = log2(c(45, 50, 30, 88, 120)),  
  related_diseases = c("Breast Cancer", "Ovarian Cancer")  
)  
  
# Now we can easily work with this structured information  
cat("Analyzing gene:", brca1_gene$gene_symbol, "\n")
```

Analyzing gene: BRCA1

```
cat("Located on chromosome:", brca1_gene$chromosome, "\n")
```

Located on chromosome: 17

```
# Calculate the average log2 expression  
avg_expression <- mean(brca1_gene$expression_log2)  
cat("Average log2 expression:", avg_expression, "\n")
```

Average log2 expression: 5.881784

Each `cat()` line prints a label followed by a value pulled straight out of the list with `$`, and the average log2 expression comes out around 5.9.

i A couple of helper functions used above

- **cat()** concatenates and prints its arguments to the console. Unlike **print()**, it lets you seamlessly join text and variables on one line, and the `"\n"` character adds a newline (a line break).
- **log2()** calculates the base-2 logarithm. It's very common in gene-expression analysis to transform skewed count data with **log2()** to make it more symmetric

and easier to model.

This simple `brca1_gene` list is now a complete, portable record. You could imagine creating a list of these gene records, building a powerful, hierarchical database for your entire project.

9.7. Exercises

For these exercises, keep the `brca1_gene` list from above handy. Each one practices a move you'll use constantly when pulling apart analysis results.

1. **Extract by name.** Pull the `related_diseases` element *out* of `brca1_gene` so that you end up with a plain character vector (not a list). Confirm its class.

i Solution

```
diseases <- brca1_gene[["related_diseases"]]
diseases
```

```
[1] "Breast Cancer" "Ovarian Cancer"
```

```
class(diseases)
```

```
[1] "character"
```

Using `[[ ]]` (or `brca1_gene$related_diseases`) extracts the element itself, so `class()` reports "character". Had you used single `[ ]`, you'd have gotten a one-element *list* back instead.

2. **Add an element.** A collaborator tells you BRCA1 sits on the *minus* strand. Add a new element called `strand` with the value `"-"` to `brca1_gene`, then confirm it's there.

i Solution

```
brca1_gene$strand <- "-"
names(brca1_gene)
```

```
[1] "gene_symbol"      "full_name"        "chromosome"       "expression_log2"
```

```
[5] "related_diseases" "strand"
```

Assigning to a name that doesn't yet exist *adds* a new compartment. Checking `names()` is a quick way to confirm the addition landed.

9. Lists: a catch-all container

3. **Nest a vector inside a list.** Add an element called `samples` holding the character vector `c("Tumor", "Normal", "Tumor")`. Then count how many samples are tumors.

i Solution

```
brca1_gene$samples <- c("Tumor", "Normal", "Tumor")
sum(brca1_gene$samples == "Tumor")
```

```
[1] 2
```

A list element can be a whole vector. Once you *extract* it with `$`, it behaves like any ordinary vector — here we compare it to "Tumor" and `sum()` the TRUEs to get 2.

4. **Spot the bracket bug.** A labmate runs `mean(brca1_gene["expression_log2"])` and gets an error. Explain why, then fix it.

i Solution

```
mean(brca1_gene[["expression_log2"]])
```

```
[1] 5.881784
```

Single `[ ]` returns a *list* containing the vector, and `mean()` can't average a list. Switching to `[[ ]]` (or `$`) extracts the numeric vector itself, so `mean()` works.

9.8. Summary

You can now reach for a list whenever you need to keep mixed, named things together in one object — which is exactly what a real analysis result looks like. Specifically, you can:

- **Build** a named list with `list(name = value, ...)`.
- **Inspect** it with `str()` (the structure), `length()`, `names()`, and `class()`.
- **Extract** a single element with `[[ ]]` or `$` (you get the item itself), and take a **sub-list** with single `[ ]` (you get a smaller list).
- **Modify** a list by assigning to `$name` / `[[ "name" ]]` to add or update, and by assigning `NULL` to remove.
- **Assemble** a self-contained gene record and pull values back out for calculation.

The one idea to carry forward: `[[ ]]` reaches in and hands you the thing; `[ ]` hands you a smaller list. Get that straight, and lists — and all the model and test results built on

9. *Lists: a catch-all container*

them — stop being mysterious.

10. Data Frames

While R has many different data types, the one that is central to much of the power and popularity of R is the `data.frame`. A `data.frame` looks a bit like an R matrix in that it has two dimensions, rows and columns. However, `data.frames` are usually viewed as a set of columns representing variables and the rows representing the values of those variables. Importantly, a `data.frame` may contain *different* data types in each of its columns; matrices **must** contain only one data type. This distinction is important to remember, as there are *specific* approaches to working with R `data.frames` that may be different than those for working with matrices.

10.1. Learning goals

- Understand how `data.frames` are different from matrices.
- Know a few functions for examining the contents of a `data.frame`.
- List approaches for subsetting `data.frames`.
- Be able to load and save tabular data from and to disk.
- Show how to create a `data.frame` from scratch.

10.2. Learning objectives

- Load the yeast growth dataset into R using `read.csv`.
- Examine the contents of the dataset.
- Use subsetting to find genes that may be involved with nutrient metabolism and transport.
- Summarize data measurements by categories.

10.3. Dataset

The data used here are borrowed directly from the [fantastic Bioconnector tutorials](#) and are a cleaned up version of the data from [Brauer et al. Coordination of Growth Rate, Cell](#)

10. Data Frames

[Cycle, Stress Response, and Metabolic Activity in Yeast \(2008\) Mol Biol Cell 19:352-367](#). These data are from a gene expression microarray, and in this paper the authors examine the relationship between growth rate and gene expression in yeast cultures limited by one of six different nutrients (glucose, leucine, ammonium, sulfate, phosphate, uracil). If you give yeast a rich media loaded with nutrients except restrict the supply of a single nutrient, you can control the growth rate to any rate you choose. By starving yeast of specific nutrients you can find genes that:

1. Raise or lower their expression in response to growth rate. Growth-rate dependent expression patterns can tell us a lot about cell cycle control, and how the cell responds to stress. The authors found that expression of >25% of all yeast genes is linearly correlated with growth rate, independent of the limiting nutrient. They also found that the subset of negatively growth-correlated genes is enriched for peroxisomal functions, and positively correlated genes mainly encode ribosomal functions.
2. Respond differently when different nutrients are being limited. If you see particular genes that respond very differently when a nutrient is sharply restricted, these genes might be involved in the transport or metabolism of that specific nutrient.

The dataset can be downloaded directly from:

- [brauer2007_tidy.csv](#)

We are going to read this dataset into R and then use it as a playground for learning about data.frames.

10.4. Reading in data

R has many capabilities for reading in data. Many of the functions have names that help us to understand what data format is to be expected. In this case, the filename that we want to read ends in `.csv`, meaning comma-separated-values. The `read.csv()` function reads in `.csv` files. As usual, it is worth reading `help('read.csv')` to get a better sense of the possible bells-and-whistles.

The `read.csv()` function can read directly from a URL, so we do not need to download the file directly. This dataset is relatively large (about 16MB), so this may take a bit depending on your network connection speed.

```
options(width=60)
```

```
url = paste0(
  'https://raw.githubusercontent.com',
  '/biocompare/workshops/master/data/brauer2007_tidy.csv'
)
ydat <- read.csv(url)
```

Our variable, `ydat`, now “contains” the downloaded and read data. We can check to see what data type `read.csv` gave us:

```
class(ydat)
```

```
[1] "data.frame"
```

10.5. Inspecting data.frames

Our `ydat` variable is a `data.frame`. As I mentioned, the dataset is fairly large, so we will not be able to look at it all at once on the screen. However, R gives us many tools to inspect a `data.frame`.

- Overviews of content
 - `head()` to show first few rows
 - `tail()` to show last few rows
- Size
 - `dim()` for dimensions (rows, columns)
 - `nrow()`
 - `ncol()`
 - `object.size()` for power users interested in the memory used to store an object
- Data and attribute summaries
 - `colnames()` to get the names of the columns
 - `rownames()` to get the “names” of the rows—may not be present
 - `summary()` to get per-column summaries of the data in the `data.frame`.

```
head(ydat)
```

10. Data Frames

```

symbol systematic_name nutrient rate expression
1   SFB2          YNL049C  Glucose 0.05      -0.24
2   <NA>          YNL095C  Glucose 0.05       0.28
3   QRI7          YDL104C  Glucose 0.05      -0.02
4   CFT2          YLR115W  Glucose 0.05      -0.33
5   SS02          YMR183C  Glucose 0.05       0.05
6   PSP2          YML017W  Glucose 0.05      -0.69

```

bp

```

1   ER to Golgi transport
2   biological process unknown
3   proteolysis and peptidolysis
4   mRNA polyadenylation*
5   vesicle fusion*
6   biological process unknown

```

mf

```

1   molecular function unknown
2   molecular function unknown
3   metalloendopeptidase activity
4   RNA binding
5   t-SNARE activity
6   molecular function unknown

```

```
tail(ydat)
```

```

symbol systematic_name nutrient rate expression
198425 DOA1          YKL213C  Uracil 0.3       0.14
198426 KRE1          YNL322C  Uracil 0.3       0.28
198427 MTL1          YGR023W  Uracil 0.3       0.27
198428 KRE9          YJL174W  Uracil 0.3       0.43
198429 UTH1          YKR042W  Uracil 0.3       0.19
198430 <NA>          YOL111C  Uracil 0.3       0.04

```

bp

```

198425 ubiquitin-dependent protein catabolism*
198426 cell wall organization and biogenesis
198427 cell wall organization and biogenesis
198428 cell wall organization and biogenesis*
198429 mitochondrion organization and biogenesis*
198430 biological process unknown

```

mf

```

198425 molecular function unknown

```

10. Data Frames

```
198426 structural constituent of cell wall
198427      molecular function unknown
198428      molecular function unknown
198429      molecular function unknown
198430      molecular function unknown
```

```
dim(ydat)
```

```
[1] 198430      7
```

```
nrow(ydat)
```

```
[1] 198430
```

```
ncol(ydat)
```

```
[1] 7
```

```
colnames(ydat)
```

```
[1] "symbol"      "systematic_name" "nutrient"
[4] "rate"        "expression"      "bp"
[7] "mf"
```

```
summary(ydat)
```

symbol	systematic_name	nutrient
Length:198430	Length:198430	Length:198430
Class :character	Class :character	Class :character
Mode :character	Mode :character	Mode :character

rate	expression	bp
Min. :0.0500	Min. :-6.500000	Length:198430
1st Qu.:0.1000	1st Qu.: -0.290000	Class :character

10. Data Frames

```
Median :0.2000   Median : 0.000000   Mode  :character
Mean   :0.1752   Mean    : 0.003367
3rd Qu.:0.2500   3rd Qu.: 0.290000
Max.   :0.3000   Max.    : 6.640000
      mf
Length:198430
Class :character
Mode  :character
```

In RStudio, there is an additional function, `View()` (note the capital “V”) that opens the first 1000 rows (default) in the RStudio window, akin to a spreadsheet view.

```
View(ydat)
```

10.6. Accessing variables (columns) and subsetting

In R, data.frames can be subset similarly to other two-dimensional data structures. The `[` in R is used to denote subsetting of any kind. When working with two-dimensional data, we need two values inside the `[ ]` to specify the details. The specification is `[rows, columns]`. For example, to get the first three rows of `ydat`, use:

```
ydat[1:3, ]
```

```
symbol systematic_name nutrient rate expression
1   SFB2           YNL049C  Glucose 0.05      -0.24
2   <NA>           YNL095C  Glucose 0.05       0.28
3   QRI7           YDL104C  Glucose 0.05     -0.02
      bp
1      ER to Golgi transport
2  biological process unknown
3 proteolysis and peptidolysis
      mf
1  molecular function unknown
2  molecular function unknown
3 metalloendopeptidase activity
```

10. Data Frames

Note how the second number, the columns, is blank. R takes that to mean “all the columns”. Similarly, we can combine rows and columns specification arbitrarily.

```
ydat[1:3, 1:3]
```

```
  symbol systematic_name nutrient
1  SFB2          YNL049C  Glucose
2  <NA>          YNL095C  Glucose
3  QRI7          YDL104C  Glucose
```

Because selecting a single variable, or column, is such a common operation, there are two shortcuts for doing so *with data.frames*. The first, the `$` operator works like so:

```
# Look at the column names, just to refresh memory
colnames(ydat)
```

```
[1] "symbol"          "systematic_name" "nutrient"
[4] "rate"            "expression"      "bp"
[7] "mf"
```

```
# Note that I am using "head" here to limit the output
head(ydat$symbol)
```

```
[1] "SFB2" NA      "QRI7" "CFT2" "SS02" "PSP2"
```

```
# What is the actual length of "symbol"?
length(ydat$symbol)
```

```
[1] 198430
```

The second is related to the fact that, in R, `data.frames` are also lists. We subset a list by using `[[ ]]` notation. To get the second column of `ydat`, we can use:

```
head(ydat[[2]])
```

```
[1] "YNL049C" "YNL095C" "YDL104C" "YLR115W" "YMR183C"
[6] "YML017W"
```

10. Data Frames

Alternatively, we can use the column name:

```
head(ydat[["systematic_name"]])
```

```
[1] "YNL049C" "YNL095C" "YDL104C" "YLR115W" "YMR183C"  
[6] "YML017W"
```

10.6.1. Some data exploration

There are a couple of columns that include numeric values. Which columns are numeric?

```
class(ydat$symbol)
```

```
[1] "character"
```

```
class(ydat$rate)
```

```
[1] "numeric"
```

```
class(ydat$expression)
```

```
[1] "numeric"
```

Make histograms of: - the expression values - the rate values

What does the `table()` function do? Could you use that to look at the `rate` column given that that column appears to have repeated values?

What `rate` corresponds to the most nutrient-starved condition?

10.6.2. More advanced indexing and subsetting

We can use, for example, logical values (TRUE/FALSE) to subset data.frames.

```
head(ydat[ydat$symbol == 'LEU1', ])
```

	symbol	systematic_name	nutrient	rate	expression	bp
NA	<NA>	<NA>	<NA>	NA	NA	<NA>
NA.1	<NA>	<NA>	<NA>	NA	NA	<NA>
NA.2	<NA>	<NA>	<NA>	NA	NA	<NA>
NA.3	<NA>	<NA>	<NA>	NA	NA	<NA>
NA.4	<NA>	<NA>	<NA>	NA	NA	<NA>
NA.5	<NA>	<NA>	<NA>	NA	NA	<NA>
	mf					
NA	<NA>					
NA.1	<NA>					
NA.2	<NA>					
NA.3	<NA>					
NA.4	<NA>					
NA.5	<NA>					

```
tail(ydat[ydat$symbol == 'LEU1', ])
```

	symbol	systematic_name	nutrient	rate	expression	bp	mf
NA.47244	<NA>	<NA>	<NA>	NA	NA		
NA.47245	<NA>	<NA>	<NA>	NA	NA		
NA.47246	<NA>	<NA>	<NA>	NA	NA		
NA.47247	<NA>	<NA>	<NA>	NA	NA		
NA.47248	<NA>	<NA>	<NA>	NA	NA		
NA.47249	<NA>	<NA>	<NA>	NA	NA		
						bp	mf
NA.47244	<NA>	<NA>	<NA>				
NA.47245	<NA>	<NA>	<NA>				
NA.47246	<NA>	<NA>	<NA>				
NA.47247	<NA>	<NA>	<NA>				
NA.47248	<NA>	<NA>	<NA>				
NA.47249	<NA>	<NA>	<NA>				

What is the problem with this approach? It appears that there are a bunch of NA values. Taking a quick look at the `symbol` column, we see what the problem.

10. Data Frames

```
summary(ydat$symbol)
```

```
Length      Class      Mode
198430 character character
```

Using the `is.na()` function, we can make filter further to get down to values of interest.

```
head(ydat[ydat$symbol == 'LEU1' & !is.na(ydat$expression), ])
```

	symbol	systematic_name	nutrient	rate	expression
1526	LEU1	YGL009C	Glucose	0.05	-1.12
7043	LEU1	YGL009C	Glucose	0.10	-0.77
12555	LEU1	YGL009C	Glucose	0.15	-0.67
18071	LEU1	YGL009C	Glucose	0.20	-0.59
23603	LEU1	YGL009C	Glucose	0.25	-0.20
29136	LEU1	YGL009C	Glucose	0.30	0.03

bp

1526	leucine biosynthesis
7043	leucine biosynthesis
12555	leucine biosynthesis
18071	leucine biosynthesis
23603	leucine biosynthesis
29136	leucine biosynthesis

mf

1526	3-isopropylmalate dehydratase activity
7043	3-isopropylmalate dehydratase activity
12555	3-isopropylmalate dehydratase activity
18071	3-isopropylmalate dehydratase activity
23603	3-isopropylmalate dehydratase activity
29136	3-isopropylmalate dehydratase activity

Sometimes, looking at the data themselves is not that important. Using `dim()` is one possibility to look at the number of rows and columns after subsetting.

```
dim(ydat[ydat$expression > 3, ])
```

```
[1] 714 7
```

10. Data Frames

Find the high expressed genes when leucine-starved. For this task we can also use `subset` which allows us to treat column names as R variables (no `$` needed).

```
subset(ydat, nutrient == 'Leucine' & rate == 0.05 & expression > 3)
```

	symbol	systematic_name	nutrient	rate	expression
133768	QDR2	YIL121W	Leucine	0.05	4.61
133772	LEU1	YGL009C	Leucine	0.05	3.84
133858	BAP3	YDR046C	Leucine	0.05	4.29
135186	<NA>	YPL033C	Leucine	0.05	3.43
135187	<NA>	YLR267W	Leucine	0.05	3.23
135288	HXT3	YDR345C	Leucine	0.05	5.16
135963	TP02	YGR138C	Leucine	0.05	3.75
135965	YR02	YBR054W	Leucine	0.05	4.40
136102	GPG1	YGL121C	Leucine	0.05	3.08
136109	HSP42	YDR171W	Leucine	0.05	3.07
136119	HXT5	YHR096C	Leucine	0.05	4.90
136151	<NA>	YJL144W	Leucine	0.05	3.06
136152	MOH1	YBL049W	Leucine	0.05	3.43
136153	<NA>	YBL048W	Leucine	0.05	3.95
136189	HSP26	YBR072W	Leucine	0.05	4.86
136231	NCA3	YJL116C	Leucine	0.05	4.03
136233	<NA>	YBR116C	Leucine	0.05	3.28
136486	<NA>	YGR043C	Leucine	0.05	3.07
137443	ADH2	YMR303C	Leucine	0.05	4.15
137448	ICL1	YER065C	Leucine	0.05	3.54
137451	SFC1	YJR095W	Leucine	0.05	3.72
137569	MLS1	YNL117W	Leucine	0.05	3.76

	bp
133768	multidrug transport
133772	leucine biosynthesis
133858	amino acid transport
135186	meiosis*
135187	biological process unknown
135288	hexose transport
135963	polyamine transport
135965	biological process unknown
136102	signal transduction
136109	response to stress*
136119	hexose transport

10. Data Frames

```
136151          response to dessication
136152          biological process unknown
136153                      <NA>
136189          response to stress*
136231 mitochondrion organization and biogenesis
136233                      <NA>
136486          biological process unknown
137443          fermentation*
137448          glyoxylate cycle
137451          fumarate transport*
137569          glyoxylate cycle
                      mf
133768          multidrug efflux pump activity
133772 3-isopropylmalate dehydratase activity
133858          amino acid transporter activity
135186          molecular function unknown
135187          molecular function unknown
135288          glucose transporter activity*
135963          spermine transporter activity
135965          molecular function unknown
136102          signal transducer activity
136109          unfolded protein binding
136119          glucose transporter activity*
136151          molecular function unknown
136152          molecular function unknown
136153                      <NA>
136189          unfolded protein binding
136231          molecular function unknown
136233                      <NA>
136486          transaldolase activity
137443          alcohol dehydrogenase activity
137448          isocitrate lyase activity
137451 succinate:fumarate antiporter activity
137569          malate synthase activity
```

10.7. Aggregating data

Aggregating data, or summarizing by category, is a common way to look for trends or differences in measurements between categories. Use `aggregate` to find the mean expression

10. Data Frames

by gene symbol.

```
head(aggregate(ydat$expression, by=list( ydat$symbol), mean))
```

	Group.1	x
1	AAC1	0.52888889
2	AAC3	-0.21628571
3	AAD10	0.43833333
4	AAD14	-0.07166667
5	AAD16	0.24194444
6	AAD4	-0.79166667

```
# or  
head(aggregate(expression ~ symbol, mean, data=ydat))
```

	symbol	expression
1	AAC1	0.52888889
2	AAC3	-0.21628571
3	AAD10	0.43833333
4	AAD14	-0.07166667
5	AAD16	0.24194444
6	AAD4	-0.79166667

10.8. Creating a data.frame from scratch

Sometimes it is useful to combine related data into one object. For example, let's simulate some data.

```
smoker = factor(rep(c("smoker", "non-smoker"), each=50))  
smoker_numeric = as.numeric(smoker)  
x = rnorm(100)  
risk = x + 2*smoker_numeric
```

We have two variables, `risk` and `smoker` that are related. We can make a data.frame out of them:

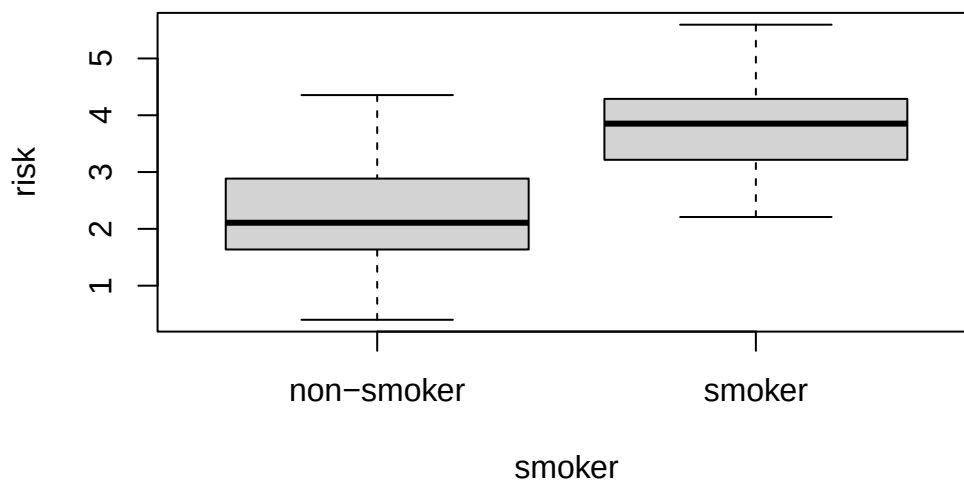
10. Data Frames

```
smoker_risk = data.frame(smoker = smoker, risk = risk)
head(smoker_risk)
```

```
  smoker    risk
1 smoker 3.438347
2 smoker 3.570912
3 smoker 4.511678
4 smoker 4.131091
5 smoker 2.582227
6 smoker 4.046004
```

R also has plotting shortcuts that work with data.frames to simplify plotting

```
plot(risk ~ smoker, data=smoker_risk)
```



10.9. Saving a data.frame

Once we have a data.frame of interest, we may want to save it. The most portable way to save a data.frame is to use one of the `write` functions. In this case, let's save the data as a `.csv` file.

```
write.csv(smoker_risk, "smoker_risk.csv")
```

11. Factors

Open almost any biological dataset and you will meet columns that are not numbers but *labels*: a sample is **treated** or **control**, a mouse is **wild-type**, **heterozygous**, or **homozygous**, a tumor is stage **I**, **II**, **III**, or **IV**. These categorical variables are the backbone of experiments — they say which group each sample belongs to. R has a special tool built exactly for them: the **factor**.

You could store these labels as ordinary text, and R would let you. But the moment you want to count how many samples are in each group, run an analysis of variance, or fit a model that compares treatment to control, R needs to know that “treated” and “control” are *categories* — a fixed set of possible values — not free-form strings. A factor is how you tell it that. This chapter shows you what factors are, how they behave, and the one quirk that trips up nearly every beginner.

11.1. What you’ll learn

- Create a factor from a character vector with `factor()`.
- Explain how a factor stores its data as integer codes plus a lookup table of **levels**.
- Use `table()` to count how many observations fall in each category.
- Control the set of levels, and understand how missing values arise.
- Avoid the classic `as.numeric()` trap that returns codes instead of labels.

11.2. A vector with a built-in dictionary

Imagine you are recording the country of origin for nine samples in a study. The raw data is just text:

```
# country of origin for nine samples
citizen <- c("uk", "us", "no", "au", "uk", "us", "us", "no", "au")
citizen
```

```
[1] "uk" "us" "no" "au" "uk" "us" "us" "no" "au"
```

11. Factors

That is a plain character vector — nine strings, with the quotation marks R always uses to show text. Now turn it into a factor:

```
citizen_f <- factor(citizen)
citizen_f
```

```
[1] uk us no au uk us us no au
Levels: au no uk us
```

Two things changed. The values printed *without* quotation marks, and a new line appeared: `Levels: au no uk us`. That `Levels:` line is the heart of the whole idea.

! The mental model: codes plus a lookup table

A factor *looks* like text but is stored as **integers** behind a **lookup table**. Think of a coat check at a theater: you hand over your coat and get a numbered ticket. The tickets (1, 2, 3, ...) are small and easy to handle; the coats themselves stay on a rack, each hanging at the number on its ticket.

A factor works the same way. The **levels** are the rack — the unique categories, stored once, in alphabetical order by default (`au`, `no`, `uk`, `us`). Each element of your data is just a *ticket number* pointing at one of those levels. So `"uk"` is not stored nine times; it is stored as the integer 3, and R looks up level 3 to show you `uk`.

We can pull back the curtain and see the tickets directly. Every R object can carry hidden **attributes**, and a factor's class and levels live there:

```
attributes(citizen_f)
```

```
$levels
[1] "au" "no" "uk" "us"
```

```
$class
[1] "factor"
```

```
class(citizen_f)
```

```
[1] "factor"
```

11. Factors

The `attributes()` call shows the `levels` (the rack) and confirms the object's `class` is `"factor"`. If we strip that class away with `unclass()`, the disguise drops and the raw ticket numbers appear:

```
unclass(citizen_f)
```

```
[1] 3 4 2 1 3 4 4 2 1
attr(,"levels")
[1] "au" "no" "uk" "us"
```

Read that output against the levels `au no uk us`. The first sample was `"uk"`, which is the **3rd** level, so its code is 3. The third sample was `"no"`, the **2nd** level, so its code is 2. Each number is simply a pointer into the lookup table.

11.3. Counting categories with `table()`

The single most useful thing you will do with a factor is count how many samples fall in each category. That is exactly what `table()` does:

```
table(citizen_f)
```

```
citizen_f
au no uk us
 2  2  2  3
```

At a glance you can see the shape of your sample set: two samples each from Australia and Norway, two from the UK, and three from the US. For a real experiment this is how you check that your treatment and control groups are balanced before you analyze anything.

11.4. Converting a factor back

You will often need to turn a factor back into something else — and here is where people get burned. There are two ways to convert, and they give very different answers.

To recover the original labels, use `as.character()`:

11. Factors

```
as.character(citizen_f)
```

```
[1] "uk" "us" "no" "au" "uk" "us" "us" "no" "au"
```

That gives back the text you started with — "uk", "us", and so on. But watch what happens if you reach for `as.numeric()` instead:

```
as.numeric(citizen_f)
```

```
[1] 3 4 2 1 3 4 4 2 1
```

 The classic gotcha: `as.numeric()` returns the codes, not the labels

`as.numeric()` on a factor returns the **ticket numbers**, not the values. You get 3 4 2 1 3 4 4 2 1 — the integer codes — *not* the countries, and certainly not any number the labels might have looked like.

This bites hardest when a factor's labels are themselves numbers. Suppose a dose column reads "10", "20", "50" but was accidentally stored as a factor. `as.numeric()` would return 1, 2, 3 (the codes), not 10, 20, 50 (the doses) — a silent, dangerous error. The safe recipe when labels are numeric is to go through character first: `as.numeric(as.character(dose_f))`.

11.5. Controlling the levels

By default, the levels are every unique value in your data, sorted alphabetically. But you are allowed to specify the levels yourself — and this is genuinely useful. Maybe you only care about the US and UK samples for one analysis:

```
citizen_f2 <- factor(citizen, levels = c("us", "uk"))
citizen_f2
```

```
[1] uk   us   <NA> <NA> uk   us   us   <NA> <NA>
Levels: us uk
```

11. Factors

Look closely: the values that were not in your chosen levels — the **no** and **au** samples — turned into `<NA>`, R’s marker for a missing value. By naming only **us** and **uk** as legal levels, you told R that every other value has no category to belong to, so it became missing.

```
table(citizen_f2)
```

```
citizen_f2
us uk
3  2
```

By default, `table()` quietly drops those missing values, so the count reflects only the four samples that matched. If you *want* to see the missing ones, add them as an explicit level with `addNA()`:

```
table(addNA(citizen_f2))
```

```
us  uk <NA>
3   2     4
```

Now the `<NA>` category appears with its own count of 4 — the five samples that fell outside your chosen levels.

i Why specifying levels matters

Setting the levels yourself does more than filter. It also fixes their **order**, which controls how categories appear in tables and plots and which group a model treats as the baseline. For a treatment study you might write `factor(group, levels = c("control", "treated"))` so that “control” is the reference everything else is compared against. The default settings are not always what your analysis needs — knowing what they are, and overriding them on purpose, is part of doing the analysis correctly.

11.6. Exercises

1. **Genotypes.** A small genotyping study records these calls for eight mice: `"wt", "het", "hom", "wt", "wt", "het", "hom", "wt"`. Make a factor from this vector and use `table()` to count how many mice carry each genotype.

i Solution

```
genotype <- c("wt", "het", "hom", "wt", "wt", "het", "hom", "wt")
genotype_f <- factor(genotype)
table(genotype_f)
```

```
genotype_f
het hom wt
  2  2  4
```

`table()` counts each category for you: four `wt`, two `het`, and two `hom`. The factor turned a list of labels into a tidy summary of the sample set.

2. **Read the codes.** Using the `genotype_f` factor from Exercise 1, predict what `unclass(genotype_f)` will print *before* you run it. Then run it and check.

i Solution

```
unclass(genotype_f)
```

```
[1] 3 1 2 3 3 1 2 3
attr("levels")
[1] "het" "hom" "wt"
```

The levels are alphabetical — `het`, `hom`, `wt` — so `het` is code 1, `hom` is 2, and `wt` is 3. The first mouse (`wt`) prints as 3. Each integer is a ticket pointing into the levels table, not a measurement.

3. **The numeric trap.** A colleague stored a dose column as a factor with labels "5", "10", "20" and ran `as.numeric()` on it to get the doses back. Why are their numbers wrong, and what should they do instead?

i Solution

```
dose_f <- factor(c("5", "10", "20", "5", "20"))
as.numeric(dose_f) # WRONG: returns the integer codes
```

```
[1] 3 1 2 3 2
```

```
as.numeric(as.character(dose_f)) # RIGHT: recovers the real doses
```

```
[1] 5 10 20 5 20
```

`as.numeric()` returns the level codes, and because levels are sorted, "10" sorts

before "5" — so the codes do not even match the order you expect. Converting to character first turns each label back into its text, which `as.numeric()` can then read as the true number.

4. **Set the reference group.** You have a `group` vector of "treated" and "control" labels. Build a factor whose levels are ordered so that `control` comes first (the baseline a model would compare against).

i Solution

```
group <- c("treated", "control", "treated", "control", "treated")
group_f <- factor(group, levels = c("control", "treated"))
group_f
```

```
[1] treated control treated control treated
Levels: control treated
```

By naming `control` first in `levels`, you make it level 1 — the reference category. Most modeling functions in R compare every other group against the first level, so setting it deliberately keeps your results interpretable.

11.7. Summary

You can now recognize and build the categorical variables at the center of nearly every biological experiment:

- **Create** a factor from a character vector with `factor()`, and recognize it by the `Levels:` line and the missing quotation marks.
- **Picture** it as codes plus a lookup table — integer tickets pointing at a rack of levels — which explains every behavior in this chapter.
- **Count** categories with `table()` to check group sizes and spot imbalance.
- **Control** the levels to filter categories, set a reference group, and decide how missing values are handled.
- **Convert** safely: `as.character()` gives the labels back, while `as.numeric()` gives the codes — so reach for `as.numeric(as.character(x))` whenever a factor's labels are really numbers.

Keep the coat-check picture in mind and factors stop being mysterious: a factor is just a vector of tickets with a table of what each ticket stands for.

Part III.

Exploratory data analysis

Imagine you're on an adventure, about to embark on a journey into the unknown. You've just been handed a treasure map, with the promise of valuable insights waiting to be discovered. This map is your data set, and the journey is exploratory data analysis (EDA).

As you begin your exploration, you start by getting a feel for the terrain. You take a broad, bird's-eye view of the data, examining its structure and dimensions. Are you dealing with a vast landscape or a small, confined area? Are there any missing pieces in the map that you'll need to account for? Understanding the overall context of your data set is crucial before venturing further.

With a sense of the landscape, you now zoom in to identify key landmarks in the data. You might look for unusual patterns, trends, or relationships between variables. As you spot these landmarks, you start asking questions: What's causing that spike in values? Are these two factors related, or is it just a coincidence? By asking these questions, you're actively engaging with the data and forming hypotheses that could guide future analysis or experiments.

As you continue your journey, you realize that the map alone isn't enough to fully understand the terrain. You need more tools to bring the data to life. You start visualizing the data using charts, plots, and graphs. These visualizations act as your binoculars, allowing you to see patterns and relationships more clearly. Through them, you can uncover the hidden treasures buried within the data.

EDA isn't a linear path from start to finish. As you explore, you'll find yourself circling back to previous points, refining your questions, and digging deeper. The process is iterative, with each new discovery informing the next. And as you go, you'll gain a deeper understanding of the data's underlying structure and potential.

Finally, after your thorough exploration, you'll have a solid foundation to build upon. You'll be better equipped to make informed decisions, test hypotheses, and draw meaningful conclusions. The insights you've gained through EDA will serve as a compass, guiding you towards the true value hidden within your data. And with that, you've successfully completed your journey through exploratory data analysis.

12. Introduction to dplyr: mammal sleep dataset

The dataset we will be using to introduce the *dplyr* package is an updated and expanded version of the mammals sleep dataset. Updated sleep times and weights were taken from V. M. Savage and G. B. West. A quantitative, theoretical framework for understanding mammalian sleep¹.

12.1. Learning goals

- Know that `dplyr` is just a different approach to manipulating data in `data.frames`.
- List the commonly used `dplyr` verbs and how they can be used to manipulate `data.frames`.
- Show how to aggregate and summarized data using `dplyr`
- Know what the piping operator, `|>`, is and how it can be used.

12.2. Learning objectives

- Select subsets of the mammal sleep dataset.
- Reorder the dataset.
- Add columns to the dataset based on existing columns.
- Summarize the amount of sleep by categorical variables using `group_by` and `summarize`.

¹A quantitative, theoretical framework for understanding mammalian sleep. Van M. Savage, Geoffrey B. West. Proceedings of the National Academy of Sciences Jan 2007, 104 (3) 1051-1056; DOI: [10.1073/pnas.0610080104](https://doi.org/10.1073/pnas.0610080104)

12.3. What is dplyr?

The *dplyr* package is a specialized package for working with `data.frames` (and the related `tibble`) to transform and summarize tabular data with rows and columns. For another explanation of dplyr see the dplyr package vignette: [Introduction to dplyr](#)

12.4. Why Is dplyr useful?

dplyr contains a set of functions—commonly called the dplyr “verbs”—that perform common data manipulations such as filtering for rows, selecting specific columns, re-ordering rows, adding new columns and summarizing data. In addition, dplyr contains a useful function to perform another common task which is the “split-apply-combine” concept.

Compared to base functions in R, the functions in dplyr are often easier to work with, are more consistent in the syntax and are targeted for data analysis around data frames, instead of just vectors.

12.5. Data: Mammals Sleep

The `msleep` (mammals sleep) data set contains the sleep times and weights for a set of mammals and is available in the `dagdata` repository on github. This data set contains 83 rows and 11 variables. The data happen to be available as a `dataset` in the *ggplot2* package. To get access to the `msleep` dataset, we need to first install the `ggplot2` package.

```
install.packages('ggplot2')
```

Then, we can load the library.

```
library(ggplot2)
data(msleep)
```

As with many datasets in R, “help” is available to describe the dataset itself.

```
?msleep
```

12. Introduction to dplyr: mammal sleep dataset

The columns are described in the help page, but are included here, also.

column name	Description
name	common name
genus	taxonomic rank
vore	carnivore, omnivore or herbivore?
order	taxonomic rank
conservation	the conservation status of the mammal
sleep_total	total amount of sleep, in hours
sleep_rem	rem sleep, in hours
sleep_cycle	length of sleep cycle, in hours
awake	amount of time spent awake, in hours
brainwt	brain weight in kilograms
bodywt	body weight in kilograms

12.6. dplyr verbs

The dplyr verbs are listed here. There are many other functions available in dplyr, but we will focus on just these.

dplyr verbs	Description
<code>select()</code>	select columns
<code>filter()</code>	filter rows
<code>arrange()</code>	re-order or arrange rows
<code>mutate()</code>	create new columns
<code>summarise()</code>	summarise values
<code>group_by()</code>	allows for group operations in the “split-apply-combine” concept

12.7. Using the dplyr verbs

The two most basic functions are `select()` and `filter()`, which selects columns and filters rows respectively. What are the equivalent ways to select columns without dplyr? And filtering to include only specific rows?

Before proceeding, we need to install the dplyr package:

12. Introduction to dplyr: mammal sleep dataset

```
install.packages('dplyr')
```

And then load the library:

```
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

12.7.1. Selecting columns: select()

Select a set of columns such as the `name` and the `sleep_total` columns.

```
sleepData <- select(msleep, name, sleep_total)
head(sleepData)
```

```
# A tibble: 6 x 2
  name                sleep_total
  <chr>              <dbl>
1 Cheetah             12.1
2 Owl monkey          17
3 Mountain beaver     14.4
4 Greater short-tailed shrew 14.9
5 Cow                  4
6 Three-toed sloth     14.4
```

To select all the columns *except* a specific column, use the “-” (subtraction) operator (also known as negative indexing). For example, to select all columns except `name`:

12. Introduction to dplyr: mammal sleep dataset

```
head(select(msleep, -name))
```

```
# A tibble: 6 x 10
```

	genus	vore	order	conservation	sleep_total	sleep_rem	sleep_cycle	awake
	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	Acinonyx	carni	Carnivo~	lc	12.1	NA	NA	11.9
2	Aotus	omni	Primates	<NA>	17	1.8	NA	7
3	Aplodontia	herbi	Rodentia	nt	14.4	2.4	NA	9.6
4	Blarina	omni	Soricom~	lc	14.9	2.3	0.133	9.1
5	Bos	herbi	Artioda~	domesticated	4	0.7	0.667	20
6	Bradypus	herbi	Pilosa	<NA>	14.4	2.2	0.767	9.6

```
# i 2 more variables: brainwt <dbl>, bodywt <dbl>
```

To select a range of columns by name, use the “:” operator. Note that dplyr allows us to use the column names without quotes and as “indices” of the columns.

```
head(select(msleep, name:order))
```

```
# A tibble: 6 x 4
```

	name	genus	vore	order
	<chr>	<chr>	<chr>	<chr>
1	Cheetah	Acinonyx	carni	Carnivora
2	Owl monkey	Aotus	omni	Primates
3	Mountain beaver	Aplodontia	herbi	Rodentia
4	Greater short-tailed shrew	Blarina	omni	Soricomorpha
5	Cow	Bos	herbi	Artiodactyla
6	Three-toed sloth	Bradypus	herbi	Pilosa

To select all columns that start with the character string “sl”, use the function `starts_with()`.

```
head(select(msleep, starts_with("sl")))
```

```
# A tibble: 6 x 3
```

	sleep_total	sleep_rem	sleep_cycle
	<dbl>	<dbl>	<dbl>
1	12.1	NA	NA
2	17	1.8	NA

12. Introduction to dplyr: mammal sleep dataset

3	14.4	2.4	NA
4	14.9	2.3	0.133
5	4	0.7	0.667
6	14.4	2.2	0.767

Some additional options to select columns based on a specific criteria include:

1. `ends_with()` = Select columns that end with a character string
2. `contains()` = Select columns that contain a character string
3. `matches()` = Select columns that match a regular expression
4. `one_of()` = Select column names that are from a group of names

12.7.2. Selecting rows: `filter()`

The `filter()` function allows us to filter rows to include only those rows that *match* the filter. For example, we can filter the rows for mammals that sleep a total of more than 16 hours.

```
filter(msleep, sleep_total >= 16)
```

```
# A tibble: 8 x 11
  name      genus vore order conservation sleep_total sleep_rem sleep_cycle awake
  <chr>    <chr> <chr> <chr> <chr>          <dbl>      <dbl>      <dbl> <dbl>
1 Owl mo~ Aotus omni Prim~ <NA>         17         1.8        NA      7
2 Long-n~ Dasy~ carni Cing~ lc          17.4        3.1        0.383   6.6
3 North ~ Dide~ omni Dide~ lc          18         4.9        0.333   6
4 Big br~ Epte~ inse~ Chir~ lc          19.7        3.9        0.117   4.3
5 Thick~~ Lutr~ carni Dide~ lc          19.4        6.6        NA      4.6
6 Little~ Myot~ inse~ Chir~ <NA>         19.9        2          0.2    4.1
7 Giant ~ Prio~ inse~ Cing~ en          18.1        6.1        NA      5.9
8 Arctic~ Sper~ herbi Rode~ lc          16.6        NA         NA      7.4
# i 2 more variables: brainwt <dbl>, bodywt <dbl>
```

Filter the rows for mammals that sleep a total of more than 16 hours *and* have a body weight of greater than 1 kilogram.

```
filter(msleep, sleep_total >= 16, bodywt >= 1)
```


12. Introduction to dplyr: mammal sleep dataset

```
# A tibble: 3 x 11
  name      genus vore  order conservation sleep_total sleep_rem sleep_cycle awake
  <chr>    <chr> <chr> <chr> <chr>          <dbl>    <dbl>    <dbl> <dbl>
1 Long-n~ Dasy~ carni Cing~ lc          17.4      3.1      0.383  6.6
2 North ~ Dide~ omni  Dide~ lc          18        4.9      0.333   6
3 Giant ~ Prio~ inse~ Cing~ en          18.1      6.1      NA      5.9
# i 2 more variables: brainwt <dbl>, bodywt <dbl>
```

Filter the rows for mammals in the Perissodactyla and Primates taxonomic order. The `%in%` operator is a logical operator that returns TRUE for values of a vector that are present in a second vector.

```
filter(msleep, order %in% c("Perissodactyla", "Primates"))
```

```
# A tibble: 15 x 11
  name      genus vore  order conservation sleep_total sleep_rem sleep_cycle awake
  <chr>    <chr> <chr> <chr> <chr>          <dbl>    <dbl>    <dbl> <dbl>
1 Owl m~ Aotus omni  Prim~ <NA>          17      1.8      NA      7
2 Grivet Cerc~ omni  Prim~ lc          10      0.7      NA     14
3 Horse Equus herbi Peri~ domesticated  2.9      0.6      1     21.1
4 Donkey Equus herbi Peri~ domesticated  3.1      0.4      NA     20.9
5 Patas~ Eryt~ omni  Prim~ lc          10.9     1.1      NA     13.1
6 Galago Gala~ omni  Prim~ <NA>          9.8     1.1     0.55  14.2
7 Human Homo omni  Prim~ <NA>          8      1.9     1.5    16
8 Mongo~ Lemur herbi Prim~ vu          9.5     0.9      NA     14.5
9 Macaq~ Maca~ omni  Prim~ <NA>          10.1     1.2     0.75  13.9
10 Slow ~ Nyct~ carni Prim~ <NA>          11      NA      NA     13
11 Chimp~ Pan omni  Prim~ <NA>          9.7     1.4     1.42  14.3
12 Baboon Papio omni  Prim~ <NA>          9.4      1     0.667  14.6
13 Potto Pero~ omni  Prim~ lc          11      NA      NA     13
14 Squir~ Saim~ omni  Prim~ <NA>          9.6     1.4      NA     14.4
15 Brazi~ Tapi~ herbi Peri~ vu          4.4      1     0.9    19.6
# i 2 more variables: brainwt <dbl>, bodywt <dbl>
```

You can use the boolean operators (e.g. `>`, `<`, `>=`, `<=`, `!=`, `%in%`) to create the logical tests.

12.8. “Piping” with `|>`

It is not unusual to want to perform a set of operations using dplyr. The pipe operator `|>` allows us to “pipe” the output from one function into the input of the next. While there is nothing special about how R treats operations that are written in a pipe, the idea of piping is to allow us to read multiple functions operating one after another from left-to-right. Without piping, one would either 1) save each step in set of functions as a temporary variable and then pass that variable along the chain or 2) have to “nest” functions, which can be hard to read.

Here’s an example we have already used:

```
head(select(msleep, name, sleep_total))
```

```
# A tibble: 6 x 2
  name                sleep_total
  <chr>                <dbl>
1 Cheetah              12.1
2 Owl monkey           17
3 Mountain beaver     14.4
4 Greater short-tailed shrew 14.9
5 Cow                  4
6 Three-toed sloth    14.4
```

Now in this case, we will pipe the `msleep` data frame to the function that will select two columns (`name` and `sleep_total`) and then pipe the new data frame to the function `head()`, which will return the head of the new data frame.

```
msleep |>
  select(name, sleep_total) |>
  head()
```

```
# A tibble: 6 x 2
  name                sleep_total
  <chr>                <dbl>
1 Cheetah              12.1
2 Owl monkey           17
3 Mountain beaver     14.4
4 Greater short-tailed shrew 14.9
5 Cow                  4
6 Three-toed sloth    14.4
```

12. Introduction to dplyr: mammal sleep dataset

You will soon see how useful the pipe operator is when we start to combine many functions.

Now that you know about the pipe operator (`|>`), we will use it throughout the rest of this tutorial.

12.8.1. Arrange Or Re-order Rows Using `arrange()`

To arrange (or re-order) rows by a particular column, such as the taxonomic order, list the name of the column you want to arrange the rows by:

```
msleep |> arrange(order) |> head()
```

```
# A tibble: 6 x 11
  name      genus vore  order conservation sleep_total sleep_rem sleep_cycle awake
  <chr>    <chr> <chr> <chr> <chr>           <dbl>    <dbl>    <dbl> <dbl>
1 Tenrec  Tenr~ omni  Afro~ <NA>           15.6     2.3     NA     8.4
2 Cow     Bos   herbi Arti~ domesticated    4       0.7     0.667  20
3 Roe de~ Capr~ herbi Arti~ lc             3       NA     NA     21
4 Goat    Capri herbi Arti~ lc             5.3     0.6     NA     18.7
5 Giraffe Gira~ herbi Arti~ cd             1.9     0.4     NA     22.1
6 Sheep   Ovis  herbi Arti~ domesticated    3.8     0.6     NA     20.2
# i 2 more variables: brainwt <dbl>, bodywt <dbl>
```

Now we will select three columns from `msleep`, arrange the rows by the taxonomic order and then arrange the rows by `sleep_total`. Finally, show the head of the final data frame:

```
msleep |>
  select(name, order, sleep_total) |>
  arrange(order, sleep_total) |>
  head()
```

```
# A tibble: 6 x 3
  name      order      sleep_total
  <chr>    <chr>          <dbl>
1 Tenrec  Afrosoricida    15.6
2 Giraffe Artiodactyla  1.9
3 Roe deer Artiodactyla  3
4 Sheep   Artiodactyla    3.8
```

12. Introduction to dplyr: mammal sleep dataset

```
5 Cow      Artiodactyla      4
6 Goat     Artiodactyla     5.3
```

Same as above, except here we filter the rows for mammals that sleep for 16 or more hours, instead of showing the head of the final data frame:

```
msleep |>
  select(name, order, sleep_total) |>
  arrange(order, sleep_total) |>
  filter(sleep_total >= 16)
```

```
# A tibble: 8 x 3
  name          order      sleep_total
  <chr>         <chr>         <dbl>
1 Big brown bat  Chiroptera     19.7
2 Little brown bat Chiroptera     19.9
3 Long-nosed armadillo Cingulata     17.4
4 Giant armadillo  Cingulata     18.1
5 North American Opossum Didelphimorphia 18
6 Thick-tailed opossum Didelphimorphia 19.4
7 Owl monkey     Primates       17
8 Arctic ground squirrel Rodentia       16.6
```

For something slightly more complicated do the same as above, except arrange the rows in the `sleep_total` column in a descending order. For this, use the function `desc()`

```
msleep |>
  select(name, order, sleep_total) |>
  arrange(order, desc(sleep_total)) |>
  filter(sleep_total >= 16)
```

```
# A tibble: 5 x 3
  name          order      sleep_total
  <chr>         <chr>         <dbl>
1 Little brown bat Chiroptera     19.9
2 Big brown bat    Chiroptera     19.7
3 Giant armadillo  Cingulata     18.1
4 Long-nosed armadillo Cingulata     17.4
5 Thick-tailed opossum Didelphimorphia 19.4
```

6	North American Opossum	Didelphimorphia	18
7	Owl monkey	Primates	17
8	Arctic ground squirrel	Rodentia	16.6

12.9. Create New Columns Using mutate()

The `mutate()` function will add new columns to the data frame. Create a new column called `rem_proportion`, which is the ratio of rem sleep to total amount of sleep.

```
msleep |>
  mutate(rem_proportion = sleep_rem / sleep_total) |>
  head()
```

```
# A tibble: 6 x 12
  name      genus vore  order conservation sleep_total sleep_rem sleep_cycle awake
  <chr>    <chr> <chr> <chr> <chr>           <dbl>    <dbl>    <dbl> <dbl>
1 Cheetah Acin~ carni Carn~ lc           12.1      NA      NA      11.9
2 Owl mo~ Aotus omni  Prim~ <NA>         17        1.8    NA       7
3 Mounta~ Aplo~ herbi Rode~ nt          14.4      2.4    NA      9.6
4 Greate~ Blar~ omni  Sori~ lc          14.9      2.3    0.133   9.1
5 Cow     Bos   herbi Arti~ domesticated  4         0.7    0.667   20
6 Three~ Brad~ herbi Pilo~ <NA>         14.4      2.2    0.767   9.6
# i 3 more variables: brainwt <dbl>, bodywt <dbl>, rem_proportion <dbl>
```

You can add many new columns using `mutate` (separated by commas). Here we add a second column called `bodywt_grams` which is the `bodywt` column in grams.

```
msleep |>
  mutate(rem_proportion = sleep_rem / sleep_total,
         bodywt_grams = bodywt * 1000) |>
  head()
```

```
# A tibble: 6 x 13
  name      genus vore  order conservation sleep_total sleep_rem sleep_cycle awake
  <chr>    <chr> <chr> <chr> <chr>           <dbl>    <dbl>    <dbl> <dbl>
1 Cheetah Acin~ carni Carn~ lc           12.1      NA      NA      11.9
2 Owl mo~ Aotus omni  Prim~ <NA>         17        1.8    NA       7
3 Mounta~ Aplo~ herbi Rode~ nt          14.4      2.4    NA      9.6
```

12. Introduction to dplyr: mammal sleep dataset

```
4 Greate~ Blar~ omni Sori~ lc          14.9      2.3      0.133  9.1
5 Cow      Bos   herbi Arti~ domesticated      4      0.7      0.667  20
6 Three-- Brad~ herbi Pilo~ <NA>          14.4      2.2      0.767  9.6
# i 4 more variables: brainwt <dbl>, bodywt <dbl>, rem_proportion <dbl>,
#   bodywt_grams <dbl>
```

Is there a relationship between `rem_proportion` and `bodywt`? How about `sleep_total`?

12.9.1. Create summaries: `summarise()`

The `summarise()` function will create summary statistics for a given column in the data frame such as finding the mean. For example, to compute the average number of hours of sleep, apply the `mean()` function to the column `sleep_total` and call the summary value `avg_sleep`.

```
msleep |>
  summarise(avg_sleep = mean(sleep_total))
```

```
# A tibble: 1 x 1
  avg_sleep
    <dbl>
1      10.4
```

There are many other summary statistics you could consider such `sd()`, `min()`, `max()`, `median()`, `sum()`, `n()` (returns the length of vector), `first()` (returns first value in vector), `last()` (returns last value in vector) and `n_distinct()` (number of distinct values in vector).

```
msleep |>
  summarise(avg_sleep = mean(sleep_total),
            min_sleep = min(sleep_total),
            max_sleep = max(sleep_total),
            total = n())
```

```
# A tibble: 1 x 4
  avg_sleep min_sleep max_sleep total
    <dbl>      <dbl>      <dbl> <int>
1      10.4       1.9      19.9     83
```

12.10. Grouping data: group_by()

The `group_by()` verb is an important function in dplyr. The `group_by` allows us to use the concept of “split-apply-combine”. We literally want to split the data frame by some variable (e.g. taxonomic order), apply a function to the individual data frames and then combine the output. This approach is similar to the `aggregate` function from R, but `group_by` integrates with dplyr.

Let’s do that: split the `msleep` data frame by the taxonomic order, then ask for the same summary statistics as above. We expect a set of summary statistics for each taxonomic order.

```
msleep |>
  group_by(order) |>
  summarise(avg_sleep = mean(sleep_total),
            min_sleep = min(sleep_total),
            max_sleep = max(sleep_total),
            total = n())
```

```
# A tibble: 19 x 5
```

	order	avg_sleep	min_sleep	max_sleep	total
	<chr>	<dbl>	<dbl>	<dbl>	<int>
1	Afrosoricida	15.6	15.6	15.6	1
2	Artiodactyla	4.52	1.9	9.1	6
3	Carnivora	10.1	3.5	15.8	12
4	Cetacea	4.5	2.7	5.6	3
5	Chiroptera	19.8	19.7	19.9	2
6	Cingulata	17.8	17.4	18.1	2
7	Didelphimorphia	18.7	18	19.4	2
8	Diprotodontia	12.4	11.1	13.7	2
9	Erinaceomorpha	10.2	10.1	10.3	2
10	Hyracoidea	5.67	5.3	6.3	3
11	Lagomorpha	8.4	8.4	8.4	1
12	Monotremata	8.6	8.6	8.6	1
13	Perissodactyla	3.47	2.9	4.4	3
14	Pilosa	14.4	14.4	14.4	1
15	Primates	10.5	8	17	12
16	Proboscidea	3.6	3.3	3.9	2
17	Rodentia	12.5	7	16.6	22
18	Scandentia	8.9	8.9	8.9	1
19	Soricomorpha	11.1	8.4	14.9	5

13. Case Study: Behavioral Risk Factor Surveillance System

The Behavioral Risk Factor Surveillance System (BRFSS) is a large health survey run every year by the U.S. Centers for Disease Control and Prevention (CDC). Each year it phones hundreds of thousands of adults and asks about their health behaviors, chronic conditions, and use of preventive care. Public-health researchers lean on it to track risk factors, shape policy, and judge whether prevention programs are working.

In this chapter you take on the role of one of those researchers. You'll work with a small slice of the BRFSS data and walk it through a full **exploratory data analysis (EDA)**: load it, look at it, summarize it, plot it, and ask whether the variables relate to one another. EDA is the first thing you do with any new dataset — before any fancy modeling — because it's how you learn the shape of your data and catch problems early. By the end you'll go one step further and fit a simple statistical model to a question the data can actually answer.

Everything here uses **base R** — no extra packages — so you can follow along on a fresh R install.

13.1. What you'll learn

- Download a dataset and load a CSV into R with `download.file()` and `read.csv()`.
- Inspect a data frame's size and structure with `head()`, `dim()`, and `summary()`.
- Visualize single variables with `hist()` and `boxplot()`, and pairs of variables with `plot()`.
- Measure the strength of a linear relationship with `cor()`, and handle the missing-value trap that makes it return `NA`.
- Compare a measurement between two groups with a `t.test()`, and fit and read a simple linear regression with `lm()`.

13.2. Loading the dataset

First you need the data. You can download it from [this link](https://raw.githubusercontent.com/seandavi/ITR/master/BRFSS-subset.csv) by hand, or — better — let R fetch it for you straight from the command line. The `download.file()` function takes a URL and a local filename to save it under:

```
download.file('https://raw.githubusercontent.com/seandavi/ITR/master/BRFSS-subset.csv',
              destfile = 'BRFSS-subset.csv')
```

That writes a file called `BRFSS-subset.csv` into your working directory. Now you point R at that file and read it in with `read.csv()`, storing the result in a data frame called `brfss`:

```
path <- "BRFSS-subset.csv"
stopifnot(file.exists(path))
brfss <- read.csv(path)
```

The `stopifnot(file.exists(path))` line is a small safety check: if the file isn't where you said it is, R stops immediately with a clear message instead of failing in some confusing way two steps later.

💡 Working interactively? Let R pop up a file picker

If you downloaded the file by hand and aren't sure of its exact path, you can let R open a file-chooser dialog instead of typing the path:

```
path <- file.choose() # opens a dialog; navigate to BRFSS-subset.csv
brfss <- read.csv(path)
```

That's handy when you're poking around interactively, but for a script or a reproducible document you want a fixed path like the one above so it runs the same way every time.

13.3. Inspecting the data

Whenever you load a dataset, your very first move should be to *look at it*. Start with `head()`, which prints the first six rows:

13. Case Study: Behavioral Risk Factor Surveillance System

```
head(brfss)
```

```
   Age   Weight   Sex Height Year
1  31 48.98798 Female 157.48 1990
2  57 81.64663 Female 157.48 1990
3  43 80.28585   Male 177.80 1990
4  72 70.30682   Male 170.18 1990
5  31 49.89516 Female 154.94 1990
6  58 54.43108 Female 154.94 1990
```

That gives you a feel for what each column holds and what type of value it is. Here you can see an *Age*, a *Weight*, a *Height*, a *Sex*, and a *Year*.

Next, check how big the dataset is with `dim()`, which reports the number of rows and columns:

```
dim(brfss)
```

```
[1] 20000    5
```

Knowing the size matters: it tells you whether you have a handful of records or many thousands, which shapes what analyses make sense.

13.4. Summary statistics

Now that you know the structure, get a quick numeric overview of every column at once with `summary()`:

```
summary(brfss)
```

Age		Weight		Sex		Height	
Min.	:18.00	Min.	: 34.93	Length	:20000	Min.	:105.0
1st Qu.:	36.00	1st Qu.:	61.69	N.unique	: 2	1st Qu.:	162.6
Median	:51.00	Median	: 72.57	N.blank	: 0	Median	:168.0
Mean	:50.99	Mean	: 75.42	Min.nchar:	4	Mean	:169.2
3rd Qu.:	65.00	3rd Qu.:	86.18	Max.nchar:	6	3rd Qu.:	177.8
Max.	:99.00	Max.	:278.96			Max.	:218.0

13. Case Study: Behavioral Risk Factor Surveillance System

```
NAs      :139      NAs      :649      NAs      :184
Year
Min.     :1990
1st Qu.  :1990
Median   :2000
Mean     :2000
3rd Qu.  :2010
Max.     :2010
```

For each **numeric** column (**Age**, **Weight**, **Height**) this prints the minimum, first quartile, median, mean, third quartile, and maximum. For text/categorical columns you get a simpler count. Two things to notice right away: the **Weight** and **Height** columns list a number of NA's (missing values), and the spread between the median and the maximum hints at the *shape* of each distribution. Both of those observations will matter shortly.

13.5. Visualizing single variables

Numbers in a summary table only take you so far. A picture of a single variable — its **distribution** — tells you far more about its shape, center, and spread. Start with a histogram of **Age** using `hist()`:

```
hist(brfss$Age, main = "Age Distribution",
      xlab = "Age", col = "lightblue")
```

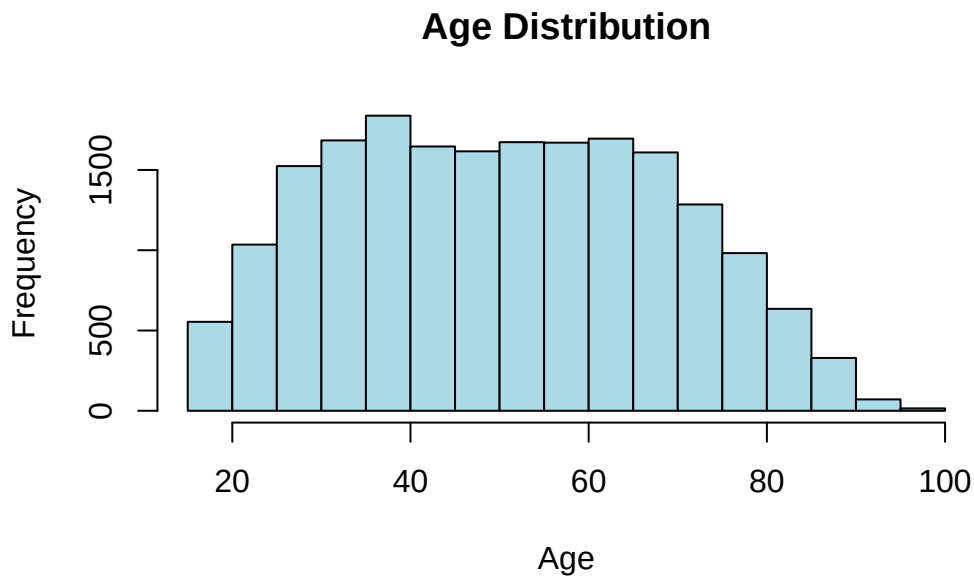


Figure 13.1.: Distribution of respondent ages in the BRFSS subset.

Each bar counts how many respondents fall in that age range. You can read the center, the spread, and whether the distribution is roughly symmetric or lopsided straight off the picture.

💡 A function has many options — read its help page

`hist()` has many options: the number of bins, the bar color, the title, the axis labels, and more. You can pull up the full list any time by typing `?hist` in the R console. This is a habit worth building for *every* function you use. The documentation for any function `f` is one keystroke away with `?f` or `help("f")`, and it's the fastest way to learn what a function can do.

Next, compare a variable *across groups*. A boxplot is perfect for this. Here you compare the `Weight` distribution between the two sexes. The formula `Weight ~ Sex` reads as “weight broken down by sex”:

```
boxplot(brfss$Weight ~ brfss$Sex, main = "Weight Distribution by Sex",
        xlab = "Sex", ylab = "Weight", col = c("pink", "lightblue"))
```

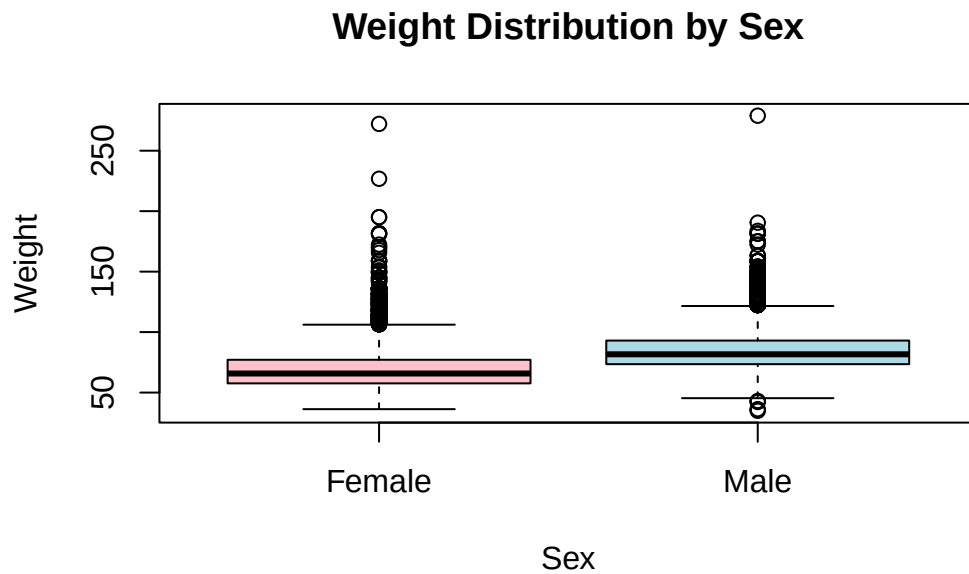


Figure 13.2.: Weight distribution compared between the sexes.

Each box spans the middle 50% of weights, the thick line is the median, and the whiskers reach out toward the extremes. At a glance you can see whether one group tends to weigh more than the other and whether the spreads differ.

13.6. Analyzing relationships between variables

So far you've looked at variables one at a time. EDA gets really interesting when you ask whether *two* variables move together. Start with a scatterplot of age against weight using `plot()`:

```
plot(brfss$Age, brfss$Weight, main = "Scatterplot of Age and Weight",  
     xlab = "Age", ylab = "Weight", col = "darkblue")
```

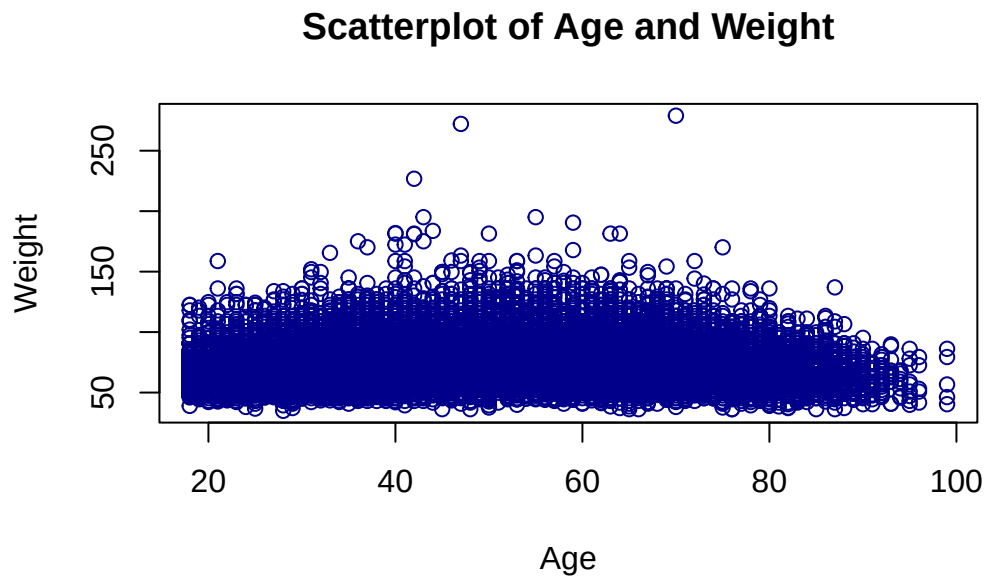


Figure 13.3.: Scatterplot of age versus weight.

Each point is one respondent. A cloud with no obvious slope means little relationship; a tilted, tightening band means the two variables tend to rise or fall together.

To put a single number on that, compute the **correlation coefficient** with `cor()`. It runs from -1 (perfect negative relationship) through 0 (none) to $+1$ (perfect positive relationship):

```
cor(brfss$Age, brfss$Weight)
```

```
[1] NA
```

That's not a number — it's NA. This is one of the most common surprises in R, so it's worth pausing on.

⚠ Missing values make `cor()` return NA

Remember those NA's the `summary()` flagged in the `Weight` column? By default `cor()` refuses to guess what a missing value should be, so if *any* pair has a missing value the

13. Case Study: Behavioral Risk Factor Surveillance System

whole result comes back NA. A quick look at `help("cor")` points to the fix: the `use` argument. Setting `use = "complete.obs"` tells `cor()` to drop the rows with missing values and compute the correlation on the rest.

```
cor(brfss$Age, brfss$Weight, use = "complete.obs")
```

```
[1] 0.02699989
```

Now you get a real number. It's small and positive, telling you age and weight are only weakly related in this dataset.

13.7. Comparing two groups: a t-test

EDA naturally raises questions that a picture alone can't settle. Here's one: the BRFSS subset spans two survey years, 1990 and 2010. **Did the average weight of female respondents change between those years?**

R read the `Year` column as a plain integer, but for grouping it's really a **category** with two levels. Convert it to a factor so R treats it that way:

```
brfss$Year <- factor(brfss$Year)
```

Now pull out just the female respondents into their own data frame so you can compare them across years:

```
brfssFemale <- brfss[brfss$Sex == "Female", ]  
summary(brfssFemale)
```

Age		Weight		Sex		Height		Year	
Min.	:18.00	Min.	: 36.29	Length	:12039	Min.	:105.0	1990:	5718
1st Qu.:	37.00	1st Qu.:	57.61	N.unique	: 1	1st Qu.:	157.5	2010:	6321
Median	:52.00	Median	: 65.77	N.blank	: 0	Median	:163.0		
Mean	:51.92	Mean	: 69.05	Min.nchar:	6	Mean	:163.3		
3rd Qu.:	67.00	3rd Qu.:	77.11	Max.nchar:	6	3rd Qu.:	168.0		
Max.	:99.00	Max.	:272.16			Max.	:200.7		
NAs	:103	NAs	:560			NAs	:140		

13. Case Study: Behavioral Risk Factor Surveillance System

A boxplot of weight by year shows the comparison at a glance. The formula `Weight ~ Year` again reads “weight broken down by year”:

```
plot(Weight ~ Year, brfssFemale)
```

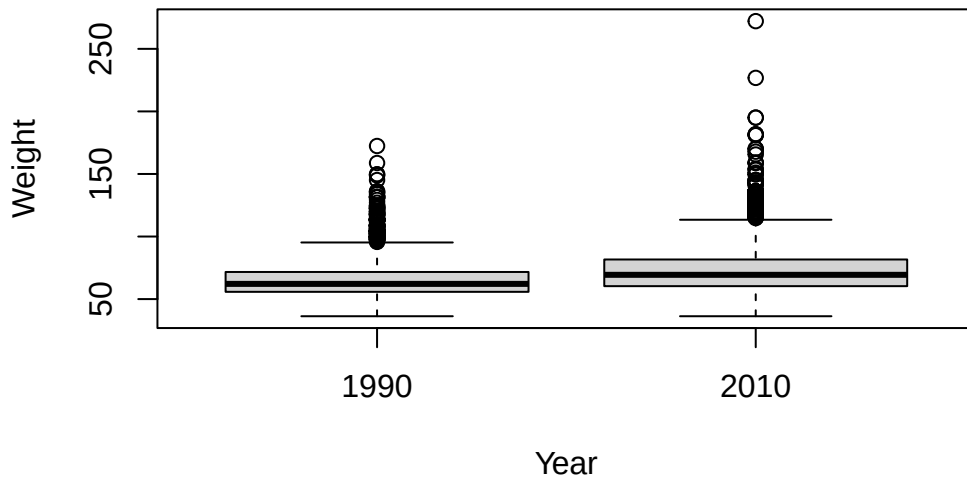


Figure 13.4.: Female respondent weight in 1990 versus 2010.

The 2010 box looks shifted up from the 1990 box — average weight appears higher in the later year. But “appears higher” isn’t an answer. To decide whether that gap is larger than you’d expect from chance alone, use a **two-sample t-test**. (We cover the t-test in depth in Section 17.6; here we just put it to work.)

```
t.test(Weight ~ Year, brfssFemale)
```

Welch Two Sample t-test

data: Weight by Year

t = -27.133, df = 11079, p-value < 2.2e-16

alternative hypothesis: true difference in means between group 1990 and group 2010 is not e

13. Case Study: Behavioral Risk Factor Surveillance System

```
95 percent confidence interval:
-8.723607 -7.548102
sample estimates:
mean in group 1990 mean in group 2010
64.81838          72.95424
```

Read the output from the bottom up. The two group means show 2010 females weigh more on average than 1990 females. The **p-value** is tiny — far below the usual 0.05 threshold — so the difference is *statistically significant*: a gap this large is very unlikely to be a fluke of sampling. The **confidence interval** for the difference doesn't include zero, telling the same story. You can now say, with evidence, that average female weight was higher in 2010 than in 1990.

13.8. Fitting a model: weight and height

A t-test compares groups. A different kind of question asks how one *continuous* variable depends on another: **for 2010 males, how does weight change with height?** That's a job for **linear regression**.

First, subset to 2010 males with `subset()`, then look at the pieces:

```
brfss2010Male <- subset(brfss, Year == 2010 & Sex == "Male")
summary(brfss2010Male)
```

Age		Weight		Sex		Height		Year	
Min.	:18.00	Min.	: 36.29	Length	:3679	Min.	:135	1990:	0
1st Qu.:	45.00	1st Qu.:	77.11	N.unique	: 1	1st Qu.:	173	2010:	3679
Median	:57.00	Median	: 86.18	N.blank	: 0	Median	:178		
Mean	:56.25	Mean	: 88.85	Min.nchar:	4	Mean	:178		
3rd Qu.:	68.00	3rd Qu.:	99.79	Max.nchar:	4	3rd Qu.:	183		
Max.	:99.00	Max.	:278.96			Max.	:218		
NAs	:30	NAs	:49			NAs	:31		

Look at each variable on its own first, then look at the relationship:

```
hist(brfss2010Male$Weight)
```

13. Case Study: Behavioral Risk Factor Surveillance System

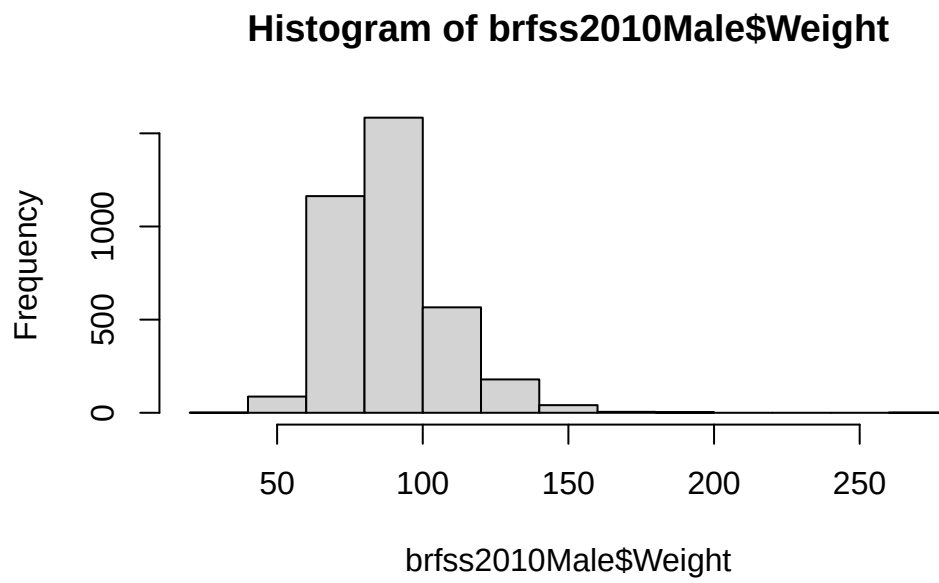


Figure 13.5.: Weight distribution for 2010 male respondents.

```
hist(brfss2010Male$Height)
```

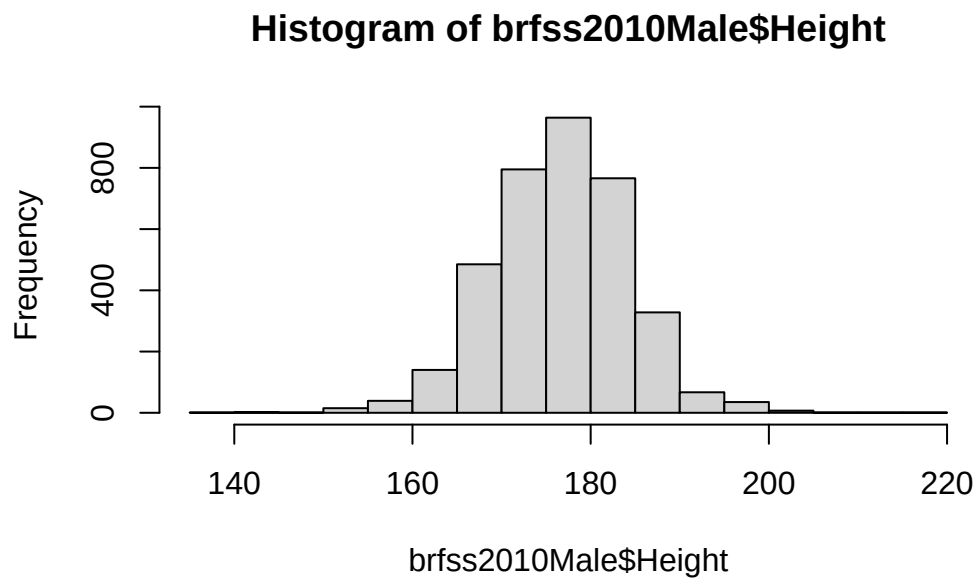


Figure 13.6.: Height distribution for 2010 male respondents.

```
plot(Weight ~ Height, brfss2010Male)
```

13. Case Study: Behavioral Risk Factor Surveillance System

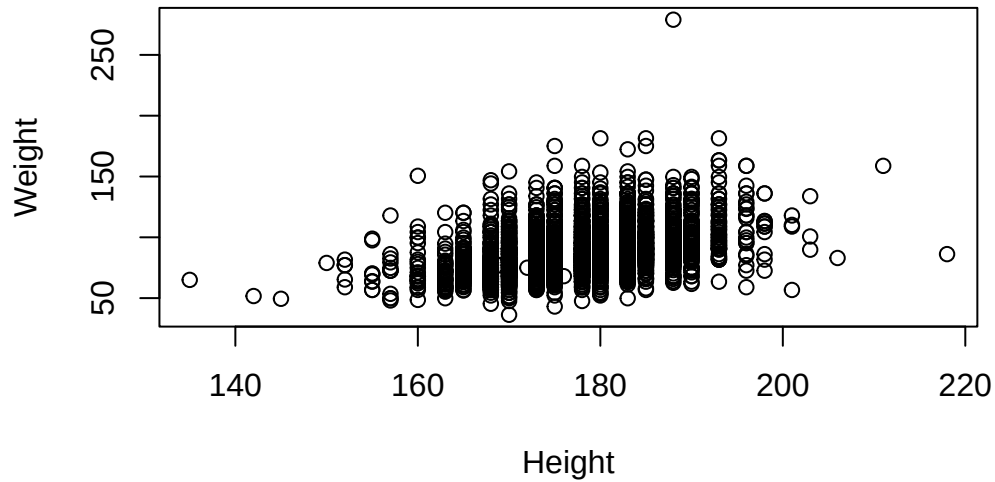


Figure 13.7.: Weight versus height for 2010 male respondents.

The scatterplot shows the expected upward trend: taller men tend to weigh more. Now quantify that trend by fitting a straight line through the cloud with `lm()` (“linear model”). The formula `Weight ~ Height` reads “model weight as a function of height”:

```
fit <- lm(Weight ~ Height, brfss2010Male)
fit
```

Call:

```
lm(formula = Weight ~ Height, data = brfss2010Male)
```

Coefficients:

(Intercept)	Height
-86.8747	0.9873

The printout gives two **coefficients**. The **(Intercept)** is where the line crosses zero height (not physically meaningful here, just where the math anchors the line). The **Height**

13. Case Study: Behavioral Risk Factor Surveillance System

coefficient is the **slope**: for each additional centimeter of height, predicted weight goes up by that many kilograms. That single number is the relationship, summarized.

You can lay out the same fit as an **ANOVA table**, which partitions the variability in weight and tests whether height explains a significant share of it:

```
anova(fit)
```

Analysis of Variance Table

Response: Weight

	Df	Sum Sq	Mean Sq	F value	Pr(>F)
Height	1	197664	197664	693.8	< 2.2e-16 ***
Residuals	3617	1030484	285		

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

The small p-value on the **Height** row confirms what the scatterplot suggested: height explains a statistically significant amount of the variation in weight.

To *see* the fitted model, redraw the scatterplot and superimpose the regression line with `abline()`, which knows how to read the line straight out of a fitted model object:

```
plot(Weight ~ Height, brfss2010Male)
abline(fit, col = "blue", lwd = 2)
# Substitute your own weight and height to see where you land...
points(73 * 2.54, 178 / 2.2, col = "red", cex = 4, pch = 20)
```

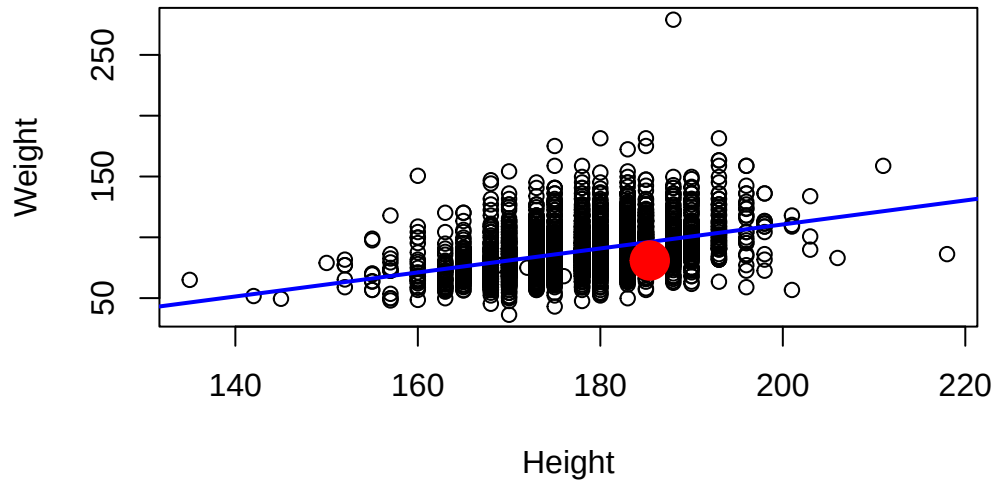


Figure 13.8.: The fitted regression line through the weight-versus-height data, with one point marked in red.

The blue line is the model's best guess at average weight for any given height. The red dot is one person's height and weight (converted to centimeters and kilograms); swap in your own numbers to see where you'd fall relative to the trend.

i Reading the model's diagnostics

A fitted model is a *noun*; the things you can do to it are *verbs* (R calls them **methods**). When you call `plot()` on a linear model, R quietly routes it to a special version, `plot.lm()`, that draws diagnostic plots instead of a scatterplot. The first one, **Residuals vs Fitted**, is the most useful: it plots the model's errors against its predictions. You want a shapeless cloud centered on zero — any strong pattern (a curve, a funnel) is a sign the straight-line model is missing something.

```
plot(fit, which = 1)
```

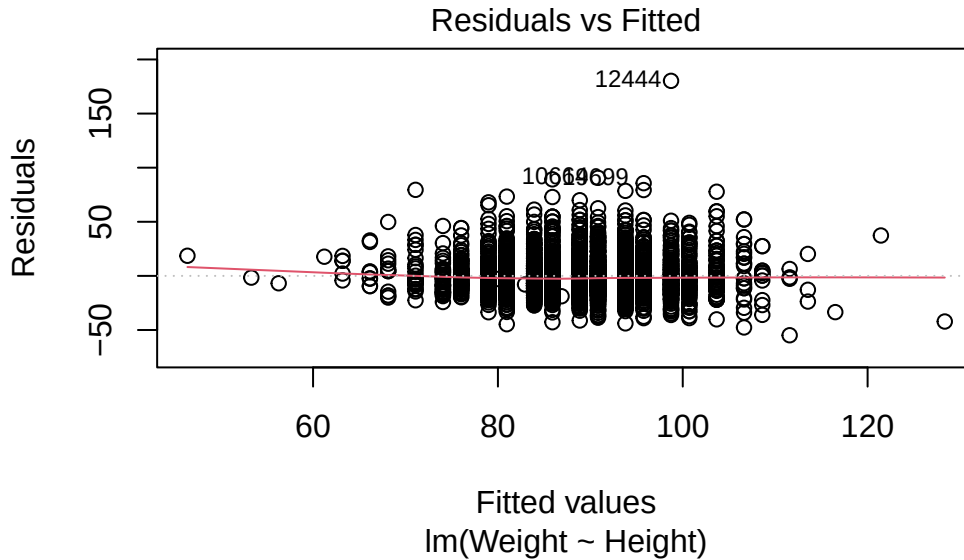


Figure 13.9.: Residuals-versus-fitted diagnostic plot for the weight-versus-height model.

You can see the full set of diagnostics with `plot(fit)` (it draws four), and read about them with `?plot.lm`.

13.9. Exercises

1. What is the mean weight in this dataset? How about the median? What is the difference between the two, and what does that tell you about the shape of the weight distribution?

```
mean(brfss$Weight, na.rm = TRUE)
```

```
[1] 75.42455
```

```
median(brfss$Weight, na.rm = TRUE)
```

```
[1] 72.57478
```

13. Case Study: Behavioral Risk Factor Surveillance System

```
mean(brfss$Weight, na.rm = TRUE) - median(brfss$Weight, na.rm = TRUE)
```

```
[1] 2.849774
```

The mean sits above the median, which is the signature of a distribution with a long right tail — a few very heavy individuals pull the mean up. (Note the `na.rm = TRUE`: just like `cor()`, these functions need to be told to ignore the missing values.)

2. Using what you just found about the mean and median, draw a histogram of the weight distribution with `hist()`. How would you describe its shape?

```
hist(brfss$Weight, xlab = "Weight (kg)", breaks = 30)
```

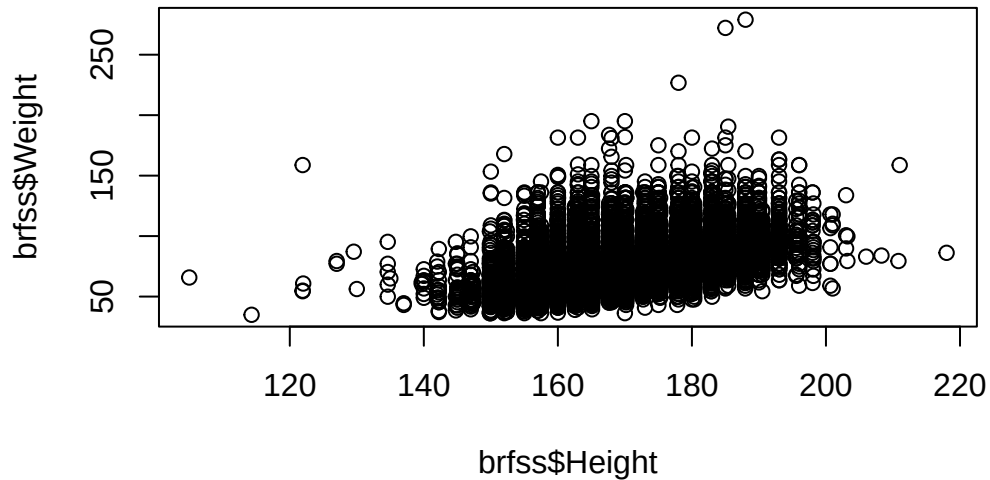


The histogram is **right-skewed**: most respondents cluster at lower weights with a tail stretching toward the high end — exactly what the mean-above-median result predicted.

3. Use `plot()` to examine the relationship between height and weight.

```
plot(brfss$Height, brfss$Weight)
```


13. Case Study: Behavioral Risk Factor Surveillance System



Taller people tend to be heavier, so the points drift upward to the right.

4. What is the correlation between height and weight? What does it tell you about their relationship?

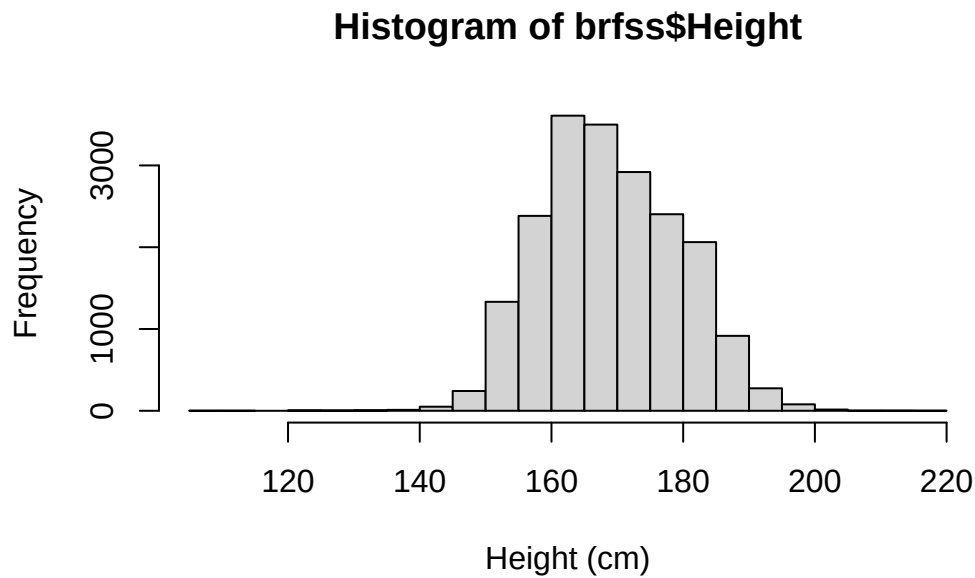
```
cor(brfss$Height, brfss$Weight, use = "complete.obs")
```

```
[1] 0.5140928
```

A moderate-to-strong positive correlation, confirming numerically what the scatterplot showed. (Don't forget `use = "complete.obs"` to skip the missing values.)

5. Draw a histogram of the height distribution. How would you describe its shape compared to weight?

```
hist(brfss$Height, xlab = "Height (cm)", breaks = 30)
```



Height is far more symmetric than weight — closer to the bell-shaped normal curve, with no strong tail to either side.

13.10. Summary

You've now run a complete exploratory data analysis on a real public-health dataset, end to end:

- You **loaded** the data with `download.file()` and `read.csv()`, and guarded the load with `stopifnot(file.exists(path))`.
- You **inspected** it with `head()`, `dim()`, and `summary()`, which flagged the missing values that mattered later.
- You **visualized** single variables with `hist()` and `boxplot()`, and pairs of variables with `plot()`.
- You **measured** a relationship with `cor()` — and learned that missing values force you to add `use = "complete.obs"`.
- You went **beyond description** to answer real questions: a `t.test()` showed female weight rose between 1990 and 2010, and an `lm()` quantified how male weight rises with height.

13. Case Study: Behavioral Risk Factor Surveillance System

EDA is the beginning of the data-analysis process, not the end — but it’s the part that keeps every later step honest. Knowing the shape of your data, its missing values, and its obvious relationships is what lets you choose the right model and trust the answer it gives. For more on the t-test you used here, see [Section 17.6](#).

14. Exploring data with ggplot2

This chapter is based on the [Intro to ggplot2 chapter](#) from the book [Modern Data Visualization with R](#) by Robert Kabacoff, which is licensed under the [Creative Commons Attribution-NonCommercial 4.0 International License](#). The original chapter has been modified to fit the context of this book.

The [insurance dataset](#) is described in the book *Machine Learning with R* by Brett Lantz. A cleaned version of the dataset is also available on Kaggle. The dataset describes medical information and costs billed by health insurance companies in 2013, as compiled by the United States Census Bureau. Variables include age, sex, body mass index, number of children covered by health insurance, smoker status, US region, and individual medical costs billed by health insurance for 1338 individuals.

In this chapter, we will explore the dataset using `ggplot2`, a powerful visualization package in R.

To get started, we need to install and load the `ggplot2` package. If you haven't installed it yet, you can do so using the following command:

```
install.packages("ggplot2")
```

Once installed, load the package:

```
library(ggplot2)
```

Next, we will read the insurance dataset into R. We'll use a convenient online version of the dataset and use the `read.csv` function to load it:

```
# load the data
url <- "https://tinyurl.com/mtktm8e5"
insurance <- read.csv(url)
```

In Rstudio, you can use the `View()` function to inspect the dataset:

14. Exploring data with *ggplot2*

```
# view the dataset  
View(insurance)
```

Next, we'll add a variable indicating if the patient is obese or not. Obesity will be defined as a body mass index greater than or equal to 30.

```
# create an obesity variable  
insurance$obese <- ifelse(insurance$bmi >= 30,  
                          "obese", "not obese")
```

In building a *ggplot2* graph, only the first two functions described below are required. The others are optional and can appear in any order.

14.1. *ggplot()*

The *ggplot()* function initializes the plot. It takes a data frame as its first argument and can also include aesthetic mappings (*aes*) that define how variables in the data are mapped to visual properties of the plot, such as x and y axes, color, size, etc.

```
# initialize the plot  
ggplot(data = insurance, aes(x = age, y = expenses))
```

14. Exploring data with *ggplot2*

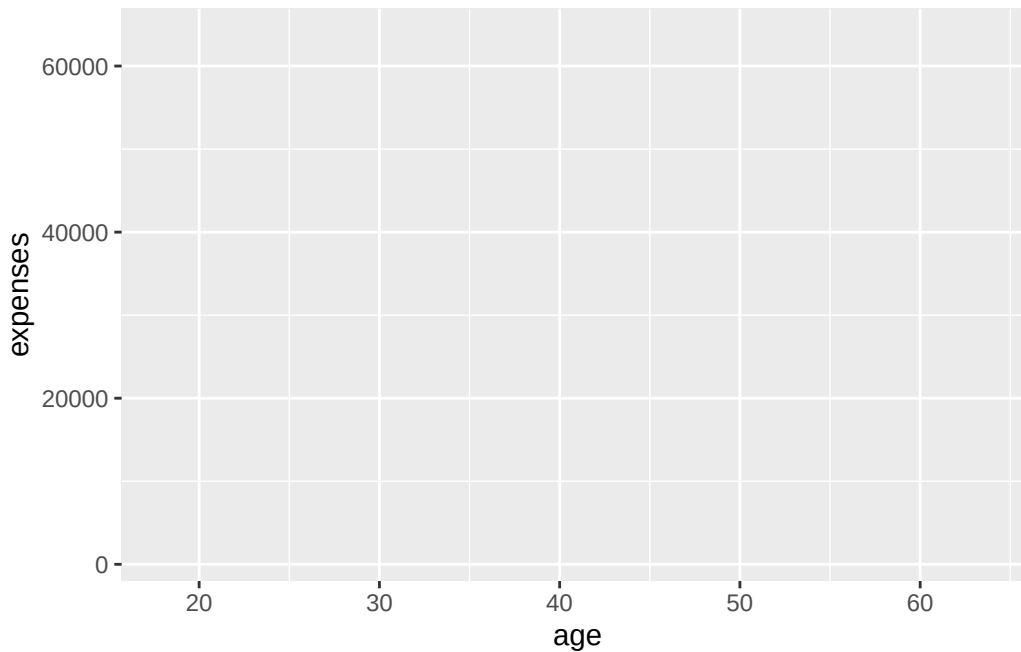


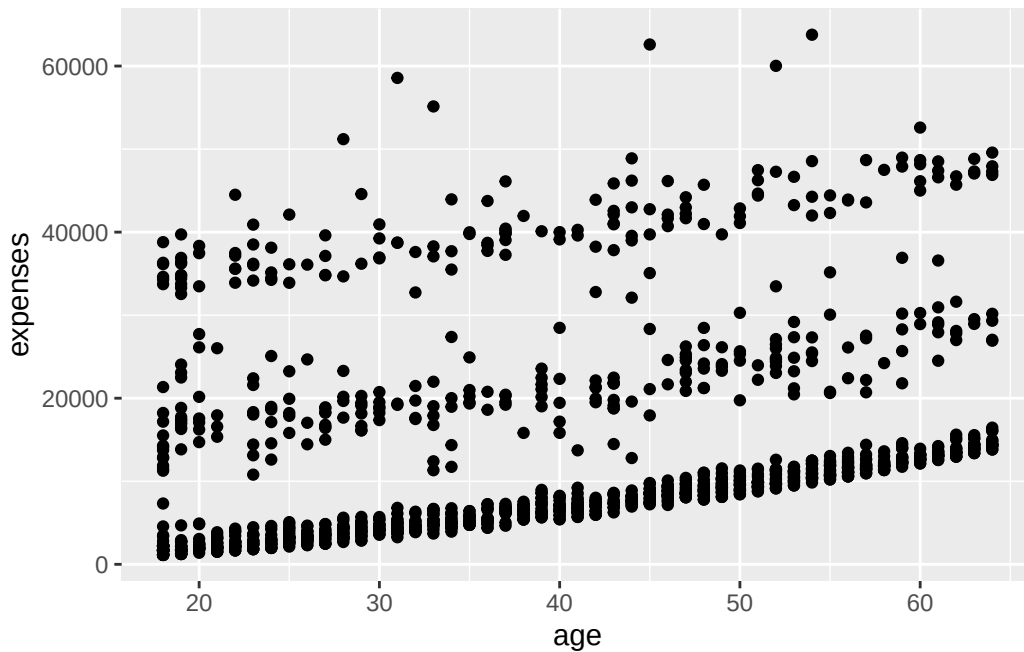
Figure 14.1.: Initialize the plot with `ggplot()`

Why is Figure 14.1 not showing a plot? Because we have not added any layers to the plot yet. The `ggplot()` function only initializes the plot; it does not display anything until we add layers. We specified that the `age` variable should be mapped to the x-axis and that the `expenses` should be mapped to the y-axis, but we haven't yet specified what we wanted placed on the graph.

14.2. `geom_*()`

The `geom_*()` functions add layers to the plot. Each `geom_*()` function corresponds to a specific type of geometric object, such as points, lines, bars, etc. For example, `geom_point()` adds points to the plot, while `geom_line()` adds lines.

```
# add points to the plot
ggplot(data = insurance,
       mapping = aes(x = age, y = expenses)) +
  geom_point()
```

Figure 14.2.: Add points to the plot with `geom_point()`

In Figure 14.2, we added points to the plot using `geom_point()`. The `+` operator is used to add layers to the plot. The `mapping` argument in `aes()` specifies how variables in the data are mapped to visual properties of the plot.

We can see in Figure 14.2 that insurance expenses increase with age, but there is a lot of variability in the data.

A number of parameters (options) can be specified in a `geom_` function. Options for the `geom_point` function include `color`, `size`, and `alpha`. These control the point color, size, and transparency, respectively. Transparency ranges from 0 (completely transparent) to 1 (completely opaque). Adding a degree of transparency can help visualize overlapping points.

```
# make points blue, larger, and semi-transparent
ggplot(data = insurance,
       mapping = aes(x = age, y = expenses)) +
  geom_point(color = "cornflowerblue",
            alpha = .7,
            size = 2)
```

14. Exploring data with *ggplot2*

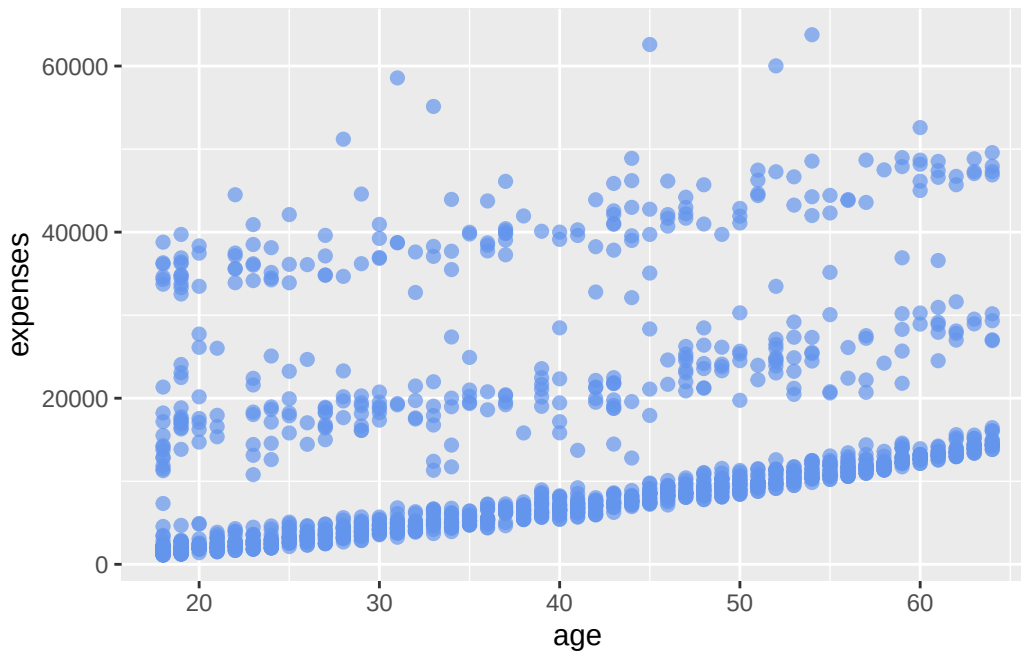
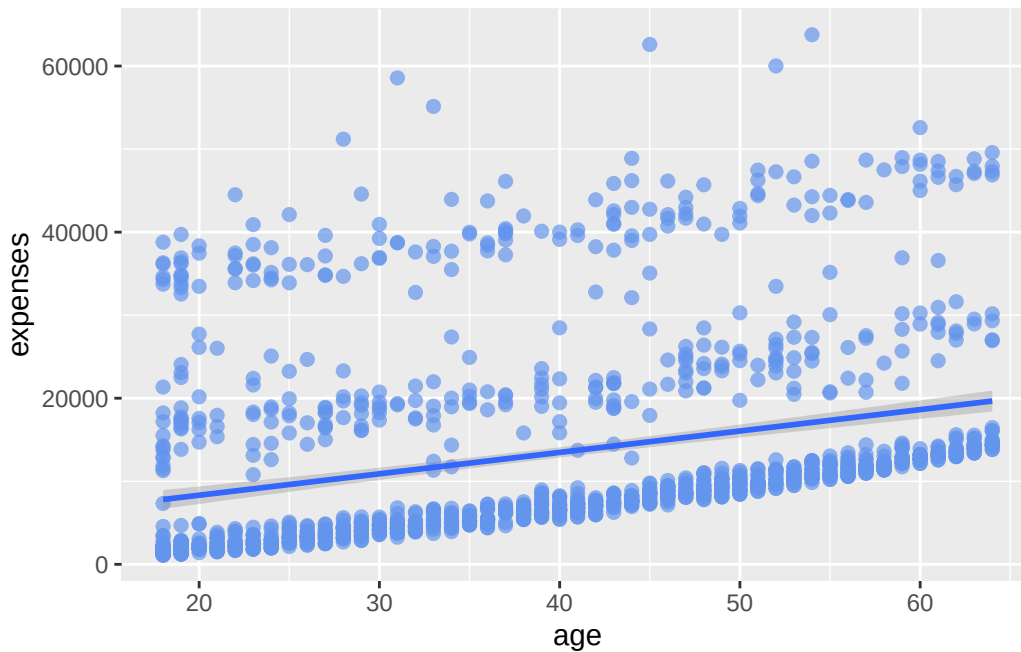


Figure 14.3.: Modify point color, size, and transparency.

Next, we can add a line of best fit; in essence, we will layer on a regression fit. We can do this with the `geom_smooth` function. Options control the type of line (linear, quadratic, nonparametric), the thickness of the line, the line's color, and the presence or absence of a confidence interval. Here we request a linear regression (`method = lm`) line (where `lm` stands for linear model).

```
# add a line of best fit
ggplot(data = insurance,
       mapping = aes(x = age, y = expenses)) +
  geom_point(color = "cornflowerblue",
            alpha = .7,
            size = 2) +
  geom_smooth(method = "lm")
```

```
`geom_smooth()` using formula = 'y ~ x'
```


Figure 14.4.: Add a line of best fit with `geom_smooth()`

In Figure 14.4, we added a line of best fit using `geom_smooth(method = "lm")`. The `method` argument specifies the type of smoothing to apply. In this case, we used a linear model (lm) to fit the line.

14.3. Grouping

In addition to mapping variables to the x and y axes, groups of observations can be mapped to the color, shape, size, transparency, and other visual characteristics of geometric objects. This allows groups of observations to be superimposed in a single graph.

Let's add smoker status to the plot and represent it by color.

```
# group points by smoker status
ggplot(data = insurance,
       mapping = aes(x = age, y = expenses, color = smoker)) +
  geom_point(alpha = .7, size = 2) +
  geom_smooth(method = "lm", se = FALSE)
```

```
`geom_smooth()` using formula = 'y ~ x'
```

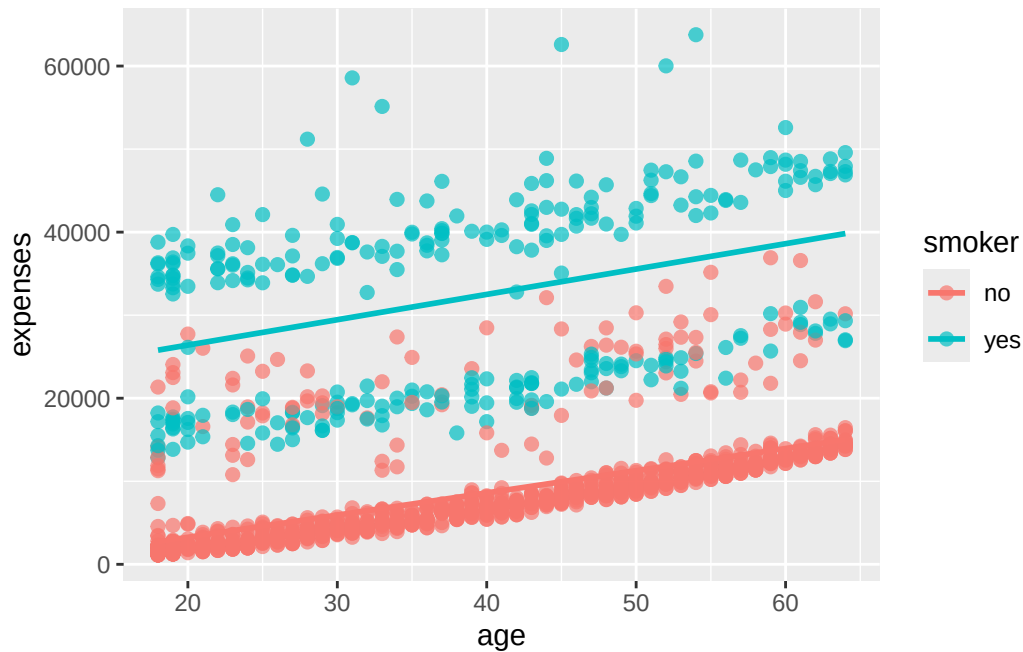


Figure 14.5.: Group points by smoker status.

In Figure 14.5, we added the `color` aesthetic to the `aes()` function to map the `smoker` variable to the color of the points and the line of best fit. This allows us to see how the relationship between age and expenses differs for smokers and non-smokers. It probably comes as no surprise that smokers appear to incur greater expenses than non-smokers.

14.4. Scales

Scales control how variables are mapped to the visual characteristics of the plot. Scale functions (which start with `scale_`) allow you to modify this mapping. In the next plot, we'll change the x and y axis scaling, and the colors employed.

```
# modify scales for x and y axes, and colors
# modify the x and y axes and specify the colors to be used
ggplot(data = insurance,
       mapping = aes(x = age,
                     y = expenses,
                     color = smoker)) +
  geom_point(alpha = .5,
```

14. Exploring data with ggplot2

```
size = 2) +  
geom_smooth(method = "lm",  
            se = FALSE,  
            size = 1.5) +  
scale_x_continuous(breaks = seq(0, 70, 10)) +  
scale_y_continuous(breaks = seq(0, 60000, 20000),  
                  label = scales::dollar) +  
scale_color_manual(values = c("indianred3",  
                              "cornflowerblue"))
```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.

`geom\_smooth()` using formula = 'y ~ x'

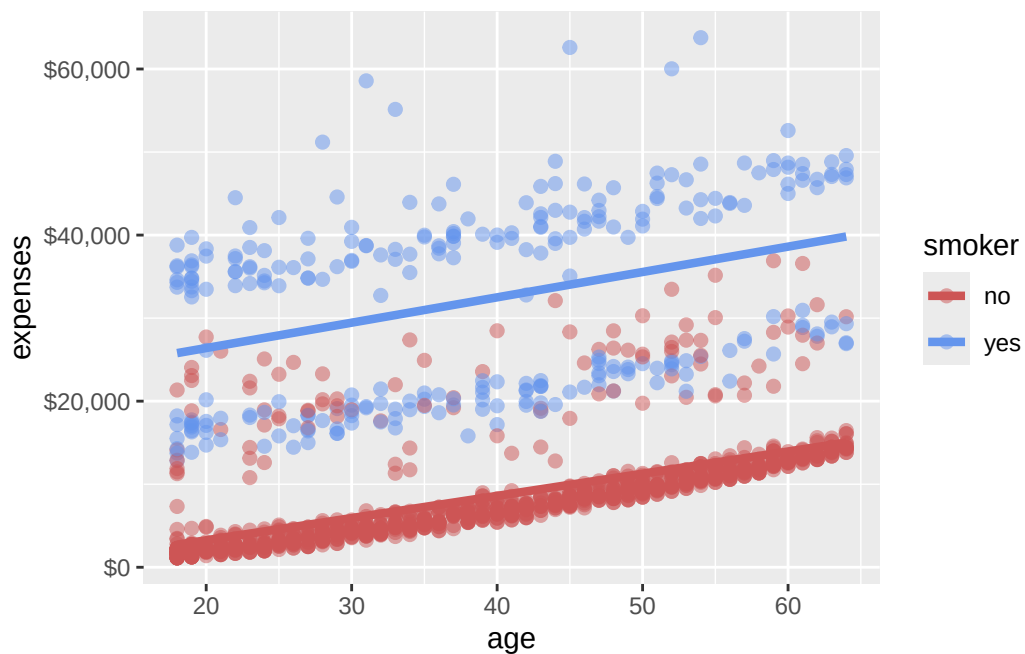


Figure 14.6.: Modify scales for x and y axes, and colors.

In Figure 14.6, we used `scale_x_continuous()` and `scale_y_continuous()` to modify the x and y axes, respectively. The `breaks` argument specifies the tick marks on the axes, and the `label` argument in `scale_y_continuous()` formats the y-axis labels as dollar

amounts using the `scales::dollar` function. We also used `scale_color_manual()` to specify custom colors for the points based on smoker status.

14.5. Facets

Faceting allows you to create multiple plots based on a categorical variable. This is useful for comparing distributions or relationships across different groups. The `facet_wrap()` function is commonly used for this purpose.

```
# create facets based on the obese variable
# reproduce plot for each obese and non-obese individuals
ggplot(data = insurance,
       mapping = aes(x = age,
                     y = expenses,
                     color = smoker)) +
  geom_point(alpha = .5) +
  geom_smooth(method = "lm",
             se = FALSE) +
  scale_x_continuous(breaks = seq(0, 70, 10)) +
  scale_y_continuous(breaks = seq(0, 60000, 20000),
                    label = scales::dollar) +
  scale_color_manual(values = c("indianred3",
                                "cornflowerblue")) +
  facet_wrap(~obese)
```

``geom_smooth()`` using formula = 'y ~ x'

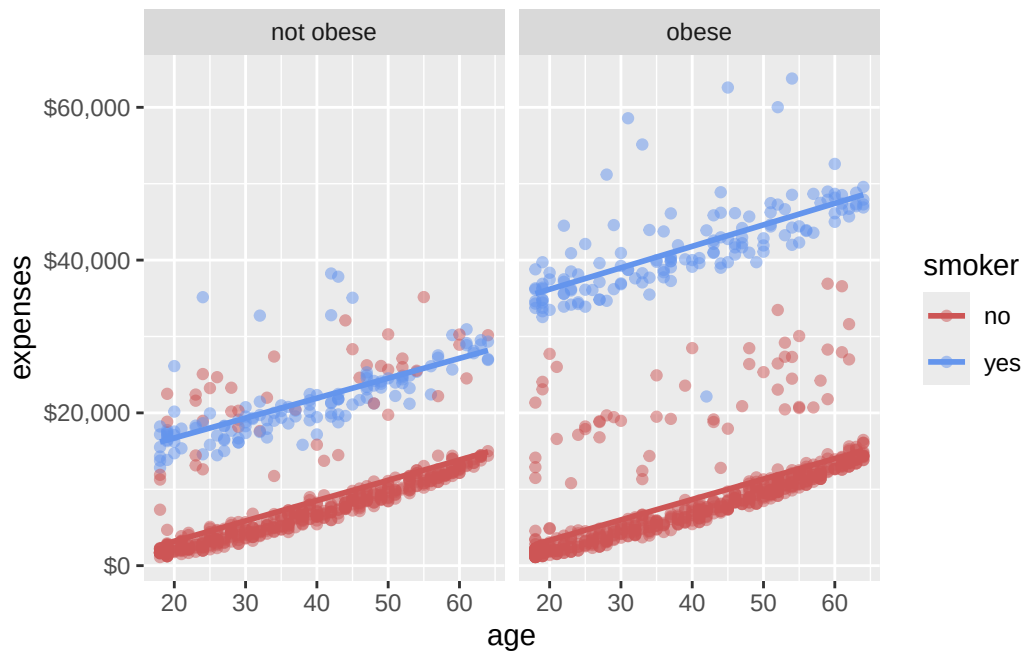


Figure 14.7.: Create facets based on the obese variable.

In Figure 14.7, we used `facet_wrap(~obese)` to create separate plots for obese and non-obese individuals. This allows us to compare the relationship between age and expenses for these two groups side by side. Pretty cool, right? We have now created a plot that shows the relationships among age, smoking status, obesity, and annual medical expenses. In essence, we have placed four dimensions of data into a two-dimensional plot!

14.6. Labels and Titles

Labels and titles are important for making your plots informative and easy to understand. The `labs()` function is used to add labels to the x and y axes, as well as a title for the plot.

```
# add labels and title to the plot
# add informative labels
ggplot(data = insurance,
       mapping = aes(x = age,
                     y = expenses,
                     color = smoker)) +
```

14. Exploring data with ggplot2

```
geom_point(alpha = .5) +  
geom_smooth(method = "lm",  
            se = FALSE) +  
scale_x_continuous(breaks = seq(0, 70, 10)) +  
scale_y_continuous(breaks = seq(0, 60000, 20000),  
                  label = scales::dollar) +  
scale_color_manual(values = c("indianred3",  
                              "cornflowerblue")) +  
facet_wrap(~obese) +  
labs(title = "Relationship between patient demographics and medical costs",  
      subtitle = "US Census Bureau 2013",  
      caption = "source: http://mosaic-web.org/",  
      x = "Age (years)",  
      y = "Annual expenses",  
      color = "Smoker?")
```

`geom\_smooth()` using formula = 'y ~ x'

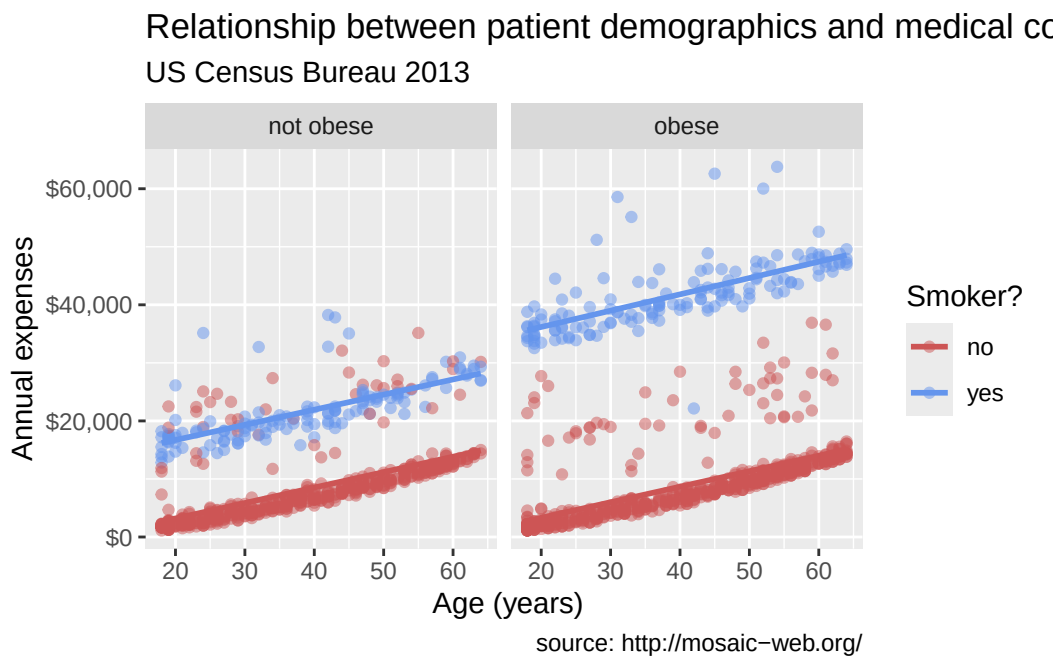


Figure 14.8.: Add labels and title to the plot.

In Figure 14.8, we used the `labs()` function to add a title, subtitle, caption, and labels for the x and y axes. This makes the plot more informative and easier to interpret.

14.7. Theming

Finally, we can fine tune the appearance of the graph using themes. Theme functions (which start with `theme_`) control background colors, fonts, grid-lines, legend placement, and other non-data related features of the graph. Let's use a cleaner theme.

```
# customize the plot's appearance with themes
# use a minimalist theme
ggplot(data = insurance,
       mapping = aes(x = age,
                     y = expenses,
                     color = smoker)) +
  geom_point(alpha = .5) +
  geom_smooth(method = "lm",
             se = FALSE) +
  scale_x_continuous(breaks = seq(0, 70, 10)) +
  scale_y_continuous(breaks = seq(0, 60000, 20000),
                    label = scales::dollar) +
  scale_color_manual(values = c("indianred3",
                                "cornflowerblue")) +
  facet_wrap(~obese) +
  labs(title = "Relationship between age and medical expenses",
       subtitle = "US Census Data 2013",
       caption = "source: https://github.com/dataspelunking/MLwR",
       x = "Age (years)",
       y = "Medical Expenses",
       color = "Smoker?") +
  theme_minimal()
```

```
`geom_smooth()` using formula = 'y ~ x'
```

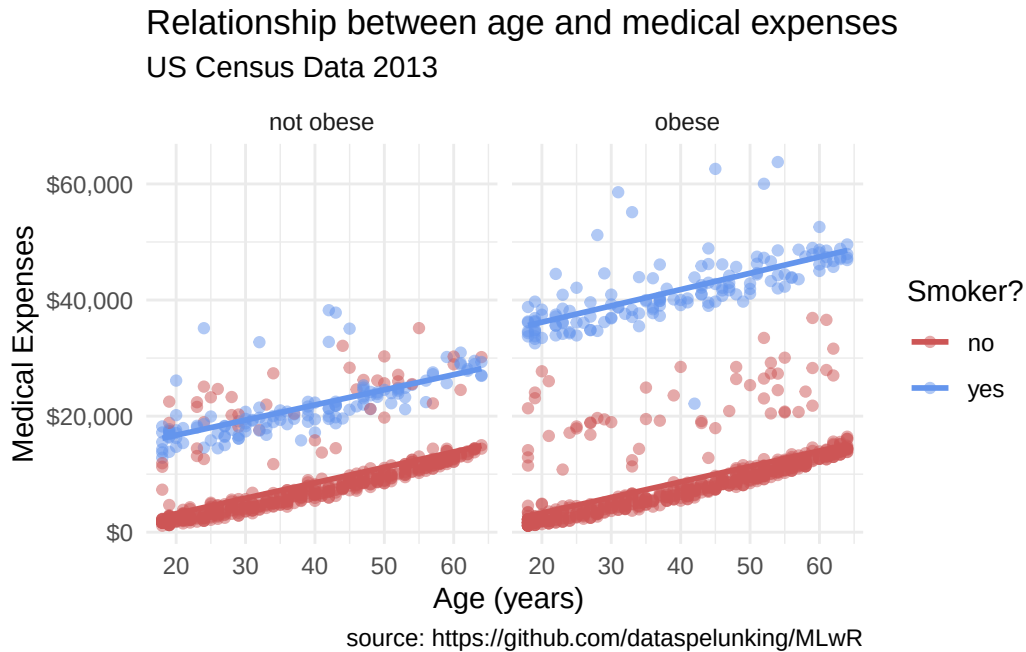


Figure 14.9.: Customize the plot's appearance with themes.

14.8. Conclusion

In this chapter, we explored the insurance dataset using *ggplot2*, a powerful visualization package in R. We learned how to initialize a plot with `ggplot()`, add layers with `geom_*()` functions, group observations, modify scales, create facets, add labels and titles, and customize the plot's appearance with themes.

Our final plot:

Now we have something. From Figure 14.9 it appears that:

- There is a positive linear relationship between age and expenses. The relationship is constant across smoking and obesity status (i.e., the slope doesn't change).
- Smokers and obese patients have higher medical expenses.
- There is an interaction between smoking and obesity. Non-smokers look fairly similar across obesity groups. However, for smokers, obese patients have much higher expenses.
- There are some very high outliers (large expenses) among the obese smoker group.

14. Exploring data with ggplot2

These findings are tentative. They are based on a limited sample size and do not involve statistical testing to assess whether differences may be due to chance variation.

15. A self-guided tour of data visualization in R

Imagine you have just finished an experiment. You have a spreadsheet with a thousand rows of penguin measurements, or a matrix of gene-expression values across dozens of samples, and you need to *see* what is going on. Which species are bigger? Are two genes switched on together? Staring at a wall of numbers will not tell you — but the right picture will. Data visualization is the bridge between raw measurements and human understanding, and it is one of the most useful skills a biologist can own.

This chapter is a guided tour. We start with two ideas that make *any* plot better, then build up `ggplot2` — R’s standard plotting system — one layer at a time so you understand *why* the code looks the way it does. From there we tour a few specialized plot types you’ll meet in genomics: UpSet plots for set overlaps, heatmaps for expression matrices, and a pointer to genome-track visualization. The companion chapter [Exploring data with ggplot2](#) walks a single dataset end to end; here we focus on the *grammar* and on the wider toolbox.

15.1. What you’ll learn

- Apply two design principles — the data-ink ratio and clear labeling — to make any plot more readable.
- Read a `ggplot2` call as a stack of **layers**: data → aesthetics (`aes()`) → geoms → scales → facets → themes.
- Build a plot incrementally, adding one layer at a time and interpreting each result.
- Choose a colorblind-safe palette with `RColorBrewer`.
- Reach for the right specialized plot — an UpSet plot for many sets, a heatmap for a data matrix — and produce a runnable example of each.

15.2. Two principles that make any plot better

Before we touch a single `ggplot()` call, it helps to know what we are aiming for. Two ideas carry most of the weight.

15.2.1. Maximize the data-ink ratio

The statistician Edward Tufte coined the term **data-ink ratio**: the proportion of a graphic's ink that is actually showing your data, rather than decoration. The goal is to keep that ratio high. Put simply, **every mark on the plot should earn its place**.

Common “chart junk” to delete:

- Redundant or heavy grid lines
- Busy backgrounds or unnecessary fill colors
- 3D effects on what is really 2D data
- Drop shadows and other decoration

Let's see the difference. We'll use R's built-in `mpg` dataset (fuel-economy records for various car models) — the relationship is generic, but the lesson about ink transfers directly to a scatterplot of, say, gene length versus expression. First, a cluttered version:

```
library(ggplot2)

ggplot(mpg, aes(x = displ, y = hwy, color = class)) +
  geom_point() +
  theme_gray() +
  labs(title = "Fuel efficiency vs. engine displacement",
       subtitle = "This is a very busy plot",
       x = "Engine displacement (litres)",
       y = "Highway miles per gallon",
       color = "Vehicle class")
```

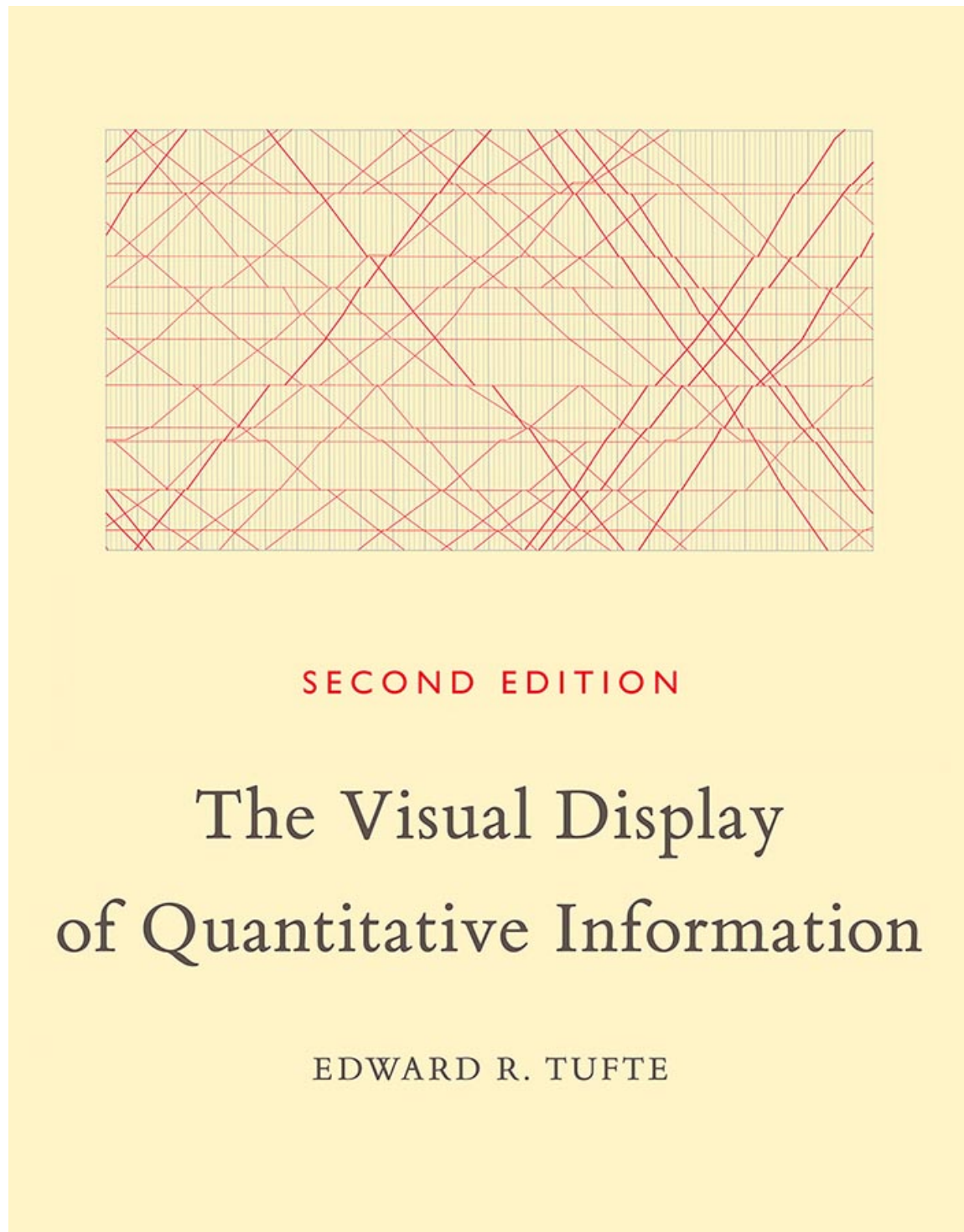


Figure 15.1.: Edward Tufte’s landmark book, *The Visual Display of Quantitative Information*, popularized the data-ink ratio.

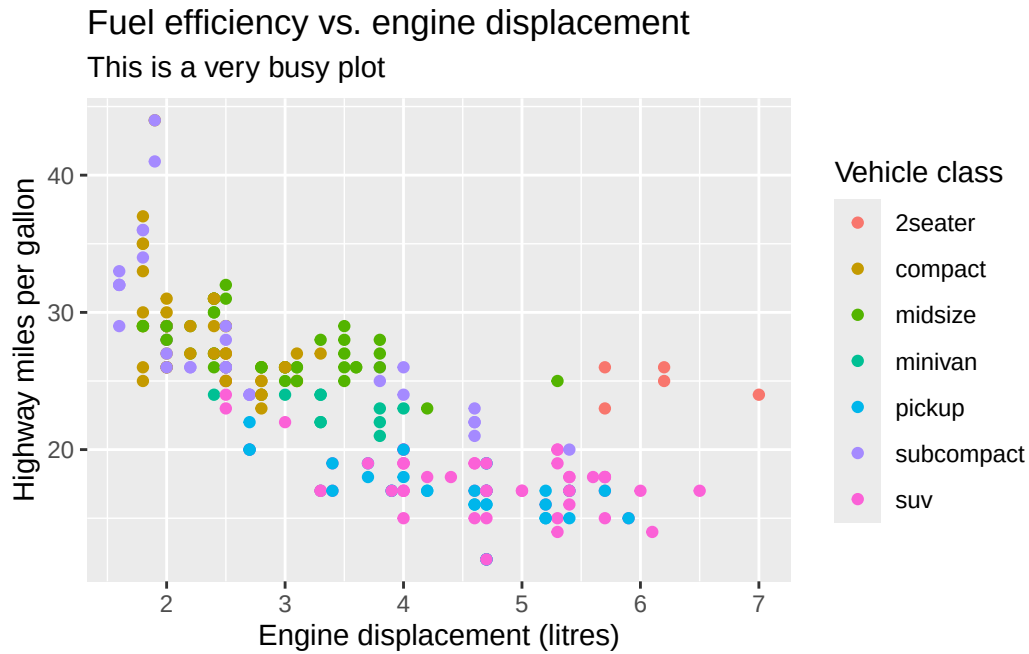


Figure 15.2.: A cluttered plot with a low data-ink ratio. The gray background and heavy gridlines are decoration, not information.

Now the same data with the decoration stripped away:

```
ggplot(mpg, aes(x = displ, y = hwy, color = class)) +
  geom_point() +
  theme_minimal() +
  labs(title = "Fuel efficiency vs. engine displacement",
       x = "Engine displacement (litres)",
       y = "Highway miles per gallon",
       color = "Class")
```

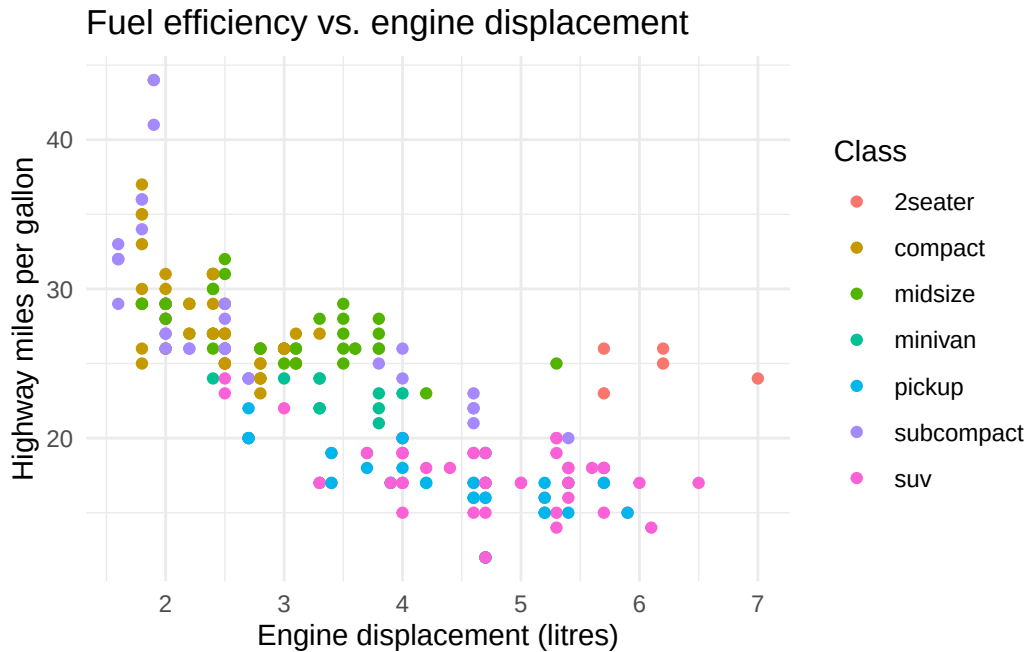


Figure 15.3.: The same data with a high data-ink ratio. The eye goes straight to the points.

The only change between Figure 15.2 and Figure 15.3 is the theme, yet the second reads more easily: the trend (bigger engines get worse mileage) jumps out because nothing competes with it. This is a guideline, not a law — a faint gridline can genuinely help a reader estimate values — but “could I delete this?” is always worth asking.

15.2.2. Label everything

A plot without labels is just a picture. To be analysis, it has to carry its context with it. Every finished plot should have:

- A clear, descriptive title.
- Axis labels *with units* (e.g., “Body mass (g)”).
- A legend whenever color, shape, or size encodes a variable.

💡 Labels are for future-you

The most common audience for your plot is yourself, three months later, trying to remember what it showed. Spelling out units and a title costs seconds now and saves

an embarrassing email later.

15.2.3. Use color deliberately

Color is powerful and easy to overuse. Two rules of thumb: don't use more colors than you have meaningful groups, and pick palettes that survive printing and colorblindness. The `RColorBrewer` package ships curated palettes in three families:

- **Sequential** palettes run light → dark and suit ordered data (a gradient from low to high expression).
- **Qualitative** palettes use distinct hues for *categories* with no order (species, treatment group).
- **Diverging** palettes emphasize a meaningful midpoint with two contrasting directions (log-fold-changes around zero).

```
library(RColorBrewer)

display.brewer.all()
```

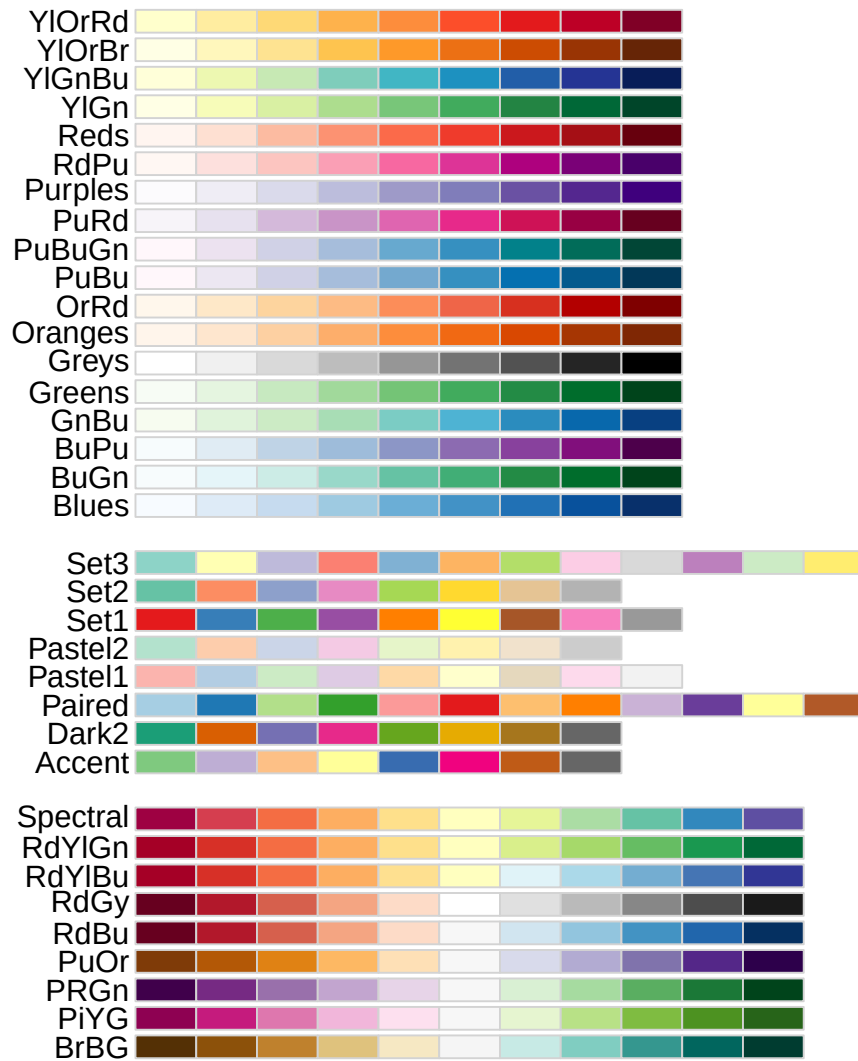


Figure 15.4.: RColorBrewer palettes, grouped into sequential (top), qualitative (middle), and diverging (bottom) families.

Figure 15.4 is a menu: pick a palette name (like "Set2" or "RdBu") and hand it to a `scale_color_brewer()` or `scale_fill_brewer()` layer.

 Roughly 1 in 12 men is red-green colorblind

A red/green scheme that looks fine to you can be unreadable to a chunk of your audience. `RColorBrewer` can show only the palettes that stay distinguishable under common color-vision deficiencies — restrict yourself to these when color carries meaning.

```
display.brewer.all(colorblindFriendly = TRUE)
```

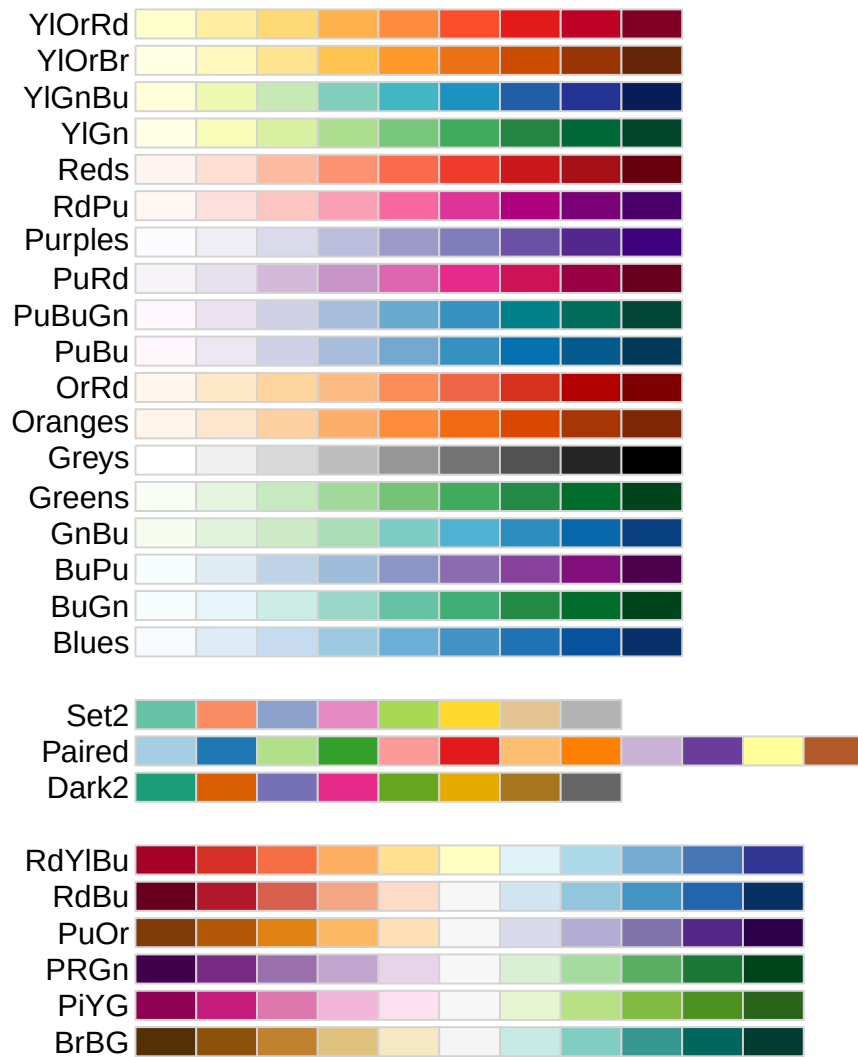


Figure 15.5.: The subset of RColorBrewer palettes that remain distinguishable for viewers with common color-vision deficiencies.

The palettes in Figure 15.5 are the safe default set. When in doubt, choose from here.

15.3. The grammar of graphics, layer by layer

`ggplot2` is built on an idea called the **grammar of graphics**: every plot is assembled from a small set of independent parts, the way a sentence is built from nouns, verbs, and adjectives. Once you know the parts, you can describe an enormous variety of plots with the same handful of “words.” The parts are:

1. **Data** — the data frame you’re plotting.
2. **Aesthetics** (`aes()`) — which *variable* maps to which *visual channel* (x position, y position, color, size, shape).
3. **Geoms** — the geometric shapes that actually get drawn (points, lines, bars).
4. **Scales** — how data values translate into pixels and colors (axis ranges, color palettes).
5. **Facets** — splitting one plot into small multiples by a grouping variable.
6. **Themes** — the non-data styling (fonts, background, gridlines).

Think of it like ordering a coffee: the **data** is the beans, the **aesthetics** say what goes in which cup, the **geom** is whether you get it as espresso or latte, and the **theme** is the mug it’s served in. We’ll build one plot up through these layers, adding exactly one new idea per step.

We’ll use the `palmerpenguins` dataset — body measurements for three penguin species (*Adelie*, *Chinstrap*, *Gentoo*) recorded near Palmer Station, Antarctica. It’s real organismal biology and small enough to reason about.

```
library(palmerpenguins)
library(dplyr)

# drop the few rows with missing measurements so plots don't warn
penguins <- na.omit(penguins)
glimpse(penguins)
```

```
Rows: 333
Columns: 8
$ species      <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel~
$ island       <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgerse~
$ bill_length_mm <dbl> 39.1, 39.5, 40.3, 36.7, 39.3, 38.9, 39.2, 41.1, 38.6~
$ bill_depth_mm <dbl> 18.7, 17.4, 18.0, 19.3, 20.6, 17.8, 19.6, 17.6, 21.2~
$ flipper_length_mm <int> 181, 186, 195, 193, 190, 181, 195, 182, 191, 198, 18~
```

15. A self-guided tour of data visualization in R

```
$ body_mass_g      <int> 3750, 3800, 3250, 3450, 3650, 3625, 4675, 3200, 3800~  
$ sex              <fct> male, female, female, female, male, female, male, fe~  
$ year            <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007~
```

`glimpse()` shows each penguin has a species, an island, bill and flipper measurements, a body mass, and a sex. We'll ask a biological question: **how do the species differ in body size?**

15.3.1. Step 1: data and aesthetics

We start by telling `ggplot()` *what data* to use and *which variables map to which axes*. Here, flipper length on x and body mass on y:

```
ggplot(data = penguins,  
       mapping = aes(x = flipper_length_mm, y = body_mass_g))
```

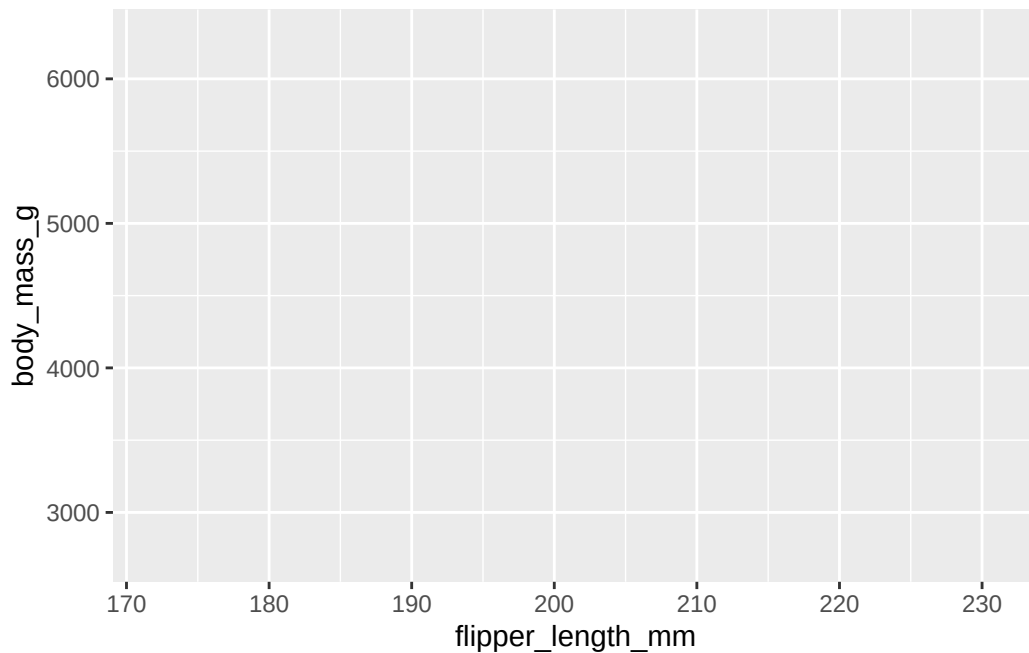


Figure 15.6.: Data plus aesthetics, but no geom: `ggplot()` sets up the axes but draws nothing yet.

Figure 15.6 looks empty — and that’s the point. We’ve declared the axes (`aes()` mapped `flipper_length_mm` to `x` and `body_mass_g` to `y`), but we haven’t said *what shape* to draw. A plot with no geom has nothing to show.

15.3.2. Step 2: add a geom

A **geom** is the layer that draws something. We add layers with `+`. For two continuous variables, points make a scatterplot:

```
ggplot(data = penguins,  
       mapping = aes(x = flipper_length_mm, y = body_mass_g)) +  
  geom_point()
```

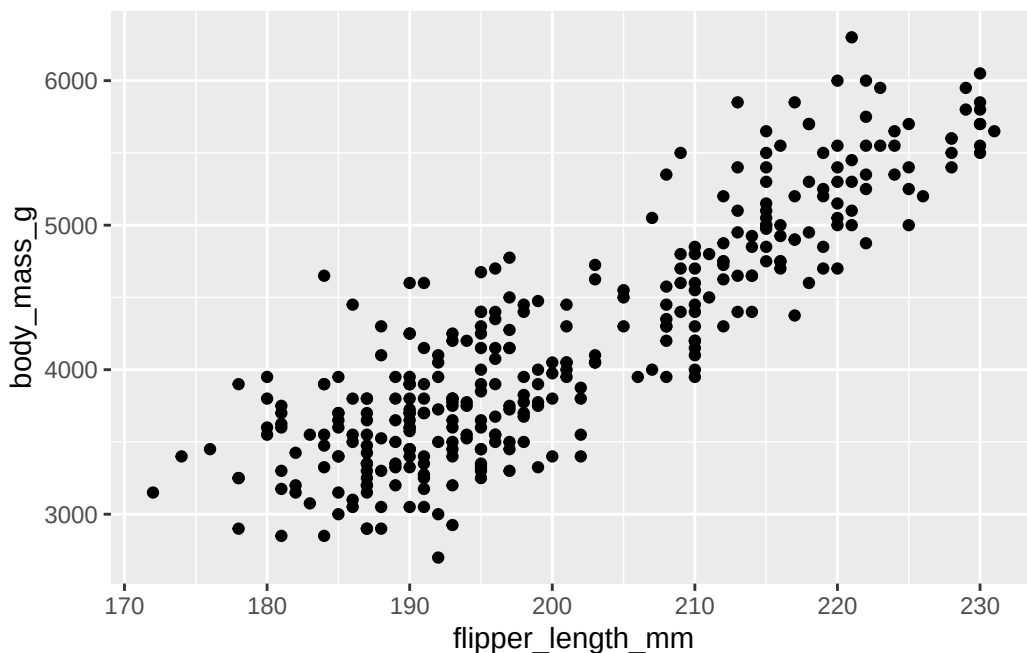


Figure 15.7.: Adding `geom_point()` draws one point per penguin.

Now Figure 15.7 tells a story: heavier penguins tend to have longer flippers, a clear positive relationship. But the points seem to fall into a couple of clusters. Could that be the species?

15.3.3. Step 3: map a third variable to color

Aesthetics aren't limited to x and y. We can map a *categorical* variable onto **color** to encode a third dimension on the same flat plot:

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point()
```

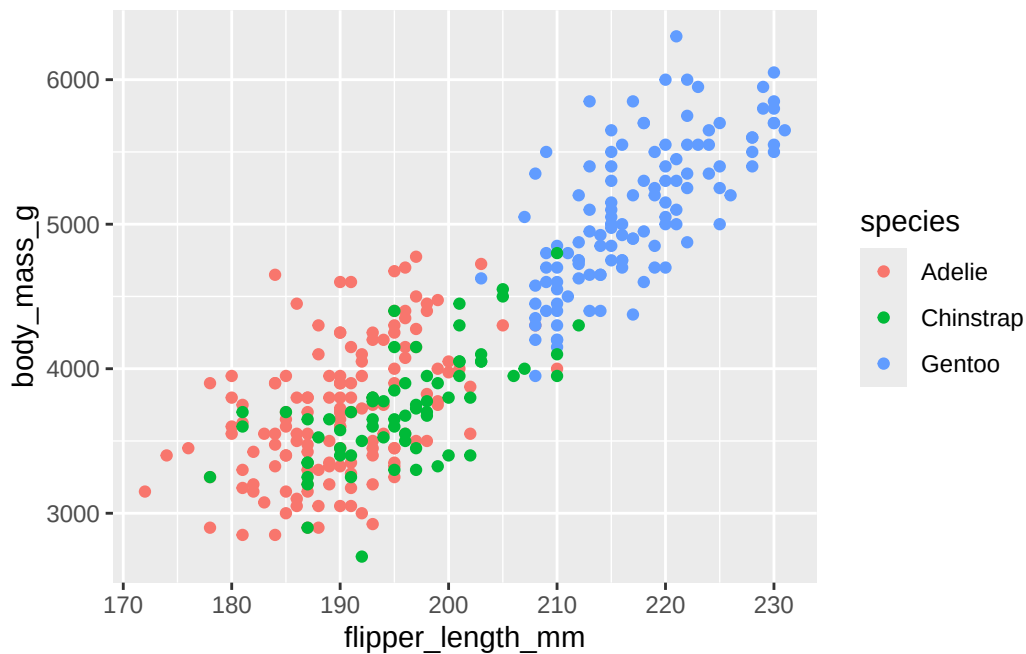


Figure 15.8.: Mapping species to the color aesthetic reveals three clusters.

The clusters in Figure 15.8 resolve cleanly: Gentoo penguins are the large ones (long flippers, heavy), while Adelie and Chinstrap overlap at the smaller end. We answered our biological question by adding *one word* to `aes()`.

i Inside `aes()` vs. outside it

Put a setting **inside** `aes()` when it should vary *with the data* (`color = species`). Put it **outside** `aes()`, in the geom, when it's a fixed constant (`geom_point(color = "blue")` makes every point blue). Mixing these up is the single most common

ggplot2 confusion.

15.3.4. Step 4: layer on a second geom

Layers stack. We can keep the points and add a `geom_smooth()` trend line on top. Because `color = species` is set in the shared `aes()`, we get one line per species for free:

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = FALSE)
```

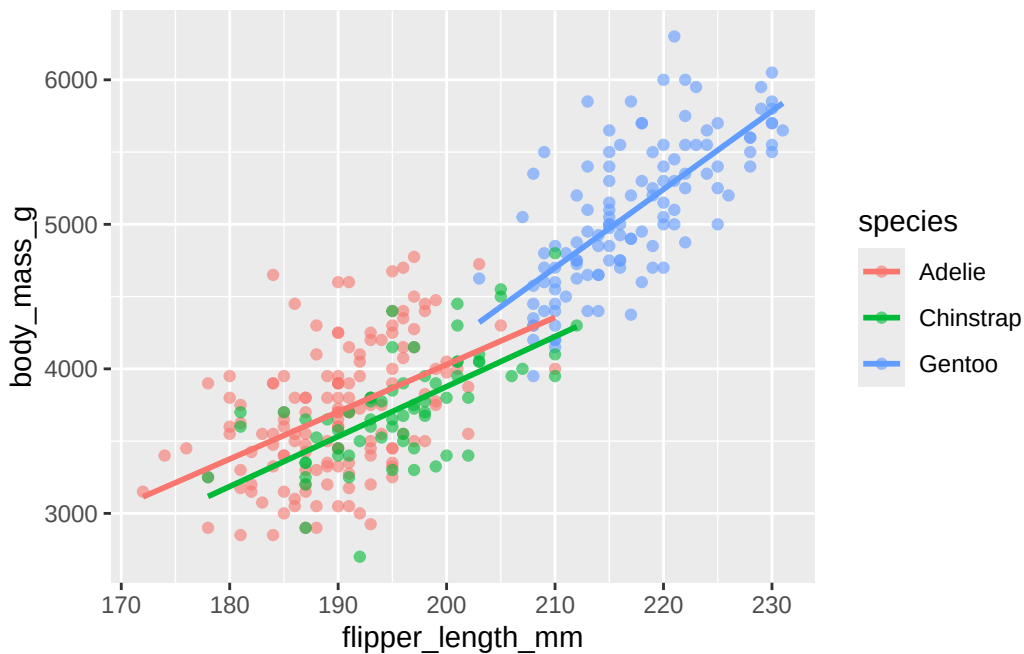


Figure 15.9.: A second geom (a linear trend line) layered over the points, one line per species.

Figure 15.9 adds three regression lines (`method = "lm"`). All three slope upward, confirming the flipper-mass relationship holds *within* each species, not just across the pooled cloud. We also set `alpha = 0.6` (outside `aes()`, so it's constant) to make overlapping points semi-transparent.

15.3.5. Step 5: control scales

Scales govern the translation from data to pixels and colors. Let's swap in a colorblind-safe Brewer palette and label the axis ticks with units:

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = FALSE) +
  scale_color_brewer(palette = "Set2") +
  scale_y_continuous(breaks = seq(3000, 6000, 1000))
```

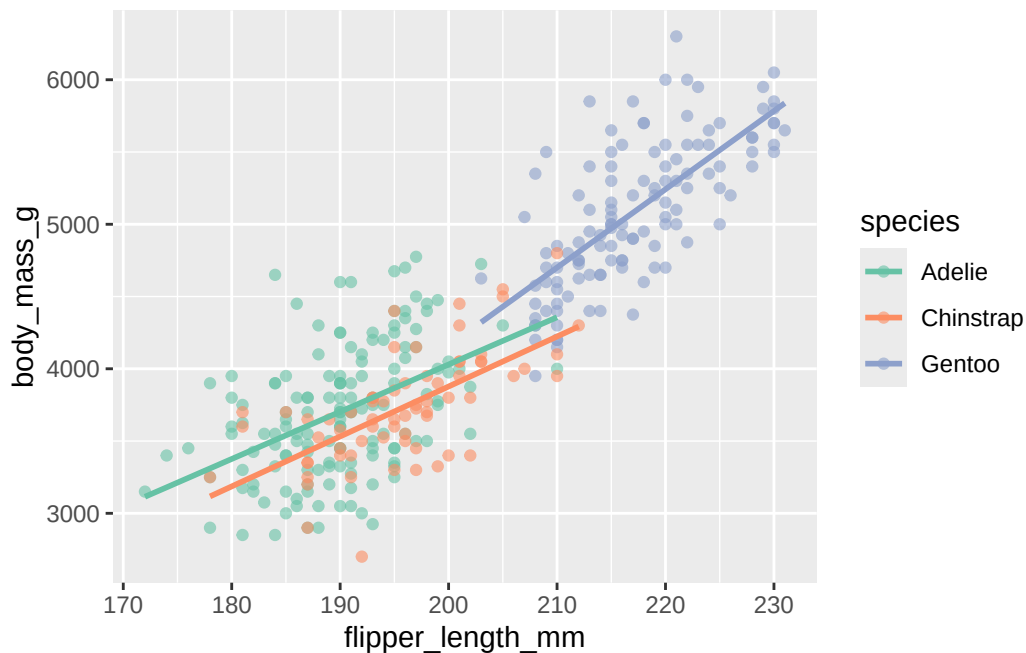


Figure 15.10.: A Brewer qualitative palette via `scale_color_brewer()` and tidy axis breaks.

In Figure 15.10 the colors come from the qualitative "Set2" palette (safe under color blindness, per Section 15.2), and `scale_y_continuous()` sets even gridline breaks. Scales are where you take control of *how* the mapping looks without changing *what* is mapped.

15.3.6. Step 6: facet into small multiples

Faceting splits one plot into a grid of panels, one per group — often clearer than cramming everything together. `facet_wrap(~ species)` gives each species its own panel:

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = FALSE) +
  scale_color_brewer(palette = "Set2") +
  facet_wrap(~ species)
```

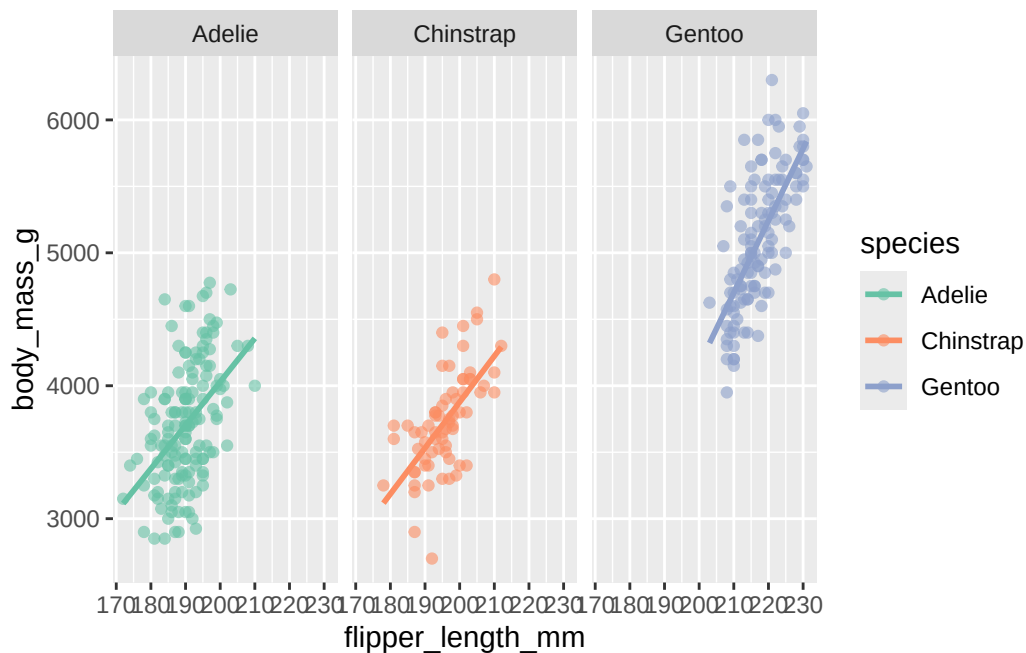


Figure 15.11.: Faceting by species puts each species in its own panel for side-by-side comparison.

Figure 15.11 makes the per-species spread easy to compare: Gentoo not only sit higher and to the right, they also span a wider range of body mass than the other two.

15.3.7. Step 7: labels and theme

Finally, the **theme** controls non-data styling, and `labs()` supplies the human context from Section 15.2. Putting it together:

```
ggplot(data = penguins,
       mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = FALSE) +
  scale_color_brewer(palette = "Set2") +
  facet_wrap(~ species) +
  labs(title = "Body mass increases with flipper length in all three species",
       x = "Flipper length (mm)",
       y = "Body mass (g)",
       color = "Species") +
  theme_minimal()
```

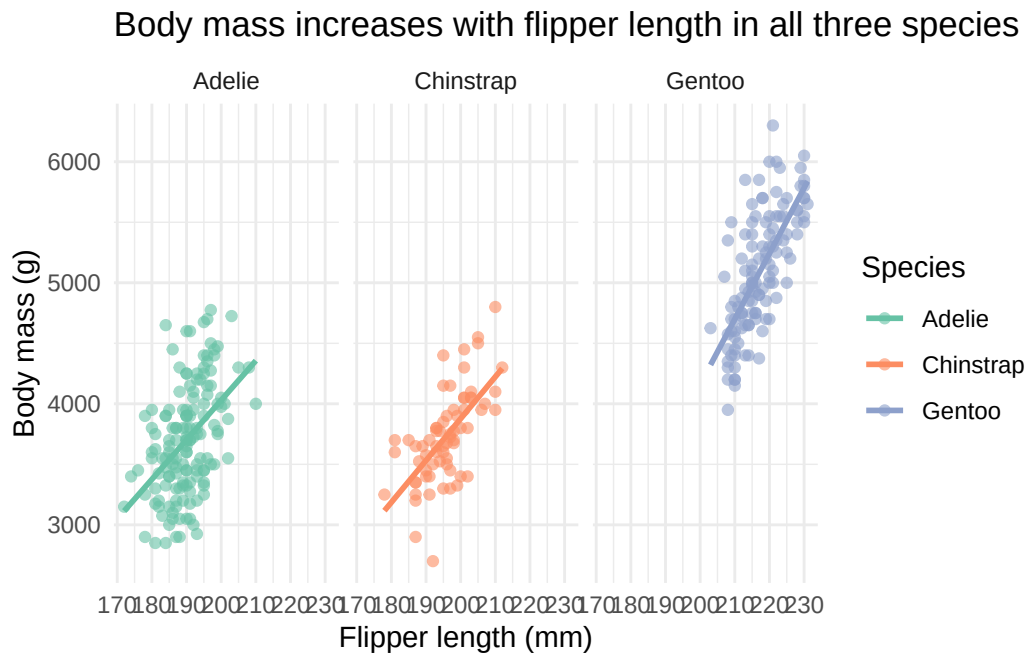


Figure 15.12.: The finished plot: data, aesthetics, two geoms, a scale, facets, labels, and a clean theme.

Figure 15.12 is publication-grade, and you built it one layer at a time. Read the call top

to bottom and you can *name* every part: data, the `aes()` mapping, two geoms, a color scale, facets, labels, and a minimal theme. That is the whole grammar of graphics — every `ggplot2` plot you ever write is some combination of these same pieces.

💡 Combining whole plots: `patchwork`

Faceting splits *one* plot by a variable. To place *different* plots side by side, the `patchwork` package lets you add them with `+` or stack them with `/`, e.g. `plot_a + plot_b`. It's the easiest way to assemble multi-panel figures.

15.4. Specialized plots for biology

`ggplot2` covers most everyday needs, but a few questions call for purpose-built plots. Here are two you'll meet often in genomics.

15.4.1. Sets and intersections: UpSet plots

Suppose you have several gene lists — genes up in treatment A, up in treatment B, bound by a transcription factor — and you want to know which genes are shared. The classic tool is a Venn diagram, but Venn diagrams become unreadable past three sets. An **UpSet plot** scales to many sets: a bar chart shows the size of each intersection, and a dot matrix beneath it shows *which* sets each bar combines.

`UpSetR` takes a data frame of 0s and 1s — one row per element, one column per set, a 1 marking membership. We'll use the `movies` dataset bundled with the package (already in that binary format) as a stand-in for “genes × gene-lists”:

```
library(UpSetR)

movies <- read.csv(system.file("extdata", "movies.csv", package = "UpSetR"),
                  header = TRUE, sep = ";")

upset(movies,
      nsets = 5,
      order.by = "freq",
      mainbar.y.label = "Intersection size",
      sets.x.label = "Total movies in genre")
```

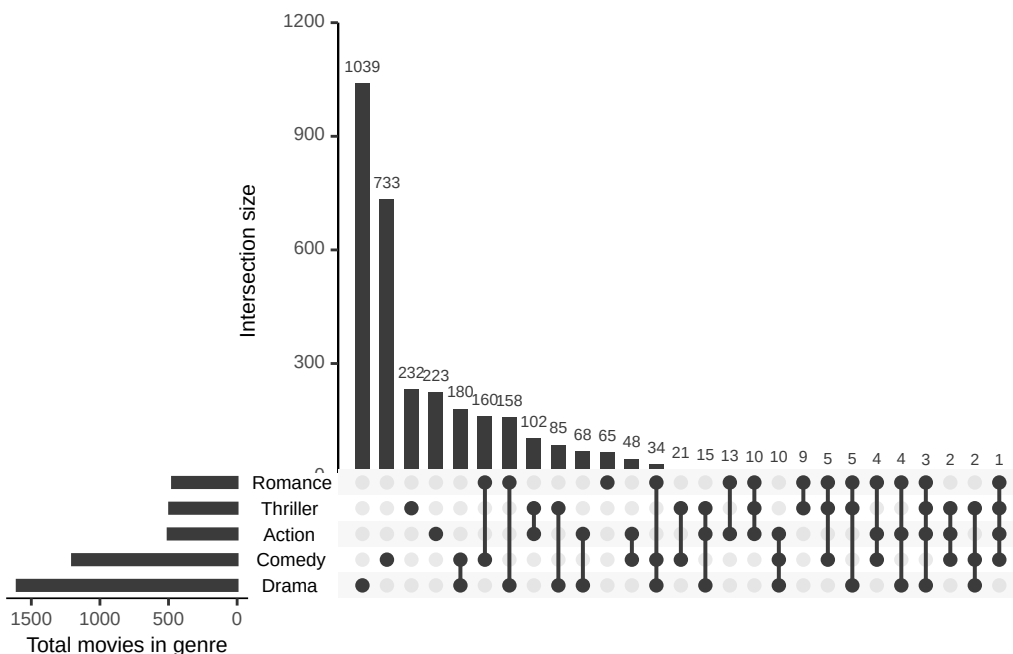


Figure 15.13.: An UpSet plot of movie genres. Each bar is the size of one intersection; the dots below say which genres it combines. Read it as a scalable Venn diagram.

Figure 15.13 reads left to right by decreasing intersection size. The tallest bars with a single filled dot are the “only this set” groups; bars over two or more filled dots are the overlaps. You can see at a glance, for example, how many movies are *both* Drama and Comedy versus Drama alone — a comparison that a five-circle Venn diagram simply cannot show clearly. Swap the movie genres for your gene lists and the reading is identical.

15.4.2. Heatmaps for data matrices

A **heatmap** turns a matrix of numbers into a grid of colored tiles, so the eye can pick out blocks and patterns that a table hides. In genomics, the rows are typically genes, the columns are samples, and the color is expression level. Heatmaps also usually *cluster* similar rows and columns together, so co-behaving genes and similar samples sit next to each other.

i What is a matrix?

A matrix is a two-dimensional grid of values addressed by row and column, and — unlike a data frame — every entry must be the *same* type (here, all numbers). Heatmap functions expect a numeric matrix, not a data frame.

Let’s make a runnable example with `pheatmap` (“pretty heatmap”). We’ll use the `mtcars` data as a numeric matrix — treat the cars as “samples” and the measurements as “features,” exactly as you would genes and samples. The catch: the columns are on wildly different scales (horsepower in the hundreds, number of cylinders in single digits), so we **scale each column** to comparable units first. Scaling is routine for expression data too.

```
library(pheatmap)

# build a numeric matrix and scale each column to mean 0, sd 1
car_matrix <- scale(as.matrix(mtcars))

pheatmap(car_matrix,
  main = "Scaled car measurements",
  color = colorRampPalette(rev(brewer.pal(7, "RdBu")))(100))
```

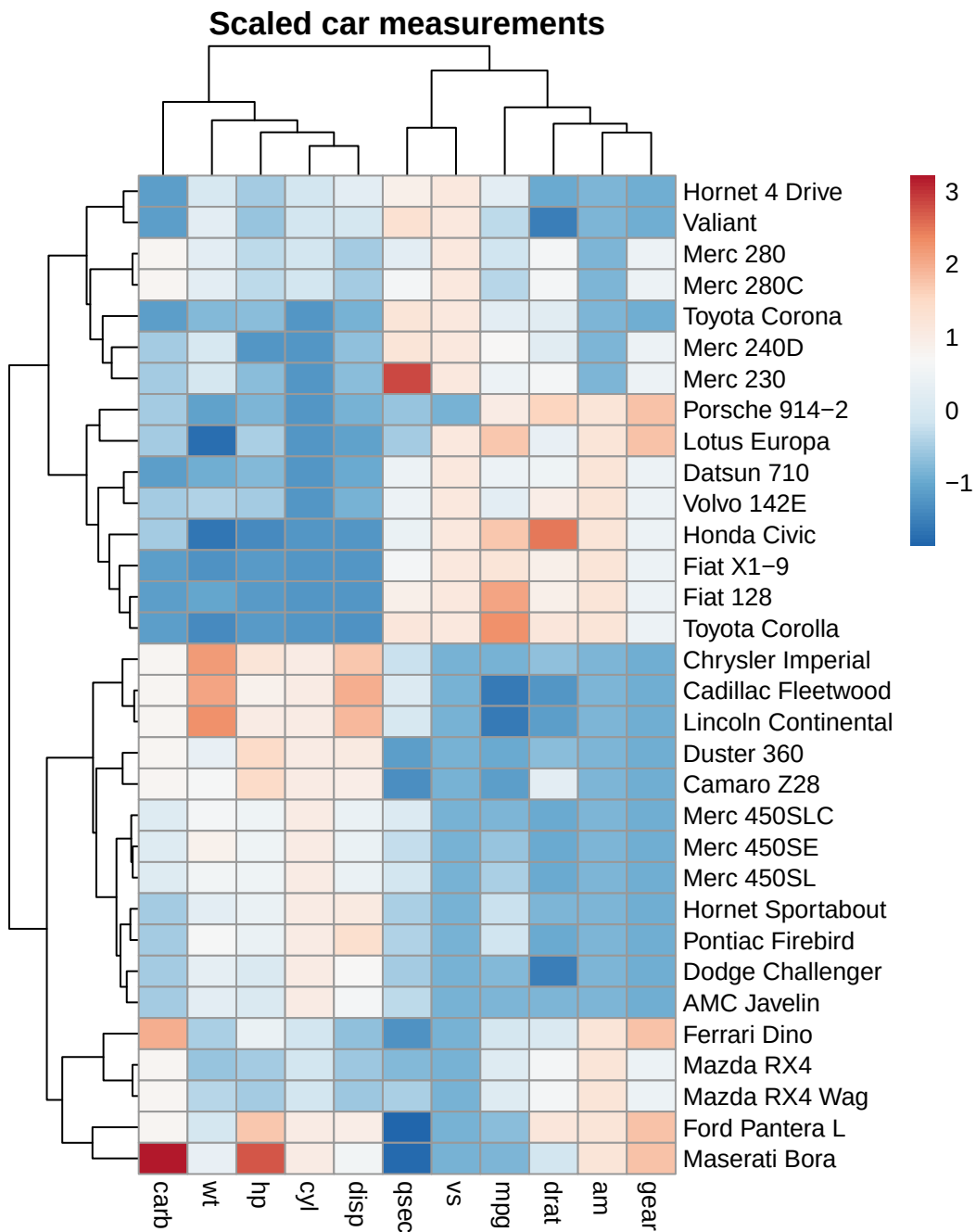


Figure 15.14.: A clustered heatmap of scaled car measurements. Rows (cars) and columns (features) are reordered so similar profiles sit together; the color shows how far above (red) or below (blue) average each value is.

Figure 15.14 clusters the rows and columns automatically. The tree diagrams (**dendrograms**) on the top and left show how cars and features were grouped. You can read off blocks: the fuel-efficiency feature (`mpg`) sits opposite the power/weight features, and the cars split into a heavy-and-powerful block versus a light-and-efficient one. Read genes for cars and samples for features, and this is exactly how an expression heatmap is interpreted.

For richer needs — multiple annotation bars, split heatmaps, precise control over legends — Bioconductor’s `ComplexHeatmap` is the standard. It uses the same mental model (a matrix plus optional row/column annotations) with far more knobs; its [reference book](#) is the place to go when `pheatmap` runs out of room.

15.4.3. Visualizing the genome (further exploration)

The plots above are general-purpose. Some genomic questions need a view organized along **genomic coordinates** — gene models, read coverage, and peaks lined up against a chromosome. This is a heavier topic that leans on Bioconductor, so we flag it as further exploration rather than cover it in depth here:

- The [Gviz package](#) builds publication-quality, multi-track genome browser views — stacking gene annotations, alignments, and quantitative data over a coordinate axis.
- The [GenomicDistributions package](#) summarizes *where* a set of genomic regions falls — for example, how ChIP-seq or ATAC-seq peaks distribute relative to transcription start sites.

When you reach a project that needs these, start from their vignettes; the grammar-of-graphics intuition from Section 15.3 still helps you read the layered tracks they produce.

15.5. Exercises

Use the cleaned `penguins` data frame from Section 15.3 for the first three exercises (`library(palmerpenguins)`
`library(ggplot2); penguins <- na.omit(penguins)`).

1. **A different geom.** Make a scatterplot of bill length (`bill_length_mm`, x) versus bill depth (`bill_depth_mm`, y), colored by species. What do you notice?

15. A self-guided tour of data visualization in R

i Solution

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm, color = species)) +  
  geom_point() +  
  theme_minimal()
```

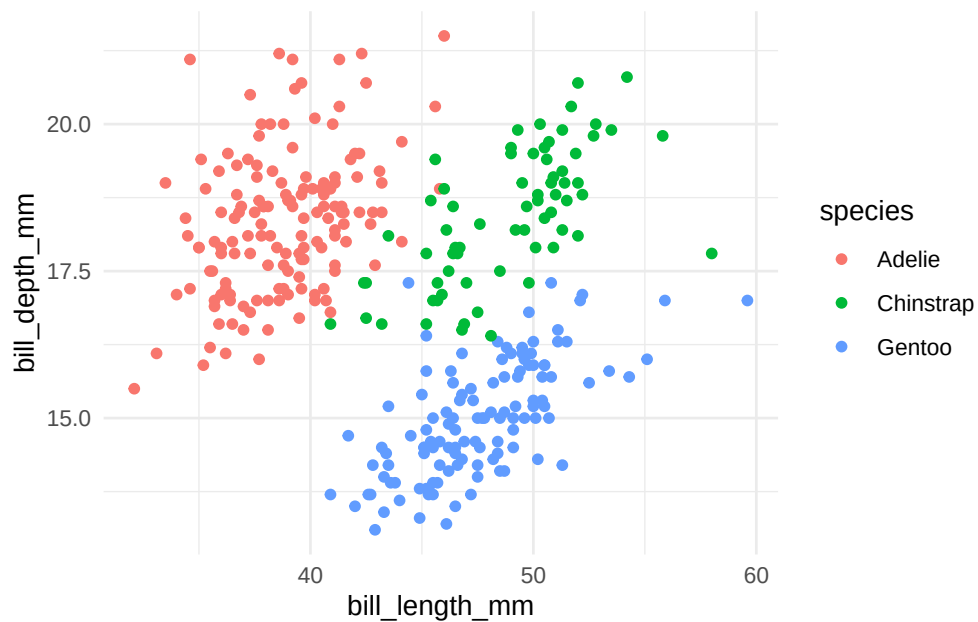


Figure 15.15.: Bill length vs. depth, colored by species.

Within each species, deeper bills go with longer bills, but the three species sit in clearly separated regions — bill shape is a good species fingerprint.

2. **A distribution, not a relationship.** Use `geom_boxplot()` to compare body mass across the three species (species on x, `body_mass_g` on y). Which species is heaviest?

i Solution

```
ggplot(penguins, aes(x = species, y = body_mass_g, fill = species)) +  
  geom_boxplot() +  
  theme_minimal()
```

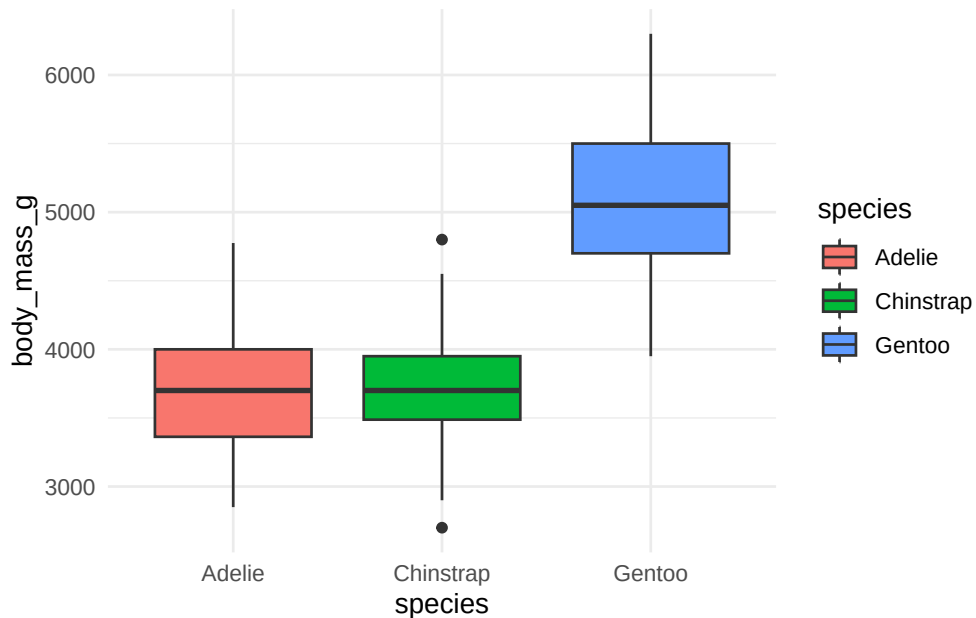



Figure 15.16.: Body-mass distribution by species.

Gentoo penguins are clearly the heaviest; Adelie and Chinstrap have similar, lower body masses. A boxplot summarizes a *distribution* (median, spread, outliers) rather than a point-to-point relationship.

3. **Add the labels.** Take your boxplot from Exercise 2 and give it a title and properly-labeled axes with units, following Section 15.2.

i Solution

```
ggplot(penguins, aes(x = species, y = body_mass_g, fill = species)) +
  geom_boxplot() +
  labs(title = "Gentoo penguins are the heaviest of the three species",
       x = "Species",
       y = "Body mass (g)",
       fill = "Species") +
  theme_minimal()
```

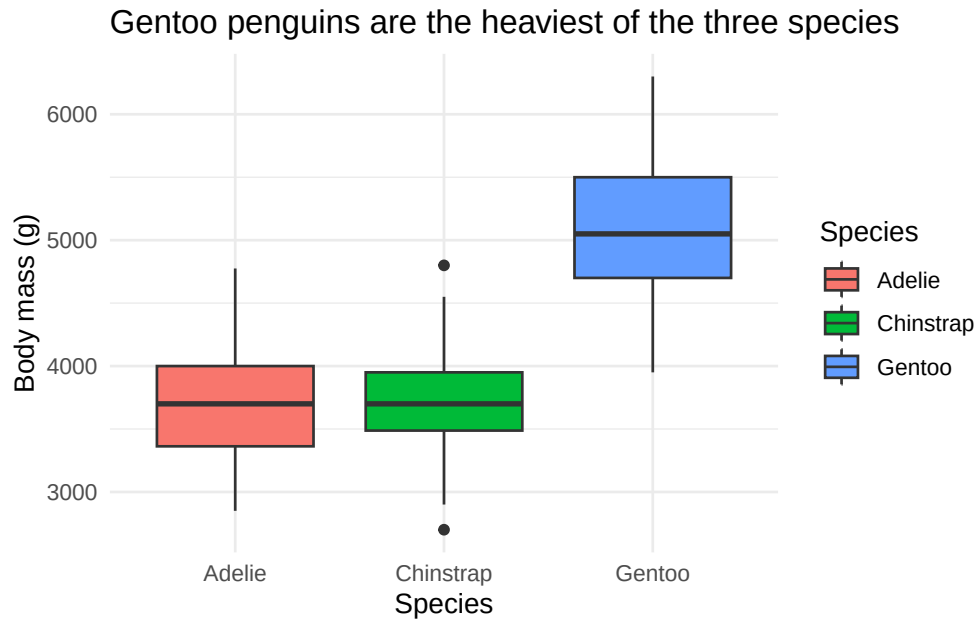


Figure 15.17.: The boxplot, now properly labeled.

The title states the *finding*, not just the variables — a good title does the reader's work for them.

4. **Pick the right palette family.** You're plotting log-fold-changes that range from strongly negative to strongly positive, with zero as the meaningful midpoint. Which RColorBrewer palette *family* should you use, and why?

i Solution

A **diverging** palette (such as "RdBu"). Diverging palettes give two contrasting colors that meet at a neutral midpoint, so zero reads as “no change” and the two directions of change are immediately distinguishable. A sequential palette would hide the sign of the change.

15.6. Summary

You now have both the principles and the toolbox for visualizing data in R:

15. A self-guided tour of data visualization in R

- You can make any plot more readable by **maximizing the data-ink ratio** (deleting decoration) and **labeling everything** with a clear title and units.
- You can read and write a `ggplot2` call as a stack of **layers** — data, aesthetics, geoms, scales, facets, themes — and you built one up one layer at a time on real penguin data.
- You can map extra variables onto **color** and other aesthetics, and you know to put data-driven settings *inside* `aes()` and constants *outside* it.
- You can choose a **colorblind-safe palette** from `RColorBrewer`.
- You can reach for a specialized plot when you need one: an **UpSet plot** for intersections of many sets, and a clustered **heatmap** for an expression-style matrix.

The grammar of graphics is the throughline: once a plot is just data plus a stack of layers, the way to build *any* new plot is to ask which layer you need to add next. For more depth on a single dataset, see [Exploring data with ggplot2](#); for inspiration, browse the [R Graph Gallery](#) and [From Data to Viz](#).

Part IV.
statistics

16. Working with distribution functions

Heights, blood pressures, measurement errors, log-transformed gene-expression values — measure enough of almost anything in biology and the values tend to pile up into the same bell-shaped curve: the **normal distribution**. Once your data follow that curve, you'll want to ask questions of it. *What fraction of samples fall below some cutoff? Which value marks the top 5%? Give me twenty random draws to simulate an experiment.* R answers each of those with a different function.

The catch is remembering which function does which job. This chapter is a **visual guide**: for every function we pair the R call with a picture of exactly what it measures on the bell curve, because the difference between a *probability*, a *density*, and a *quantile* is far easier to see than to memorize. It's genuinely hard to reason about these functions without a shape to point at — so a shape is what we'll build for each one.

16.1. What you'll learn

- Use `pnorm()` to turn a value into a probability (the area under the curve to its left).
- Use `dnorm()` to get the height (density) of the curve at a value.
- Use `qnorm()` to go the other direction — from a probability back to a value.
- Use `rnorm()` to draw random samples from a normal distribution.
- Combine these functions to answer real questions about normally distributed data, such as IQ scores.

💡 One naming rule for *every* distribution

R builds these names from a one-letter prefix plus the distribution: **d** for density, **p** for probability, **q** for quantile, **r** for random. Learn it once and it transfers everywhere — `dbinom/pbinom/qbinom/rbinom` for the binomial, `dt/pt/qt/rt` for the t-distribution we meet in the next chapter, `dpois/ppois/...` for the Poisson. Here we use the `...norm` family for the normal.

The four functions, at a glance:

16. Working with distribution functions

Table 16.1.: Functions for the normal distribution

Function	Input	Output
<code>pnorm</code>	a value <code>q</code>	$P(X < q)$: area under the curve to the left of <code>q</code>
<code>dnorm</code>	a value <code>x</code>	the height of the density curve at <code>x</code>
<code>qnorm</code>	a probability from 0 to 1	the value <code>x</code> where $P(X < x)$ equals that probability
<code>rnorm</code>	a count <code>n</code>	<code>n</code> random samples from the distribution

In one sentence each: `pnorm` gives the area to the left of a point, `dnorm` gives the height of the curve at a point, `qnorm` is the inverse of `pnorm` (probability back to value), and `rnorm` generates random samples. The rest of the chapter unpacks them one at a time.

16.2. `pnorm`

`pnorm` answers the question “what fraction of the distribution lies to the **left** of this value?” If you don’t supply a mean and standard deviation, R assumes the *standard normal* (mean 0, standard deviation 1). The help page gives this template:

```
pnorm(q, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)
```

You hand it `q`, a value on the x-axis, and it returns the area under the curve to the left of that point. The mean sits dead centre and splits the curve in half, so:

```
pnorm(0)      # half the distribution is below the mean
```

```
[1] 0.5
```

```
pnorm(1)      # ~84% is below one standard deviation above the mean
```

```
[1] 0.8413447
```

16. Working with distribution functions

```
pnorm(-1) # ~16% is below one standard deviation below it
```

```
[1] 0.1586553
```

The figures below show each of those as a shaded area.

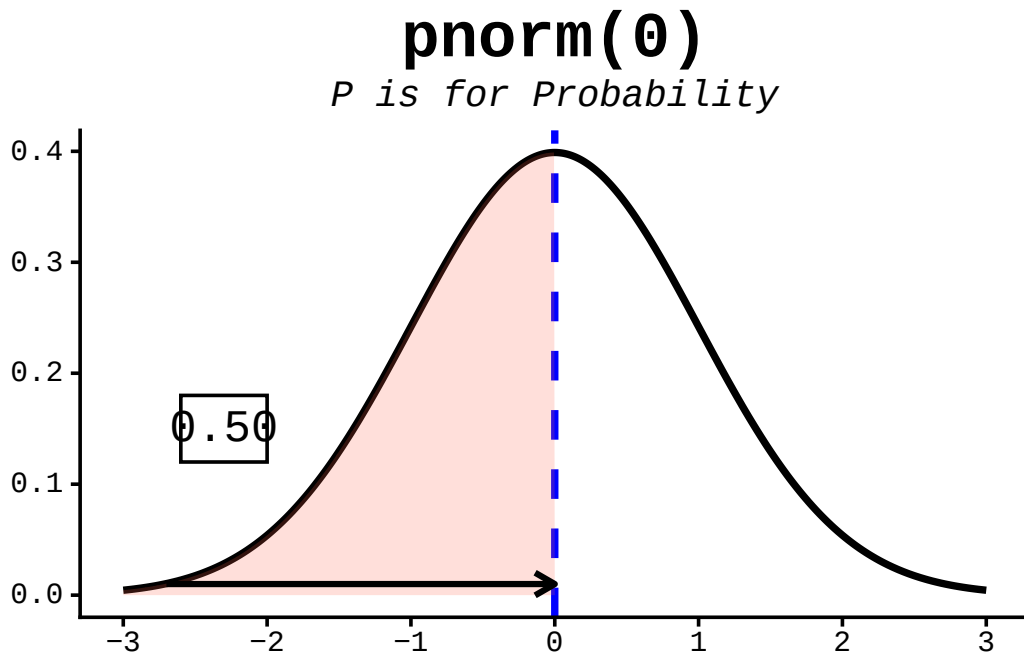


Figure 16.1.: The `pnorm` function takes a quantile (value on the x-axis) and returns the area under the curve to the left of that value.

The option `lower.tail = TRUE` tells R to use the area to the *left* of the given point — the default. To get the area to the **right** instead, either switch to `lower.tail = FALSE` or subtract from 1:

```
pnorm(1, lower.tail = FALSE) # area to the RIGHT of 1
```

```
[1] 0.1586553
```

```
1 - pnorm(1) # exactly the same thing
```

```
[1] 0.1586553
```

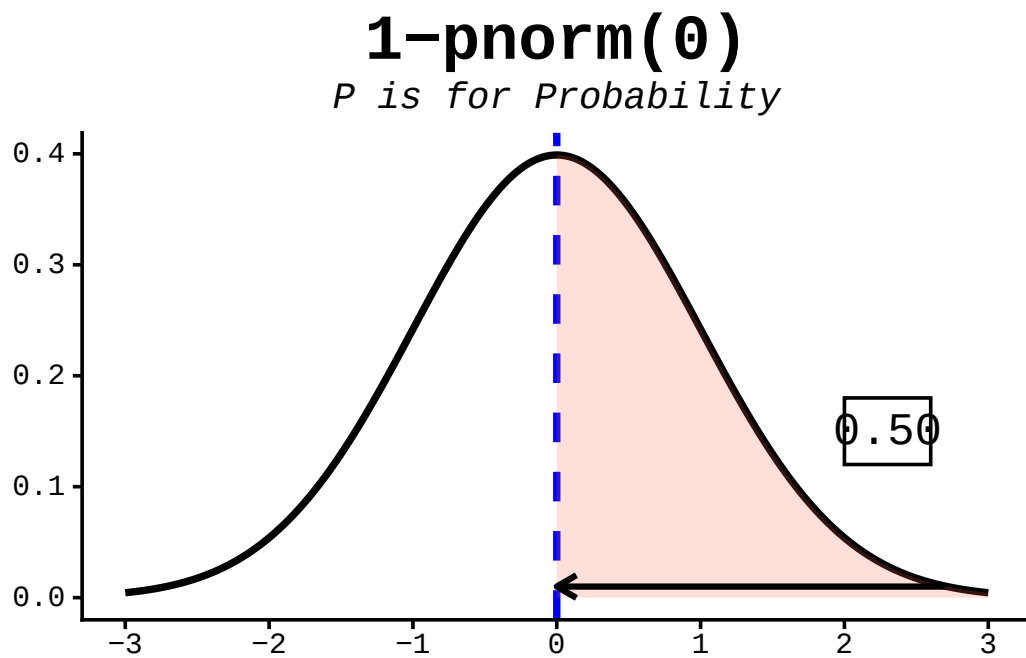


Figure 16.2.: The pnorm function takes a quantile (value on the x-axis) and returns the area under the curve to the left of that value.

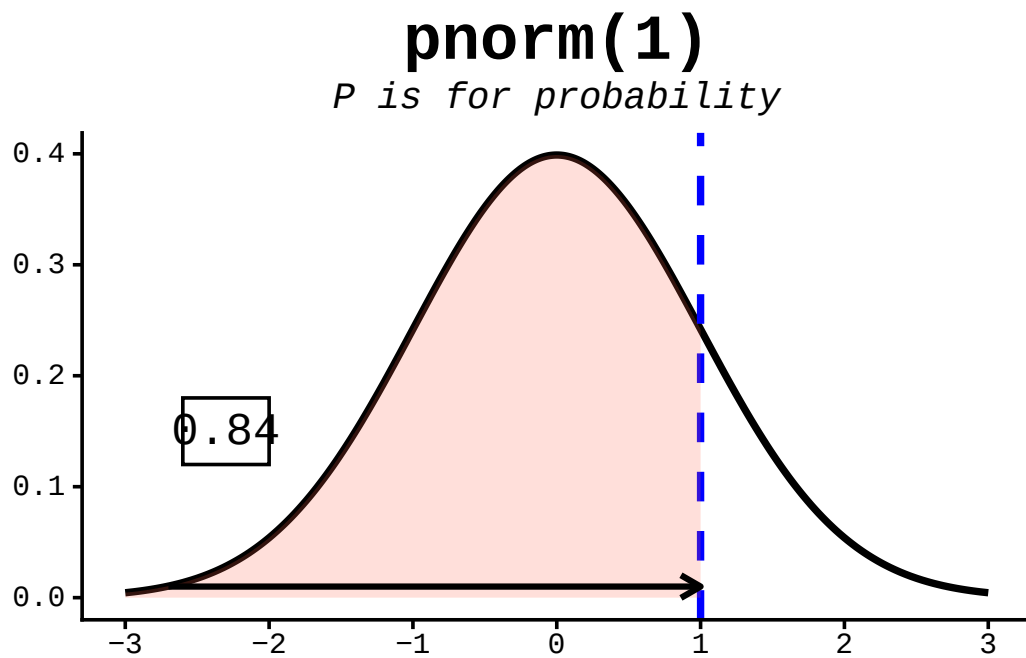


Figure 16.3.: The pnorm function takes a quantile (value on the x-axis) and returns the area under the curve to the left of that value.

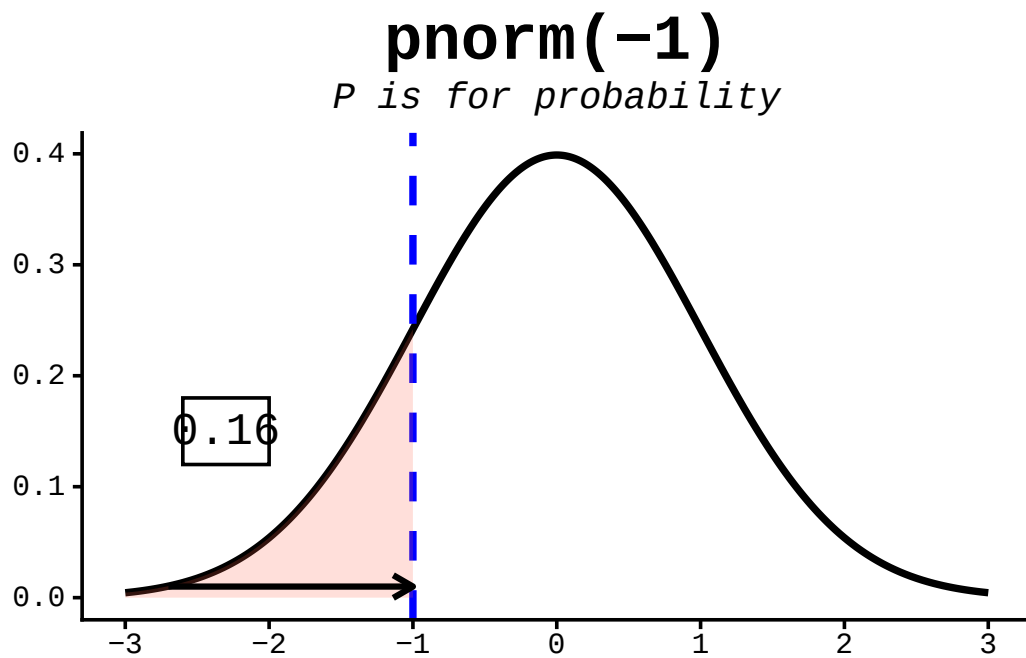


Figure 16.4.: The pnorm function takes a quantile (value on the x-axis) and returns the area under the curve to the left of that value.

16.3. dnorm

`dnorm` gives the **height** of the bell curve at a value — the probability *density*. The curve is tallest at the mean and falls off symmetrically on either side. Unlike `pnorm`, the result is not a probability you can read as a percentage; it's the height of the curve, used mainly for *drawing* the distribution or in likelihood calculations.

```
dnorm(0)      # the peak of the standard normal
```

```
[1] 0.3989423
```

```
dnorm(1)      # lower, one SD out...
```

```
[1] 0.2419707
```

```
dnorm(-1)     # ...and the same height on the other side, by symmetry
```

```
[1] 0.2419707
```

The figures below mark each of these heights on the curve.

16.4. qnorm

`qnorm` runs `pnorm` in reverse: hand it a probability and it returns the value with that much area to its left. Since `pnorm(1)` is about 0.84, `qnorm(0.84)` is about 1 — the two functions undo each other. This is how you find **percentiles**.

```
qnorm(0.5)     # the median: half the area lies to its left
```

```
[1] 0
```

```
qnorm(0.25)    # the first quartile
```

```
[1] -0.6744898
```

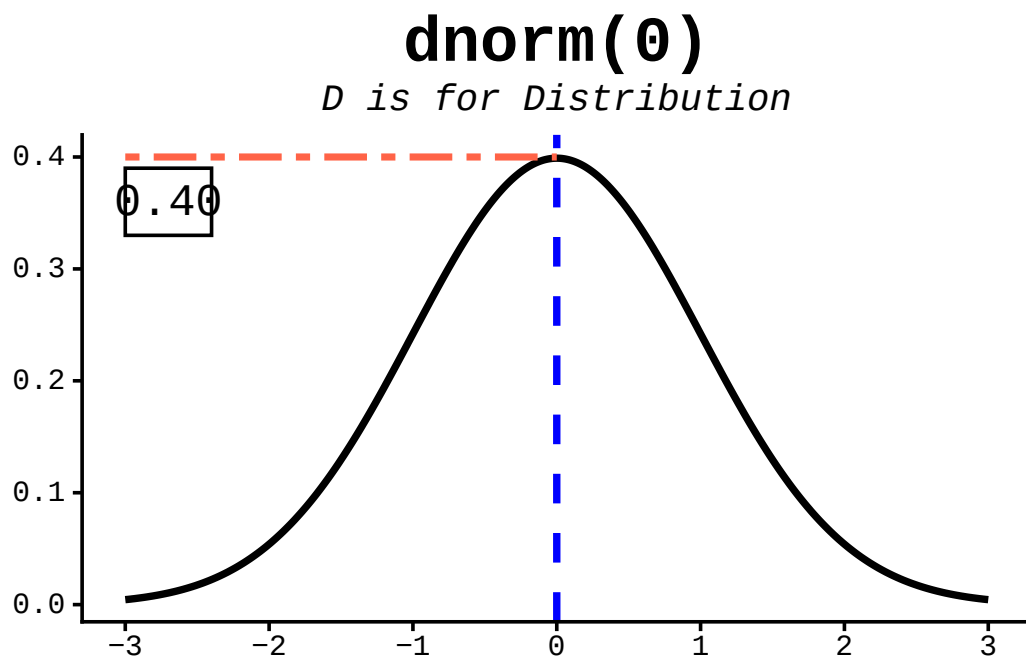


Figure 16.5.: The `dnorm` function returns the height of the normal distribution at a given point.

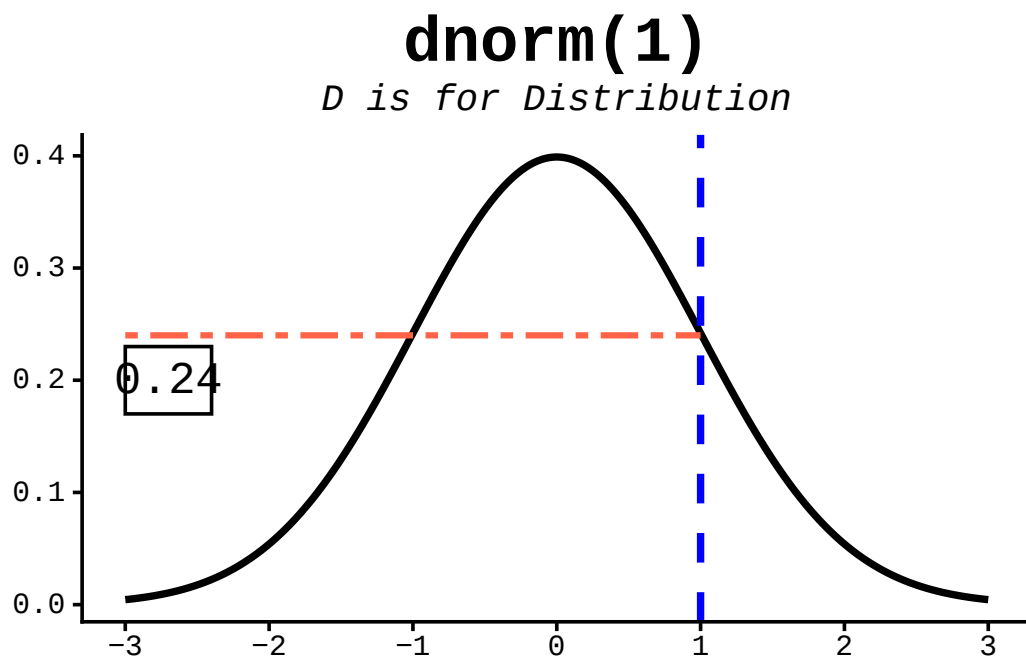


Figure 16.6.: The `dnorm` function returns the height of the normal distribution at a given point.

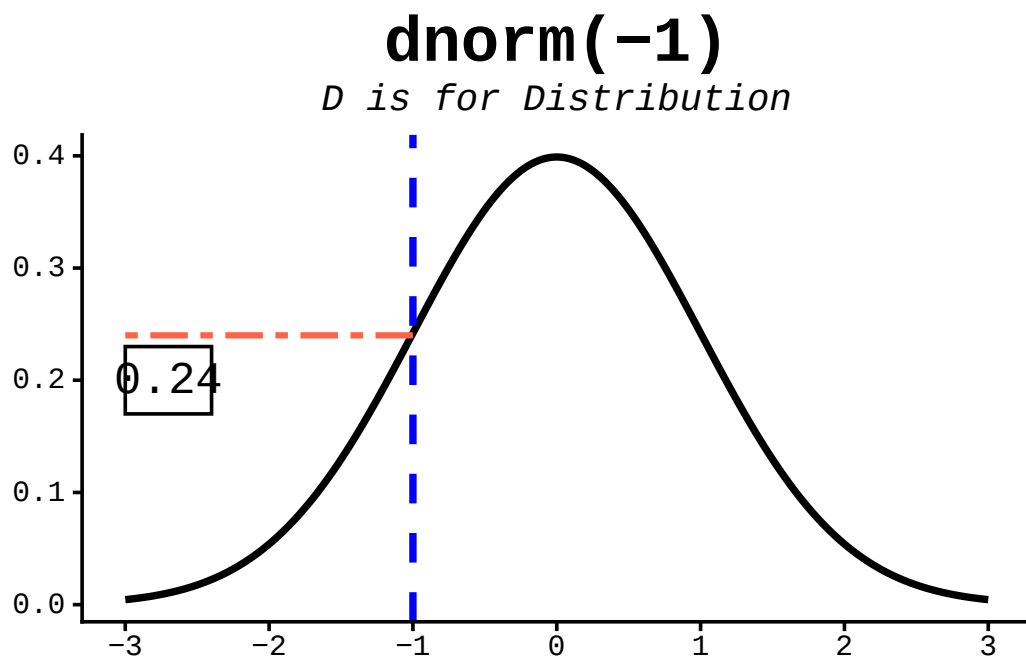


Figure 16.7.: The `dnorm` function returns the height of the normal distribution at a given point.

16. Working with distribution functions

```
qnorm(0.975) # the value cutting off the top 2.5% - the famous 1.96
```

```
[1] 1.959964
```

The figures below show several quantiles as shaded areas.

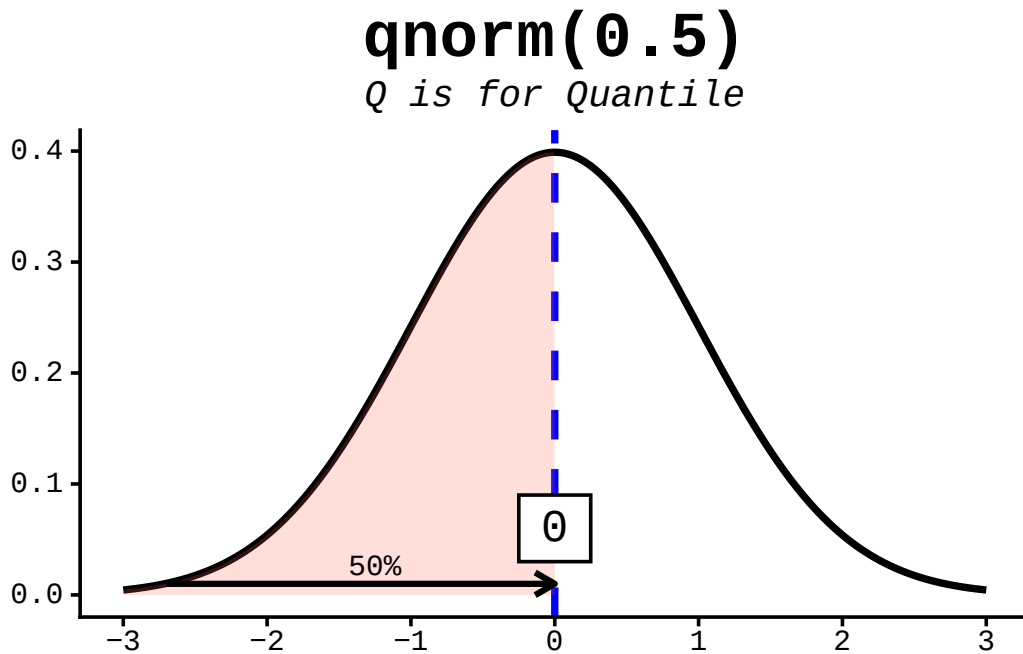


Figure 16.8.: The `qnorm` function is the inverse of the `pnorm` function in that it takes a probability and gives the quantile.

16.5. `rnorm`

`rnorm` draws random samples from the distribution — the workhorse for *simulating* data (we leaned on it heavily in the t-distribution chapter). Give it a count `n` and it hands back that many random values:

```
rnorm(5)
```

```
[1] -0.01976095 -0.13131999  0.42682316  1.49509130  0.46830778
```

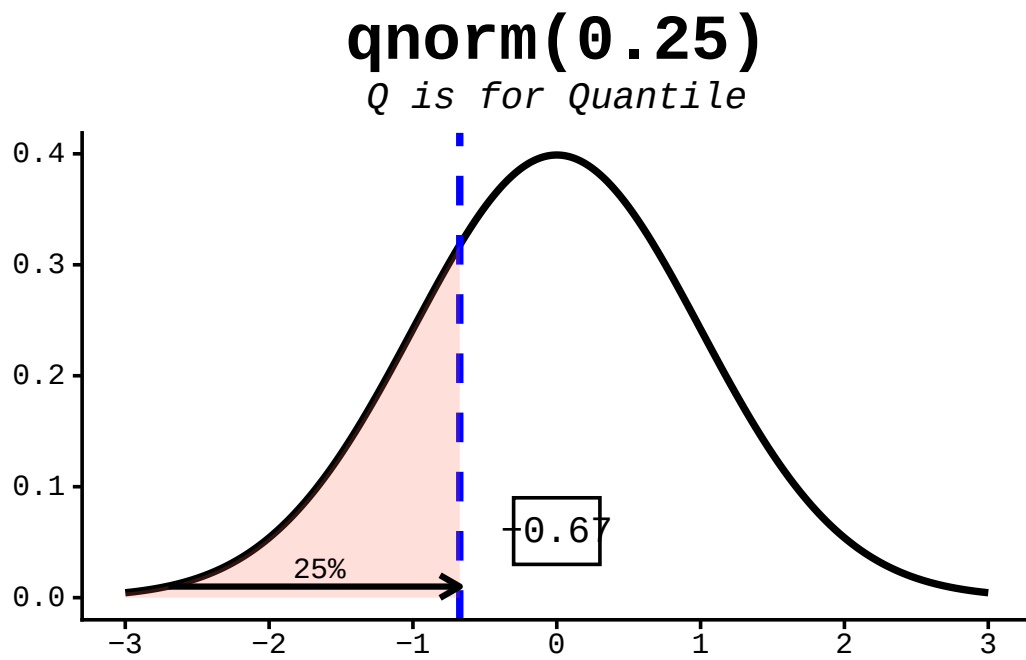


Figure 16.9.: The qnorm function is the inverse of the pnorm function in that it takes a probability and gives the quantile.

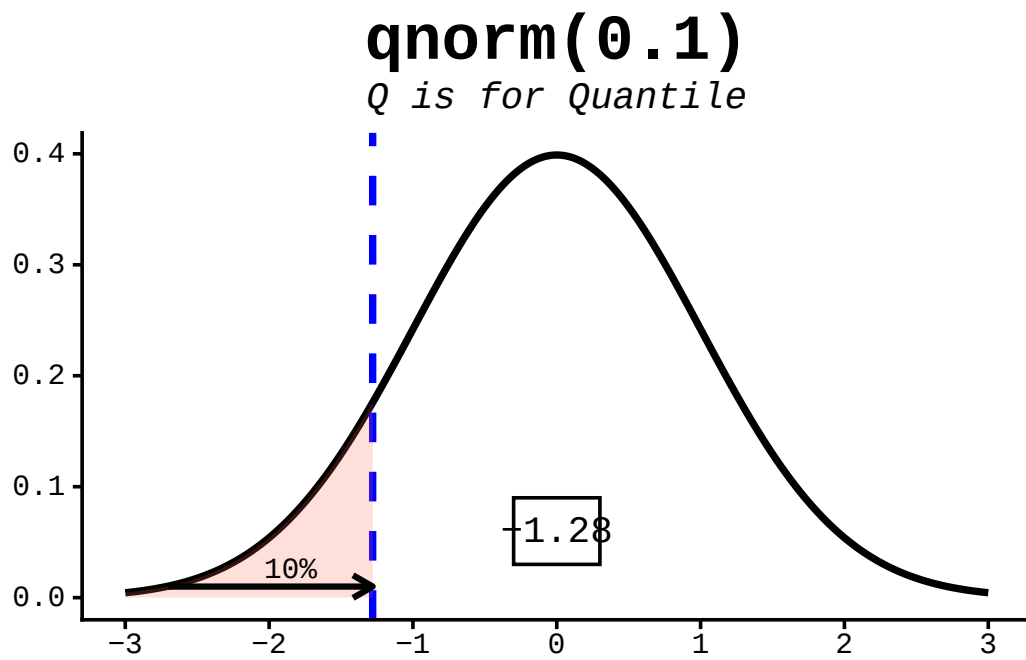


Figure 16.10.: The qnorm function is the inverse of the pnorm function in that it takes a probability and gives the quantile.

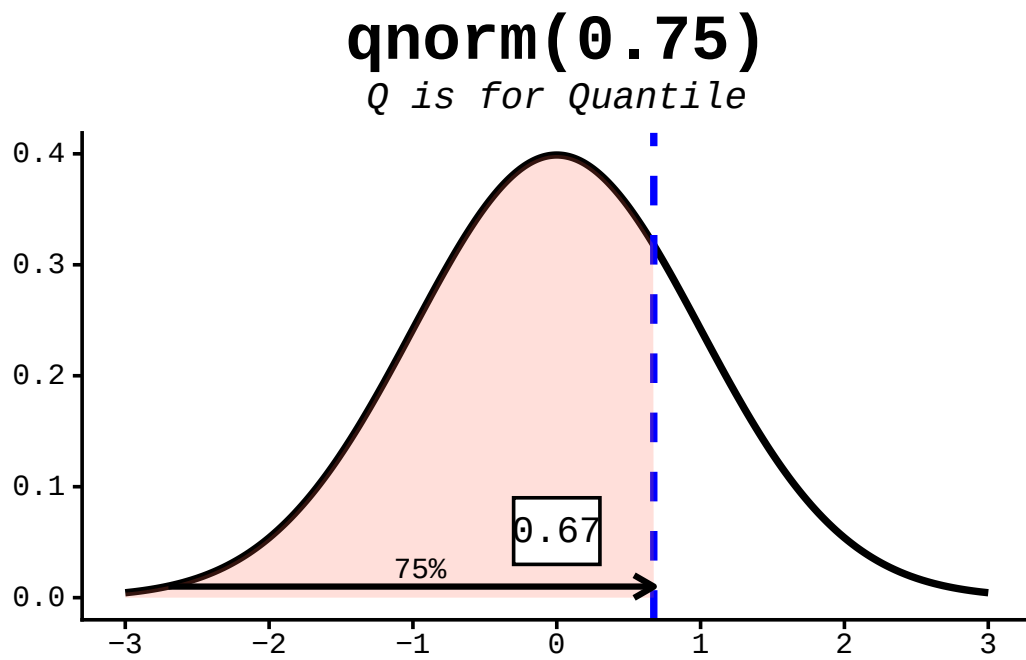


Figure 16.11.: The qnorm function is the inverse of the pnorm function in that it takes a probability and gives the quantile.

16. Working with distribution functions

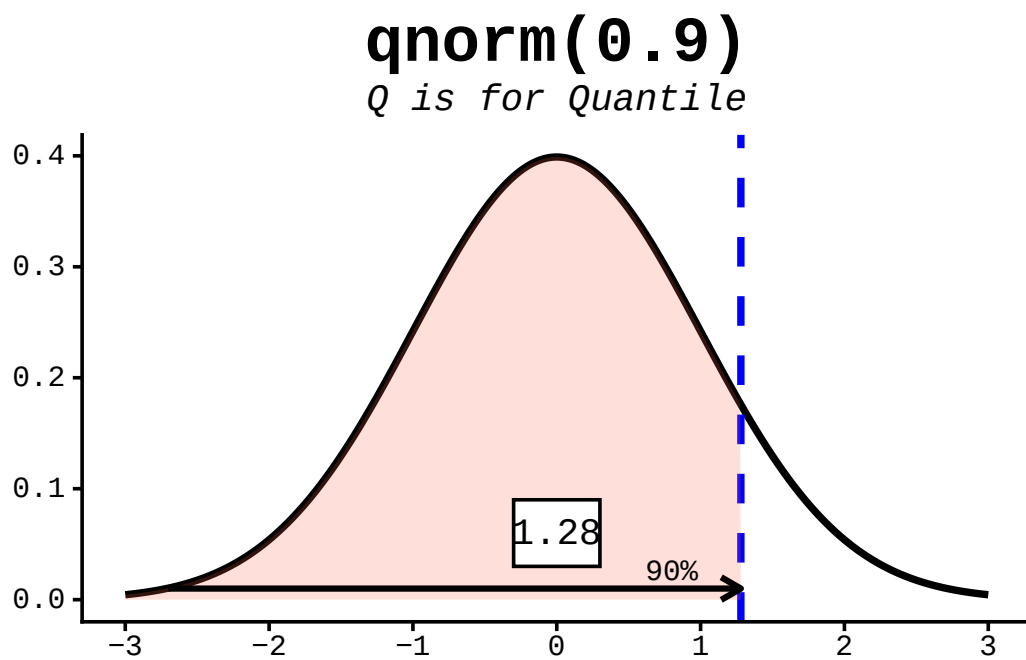


Figure 16.12.: The qnorm function is the inverse of the pnorm function in that it takes a probability and gives the quantile.

16. Working with distribution functions

Run that again and you'll get five *different* numbers. To make a simulation reproducible — so you and a reader see the same “random” draws — set a seed first:

```
set.seed(42)
rnorm(5)
```

```
[1]  1.3709584 -0.5646982  0.3631284  0.6328626  0.4042683
```

And just like the other functions, `mean` and `sd` shift and scale the draws:

```
rnorm(5, mean = 100, sd = 15) # e.g. five simulated IQ scores
```

```
[1]  98.40813 122.67283  98.58011 130.27636  99.05929
```

```
print(r1)
```

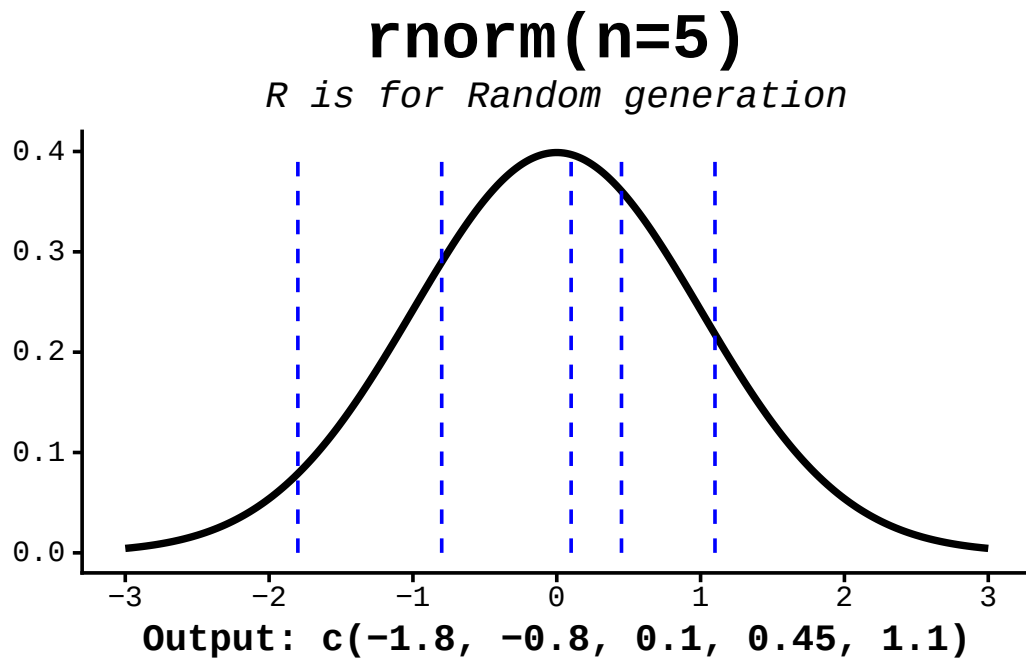


Figure 16.13.: The `rnorm` function takes a number of samples and returns a vector of random numbers from the normal distribution (with `mean=0`, `sd=1` as defaults)

16.6. Worked example: IQ scores

The normal distribution (also called the Gaussian distribution) is defined by two numbers: the **mean** (μ), which sets the centre, and the **standard deviation** (σ), which sets the spread. IQ scores are deliberately built to be normal with a **mean of 100** and a **standard deviation of 15**, which makes them a perfect playground for the four functions.

A handy rule of thumb — the “68–95–99.7 rule” — says about 68% of values fall within one standard deviation of the mean (here, 85 to 115) and about 95% within two (70 to 130). Let’s check the 68% with `pnorm`. The area *between* 85 and 115 is the area below 115 minus the area below 85:

```
pnorm(115, mean = 100, sd = 15) - pnorm(85, mean = 100, sd = 15)
```

```
[1] 0.6826895
```

About 0.68 — exactly what the rule promises. Notice the pattern: a “between” question becomes one `pnorm` minus another.

16.7. Exercises

Use the IQ distribution (mean 100, sd 15) for each. Each solution names *which* of the four functions fits the question — that choice is the real skill.

1. **Top 10%.** Which IQ score marks the 90th percentile — the cutoff above which only the top 10% fall?

i Solution

```
qnorm(0.9, mean = 100, sd = 15)
```

```
[1] 119.2233
```

We want a *value* from a *probability*, so this is a `qnorm` job. About 119.

2. **Above 130.** What fraction of people have an IQ above 130 (two standard deviations up)?

16. Working with distribution functions

i Solution

```
1 - pnorm(130, mean = 100, sd = 15)
```

```
[1] 0.02275013
```

```
# or, equivalently:
```

```
pnorm(130, mean = 100, sd = 15, lower.tail = FALSE)
```

```
[1] 0.02275013
```

About 2.3% — consistent with the 95% rule leaving roughly 2.5% in each tail.

3. **Below 70.** What fraction have an IQ below 70?

i Solution

```
pnorm(70, mean = 100, sd = 15)
```

```
[1] 0.02275013
```

Also about 2.3%. The curve is symmetric, so the lower tail past 70 mirrors the upper tail past 130.

4. **Simulate a classroom.** Draw 30 random IQ scores and count how many land above 115. (Set a seed so your answer is reproducible.)

i Solution

```
set.seed(1)
scores <- rnorm(30, mean = 100, sd = 15)
sum(scores > 115)
```

```
[1] 3
```

Theory predicts about 16% above 115 — roughly 5 of 30 — but any single small sample wobbles around that. Re-run with a different seed to feel the sampling variability for yourself.

16.8. Summary

You now have the four-function toolkit for the normal distribution — and, thanks to R's naming rule, the blueprint for every other distribution too:

16. Working with distribution functions

- `pnorm(x)` — area to the **left** of `x` (a probability). Use `lower.tail = FALSE` or `1 - pnorm(x)` for the right tail, and one `pnorm` minus another for a range.
- `dnorm(x)` — the **height** of the curve at `x` (a density).
- `qnorm(p)` — the **value** at probability `p` (the inverse of `pnorm`); your tool for percentiles.
- `rnorm(n)` — `n` random **draws**; pair it with `set.seed()` for reproducible simulations.

Keep the pictures in mind: `pnorm` is an area, `dnorm` is a height, `qnorm` reads the area axis backwards to a value, and `rnorm` scatters points along the curve.

17. The t-statistic and t-distribution

Suppose you measure a gene's expression in five treated samples and five controls, and the treated mean comes out higher. Is that a real effect, or just the kind of gap you'd expect by chance from five noisy measurements? With only a handful of samples you can't lean on the normal distribution as-is, because you had to *estimate* the variability from the same tiny sample you're testing. The **t-distribution** is the honest accounting for that extra uncertainty — and in this chapter we'll *build* it by simulation so you can see exactly where it comes from, then use R's `t.test()` to put it to work.

The distribution that the t-statistic follows was described in a famous 1908 paper (Student 1908) by “Student” — a pseudonym for [William Sealy Gosset](#), who worked at the Guinness brewery and published anonymously because his employer treated statistical methods as a trade secret.

17.1. What you'll learn

- Explain why estimating the standard deviation from a small sample forces us off the normal distribution and onto the t-distribution.
- Read a Z-score and a t-score and say what each one measures.
- Use simulation to show that t-based p-values are calibrated (uniform under the null hypothesis) when the normal-based ones are not.
- Run one-sample, two-sample, and formula-based tests with `t.test()`.
- Estimate statistical power with `power.t.test()` and by simulation.

17.2. Background

A **t-test** is a [statistical hypothesis test](#) you reach for when you want to compare means and you only have a sample to work from. It comes in three everyday flavors:

- **One-sample** — compare a sample mean to a single hypothesized or target value.
- **Two-sample** — compare the means of two groups.

17. The *t*-statistic and *t*-distribution

- **Paired** — compare two measurements on the same units (e.g., before-and-after on the same patients).

A *t*-test combines three ingredients — the **t-statistic**, the **t-distribution**, and the **degrees of freedom** — to decide whether a difference is bigger than chance would comfortably produce. (To compare three or more means at once you’d reach instead for an analysis of variance, or ANOVA.)

We’ll get to `t.test()`, but the point of this chapter is to first see *why* the *t*-distribution exists. That story starts with something you may already have met: the Z-score.

17.3. The Z-score and probability

Before we talk about *t*-scores, let’s review the Z-score, its relationship to the normal distribution, and how it turns into a probability.

The Z-score is defined as:

$$Z = \frac{x - \mu}{\sigma} \quad (17.1)$$

where μ is the population mean that x is drawn from and σ is the population standard deviation — **taken as known**, not estimated from the data. In words: a Z-score says how many standard deviations a value sits from the mean.

The probability of observing a Z-score of z or smaller can be calculated with `pnorm(z, mu, sigma)`.

For example, suppose our “population” is known and truly has a mean of 0 and a standard deviation of 1 (the *standard normal*). For any observation drawn from that population, we can assign a probability of seeing a value that extreme by random chance — *under the assumption that the null hypothesis is **TRUE***.

```
zscore <- seq(-5, 5, 1)
```

For each value of `zscore`, let’s calculate the probability and collect the results in a `data.frame`. We’ll name it `pdat` (for “probability data”) so it doesn’t collide with the `df` we use later for degrees of freedom.

17. The *t*-statistic and *t*-distribution

```
pdat <- data.frame(  
  zscore = zscore,  
  pval   = pnorm(zscore, 0, 1)  
)  
pdat
```

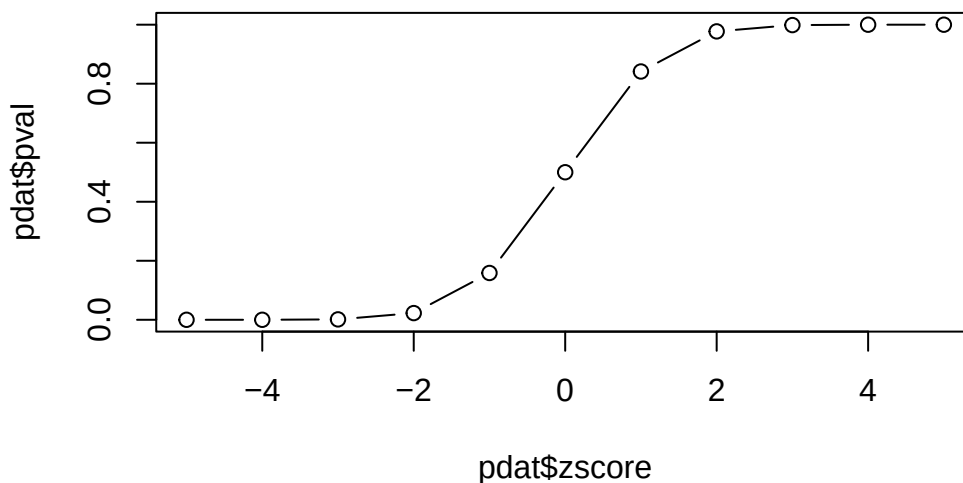
	zscore	pval
1	-5	2.866516e-07
2	-4	3.167124e-05
3	-3	1.349898e-03
4	-2	2.275013e-02
5	-1	1.586553e-01
6	0	5.000000e-01
7	1	8.413447e-01
8	2	9.772499e-01
9	3	9.986501e-01
10	4	9.999683e-01
11	5	9.999997e-01

Why is the p-value for something 5 standard deviations *above* the mean (`zscore = 5`) nearly 1 here? Because `pnorm` reports the *lower* tail by default — the probability of a value *at or below* *z*. A value 5 standard deviations above the mean has almost the entire distribution below it, so that probability is nearly 1.

Let's plot probability against Z-score:

```
plot(pdat$zscore, pdat$pval, type = 'b')
```

17. The *t*-statistic and *t*-distribution



This curve is the *cumulative distribution function* (CDF). Read it like a lookup table: pick a Z-score on the x-axis and read off the probability of seeing a value that small. Because we built our example to follow the standard normal, this is also the CDF of the standard normal distribution.

💡 What a p-value is (and isn't)

A p-value is the probability of a result *at least this extreme* **if the null hypothesis were true**. It is *not* the probability that the null hypothesis is true, and a small p-value is not a measure of effect size. For an accessible, authoritative take on the common misreadings, see the American Statistical Association's statement on p-values (Wasserstein and Lazar 2016).

17.3.1. Small diversion: a two-sided `pnorm`

`pnorm` returns a *one-sided* probability using the lower tail by default. Often we don't care which direction a value deviates — only that it's far from the mean — so we want a *two-sided* p-value. We can build one in three steps:

1. Take the absolute value of `x`, so direction no longer matters.
2. Call `pnorm` with `lower.tail = FALSE`, so larger `|x|` gives smaller probabilities.

17. The *t*-statistic and *t*-distribution

3. Multiply by 2 to count *both* tails.

```
twosidedpnorm <- function(x, mu = 0, sd = 1) {  
  2 * pnorm(abs(x), mu, sd, lower.tail = FALSE)  
}
```

Now we can ask how surprising it is to land 2 or 3 standard deviations from the mean in either direction:

```
twosidedpnorm(2)
```

```
[1] 0.04550026
```

```
twosidedpnorm(3)
```

```
[1] 0.002699796
```

17.4. The **t**-distribution

Everything above leaned on one big assumption: that we *know* the standard deviation σ . Recall the Z-score:

$$Z = \frac{x - \mu}{\sigma}$$

and the *population* standard deviation:

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \mu)^2} \quad (17.2)$$

When we genuinely know σ , the Z-score is the right tool. But usually we only have a *sample*, not the whole population. Then we swap the Z-score for the **t-score**:

$$t = \frac{x - \bar{x}}{s} \quad (17.3)$$

This looks almost identical to the Z-score, with two changes: we use the *sample* mean $\bar{x}$ in place of μ , and we *estimate* the standard deviation as s from the data:

17. The *t*-statistic and *t*-distribution

$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2} \quad (17.4)$$

In words: the sample standard deviation s is the typical distance of a data point from the sample mean. The only change from the population formula is the divisor — $N-1$ instead of N — which corrects for the fact that we used the data twice: once to estimate the mean, and again to measure the spread around it.

! The whole point of the *t*-distribution

Because we *estimate* the standard deviation from a small sample, our test statistic is more uncertain than a Z-score. The *t*-distribution captures that by having **fatter tails** for small samples — so it takes a more extreme result to reach significance. As the sample grows, the estimate sharpens and the *t*-distribution slides back toward the normal.

We can *see* those fatter tails. Let's plot the *t*-distribution densities for 3, 6, 10, and 20 degrees of freedom (smaller degrees of freedom means a smaller sample) alongside the normal:

```
library(dplyr)
library(ggplot2)
t_values <- seq(-6, 6, 0.01)
tdens <- data.frame(
  value = t_values,
  t_3   = dt(t_values, 3),
  t_6   = dt(t_values, 6),
  t_10  = dt(t_values, 10),
  t_20  = dt(t_values, 20),
  Normal = dnorm(t_values)
) |>
  tidyr::pivot_longer(-value, names_to = "Distribution", values_to = "density")
ggplot(tdens, aes(x = value, y = density, color = Distribution)) +
  geom_line()
```

17. The t -statistic and t -distribution

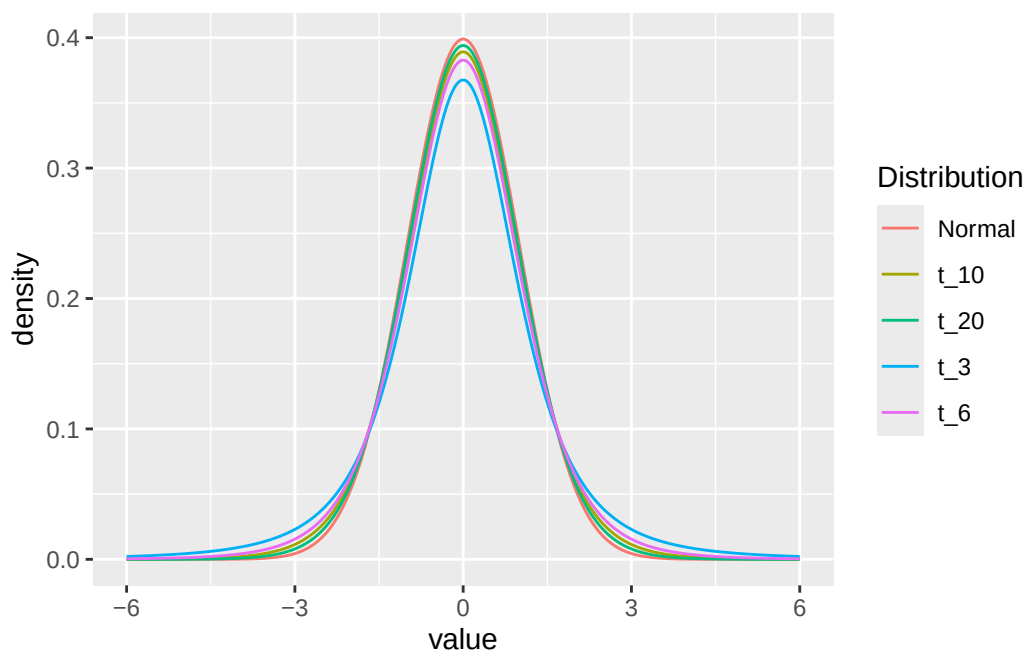


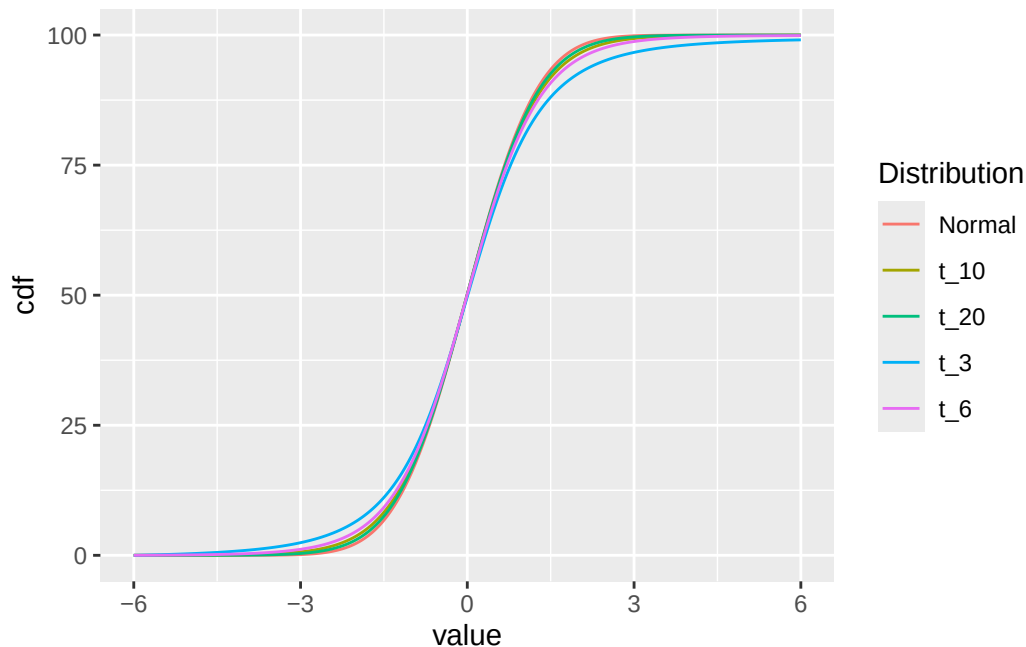
Figure 17.1.: t -distributions for various degrees of freedom. The tails are fatter for smaller degrees of freedom — the visible cost of estimating the standard deviation from the data.

The `dt` and `dnorm` functions give the *density* (the height of the curve) at each point. Notice how the `t_3` curve sits lowest in the middle and highest in the tails — that extra tail weight is the uncertainty from a tiny sample, drawn to scale.

We can also look at the cumulative version of each curve:

```
tcdf <- tdens |>
  group_by(Distribution) |>
  arrange(value) |>
  mutate(cdf = cumsum(density))
ggplot(tcdf, aes(x = value, y = cdf, color = Distribution)) +
  geom_line()
```

17. The t-statistic and t-distribution



17.4.1. p-values based on Z vs t

When we have a *sample* and want to test how far its mean sits from a population mean, we scale by the standard error (the standard deviation of the sample mean), $\sigma/\sqrt{n}$:

$$z = \frac{x - \mu}{\sigma/\sqrt{n}}$$

Let's compare the p-value we'd get from the normal distribution against the one from the t-distribution, for a single small sample.

```
set.seed(5432)
samp <- rnorm(5, mean = 0.5)
z <- sqrt(length(samp)) * mean(samp) # simplifying assumption: sigma = 1, mu = 0
```

The p-value if we *pretend* we know the standard deviation:

```
pnorm(z, lower.tail = FALSE)
```

```
[1] 0.02428316
```

17. The *t*-statistic and *t*-distribution

In reality we don't know it, so we estimate it from the data with `sd()` and use the *t*-distribution (with `length(samp) - 1` degrees of freedom):

```
ts <- sqrt(length(samp)) * mean(samp) / sd(samp)
pnorm(ts, lower.tail = FALSE) # wrong: treats the estimate as known
```

```
[1] 0.0167297
```

```
pt(ts, df = length(samp) - 1, lower.tail = FALSE) # right: accounts for the estimate
```

```
[1] 0.0503001
```

The two numbers differ, and that difference is the whole story: using the normal distribution on an *estimated* standard deviation quietly understates our uncertainty. The next section makes that mistake visible at scale.

17.4.2. Experiment: does the choice of distribution actually matter?

Here's a claim worth testing for yourself rather than taking on faith: *if you estimate the standard deviation but compute p-values from the normal distribution, your p-values will be wrong*. Let's find out how wrong.

There's a tidy way to check. If a test is working correctly, then **under the null hypothesis its p-values should be uniformly distributed between 0 and 1** — every value equally likely. So we'll simulate many samples where the null is *true*, compute p-values three ways, and look at the histograms. A flat histogram means the test is honest; a lopsided one means it isn't.

The plan:

1. Draw many samples of size `n` from the standard normal distribution.
2. Score each with the **Z**-statistic (pretending we know σ).
3. Score each with the **t**-statistic (estimating σ from the data), then compute p-values *two* ways — from the normal, and from the *t*-distribution.

First, a function that draws a sample and returns its Z-score:

```
zf <- function(n) {
  samp <- rnorm(n)
  sqrt(length(samp)) * mean(samp) / 1 # simplifying assumption: sigma = 1, mu = 0
}
```


17. The *t*-statistic and *t*-distribution

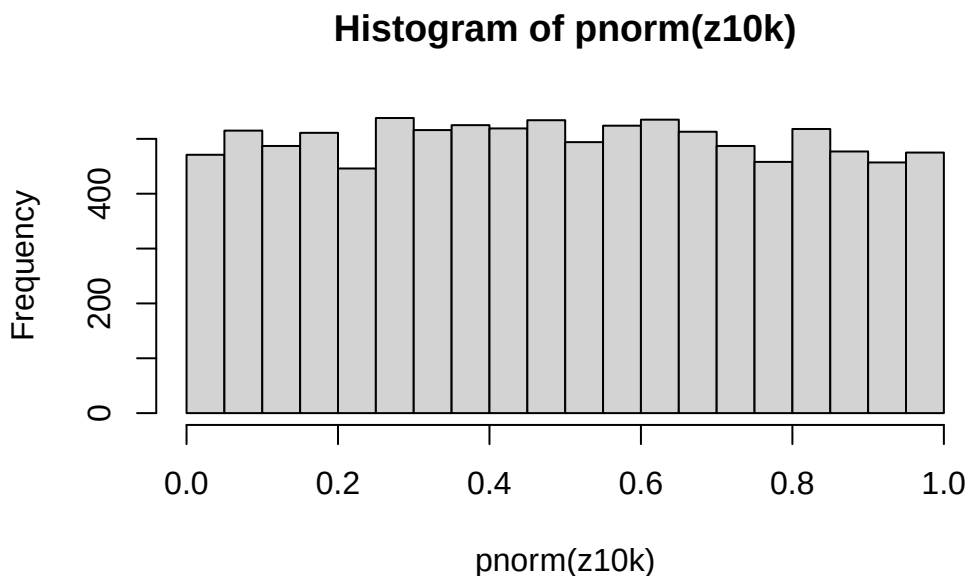
Give it a try:

```
zf(5)
```

```
[1] 0.7406094
```

Run 10,000 replicates and look at the p-value histogram. Here we *do* know the population standard deviation (we're sampling from the standard normal), so the normal distribution is the correct choice — and the histogram should be flat:

```
z10k <- replicate(10000, zf(5))  
hist(pnorm(z10k))
```



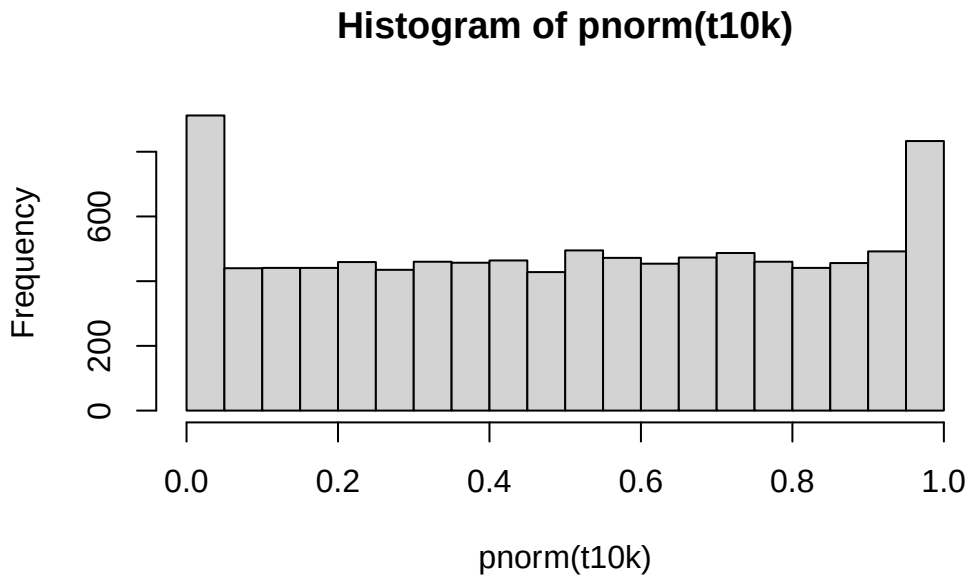
Flat, as promised. Now the *t*-score function — same idea, but we *estimate* the standard deviation with `sd()` instead of assuming it:

```
tf <- function(n) {  
  samp <- rnorm(n)  
  # use the sample standard deviation, since we "don't know" the population one  
  sqrt(length(samp)) * mean(samp) / sd(samp)  
}
```

17. The *t*-statistic and *t*-distribution

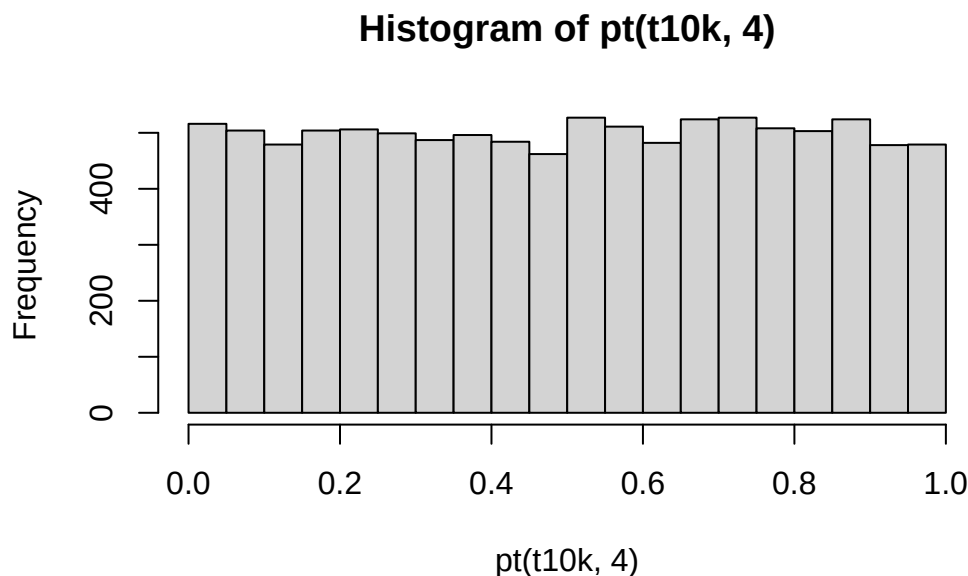
If we feed those *t*-scores to the **normal** distribution — the common mistake — the histogram is no longer flat:

```
t10k <- replicate(10000, tf(5))  
hist(pnorm(t10k))
```



The pile-ups near 0 and 1 mean this test rejects the null too often: it's overconfident, exactly because it treats an *estimated* standard deviation as if it were known. Now feed the same *t*-scores to the **t-distribution** (with $n - 1 = 4$ degrees of freedom):

```
hist(pt(t10k, 4))
```



Flat again. The *t*-distribution restores honest, uniform p-values — *that* is the payoff for all the math above.

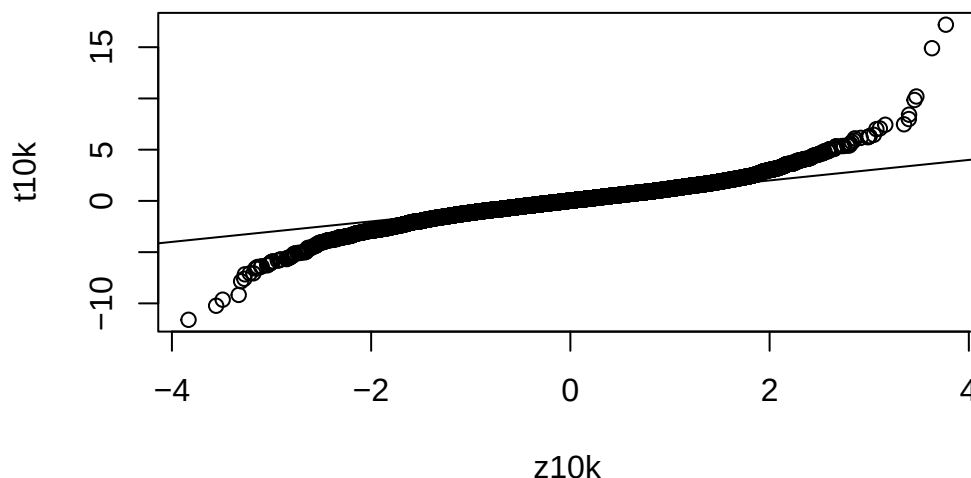
We can also compare the two sets of scores directly with a Q-Q plot.

A **Q-Q plot** plots the quantiles of two distributions against each other. If the two distributions are identical, the points fall on a straight line; where they differ, the points peel away from it.

Here we compare our simulated z-scores against our simulated t-scores. Because the t-scores carry the extra uncertainty from estimating the standard deviation, we expect them to disagree with the z-scores in the tails.

```
qqplot(z10k, t10k)
abline(0, 1)
```

17. The *t*-statistic and *t*-distribution



The points bow away from the line at both ends — the *t*-distribution’s fatter tails, made visible. What would happen if you raised the sample size from 5 to 50? Try it: the bow should shrink as the *t*-distribution approaches the normal.

17.5. Checkpoint: **t** vs normal

Before we start using `t.test()`, let’s pin down the one idea this chapter has been circling.

The *t*-distribution is a *family* of distributions indexed by the **degrees of freedom**, which track the sample size. It has heavier tails than the normal for small samples and approaches the normal as the sample grows. Those heavier tails make it more conservative — it takes a more extreme result to reach significance — which is the correct response to the uncertainty introduced by *estimating* the standard deviation from the data. That correction matters most exactly when you have the least data.

17.6. `t.test`

Now the practical part. R’s `t.test()` packages everything above into one function — you hand it data, it returns a *t*-statistic, degrees of freedom, a *p*-value, and a confidence

interval.

17.6.1. One-sample

A one-sample test compares a sample mean to a single value (zero, by default). We'll simulate a sample whose true mean is 1, so we *know* the answer the test should be reaching for.

```
x <- rnorm(20, 1)
# small sample: just the first 5 values
t.test(x[1:5])
```

One Sample t-test

```
data:  x[1:5]
t = 0.97599, df = 4, p-value = 0.3843
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 -1.029600  2.145843
sample estimates:
mean of x
0.5581214
```

We rigged this so the null hypothesis ($\text{mean} = 0$) is **false** — the true mean is 1. But with only 5 values the evidence is thin, so the p-value is modest. Add more data and the effect comes through clearly:

```
t.test(x[1:20])
```

One Sample t-test

```
data:  x[1:20]
t = 3.8245, df = 19, p-value = 0.001144
alternative hypothesis: true mean is not equal to 0
95 percent confidence interval:
 0.3541055 1.2101894
sample estimates:
```

17. The *t*-statistic and *t*-distribution

```
mean of x  
0.7821474
```

17.6.2. Two-sample

A two-sample test compares the means of two groups:

```
x <- rnorm(10, 0.5)  
y <- rnorm(10, -0.5)  
t.test(x, y)
```

Welch Two Sample t-test

```
data: x and y  
t = 3.4296, df = 17.926, p-value = 0.003003  
alternative hypothesis: true difference in means is not equal to 0  
95 percent confidence interval:  
 0.5811367 2.4204048  
sample estimates:  
 mean of x mean of y  
 0.7039205 -0.7968502
```

17.6.3. From a data frame

Often your data and group labels live in two columns of a data frame rather than in two separate vectors:

```
tdata <- data.frame(  
  value = c(x, y),  
  group = as.factor(rep(c('g1', 'g2'), each = 10))  
)  
tdata
```

```
      value group  
1  1.12896674   g1  
2 -1.26838101   g1  
3  1.04577597   g1
```

17. The *t*-statistic and *t*-distribution

```
4  1.69075585  g1
5  0.18672204  g1
6  1.99715092  g1
7  1.15424947  g1
8  0.37671442  g1
9 -0.09565723  g1
10 0.82290783  g1
11 -1.48530261 g2
12 -1.29200440 g2
13 -0.18778362 g2
14 0.59205742  g2
15 -2.10065248 g2
16 -0.29961560 g2
17 -0.38985115 g2
18 -2.47126235 g2
19 -0.63654380 g2
20 0.30245611  g2
```

R lets you run the same test with **formula notation**:

```
t.test(value ~ group, data = tdat)
```

Welch Two Sample t-test

```
data:  value by group
t = 3.4296, df = 17.926, p-value = 0.003003
alternative hypothesis: true difference in means between group g1 and group g2 is not equal
95 percent confidence interval:
 0.5811367 2.4204048
sample estimates:
mean in group g1 mean in group g2
 0.7039205      -0.7968502
```

Read `value ~ group` as “*value is a function of group*” — in practice, do a *t*-test of the values in `g1` against those in `g2`.

17.6.4. Equivalence to a linear model

A two-sample *t*-test (with equal variances assumed) is really a linear model in disguise. Run the test:

```
t.test(value ~ group, data = tdat, var.equal = TRUE)
```

Two Sample t-test

```
data: value by group
t = 3.4296, df = 18, p-value = 0.002989
alternative hypothesis: true difference in means between group g1 and group g2 is not equal
95 percent confidence interval:
 0.5814078 2.4201337
sample estimates:
mean in group g1 mean in group g2
    0.7039205      -0.7968502
```

and compare it to fitting a line:

```
res <- lm(value ~ group, data = tdat)
summary(res)
```

Call:

```
lm(formula = value ~ group, data = tdat)
```

Residuals:

```
      Min       1Q   Median       3Q      Max
-1.9723 -0.5600  0.2511  0.5252  1.3889
```

Coefficients:

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    0.7039     0.3094   2.275  0.03538 *
groupg2       -1.5008     0.4376  -3.430  0.00299 **
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```


17. The *t*-statistic and *t*-distribution

```
Residual standard error: 0.9785 on 18 degrees of freedom
Multiple R-squared:  0.3952,    Adjusted R-squared:  0.3616
F-statistic: 11.76 on 1 and 18 DF,  p-value: 0.002989
```

The `groupg2` coefficient is the difference in means, and its p-value matches the t-test's. Same question, same answer, two front doors.

17.7. Power calculations

The **power** of a test is the probability that it *correctly* rejects the null hypothesis when the alternative is true — in other words, the probability of *not* missing a real effect (not making a Type II error). Power rises with the significance level (**alpha**), the sample size, and the effect size. It's what you compute *before* an experiment to ask “how many samples do I need?”

`power.t.test()` ties these quantities together. From `help("power.t.test")`, the main arguments are:

- **n** — sample size
- **delta** — effect size (difference in means)
- **sd** — standard deviation
- **sig.level** — significance level
- **power** — power

You supply any four and it solves for the fifth. For example, the power of a one-sample test with `n = 5`, `sd = 1`, and an effect size of 1:

```
power.t.test(n = 5, delta = 1, sd = 1, sig.level = 0.05)
```

Two-sample t test power calculation

```
      n = 5
  delta = 1
      sd = 1
sig.level = 0.05
  power = 0.2859276
alternative = two.sided
```

NOTE: `n` is number in *each* group

17. The *t*-statistic and *t*-distribution

That prints a full summary. To grab just the power value, use `$`:

```
power.t.test(n = 5, delta = 1, sd = 1,
             sig.level = 0.05, type = 'one.sample')$power
```

```
[1] 0.4013203
```

Tip

When a function returns a result that doesn't look “computable” — like the summary from `power.t.test` — you can pull out a single piece with `$`. How do you know what's available to extract? Use `names()` for the labels, or `str()` for the full structure:

```
names(power.t.test(n = 5, delta = 1, sd = 1,
                  sig.level = 0.05, type = 'one.sample'))
```

```
[1] "n"          "delta"      "sd"         "sig.level"  "power"
[6] "alternative" "note"       "method"
```

```
# or
str(power.t.test(n = 5, delta = 1, sd = 1,
                sig.level = 0.05, type = 'one.sample'))
```

List of 8

```
$ n          : num 5
$ delta      : num 1
$ sd         : num 1
$ sig.level  : num 0.05
$ power      : num 0.401
$ alternative: chr "two.sided"
$ note       : NULL
$ method     : chr "One-sample t test power calculation"
- attr(*, "class")= chr "power.htest"
```

Just as often you'll flip the question around: *how big a sample* do I need to hit a target power? Solve for `n` by supplying `power` instead:

17. The *t*-statistic and *t*-distribution

```
power.t.test(delta = 1, sd = 1, sig.level = 0.05, power = 0.8, type = "one.sample")
```

One-sample t test power calculation

```
      n = 9.937864
delta = 1
sd = 1
sig.level = 0.05
power = 0.8
alternative = two.sided
```

`power.t.test` is convenient and fast. But as we saw with the *t*-distribution itself, sometimes the distribution of a test statistic is **not** easily calculated in closed form. In those cases we can estimate power by *simulation* — run the experiment thousands of times and count how often it rejects the null:

```
sim_t_test_pval <- function(n = 5, delta = 1, sd = 1, sig.level = 0.05) {
  x <- rnorm(n, delta, sd)
  t.test(x)$p.value <= sig.level
}
pow <- mean(replicate(1000, sim_t_test_pval()))
pow
```

```
[1] 0.405
```

Step by step: `sim_t_test_pval` simulates one sample of size `n` from a normal distribution with mean `delta` and standard deviation `sd`, runs a one-sample *t*-test, and returns `TRUE` when the *p*-value clears the significance threshold. `replicate` repeats that 1000 times, and `mean` turns the resulting `TRUE/FALSE` vector into the *proportion* of significant results — our estimate of power.

The two approaches should land in the same place:

```
power.t.test(n = 5, delta = 1, sd = 1, sig.level = 0.05, type = 'one.sample')$power
```

```
[1] 0.4013203
```

17. The *t*-statistic and *t*-distribution

```
mean(replicate(1000, sim_t_test_pval(n = 5, delta = 1, sd = 1, sig.level = 0.05)))
```

```
[1] 0.414
```

Close — and the simulation route keeps working in situations where no neat formula exists.

17.8. Exercises

Note

Several exercises reuse objects (`samp`, `t10k`, `tdata`) built earlier in the chapter. Run the chapter’s chunks first, or recreate the objects in your session.

1. **One-sided vs two-sided.** Use `twosidedpnorm()` to find the two-sided p-value for a Z-score of 1.96. Does the answer match the “95%” rule of thumb you may have heard?

Solution

```
twosidedpnorm(1.96)
```

```
[1] 0.04999579
```

It’s almost exactly 0.05 — which is *why* 1.96 is the classic cutoff for a two-sided test at the 5% level.

2. **Degrees of freedom matter.** Compute the two-sided p-value for $t = 2.5$ from the *t*-distribution with 4 degrees of freedom and again with 30. Which is larger, and does that fit the “fatter tails” story?

Solution

```
2 * pt(2.5, df = 4, lower.tail = FALSE)
```

```
[1] 0.06676654
```

```
2 * pt(2.5, df = 30, lower.tail = FALSE)
```

```
[1] 0.01811565
```

The 4-df p-value is larger. Fewer degrees of freedom means fatter tails, so the same *t*-statistic is *less* surprising — it takes more extreme evidence to reach significance with a small sample.

3. **A paired test.** Imagine the same 8 patients measured before and after a treatment. Simulate the pairs below and run a paired *t*-test with `t.test(after, before, paired = TRUE)`. How does the result compare to treating the two groups as independent (`paired = FALSE`)?

```
set.seed(11)
before <- rnorm(8, 10, 2)
after <- before + rnorm(8, 1, 0.5) # a real, consistent bump per patient
```

i Solution

```
t.test(after, before, paired = TRUE)
```

Paired t-test

```
data: after and before
t = 6.7665, df = 7, p-value = 0.0002611
alternative hypothesis: true mean difference is not equal to 0
95 percent confidence interval:
 0.4408169 0.9144199
sample estimates:
mean difference
 0.6776184
```

```
t.test(after, before, paired = FALSE)
```

Welch Two Sample t-test

```
data: after and before
t = 0.62439, df = 13.955, p-value = 0.5424
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
-1.650735  3.005972
sample estimates:
mean of x mean of y
```

17. The *t*-statistic and *t*-distribution

```
10.364923  9.687305
```

The paired test has a much smaller p-value. By matching each patient to themselves it removes the patient-to-patient variability, leaving only the change due to treatment — so it detects the effect that the unpaired test can barely see.

4. **Power and sample size.** Using `power.t.test`, find the sample size needed to detect an effect of `delta = 0.5` (with `sd = 1`) at 80% power for a one-sample test. Then halve the effect size to 0.25 and recompute. Roughly how does the required `n` change?

i Solution

```
power.t.test(delta = 0.5, sd = 1, sig.level = 0.05, power = 0.8, type = "one.sample")
```

```
[1] 33.3672
```

```
power.t.test(delta = 0.25, sd = 1, sig.level = 0.05, power = 0.8, type = "one.sample")
```

```
[1] 127.5161
```

Halving the effect size roughly *quadruples* the required sample size — small effects are expensive to detect. This is the single most useful intuition to carry out of a power calculation.

17.9. Summary

You can now:

- Explain why estimating the standard deviation from a small sample pushes us from the normal distribution onto the *t*-distribution, and what “fatter tails” buys you.
- Distinguish a Z-score from a *t*-score and turn either into a p-value with `pnorm` and `pt`.
- Use simulation to *check* that a test is honest — uniform p-values under the null — rather than taking it on trust.
- Run one-sample, two-sample, paired, and formula-based tests with `t.test()`, and recognize the two-sample *t*-test as a linear model.
- Estimate power and required sample size with `power.t.test()` and by simulation.

The thread tying it all together: when you estimate uncertainty from your data instead of knowing it, you have to pay for that estimate — and the *t*-distribution is how statistics

keeps the books honest.

17.10. Resources

- The [pwr package](#) for a fuller toolkit of power calculations.
- The American Statistical Association’s statement on p-values (Wasserstein and Lazar 2016) for what a p-value does and doesn’t tell you.

18. K-means clustering

When yeast cells run low on glucose, they flip their entire metabolic program — switching from fermentation to respiration in an event called the **diauxic shift**. DeRisi et al. (1997) watched this happen by measuring the expression of roughly 6,400 genes at seven time points across the shift. That leaves you with a wall of numbers: 6,400 genes, each with its own little trajectory over time. Some genes ramp up as the cells switch fuels, some shut down, and most barely move at all. How do you find the *groups* of genes that rise and fall together — the ones that might be co-regulated, taking part in the same biological response?

That is exactly the kind of question **k-means clustering** answers. It is an unsupervised method: you don't tell it which genes belong together, you just ask it to sort the genes into a handful of groups so that genes with similar behavior land in the same group. In this chapter you'll use k-means on the DeRisi yeast data — the same dataset you met in the SummarizedExperiment chapter — to pull out clusters of co-regulated genes and then read their temporal profiles back as biology.

18.1. What you'll learn

- Explain in plain language what k-means clustering does and when it's useful.
- Pull a public dataset from NCBI GEO with `getGEO()` and convert it to a `SummarizedExperiment`.
- Filter a gene-expression matrix down to its most variable (most informative) genes using the standard deviation.
- Run `kmeans()` on real data and make the result **reproducible** with `set.seed()`.
- Plot each cluster's expression profile and interpret what the clusters say about the diauxic shift.

18.2. What k-means does

K-means clustering is a method for finding groups in a dataset. It is an *unsupervised* learning technique: it doesn't rely on previously labeled data. Instead, it finds structure directly from the data, based on how similar the data points are to one another (Figure 18.1).

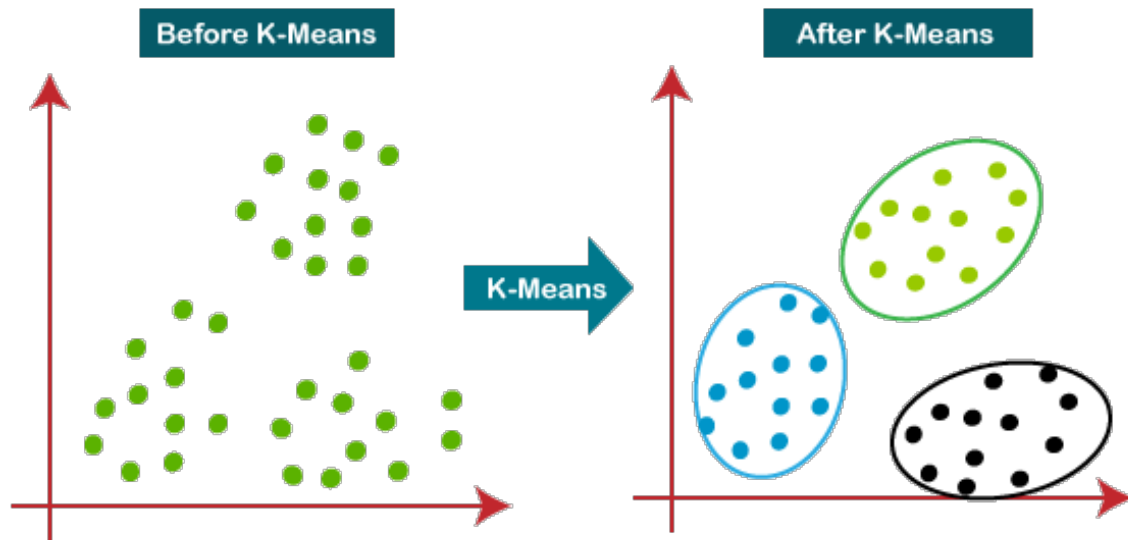


Figure 18.1.: K-means clustering takes a dataset and divides it into k clusters.

The idea is to split the data into k groups, called **clusters**, so that each point belongs to the cluster whose average it is closest to. The algorithm tries to make points within a cluster as similar as possible while keeping different clusters as distinct as possible. For our yeast data, a “point” is a single gene, and “similar” means *follows a similar expression pattern over the time course*.

💡 An analogy for the algorithm

Imagine k party hosts scattered around a room full of guests. Each guest walks to the nearest host (**assignment**). Each host then moves to the middle of their own group (**update**). Guests re-check who is now closest and move again, and the hosts re-center. Repeat until nobody needs to move. The final huddles are your clusters, and each host’s position is a **centroid** — the average of its group.

18.3. The k-means algorithm

Spelled out, k-means follows these steps:

1. Choose the number of clusters, k .
2. Place k centroids at random (often by picking k of the data points).

18. K-means clustering

3. **Assign** each data point to its nearest centroid.
4. **Update** each centroid to the mean of the points assigned to it.
5. Repeat steps 3 and 4 until the centroids stop moving (or a maximum number of iterations is reached).

When the centroids stabilize, the algorithm has *converged*, and the final clusters represent the patterns in the data.

 k-means is random — seed it

Step 2 starts from a **random** placement of centroids, so two runs can give slightly different clusters (and the cluster *numbers* can be shuffled). Whenever you want a result you can reproduce — and a frozen figure that doesn't change on every render — call `set.seed()` before `kmeans()`.

18.4. Pros and cons of k-means clustering

Compared to other clustering algorithms, k-means has several advantages:

- **Simplicity** — the algorithm is straightforward and easy to implement, even on large datasets.
- **Scalability** — it adapts well to large datasets with optimization or parallel processing.
- **Speed** — it is generally fast, especially when **k** is small.
- **Interpretability** — the results are easy to read: every point gets a single cluster label.

It also has real limitations:

- **Choice of k** — you have to pick the number of clusters yourself, and a poor choice gives poor results. Domain knowledge and experimentation help.
- **Sensitivity to initial conditions** — different random starts can give different answers, which is why you seed and sometimes re-run.
- **Assumes round, even clusters** — k-means expects roughly spherical, similarly sized clusters, and struggles when the real groups have other shapes.
- **Sensitivity to outliers** — a few extreme points can drag a centroid around, so cleaning or filtering the data first helps.

Despite these limits, k-means remains a popular, widely used tool for exploring data — and it is especially handy in biology, where spotting groups of co-behaving genes can hint at shared function or regulation.

18.5. A worked example: the diauxic shift

18.5.1. The data and experimental background

The data come from DeRisi et al. (1997). From their abstract:

DNA microarrays containing virtually every gene of *Saccharomyces cerevisiae* were used to carry out a comprehensive investigation of the temporal program of gene expression accompanying the metabolic shift from fermentation to respiration. The expression profiles observed for genes with known metabolic functions pointed to features of the metabolic reprogramming that occur during the diauxic shift, and the expression patterns of many previously uncharacterized genes provided clues to their possible functions.

These data are available from NCBI GEO as [GSE28](#).

Here is the biology in brief. When baker's yeast grows on glucose with plenty of oxygen, it still *prefers* to ferment the glucose rather than respire it — even though respiration is more efficient. Once the glucose runs low, the cells switch metabolic gears to respiration. That switch is the [diauxic shift](#). This experiment profiles about 6,400 genes across a time course spanning the shift.

These data have no replicates; they are a single time course. Watching how *groups* of genes behave across that course can reveal biology — both about the system and about individual genes. So we'll use clustering to group genes with similar expression patterns over time, then look at those groups.

Your goal: use `kmeans()` to divide the most variable (most informative) genes into a few groups, then visualize each group's temporal profile.

18.5.2. Getting the data

These data were deposited at NCBI GEO back in 2002. `GEOquery` can pull them out in one call.

```
library(GEOquery)
gse <- getGEO("GSE28")[[1]]
class(gse)
```

```
[1] "ExpressionSet"
attr(,"package")
[1] "Biobase"
```

18. K-means clustering

GEOquery predates the SummarizedExperiment class, so it hands you an older ExpressionSet. Bioconductor can convert that into the SummarizedExperiment you already know how to work with.

```
library(SummarizedExperiment)
gse <- as(gse, "SummarizedExperiment")
gse
```

```
class: SummarizedExperiment
dim: 6400 7
metadata(3): experimentData annotation protocolData
assays(1): exprs
rownames(6400): 1 2 ... 6399 6400
rowData names(20): ID ORF ... FAILED IS_CONTAMINATED
colnames(7): GSM887 GSM888 ... GSM892 GSM893
colData names(33): title geo_accession ... supplementary_file
                  data_row_count
```

A quick look at the sample descriptions in colData() suggests the columns aren't in time order:

```
colData(gse)$title
```

```
[1] "diauxic shift timecourse: 15.5 hr" "diauxic shift timecourse: 0 hr"
[3] "diauxic shift timecourse: 18.5 hr" "diauxic shift timecourse: 9.5 hr"
[5] "diauxic shift timecourse: 11.5 hr" "diauxic shift timecourse: 13.5 hr"
[7] "diauxic shift timecourse: 20.5 hr"
```

So reorder them by hand so the columns run in time-course order:

```
gse <- gse[, c(2, 4, 5, 6, 1, 3, 7)]
```

18.5.3. Preprocessing: keep only the informative genes

Most of those 6,400 genes barely change across the time course. Feeding all of them to the clustering algorithm is slow and, worse, lets thousands of flat, noisy genes drown out the few hundred that actually respond to the diauxic shift.

18. K-means clustering

So before clustering, you **filter** down to the genes that carry signal. A simple, effective filter is the **standard deviation** of each gene across the samples: a gene whose expression swings widely over the time course has a high standard deviation, while a gene that stays flat has a low one.

! Why filter on variability first

Genes that change a lot across conditions are the ones most likely to be doing something biologically interesting. Dropping the flat, low-variance genes does two good things at once: it shrinks the dataset so clustering is faster, and it removes noise so real patterns stand out. Filtering on a gene-level statistic like the standard deviation — one that ignores the grouping you'll later test — can also genuinely *increase statistical power* and reduce false positives (Bourgon et al. 2010).

The recipe has three steps:

1. Compute each gene's standard deviation across the samples.
2. Pick a cutoff (this is somewhat arbitrary — try a few and see how sensitive your results are).
3. Keep only the genes above the cutoff.

Start with step 1 and look at the distribution of standard deviations:

```
sds <- apply(assays(gse)[[1]], 1, sd)
hist(sds)
```

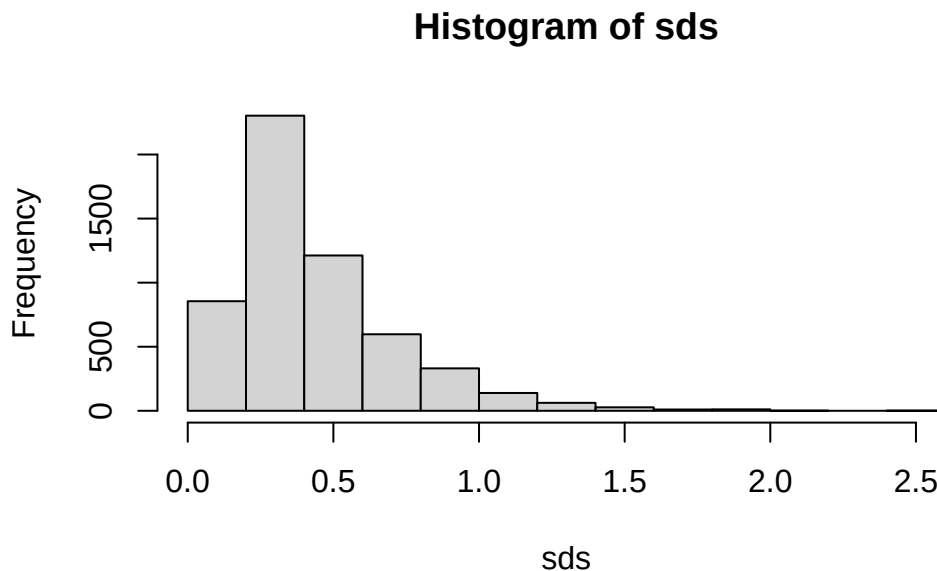


Figure 18.2.: Histogram of per-gene standard deviations across the time course. Most genes barely vary (the tall bars on the left); a long right tail marks the genes that change the most.

Figure 18.2 tells the story directly. The bulk of the genes pile up near zero — they hardly change across the time course and carry little information. A long tail stretches off to the right: these are the genes whose expression swings the most. A cutoff of about `sd > 0.8` sits past the main pile and keeps that informative right tail. The exact value is a judgment call, which is why it's worth re-running with a different cutoff later (one of the exercises does just that).

Now apply the cutoff (step 3) and build a smaller `SummarizedExperiment` holding just the highly variable genes:

```
idx <- sds > 0.8 & !is.na(sds)
gse_sub <- gse[idx, ]
nrow(gse_sub)
```

```
[1] 582
```

That trims thousands of genes down to a few hundred — a far more workable set.

18.5.4. Clustering

`gse_sub` now holds just the informative genes. The `kmeans()` function takes a matrix (rows = genes, columns = samples) and the number of clusters. Remember to seed first so the result is reproducible.

```
k <- 4
set.seed(42)
km <- kmeans(assays(gse_sub)[[1]], centers = k)
```

The result, `km`, carries a vector `km$cluster` that gives the cluster label (1 through `k`) for each gene. With four clusters, you can plot every cluster's genes side by side and see how each group moves over time.

```
expression_values <- assays(gse_sub)[[1]]
par(mfrow = c(2, 2), mar = c(3, 4, 1, 2)) # 2x2 grid of plots
for (i in 1:k) {
  matplot(t(expression_values[km$cluster == i, ]), type = "l",
          ylim = c(-3, 3), ylab = paste("cluster", i))
}
```

18. K-means clustering

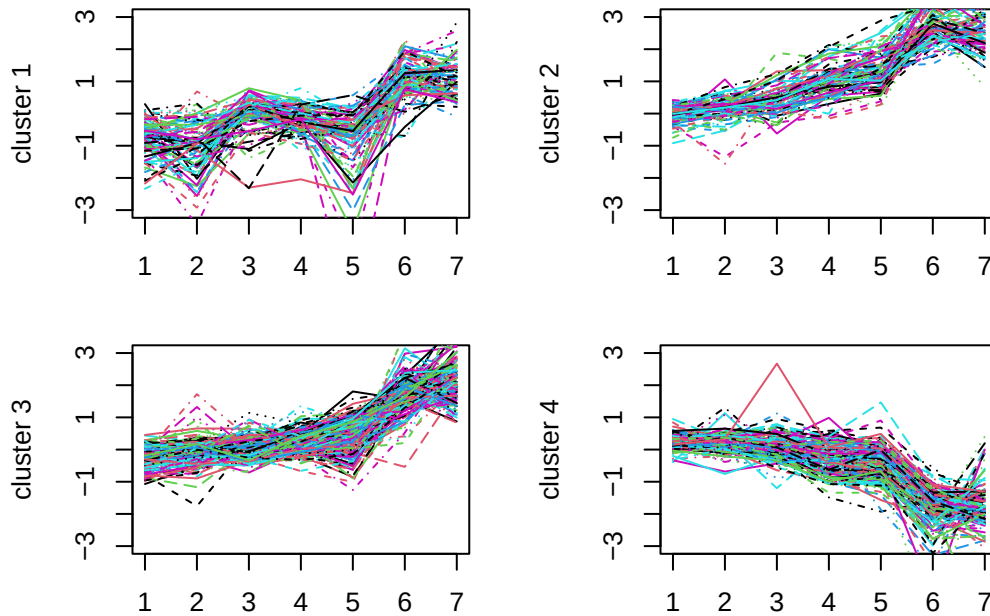


Figure 18.3.: Expression profiles for the four clusters found by k-means. Each line is one gene; the x-axis runs left to right across the time course, and the y-axis is the (standardized) expression level. Genes within a cluster share a trend, which suggests they may be co-regulated.

18.5.5. Reading the clusters as biology

This is the step that the numbers were building toward. Each panel in Figure 18.3 is a group of genes that move *together* across the time course — and “together” is the whole point, because co-regulated genes are often part of the same pathway or controlled by the same switch.

Look at the *shapes* rather than the exact lines:

- Some clusters **rise** toward the late time points. Those are genes that get switched **on** as the glucose runs out and the cells shift to respiration — candidate members of the respiratory and stress-response programs.
- Other clusters **fall** across the course. Those genes are being switched **off** during the shift — for example, the fermentation and growth machinery the cell no longer needs once glucose is scarce.

18. K-means clustering

- The lines within each panel hug one another tightly, which is the visual signature of co-regulation: many genes responding in step to the same metabolic cue.

That is the payoff of clustering. You started with 6,400 noisy trajectories and ended with a handful of clean, interpretable patterns — rising groups, falling groups — that map directly onto the biology of the diauxic shift. If you wanted to go further, you could pull the gene names out of each cluster (`rownames(gse_sub)[km$cluster == i]`) and ask whether the “rising” cluster is enriched for respiration genes.

i Why standardized expression?

The y-axis runs from about -3 to 3 because these microarray values are log-ratios centered near zero: positive means “higher than the reference,” negative means “lower.” Clustering on values like these groups genes by the *shape* of their response over time rather than by their absolute brightness, which is usually what you want.

18.6. Exercises

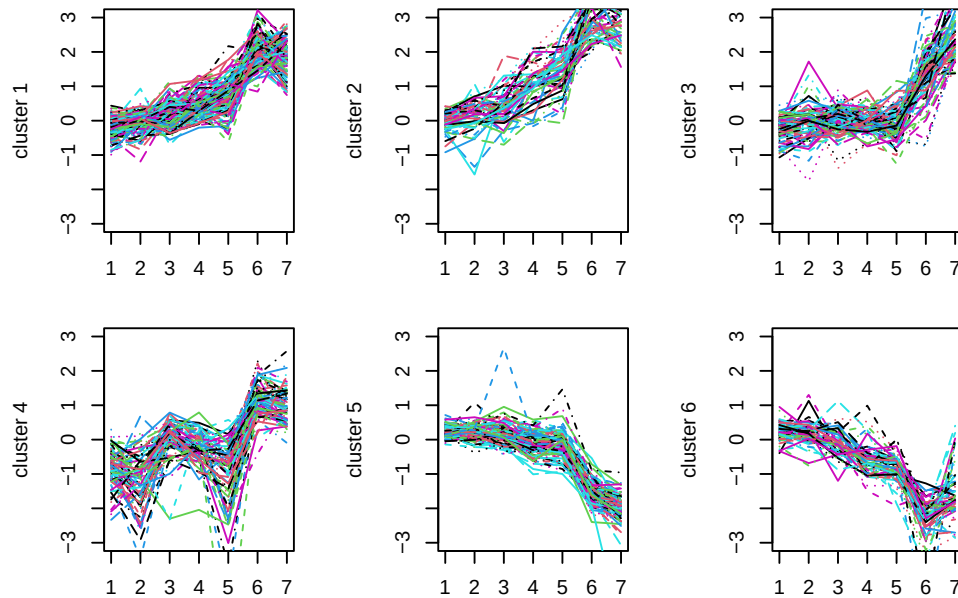
For each exercise, reuse the `gse` and `gse_sub` objects you built above.

1. **Try a different k.** Re-run the clustering with `k <- 6` instead of 4 and re-make the cluster plots. Do the extra clusters reveal new patterns, or do they mostly split the existing ones?

i Solution

```
k6 <- 6
set.seed(42)
km6 <- kmeans(assays(gse_sub)[[1]], centers = k6)
par(mfrow = c(2, 3), mar = c(3, 4, 1, 2))
for (i in 1:k6) {
  matplot(t(expression_values[km6$cluster == i, ]), type = "l",
          ylim = c(-3, 3), ylab = paste("cluster", i))
}
```

18. K-means clustering



With more clusters, the broad “rising” and “falling” groups tend to break into finer sub-patterns (e.g. early vs. late risers). There is no single correct k ; the right number depends on how fine a distinction is biologically useful.

2. **Try a different cutoff.** Lower the standard-deviation cutoff to 0.5 so you keep *more* genes, rebuild the subset, and cluster again with $k = 4$. How many genes do you keep now, and do the cluster shapes get cleaner or noisier?

i Solution

```
idx2 <- sds > 0.5 & !is.na(sds)
gse_sub2 <- gse[idx2, ]
nrow(gse_sub2)
```

```
[1] 1657
```

```
set.seed(42)
km2 <- kmeans(assays(gse_sub2)[[1]], centers = 4)
```

A lower cutoff keeps many more genes, including less-variable ones. The extra genes are noisier, so the cluster profiles usually look a little messier. This is the sensitivity check the chapter warned about: the cutoff is a knob, and it’s worth turning it to see whether your conclusions hold.

3. **How big is each cluster?** Using your original `km` result, count how many genes fall in each of the four clusters.

i Solution

```
table(km$cluster)
```

```
 1   2   3   4
98 102 203 179
```

`table()` tallies the cluster labels. Uneven cluster sizes are normal and fine — a cluster of co-regulated genes is just as real whether it has 30 members or

300.

4. **Reproducibility.** Run `kmeans()` on `gse_sub` twice *without* calling `set.seed()` first, and compare `table()` of the cluster labels between the two runs. What changes, and why does that matter for a printed result?

i Solution

```
km_a <- kmeans(assays(gse_sub)[[1]], centers = 4)
km_b <- kmeans(assays(gse_sub)[[1]], centers = 4)
table(km_a$cluster)
```

```
 1   2   3   4
91  88 165 238
```

```
table(km_b$cluster)
```

```
 1   2   3   4
165  91  88 238
```

The two runs start from different random centroids, so the cluster *labels* (and sometimes the exact membership) differ. Seeding with `set.seed()` pins the random start so anyone re-running your code sees the same clusters — which is exactly why we seeded the worked example.

18.7. Summary

You set out to find groups of genes that behave alike across the yeast diauxic shift, and you did it with k-means. Looking back at the objectives:

- You can describe k-means as an **unsupervised** method that sorts points into **k** groups around moving **centroids**, and you know its main trade-offs (you pick **k**; it likes round, even clusters; it's sensitive to outliers and to the random start).
- You pulled a real dataset from GEO with `getGEO()` and converted it to a `SummarizedExperiment`.
- You **filtered** the genes down to the high-variance, informative ones using their standard deviation, and you saw on Figure 18.2 why a cutoff past the main pile makes sense.
- You ran `kmeans()` reproducibly with `set.seed()` and plotted each cluster.
- You read Figure 18.3 as biology: clusters that **rise** are genes switched on for respiration as glucose runs out, and clusters that **fall** are the fermentation/growth machinery being switched off — co-regulated genes moving in step through the diauxic shift.

That wraps up the statistics part of the book. Clustering is your first taste of unsupervised learning; the machine-learning chapters pick up that thread and build on it.

Part V.

Machine Learning

19. Introduction

Machine learning represents a fundamental shift in how we approach problem-solving with computers. Rather than explicitly programming every rule and decision path, machine learning allows algorithms to discover patterns in data and make predictions or decisions based on these learned patterns. This approach has proven particularly powerful for complex problems where traditional rule-based programming becomes unwieldy or where the underlying patterns are too subtle for humans to easily codify.

At its core, machine learning is about generalization. We want to build models that can take what they've learned from historical data and apply that knowledge to new, previously unseen situations. This capability makes machine learning invaluable across diverse fields, from predicting stock prices and diagnosing diseases to recognizing speech and recommending movies.

Machine learning in biology is a really broad topic. Greener et al. (2022) present a nice overview of the different types of machine learning methods that are used in biology. Libbrecht and Noble (2015) also present an early review of machine learning in genetics and genomics.

19.1. Types of Machine Learning

The field of machine learning encompasses several distinct approaches, each suited to different types of problems and data structures. Understanding these categories helps practitioners choose appropriate methods and set realistic expectations for their projects.

Supervised learning forms the foundation of most practical machine learning applications. In supervised learning, we have access to both input features and the correct answers (labels or targets) for our training examples. The algorithm learns to map inputs to outputs by studying these example pairs. Classification problems, where we predict discrete categories, and regression problems, where we predict continuous numerical values, both fall under supervised learning. For instance, predicting whether an email is spam (classification) or forecasting house prices (regression) are classic supervised learning tasks.

Unsupervised learning tackles scenarios where we have input data but no predetermined correct answers. Instead of learning to predict specific outputs, unsupervised algorithms

seek to discover hidden structures or patterns within the data itself. Clustering algorithms group similar data points together, while dimensionality reduction techniques identify the most important underlying factors that explain variation in the data. These methods often serve as exploratory tools, helping analysts understand their data better before applying supervised techniques.

Reinforcement learning takes a different approach entirely, focusing on learning through interaction with an environment. Rather than learning from fixed examples, reinforcement learning agents take actions and receive rewards or penalties, gradually improving their decision-making through trial and error. This approach has achieved remarkable success in game-playing scenarios and robotics, though it requires specialized techniques and considerable computational resources.

Learning Type	Data Requirements	Goal	Common Applications
Supervised	Input-output pairs	Predict labels/values	Classification, regression
Unsupervised	Input data only	Discover patterns	Clustering, dimensionality reduction
Reinforcement	Environment interaction	Optimize decisions	Game playing, robotics

When thinking about machine learning, it can help to have a simple framework in mind. In Figure 19.1, we present a simple view of machine learning according to the [scikit-learn](#) package.

i Terminology and Concepts

- Data

: Data is the foundation of machine learning and can be structured (tabular) or unstructured (text, images, audio). It is usually divided into training, validation, and testing sets for model development and evaluation.

- Features

: Features are the variables or attributes used to describe the data points. Feature

19. Introduction

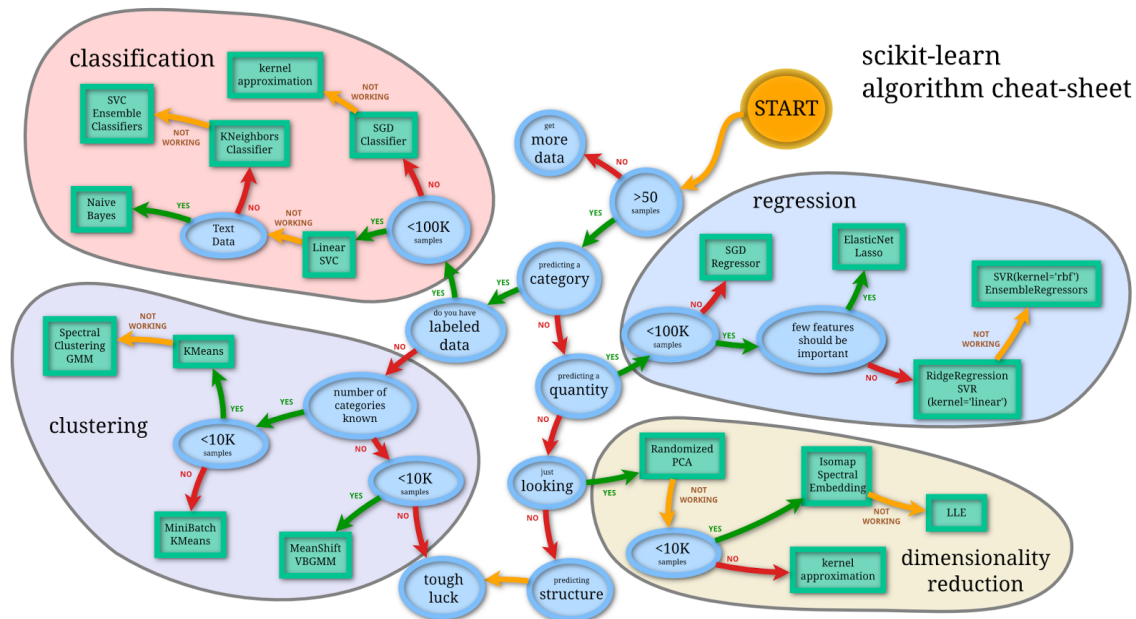


Figure 19.1.: A simple view of machine learning according the sklearn.

engineering and selection are crucial steps in machine learning to improve model performance and interpretability.

- Models and Algorithms

: Models are mathematical representations of the relationship between features and the target variable(s). Algorithms are the methods used to train models, such as linear regression, decision trees, and neural networks.

- Hyperparameters and Tuning

: Hyperparameters are adjustable parameters that control the learning process of an algorithm. Tuning involves finding the optimal set of hyperparameters to improve model performance.

- Evaluation Metrics

: Evaluation metrics quantify the performance of a model, such as accuracy, precision, recall, F1-score (for classification), and mean squared error, R-squared (for regression).

19.2. The Machine Learning Workflow

Successful machine learning projects follow a structured workflow that ensures robust, reliable results. This process begins long before any algorithms are trained and extends well beyond initial model development.

i The Machine Learning Workflow

In a nutshell, the machine learning workflow consists of the following steps:

1. **Problem Definition:** Clearly define the problem and determine if machine learning is the right approach.
2. **Data Collection and Preparation:** Gather relevant data and preprocess it to make it suitable for modeling.
3. **Data Splitting:** Divide the data into training, validation, and test sets to ensure unbiased evaluation.
4. **Model Selection:** Choose the appropriate machine learning algorithm and configure its hyperparameters.
5. **Training:** Fit the model to the training data, allowing it to learn patterns and relationships.
6. **Evaluation:** Assess model performance using validation data and appropriate metrics.
7. **Deployment:** Integrate the model into production systems for real-world use.

The journey starts with problem definition, where practitioners must clearly articulate what they're trying to achieve and whether machine learning is the appropriate solution. Not every problem requires machine learning; sometimes simpler statistical methods or rule-based systems provide better solutions with less complexity and maintenance overhead.

Data collection and preparation typically consume the majority of time in real-world projects. Raw data rarely arrives in a format suitable for machine learning algorithms. Common preprocessing steps include handling missing values, encoding categorical variables, scaling numerical features, and addressing outliers. The quality of this preprocessing often determines the success or failure of the entire project.

Data splitting deserves special attention because it directly impacts the reliability of your results. The training set teaches the algorithm, but if we evaluate performance on the same data used for training, we get an overly optimistic view of how well our model will perform on new data. This is analogous to letting students see exam questions while studying and then using the same questions for the actual exam.

! The Critical Importance of Data Splitting

One of the most crucial steps in the machine learning workflow is properly splitting your data into separate sets for training, validation, and testing. This separation serves as the foundation for honest evaluation and prevents overfitting.

- **Training data** is used to fit the model parameters.
- **Validation data** helps select the best model architecture and hyperparameters.
- **Test data** provides a final, unbiased estimate of model performance on new data.

The golden rule: never use test data for any decision-making during model development. Reserve it exclusively for final evaluation.

A common approach involves splitting data into 80% for training and 20% for testing. For more complex projects, a three-way split might use 60% for training, 20% for validation (model selection), and 20% for final testing. Cross-validation provides an alternative approach where the training data is repeatedly split into smaller training and validation sets, providing more robust estimates of model performance while still preserving an untouched test set.

Model selection involves choosing both the type of algorithm and its specific configuration. Different algorithms make different assumptions about the data and excel in different scenarios. Linear models work well when relationships are approximately linear and interpretability is important. Tree-based methods handle nonlinear relationships and interactions naturally but may overfit with limited data. Neural networks can model complex patterns but require large datasets and careful tuning. Hyperparameters are the parameters that describe the details of the model such as the depth of a decision tree, the choice of k in k -nearest neighbors, or the learning rate in gradient descent. Hyperparameter tuning is the process of finding the best values for these parameters, often using techniques like grid search or random search combined with cross-validation.

Training is where the algorithm actually learns from the data. During this phase, the model adjusts its internal parameters to minimize prediction errors on the training set. However, the goal isn't to achieve perfect performance on training data. Models that fit training data too closely often fail to generalize to new examples, a problem known as overfitting.

Evaluation determines whether our model is ready for deployment. This involves multiple metrics beyond simple accuracy, including precision, recall, F1-score for classification, or mean squared error and R-squared for regression. We also examine model behavior across different subgroups in our data to ensure fair and consistent performance.

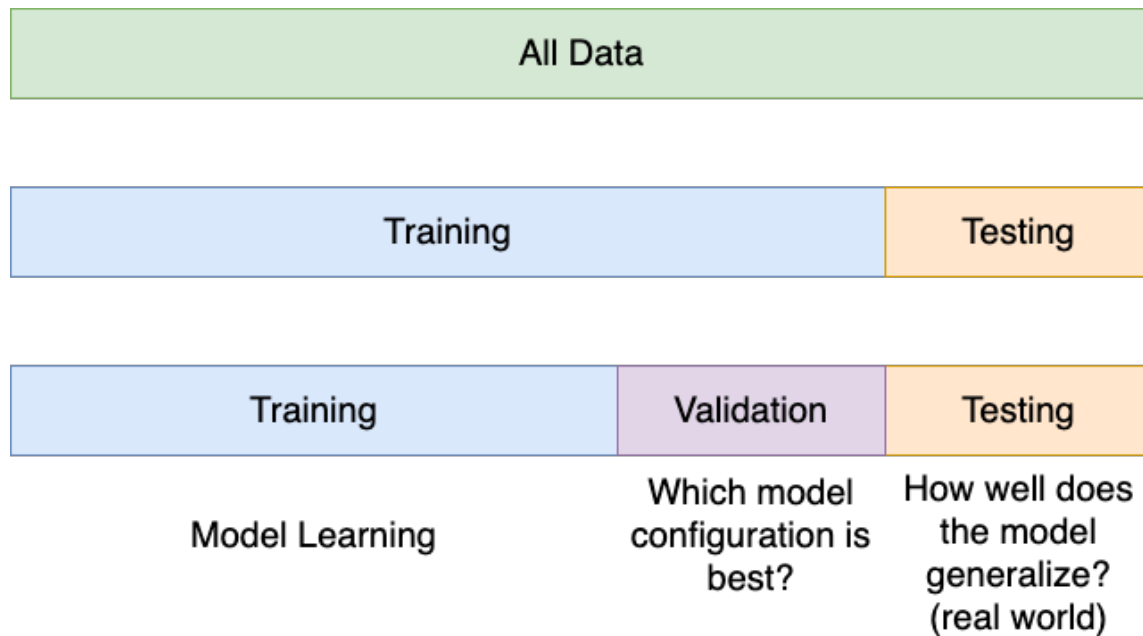


Figure 19.2.: Data splitting and train/validate/test paradigm. For models without “hyperparameters”, only the training/testing sets are necessary. For models that require learning model *structure* (such as the k in k -nearest-neighbor) as well as model parameters (like betas in linear regression), a separate validation set is essential.

19.3. Understanding Overfitting and Underfitting

Recall that the goal of machine learning is to build models that generalize well to new, unseen data. Any model that is complex enough can perfectly model data given to it. However, achieving this balance between fitting the training data and maintaining good performance on *new examples* (ie., generalization) is the goal, not just performing well on the training data.

The concept of overfitting represents one of the central challenges in machine learning. When a model overfits, it learns the training data too well, memorizing specific examples rather than generalizing patterns. This results in excellent performance on training data but poor performance on new, unseen data.

Imagine teaching someone to recognize cats by showing them 100 photos. An overfitted learner might memorize every pixel of those specific photos rather than learning general features like whiskers, pointed ears, and fur patterns. When shown new cat photos, they would struggle because they focused on irrelevant details specific to the training images.

Underfitting represents the opposite problem. An underfitted model is too simple to capture the underlying patterns in the data. It performs poorly on both training and test data because it lacks the complexity needed to model the relationships present in the data. Continuing our cat recognition analogy, an underfitted model might only look at image brightness and miss all the important visual features that distinguish cats from other animals.

```
set.seed(123)
sinsim <- function(n,sd=0.1) {
  x <- seq(0,1,length.out=n)
  y <- sin(2*pi*x) + rnorm(n,0,sd)
  return(data.frame(x=x,y=y))
}
dat <- sinsim(100,0.25)
library(ggplot2)
library(patchwork)
p_base <- ggplot(dat,aes(x=x,y=y)) +
  geom_point(alpha=0.7) +
  theme_bw()
p_lm <- p_base +
  geom_smooth(method="lm", se=FALSE, alpha=0.6, formula = y ~ x)
p_lmsin <- p_base +
  geom_smooth(method="lm",formula=y~sin(2*pi*x), se=FALSE, alpha=0.6)
p_loess_wide <- p_base +
```

```
geom_smooth(method="loess",span=0.5, se=FALSE, alpha=0.6, formula = y ~ x)
p_loess_narrow <- p_base +
  geom_smooth(method="loess",span=0.05, se=FALSE, alpha=0.6, formula = y ~ x)
p_lm + p_lmsin + p_loess_wide + p_loess_narrow + plot_layout(ncol=2) +
  plot_annotation(tag_levels = 'A') &
  theme(plot.tag = element_text(size = 8))
```

Warning in simpleLoess(y, x, w, span, degree = degree, parametric = parametric,
: k-d tree limited by memory. ncmx= 200

Warning in sqrt(sum.squares/one.delta): NaNs produced

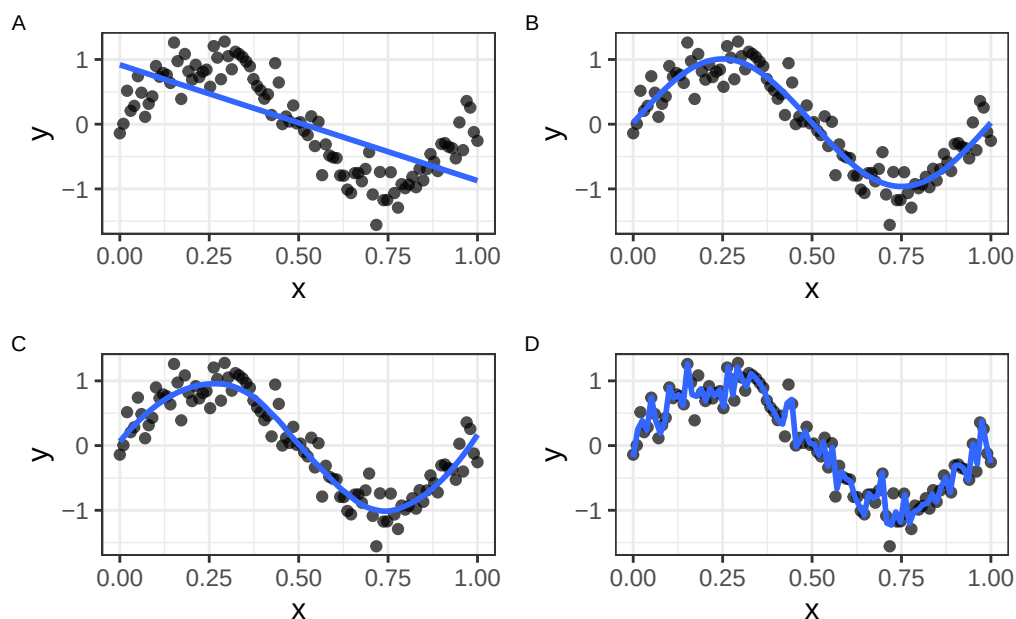


Figure 19.3.: Data simulated according to the function $f(x) = \sin(2\pi x) + N(0, 0.25)$ fitted with four different models. A) A simple linear model demonstrates *underfitting*. B) A linear model with a sin function ($y = \sin(2\pi x)$) and C) a loess model with a wide span (0.5) demonstrate *good fits*. D) A loess model with a narrow span (0.05) is a good example of *overfitting*.

In Figure 19.3, we simulate data according to the function $f(x) = \sin(2\pi x) + N(0, 0.25)$ and fit four different models. Choosing a model that is too simple (A) will result in *underfitting*

the data, while choosing a model that is too complex (D) will result in *overfitting* the data. For model (D), the loess model with a narrow span, the fitted line follows the noise in the data too closely, capturing random fluctuations rather than the underlying trend. In contrast, models (B) and (C) demonstrate good fits, capturing the essential pattern without being overly complex.

The relationship between model complexity and performance often follows a characteristic U-shaped curve. Very simple models underperform due to underfitting. As complexity increases, performance improves as the model captures more relevant patterns. However, beyond an optimal point, additional complexity leads to overfitting and degraded performance on new data.

Detecting Overfitting

Overfitting reveals itself through a growing gap between training and validation performance. If your model achieves 95% accuracy on training data but only 70% on validation data, you're likely overfitting.

Monitor both training and validation metrics throughout model development. The best models show similar performance on both sets, indicating good generalization.

Several strategies help combat overfitting. Regularization techniques add penalties for model complexity, encouraging simpler solutions. Cross-validation provides more robust estimates of model performance by training and validating on multiple data splits. Early stopping halts training when validation performance begins to degrade, even if training performance continues improving. Feature selection reduces the number of input variables, focusing the model on the most relevant information.

The bias-variance tradeoff provides a theoretical framework for understanding these phenomena. Bias refers to systematic errors due to overly simple models, while variance refers to sensitivity to small changes in training data. High-bias models underfit, while high-variance models overfit. The optimal model balances these competing sources of error.

19.4. Cross-Validation and Model Selection

Cross-validation addresses a fundamental challenge in machine learning: how do we reliably estimate model performance when we have limited data? Simple train-test splits can be misleading, especially with small datasets, because performance estimates depend heavily on which specific examples end up in each set.

K-fold cross-validation provides a more robust solution. The training data is divided into k equal-sized subsets (folds). The model is trained k times, each time using k-1 folds for

training and the remaining fold for validation. This process yields k performance estimates, which can be averaged to get a more stable assessment of model quality.

Five-fold and ten-fold cross-validation are common choices, providing good balance between computational efficiency and reliable estimates. Leave-one-out cross-validation represents an extreme case where k equals the number of training examples. While this maximizes the use of training data, it can be computationally expensive and may provide overly optimistic estimates for some types of models.

Stratified cross-validation ensures that each fold maintains the same proportion of examples from each class, which is particularly important for classification problems with imbalanced datasets. Time series data requires special consideration, as temporal order matters. Time series cross-validation uses only past data to predict future values, respecting the temporal structure.

Cross-validation serves multiple purposes in the model development process. It helps select the best algorithm from a set of candidates, tune hyperparameters within a chosen algorithm, and provide realistic estimates of expected performance on new data. However, it's crucial to remember that cross-validation results still come from the training data. Final model evaluation should always use a completely separate test set.

20. Supervised Learning

Imagine you have measured a few hundred penguins on a remote island — the length and depth of each bird’s bill, the length of its flipper. Now a new bird shows up. Can you predict its **body mass** from those measurements alone? Or guess which **species** it belongs to? That is exactly the kind of question supervised learning answers: you have examples where you know the answer, and you want a rule that predicts the answer for a new case you haven’t seen.

The same shape of problem shows up everywhere in biology. Predict a tumour’s response to a drug from its gene-expression profile. Predict a patient’s blood pressure from a panel of clinical measurements. Classify a cell as healthy or diseased from its morphology. In every case you have **features** (the inputs you measure) and a **target** (the answer you want to predict), and a pile of past examples to learn from.

In the [previous chapter](#) we met the three paradigms of machine learning and the all-important habit of splitting data to avoid fooling ourselves. This chapter is a guided tour of the workhorse **supervised** algorithms — linear regression, k-nearest neighbours, ridge/LASSO/elastic-net, decision trees, and random forests. For each one you’ll get a plain-language intuition and a small, runnable example on a single shared dataset, so you can see the algorithms as variations on one theme rather than a zoo of unrelated tricks. (The later [mlr3verse chapter](#) shows how to run all of these through one tidy interface; here we stay close to base R so the moving parts stay visible.)

20.1. What you’ll learn

- Describe any supervised algorithm in terms of its **hypothesis space**, **objective function**, and **learning algorithm**.
- Fit and interpret a **linear regression** with `lm()`.
- Make predictions with **k-nearest neighbours** using `class::knn()`, and explain why feature scaling matters.
- Fit **ridge**, **LASSO**, and **elastic-net** models with `glmnet()`, and read off which features each one keeps.
- Grow and read a **decision tree** with `rpart`, and a **random forest** with `ranger`.

20.2. A framework for understanding supervised algorithms

Every supervised algorithm in this chapter — and almost every one you’ll meet elsewhere — can be taken apart into the same three pieces. Once you can name those three pieces for an algorithm, you understand most of what makes it tick, and you can compare any two algorithms cleanly.

! The three pillars of a supervised algorithm

1. **Hypothesis space (the model family).** The set of all candidate models the algorithm is allowed to consider. For linear regression it’s “all straight lines”; for a decision tree it’s “all branching yes/no rule sets.” This choice bakes in an assumption — a *bias* — about what the answer can look like.
2. **Objective function (the goal).** A number that says how *good* a candidate model is on the training data — usually how badly it errs, so smaller is better. For regression that’s often the *mean squared error*. Learning means finding the model that makes this number as small as possible.
3. **Learning algorithm (the optimiser).** The actual procedure that searches the hypothesis space for the model with the best objective value. Sometimes it’s a one-shot formula (ordinary linear regression); sometimes it’s a long iterative grind (neural networks).

Here’s the payoff of this framing. Ridge and LASSO regression, which we’ll meet shortly, use the *same* hypothesis space and *same* style of optimiser as plain linear regression. The *only* thing they change is the objective function — they add a penalty for being too complicated. That one small change is what lets them avoid overfitting. Keep the three pillars in mind and each new algorithm becomes a small variation, not a whole new world.

20.3. Our shared dataset: Palmer penguins

To keep the tour concrete, we’ll use one small, friendly biological dataset throughout: measurements of penguins from three species on islands in the Palmer Archipelago, Antarctica. It lives in the `palmerpenguins` package.

```
library(palmerpenguins)
library(dplyr)

penguins_clean <- penguins |>
```

20. Supervised Learning

```
na.omit()          # drop the handful of rows with missing measurements

glimpse(penguins_clean)
```

```
Rows: 333
Columns: 8
$ species      <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adel~
$ island       <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgerse~
$ bill_length_mm <dbl> 39.1, 39.5, 40.3, 36.7, 39.3, 38.9, 39.2, 41.1, 38.6~
$ bill_depth_mm <dbl> 18.7, 17.4, 18.0, 19.3, 20.6, 17.8, 19.6, 17.6, 21.2~
$ flipper_length_mm <int> 181, 186, 195, 193, 190, 181, 195, 182, 191, 198, 18~
$ body_mass_g   <int> 3750, 3800, 3250, 3450, 3650, 3625, 4675, 3200, 3800~
$ sex          <fct> male, female, female, female, male, female, male, fe~
$ year         <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007~
```

Each row is one bird. We'll use two prediction problems throughout:

- **Regression:** predict `body_mass_g` (a continuous number) from the body measurements.
- **Classification:** predict `species` (one of three categories) from the same measurements.

As we insisted in the previous chapter, we never judge a model on the same data we trained it on. So let's split once, here, into a **training** set and a **test** set, and reuse that split for every algorithm.

```
set.seed(42)
n <- nrow(penguins_clean)
train_idx <- sample(seq_len(n), size = 0.7 * n) # 70% for training
train <- penguins_clean[train_idx, ]
test <- penguins_clean[-train_idx, ]
c(train = nrow(train), test = nrow(test))
```

```
train  test
233    100
```

We now have 233 birds to learn from and 100 held back to test on honestly.

20.4. Linear regression

Plain-language intuition. Linear regression draws the best straight-line relationship between your features and a continuous target. “Best” means the line that makes the total squared prediction error as small as possible. Each feature gets a *coefficient* — a slope — telling you how much the target changes when that feature goes up by one unit. It is the simplest, most interpretable model in the toolbox, and a sensible first thing to try.

In the language of the three pillars: the hypothesis space is “all straight-line combinations of the features,” the objective function is the sum of squared errors (Equation 20.1), and the learning algorithm is a tidy formula that R solves instantly.

$$L = \sum_{i=1}^n (\hat{y}_i - y_i)^2 \quad (20.1)$$

Here $\hat{y}_i$ is the predicted value, y_i is the true value, and n is the number of observations. Let’s predict penguin body mass from flipper length and bill dimensions:

```
fit_lm <- lm(body_mass_g ~ flipper_length_mm + bill_length_mm + bill_depth_mm,
             data = train)
coef(fit_lm)
```

(Intercept)	flipper_length_mm	bill_length_mm	bill_depth_mm
-6359.49030	50.52663	1.88153	18.08102

Read the coefficients as “holding the others fixed, one extra millimetre of flipper length is associated with about 51 grams more body mass.” That is the chief virtue of linear regression: the model *is* its explanation.

Now the honest test — how far off are the predictions on the held-out birds? We summarise that with the **root mean squared error** (RMSE), roughly the typical prediction error in grams:

```
pred_lm <- predict(fit_lm, newdata = test)
rmse <- function(actual, predicted) sqrt(mean((actual - predicted)^2))
rmse(test$body_mass_g, pred_lm)
```

```
[1] 387.8589
```

So the model’s predictions are typically off by around that many grams — for birds weighing 3,000–6,000 g, not bad for three measurements and a straight line.

20.5. K-nearest neighbours

Plain-language intuition. k-nearest neighbours (k-NN) is the “you are who your neighbours are” algorithm. To predict the answer for a new penguin, it finds the k most similar penguins in the training data — the ones closest to it in measurement space — and lets them vote. For classification, the new bird is assigned the most common species among its k neighbours. For regression, it gets the average of their values.

k-NN is unusual: it has no real “training” step and no equation to fit. It just stores the data and measures distances at prediction time. Its one knob is k : a small k (say 1) follows every wiggle in the data and can overfit; a large k smooths things out and can underfit. k is a **hyperparameter** you tune, exactly like we discussed in the previous chapter.

Scale your features first

k-NN decides “closeness” using distance, so a feature measured in large numbers will dominate one measured in small numbers — purely because of its units, not its importance. Flipper length (200 mm) would swamp bill depth (17 mm) if we left them as-is. The fix is to **standardise** every feature to a common scale before computing distances. Forgetting this is the single most common k-NN mistake.

Let’s classify penguin **species** from the three measurements. We standardise using the training set’s mean and standard deviation, then apply the *same* shift and scale to the test set:

```
library(class)

features <- c("flipper_length_mm", "bill_length_mm", "bill_depth_mm")

train_means <- colMeans(train[features])
train_sds <- apply(train[features], 2, sd)

scale_with <- function(df) scale(df[features], center = train_means, scale = train_sds)
train_x <- scale_with(train)
test_x <- scale_with(test)

set.seed(42)
knn_pred <- knn(train = train_x, test = test_x, cl = train$species, k = 5)

mean(knn_pred == test$species) # accuracy: fraction classified correctly
```

```
[1] 0.98
```

With $k = 5$ neighbours, k-NN gets the species right for the large majority of held-out birds. The three penguin species really do cluster into distinct regions of measurement space, which is exactly the situation where “ask your neighbours” works beautifully.

20.6. Penalized regression

Some material in this section is adapted from the [STHDA penalized-regression tutorial](#).

Plain linear regression has a weakness: when you have many features — especially more features than observations, the common situation in genomics — it can chase noise and overfit. **Penalized regression** fixes this by tweaking the *objective function* (pillar two). It keeps the squared-error term but adds a **penalty** that grows as the coefficients get large, gently pressuring the model to stay simple.

A single dial, **lambda** (λ), controls how hard that pressure pushes. When $\lambda = 0$ there’s no penalty and you’re back to ordinary linear regression. As λ grows, coefficients are squeezed toward zero. The three methods below — ridge, LASSO, and elastic net — differ only in the *shape* of the penalty.

We’ll fit all three with the `glmnet` package. `glmnet` does not take a formula; it wants a numeric **matrix** of features $\mathbf{x}$ and a response vector $\mathbf{y}$. The `model.matrix()` step below builds that matrix (and drops the intercept column with `[, -1]`):

```
library(glmnet)

x_train <- model.matrix(body_mass_g ~ flipper_length_mm + bill_length_mm +
                        bill_depth_mm, data = train)[, -1]
y_train <- train$body_mass_g

x_test <- model.matrix(body_mass_g ~ flipper_length_mm + bill_length_mm +
                      bill_depth_mm, data = test)[, -1]
y_test <- test$body_mass_g
```

20.6.1. Ridge regression

Plain-language intuition. Ridge adds an **L2 penalty**: the sum of the *squared* coefficients (Equation 20.2). This shrinks every coefficient toward zero but never quite *to* zero

20. Supervised Learning

— so ridge keeps all your features, just turns down their volume. It's the right choice when you believe many features each contribute a little.

$$L = \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^k \beta_j^2 \quad (20.2)$$

In `glmnet`, ridge is `alpha = 0`. We use `cv.glmnet()` to let cross-validation pick a good λ automatically:

```
set.seed(42)
fit_ridge <- cv.glmnet(x_train, y_train, alpha = 0)
coef(fit_ridge, s = "lambda.min")
```

```
4 x 1 sparse Matrix of class "dgCMatrix"
               lambda.min
(Intercept)   -4592.10682
flipper_length_mm    41.68954
bill_length_mm      13.28295
bill_depth_mm     -10.47499
```

Notice that all three coefficients are present and non-zero — ridge shrank them but kept them. Its test error:

```
pred_ridge <- predict(fit_ridge, newx = x_test, s = "lambda.min")
rmse(y_test, pred_ridge)
```

```
[1] 396.0116
```

20.6.2. LASSO regression

Plain-language intuition. LASSO (Least Absolute Shrinkage and Selection Operator) adds an **L1 penalty**: the sum of the *absolute* coefficients (Equation 20.3). The difference from ridge sounds small but matters a lot: the L1 penalty can push a coefficient *exactly* to zero, effectively deleting that feature from the model. So LASSO does **automatic feature selection** — it hands you a simpler model that uses only the features that earn their keep.

20. Supervised Learning

$$L = \sum_{i=1}^n (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^k |\beta_j| \quad (20.3)$$

LASSO is `alpha = 1` in `glmnet`:

```
set.seed(42)
fit_lasso <- cv.glmnet(x_train, y_train, alpha = 1)
coef(fit_lasso, s = "lambda.min")
```

```
4 x 1 sparse Matrix of class "dgCMatrix"
      lambda.min
(Intercept) -5499.365152
flipper_length_mm 47.897811
bill_length_mm 1.414023
bill_depth_mm .
```

Any coefficient shown as `.` has been zeroed out and dropped. With only three strong features here, LASSO may keep all of them — but on a genomics dataset with thousands of features, this is how you go from “20,000 genes” to “the handful that actually predict the outcome.”

💡 Ridge or LASSO? Let the data decide

Neither always wins. LASSO shines when a few features carry the signal and the rest are noise (so zeroing them helps). Ridge shines when *many* features each contribute a bit. When you’re unsure — which is most of the time — fit both with cross-validation and compare their test error, exactly as we’re doing here.

20.6.3. Elastic net

Plain-language intuition. Why choose? Elastic net **blends** the L1 and L2 penalties, so it can zero out useless features (the LASSO trick) *while* gracefully handling groups of correlated features (the ridge strength). When two features are highly correlated, pure LASSO tends to arbitrarily keep one and drop the other; elastic net is happier keeping both, which is common and useful in biology where genes in the same pathway move together.

20. Supervised Learning

The mixing dial is `alpha`, between 0 and 1: `alpha = 0` is pure ridge, `alpha = 1` is pure LASSO, and anything in between is elastic net. Let's use a 50/50 blend, `alpha = 0.5`:

```
set.seed(42)
fit_enet <- cv.glmnet(x_train, y_train, alpha = 0.5)

coef(fit_enet, s = "lambda.min")
```

```
4 x 1 sparse Matrix of class "dgCMatrix"
               lambda.min
(Intercept)    -6166.200623
flipper_length_mm  49.790369
bill_length_mm    2.389242
bill_depth_mm     14.153060
```

```
pred_enet <- predict(fit_enet, newx = x_test, s = "lambda.min")
rmse(y_test, pred_enet)
```

```
[1] 388.1342
```

Elastic net gives you a coefficient profile somewhere between ridge's “keep everything, shrunk” and LASSO's “keep a few.” On this small three-feature problem all four regression models land in roughly the same place; the penalized methods earn their keep when features are many and correlated.

20.7. Classification and regression trees (CART)

Plain-language intuition. A decision tree predicts by asking a series of simple yes/no questions about the features, like a flow chart or the game of Twenty Questions. “Is the flipper longer than 206 mm? If yes, is the bill deeper than 17 mm?” Each question splits the data into purer and purer groups, until a final **leaf** gives the prediction — the majority species (for classification) or the average value (for regression).

In three-pillar terms: the hypothesis space is “all such question trees,” the objective at each split is to make the resulting groups as *pure* as possible (measured by the Gini index or entropy), and the learning algorithm greedily picks the best split at each step. Trees are prized because you can *read* them — they match how people naturally reason.

We'll grow a classification tree for species with `rpart` (which ships with R):

20. Supervised Learning

```
library(rpart)

set.seed(42)
fit_tree <- rpart(species ~ flipper_length_mm + bill_length_mm + bill_depth_mm,
                  data = train, method = "class")

tree_pred <- predict(fit_tree, newdata = test, type = "class")
mean(tree_pred == test$species) # test accuracy
```

```
[1] 0.94
```

The tree learned a short list of measurement thresholds that separate the species, and it classifies the held-out birds with high accuracy. We can even print the learned rules:

```
fit_tree
```

```
n= 233
```

```
node), split, n, loss, yval, (yprob)
```

```
* denotes terminal node
```

```
1) root 233 131 Adelie (0.43776824 0.19742489 0.36480687)
  2) flipper_length_mm< 206.5 142 42 Adelie (0.70422535 0.29577465 0.00000000)
    4) bill_length_mm< 44.2 101 3 Adelie (0.97029703 0.02970297 0.00000000) *
    5) bill_length_mm>=44.2 41 2 Chinstrap (0.04878049 0.95121951 0.00000000) *
  3) flipper_length_mm>=206.5 91 6 Gentoo (0.02197802 0.04395604 0.93406593)
    6) bill_depth_mm>=17.2 7 3 Chinstrap (0.28571429 0.57142857 0.14285714) *
    7) bill_depth_mm< 17.2 84 0 Gentoo (0.00000000 0.00000000 1.00000000) *
```

Figure 20.1 shows the same idea drawn out for a clinical example: each internal node tests one feature, and following the branches leads to a leaf with a predicted class.

A single tree is twitchy

Decision trees have a real flaw: they're **unstable**. Change a few training rows and you can get a noticeably different tree. A single deep tree also overfits easily. The fix — averaging many trees together — is exactly the idea behind random forests, next.

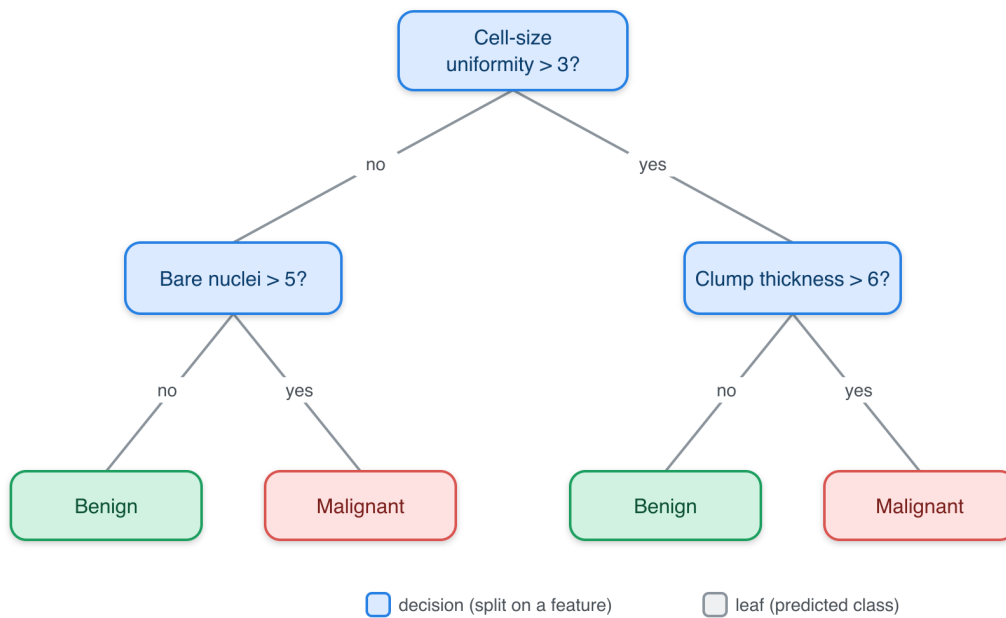


Figure 20.1.: A classification tree for a breast-cancer screening example. Each internal (blue) node tests one feature; following the branch that matches a sample leads, eventually, to a leaf (green/red) that gives the predicted class.

20.8. Random forests

Plain-language intuition. If one decision tree is twitchy, build *hundreds* of them and let them vote. A **random forest** grows many trees, each on a random bootstrap sample of the data and each allowed to consider only a random subset of features at every split. That deliberate randomness makes the individual trees disagree in their mistakes — and when you average many imperfect-but-different predictions, the errors cancel out. The result is far more stable and accurate than any single tree, while needing very little tuning. Figure 20.2 sketches the idea: many trees, one combined vote.

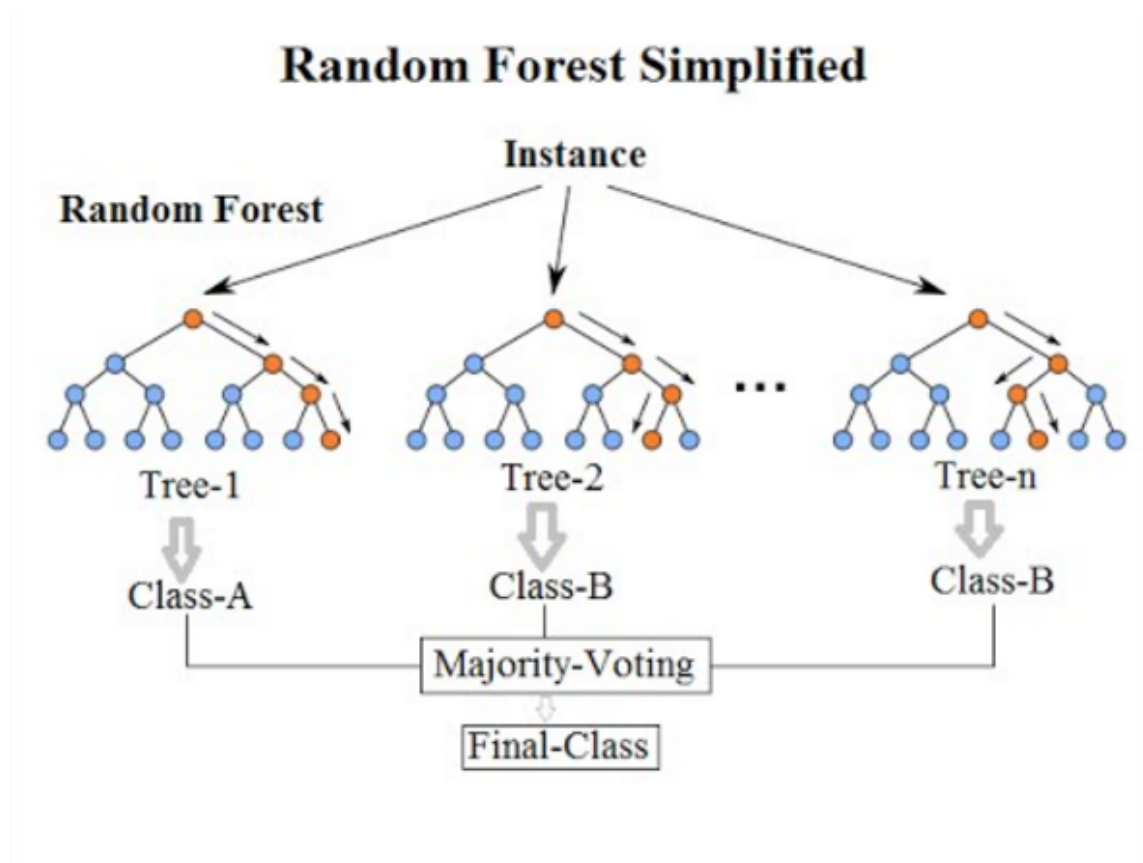


Figure 20.2.: Graphical representation of a random forest: many decision trees, each grown on a slightly different view of the data, combine their votes into one prediction.

We'll use the fast **ranger** package. Random forests are randomised, so we set a seed for a reproducible result:

```
library(ranger)

set.seed(42)
fit_rf <- ranger(species ~ flipper_length_mm + bill_length_mm + bill_depth_mm,
                 data = train)

rf_pred <- predict(fit_rf, data = test)$predictions
mean(rf_pred == test$species) # test accuracy
```

```
[1] 0.97
```

The forest typically matches or beats the single tree on the held-out birds, with no manual tuning — which is exactly why random forests are such a popular, reliable default for tabular biological data.

20.9. Summary

You now have a working map of the supervised-learning landscape, and you’ve run every algorithm on it with your own hands:

- Every algorithm is a **hypothesis space** + an **objective function** + a **learning algorithm**. Name those three and you understand it.
- **Linear regression** (`lm()`) fits an interpretable straight-line model; coefficients tell you how each feature moves the target.
- **k-nearest neighbours** (`class::knn()`) predicts from the most similar training examples — remember to **scale** your features first.
- **Penalized regression** (`glmnet()`) adds a complexity penalty to fight overfitting: **ridge** (`alpha = 0`) shrinks all coefficients, **LASSO** (`alpha = 1`) zeroes some out for feature selection, and **elastic net** (`0 < alpha < 1`) blends the two.
- **Decision trees** (`rpart`) are readable flow charts but unstable on their own; **random forests** (`ranger`) average many trees into a stable, accurate, low-fuss predictor.

In every case we trained on one split and judged on a held-out split — the habit that keeps you honest. The next chapters show how to run all of these through the unified `mlr3verse` interface, so swapping one algorithm for another becomes a one-line change.

20.10. Exercises

1. **A simpler linear model.** Refit the linear regression for `body_mass_g` using *only* `flipper_length_mm`. Compute its test RMSE and compare it to the three-feature model. Did dropping two features hurt much?

i Solution

```
fit_simple <- lm(body_mass_g ~ flipper_length_mm, data = train)
pred_simple <- predict(fit_simple, newdata = test)
rmse(test$body_mass_g, pred_simple)
```

```
[1] 389.7472
```

Flipper length alone is a strong predictor of body mass, so the simpler model is usually only a little worse. That's a useful reminder: more features are not automatically better.

2. **Tuning `k` in `k`-NN.** Try `k = 1`, `k = 5`, and `k = 25` for the species classifier. Which gives the best test accuracy? What goes wrong at the extremes?

i Solution

```
set.seed(42)
for (k in c(1, 5, 25)) {
  pred_k <- knn(train = train_x, test = test_x, cl = train$species, k = k)
  cat("k =", k, " accuracy =", round(mean(pred_k == test$species), 3), "\n")
}
```

```
k = 1  accuracy = 0.98
```

```
k = 5  accuracy = 0.98
```

```
k = 25 accuracy = 0.96
```

Very small `k` (like 1) follows noise and can overfit; very large `k` blurs the class boundaries and underfits. A middle value usually wins — which is why `k` is a hyperparameter worth tuning.

3. **Watch LASSO select.** Push the LASSO penalty up by using `lambda.1se` (a more aggressive, simpler model) instead of `lambda.min`, and look at the coefficients. Are any features dropped?

i Solution

```
coef(fit_lasso, s = "lambda.1se")
```

```
4 x 1 sparse Matrix of class "dgCMatrix"
```

```
              lambda.1se
(Intercept)  -4070.4745
flipper_length_mm  41.1108
bill_length_mm      .
bill_depth_mm      .
```

`lambda.1se` is the largest penalty whose error is still within one standard error of the best — a deliberately simpler model. Any coefficient printed as `.` has been set to exactly zero and removed. With strong features here it may keep them all, but the mechanism is the same one that prunes thousands of genes down to a few.

4. **Feature importance from the forest.** Refit the random forest with `importance = "impurity"` and print `fit_rf$variable.importance`. Which measurement matters most for telling the species apart?

i Solution

```
set.seed(42)
fit_rf_imp <- ranger(species ~ flipper_length_mm + bill_length_mm + bill_depth_mm,
                     data = train, importance = "impurity")
sort(fit_rf_imp$variable.importance, decreasing = TRUE)
```

```
      bill_length_mm flipper_length_mm      bill_depth_mm
           58.71576           47.88220           40.17413
```

The feature with the largest importance score contributed most to splitting the species apart across the forest's trees. Importance scores are one of the friendliest things random forests give you — a ranked list of which measurements carry the signal.

5. **Regression, three ways.** You have test RMSE for linear regression, ridge, and elastic net. Compute it for LASSO too, then state which model you'd ship and why.

i Solution

```
pred_lasso <- predict(fit_lasso, newx = x_test, s = "lambda.min")
c(linear = rmse(y_test, pred_lm),
  ridge  = rmse(y_test, pred_ridge),
  lasso  = rmse(y_test, pred_lasso),
  enet   = rmse(y_test, pred_enet))
```

```
linear    ridge    lasso    enet
387.8589  396.0116  390.8471  388.1342
```

On this small, low-dimensional problem the four are nearly tied, so you'd pick the simplest and most interpretable — plain linear regression. The penalized methods pull ahead only when features are numerous and correlated, as in real genomic data.

21. The mlr3verse

The R ecosystem offers numerous packages for machine learning, each with its own interface, conventions, and capabilities. While this diversity provides flexibility, it also creates challenges for practitioners who must learn different APIs for different algorithms and spend time on repetitive data preparation tasks. The mlr3verse addresses these challenges by providing a unified, consistent framework for machine learning in R.

21.1. The Philosophy of mlr3verse

The mlr3verse follows a modular design philosophy that separates different aspects of the machine learning workflow into distinct, interchangeable components. This separation of concerns makes it easier to experiment with different combinations of preprocessing steps, algorithms, and evaluation strategies without rewriting code.

Rather than monolithic functions that handle everything from data preprocessing to model evaluation, mlr3verse provides specialized objects for each component of the machine learning pipeline. Tasks define the problem and data, learners implement algorithms, measures specify evaluation metrics, and resamplings control validation strategies. This modular approach promotes code reusability and makes it easier to understand and maintain complex machine learning workflows.

The framework emphasizes object-oriented design, leveraging R6 classes to provide consistent interfaces across different components. This design ensures that similar operations work the same way regardless of which specific algorithm or evaluation metric you're using. Once you learn the basic patterns, you can easily work with new algorithms and techniques.

Reproducibility receives special attention in mlr3verse. All random operations can be controlled through seed settings, and the framework provides tools to track and reproduce experimental results. This focus on reproducibility is essential for scientific applications and helps practitioners debug and iterate on their models.

21.2. The *mlr3*verse Ecosystem

The *mlr3*verse consists of several interconnected packages, each focused on specific aspects of machine learning. Understanding these components helps practitioners choose the right tools for their projects and leverage the full power of the ecosystem.

Package	Purpose	Key Features
<i>mlr3</i>	Core framework	Tasks, learners, measures, resampling
<i>mlr3learners</i>	Extended algorithms	Additional ML algorithms beyond base <i>mlr3</i>
<i>mlr3pipelines</i>	Preprocessing & pipelines	Feature engineering, model stacking
<i>mlr3tuning</i>	Hyperparameter optimization	Grid search, random search, Bayesian optimization
<i>mlr3measures</i>	Additional metrics	Extended evaluation measures
<i>mlr3viz</i>	Visualization	Plotting utilities for models and results
<i>mlr3filters</i>	Feature selection	Filter-based feature selection methods

The core *mlr3* package provides the foundation, implementing basic tasks, learners, and evaluation procedures. It includes common algorithms like linear regression, logistic regression, and decision trees, along with fundamental evaluation metrics and resampling strategies.

mlr3learners extends the available algorithms significantly, providing interfaces to popular R packages like *randomForest*, *glmnet*, and *xgboost*. This extension allows practitioners to access state-of-the-art algorithms while maintaining the consistent *mlr3* interface.

mlr3pipelines introduces powerful preprocessing and model composition capabilities. It allows users to chain together multiple preprocessing steps, combine different algorithms, and create complex model architectures. This package particularly shines in scenarios requiring sophisticated feature engineering or ensemble methods.

mlr3tuning automates the search for optimal hyperparameters, implementing various optimization strategies from simple grid search to sophisticated Bayesian optimization. Hyperparameter tuning can dramatically improve model performance, but it requires careful validation to avoid overfitting.

21.3. Core *mlr3* Concepts and Objects

The *mlr3* framework organizes machine learning workflows around four fundamental object types:

- **Tasks** provide the data and problem definition.
- **Learners** implement the algorithms.
- **Measures** evaluate performance.
- **Resamplings** ensure robust validation.

Understanding these objects and how they interact forms the foundation for effective use of the *mlr3*verse. The goal is to provide a consistent, reusable interface for machine learning tasks, allowing practitioners to focus on solving problems rather than learning different APIs.

Tasks encapsulate the machine learning problem, combining data with metadata about the prediction target and feature types. Classification tasks specify categorical targets, while regression tasks involve continuous targets. Tasks handle many routine data management operations automatically, such as identifying feature types and managing factor levels. They also provide methods for data manipulation, including filtering rows, selecting features, and creating subsets.

Loading required package: *mlr3*

```
# Create a classification task with Palmer Penguins data
task <- as_task_classif(penguins, target = "species")

# Examine task properties
task$nrow # Number of observations
```

```
[1] 333
```

```
task$ncol # Number of features
```

```
[1] 8
```

```
task$feature_names # Names of predictor variables
```

```
[1] "bill_depth_mm"      "bill_length_mm"    "body_mass_g"
[4] "flipper_length_mm" "island"             "sex"
[7] "year"
```

```
task$target_names # Name of target variable
```

```
[1] "species"
```

Learners implement machine learning algorithms, providing a consistent interface regardless of the underlying implementation. Each learner specifies its capabilities, including which task types it supports, whether it can handle missing values, and what hyperparameters are available for tuning. Learners maintain information about their training state and can generate predictions on new data.

The learner registry provides a convenient way to discover available algorithms. You can query the registry to find learners for specific task types or search for algorithms with particular capabilities.

```
# Explore available classification learners
classif_learners <- mlr_learners$keys("classif")
head(classif_learners, 10)
```

```
[1] "classif.cv_glmnet"    "classif.debug"      "classif.featureless"
[4] "classif.glmnet"      "classif.kknn"       "classif.lda"
[7] "classif.log_reg"     "classif.multinom"   "classif.naive_bayes"
[10] "classif.nnet"
```

```
# Examine a specific learner
rpart_learner <- lrn("classif.rpart")
rpart_learner$param_set$ids() # Available hyperparameters
```

```
[1] "cp"                "keep_model"         "maxcompete"         "maxdepth"
[5] "maxsurrogate"      "minbucket"          "minsplit"           "surrogatestyle"
[9] "usesurrogate"      "xval"
```

i Learner Naming Convention

mlr3 uses a consistent naming scheme for learners: `[task_type].[algorithm]`. For example, `classif.rpart` implements decision trees for classification, while `regr.lm` provides linear regression. This naming makes it easy to find appropriate algorithms for your task type.

Measures define evaluation metrics for assessing model performance. Different measures emphasize different aspects of model quality, and the choice of measure can significantly impact model selection and hyperparameter tuning. Classification measures include accuracy, precision, recall, and F1-score, while regression measures encompass mean squared error, mean absolute error, and R-squared.

```
# Common classification measures
acc_measure <- msr("classif.acc") # Accuracy
ce_measure  <- msr("classif.ce")  # Classification error
auc_measure <- msr("classif.auc") # Area under ROC curve
```

Resamplings control how data is split for training and validation. They implement various strategies including simple holdout splits, k-fold cross-validation, and bootstrap sampling. The choice of resampling strategy affects both the reliability of performance estimates and computational requirements.

```
# Different resampling strategies
holdout <- rsmp("holdout", ratio = 0.8) # 80/20 split
cv5 <- rsmp("cv", folds = 5)           # 5-fold cross-validation
bootstrap <- rsmp("bootstrap", repeats = 30) # Bootstrap sampling

# Examine resampling properties
cv5$param_set
```

```
<ParamSet(1)>
      id   class lower upper nlevels      default  value
<char> <char> <int> <num>   <num>      <list> <list>
1: folds ParamInt      2   Inf     Inf <NoDefault[0]>      5
```

These four object types work together to create complete machine learning workflows. Tasks provide the data and problem definition, learners implement the algorithms, measures evaluate performance, and resamplings ensure robust validation. This modular design allows practitioners to mix and match components easily, experimenting with different combinations to find optimal solutions for their specific problems.

The interaction between these objects follows predictable patterns. Learners train on tasks, producing fitted models that can generate predictions. These predictions are evaluated using measures, with resampling strategies ensuring that evaluation results generalize beyond the specific training data used. Understanding these interactions enables practitioners to build sophisticated machine learning pipelines while maintaining clear, readable code.

! Key Advantages of *mlr3*verse

The *mlr3*verse provides several crucial advantages for machine learning practitioners:

Consistency: All algorithms use the same interface, reducing learning overhead and making code more maintainable.

Flexibility: Modular design allows easy experimentation with different combinations of preprocessing, algorithms, and evaluation strategies.

Reproducibility: Built-in support for controlling randomness and tracking experimental results.

Extensibility: Easy to add new algorithms, measures, and preprocessing steps while maintaining compatibility with existing code.

Best Practices: Framework encourages proper practices like train/test splitting and cross-validation through its core design.

This foundation in *mlr3*verse concepts prepares practitioners to tackle real machine learning problems with confidence. The consistent interfaces and modular design reduce the cognitive load associated with learning new algorithms, allowing focus on the more important aspects of problem-solving: understanding the data, choosing appropriate methods, and interpreting results. In the hands-on sections that follow, these concepts will come together to demonstrate how *mlr3*verse facilitates efficient, robust machine learning workflows.

22. Examples

22.1. Overview

In this chapter, we focus on practical aspects of machine learning. The goal is to provide a hands-on introduction to the application of machine learning techniques to real-world data. While the theoretical foundations of machine learning are important, the ability to apply these techniques to solve practical problems is equally crucial. In this chapter, we will use the `mlr3` package in R to build and evaluate machine learning models for classification and regression tasks.

We will use three examples to illustrate the machine learning workflow:

1. **Cancer types classification:** We will classify different types of cancer based on gene expression data.
2. **Age prediction from DNA methylation:** We will predict the chronological age of individuals based on DNA methylation patterns.
3. **Gene expression prediction:** We will predict gene expression levels based on histone modification data.

We'll be applying knn, decision trees, and random forests, linear regression, and penalized regression models to these datasets.

The `mlr3` R package is a modern, object-oriented machine learning framework in R that builds on the success of its predecessor, the `mlr` package. It provides a flexible and extensible platform for handling common machine learning tasks such as data preprocessing, model training, hyperparameter tuning, and model evaluation Figure 22.1. The package is designed to guide and standardize the process of using complex machine learning pipelines.

22.1.1. Key features of `mlr3`

- **Task abstraction** `mlr3` encapsulates different types of learning problems like classification, regression, and survival analysis into “Task” objects, making it easier

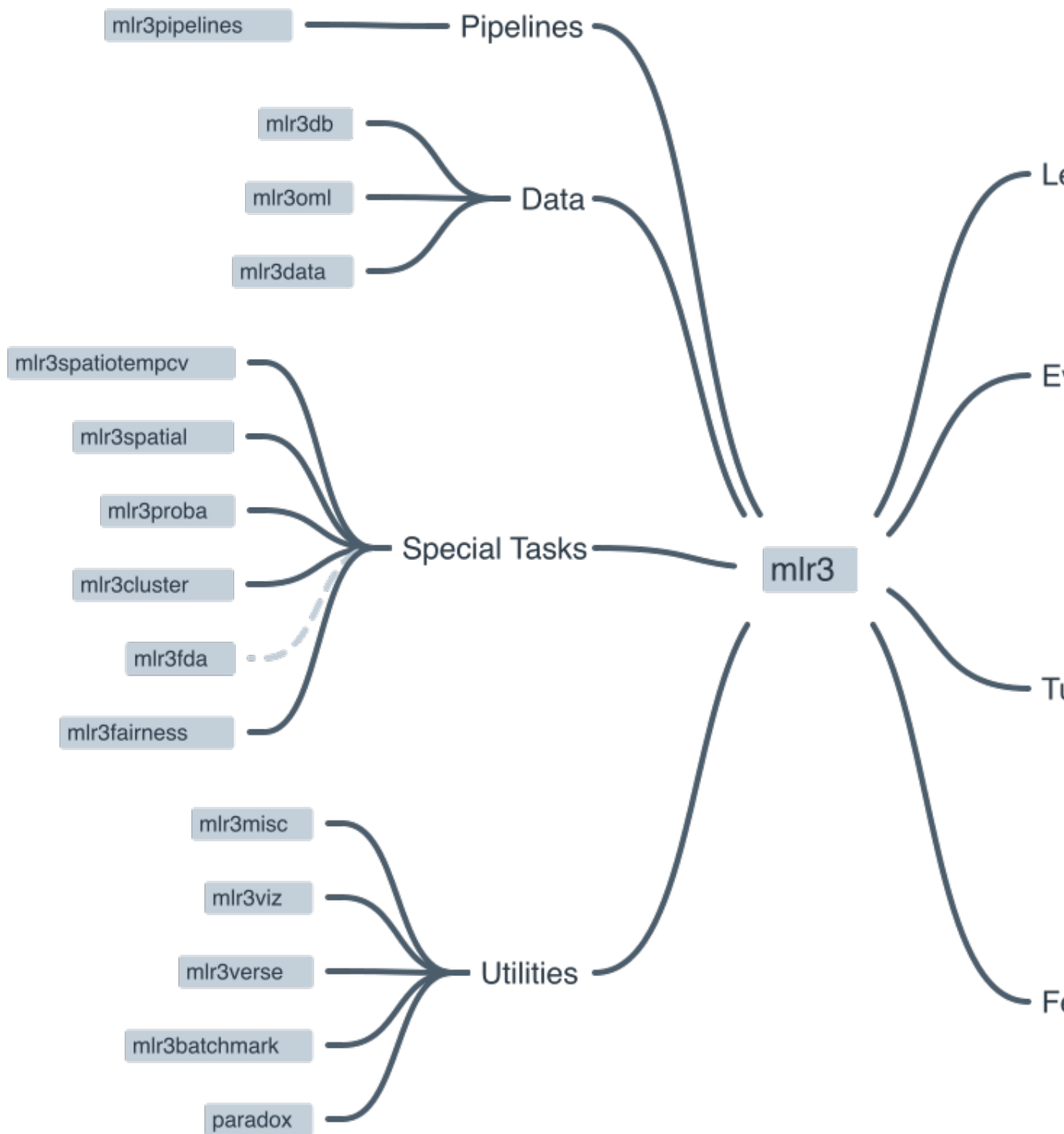


Figure 22.1.: The mlr3 ecosystem.

22. Examples

to handle various learning scenarios. Examples of tasks include classification tasks, regression tasks, and survival tasks.

- **Modular design** The package follows a modular design, allowing users to quickly swap out different components such as learners (algorithms), measures (performance metrics), and resampling strategies. Examples of learners include linear regression, logistic regression, and random forests. Examples of measures include accuracy, precision, recall, and F1 score. Examples of resampling strategies include cross-validation, bootstrapping, and holdout validation.
- **Extensibility** Users can extend the functionality of `mlr3` by adding custom components like learners, measures, and preprocessing steps via the R6 object-oriented system.
- **Preprocessing** `mlr3` provides a flexible way to preprocess data using “PipeOps” (pipeline operations), allowing users to create reusable preprocessing pipelines.
- **Tuning and model selection** `mlr3` supports hyperparameter tuning and model selection using various search strategies like grid search, random search, and Bayesian optimization.
- **Parallelization** The package allows for parallelization of model training and evaluation, making it suitable for large-scale machine learning tasks.
- **Benchmarking** `mlr3` facilitates benchmarking of multiple algorithms on multiple tasks, simplifying the process of comparing and selecting the best models.

You can find more information, including tutorials and examples, on the official `mlr3` GitHub repository¹ and the `mlr3` book².

22.2. The `mlr3` workflow

The `mlr3` package is designed to simplify the process of creating and deploying complex machine learning pipelines. The package follows a modular design, which means that users can quickly swap out different components such as learners (algorithms), measures (performance metrics), and resampling strategies. The package also supports parallelization of model training and evaluation, making it suitable for large-scale machine learning tasks.

The following sections describe each of these steps in detail.

¹<https://github.com/mlr-org/mlr3>

²<https://mlr3book.ml-org.com/>

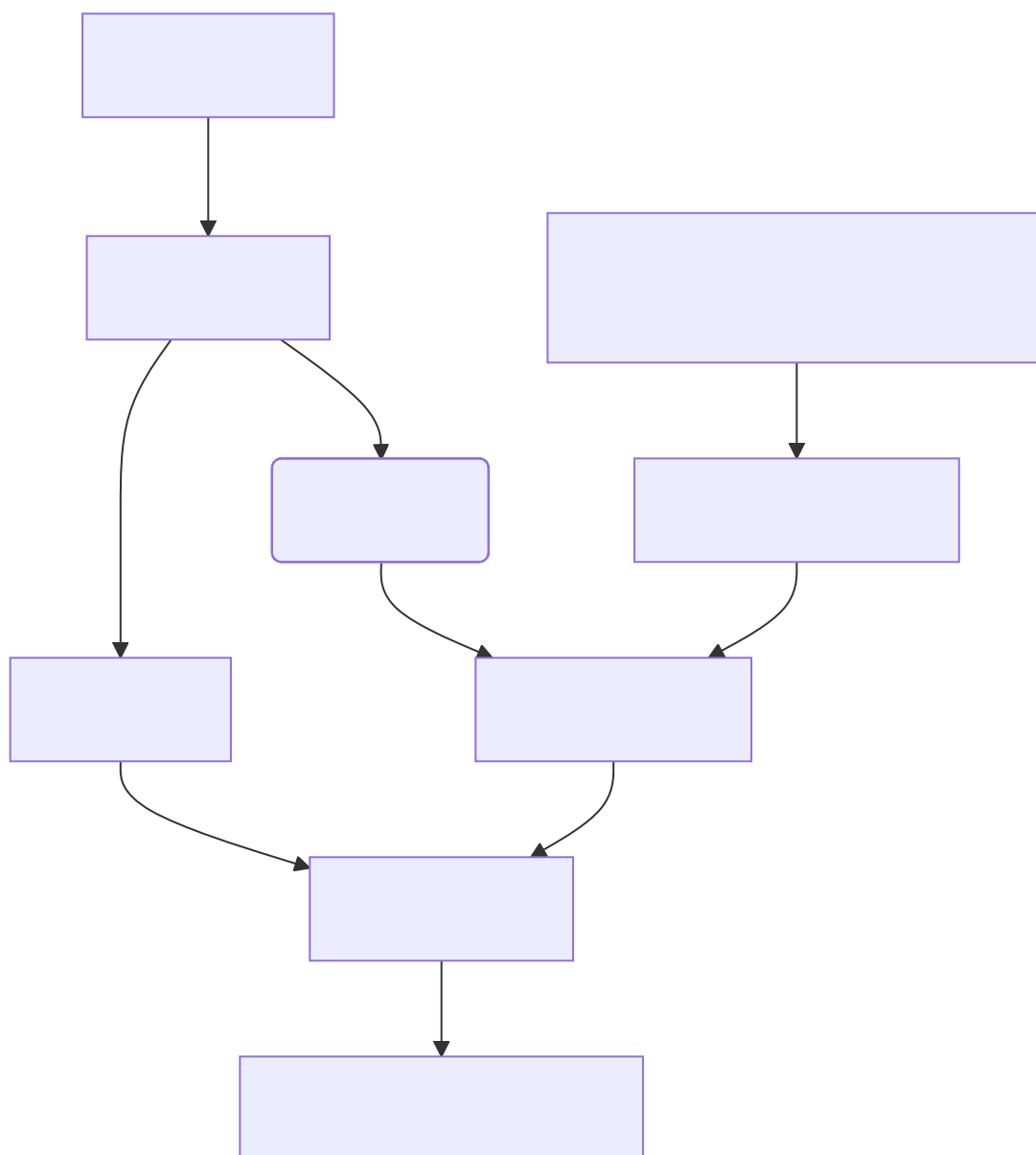


Figure 22.2.: The simplified workflow of a machine learning pipeline using mlr3.

22.2.1. The machine learning Task

Imagine you want to teach a computer how to make predictions or decisions, similar to how you might teach a student. To do this, you need to clearly define what you want the computer to learn and work on. This is called defining a “task.” Let’s break down what this involves and why it’s important.

22.2.1.1. Step 1: Understand the Problem

First, think about what problem you want to solve or what question you want the computer to answer. For example: - Do you want to predict the weather for tomorrow? - Are you trying to figure out if an email is spam or not? - Do you want to know how much a house might sell for?

These questions define your **task type**. In machine learning, there are several common task types:

- **Classification:** Deciding which category something belongs to (e.g., spam or not spam).
- **Regression:** Predicting a number (e.g., the price of a house).
- **Clustering:** Grouping similar items together (e.g., customer segmentation).

22.2.1.2. Step 2: Choose Your Data

Next, you need data that is related to your problem. Think of data as the information or examples you’ll use to teach the computer. For instance, if your task is to predict house prices, your data might include:

- The size of the house
- The number of bedrooms
- The location of the house
- The age of the house

These pieces of information are called **features**. Features are the input that the computer uses to make predictions.

22.2.1.3. Step 3: Define the Target

Along with features, you need to define the **target**. The target is what you want to predict or decide. In the house price example, the target would be the actual price of the house.

22.2.1.4. Step 4: Create the Task

Now that you have your problem, data, and target, you can create the task. In `mlr3`, a task brings together the type of problem (task type), the features (input data), and the target (what you want to predict).

Here's a simple summary:

1. **Task Type:** What kind of problem are you solving? (e.g., classification, regression)
2. **Features:** What information do you have to make the prediction? (e.g., size, location)
3. **Target:** What are you trying to predict? (e.g., house price)

By clearly defining these elements, you set a solid foundation for the machine learning process. This helps ensure that the computer can learn effectively and make accurate predictions.

22.2.1.5. `mlr3` and Tasks

The `mlr3` package uses the concept of “Tasks” to encapsulate different types of learning problems like classification, regression, and survival analysis. A Task contains the data (features and target variable) and additional metadata to define the machine learning problem. For example, in a classification task, the target variable is a label (stored as a character or factor), while in a regression task, the target variable is a numeric quantity (stored as an integer or numeric).

There are a number of [Task Types](#) that are supported by `mlr3`. To create a task from a `data.frame()`, `data.table()` or `Matrix()`, you first need to select the right task type:

- **Classification Task:** The target is a label (stored as `character` or `factor`) with only relatively few distinct values → [TaskClassif](#).
- **Regression Task:** The target is a numeric quantity (stored as `integer` or `numeric`) → [TaskRegr](#).
- **Survival Task:** The target is the (right-censored) time to an event. More censoring types are currently in development → [mlr3proba::TaskSurv](#) in add-on package [mlr3proba](#).
- **Density Task:** An unsupervised task to estimate the density → [mlr3proba::TaskDens](#) in add-on package [mlr3proba](#).

- **Cluster Task:** An unsupervised task type; there is no target and the aim is to identify similar groups within the feature space → `mlr3cluster::TaskClust` in add-on package `mlr3cluster`.
- **Spatial Task:** Observations in the task have spatio-temporal information (e.g. coordinates) → `mlr3spatiotempcv::TaskRegrST` or `mlr3spatiotempcv::TaskClassifST` in add-on package `mlr3spatiotempcv`.
- **Ordinal Regression Task:** The target is ordinal → `TaskOrdinal` in add-on package `mlr3ordinal` (still in development).

22.2.2. The “Learner” in Machine Learning

After you’ve defined your task, the next step in teaching a computer to make predictions or decisions is to choose a “learner.” Let’s explore what a learner is and how it fits into the `mlr3` package.

22.2.2.1. What is a “Learner”?

Think of a learner as the method or tool that the computer uses to learn from the data. Another common name for a “learner” is a “model.” It’s similar to choosing a tutor or a teacher for a student. Different learners have different ways of understanding and processing information. For example:

- Some learners might be great at recognizing patterns in data, like a tutor who is excellent at spotting trends.
- Others might be good at making decisions based on rules, like a tutor who uses step-by-step logic.

In machine learning, there are many types of learners, each with its own strengths and weaknesses. Here are a few examples:

- **Decision Trees:** These learners make decisions by asking a series of questions, like “Is the house larger than 1000 square feet?” and “Does it have more than 3 bedrooms?”
- **k-Nearest Neighbors:** These learners make predictions based on the similarity of new data points to existing data points.
- **Linear Regression:** This learner tries to fit a straight line through the data points to make predictions about numbers.
- **Random Forests:** These are like a group of decision trees working together to make more accurate predictions.

22. Examples

- **Support Vector Machines:** These learners find the best boundary that separates different categories in the data.

22.2.2.2. Choosing the Right Learner

Selecting the right learner is crucial because different learners work better for different types of tasks and data. For example:

- If your task is to classify emails as spam or not spam, a decision tree or a support vector machine might work well.
- If you're predicting house prices, linear regression or random forests could be good choices.

The goal is to find a learner that can understand the patterns in your data and make accurate predictions. This is where the `mlr3` package comes in handy. It provides a wide range of learners that you can choose from, making it easier to experiment and find the best learner for your task.

22.2.2.3. Learners in `mlr3`

In the `mlr3` package, learners are pre-built tools that you can easily use for your tasks. Here's how it works:

1. **Select a Learner:** `mlr3` provides a variety of learners to choose from, like decision trees, linear regression, and more.
2. **Train the Learner:** Once you've selected a learner, you provide it with your task (the problem, data, and target). The learner uses this information to learn and make predictions.
3. **Evaluate and Improve:** After training, you can test how well the learner performs and make adjustments if needed, such as trying a different learner or fine-tuning the current one.

22.2.2.4. `mlr3` and Learners

Objects of class `Learner` provide a unified interface to many popular machine learning algorithms in R. They consist of methods to train and predict a model for a `Task` and provide meta-information about the learners, such as the hyperparameters (which control the behavior of the learner) you can set.

22. Examples

The base class of each learner is `Learner`, specialized for regression as `LearnerRegr` and for classification as `LearnerClassif`. Other types of learners, provided by extension packages, also inherit from the `Learner` base class, e.g. `mlr3proba::LearnerSurv` or `mlr3cluster::LearnerClust`.

All Learners work in a two-stage procedure:

- **Training stage:** The training data (features and target) is passed to the Learner's `$train()` function which trains and stores a model, i.e. the relationship of the target and features.
- **Predict stage:** The new data, usually a different slice of the original data than used for training, is passed to the `$predict()` method of the Learner. The model trained in the first step is used to predict the missing target, e.g. labels for classification problems or the numerical value for regression problems.

There are a number of [predefined learners](#). The `mlr3` package ships with the following set of classification and regression learners. We deliberately keep this small to avoid unnecessary dependencies:

- `classif.featureless`: Simple baseline classification learner. The default is to always predict the label that is most frequent in the training set. While this is not very useful by itself, it can be used as a “[fallback learner](#)” to make predictions in case another, more sophisticated, learner failed for some reason.
- `regr.featureless`: Simple baseline regression learner. The default is to always predict the mean of the target in training set. Similar to `mlr_learners_classif.featureless`, it makes for a good “[fallback learner](#)”
- `classif.rpart`: Single classification tree from package `rpart`.
- `regr.rpart`: Single regression tree from package `rpart`.

This set of baseline learners is usually insufficient for a real data analysis. Thus, we have cherry-picked implementations of the most popular machine learning method and collected them in the `mlr3learners` package:

- Linear (`regr.lm`) and logistic (`classif.log_reg`) regression
- Penalized Generalized Linear Models (`regr.glmnet`, `classif.glmnet`), possibly with built-in optimization of the penalization parameter (`regr.cv_glmnet`, `classif.cv_glmnet`)
- (Kernelized) k-Nearest Neighbors regression (`regr.kknn`) and classification (`classif.kknn`).
- Kriging / Gaussian Process Regression (`regr.km`)
- Linear (`classif.lda`) and Quadratic (`classif.qda`) Discriminant Analysis
- Naive Bayes Classification (`classif.naive_bayes`)
- Support-Vector machines (`regr.svm`, `classif.svm`)
- Gradient Boosting (`regr.xgboost`, `classif.xgboost`)

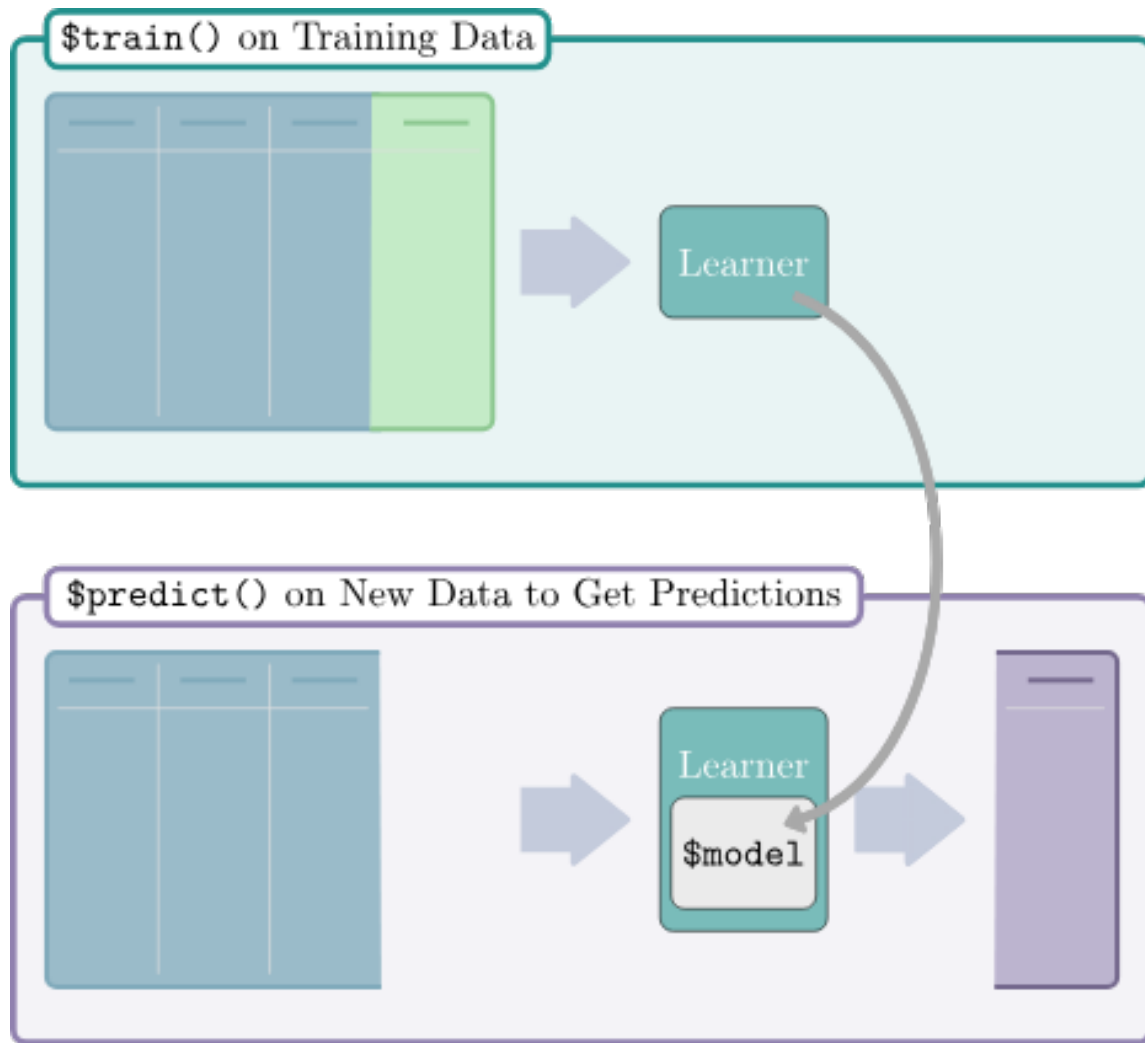


Figure 22.3.: Two stages of a learner. Top: data (features and a target) are passed to an (untrained) learner. Bottom: new data are passed to the trained model which makes predictions for the ‘missing’ target column.

22. Examples

- Random Forests for regression and classification (`regr.ranger`, `classif.ranger`)

More machine learning methods and alternative implementations are collected in the [mlr3extralearners repository](#).

22.3. Setup

```
library(mlr3verse)
library(GEOquery)
library(mlr3learners) # for knn
library(ranger) # for randomforest
set.seed(789)
```

22.4. Example: Cancer types

In this exercise, we will be classifying cancer types based on gene expression data. The data we are going to access are from Brouwer-Visser et al. (2018).

The data are from the Gene Expression Omnibus (GEO) database, a public repository of functional genomics data. The data are from a study that aimed to identify gene expression signatures that can distinguish between different types of cancer. The data include gene expression profiles from patients with different types of cancer. The goal is to build a machine learning model that can predict the cancer type based on the gene expression data.

22.4.1. Understanding the Problem

Before we start building a machine learning model, it's important to understand the problem we are trying to solve. Here are some key questions to consider:

- What are the features?
- What is the target variable?
- What type of machine learning task is this (classification, regression, clustering)?
- What is the goal of the analysis?

22.4.2. Data Preparation

Use the [GEOquery](#) package to fetch data about [GSE103512](#).

```
library(GEOquery)
gse = getGEO("GSE103512")[[1]]
```

The first step, a detail, is to convert from the older Bioconductor data structure (GEOquery was written in 2007), the `ExpressionSet`, to the newer `SummarizedExperiment`.

```
library(SummarizedExperiment)
se = as(gse, "SummarizedExperiment")
```

Examine two variables of interest, cancer type and tumor/normal status.

```
with(colData(se), table(`cancer.type.ch1`, `normal.ch1`))
```

	normal.ch1	
cancer.type.ch1	no	yes
BC	65	10
CRC	57	12
NSCLC	60	9
PCA	60	7

Before embarking on a machine learning analysis, we need to make sure that we understand the data. Things like missing values, outliers, and other problems can cause problems for machine learning algorithms.

In R, plotting, summaries, and other exploratory data analysis tools are available. PCA analysis, clustering, and other methods can also be used to understand the data. It is worth spending time on this step, as it can save time later.

22.4.3. Feature selection and data cleaning

While we could use all genes in the analysis, we will select the most informative genes using the variance of gene expression across samples. Other methods for feature selection are available, including those based on correlation with the outcome variable.

! Feature selection

Feature selection should be done on the training data only, not the test data to avoid overfitting. The test data should be used only for evaluation. In other words, the test data should be “unseen” by the model until the final evaluation.

Remember that the `apply` function applies a function to each row or column of a matrix. Here, we apply the `sd` function to each row of the expression matrix to get a vector of stan

```
sds = apply(assay(se, 'exprs'), 1, sd)
## filter out normal tissues
se_small = se[order(sds, decreasing = TRUE)[1:200],
              colData(se)$characteristics_ch1.1=='normal: no']
# remove genes with no gene symbol
se_small = se_small[rowData(se_small)$Gene.Symbol!='',]
```

To make the data easier to work with, we will use the opportunity to use one of the `rowData` columns as the rownames of the data frame. The `make.names` function is used to make sure that the rownames are valid R variable names and unique.

```
## convert to matrix for later use
dat = assay(se_small, 'exprs')
rownames(dat) = make.names(rowData(se_small)$Gene.Symbol)
```

We also need to transpose the data so that the rows are the samples and the columns are the features in order to use the data with `mlr3`.

```
feat_dat = t(dat)
tumor = data.frame(tumor_type = colData(se_small)$cancer.type.ch1, feat_dat)
```

This is another good time to check the data. Make sure that the data is in the format that you expect. Check the dimensions, the column names, and the data types.

22.4.4. Creating the “task”

The first step in using `mlr3` is to create a `task`. A task is a data set with a target variable. In this case, the target variable is the cancer type. The `mlr3` package provides a function

22. Examples

to convert a data frame into a task. These tasks can be used with any machine learning algorithm in `mlr3`.

This is a classification task, so we will use the `as_task_classif` function to create the task. The classification task requires a target variable that is categorical.

```
library(mlr3)
tumor$tumor_type = as.factor(tumor$tumor_type)
task = as_task_classif(tumor, target='tumor_type')
```

22.4.5. Splitting the data

Here, we randomly divide the data into 2/3 training data and 1/3 test data. This is a common split, but other splits can be used. The training data is used to train the model, and the test data is used to evaluate the trained model.

```
set.seed(7)
train_set = sample(task$row_ids, 0.67 * task$nrow)
test_set = setdiff(task$row_ids, train_set)
```

! Important

Training and testing on the same data is a common mistake. We want to test the model on data that it has not seen before. This is the only way to know if the model is overfitting and to get an accurate estimate of the model's performance.

In the next sections, we will train and evaluate three different models on the data: k-nearest-neighbor, classification tree, and random forest. Remember that the goal is to predict the cancer type based on the gene expression data. The `mlr3` package uses the concept of “learners” to encapsulate different machine learning algorithms.

22.4.6. Example learners

22.4.6.1. K-nearest-neighbor

The first model we will use is the k-nearest-neighbor model. This model is based on the idea that similar samples have similar outcomes. The number of neighbors to use is a parameter that can be tuned. We'll use the default value of 7, but you can try other values to see how they affect the results. In fact, `mlr3` provides the ability to tune parameters automatically, but we won't cover that here.

22.4.6.1.1. Create the learner

In `mlr3`, the machine learning algorithms are called learners. To create a learner, we use the `lrn` function. The `lrn` function takes the name of the learner as an argument. The `lrn` function also takes other arguments that are specific to the learner. In this case, we will use the default values for the arguments.

```
library(mlr3learners)
learner = lrn("classif.kknn")
```

You can get a list of all the learners available in `mlr3` by using the `lrn()` function without any arguments.

```
lrn()
```

<DictionaryLearner> with 55 stored values

Keys: `classif.cv_glmnet`, `classif.debug`, `classif.featureless`,
`classif.glmnet`, `classif.kknn`, `classif.lda`, `classif.log_reg`,
`classif.multinom`, `classif.naive_bayes`, `classif.nnet`, `classif.qda`,
`classif.ranger`, `classif.rpart`, `classif.svm`, `classif.xgboost`,
`clust.agnes`, `clust.ap`, `clust.bico`, `clust.birch`, `clust.clara`,
`clust.cmeans`, `clust.cobweb`, `clust.dbscan`, `clust.dbscan_fpc`,
`clust.diana`, `clust.em`, `clust.fanny`, `clust.featureless`, `clust.ff`,
`clust.hclust`, `clust.hdbscan`, `clust.kkmeans`, `clust.kmeans`,
`clust.kproto`, `clust.MBatchKMeans`, `clust.mclust`, `clust.meanshift`,
`clust.optics`, `clust.pam`, `clust.protoclust`, `clust.SimpleKMeans`,
`clust.specc`, `clust.xmeans`, `regr.cv_glmnet`, `regr.debug`,
`regr.featureless`, `regr.glmnet`, `regr.kknn`, `regr.km`, `regr.lm`,
`regr.nnet`, `regr.ranger`, `regr.rpart`, `regr.svm`, `regr.xgboost`

22.4.6.1.2. Train

To train the model, we use the `train` function. The `train` function takes the task and the row ids of the training data as arguments.

```
learner$train(task, row_ids = train_set)
```

Here, we can look at the trained model:

22. Examples

```
# output is large, so do this on your own
learner$model
```

22.4.6.1.3. Predict

Lets use our trained model works to predict the classes of the **training** data. Of course, we already know the classes of the training data, but this is a good way to check that the model is working as expected. It also gives us a measure of performance on the training data that we can compare to the test data to look for overfitting.

```
pred_train = learner$predict(task, row_ids=train_set)
```

And check on the test data:

```
pred_test = learner$predict(task, row_ids=test_set)
```

22.4.6.1.4. Assess

In this section, we can look at the accuracy and performance of our model on the training data and the test data. We can also look at the confusion matrix to see which classes are being confused with each other.

```
pred_train$confusion
```

	truth			
response	BC	CRC	NSCLC	PCA
BC	42	0	0	0
CRC	0	40	0	0
NSCLC	1	0	44	0
PCA	0	0	0	35

This is a multi-class confusion matrix. The rows are the true classes and the columns are the predicted classes. The diagonal shows the number of samples that were correctly classified. The off-diagonal elements show the number of samples that were misclassified.

We can also look at the accuracy of the model on the training data and the test data. The accuracy is the number of correctly classified samples divided by the total number of samples.

22. Examples

```
measures = msrs(c('classif.acc'))  
pred_train$score(measures)
```

```
classif.acc  
0.9938272
```

```
pred_test$confusion
```

	truth			
response	BC	CRC	NSCLC	PCA
BC	22	0	0	0
CRC	0	17	1	0
NSCLC	0	0	15	0
PCA	0	0	0	25

```
pred_test$score(measures)
```

```
classif.acc  
0.9875
```

Compare the accuracy on the training data to the accuracy on the test data. Do you see any evidence of overfitting?

22.4.6.2. Classification tree

We are going to use a classification tree to classify the data. A classification tree is a series of yes/no questions that are used to classify the data. The questions are based on the features in the data. The classification tree is built by finding the feature that best separates the data into the different classes. Then, the data is split based on the value of that feature. The process is repeated until the data is completely separated into the different classes.

22.4.6.2.1. Train

```
# in this case, we want to keep the model
# so we can look at it later
learner = lrn("classif.rpart", keep_model = TRUE)
```

```
learner$train(task, row_ids = train_set)
```

We can take a look at the model.

```
learner$model
```

```
n= 162
```

```
node), split, n, loss, yval, (yprob)
      * denotes terminal node
```

```
1) root 162 118 NSCLC (0.26543210 0.24691358 0.27160494 0.21604938)
  2) CDHR5>=5.101625 40    0 CRC (0.00000000 1.00000000 0.00000000 0.00000000) *
  3) CDHR5< 5.101625 122  78 NSCLC (0.35245902 0.00000000 0.36065574 0.28688525)
    6) ACPP< 6.088431 87   43 NSCLC (0.49425287 0.00000000 0.50574713 0.00000000)
      12) GATA3>=4.697803 41    1 BC (0.97560976 0.00000000 0.02439024 0.00000000) *
      13) GATA3< 4.697803 46    3 NSCLC (0.06521739 0.00000000 0.93478261 0.00000000) *
    7) ACPP>=6.088431 35    0 PCA (0.00000000 0.00000000 0.00000000 1.00000000) *
```

Decision trees are easy to visualize if they are small. Here, we can see that the tree is very simple, with only two splits.

```
library(mlr3viz)
library(ggparty)
```

```
Loading required package: ggplot2
```

```
Loading required package: partykit
```

```
Loading required package: grid
```

```
Loading required package: libcoin
```

22. Examples

Loading required package: mvtnorm

Attaching package: 'partykit'

The following object is masked from 'package:SummarizedExperiment':

width

The following object is masked from 'package:GenomicRanges':

width

The following object is masked from 'package:IRanges':

width

The following object is masked from 'package:S4Vectors':

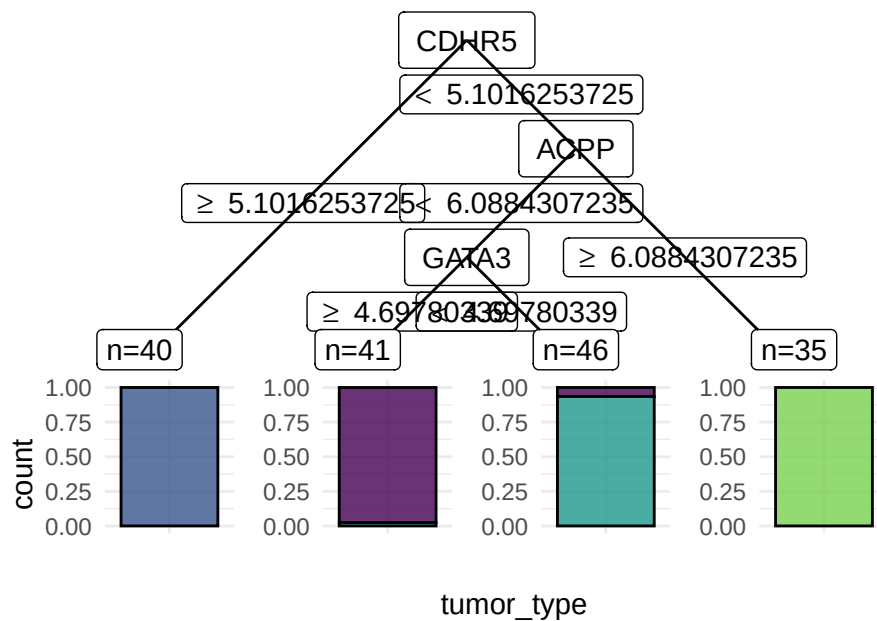
width

The following object is masked from 'package:BiocGenerics':

width

```
autoplot(learner, type='ggparty')
```

Warning in ggparty::geom_edge_label(): Ignoring unknown parameters:
`label.size`



22.4.6.2.2. Predict

Now that we have trained the model on the *training* data, we can use it to predict the classes of the training data and the test data. The `$predict` method takes a `task` and produces a prediction based on the *trained* model, in this case, called `learner`.

```
pred_train = learner$predict(task, row_ids=train_set)
```

Remember that we split the data into training and test sets. We can use the trained model to predict the classes of the test data. Since the *test* data was not used to train the model, it is not “cheating” like what we just did where we did the prediction on the *training* data.

```
pred_test = learner$predict(task, row_ids=test_set)
```

22.4.6.2.3. Assess

For classification tasks, we often look at a confusion matrix of the *truth* vs the *predicted* classes for the samples.

! Important

Assessing the performance of a model should **always** be **reported** from assessment on an independent test set.

```
pred_train$confusion
```

	truth			
response	BC	CRC	NSCLC	PCA
BC	40	0	1	0
CRC	0	40	0	0
NSCLC	3	0	43	0
PCA	0	0	0	35

- What does this confusion matrix tell you?

We can also ask for several “measures” of the performance of the model. Here, we ask for the accuracy of the model. To get a complete list of measures, use `msr()`.

```
measures = msrs(c('classif.acc'))
pred_train$score(measures)
```

```
classif.acc
0.9753086
```

- How does the accuracy compare to the confusion matrix?
- How does this accuracy compare to the accuracy of the k-nearest-neighbor model?
- How about the decision tree model?

```
pred_test$confusion
```

	truth			
response	BC	CRC	NSCLC	PCA
BC	20	0	1	0
CRC	0	17	3	0
NSCLC	2	0	12	0
PCA	0	0	0	25

```
pred_test$score(measures)
```

```
classif.acc
0.925
```

- What does the confusion matrix in the *test* set tell you?
- How do the assessments of the *test* and *training* sets differ?

💡 Overfitting

When the assessment of the test set is worse than the evaluation of the training set, the model may be *overfit*. How to address overfitting varies by model type, but it is a sign that you should pay attention to model selection and parameters.

22.4.6.3. RandomForest

```
learner = lrn("classif.ranger", importance = "impurity")
```

22.4.6.3.1. Train

```
learner$train(task, row_ids = train_set)
```

Again, you can look at the model that was trained.

```
learner$model
```

Ranger result

Call:

```
ranger::ranger(dependent.variable.name = task$target_names, data = task$data(), proba
```

Type:	Classification
Number of trees:	500
Sample size:	162
Number of independent variables:	192
Mtry:	13

22. Examples

```
Target node size:          1
Variable importance mode:  impurity
Splitrule:                gini
OOB prediction error:      0.62 %
```

For more details, the `mlr3` random forest approach is based on the `ranger` package. You can look at the `ranger` documentation.

- What is the OOB error in the output?

Random forests are a collection of decision trees. Since predictors enter the trees in a random order, the trees are different from each other. The random forest procedure gives us a measure of the “importance” of each variable.

```
head(learner$importance(), 15)
```

LGALS4	FABP1	KRT20	EFHD1	CDHR5	TRPS1	EPS8L3	TRPS1.1
4.059718	3.910380	3.570283	3.533239	3.460209	3.174617	3.048722	2.949918
MUC12	SFTPB	GATA3	SFTPA2	CCL18.1	KLK2.1	CFTR	
2.943308	2.881058	2.571155	2.234561	2.138951	2.072283	2.057455	

More “important” variables are those that are more often used in the trees. Are the most important variables the same as the ones that were important in the decision tree?

If you are interested, look up a few of the important variables in the model to see if they make biological sense.

22.4.6.3.2. Predict

Again, we can use the trained model to predict the classes of the training data and the test data.

```
pred_train = learner$predict(task, row_ids=train_set)
```

```
pred_test = learner$predict(task, row_ids=test_set)
```

22.4.6.3.3. Assess

```
pred_train$confusion
```

	truth			
response	BC	CRC	NSCLC	PCA
BC	43	0	0	0
CRC	0	40	0	0
NSCLC	0	0	44	0
PCA	0	0	0	35

```
measures = msrs(c('classif.acc'))
pred_train$score(measures)
```

```
classif.acc
1
```

```
pred_test$confusion
```

	truth			
response	BC	CRC	NSCLC	PCA
BC	22	0	0	0
CRC	0	17	0	0
NSCLC	0	0	16	0
PCA	0	0	0	25

```
pred_test$score(measures)
```

```
classif.acc
1
```

22.5. Example Predicting age from DNA methylation

We will be building a regression model for chronological age prediction, based on DNA methylation. This is based on the work of [Jana Naue et al. 2017](#), in which biomarkers are examined to predict the chronological age of humans by analyzing the DNA methylation patterns. Different machine learning algorithms are used in this study to make an age prediction.

22. Examples

It has been recognized that within each individual, the level of [DNA methylation](#) changes with age. This knowledge is used to select useful biomarkers from DNA methylation datasets. The [CpG sites](#) with the highest correlation to age are selected as the biomarkers (and therefore features for building a regression model). In this tutorial, specific biomarkers are analyzed by machine learning algorithms to create an age prediction model.

The data are taken from [this tutorial](#).

```
library(data.table)
meth_age = rbind(
  fread('https://zenodo.org/record/2545213/files/test_rows_labels.csv'),
  fread('https://zenodo.org/record/2545213/files/train_rows.csv')
)
```

Let's take a quick look at the data.

```
head(meth_age)
```

	RPA2_3	ZYG11A_4	F5_2	HOXC4_1	NKIRAS2_2	MEIS1_1	SAMD10_2	GRM2_9	TRIM59_5
	<num>	<num>	<num>	<num>	<num>	<num>	<num>	<num>	<num>
1:	65.96	18.08	41.57	55.46	30.69	63.42	40.86	68.88	44.32
2:	66.83	20.27	40.55	49.67	29.53	30.47	37.73	53.30	50.09
3:	50.30	11.74	40.17	33.85	23.39	58.83	38.84	35.08	35.90
4:	65.54	15.56	33.56	36.79	20.23	56.39	41.75	50.37	41.46
5:	59.01	14.38	41.95	30.30	24.99	54.40	37.38	30.35	31.28
6:	81.30	14.68	35.91	50.20	26.57	32.37	32.30	55.19	42.21

	LDB2_3	ELOVL2_6	DDO_1	KLF14_2	Age
	<num>	<num>	<num>	<num>	<int>
1:	56.17	62.29	40.99	2.30	40
2:	58.40	61.10	49.73	1.07	44
3:	58.81	50.38	63.03	0.95	28
4:	58.05	50.58	62.13	1.99	37
5:	65.80	48.74	41.88	0.90	24
6:	70.15	61.36	33.62	1.87	43

As before, we create the `task` object, but this time we use `as_task_regr()` to create a regression task.

- Why is this a regression task?

22. Examples

```
task = as_task_regr(meth_age, target = 'Age')
```

```
set.seed(7)
train_set = sample(task$row_ids, 0.67 * task$nrow)
test_set = setdiff(task$row_ids, train_set)
```

22.5.1. Example learners

22.5.1.1. Linear regression

We will start with a simple linear regression model.

```
learner = lrn("regr.lm")
```

22.5.1.1.1. Train

```
learner$train(task, row_ids = train_set)
```

When you train a linear regression model, we can evaluate some of the diagnostic plots to see if the model is appropriate (Figure [22.4](#)).

```
par(mfrow=c(2,2))
plot(learner$model)
```

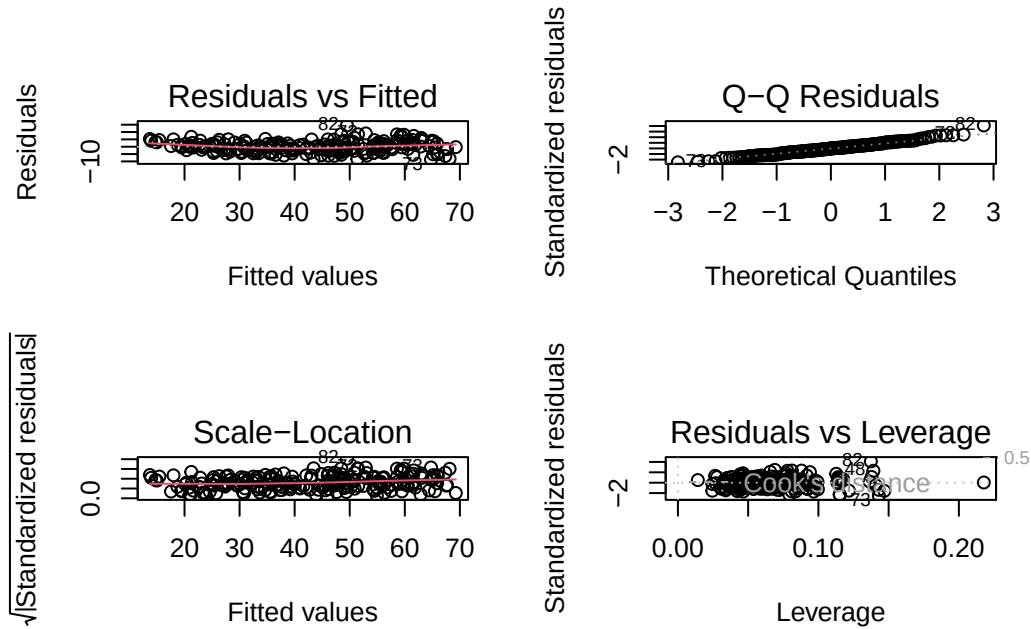


Figure 22.4.: Regression diagnostic plots. The top left plot shows the residuals vs. fitted values. The top right plot shows the normal Q-Q plot. The bottom left plot shows the scale-location plot. The bottom right plot shows the residuals vs. leverage.

22.5.1.1.2. Predict

```
pred_train = learner$predict(task, row_ids=train_set)
```

```
pred_test = learner$predict(task, row_ids=test_set)
```

22.5.1.1.3. Assess

```
pred_train
```

```
-- <PredictionRegr> for 209 observations: -----
row_ids truth response
  298    29 31.40565
```

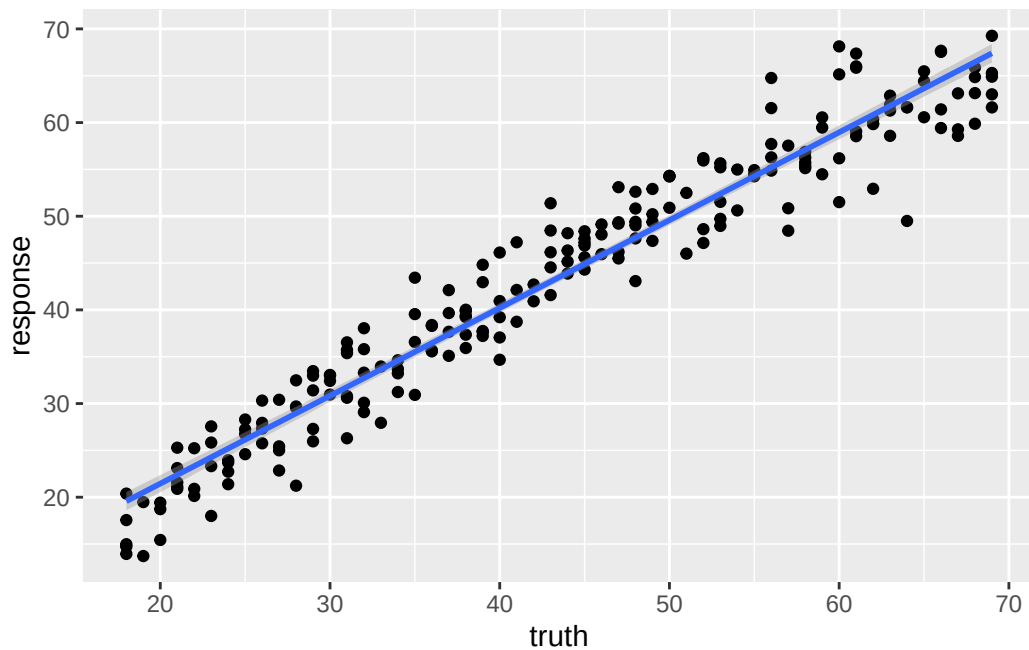

22. Examples

103	58	56.26019
194	53	48.96480
---	---	---
312	48	52.61195
246	66	67.66312
238	38	39.38414

We can plot the relationship between the `truth` and `response`, or predicted value to see visually how our model performs.

```
library(ggplot2)
ggplot(pred_train, aes(x=truth, y=response)) +
  geom_point() +
  geom_smooth(method='lm')
```

``geom_smooth()`` using formula = 'y ~ x'



We can use the r-squared of the fit to roughly compare two models.

22. Examples

```
measures = msrs(c('regr.rsq'))  
pred_train$score(measures)
```

```
regr.rsq  
0.9376672
```

```
pred_test
```

```
-- <PredictionRegr> for 103 observations: -----  
row_ids truth response  
    4    37 37.64301  
    5    24 28.34777  
    7    34 33.22419  
---    ---    ---  
  306    42 41.65864  
  307    63 58.68486  
  309    68 70.41987
```

```
pred_test$score(measures)
```

```
regr.rsq  
0.9363526
```

22.5.1.2. Regression tree

```
learner = lrn("regr.rpart", keep_model = TRUE)
```

22.5.1.2.1. Train

```
learner$train(task, row_ids = train_set)
```

```
learner$model
```

22. Examples

n= 209

```
node), split, n, deviance, yval
  * denotes terminal node
```

```
1) root 209 45441.4500 43.27273
  2) ELOVL2_6< 56.675 98 5512.1220 30.24490
    4) ELOVL2_6< 47.24 47 866.4255 24.23404
      8) GRM2_9< 31.3 34 289.0588 22.29412 *
      9) GRM2_9>=31.3 13 114.7692 29.30769 *
    5) ELOVL2_6>=47.24 51 1382.6270 35.78431
      10) F5_2>=39.295 35 473.1429 33.28571 *
      11) F5_2< 39.295 16 213.0000 41.25000 *
  3) ELOVL2_6>=56.675 111 8611.3690 54.77477
    6) ELOVL2_6< 65.365 63 3101.2700 49.41270
      12) KLF14_2< 3.415 37 1059.0270 46.16216 *
      13) KLF14_2>=3.415 26 1094.9620 54.03846 *
    7) ELOVL2_6>=65.365 48 1321.3120 61.81250 *
```

What is odd about using a regression tree here is that we end up with only a few discrete estimates of age. Each “leaf” has a value.

22.5.1.2.2. Predict

```
pred_train = learner$predict(task, row_ids=train_set)
```

```
pred_test = learner$predict(task, row_ids=test_set)
```

22.5.1.2.3. Assess

```
pred_train
```

```
-- <PredictionRegr> for 209 observations: -----
row_ids truth response
  298    29 33.28571
  103    58 61.81250
  194    53 46.16216
```

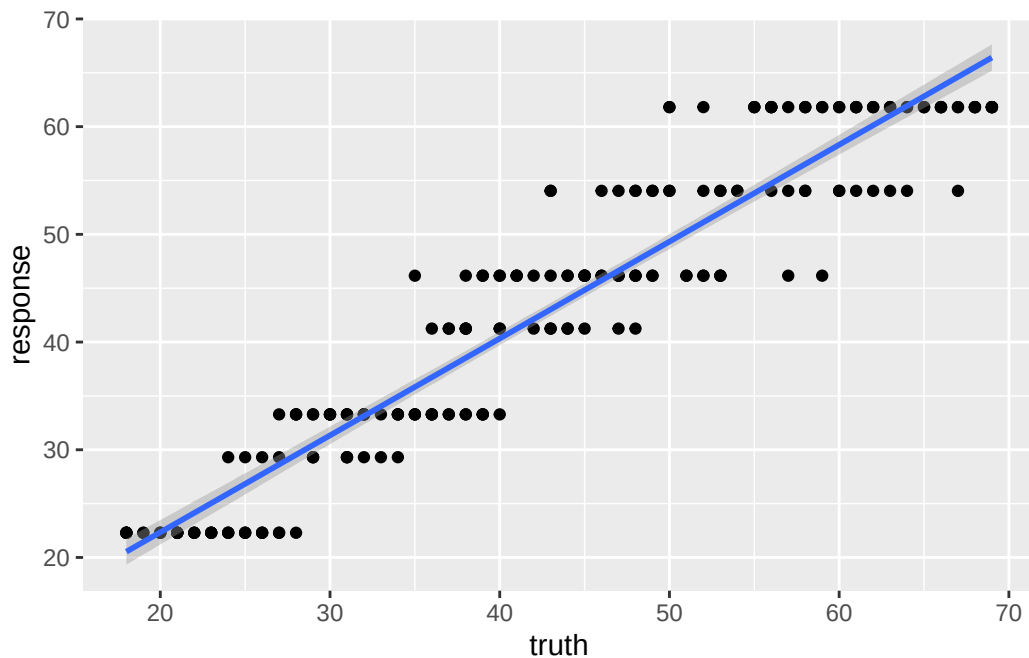
22. Examples

```
---      ---      ---  
312      48 54.03846  
246      66 61.81250  
238      38 41.25000
```

We can see the effect of the discrete values much more clearly here.

```
library(ggplot2)  
ggplot(pred_train, aes(x=truth, y=response)) +  
  geom_point() +  
  geom_smooth(method='lm')
```

`geom\_smooth()` using formula = 'y ~ x'



And the r-squared values for this model prediction shows quite a bit of difference from the linear regression above.

```
measures = msrs(c('regr.rsq'))  
pred_train$score(measures)
```

22. Examples

```
regr.rsq  
0.8995351
```

```
pred_test
```

```
-- <PredictionRegr> for 103 observations: -----  
row_ids truth response  
    4    37 41.25000  
    5    24 33.28571  
    7    34 33.28571  
   ---   ---   ---  
  306    42 46.16216  
  307    63 61.81250  
  309    68 61.81250
```

```
pred_test$score(measures)
```

```
regr.rsq  
0.8545402
```

22.5.1.3. RandomForest

Randomforest is also tree-based, but unlike the single regression tree above, randomforest is a “forest” of trees which will eliminate the discrete nature of a single tree.

```
learner = lrn("regr.ranger", mtry=2, min.node.size=20)
```

22.5.1.3.1. Train

```
learner$train(task, row_ids = train_set)
```

```
learner$model
```

22. Examples

\$model

Ranger result

Call:

```
ranger::ranger(dependent.variable.name = task$target_names, data = data, min.node.size = 20)
```

Type:	Regression
Number of trees:	500
Sample size:	209
Number of independent variables:	13
Mtry:	2
Target node size:	20
Variable importance mode:	none
Splitrule:	variance
OOB prediction error (MSE):	18.71583
R squared (OOB):	0.9143317

22.5.1.3.2. Predict

```
pred_train = learner$predict(task, row_ids=train_set)
```

```
pred_test = learner$predict(task, row_ids=test_set)
```

22.5.1.3.3. Assess

```
pred_train
```

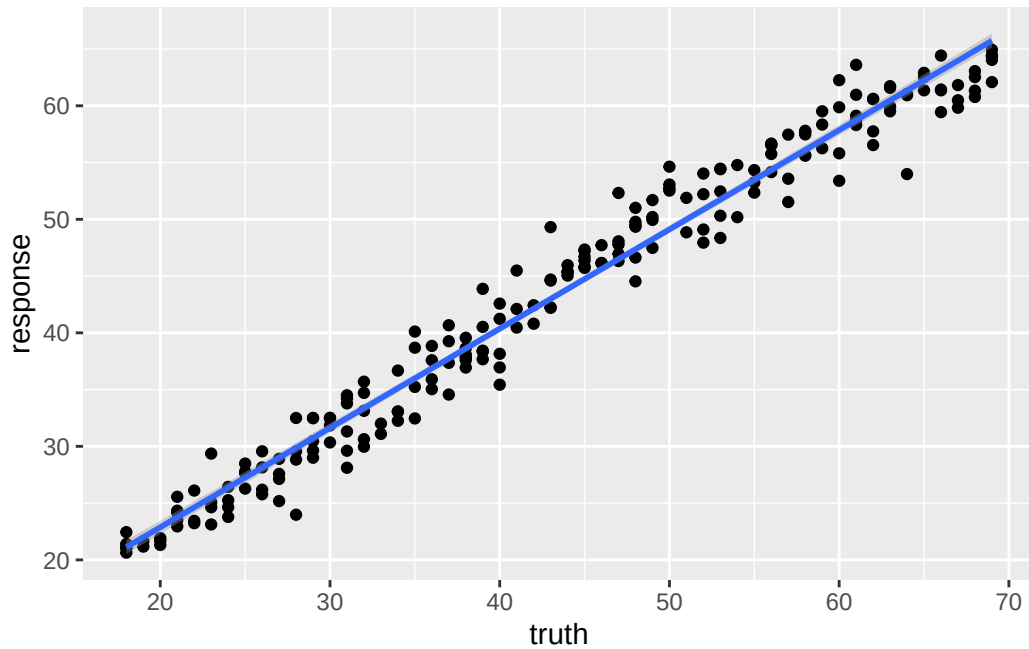
```
-- <PredictionRegr> for 209 observations: -----
```

row_ids	truth	response
298	29	30.45593
103	58	57.68655
194	53	48.35907
---	---	---
312	48	51.00870
246	66	64.42354
238	38	37.83828

22. Examples

```
ggplot(pred_train, aes(x=truth, y=response)) +  
  geom_point() +  
  geom_smooth(method='lm')
```

``geom_smooth()`` using formula = `'y ~ x'`



```
measures = msrs(c('regr.rsq'))  
pred_train$score(measures)
```

```
regr.rsq  
0.9613337
```

```
pred_test
```

```
-- <PredictionRegr> for 103 observations: -----  
row_ids truth response  
4      37 37.58435
```

22. Examples

5	24	29.14633
7	34	33.46846
---	---	---
306	42	40.62838
307	63	58.24280
309	68	63.06995

```
pred_test$score(measures)
```

```
regr.rsq  
0.9206752
```

22.6. Example: Expression prediction from histone modification data

In this little set of exercises, you will be using histone marks near a gene to predict its expression (Figure 22.5).

$$y \approx h_1 + h_2 + h_3 + \dots \quad (22.1)$$

The data are from a study that aimed to predict gene expression from histone modification data. The data include gene expression levels and histone modification data for a set of genes. The goal is to build a machine learning model that can predict gene expression levels based on the histone modification data. The histone modification data are simply summaries of the histone marks within the promoter, defined as the region 2kb upstream of the transcription start site for this exercise.

We will try a couple of different approaches:

1. Penalized regression
2. RandomForest

22.6.1. The Data

The data in this exercise were developed by Anshul Kundaje. We are not going to focus on the details of the data collection, etc. Instead, this is

Relationship between chromatin marks and gene expression

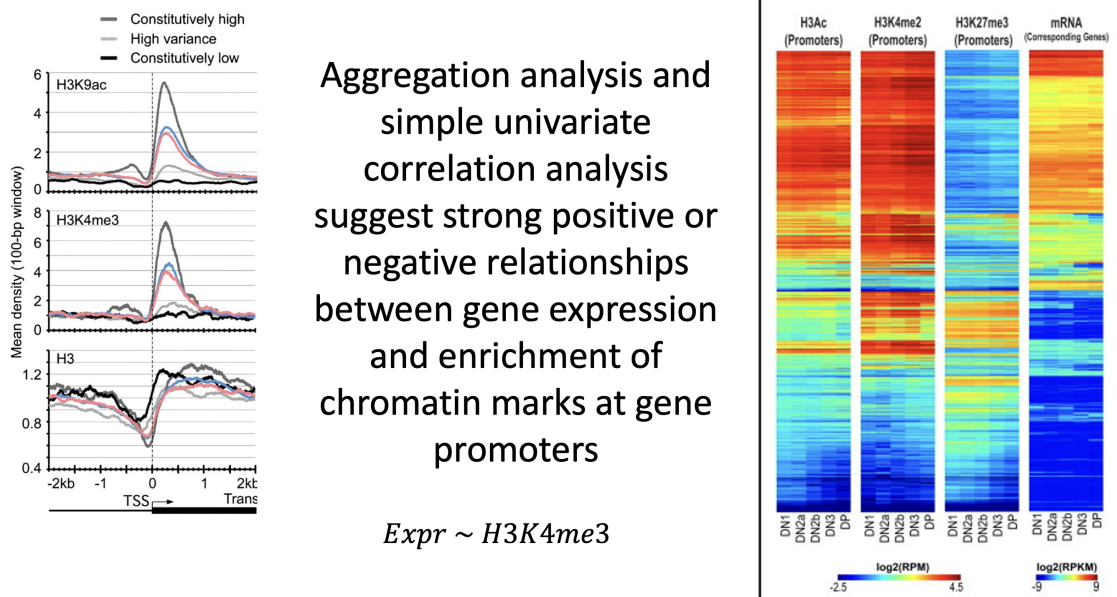


Figure 22.5.: What is the combined effect of histone marks on gene expression?

22. Examples

```
fullFeatureSet <- read.table("http://seandavi.github.io/ITR/expression-prediction/features.
```

What are the column names of the predictor variables?

```
colnames(fullFeatureSet)
```

```
[1] "Control"  "Dnase"    "H2az"     "H3k27ac"  "H3k27me3" "H3k36me3"  
[7] "H3k4me1"  "H3k4me2"  "H3k4me3"  "H3k79me2" "H3k9ac"   "H3k9me1"  
[13] "H3k9me3"  "H4k20me1"
```

These are going to be predictors combined into a model. Some of our learners will rely on predictors being on a similar scale. Are our data already there?

To perform centering and scaling by column, we can convert to a matrix and then use `scale`.

```
par(mfrow=c(1,2))  
scaled_features <- scale(as.matrix(fullFeatureSet))  
boxplot(fullFeatureSet, title='Original data')  
boxplot(scaled_features, title='Centered and scaled data')
```

22. Examples

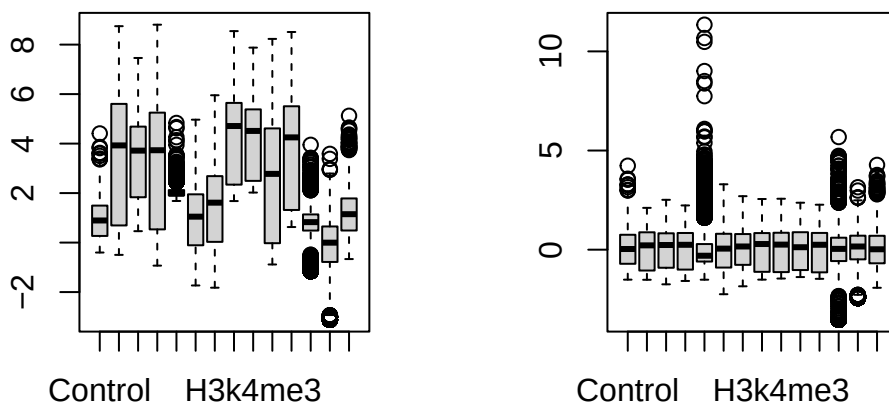


Figure 22.6.: Boxplots of original and scaled data.

There is a row for each gene and a column for each histone mark and we can see that the data are centered and scaled by column. We can also see some patterns in the data (see Figure 22.7).

```
sampled_features <- fullFeatureSet[sample(nrow(scaled_features), 500),]
library(ComplexHeatmap)
```

=====

ComplexHeatmap version 2.28.0

Bioconductor page: <http://bioconductor.org/packages/ComplexHeatmap/>

Github page: <https://github.com/jokergoo/ComplexHeatmap>

Documentation: <http://jokergoo.github.io/ComplexHeatmap-reference>

If you use it in published research, please cite either one:

- Gu, Z. Complex Heatmap Visualization. iMeta 2022.
- Gu, Z. Complex heatmaps reveal patterns and correlations in multidimensional genomic data. Bioinformatics 2016.

22. Examples

The new InteractiveComplexHeatmap package can directly export static complex heatmaps into an interactive Shiny app with zero effort. Have a try!

This message can be suppressed by:

```
suppressPackageStartupMessages(library(ComplexHeatmap))
=====
```

```
Heatmap(sampled_features, name='histone marks', show_row_names=FALSE)
```

Warning: The input is a data frame-like object, convert it to a matrix.

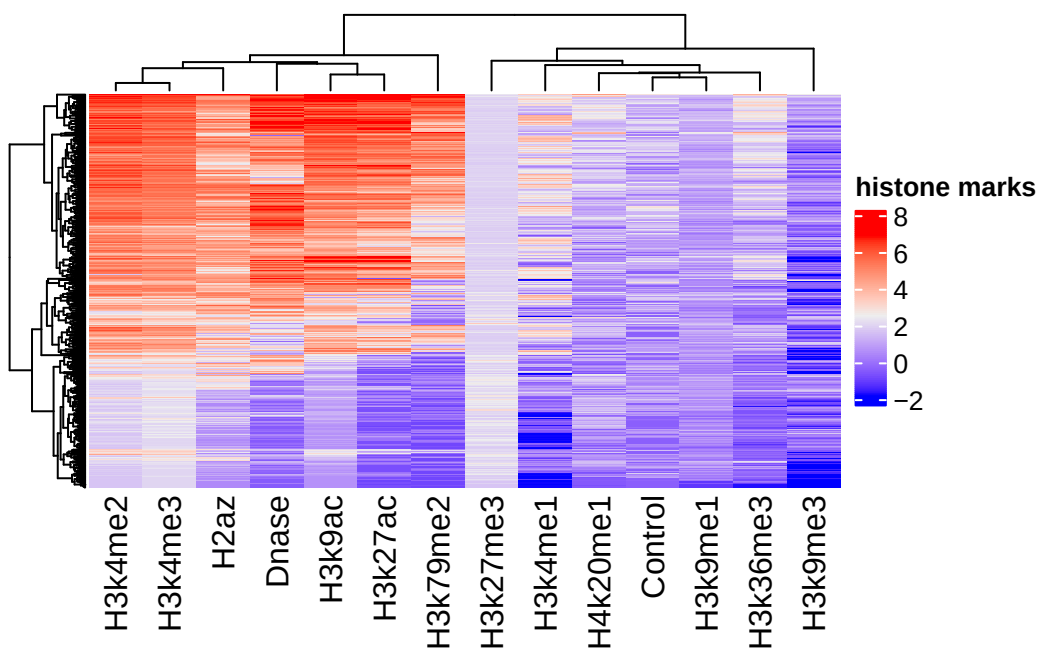


Figure 22.7.: Heatmap of 500 randomly sampled rows of the data. Columns are histone marks and there is a row for each gene.

Now, we can read in the associated gene expression measures that will become our “target” for prediction.

22. Examples

```
target <- scan(url("http://seandavi.github.io/ITR/expression-prediction/target.txt"), skip=
# make into a dataframe
exp_pred_data <- data.frame(gene_expression=target, scaled_features)
```

And the first few rows of the target data frame using:

```
head(exp_pred_data,3)
```

	gene_expression	Control	Dnase	H2az
ENSG000000000419.7.49575069	6.082343	0.7452926	0.7575546	1.0728432
ENSG000000000457.8.169863093	2.989145	1.9509786	1.0216546	0.3702787
ENSG000000000938.7.27961645	-5.058894	-0.3505542	-1.4482958	-1.0390775
	H3k27ac	H3k27me3	H3k36me3	H3k4me1
ENSG000000000419.7.49575069	1.0950440	-0.5125312	1.1334793	0.4127984
ENSG000000000457.8.169863093	0.7142157	-0.4079244	0.8739005	1.1649282
ENSG000000000938.7.27961645	-1.0173283	1.4117293	-0.5157582	-0.5017450
	H3k4me2	H3k4me3	H3k79me2	H3k9ac
ENSG000000000419.7.49575069	1.2136176	1.1202901	1.5155803	1.2468256
ENSG000000000457.8.169863093	0.6456572	0.6508561	0.7976487	0.5792891
ENSG000000000938.7.27961645	-0.1878255	-0.6560973	-1.3803974	-1.0067972
	H3k9me1	H3k9me3	H4k20me1	
ENSG000000000419.7.49575069	0.1426980	1.185622	1.9599992	
ENSG000000000457.8.169863093	0.3630902	1.014923	-0.2695111	
ENSG000000000938.7.27961645	0.6564520	-1.370871	-1.8773178	

22.6.2. Create task

```
exp_pred_task = as_task_regr(exp_pred_data, target='gene_expression')
```

Partition the data into test and training sets. We will use $\frac{1}{3}$ and $\frac{2}{3}$ of the data for testing.

```
split = partition(exp_pred_task)
```

22.6.3. Example learners**22.6.3.1. Linear regression**

```
learner = lrn("regr.lm")
```

22.6.3.1.1. Train

```
learner$train(exp_pred_task, split$train)
```

22.6.3.1.2. Predict

```
pred_train = learner$predict(exp_pred_task, split$train)
pred_test = learner$predict(exp_pred_task, split$test)
```

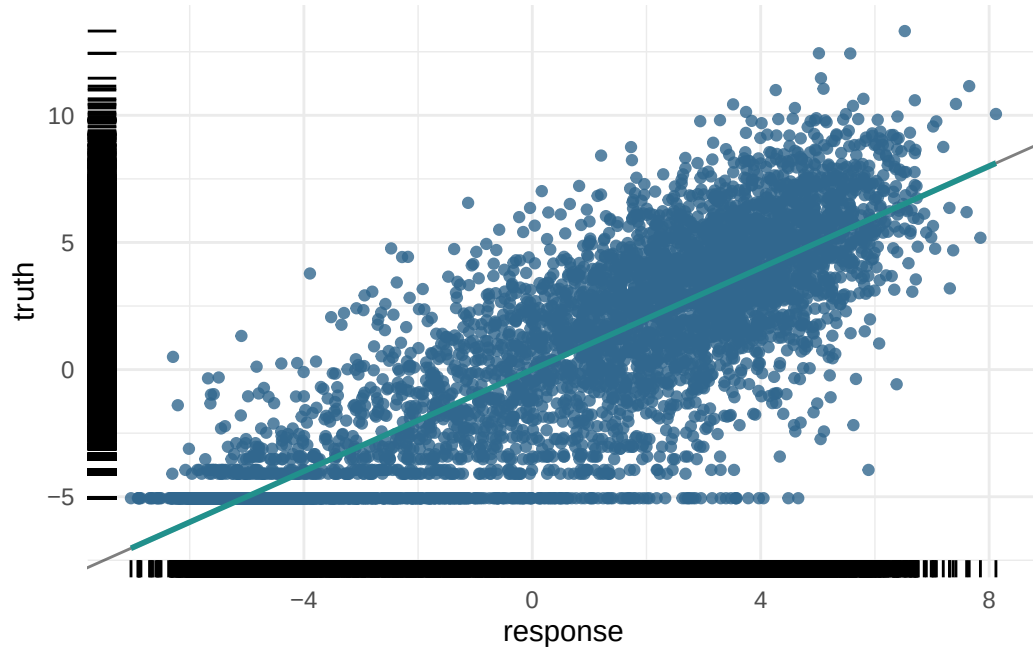
22.6.3.1.3. Assess

```
pred_train
```

```
-- <PredictionRegr> for 5789 observations: -----
row_ids      truth  response
      1  6.082343  5.182373
      2  2.989145  2.970278
      3 -5.058894 -5.283509
    ---      ---      ---
 8637 -5.058894 -3.955237
 8638  6.089159  4.809801
 8640 -3.114148 -4.723061
```

```
plot(pred_train)
```

22. Examples



For the training data:

```
measures = msrs(c('regr.rsq'))  
pred_train$score(measures)
```

```
regr.rsq  
0.7505366
```

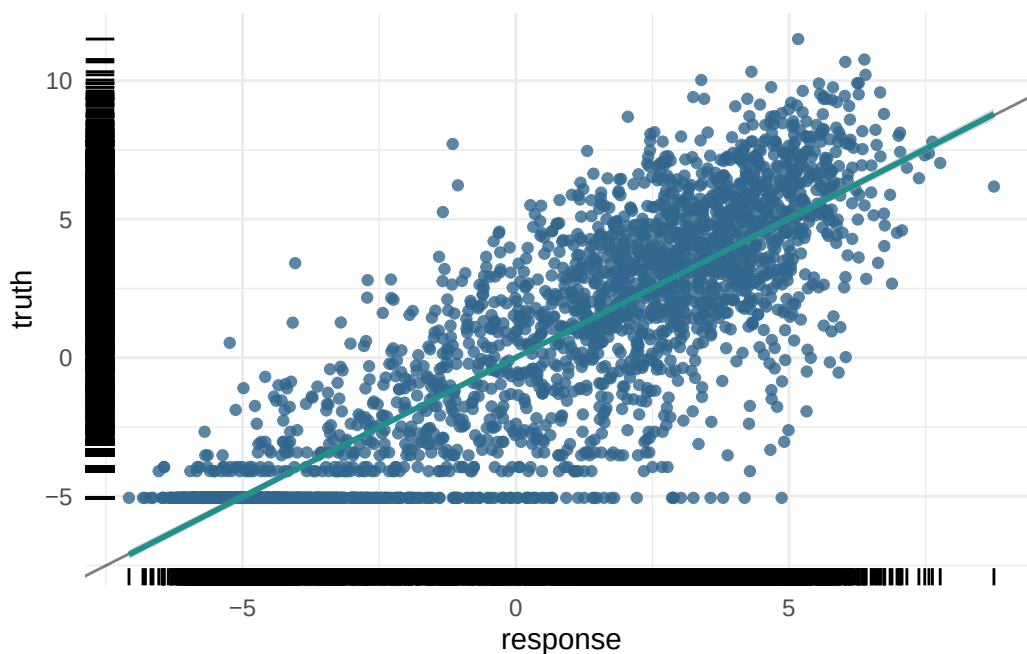
And the test data:

```
pred_test$score(measures)
```

```
regr.rsq  
0.7510334
```

And the plot of the test data predictions:

```
plot(pred_test)
```



22.6.3.2. Penalized regression

Imagine you want to teach a computer to predict house prices based on various features like size, number of bedrooms, and location. You decide to use **regression**, which finds a relationship between these features and the house prices. But what if your model becomes too complicated? This is where **penalized regression** comes in.

22.6.3.2.1. The Problem with Overfitting

Sometimes, the model tries too hard to fit every single data point perfectly. This can make the model very complex, like trying to draw a perfect line through a very bumpy path. This problem is called **overfitting**. An overfitted model works well on the data it has seen (training data) but performs poorly on new, unseen data (testing data).

22.6.3.2.2. Introducing Penalized Regression

Penalized regression helps prevent overfitting by adding a “penalty” to the model for being too complex. Think of it as a way to encourage the model to be simpler and more general. There are three common types of penalized regression:

22. Examples

1. Ridge Regression (L2 Penalty):

- Adds a penalty based on the size of the coefficients. It tries to keep all coefficients small.
- If the model's equation looks too complicated, Ridge Regression will push it towards a simpler form by shrinking the coefficients.
- Imagine you have a rubber band that pulls the coefficients towards zero, making the model less likely to overfit.

2. Lasso Regression (L1 Penalty):

- Adds a penalty that can shrink some coefficients all the way to zero.
- This means Lasso Regression can completely remove some features from the model, making it simpler.
- Imagine you have a pair of scissors that can cut off the least important features, leaving only the most important ones.

3. Elastic Net:

- Combines both Ridge and Lasso penalties. It adds penalties for both the size and the number of coefficients.
- This method balances between shrinking coefficients and eliminating some altogether, offering the benefits of both Ridge and Lasso.
- Think of Elastic Net as using both the rubber band (Ridge) and scissors (Lasso) to simplify the model.

With our data, the number of predictors is not huge, but we might be interested in 1) reducing overfitting, 2) improving interpretability, or 3) both by minimizing the number of predictors in our model without drastically affecting our prediction accuracy. Without penalized regression, the model might come up with a very complex equation. With Ridge, Lasso, or Elastic Net, the model simplifies this equation by either shrinking the coefficients (Ridge), removing some of them (Lasso), or balancing both (Elastic Net).

Here's a simple summary:

- **Ridge Regression:** Reduces the impact of less important features by shrinking their coefficients.
- **Lasso Regression:** Can eliminate some features entirely by setting their coefficients to zero.
- **Elastic Net:** Combines the effects of Ridge and Lasso, shrinking some coefficients and eliminating others.

Using penalized regression in machine learning ensures that your model:

1. **Performs Better on New Data:** By avoiding overfitting, the model can make more accurate predictions on new, unseen data.
2. **Is Easier to Interpret:** A simpler model with fewer features is easier to understand and explain.

22.6.3.3. Penalized Regression with `mlr3`

In the `mlr3` package, you can easily apply penalized regression methods to your tasks. Here's how:

1. **Select Penalized Regression Learners:** `mlr3` provides learners for Ridge, Lasso, and Elastic Net Regression.
2. **Train the Learner:** Use your data to train the chosen penalized regression model.
3. **Evaluate and Adjust:** Check how well the model performs and make adjustments if needed.

This description explains penalized regression, including Ridge, Lasso, and Elastic Net, in an intuitive way, highlighting their benefits and how they work, while relating them to familiar concepts and the `mlr3` package.

Recall that we can use penalized regression to select the most important predictors from a large set of predictors. In this case, we will use the `glmnet` package to perform penalized regression, but we will use the `mlr` interface to `glmnet` to make it easier to use.

The `nfolds` parameter is the number of folds to use in the cross-validation procedure.

What is Cross-Validation? Cross-validation is a technique used to assess how well a model will perform on unseen data. It involves splitting the data into multiple parts, training the model on some of these parts, and validating it on the remaining parts. This process is repeated several times to ensure the model's performance is consistent.

Why Use Cross-Validation? Cross-validation helps to:

- **Avoid Overfitting:** By testing the model on different subsets of the data, cross-validation helps ensure that the model does not memorize the training data but learns to generalize from it.
- **Select the Best Model Parameters:** Penalized regression models, such as those trained with `glmnet`, have parameters that control the strength of the penalty (e.g., `lambda`). Cross-validation helps find the best values for these parameters.

When using the `glmnet` package, cross-validation can be performed using the `cv.glmnet` function. Here's how the process works:

22. Examples

1. Split the Data: The data is divided into k folds (common choices are 5 or 10 folds). Each fold is a subset of the data.
2. Train and Validate: The model is trained k times. In each iteration, $k-1$ folds are used for training, and the remaining fold is used for validation. This process is repeated until each fold has been used as the validation set exactly once.
3. Calculate Performance: The performance of the model (e.g., mean squared error for regression) is calculated for each fold. The average performance across all folds is computed to get an overall measure of how well the model is expected to perform on unseen data.
4. Select the Best Parameters: The `cv.glmnet` function evaluates different values of the penalty parameter (λ). It selects the λ value that results in the best average performance across the folds.

In this case, we will use the `cv_glmnet` learner, which will automatically select the best value of the penalization parameters. When the `alpha` parameter is set to 0, the model is a Ridge regression model. When the `alpha` parameter is set to 1, the model is a Lasso regression model.

```
learner = lrn("regr.cv_glmnet", nfolds=10, alpha=0)
```

22.6.3.3.1. Train

```
learner$train(exp_pred_task)
```

```
measures = msrs(c('regr.rsq', 'regr.mse', 'regr.rmse'))  
pred_train$score(measures)
```

```
regr.rsq  regr.mse  regr.rmse  
0.7505366 4.8335493 2.1985334
```

In the case of the penalized regression, we can also look at the coefficients of the model.

```
coef(learner$model)
```

22. Examples

15 x 1 sparse Matrix of class "dgCMatix"

```
      lambda.1se
(Intercept)  0.10173828
Control      -0.07049573
Dnase        0.87495827
H2az         0.33778155
H3k27ac      0.19454755
H3k27me3     -0.26196627
H3k36me3     0.70067015
H3k4me1      -0.06753216
H3k4me2      0.15796965
H3k4me3      0.38806405
H3k79me2     0.94399114
H3k9ac       0.50694074
H3k9me1      -0.07422291
H3k9me3      -0.17013775
H4k20me1     0.11617186
```

Note that the coefficients are all zero for the histone marks that were not selected by the model. In this case, we can use the model not to predict new data, but to help us understand the data.

```
pred_train = learner$predict(exp_pred_task, split$train)
pred_test  = learner$predict(exp_pred_task, split$test)
```

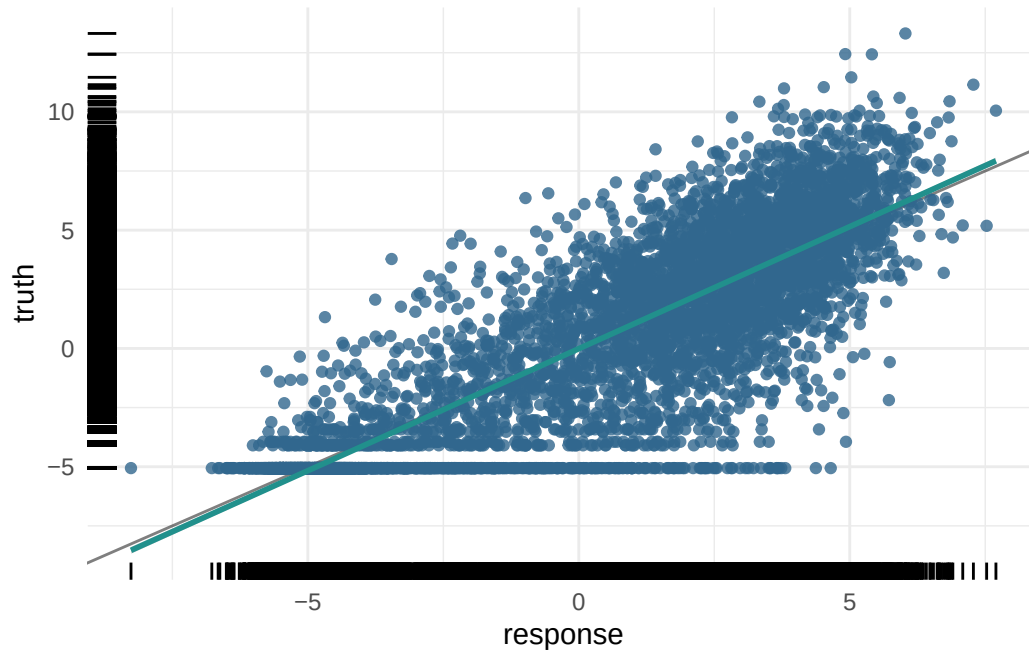
22.6.3.3.2. Assess

```
pred_train
```

```
-- <PredictionRegr> for 5789 observations: -----
row_ids      truth  response
    1  6.082343  4.892642
    2  2.989145  2.932909
    3 -5.058894 -4.518292
---          ---
 8637 -5.058894 -4.341742
 8638  6.089159  4.696146
 8640 -3.114148 -4.153795
```

22. Examples

```
plot(pred_train)
```



For the training data:

```
measures = msrs(c('regr.rsq'))  
pred_train$score(measures)
```

```
regr.rsq  
0.742853
```

And the test data:

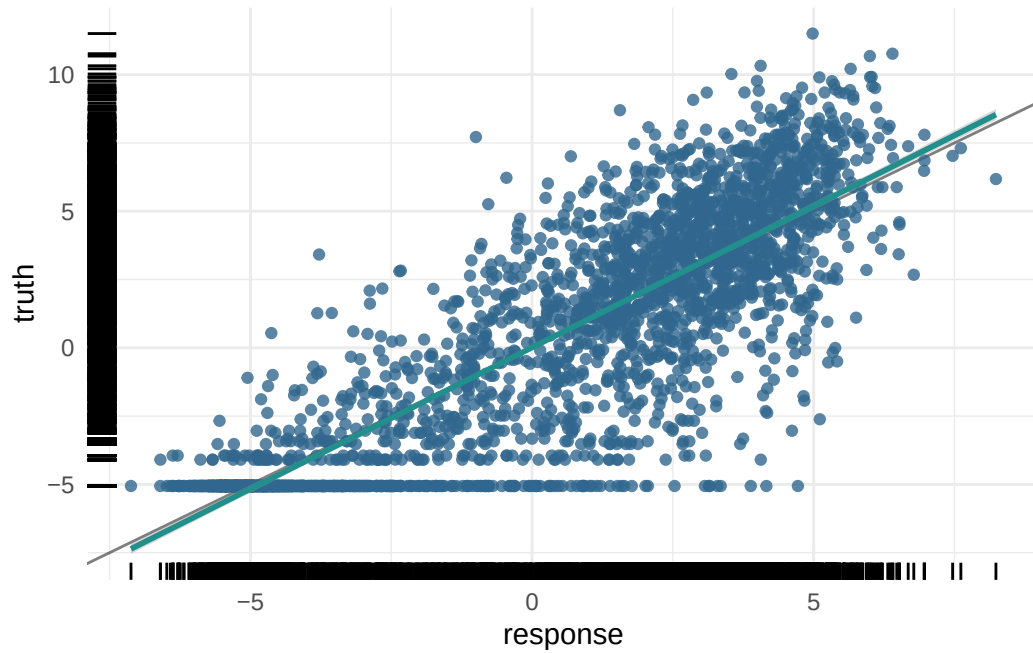
```
pred_test$score(measures)
```

```
regr.rsq  
0.7428019
```

And the plot of the test data predictions:

22. Examples

```
plot(pred_test)
```



```
# Calculate the R-squared value
truth <- pred_test$truth
predicted <- pred_test$response
rss <- sum((truth - predicted)^2) # Residual sum of squares
tss <- sum((truth - mean(truth))^2) # Total sum of squares
r_squared <- 1 - (rss / tss)
```

Part VI.

Bioconductor

23. Accessing and working with public omics data

23.1. Background

The data we are going to access are from [this paper](#).

Background: The tumor microenvironment is an important factor in cancer immunotherapy response. To further understand how a tumor affects the local immune system, we analyzed immune gene expression differences between matching normal and tumor tissue. **Methods:** We analyzed public and new gene expression data from solid cancers and isolated immune cell populations. We also determined the correlation between CD8, FoxP3 IHC, and our gene signatures. **Results:** We observed that regulatory T cells (Tregs) were one of the main drivers of immune gene expression differences between normal and tumor tissue. A tumor-specific CD8 signature was slightly lower in tumor tissue compared with normal of most (12 of 16) cancers, whereas a Treg signature was higher in tumor tissue of all cancers except liver. Clustering by Treg signature found two groups in colorectal cancer datasets. The high Treg cluster had more samples that were consensus molecular subtype 1/4, right-sided, and microsatellite-unstable, compared with the low Treg cluster. Finally, we found that the correlation between signature and IHC was low in our small dataset, but samples in the high Treg cluster had significantly more CD8+ and FoxP3+ cells compared with the low Treg cluster. **Conclusions:** Treg gene expression is highly indicative of the overall tumor immune environment. **Impact:** In comparison with the consensus molecular subtype and microsatellite status, the Treg signature identifies more colorectal tumors with high immune activation that may benefit from cancer immunotherapy.

In this little exercise, we will:

1. Access public omics data using the GEOquery package
2. Get an opportunity to work with another SummarizedExperiment object.
3. Perform a simple unsupervised analysis to visualize these public data.

23.2. GEOquery to PCA

The first step is to install the R package [GEOquery](#). This package allows us to access data from the Gene Expression Omnibus (GEO) database. GEO is a public repository of omics data.

```
BiocManager::install("GEOquery")
```

GEOquery has only one commonly used function, `getGEO()` which takes a GEO accession number as an argument. The GEO accession number is a unique identifier for a dataset.

Use the [GEOquery](#) package to fetch data about [GSE103512](#).

```
library(GEOquery)
gse = getGEO("GSE103512")[[1]]
```

You might ask why we are using `[[1]]` at the end of the `getGEO()` function. The reason is that `getGEO()` returns a list of `GSE` objects. We are only interested in the first one (and in this case, the only one). We return a list of `GSE` objects because in the early days, it was not unusual to have a single GEO accession number represent multiple datasets. While uncommon now, we've kept the convention since lots of “older” data is still quite useful.

Again, a historically-derived detail, is to convert from the older Bioconductor data structure (GEOquery was written in 2007), the `ExpressionSet`, to the newer `SummarizedExperiment`.

```
library(SummarizedExperiment)
se = as(gse, "SummarizedExperiment")
```

Use some code to determine the answers to the following:

- What is the class of `se`?
- What are the dimensions of `se`?
- What are the dimensions of the `assay` slot of `se`?
- What are the dimensions of the `colData` slot of `se`?
- What variables are in the `colData` slot of `se`?

Examine two variables of interest, cancer type and tumor/normal status. The `with` function is a convenience to allow us to access variables in a data frame by name (rather than having to do `dataframe$variable_name`). Recalling that the `table` function is a convenient way to summarize the counts of unique values in a vector, we can use `with` to access the variables of interest and `table` to summarize the counts of unique values.

23. Accessing and working with public omics data

```
with(colData(se), table(`cancer.type.ch1`, `normal.ch1`))
```

	normal.ch1	
cancer.type.ch1	no	yes
BC	65	10
CRC	57	12
NSCLC	60	9
PCA	60	7

- How many samples are there of each cancer type?
- How many samples are there of each tumor/normal status?

When performing unsupervised analysis, it is common to filter genes by variance to find the most informative genes. It is common practice to filter genes by standard deviation or some other measure of variability and keep the top X percent of them when performing dimensionality reduction. There is not a single right answer to what percentage to use, so try a few to see what happens. In the example code, I chose to use the top 500 genes by standard deviation, but you can play with the threshold to see what happens.

Recall that the `assay` function is used to access the data matrix of the `SummarizedExperiment` object.

Think through the code below and then run it.

```
sds = apply(assay(se, 'exprs'), 1, sd)
dat = assay(se, 'exprs')[order(sds, decreasing = TRUE)[1:500], ]
```

If you don't recognize the function `apply`, it is a function that applies a function to each row or column of a matrix. In this case, we are applying the `sd` function to each row of the data matrix. The `order` function is used to sort the standard deviations in decreasing order (when `decreasing=TRUE`). And the `[1:500]` is used to subset the data matrix to the top 500 genes by standard deviation.

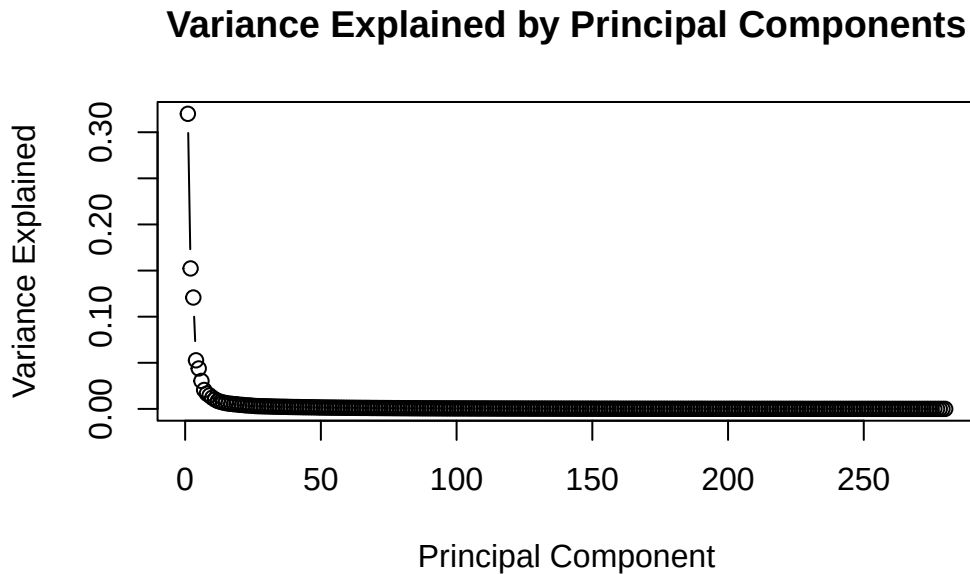
Perform PCA and prepare for plotting. We will be using `ggplot2`, so we need to make a `data.frame` before plotting.

```
pca_results <- prcomp(t(dat))
pca_df = as.data.frame(pca_results$x)
pca_df$Type = factor(colData(se)[, 'cancer.type.ch1'])
pca_df$Normal = factor(colData(se)[, 'normal.ch1'])
```

23. Accessing and working with public omics data

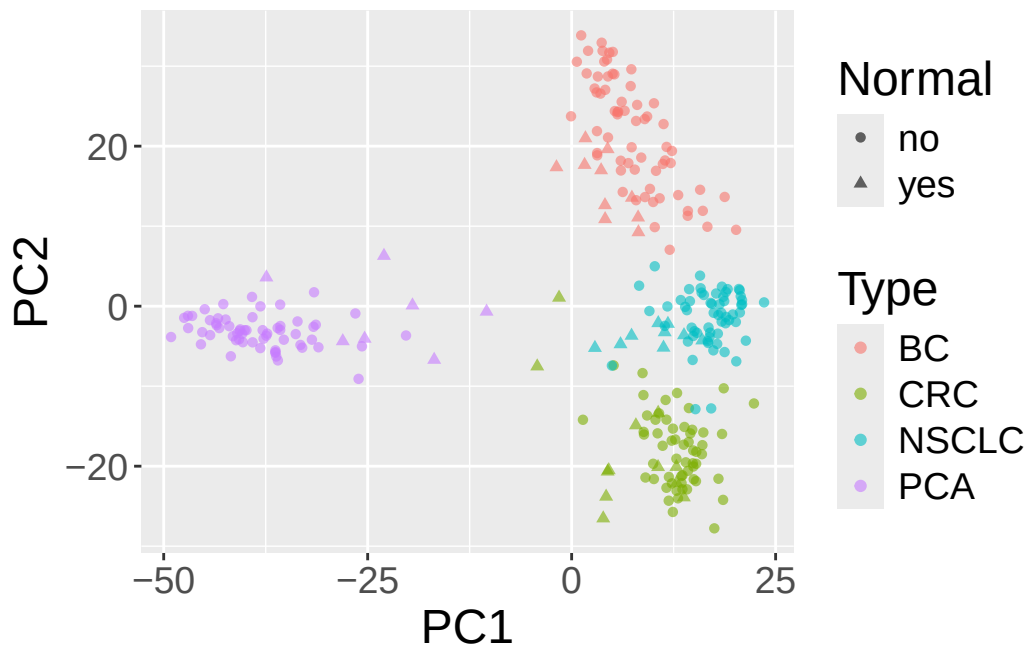
We can take a look at the variance explained by each principal component as a plot.

```
library(ggplot2)
pca_var <- pca_results$sdev^2 / sum(pca_results $sdev^2)
plot(pca_var, xlab = "Principal Component", ylab = "Variance Explained",
     main = "Variance Explained by Principal Components", type = "b")
```



Now, we are going to plot the results of the PCA, coloring the points by cancer type and using different shapes for normal and tumor samples.

```
library(ggplot2)
ggplot(pca_df, aes(x=PC1,y=PC2,shape=Normal,color=Type)) +
  geom_point(alpha=0.6) + theme(text=element_text(size = 18))
```



In this case, the x-axis is the first principal component and the y-axis is the second principal component.

- What do you see?
- What about additional principal components?
- Bonus: Try using the `GGally` package to plot principal components (using the `ggpairs` function).
- Bonus: Calculate the variance explained by each principal component and plot the results.

24. Storing experiments with SummarizedExperiment

An RNA-seq or microarray experiment hands you several things at once: a matrix of numbers (counts or intensities), a description of every feature down the side (which gene is in each row), and a description of every sample across the top (which condition, which patient). Keep these in three separate variables and they *will* drift out of sync — you drop a bad sample from the count matrix, forget to drop it from the sample table, and now your “treated vs. control” labels point at the wrong columns. Mismatches exactly like this have produced retracted papers.

SummarizedExperiment is Bioconductor’s answer: one object that holds the assay matrix, the feature data, and the sample data together, and keeps them aligned for you whenever you subset.

It is, in many ways, a modern replacement for the historical **ExpressionSet**. The main difference is that **SummarizedExperiment** is more flexible about its feature (row) information: features can be described either by genomic ranges (**GRanges**) or by an arbitrary table (**DataFrame**). That flexibility makes it a natural fit for sequencing experiments such as RNA-seq and ChIP-seq.

24.1. What you’ll learn

By the end of this chapter you will be able to:

- Describe the three coordinated parts of a **SummarizedExperiment**: `assays()`, `rowData()/rowRanges()`, and `colData()`.
- Subset an experiment by sample or feature and trust that the metadata follows.
- Reach into assays, sample data, and experiment-wide metadata with the accessor functions.
- Build a **SummarizedExperiment** from raw matrices and data frames.

24. Storing experiments with *SummarizedExperiment*

This chapter assumes you’ve met **GRanges** and **DataFrame**. If “genomic ranges” isn’t familiar yet, the *GenomicRanges* chapters come earlier in the Bioconductor part of the book — skim those first.

💡 Installing the packages

The examples below use the **SummarizedExperiment** class and the **airway** example dataset. Install both once with **BiocManager**:

```
BiocManager::install(c("SummarizedExperiment", "airway"))
```

24.2. Anatomy of a *SummarizedExperiment*

The *SummarizedExperiment* package contains two classes: **SummarizedExperiment** and **RangedSummarizedExperiment**.

SummarizedExperiment is a matrix-like container where rows represent features of interest (e.g. genes, transcripts, exons, etc.) and columns represent samples. The objects contain one or more assays, each represented by a matrix-like object of numeric or other mode. The rows of a **SummarizedExperiment** object represent features of interest. Information about these features is stored in a **DataFrame** object, accessible using the function **rowData()**. Each row of the **DataFrame** provides information on the feature in the corresponding row of the **SummarizedExperiment** object. Columns of the **DataFrame** represent different attributes of the features of interest, e.g., gene or transcript IDs, etc.

RangedSummarizedExperiment is the “child” of the **SummarizedExperiment** class, which means that all the methods on **SummarizedExperiment** also work on a **RangedSummarizedExperiment**.

The fundamental difference between the two classes is that the rows of a **RangedSummarizedExperiment** object represent genomic ranges of interest instead of a **DataFrame** of features. The **RangedSummarizedExperiment** ranges are described by a **GRanges** or a **GRangesList** object, accessible using the **rowRanges()** function.

Figure 24.1 displays the class geometry and highlights the vertical (column) and horizontal (row) relationships.

i A mental model: three linked sheets

Picture a spreadsheet workbook with three sheets that are *locked together*:

- the **assay** sheet — the matrix of numbers, features (genes) $\times$ samples;

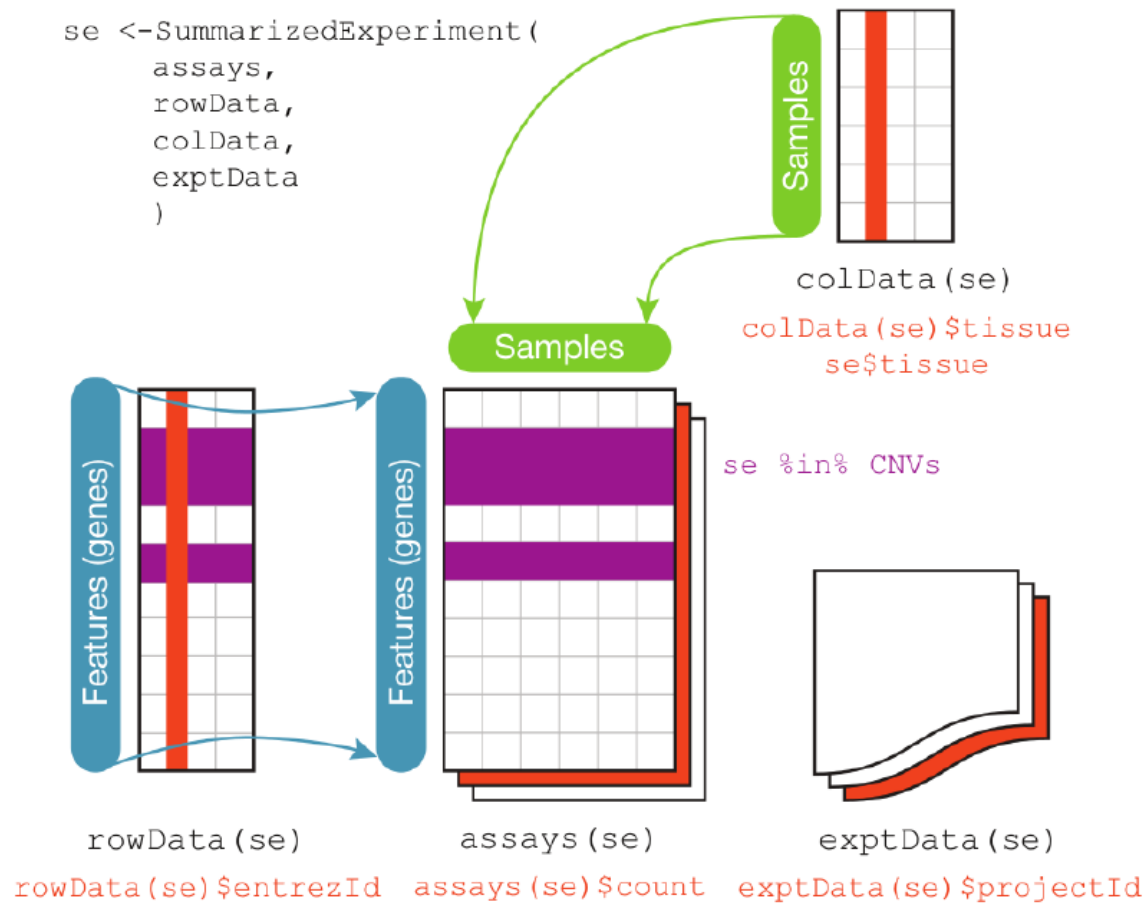


Figure 24.1.: Summarized Experiment. There are three main components, the `colData()`, the `rowData()` and the `assays()`. The accessors for the various parts of a complete `SummarizedExperiment` object match the names.

24. Storing experiments with *SummarizedExperiment*

- the **row** sheet (`rowData/rowRanges`) — one row of annotation per feature, lined up with the assay's rows;
- the **column** sheet (`colData`) — one row of annotation per sample, lined up with the assay's columns.

The magic is the lock: when you delete a sample, all three sheets update together, so the numbers and their labels can never disagree. Every accessor below is just a way of looking at one of these sheets.

24.2.1. Assays

The `airway` package contains an example dataset from an RNA-seq experiment of read counts per gene for airway smooth muscle cells. These data are stored in a `RangedSummarizedExperiment` object that holds read counts for tens of thousands of gene transcripts across 8 samples (we'll confirm the exact dimensions in a moment).

Loading required package: `airway`

```
library(SummarizedExperiment)
data(airway, package="airway")
se <- airway
se
```

```
class: RangedSummarizedExperiment
dim: 63677 8
metadata(1): ''
assays(1): counts
rownames(63677): ENSG000000000003 ENSG000000000005 ... ENSG00000273492
               ENSG00000273493
rowData names(10): gene_id gene_name ... seq_coord_system symbol
colnames(8): SRR1039508 SRR1039509 ... SRR1039520 SRR1039521
colData names(9): SampleName cell ... Sample BioSample
```

Read that printout like a label on the box: `dim` tells you it's 63,677 features across 8 samples, and the `assays`, `rowRanges`, and `colData` lines preview the three linked sheets from the model above. The sections that follow open each one in turn.

To retrieve the experiment data from a `SummarizedExperiment` object one can use the `assays()` accessor. An object can have multiple assay datasets each of which can be

24. Storing experiments with SummarizedExperiment

accessed using the `$` operator. The `airway` dataset contains only one assay (`counts`). Here each row represents a gene transcript and each column one of the samples.

```
assays(se)$counts
```

	SRR103950	SRR103951	SRR103952	SRR103953	SRR103954	SRR103955	SRR103956
ENSG000000000000379	448	873	408	1138	1047	770	572
ENSG00000000000050	0	0	0	0	0	0	0
ENSG0000000000004167	515	621	365	587	799	417	508
ENSG0000000000004360	211	263	164	245	331	233	229
ENSG0000000000004660	55	40	35	78	63	76	60
ENSG0000000000009380	0	2	0	1	0	0	0
ENSG0000000000009751	3679	6177	4252	6721	11027	5176	7995
ENSG00000000000010333	1062	1733	881	1424	1439	1359	1109
ENSG00000000000010519	380	595	493	820	714	696	704
ENSG00000000000011694	236	464	175	658	584	360	269

24.2.2. 'Row' (regions-of-interest) data

The `rowRanges()` accessor is used to view the range information for a `RangedSummarizedExperiment`. (Note if this were the parent `SummarizedExperiment` class we'd use `rowData()`). The data are stored in a `GRangesList` object, where each list element corresponds to one gene transcript and the ranges in each `GRanges` correspond to the exons in the transcript.

```
rowRanges(se)
```

```
GRangesList object of length 63677:
```

\$ENSG000000000003

GRanges object with 17 ranges and 2 metadata columns:

seqnames		ranges	strand	exon_id	exon_name
<Rle>	<IRanges>	<Rle>	<integer>	<character>	
[1]	X 99883667-99884983	-	667145	ENSE00001459322	
[2]	X 99885756-99885863	-	667146	ENSE00000868868	
[3]	X 99887482-99887565	-	667147	ENSE00000401072	
[4]	X 99887538-99887565	-	667148	ENSE00001849132	
[5]	X 99888402-99888536	-	667149	ENSE00003554016	
...	...	...	...	...	...
[13]	X 99890555-99890743	-	667156	ENSE00003512331	

24. Storing experiments with SummarizedExperiment

```
[14]      X 99891188-99891686      - |      667158 ENSE00001886883
[15]      X 99891605-99891803      - |      667159 ENSE00001855382
[16]      X 99891790-99892101      - |      667160 ENSE00001863395
[17]      X 99894942-99894988      - |      667161 ENSE00001828996
-----
seqinfo: 722 sequences (1 circular) from an unspecified genome

...
<63676 more elements>
```

24.2.3. 'Column' (sample) data

Sample meta-data describing the samples can be accessed using `colData()`, and is a `DataFrame` that can store any number of descriptive columns for each sample row.

```
colData(se)
```

DataFrame with 8 rows and 9 columns

	SampleName	cell	dex	albut	Run	avgLength
	<factor>	<factor>	<factor>	<factor>	<factor>	<integer>
SRR1039508	GSM1275862	N61311	untrt	untrt	SRR1039508	126
SRR1039509	GSM1275863	N61311	trt	untrt	SRR1039509	126
SRR1039512	GSM1275866	N052611	untrt	untrt	SRR1039512	126
SRR1039513	GSM1275867	N052611	trt	untrt	SRR1039513	87
SRR1039516	GSM1275870	N080611	untrt	untrt	SRR1039516	120
SRR1039517	GSM1275871	N080611	trt	untrt	SRR1039517	126
SRR1039520	GSM1275874	N061011	untrt	untrt	SRR1039520	101
SRR1039521	GSM1275875	N061011	trt	untrt	SRR1039521	98

	Experiment	Sample	BioSample
	<factor>	<factor>	<factor>
SRR1039508	SRX384345	SRS508568	SAMN02422669
SRR1039509	SRX384346	SRS508567	SAMN02422675
SRR1039512	SRX384349	SRS508571	SAMN02422678
SRR1039513	SRX384350	SRS508572	SAMN02422670
SRR1039516	SRX384353	SRS508575	SAMN02422682
SRR1039517	SRX384354	SRS508576	SAMN02422673
SRR1039520	SRX384357	SRS508579	SAMN02422683
SRR1039521	SRX384358	SRS508580	SAMN02422677

24. Storing experiments with *SummarizedExperiment*

Each row here is one sample, and each column is something we know about that sample — including `dex`, which records whether the cells were treated with dexamethasone. That `dex` column is what makes the next operation possible.

This sample metadata can be accessed using the `$` accessor, which makes it easy to subset the entire object by a given phenotype.

```
# subset for only those samples treated with dexamethasone
se[, se$dex == "trt"]
```

```
class: RangedSummarizedExperiment
dim: 63677 4
metadata(1): ''
assays(1): counts
rownames(63677): ENSG000000000003 ENSG000000000005 ... ENSG00000273492
               ENSG00000273493
rowData names(10): gene_id gene_name ... seq_coord_system symbol
colnames(4): SRR1039509 SRR1039513 SRR1039517 SRR1039521
colData names(9): SampleName cell ... Sample BioSample
```

24.2.4. Experiment-wide metadata

Meta-data describing the experimental methods and publication references can be accessed using `metadata()`.

```
metadata(se)
```

```
[[1]]
Experiment data
  Experimenter name: Himes BE
  Laboratory: NA
  Contact information:
  Title: RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive
  URL: http://www.ncbi.nlm.nih.gov/pubmed/24926665
  PMIDs: 24926665

  Abstract: A 226 word abstract is available. Use 'abstract' method.
```

Note that `metadata()` is just a simple list, so it is appropriate for *any* experiment wide metadata the user wishes to save, such as storing model formulas.

24. Storing experiments with SummarizedExperiment

```
metadata(se)$formula <- counts ~ dex + albut
```

```
metadata(se)
```

```
[[1]]
```

```
Experiment data
```

```
  Experimenter name: Himes BE
```

```
  Laboratory: NA
```

```
  Contact information:
```

```
  Title: RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive
```

```
  URL: http://www.ncbi.nlm.nih.gov/pubmed/24926665
```

```
  PMIDs: 24926665
```

```
  Abstract: A 226 word abstract is available. Use 'abstract' method.
```

```
$formula
```

```
counts ~ dex + albut
```

24.3. Common operations on SummarizedExperiment

24.3.1. Subsetting

- [Performs two dimensional subsetting, just like subsetting a matrix or data frame.

```
# subset the first five transcripts and first three samples
se[1:5, 1:3]
```

```
class: RangedSummarizedExperiment
```

```
dim: 5 3
```

```
metadata(2): '' formula
```

```
assays(1): counts
```

```
rownames(5): ENSG00000000003 ENSG00000000005 ENSG000000000419
```

```
  ENSG000000000457 ENSG000000000460
```

```
rowData names(10): gene_id gene_name ... seq_coord_system symbol
```

```
colnames(3): SRR1039508 SRR1039509 SRR1039512
```

```
colData names(9): SampleName cell ... Sample BioSample
```

- \$ operates on colData() columns, for easy sample extraction.

```
se[, se$cell == "N61311"]
```

```
class: RangedSummarizedExperiment
dim: 63677 2
metadata(2): '' formula
assays(1): counts
rownames(63677): ENSG000000000003 ENSG000000000005 ... ENSG00000273492
               ENSG00000273493
rowData names(10): gene_id gene_name ... seq_coord_system symbol
colnames(2): SRR1039508 SRR1039509
colData names(9): SampleName cell ... Sample BioSample
```

24.3.2. Getters and setters

- `rowRanges()` / `(rowData(), colData(), metadata())`

```
counts <- matrix(1:15, 5, 3, dimnames=list(LETTERS[1:5], LETTERS[1:3]))

dates <- SummarizedExperiment(assays=list(counts=counts),
                              rowData=DataFrame(month=month.name[1:5], day=1:5))

# Subset all January assays
dates[rowData(dates)$month == "January", ]
```

```
class: SummarizedExperiment
dim: 1 3
metadata(0):
assays(1): counts
rownames(1): A
rowData names(2): month day
colnames(3): A B C
colData names(0):
```

- `assay()` versus `assays()` There are two accessor functions for extracting the assay data from a `SummarizedExperiment` object. `assays()` operates on the entire list of assay data as a whole, while `assay()` operates on only one assay at a time. `assay(x, i)` is simply a convenience function which is equivalent to `assays(x)[[i]]`.

24. Storing experiments with SummarizedExperiment

```
assays(se)
```

```
List of length 1  
names(1): counts
```

```
assays(se)[[1]][1:5, 1:5]
```

	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG000000000003	679	448	873	408	1138
ENSG000000000005	0	0	0	0	0
ENSG000000000419	467	515	621	365	587
ENSG000000000457	260	211	263	164	245
ENSG000000000460	60	55	40	35	78

```
# assay defaults to the first assay if no i is given  
assay(se)[1:5, 1:5]
```

	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG000000000003	679	448	873	408	1138
ENSG000000000005	0	0	0	0	0
ENSG000000000419	467	515	621	365	587
ENSG000000000457	260	211	263	164	245
ENSG000000000460	60	55	40	35	78

```
assay(se, 1)[1:5, 1:5]
```

	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG000000000003	679	448	873	408	1138
ENSG000000000005	0	0	0	0	0
ENSG000000000419	467	515	621	365	587
ENSG000000000457	260	211	263	164	245
ENSG000000000460	60	55	40	35	78

24.3.3. Range-based operations

- `subsetByOverlaps()` `SummarizedExperiment` objects support all of the `findOverlaps()` methods and associated functions. This includes `subsetByOverlaps()`, which makes it easy to subset a `SummarizedExperiment` object by an interval.

In the next code block, we define a region of interest (a single genomic range) and then subset our `SummarizedExperiment` to the genes that overlap it.

```
# Subset for only the genes that fall in the interval 100,000 to 1,100,000
# of chromosome 1
roi <- GRanges(seqnames="1", ranges=IRanges(start=100000, end=1100000))
sub_se <- subsetByOverlaps(se, roi)
sub_se
```

```
class: RangedSummarizedExperiment
dim: 74 8
metadata(2): '' formula
assays(1): counts
rownames(74): ENSG00000131591 ENSG00000177757 ... ENSG00000272512
             ENSG00000273443
rowData names(10): gene_id gene_name ... seq_coord_system symbol
colnames(8): SRR1039508 SRR1039509 ... SRR1039520 SRR1039521
colData names(9): SampleName cell ... Sample BioSample
```

```
dim(sub_se)
```

```
[1] 74 8
```

Notice that `subsetByOverlaps()` worked on the *ranges* (`rowRanges`) but returned a full `SummarizedExperiment` — the assay matrix and `colData` came along automatically, kept in sync with the smaller set of genes.

24.4. Constructing a SummarizedExperiment

To construct a `SummarizedExperiment` object, you need to provide the following components: - **assays**: A list of matrices or matrix-like objects containing the data. - **rowData**: A `DataFrame` containing information about the features (rows). - **colData**: A `DataFrame`

24. Storing experiments with SummarizedExperiment

containing information about the samples (columns). - +/- **metadata**: A list containing additional metadata about the experiment.

For a nearly real example, we will use the DeRisi dataset. We'll start with the original data, which is a **data.frame** with the first couple of columns containing the gene information and the rest of the columns containing the data.

```
# Load the DeRisi dataset
deRisi <- read.csv("https://raw.githubusercontent.com/seandavi/RBiocBook/refs/heads/main/data/deRisi.csv")
head(deRisi)
```

	ORF	Name	R1	R2	R3	R4	R5	R6	R7	R1.Bkg	R2.Bkg	R3.Bkg
1	YHR007C	ERG11	7896	7484	14679	14617	9853	7599	6490	1155	1984	1323
2	YBR218C	PYC2	12144	11177	10241	4820	4950	7047	17035	1074	1694	1243
3	YAL051W	FUN43	4478	6435	6230	6848	5111	7180	4497	1140	1950	1649
4	YAL053W		6343	8243	6743	3304	3556	4694	3849	1020	1897	1196
5	YAL054C	ACS1	1542	3044	2076	1695	1753	4806	10802	1082	1940	1504
6	YAL055W		1769	3243	2094	1367	1853	3580	1956	975	1821	1185
	R4.Bkg	R5.Bkg	R6.Bkg	R7.Bkg	G1	G2	G3	G4	G5	G6	G7	G1.Bkg
1	1171	914	2445	981	8432	7173	11736	16798	12315	16111	13931	2404
2	876	1211	2444	742	11509	10226	13372	6500	6255	9024	6904	2148
3	1183	898	2637	927	5865	5895	5345	6302	5400	7933	5026	2422
4	881	1045	2518	697	6762	7454	6323	3595	4689	5660	4145	2107
5	1108	902	2610	980	3138	3785	2419	2114	2763	3561	1897	2405
6	851	1047	2536	698	2844	4069	2583	1651	2530	3484	1550	1674
	G2.Bkg	G3.Bkg	G4.Bkg	G5.Bkg	G6.Bkg	G7.Bkg						
1	2561	1598	1506	1696	2667	1244						
2	2527	1641	1196	1553	2569	848						
3	2496	1902	1501	1644	2808	1154						
4	2663	1607	1162	1577	2544	857						
5	2528	1847	1445	1713	2767	1142						
6	2648	1591	1114	1528	2668	870						

To convert this to a **SummarizedExperiment**, we need to extract the assay data, row data, and column data. The assay data will be the numeric values in the data frame, the row data will be the gene information, and the column data will be the sample information.

24.4.1. rowData, or feature information

Let's start with the **rowData**, which will be a **DataFrame** containing the gene information. We can use the first two columns of the data frame for this purpose.

24. Storing experiments with SummarizedExperiment

```
rdata <- deRisi[, 1:2]
head(rdata)
```

```
      ORF  Name
1 YHR007C ERG11
2 YBR218C  PYC2
3 YAL051W FUN43
4 YAL053W
5 YAL054C  ACS1
6 YAL055W
```

24.4.2. colData, or sample information

Next, we will create the `colData`, which will be a `DataFrame` containing the sample information. Since the sample information really isn't in the dataset, we will create a simple `DataFrame` with sample names.

```
cdata <- DataFrame(sample=paste("Sample", 0:6), timepoint = 0:6,
                    hours = c(0, 9.5, 11.5, 13.5, 15.5, 18.5, 20.5))
head(cdata)
```

DataFrame with 6 rows and 3 columns

	sample	timepoint	hours
	<character>	<integer>	<numeric>
1	Sample 0	0	0.0
2	Sample 1	1	9.5
3	Sample 2	2	11.5
4	Sample 3	3	13.5
5	Sample 4	4	15.5
6	Sample 5	5	18.5

24.4.3. assays, or the data

Remember that the DeRisi dataset has four *different* assays,

assay	description
R	Red fluorescence

24. Storing experiments with SummarizedExperiment

assay	description
G	Green fluorescence
Rb	Red background fluorescence
Gb	Green background fluorescence

We will create a list of matrices, one for each assay. The matrices will be the numeric values in the data frame, excluding the first two columns.

```
R <- as.matrix(deRisi[, 3:9])
G <- as.matrix(deRisi[, 10:16])
Rb <- as.matrix(deRisi[, 17:23])
Gb <- as.matrix(deRisi[, 24:30])
```

When we create a SummarizedExperiment object, the “constructor” will check to see that the colnames of the matrices in the list are the same as the *rownames* of the `colData` `DataFrame`, and that the rownames of the matrices in the list are the same as the *rownames* of the `rowData` `DataFrame`.

So, we need to fix that all up. Let’s start with the rownames of the `rowData` `DataFrame`:

```
rownames(rdata) <- rdata$ORF
```

Now, let’s set the rownames of the `coldata` `DataFrame` to the sample names:

```
rownames(cdata) <- cdata$sample
```

Now, we can fix the rownames and colnames of the matrices for our R, G, Rb, and Gb assays:

```
rownames(R) <- rdata$ORF
rownames(G) <- rdata$ORF
rownames(Rb) <- rdata$ORF
rownames(Gb) <- rdata$ORF
colnames(R) <- cdata$sample
colnames(G) <- cdata$sample
colnames(Rb) <- cdata$sample
colnames(Gb) <- cdata$sample
```

Take a look at the matrices to make sure they look right.

24.4.4. Putting it all together

```
se <- SummarizedExperiment(assays=list(R=R, G=G, Rb=Rb, Gb=Gb), rowData=rdata, colData=cdata)
```

24.4.5. Getting logRatios

Now that we have a `SummarizedExperiment` object, we can easily compute the log ratios of the Red and Green foreground fluorescence. This is a common operation in microarray data analysis.

```
logRatios <- log2(
  (assay(se, "R") - assay(se, "Rb")) / (assay(se, "G") - assay(se, "Gb"))
)
```

Warning: NaNs produced

```
assays(se)$logRatios <- logRatios
```

Now, we've added a new assay to the `SummarizedExperiment` object called `logRatios`. This assay contains the log ratios of the Red and Green foreground fluorescence.

```
assays(se)
```

```
List of length 5
names(5): R G Rb Gb logRatios
```

And if we want to access the log ratios, we can do so using the `assay()` method:

```
head(assay(se, "logRatios"))
```

	Sample 0	Sample 1	Sample 2	Sample 3	Sample 4	Sample 5
YHR007C	-1.2204686	NaN	NaN	2.70275677	1.6545902	5.260867
YBR218C	NaN	NaN	2.9757832	2.39231742	1.9319816	3.983313
YAL051W	0.1135715	NaN	NaN	NaN	-1.3681061	2.138654
YAL053W	-1.3753298	NaN	NaN	0.05044902	1.0906497	5.215440
YAL054C	0.2706476	0.333657387	0.0000000	0.31420165	0.3165815	NaN

24. Storing experiments with SummarizedExperiment

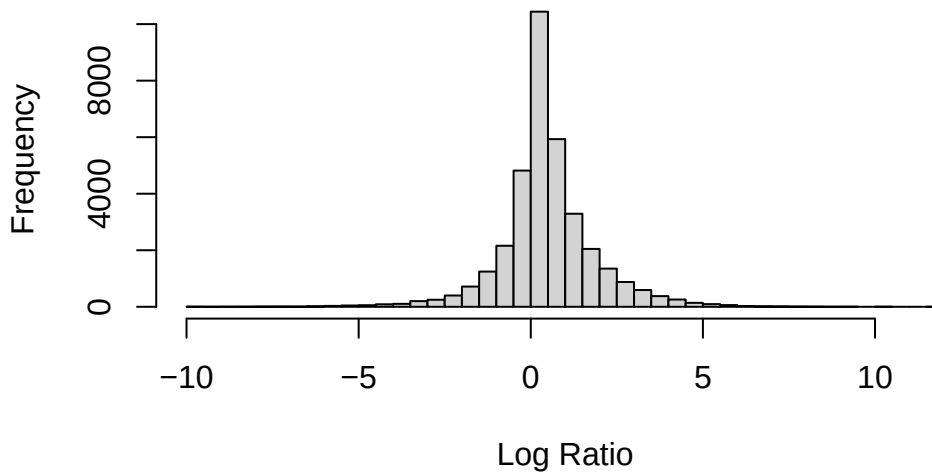
```
YAL055W 0.6209723 -0.001745548 0.2683547 0.11082813 0.4931189 NaN
      Sample 6
YHR007C 4.8223618
YBR218C      NaN
YAL051W 1.2205754
YAL053W 0.8875253
YAL054C      NaN
YAL055W      NaN
```

```
## OR
head(assays(se)$logRatios)
```

```
      Sample 0      Sample 1 Sample 2 Sample 3 Sample 4 Sample 5
YHR007C -1.2204686      NaN      NaN 2.70275677 1.6545902 5.260867
YBR218C      NaN      NaN 2.9757832 2.39231742 1.9319816 3.983313
YAL051W 0.1135715      NaN      NaN      NaN -1.3681061 2.138654
YAL053W -1.3753298      NaN      NaN 0.05044902 1.0906497 5.215440
YAL054C 0.2706476 0.333657387 0.0000000 0.31420165 0.3165815      NaN
YAL055W 0.6209723 -0.001745548 0.2683547 0.11082813 0.4931189      NaN
      Sample 6
YHR007C 4.8223618
YBR218C      NaN
YAL051W 1.2205754
YAL053W 0.8875253
YAL054C      NaN
YAL055W      NaN
```

```
hist(assays(se)$logRatios, breaks=50, main="Log Ratios of Red and Green Foreground Fluoresc
```

Log Ratios of Red and Green Foreground Fluorescence



24.5. Summary

A *SummarizedExperiment* bundles an assay matrix together with aligned feature data (*rowData/rowRanges*) and sample data (*colData*) into a single object that stays internally consistent when you subset. In this chapter you toured the accessors (`assays()`, `rowData()`, `colData()`, `metadata()`), subset an experiment by phenotype and by genomic range, and built a *SummarizedExperiment* from scratch out of the DeRisi microarray data. From here on, most Bioconductor analysis packages will hand you — or expect — an object shaped exactly like this.

24.6. Exercises

The *airway* dataset is a good sandbox. Load a fresh copy so you're not relying on any object created earlier in the chapter:

```
data(airway, package = "airway")
air <- airway
```

24. Storing experiments with *SummarizedExperiment*

1. How many genes (rows) and samples (columns) does `air` have? Which accessor tells you each sample's treatment status?
2. Subset `air` to just the untreated samples (`dex == "untrt"`) and confirm that both the assay matrix and the `colData` shrank to match.
3. Pull the `counts` assay for the first 5 genes and first 3 samples into a plain matrix.

i Solutions

```
# 1. shape and the treatment column
dim(air)           # genes x samples
```

```
[1] 63677      8
```

```
colData(air)$dex    # treatment per sample
```

```
[1] untrt trt   untrt trt   untrt trt   untrt trt
Levels: trt untrt
```

```
# 2. subsetting carries the metadata along
untrt <- air[, air$dex == "untrt"]
dim(untrt)           # fewer columns; same number of rows
```

```
[1] 63677      4
```

```
ncol(colData(untrt)) # colData has the same columns, fewer rows
```

```
[1] 9
```

```
# 3. reach into one assay and slice it
assay(air, "counts")[1:5, 1:3]
```

	SRR1039508	SRR1039509	SRR1039512
ENSG000000000003	679	448	873
ENSG000000000005	0	0	0
ENSG000000000419	467	515	621
ENSG000000000457	260	211	263
ENSG000000000460	60	55	40

The key idea in exercise 2: you subset the *object*, and `colData` follows in lockstep — you never have to remember to subset the sample table separately.

25. Microbiome analysis

The human microbiome is a complex ecosystem of bacteria, viruses, fungi, and other microorganisms that live in and on the human body. These microorganisms play a crucial role in human health and disease, influencing everything from digestion and metabolism to immune function and mental health. Understanding the composition and function of the human microbiome is a rapidly growing area of research with implications for a wide range of fields, including medicine, agriculture, and environmental science.

Microbiome analysis involves studying the microbial communities that inhabit different environments, such as the human gut, skin, mouth, and soil. The analysis of microbiome data typically involves quantifying the abundance of different microorganisms in a given sample and comparing the microbial communities across different samples or conditions. This can help researchers identify patterns, relationships, and associations between microbial communities and various factors, such as health status, diet, lifestyle, and environmental conditions.

In this chapter, we will assume that the data has already been processed and cleaned, and we will focus on analyzing the microbiome data using R.

25.1. Getting started

We'll be using the `mia` package for microbiome analysis in R. The `mia` package provides a suite of tools for analyzing microbiome data, including functions for loading, processing, and visualizing microbiome data. The package is designed to work with the `SummarizedExperiment` and `TreeSummarizedExperiment` classes, which are specialized data structures for storing microbiome data in R.

To get started, you'll need to install the `mia` package from GitHub using the `BiocManager` package. If you don't already have the `BiocManager` package installed, you can install it using the following command:

```
# Install the mia package
BiocManager::install("microbiome/mia")
```

i Note

Note: The `mia` package is part of the Bioconductor project, but in this case, we are installing it directly from GitHub using the `microbiome/mia` repository. This is a common practice when working with development versions of packages or with packages that are not yet available on Bioconductor.

Once the `mia` package is installed, you can load it into your R session using the following command:

```
# Load the mia package
library(mia)
```

Now that the `mia` package is loaded, we can load the microbiome dataset that we will be working with.

25.2. Accessing microbiome datasets

There are many publicly available microbiome datasets that you can use for analysis. Bioconductor provides several packages that contain curated microbiome datasets that facilitate access to and loading microbiome data into R for analysis.

[curatedMetagenomicData](#) is a large collection of curated human microbiome datasets, provided as TreeSE objects (Pasolli et al. 2017). The resource provides curated human microbiome data including gene families, marker abundance, marker presence, pathway abundance, pathway coverage, and relative abundance for samples from different body sites. See the package homepage for more details on data availability and access.

The [microbiomeDataSets](#) package provides a collection of curated microbiome datasets for teaching and research purposes. The package contains several example datasets that can be used to explore different aspects of microbiome analysis, such as alpha and beta diversity, differential abundance analysis, and visualization.

The [mia](#) package and several other packages provide example datasets for learning and testing the functions in the package. These datasets are typically small and easy to work with, making them ideal for learning how to analyze microbiome data in R.

25.3. Data containers

Bioconductor provides several specialized data structures for storing and working with microbiome data in R. These data structures are designed to handle the complex and high-dimensional nature of microbiome data and provide a convenient and efficient way to store, manipulate, and analyze microbiome data in R.

[SummarizedExperiment \(SE\)](#) (Morgan et al. 2020) is a generic and highly optimized container for complex data structures. It has become a common choice for analyzing various types of biomedical profiling data, such as RNAseq, ChIP-Seq, microarrays, flow cytometry, proteomics, and single-cell sequencing.

[TreeSummarizedExperiment](#) (Huang 2020) was developed as an extension to incorporate hierarchical information (such as phylogenetic trees and sample hierarchies) and reference sequences.

25.4. Loading a microbiome dataset

We will be using the `curatedMetagenomicData` package to load the `FengQ_2015` dataset. This dataset contains microbiome data from a study by [Feng et al. \(2015\)](#). The `curatedMetagenomicData` package provides a convenient interface for accessing microbiome datasets that have been pre-processed and curated for analysis.

Note

Take a moment to familiarize yourself with at least the title and abstract from the paper. We won't be reproducing the analysis in the paper, but it is helpful to connect the code in this workflow to the biology and research.

```
# Again, before using a package, we must install it  
# just once.  
BiocManager::install('curatedMetagenomicData')
```

Once installed, we can use the package.

Ignore the details of the next few lines of code for now, but suffice it to say that what the code will do for us is to load a dataset for downstream analysis.

25. Microbiome analysis

```
# Load the FengQ_2015 dataset
library(curatedMetagenomicData)
tse <- curatedMetagenomicData("FengQ_2015.relative_abundance", dryrun = FALSE, rownames = "
```

```
$`2021-03-31.FengQ_2015.relative_abundance`
```

dropping rows without rowTree matches:

```
k__Bacteria|p__Actinobacteria|c__Coriobacteriia|o__Coriobacteriales|f__Atopobiaceae|g__01
k__Bacteria|p__Actinobacteria|c__Coriobacteriia|o__Coriobacteriales|f__Coriobacteriaceae|
k__Bacteria|p__Actinobacteria|c__Coriobacteriia|o__Coriobacteriales|f__Coriobacteriaceae|
k__Bacteria|p__Firmicutes|c__Bacilli|o__Bacillales|f__Bacillales_unclassified|g__Gemella|
k__Bacteria|p__Firmicutes|c__Bacilli|o__Lactobacillales|f__Carnobacteriaceae|g__Granulica
k__Bacteria|p__Firmicutes|c__Clostridia|o__Clostridiales|f__Ruminococcaceae|g__Ruminococc
k__Bacteria|p__Firmicutes|c__Erysipelotrichia|o__Erysipelotrichales|f__Erysipelotrichacea
k__Bacteria|p__Proteobacteria|c__Betaproteobacteria|o__Burkholderiales|f__Sutterellaceae|
k__Bacteria|p__Synergistetes|c__Synergistia|o__Synergistales|f__Synergistaceae|g__Cloacib
```

Duplicated labels were made unique.

This code goes out to the internet and downloads the dataset for you. The `tse` object is a `TreeSummarizedExperiment` object that contains the microbiome data from the Feng et al. (2015) study. The `TreeSummarizedExperiment` class is a specialized data structure for storing microbiome data in R, and it is used by the `mia` package for microbiome analysis.

Tip

If you are working with your own microbiome data, you will often need to load it into R using functions like `read.csv()` or `read.table()` to read tabular data files. Make sure your data is properly formatted and cleaned before loading it into R for analysis.

Once the data is loaded into R, we can start exploring the data to understand its structure and contents. The `tse` object is a `TreeSummarizedExperiment` object, which is a specialized data structure for storing microbiome data in R. It works quite like a `data.frame` in, but with many specialized structures for storing microbiome experiment data as shown in Figure 25.1. We can use various functions to explore the data stored in the `tse` object. See the [TreeSummarizedExperiment](#) package for more details.

This section provides an introduction to these data containers. In microbiome data science, these containers link taxonomic abundance tables with rich side information on the features (microbes) and samples. Taxonomic abundance data can be obtained by 16S rRNA

25. Microbiome analysis

amplicon or metagenomic sequencing, phylogenetic microarrays, or by other means. Many microbiome experiments include multiple versions and types of data generated independently or derived from each other through transformation or agglomeration.

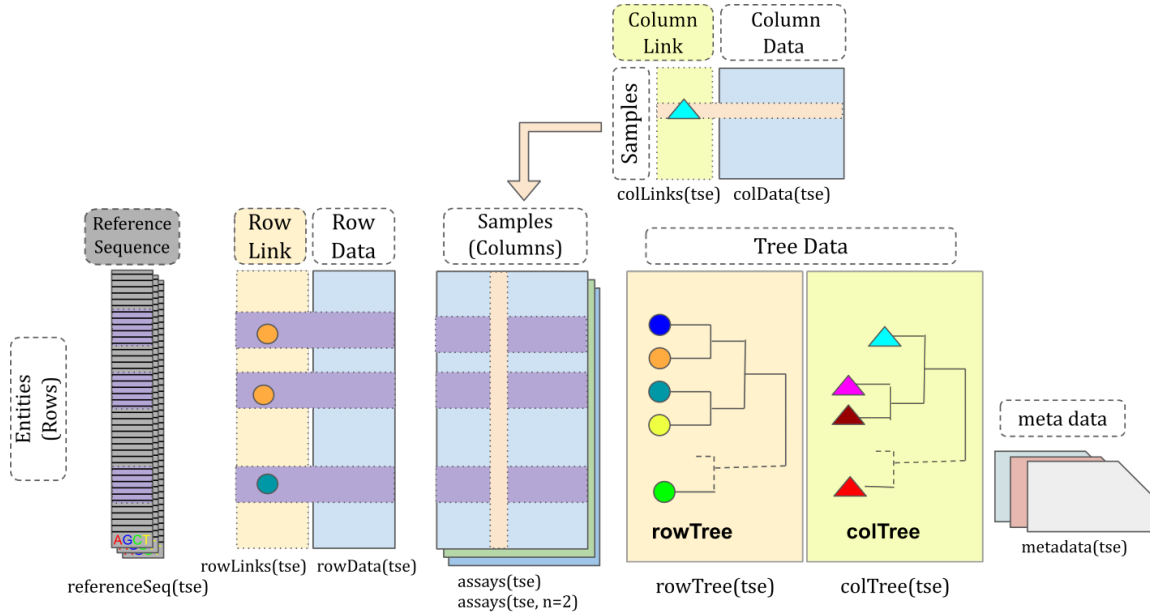


Figure 25.1.: Anatomy of a `TreeSummarizedExperiment` object. Compared to the `SingleCellExperiment` objects, `TreeSummarizedExperiment` has five additional slots. **rowTree**: the hierarchical structure on the rows of the assays. **rowLinks**: the link information between rows of the assays and the rowTree. **colTree**: the hierarchical structure on the columns of the assays. **colLinks**: the link information between columns of the assays and the colTree. **referenceSeq** (optional): the reference sequence data per feature (row).

To get an overview of the data, we can simply type the name of the object in the R console:

```
# Print the object
tse
```

```
class: TreeSummarizedExperiment
dim: 606 154
metadata(0):
assays(1): relative_abundance
rownames(606): species:Faecalibacterium prausnitzii
```

```

species:Streptococcus salivarius ... species:Lactobacillus
kullabergensis species:Lactobacillus kimbladii
rowData names(7): superkingdom phylum ... genus species
colnames(154): SID31004 SID31009 ... SID532832 SID532915
colData names(28): study_name subject_id ... triglycerides hba1c
reducedDimNames(0):
mainExpName: NULL
altExpNames(0):
rowLinks: a LinkDataFrame (606 rows)
rowTree: 1 phylo tree(s) (10430 leaves)
colLinks: NULL
colTree: NULL

```

Observe how the output of the `tse` object will show you the dimensions of the data, the metadata associated with the samples and features, etc. However, the data themselves are not printed to the console due to the large size of the dataset. The `TreeSummarizedExperiment` object is a complex data structure that contains multiple components, including the abundance data, sample information, feature information, and other metadata.

Next, we'll look at some of the key components of the `TreeSummarizedExperiment` object and how to access and manipulate them.

25.4.1. Experimental measurements: assays

When working with high-throughput biological data, the results of a workflow (often complex), is usually something resembling an Excel spreadsheet with samples as columns and features (genes, microbes, CpG sites, ...) as rows. The quantitative information (the numbers) are called “assay” data since they result from running an “assay.” The microbiome is the collection of all microbes (such as bacteria, viruses, fungi, etc.) in the body. When studying these microbes, data is needed, and that's where assays come in. An assay is a way of measuring the presence and abundance of different types of microbes in a sample. For example, if you want to know how many bacteria of a certain type are in your gut, you can use an assay to measure this. When storing assays, the original data is count-based. However, the original count-based taxonomic abundance tables may undergo different transformations, such as logarithmic, Centered Log-Ratio (CLR), or relative abundance. These are typically stored in **assays**.

The assays slot contains the experimental data as multiple count matrices. The result of assays is a list of matrices.

```
assays(tse)
```

```
List of length 1
names(1): relative_abundance
```

Individual assays can be accessed via `assay()` function. The result is a matrix. And recall that when we subset a matrix, we use `[]` and specify the `[<rows>, <columns>]`.

```
# just the first 5 rows and columns to make the results easier to see
assay(tse, "relative_abundance")[1:5, 1:5]
```

	SID31004	SID31009	SID31021	SID31030
species:Faecalibacterium prausnitzii	13.58862	5.18307	6.09213	2.79834
species:Streptococcus salivarius	11.00755	0.13897	0.17498	0.15692
species:Anaerostipes hadrus	8.26264	7.55176	8.26219	1.07529
species:Bacteroides stercoris	5.21961	0.00000	0.01189	0.59787
species:Collinsella aerofaciens	4.58481	3.68522	2.67519	2.19600
	SID31071			
species:Faecalibacterium prausnitzii	12.96581			
species:Streptococcus salivarius	0.26242			
species:Anaerostipes hadrus	3.07124			
species:Bacteroides stercoris	0.06529			
species:Collinsella aerofaciens	7.94793			

You can, of course, choose a different number of rows or columns if you like.

25.4.2. Sample information: colData

`colData` contains information about the samples used in the study. This information can include details such as the sample ID, the primers used in the analysis, the barcodes associated with the sample (truncated or complete), the type of sample (e.g. soil, fecal, mock) and a description of the sample. You can also add experimental details about samples as you see fit (e.g., the sample preparation data, prep kit lot number, or anything else you might want to track).

```
colData(tse)
```

25. Microbiome analysis

DataFrame with 154 rows and 28 columns

	study_name	subject_id	body_site	antibiotics_current_use	
	<character>	<character>	<character>		<character>
SID31004	FengQ_2015	SID31004	stool		no
SID31009	FengQ_2015	SID31009	stool		no
SID31021	FengQ_2015	SID31021	stool		no
SID31030	FengQ_2015	SID31030	stool		no
SID31071	FengQ_2015	SID31071	stool		no
...	...	...	...		...
SID532796	FengQ_2015	SID532796	stool		no
SID532802	FengQ_2015	SID532802	stool		no
SID532826	FengQ_2015	SID532826	stool		no
SID532832	FengQ_2015	SID532832	stool		no
SID532915	FengQ_2015	SID532915	stool		no
	study_condition		disease	age	age_category
	<character>		<character>	<integer>	<character>
SID31004		CRC	CRC;fatty_liver;hype..	64	adult
SID31009		control	fatty_liver;hyperten..	68	senior
SID31021		control	healthy	60	adult
SID31030		adenoma	adenoma;fatty_liver;..	70	senior
SID31071		control	fatty_liver	68	senior
...		...	...	...	...
SID532796		control	fatty_liver;hyperten..	73	senior
SID532802		control	healthy	68	senior
SID532826		control	T2D;fatty_liver;hype..	78	senior
SID532832		adenoma	adenoma;fatty_liver;..	68	senior
SID532915		control	healthy	43	adult
	gender	country	non_westernized	sequencing_platform	
	<character>	<character>	<character>		<character>
SID31004	male	AUT	no	IlluminaHiSeq	
SID31009	male	AUT	no	IlluminaHiSeq	
SID31021	female	AUT	no	IlluminaHiSeq	
SID31030	male	AUT	no	IlluminaHiSeq	
SID31071	male	AUT	no	IlluminaHiSeq	
...	...	...	...	...	...
SID532796	female	AUT	no	IlluminaHiSeq	
SID532802	male	AUT	no	IlluminaHiSeq	
SID532826	male	AUT	no	IlluminaHiSeq	
SID532832	female	AUT	no	IlluminaHiSeq	
SID532915	female	AUT	no	IlluminaHiSeq	
	DNA_extraction_kit		PMID	number_reads	number_bases

25. Microbiome analysis

	<character>	<character>	<integer>	<numeric>
SID31004	MoBio	25758642	40898340	3649611221
SID31009	MoBio	25758642	66107961	6196998053
SID31021	MoBio	25758642	60789126	5708593447
SID31030	MoBio	25758642	50300253	4741158330
SID31071	MoBio	25758642	51945426	4913627034
...	...	...	...	...
SID532796	MoBio	25758642	50845712	4754848864
SID532802	MoBio	25758642	41480415	3868696049
SID532826	MoBio	25758642	35346002	3348368467
SID532832	MoBio	25758642	42184599	3951224696
SID532915	MoBio	25758642	51594677	4886998833
	minimum_read_length	median_read_length	NCBI_accession	
	<integer>	<integer>	<character>	
SID31004	30	93	ERR688505;ERR688358	
SID31009	30	96	ERR688506;ERR688359	
SID31021	30	96	ERR688507;ERR688360	
SID31030	30	96	ERR688508;ERR688361	
SID31071	30	97	ERR688509;ERR688362	
...	...	...	...	
SID532796	30	95	ERR710428;ERR710419	
SID532802	30	96	ERR710429;ERR710420	
SID532826	30	97	ERR710430;ERR710421	
SID532832	30	96	ERR710431;ERR710422	
SID532915	30	96	ERR710432;ERR710423	
	curator	BMI	disease_subtype	diet
	<character>	<numeric>	<character>	<character>
SID31004	Paolo_Manghi;Marisa_..	29.35	carcinoma	vegetarian
SID31009	Paolo_Manghi;Marisa_..	32.00	NA	omnivore
SID31021	Paolo_Manghi;Marisa_..	22.10	NA	omnivore
SID31030	Paolo_Manghi;Marisa_..	34.11	advancedadenoma	omnivore
SID31071	Paolo_Manghi;Marisa_..	23.45	NA	omnivore
...	...	...	...	...
SID532796	Paolo_Manghi;Marisa_..	26.56	NA	omnivore
SID532802	Paolo_Manghi;Marisa_..	23.53	NA	omnivore
SID532826	Paolo_Manghi;Marisa_..	31.22	NA	omnivore
SID532832	Paolo_Manghi;Marisa_..	27.55	advancedadenoma	omnivore
SID532915	Paolo_Manghi;Marisa_..	22.65	NA	omnivore
	hdl	ldl	tnm	triglycerides
	<numeric>	<numeric>	<character>	<numeric>
SID31004	28	92	tn0m0	172
				5.2

25. Microbiome analysis

SID31009	50	157	NA	101	NA
SID31021	60	122	NA	53	NA
SID31030	74	146	NA	89	NA
SID31071	40	231	NA	258	NA
...	...	...	...	...	...
SID532796	61	114	NA	100	5.4
SID532802	67	158	NA	80	5.8
SID532826	32	90	NA	212	6.9
SID532832	51	184	NA	141	5.6
SID532915	80	132	NA	51	5.1

As you can see, there is a lot of information stored in the `colData` slot, including sample IDs, study conditions, and other metadata associated with the samples. This information is essential for understanding the context of the microbiome data and for performing downstream analyses.

💡 Use sample `colData` as database of information about your samples.

By keeping a relatively complete `colData`, you can answer questions about your findings without having to refer back to external Excel spreadsheets or lab notebooks.

25.4.3. Measured feature information: `rowData`

`rowData` contains data on the features of the analyzed samples. This is particularly important in the microbiome field for storing taxonomic information. This taxonomic information is extremely important for understanding the composition and diversity of the microbiome in each sample analyzed. It enables identification of the different types of microorganisms present in samples. It also allows you to explore the relationships between microbiome composition and various environmental or health factors.

```
head(rowData(tse))
```

DataFrame with 6 rows and 7 columns

	superkingdom	phylum	class
	<character>	<character>	<character>
species:Faecalibacterium prausnitzii	Bacteria	Firmicutes	Clostridia
species:Streptococcus salivarius	Bacteria	Firmicutes	Bacilli
species:Anaerostipes hadrus	Bacteria	Firmicutes	Clostridia
species:Bacteroides stercoris	Bacteria	Bacteroidota	Bacteroidia

25. Microbiome analysis

species:Collinsella aerofaciens	Bacteria Actinobacteria Coriobacteriia
species:Bifidobacterium longum	Bacteria Actinobacteria Actinomycetia
	order family
	<character> <character>
species:Faecalibacterium prausnitzii	Eubacteriales Oscillospiraceae
species:Streptococcus salivarius	Lactobacillales Streptococcaceae
species:Anaerostipes hadrus	Eubacteriales Lachnospiraceae
species:Bacteroides stercoris	Bacteroidales Bacteroidaceae
species:Collinsella aerofaciens	Coriobacteriales Coriobacteriaceae
species:Bifidobacterium longum	Bifidobacteriales Bifidobacteriaceae
	genus species
	<character> <character>
species:Faecalibacterium prausnitzii	Faecalibacterium Faecalibacterium pra..
species:Streptococcus salivarius	Streptococcus Streptococcus saliva..
species:Anaerostipes hadrus	Anaerostipes Anaerostipes hadrus
species:Bacteroides stercoris	Bacteroides Bacteroides stercoris
species:Collinsella aerofaciens	Collinsella Collinsella aerofaci..
species:Bifidobacterium longum	Bifidobacterium Bifidobacterium longum

25.4.4. rowTree

Phylogenetic trees also play an important role in the microbiome field. The `TreeSE` class can keep track of features and node relations via two functions, `rowTree` and `rowLinks`.

A tree can be accessed via `rowTree()` as `phylo` object.

i What is this tree of which we speak?

The “tree” in a `TreeSummarizedExperiment` is a phylogenetic tree describing the inferred evolutionary relationships among various biological species or other entities based upon similarities and differences in their physical or genetic characteristics. The tree of life is a phylogenetic tree that shows the evolutionary relationships among all living organisms on Earth.

In the context of microbiome analysis, phylogenetic trees are used to represent the evolutionary relationships between different microbial taxa based on their genetic sequences. These trees can help researchers understand the diversity and relatedness of different microbes in a sample and provide insights into the evolutionary history of the microbial community.

The `phylo` class in R is used to represent phylogenetic trees and provides functions for working with and visualizing these trees. The `ggtree` package is a popular pack-

age for visualizing phylogenetic trees in R and provides a wide range of options for customizing the appearance of the tree.

```
rowTree(tse)
```

Phylogenetic tree with 10430 tips and 10429 internal nodes.

Tip labels:

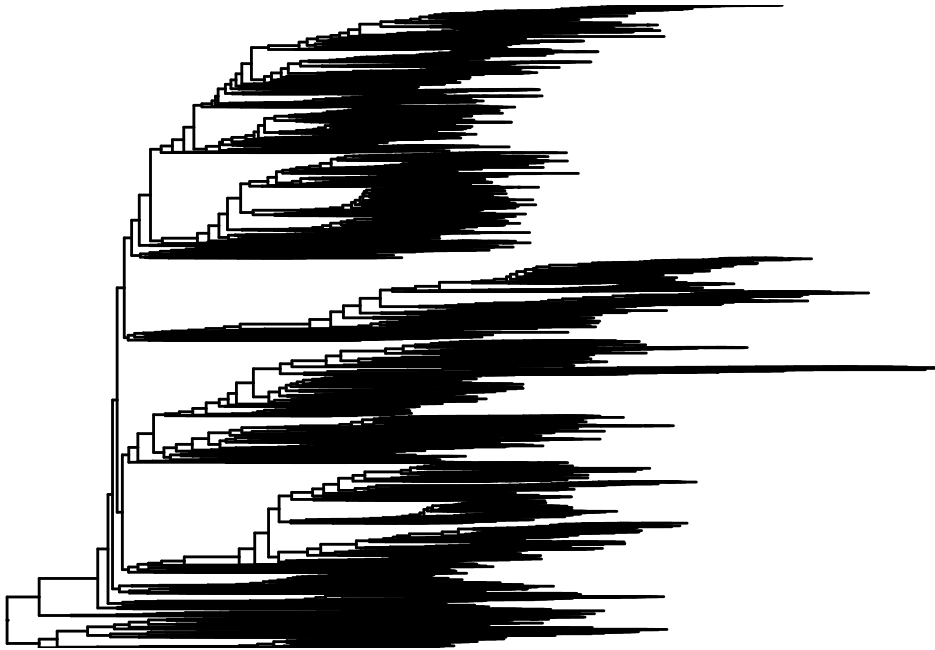
k__Archaea|p__Candidatus_Micrarchaeota|c__Candidatus_Micrarchaeota_unclassified|o__Candidatus_Micrarchaeota_unclassified

Rooted; includes branch length(s).

The `rowTree` slot contains information about the hierarchical structure of the data, such as the relationships between different microbial taxa based on their genetic sequences. This information can be used to explore the evolutionary relationships between different microbes and to visualize the diversity and relatedness of the microbial community in a sample.

We can visualize the phylogenetic tree using the `ggtree` package in R. The `ggtree` package provides functions for visualizing phylogenetic trees in a variety of formats, including circular, rectangular, and radial layouts. Here, we'll use the `ggtree` package to visualize the phylogenetic tree stored in the `rowTree` slot of the `TreeSummarizedExperiment` object.

```
library(ggtree)
ggtree(rowTree(tse))
```



See the [ggtree](#) package for more details on visualizing tree structures in R and, for more fun, the [ggtree book](#).

The `rowLink` slot contains information about the relationships between the features and the tree structure. This information can be used to link the features in the data to the nodes in the tree and to explore the relationships between the features based on their abundance profiles.

```
rowLinks(tse)
```

LinkDataFrame with 606 rows and 5 columns

	nodeLab	nodeLab_alias
	<character>	<character>
species:Faecalibacterium prausnitzii	k__Bacteria p__Firmi..	alias_3547
species:Streptococcus salivarius	k__Bacteria p__Firmi..	alias_4750
species:Anaerostipes hadrus	k__Bacteria p__Firmi..	alias_3579
species:Bacteroides stercoris	k__Bacteria p__Bacte..	alias_5690
species:Collinsella aerofaciens	k__Bacteria p__Actin..	alias_3019
...	...	...
species:Streptococcus gallolyticus	k__Bacteria p__Firmi..	alias_4745
species:Lactobacillus apis	k__Bacteria p__Firmi..	alias_4851

25. Microbiome analysis

species:Anaerostipes sp. 494a	k__Bacteria p__Firmi..	alias_3580
species:Lactobacillus kullabergensis	k__Bacteria p__Firmi..	alias_4855
species:Lactobacillus kimbladai	k__Bacteria p__Firmi..	alias_4854
	nodeNum	isLeaf
	whichTree	
	<integer>	<logical>
		<character>
species:Faecalibacterium prausnitzii	3547	TRUE
species:Streptococcus salivarius	4750	TRUE
species:Anaerostipes hadrus	3579	TRUE
species:Bacteroides stercoris	5690	TRUE
species:Collinsella aerofaciens	3019	TRUE
...	...	...
species:Streptococcus gallolyticus	4745	TRUE
species:Lactobacillus apis	4851	TRUE
species:Anaerostipes sp. 494a	3580	TRUE
species:Lactobacillus kullabergensis	4855	TRUE
species:Lactobacillus kimbladai	4854	TRUE

Both `rowTree` and `rowLinks` are optional components of the `TreeSummarizedExperiment` object, but when present, they provide valuable information about the hierarchical structure of the data and the relationships between the features in the data.

25.5. Wrangling and subsetting

One of the huge advantages of using the `SummarizedExperiment` classes in R/Bioconductor is their ability to encapsulate all the information necessary to describe the samples (`colData`), measured features (`rowData`, `rowTree`), and the measurements themselves (assays). When we manipulate or subset these objects, all the associated information about samples, features, and assays are also subsetted.

The `mia` package provides several functions for wrangling and subsetting microbiome data. These functions allow you to filter, transform, and manipulate the data to extract the information you need for analysis.

25.5.1. Subsetting samples

You can subset the samples in a `TreeSummarizedExperiment` object similarly to how you would subset a `data.frame`. For example, to subset out data based on the age category of the samples, you can use the following code:

25. Microbiome analysis

```
head(colData(tse)$age)
```

```
[1] 64 68 60 70 68 66
```

Since accessing the colData is such a common way of interacting with a `SummarizedExperiment`, we can also access the colData as if it were a `data.frame`.

```
head(tse$age)
```

```
[1] 64 68 60 70 68 66
```

```
# and get a summary of all patient ages
summary(tse$age)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
43.00	63.00	68.00	66.85	72.00	86.00

Age is a numeric variable. Let's focus on older patients.

```
# Include all rows, but only columns (samples) who are older than 50
tse_subset_by_age <- tse[, tse$age > 50]
tse_subset_by_age
```

```
class: TreeSummarizedExperiment
dim: 606 146
metadata(0):
assays(1): relative_abundance
rownames(606): species:Faecalibacterium prausnitzii
               species:Streptococcus salivarius ... species:Lactobacillus
               kullabergensis species:Lactobacillus kimbladii
rowData names(7): superkingdom phylum ... genus species
colnames(146): SID31004 SID31009 ... SID532826 SID532832
colData names(28): study_name subject_id ... triglycerides hba1c
reducedDimNames(0):
mainExpName: NULL
altExpNames(0):
rowLinks: a LinkDataFrame (606 rows)
```

25. Microbiome analysis

```
rowTree: 1 phylo tree(s) (10430 leaves)
colLinks: NULL
colTree: NULL
```

The `tse_subset_by_age` is a copy of the original `tse`, but with only the 50+ year old patients. If you want to double-check, you can look at the ages in this new `tse_subset_by_age`.

```
summary(tse_subset_by_age$age)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
52.00	63.25	68.00	68.02	72.00	86.00

25.5.2. Agglomerating data

The microbiome features (organisms) that are measured may be measured at different taxonomic levels. For example, the data may be available at the species, genus, family, or phylum level. Agglomerating data is the process of summarizing the data at a higher taxonomic level by combining the abundances of the lower-level taxa. This can be useful for simplifying the data and reducing the dimensionality of the data.

The `agglomerateByRank` function in the `mia` package can be used to agglomerate the data at a specified taxonomic rank. For example, to agglomerate the data at the phylum level, you can use the following code:

```
new_tse <- agglomerateByRank(tse, rank = "phylum")
# observe the new object, particularly the number of features (rows)
new_tse
```

```
class: TreeSummarizedExperiment
dim: 13 154
metadata(1): agglomerated_by_rank
assays(1): relative_abundance
rownames(13): Actinobacteria Ascomycota ... Synergistetes
  Verrucomicrobia
rowData names(7): superkingdom phylum ... genus species
colnames(154): SID31004 SID31009 ... SID532832 SID532915
colData names(28): study_name subject_id ... triglycerides hba1c
reducedDimNames(0):
```

```
mainExpName: NULL
altExpNames(0):
rowLinks: a LinkDataFrame (13 rows)
rowTree: 1 phylo tree(s) (13 leaves)
colLinks: NULL
colTree: NULL
```

The resulting `TreeSummarizedExperiment` object contains the data aggregated at the phylum level, with the abundances of the lower-level taxa combined to create a summary at the phylum level. This can make the data easier to work with and interpret, especially when dealing with large and complex microbiome datasets.

25.6. Community indices

In the field of microbiome ecology several indices to describe samples and community of samples are available. In this vignette we just want to give a very brief introduction.

25.6.1. Alpha diversity

diversity in microbiology is measured by several indices:

- species richness (total number of species)
- equitability (distribution of species within a microbiome)
- diversity (combination of the two)

Functions for calculating alpha and beta diversity indices are available. Using `addAlpha` multiple diversity indices are calculated by default and results are stored automatically in `colData`.

i Why do we care about alpha diversity?

Alpha diversity is a “statistic” that somehow represents how complex or “diverse” a microbiome sample is. You can imagine lots of situations that might significantly alter the diversity of a sample, such as antibiotic use, systemic chemotherapy, autoimmune conditions, dietary changes, etc. When we have two experimental conditions such as antibiotic use vs no antibiotic use, the alpha diversity in the two sample groups may differ significantly. That association, if present, could be helpful in understanding biology or even developing a biomarker.

25. Microbiome analysis

In the code below we calculate the Shannon and observed diversity indices.

```
tse <- addAlpha(tse, index = "shannon", assay.type = "relative_abundance")
tse <- addAlpha(tse, index = "observed", assay.type = "relative_abundance")
colnames(colData(tse))
```

```
[1] "study_name"      "subject_id"
[3] "body_site"       "antibiotics_current_use"
[5] "study_condition" "disease"
[7] "age"             "age_category"
[9] "gender"          "country"
[11] "non_westernized" "sequencing_platform"
[13] "DNA_extraction_kit" "PMID"
[15] "number_reads"     "number_bases"
[17] "minimum_read_length" "median_read_length"
[19] "NCBI_accession"   "curator"
[21] "BMI"              "disease_subtype"
[23] "diet"             "hdl"
[25] "ldl"              "tnm"
[27] "triglycerides"    "hba1c"
[29] "shannon"          "observed"
```

The **observed** index is a simple count of the number of species present in a sample, while the **shannon** index takes into account both the number of species and their relative abundances. These indices provide information about the diversity and richness of the microbial community in each sample.

And we can look at the resulting `colData()` which now contains the calculated alpha diversity indices.

```
colData(tse)
```

DataFrame with 154 rows and 30 columns

	study_name	subject_id	body_site	antibiotics_current_use
	<character>	<character>	<character>	<character>
SID31004	FengQ_2015	SID31004	stool	no
SID31009	FengQ_2015	SID31009	stool	no
SID31021	FengQ_2015	SID31021	stool	no
SID31030	FengQ_2015	SID31030	stool	no
SID31071	FengQ_2015	SID31071	stool	no

25. Microbiome analysis

...	...	...	...	...
SID532796	FengQ_2015	SID532796	stool	no
SID532802	FengQ_2015	SID532802	stool	no
SID532826	FengQ_2015	SID532826	stool	no
SID532832	FengQ_2015	SID532832	stool	no
SID532915	FengQ_2015	SID532915	stool	no
	study_condition	disease	age	age_category
	<character>	<character>	<integer>	<character>
SID31004	CRC	CRC;fatty_liver;hype..	64	adult
SID31009	control	fatty_liver;hyperten..	68	senior
SID31021	control	healthy	60	adult
SID31030	adenoma	adenoma;fatty_liver;..	70	senior
SID31071	control	fatty_liver	68	senior
...	...	...	...	...
SID532796	control	fatty_liver;hyperten..	73	senior
SID532802	control	healthy	68	senior
SID532826	control	T2D;fatty_liver;hype..	78	senior
SID532832	adenoma	adenoma;fatty_liver;..	68	senior
SID532915	control	healthy	43	adult
	gender	country	non_westernized	sequencing_platform
	<character>	<character>	<character>	<character>
SID31004	male	AUT	no	IlluminaHiSeq
SID31009	male	AUT	no	IlluminaHiSeq
SID31021	female	AUT	no	IlluminaHiSeq
SID31030	male	AUT	no	IlluminaHiSeq
SID31071	male	AUT	no	IlluminaHiSeq
...	...	...	...	...
SID532796	female	AUT	no	IlluminaHiSeq
SID532802	male	AUT	no	IlluminaHiSeq
SID532826	male	AUT	no	IlluminaHiSeq
SID532832	female	AUT	no	IlluminaHiSeq
SID532915	female	AUT	no	IlluminaHiSeq
	DNA_extraction_kit	PMID	number_reads	number_bases
	<character>	<character>	<integer>	<numeric>
SID31004	MoBio	25758642	40898340	3649611221
SID31009	MoBio	25758642	66107961	6196998053
SID31021	MoBio	25758642	60789126	5708593447
SID31030	MoBio	25758642	50300253	4741158330
SID31071	MoBio	25758642	51945426	4913627034
...	...	...	...	...
SID532796	MoBio	25758642	50845712	4754848864

25. Microbiome analysis

SID532802	MoBio	25758642	41480415	3868696049		
SID532826	MoBio	25758642	35346002	3348368467		
SID532832	MoBio	25758642	42184599	3951224696		
SID532915	MoBio	25758642	51594677	4886998833		
	minimum_read_length	median_read_length	NCBI_accession			
	<integer>	<integer>	<character>			
SID31004	30	93	ERR688505;ERR688358			
SID31009	30	96	ERR688506;ERR688359			
SID31021	30	96	ERR688507;ERR688360			
SID31030	30	96	ERR688508;ERR688361			
SID31071	30	97	ERR688509;ERR688362			
...	...	...	...			
SID532796	30	95	ERR710428;ERR710419			
SID532802	30	96	ERR710429;ERR710420			
SID532826	30	97	ERR710430;ERR710421			
SID532832	30	96	ERR710431;ERR710422			
SID532915	30	96	ERR710432;ERR710423			
	curator	BMI	disease_subtype	diet		
	<character>	<numeric>	<character>	<character>		
SID31004	Paolo_Manghi;Marisa_..	29.35	carcinoma	vegetarian		
SID31009	Paolo_Manghi;Marisa_..	32.00	NA	omnivore		
SID31021	Paolo_Manghi;Marisa_..	22.10	NA	omnivore		
SID31030	Paolo_Manghi;Marisa_..	34.11	advancedadenoma	omnivore		
SID31071	Paolo_Manghi;Marisa_..	23.45	NA	omnivore		
...	...	...	...	...		
SID532796	Paolo_Manghi;Marisa_..	26.56	NA	omnivore		
SID532802	Paolo_Manghi;Marisa_..	23.53	NA	omnivore		
SID532826	Paolo_Manghi;Marisa_..	31.22	NA	omnivore		
SID532832	Paolo_Manghi;Marisa_..	27.55	advancedadenoma	omnivore		
SID532915	Paolo_Manghi;Marisa_..	22.65	NA	omnivore		
	hdl	ldl	tnm	triglycerides	hba1c	shannon
	<numeric>	<numeric>	<character>	<numeric>	<numeric>	<numeric>
SID31004	28	92	t1n0m0	172	5.2	3.28935
SID31009	50	157	NA	101	NA	3.11014
SID31021	60	122	NA	53	NA	3.38207
SID31030	74	146	NA	89	NA	2.60265
SID31071	40	231	NA	258	NA	3.14500
...	...	...	...	...	...	...
SID532796	61	114	NA	100	5.4	3.66184
SID532802	67	158	NA	80	5.8	3.19522
SID532826	32	90	NA	212	6.9	3.16254

25. Microbiome analysis

SID532832	51	184	NA	141	5.6	3.21467
SID532915	80	132	NA	51	5.1	3.48822

observed
<numeric>

SID31004	101
SID31009	106
SID31021	116
SID31030	107
SID31071	113
...	...
SID532796	133
SID532802	111
SID532826	103
SID532832	94
SID532915	127

The `scater` package provides some convenient plotting functions that are designed to work with the `SummarizedExperiment` class.

```
library(scater)
```

Loading required package: `scuttle`

Loading required package: `ggplot2`

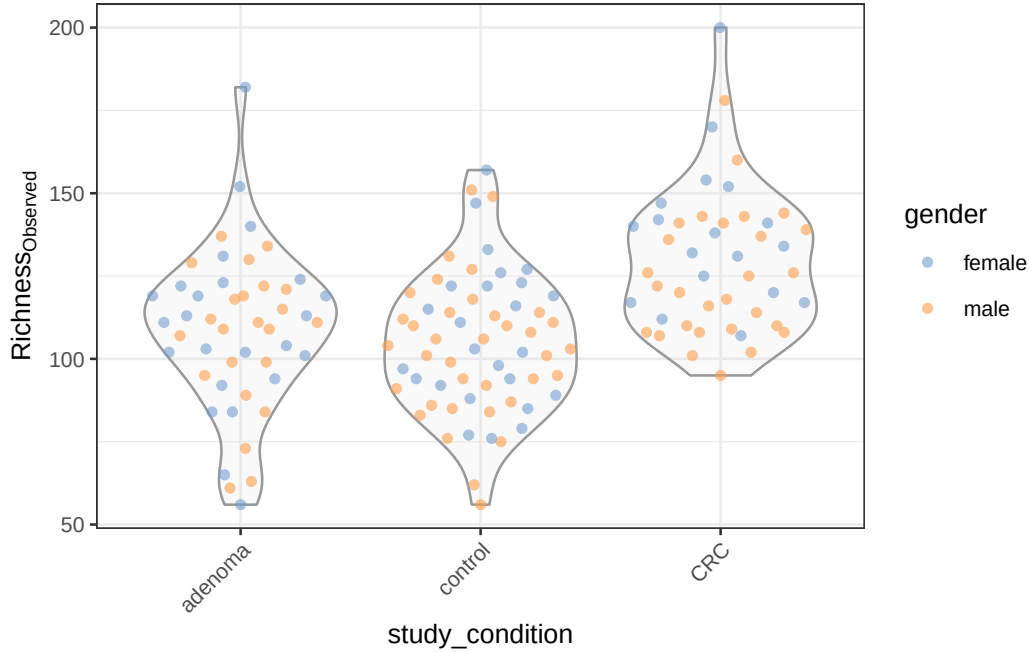
Attaching package: `'scater'`

The following object is masked from `'package:ggtree'`:

`multiplot`

```
plotColData(tse,
  "observed",          # the y-axis value that we just calculated
  "study_condition",   # The x-axis
  colour_by = "gender" # Color the points by gender
) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(y = expression(Richness[Observed])) # and give a meaningful label
```

25. Microbiome analysis



25.6.2. Beta diversity

Community similarity refers to the way microorganisms resemble each other in terms of their composition and abundance of different microbial taxa. This can help us understand to what degree different samples resemble each other and finding key information. In microbiome analysis however, it's more common to measure the dissimilarity/Beta diversity between two samples A and B using the Bray-Curtis measure which is defined as follows:

$$BC_{ij} = \frac{\sum_k |A_k - B_k|}{\sum_k (A_k + B_k)}$$

where A_k and B_k are the abundances of taxon k in samples A and B, respectively. The Bray-Curtis dissimilarity ranges from 0 (identical communities) to 1 (completely different communities).

The `mia` package provides functions for calculating beta diversity indices, such as the Bray-Curtis dissimilarity, Jaccard similarity, and UniFrac distance. These indices can be used to compare the microbial communities between different samples and to identify patterns and relationships between samples based on their microbial composition.

25. Microbiome analysis

```
# Run PCoA on relabundance assay with Bray-Curtis distances
library(vegan)

tse <- runMDS(tse,
  FUN = vegdist,
  method = "bray",
  assay.type = "relative_abundance",
  name = "MDS_bray"
)
```

This code is a bit to unpack. It calculates the Bray-Curtis dissimilarity between samples in the `tse` object using the `vegdist` function from the `vegan` package. The resulting dissimilarity matrix is then used to perform a Principal Coordinate Analysis (PCoA) using the `runMDS` function from the `scater` package. The PCoA is a dimensionality reduction technique that projects the high-dimensional Bray-Curtis dissimilarity matrix onto a lower-dimensional space while preserving the pairwise distances between samples.

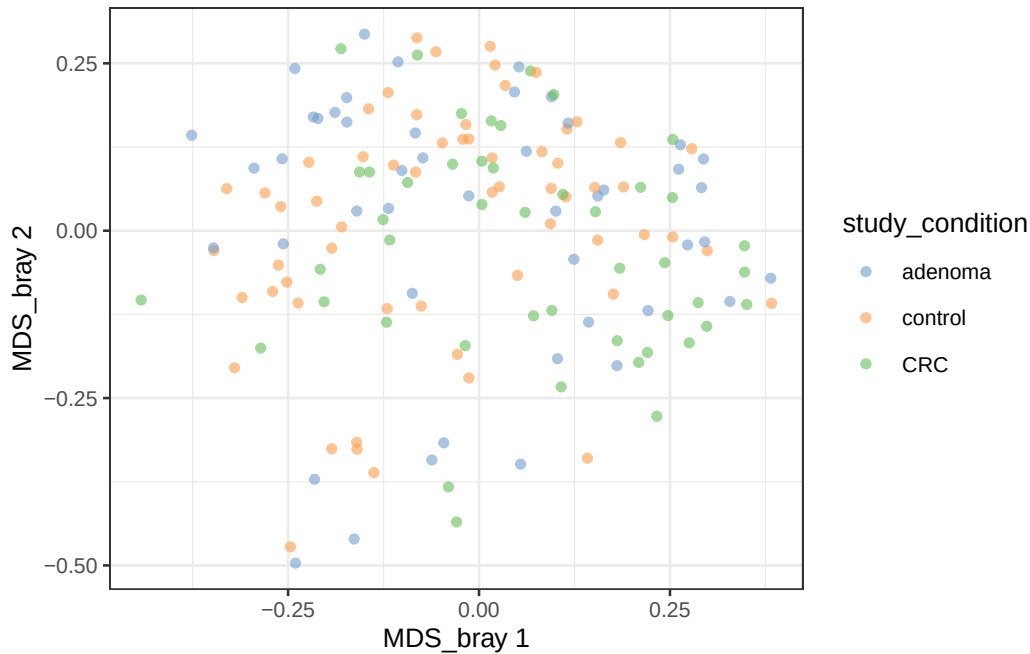
The resulting `TreeSummarizedExperiment` object now contains the Bray-Curtis dissimilarity matrix in the `MDS_bray` slot, which can be used to visualize the relationships between samples based on their microbial composition.

We can visualize the results of the PCoA using the `plotReducedDim` function from the `scater` package. This function creates a plot of the reduced dimensions (PCoA) and colors the samples based on a specified variable, such as the study condition

```
# Create ggplot object
p <- plotReducedDim(tse, "MDS_bray", colour_by = "study_condition")

print(p)
```

25. Microbiome analysis



You can experiment with different variables to color the samples and explore the relationships between samples based on their microbial composition.

25.7. Microbial composition

Let's now look at the microbial composition of the samples in the `tse` object. The microbial composition refers to the relative abundances of different microbial taxa in each sample. This information can provide insights into the diversity and structure of the microbial community in each sample and help identify patterns and relationships between samples based on their microbial composition.

The `mia` package provides functions for visualizing the microbial composition of samples, such as bar plots, heatmaps, and stacked bar plots. These plots can help you explore the relative abundances of different microbial taxa in each sample and identify patterns and relationships between samples based on their microbial composition.

25.7.1. Abundance

We can start by creating a bar plot of the top 40 most abundant taxa in the dataset. This plot shows the relative abundances of the top 40 taxa in each sample, with the taxa sorted

25. Microbiome analysis

by abundance.

```
library(miaViz)
```

Loading required package: ggraph

Attaching package: 'miaViz'

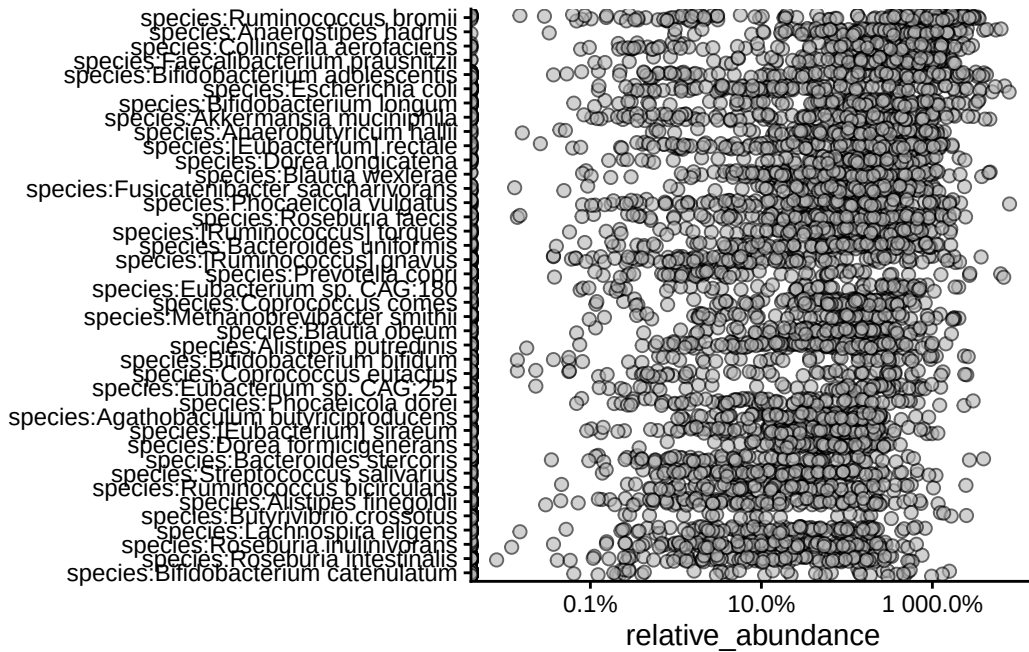
The following object is masked from 'package:mia':

plotNMDS

```
plotAbundanceDensity(tse,  
  layout = "jitter",  
  assay.type = "relative_abundance",  
  n = 40  
) +  
  scale_x_log10(label = scales::percent)
```

Warning in scale_x_log10(label = scales::percent): log-10 transformation introduced infinite values.

25. Microbiome analysis



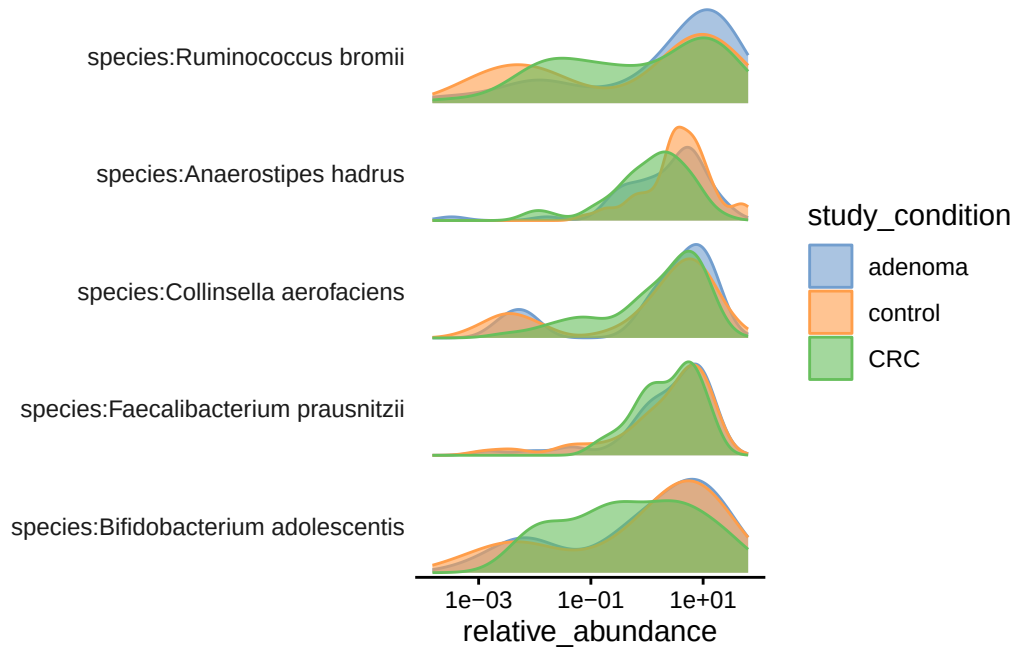
We can also look at the relative abundances of specific taxa in the dataset.

```
plotAbundanceDensity(tse,
  layout = "density",
  assay.type = "relative_abundance",
  n = 5, colour_by = "study_condition"
) +
  scale_x_log10()
```

Warning in scale_x_log10(): log-10 transformation introduced infinite values.

Warning: Removed 53 rows containing non-finite outside the scale range (`stat\_density()`).

25. Microbiome analysis



25.7.2. Prevalence

Prevalence refers to the proportion of samples in which a taxon is present. This information can help identify the most common and rare taxa in the dataset and provide insights into the distribution of different taxa across samples.

Let's agglomerate the data at the genus level and then plot the prevalence of the top 20 most prevalent genera in the dataset.

```
tse_genus <- agglomerateByRank(tse, rank = "genus")
```

```
head(getPrevalence(tse_genus,
  detection = 1 / 100,
  sort = TRUE, assay.type = "relative_abundance",
  as.relative = TRUE
))
```

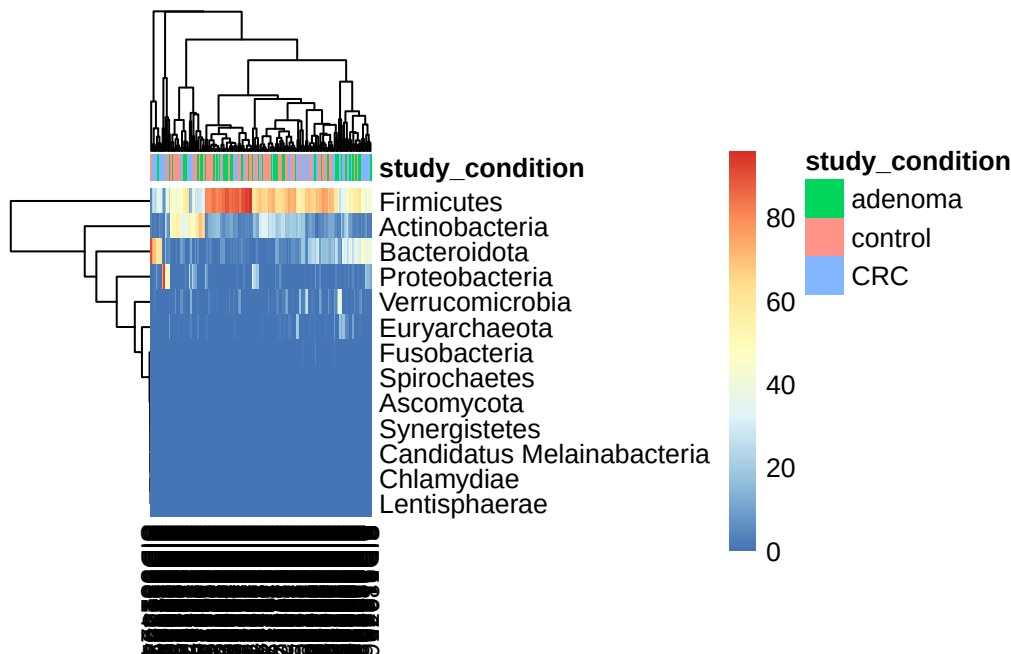
Bifidobacterium	Dorea	Faecalibacterium	Mediterraneibacter
0.7857143	0.7662338	0.7337662	0.7272727
Anaerostipes	Blautia		
0.7272727	0.7077922		

25.8. Heatmaps

Heatmaps are a common way to visualize microbiome data and can provide insights into the relative abundances of different microbial taxa in each sample. To keep things simple, we'll create a heatmap of the relative abundances of the phyla in the dataset.

We first agglomerate the data at the phylum level and then create a heatmap of the relative abundances of the phyla in the dataset.

```
library(pheatmap)
tse_phylum <- agglomerateByRank(tse, rank = "phylum")
pheatmap(
  assay(tse_phylum, "relative_abundance"),
  annotation_col = as.data.frame(colData(tse_phylum)[, "study_condition", drop = FALSE])
)
```



25.9. Further fun

The mia package folks have created a [book on microbiome analysis in R](#) that you can explore for more details on microbiome analysis in R. The book covers a wide range of

25. *Microbiome analysis*

topics related to microbiome analysis, including data loading, preprocessing, visualization, and statistical analysis. It provides detailed explanations and examples of how to work with microbiome data in R and is a valuable resource for anyone interested in learning more about microbiome analysis.

26. Genomic Ranges Introduction

26.1. Introduction

Genomic ranges are fundamental data structures in bioinformatics that represent intervals on chromosomes. They are essential for analyzing ChIP-seq peaks, gene annotations, regulatory elements, and other genomic features. In this tutorial, we'll explore the BED file format and demonstrate practical genomic range operations using R's `rtracklayer` package.

This tutorial will cover:

- Understanding the BED file format
- Loading genomic ranges from BED files
- Basic exploration of genomic ranges
- Accessing and manipulating genomic coordinates

26.2. The dataset

We'll use CTCF ChIP-seq peak data from the ENCODE project. CTCF (CCCTC-binding factor) is a key architectural protein that helps organize chromatin structure. The data is available in BED format, which we will load and analyze. Each peak represents the results of a ChIP-seq experiment, indicating regions where CTCF binds to DNA. The sample was sequenced, aligned, and peaks were called using standard ChIP-seq analysis pipelines. The peaks are stored in a BED file, which we will import into R for analysis.

26.3. The BED File Format

The Browser Extensible Data (BED) format is a standard way to represent genomic intervals. BED files are tab-delimited text files where each line represents a genomic feature.

26.3.1. BED Format Structure

The BED format has several required and optional fields:

Required fields (BED3):

- **chrom:** Chromosome name (e.g., chr1, chr2, chrX)
- **chromStart:** Start position (0-based, inclusive)
- **chromEnd:** End position (0-based, exclusive)

Optional fields:

- **name:** Feature name/identifier
- **score:** Score (0-1000)
- **strand:** Strand (+ or - OR '*')
- **thickStart:** Start of thick drawing
- **thickEnd:** End of thick drawing
- **itemRgb:** RGB color values
- **blockCount:** Number of blocks
- **blockSizes:** Block sizes
- **blockStarts:** Block start positions

26.3.2. Key Concepts

- **0-based coordinate system:** BED uses 0-based coordinates where the first base is position 0
- **Half-open intervals:** chromStart is inclusive, chromEnd is exclusive
- **Width calculation:** Width = chromEnd - chromStart

Example BED entry:

```
chr1    1000    2000    peak1    500    +
```

This represents a feature named “peak1” on chromosome 1, from position 1000 to 1999 (width = 1000 bp).

26.4. Loading Required Libraries

```
# Load required libraries
library(rtracklayer)
library(GenomicRanges)
library(ggplot2)
library(dplyr)

# Set up plotting theme
theme_set(theme_minimal())
```

26.5. Loading CTCF ChIP-seq Data

We'll work with CTCF ChIP-seq peak data from the ENCODE project. CTCF (CCCTC-binding factor) is a key architectural protein that helps organize chromatin structure.

```
# URL for the CTCF ChIP-seq BED file
bed_url <- "https://www.encodeproject.org/files/ENCFF960ZGP/@@download/ENCFF960ZGP.bed.gz"

# Let's first load this file using readr::read_table to check its structure
ctcf_peaks_raw <- readr::read_table(bed_url, col_names=FALSE)
```

```
-- Column specification -----
cols(
  X1 = col_character(),
  X2 = col_double(),
  X3 = col_double(),
  X4 = col_character(),
  X5 = col_double(),
  X6 = col_character(),
  X7 = col_double(),
  X8 = col_double(),
  X9 = col_double(),
  X10 = col_double()
)
```

```
# Display the first few rows of the raw data
head(ctcf_peaks_raw)
```

26. Genomic Ranges Introduction

A tibble: 6 x 10

	X1	X2	X3	X4	X5	X6	X7	X8	X9	X10
	<chr>	<dbl>	<dbl>	<chr>	<dbl>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	chr2	238305323	238305539	.	1000	.	7.70	-1	0.274	108
2	chr20	49982706	49982922	.	541	.	8.72	-1	0.466	108
3	chr15	39921015	39921231	.	672	.	9.08	-1	0.547	108
4	chr8	6708273	6708489	.	560	.	9.86	-1	0.662	108
5	chr9	136645956	136646172	.	584	.	10.2	-1	0.751	108
6	chr7	47669294	47669510	.	614	.	10.5	-1	0.807	108

Bioconductor's `rtracklayer` package provides a convenient way to import BED files directly into R as `GRanges` objects, which are optimized for genomic range operations. We'll use this package to load the CTCF peaks data.

```
# Load the BED file using rtracklayer
ctcf_peaks <- rtracklayer::import.bed_url, format="narrowPeak")
```

Let's take a look at the loaded CTCF peaks data. The `ctcf_peaks` object is a `GRanges` object that contains genomic ranges representing the CTCF ChIP-seq peaks.

```
ctcf_peaks
```

GRanges object with 43865 ranges and 6 metadata columns:

	seqnames	ranges	strand	name	score
	<Rle>	<IRanges>	<Rle>	<character>	<numeric>
[1]	chr2	238305324-238305539	*	<NA>	1000
[2]	chr20	49982707-49982922	*	<NA>	541
[3]	chr15	39921016-39921231	*	<NA>	672
[4]	chr8	6708274-6708489	*	<NA>	560
[5]	chr9	136645957-136646172	*	<NA>	584
...	...	...	...	...	...
[43861]	chr11	66222736-66222972	*	<NA>	1000
[43862]	chr10	75235956-75236225	*	<NA>	1000
[43863]	chr16	57649100-57649347	*	<NA>	1000
[43864]	chr17	37373596-37373838	*	<NA>	1000
[43865]	chr12	53676108-53676355	*	<NA>	1000

	signalValue	pValue	qValue	peak
	<numeric>	<numeric>	<numeric>	<integer>
[1]	7.70385	-1	0.27378	108

26. Genomic Ranges Introduction

```

[2]      8.71626      -1  0.46571      108
[3]      9.07638      -1  0.54697      108
[4]      9.86234      -1  0.66189      108
[5]     10.15488      -1  0.75082      108
...      ...      ...      ...      ...
[43861]    478.640      -1  4.87574      124
[43862]    480.183      -1  4.87574      141
[43863]    491.081      -1  4.87574      116
[43864]    491.991      -1  4.87574      127
[43865]    494.303      -1  4.87574      126
-----

```

```
seqinfo: 26 sequences from an unspecified genome; no seqlengths
```

The `ctcf_peaks` object now contains the genomic ranges of CTCF peaks, including chromosome names, start and end positions, and additional metadata such as peak scores and strand information.

26.6. Understanding GRanges Objects

To get information from a `GRanges` object, we can use various accessor functions. For example, `seqnames()` retrieves the chromosome names, `start()` and `end()` get the start and end positions, and `width()` calculates the width of each peak.

```
# Accessing chromosome names, start, end, and width
length(ctcf_peaks) # Total number of peaks
```

```
[1] 43865
```

```
seqnames(ctcf_peaks) # Chromosome names (or contig names, etc.)
```

```
factor-Rle of length 43865 with 41566 runs
```

```

Lengths:      1      1      1      1      1 ...      1      1      1      1      1
Values : chr2  chr20 chr15 chr8  chr9  ... chr11 chr10 chr16 chr17 chr12
Levels(26): chr2 chr20 chr15 ... chr1_KI270714v1_random chrUn_GL000219v1

```

```
head(start(ctcf_peaks)) # Start positions (0-based)
```


26. Genomic Ranges Introduction

```
[1] 238305324 49982707 39921016 6708274 136645957 47669295
```

```
head(end(ctcf_peaks)) # End positions (1-based)
```

```
[1] 238305539 49982922 39921231 6708489 136646172 47669510
```

```
head(width(ctcf_peaks)) # equivalent to end(ctcf_peaks) - start(ctcf_peaks)
```

```
[1] 216 216 216 216 216 216
```

What is the distribution of peak widths? We can visualize this using a histogram.

```
# Create a histogram of peak widths  
hist(width(ctcf_peaks), breaks=50, main="CTCF Peak Widths", xlab="Width (bp)", col="lightblue")
```

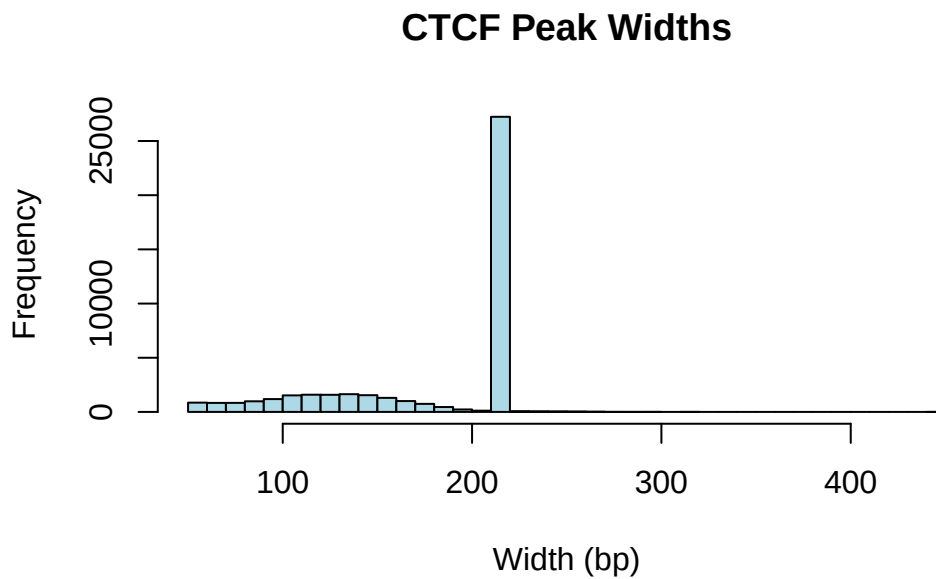


Figure 26.1.: Histogram of CTCF Peak Widths. Why is there a large peak at around 200 bp?

The “metadata” part of a `GRanges` object can be accessed using the `mcols()` function, which returns a data frame-like structure containing additional information about each genomic range.

26. Genomic Ranges Introduction

```
mcols(ctcf_peaks)
```

DataFrame with 43865 rows and 6 columns

	name	score	signalValue	pValue	qValue	peak
	<character>	<numeric>	<numeric>	<numeric>	<numeric>	<integer>
1	NA	1000	7.70385	-1	0.27378	108
2	NA	541	8.71626	-1	0.46571	108
3	NA	672	9.07638	-1	0.54697	108
4	NA	560	9.86234	-1	0.66189	108
5	NA	584	10.15488	-1	0.75082	108
...	...	...	...	...	...	...
43861	NA	1000	478.640	-1	4.87574	124
43862	NA	1000	480.183	-1	4.87574	141
43863	NA	1000	491.081	-1	4.87574	116
43864	NA	1000	491.991	-1	4.87574	127
43865	NA	1000	494.303	-1	4.87574	126

The `mcols()` function returns a data frame-like structure containing additional information about each genomic range, such as peak scores and strand orientation. This metadata can be useful for filtering or annotating peaks.

```
mcols(ctcf_peaks)[1:5, ]
```

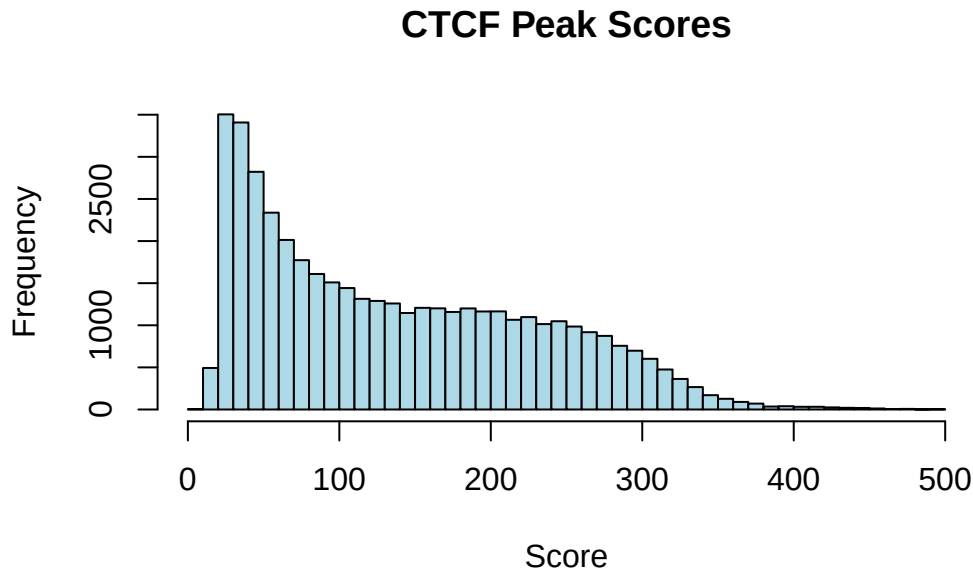
DataFrame with 5 rows and 6 columns

	name	score	signalValue	pValue	qValue	peak
	<character>	<numeric>	<numeric>	<numeric>	<numeric>	<integer>
1	NA	1000	7.70385	-1	0.27378	108
2	NA	541	8.71626	-1	0.46571	108
3	NA	672	9.07638	-1	0.54697	108
4	NA	560	9.86234	-1	0.66189	108
5	NA	584	10.15488	-1	0.75082	108

```
mcols(ctcf_peaks)$signalValue[1:5] # Accessing the score column directly
```

```
[1] 7.70385 8.71626 9.07638 9.86234 10.15488
```

```
hist(mcols(ctcf_peaks)$signalValue, breaks=50, main="CTCF Peak Scores", xlab="Score", col="
```



26.7. Exploring Peak Characteristics

26.7.1. Basic Peak Statistics

How many peaks do we have? What chromosomes are represented? What is the range of peak widths? Let's calculate some basic statistics about the CTCF peaks.

```
# Calculate basic statistics
cat("=== CTCF Peak Statistics ===\n")
```

```
=== CTCF Peak Statistics ===
```

```
cat("Total number of peaks:", length(ctcf_peaks), "\n")
```

```
Total number of peaks: 43865
```

```
cat("Number of chromosomes represented:", length(unique(seqnames(ctcf_peaks))), "\n")
```

26. Genomic Ranges Introduction

Number of chromosomes represented: 26

```
cat("Peak width range:", min(width(ctcf_peaks)), "-", max(width(ctcf_peaks)), "bp\n")
```

Peak width range: 54 - 442 bp

```
cat("Median peak width:", median(width(ctcf_peaks)), "bp\n")
```

Median peak width: 216 bp

```
cat("Mean peak width:", round(mean(width(ctcf_peaks)), 1), "bp\n")
```

Mean peak width: 181.1 bp

```
# Show chromosome names
cat("\nChromosomes present:\n")
```

Chromosomes present:

```
print(sort(unique(as.character(seqnames(ctcf_peaks)))))
```

```
[1] "chr1"                "chr1_KI270714v1_random"
[3] "chr10"               "chr11"
[5] "chr12"               "chr13"
[7] "chr14"               "chr15"
[9] "chr16"               "chr17"
[11] "chr17_GL000205v2_random" "chr18"
[13] "chr19"               "chr2"
[15] "chr20"               "chr21"
[17] "chr22"               "chr3"
[19] "chr4"                "chr5"
[21] "chr6"                "chr7"
[23] "chr8"                "chr9"
[25] "chrUn_GL000219v1"    "chrX"
```

26.7.2. Peaks Per Chromosome

```
# Count peaks per chromosome
peaks_per_chr <- table(seqnames(ctcf_peaks))
peaks_per_chr_df <- as.data.frame(peaks_per_chr)
peaks_per_chr_df
```

	Var1	Freq
1	chr2	3332
2	chr20	1236
3	chr15	1431
4	chr8	1935
5	chr9	1820
6	chr7	2140
7	chr4	2030
8	chr12	2274
9	chr5	2355
10	chr22	873
11	chr1	4207
12	chr17	2010
13	chr3	2808
14	chr11	2417
15	chr19	1662
16	chr6	2504
17	chr13	1030
18	chrX	1460
19	chr10	2037
20	chr16	1497
21	chr14	1461
22	chr21	406
23	chr18	932
24	chr17_GL000205v2_random	4
25	chr1_KI270714v1_random	1
26	chrUn_GL000219v1	3

26.8. Accessing Peak Coordinates

26.8.1. Finding Starts and Ends

```
# Extract start and end coordinates
peak_starts <- start(ctcf_peaks)
peak_ends <- end(ctcf_peaks)
peak_centers <- start(ctcf_peaks) + (width(ctcf_peaks)/2)

head(peak_starts, 10)
```

```
[1] 238305324 49982707 39921016 6708274 136645957 47669295 136784479
[8] 139453373 122421409 102583856
```

```
head(peak_ends, 10)
```

```
[1] 238305539 49982922 39921231 6708489 136646172 47669510 136784694
[8] 139453588 122421624 102584071
```

```
head(peak_centers, 10)
```

```
[1] 238305432 49982815 39921124 6708382 136646065 47669403 136784587
[8] 139453481 122421517 102583964
```

26.9. Manipulating Peak Ranges

26.9.1. Shifting Peaks

Shifting peaks is useful for various analyses, such as creating flanking regions or adjusting peak positions.

```
# Shift peaks by different amounts
peaks_shifted_100bp <- shift(ctcf_peaks, 100) # Shift right by 100bp
peaks_shifted_neg50bp <- shift(ctcf_peaks, -50) # Shift left by 50bp

cat("=== Peak Shifting Examples ===\n")
```

26. Genomic Ranges Introduction

=== Peak Shifting Examples ===

```
cat("Original peak 1:", as.character(ctcf_peaks[1]), "\n")
```

Original peak 1: chr2:238305324-238305539

```
cat("Shifted +100bp:", as.character(peaks_shifted_100bp[1]), "\n")
```

Shifted +100bp: chr2:238305424-238305639

```
cat("Shifted -50bp:", as.character(peaks_shifted_neg50bp[1]), "\n")
```

Shifted -50bp: chr2:238305274-238305489

```
# Demonstrate that width is preserved during shifting
cat("\nWidths after shifting (should be unchanged):\n")
```

Widths after shifting (should be unchanged):

```
cat("Original width:", width(ctcf_peaks[1]), "\n")
```

Original width: 216

```
cat("Shifted +100bp width:", width(peaks_shifted_100bp[1]), "\n")
```

Shifted +100bp width: 216

```
cat("Shifted -50bp width:", width(peaks_shifted_neg50bp[1]), "\n")
```

Shifted -50bp width: 216

26.9.2. Setting Peak Widths

Resizing peaks is common when standardizing peak sizes or creating fixed-width windows around peak centers.

26. Genomic Ranges Introduction

```
# Resize peaks to fixed width (200bp) centered on original peak center
peaks_200bp <- resize(ctcf_peaks, width = 200, fix = "center")

# Resize peaks to 500bp, keeping the start position fixed
peaks_500bp_start <- resize(ctcf_peaks, width = 500, fix = "start")

# Resize peaks to 300bp, keeping the end position fixed
peaks_300bp_end <- resize(ctcf_peaks, width = 300, fix = "end")

cat("=== Peak Resizing Examples ===\n")
```

=== Peak Resizing Examples ===

```
cat("Original peak 1:", as.character(ctcf_peaks[1]), "\n")
```

Original peak 1: chr2:238305324-238305539

```
cat("Resized to 200bp (center):", as.character(peaks_200bp[1]), "\n")
```

Resized to 200bp (center): chr2:238305332-238305531

```
cat("Resized to 500bp (start fixed):", as.character(peaks_500bp_start[1]), "\n")
```

Resized to 500bp (start fixed): chr2:238305324-238305823

```
cat("Resized to 300bp (end fixed):", as.character(peaks_300bp_end[1]), "\n")
```

Resized to 300bp (end fixed): chr2:238305240-238305539

```
# Verify that all peaks now have the specified width
cat("\nWidth verification:\n")
```

Width verification:

26. Genomic Ranges Introduction

```
cat("200bp resize - all widths 200?", all(width(peaks_200bp) == 200), "\n")
```

200bp resize - all widths 200? TRUE

```
cat("500bp resize - all widths 500?", all(width(peaks_500bp_start) == 500), "\n")
```

500bp resize - all widths 500? TRUE

```
cat("300bp resize - all widths 300?", all(width(peaks_300bp_end) == 300), "\n")
```

300bp resize - all widths 300? TRUE

26.9.3. Creating Flanking Regions

```
# Create flanking regions around peaks
upstream_1kb <- flank(ctcf_peaks, width = 1000, start = TRUE)
downstream_1kb <- flank(ctcf_peaks, width = 1000, start = FALSE)

# Create regions extending in both directions
extended_peaks <- resize(ctcf_peaks, width = width(ctcf_peaks) + 2000, fix = "center")

cat("=== Flanking Region Examples ===\n")
```

=== Flanking Region Examples ===

```
cat("Original peak 1:", as.character(ctcf_peaks[1]), "\n")
```

Original peak 1: chr2:238305324-238305539

```
cat("1kb upstream:", as.character(upstream_1kb[1]), "\n")
```

1kb upstream: chr2:238304324-238305323

```
cat("1kb downstream:", as.character(downstream_1kb[1]), "\n")
```

```
1kb downstream: chr2:238305540-238306539
```

```
cat("Extended  $\pm$ 1kb:", as.character(extended_peaks[1]), "\n")
```

```
Extended  $\pm$ 1kb: chr2:238304324-238306539
```

26.10. Coverage

Coverage refers to the number of times a genomic region is covered by ranges on a chromosome. A common use case is calculating coverage from ChIP-seq data, where we want to know how many reads overlap with each peak or for doing “peak calling” analysis.

We can make a toy example by simulating random reads of length 50 bp across a chromosome and then calculating coverage.

```
# Simulate random reads on chromosome 1
set.seed(42) # For reproducibility
chrom_length <- 1000000 # Length of chromosome 1
num_reads <- 100000
read_length <- 50 # Length of each read
random_starts <- sample(seq_len(chrom_length - read_length + 1), num_reads, replace = TRUE)
random_reads <- GRanges(seqnames = "chr1",
                        ranges = IRanges(start = random_starts, end = random_starts + read_length),
                        strand = "*")
random_reads
```

GRanges object with 100000 ranges and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
[1]	chr1	61413-61462	*
[2]	chr1	54425-54474	*
[3]	chr1	623844-623893	*
[4]	chr1	74362-74411	*
[5]	chr1	46208-46257	*
...	...	...	...
[99996]	chr1	663489-663538	*

26. Genomic Ranges Introduction

```
[99997]      chr1 886082-886131      *
[99998]      chr1 933222-933271      *
[99999]      chr1 486340-486389      *
[100000]     chr1 177556-177605      *
-----
```

```
seqinfo: 1 sequence from an unspecified genome; no seqlengths
```

We are now ready to calculate the coverage of these random reads across the chromosome. The `coverage()` function from the `GenomicRanges` package computes the coverage of ranges in a `GRanges` object.

```
random_coverage <- coverage(random_reads, width = chrom_length)
head(random_coverage)
```

```
RleList of length 1
```

```
$chr1
```

```
integer-Rle of length 1000000 with 173047 runs
```

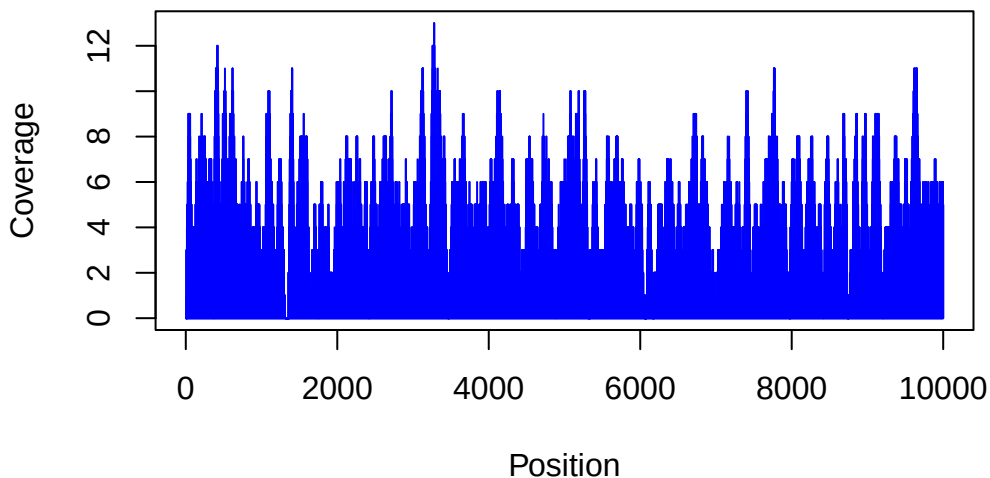
```
Lengths:  7  1  6  3  8  8  1  1 22  1  4 ...  3  1  2  8 12 23  5  2  8  6
```

```
Values  :  0  1  3  4  5  6  7  8  9  8  6 ...  3  2  3  4  5  4  3  2  1  0
```

We can visualize the coverage of the random reads on chromosome 1. The `coverage()` function returns a `Rle` object, which is a run-length encoding of the coverage values.

```
plot(random_coverage[[1]][1:10000], main="Coverage of Random Reads on Chromosome 1", xlab="Position")
```

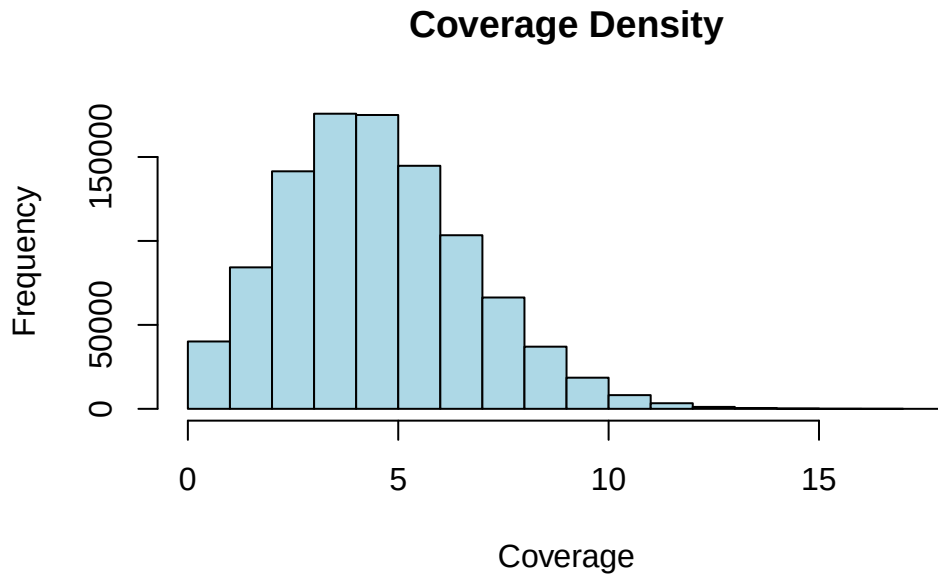
Coverage of Random Reads on Chromosome 1



26. Genomic Ranges Introduction

This plot shows the coverage of random reads across the first 10,000 bases of chromosome 1. The y-axis represents the number of reads covering each position, while the x-axis represents the genomic positions.

```
# Calculate coverage density  
hist(as.numeric(random_coverage[[1]]), main="Coverage Density", xlab="Coverage", ylab="Frequency")
```



26.11. Key Takeaways

This tutorial demonstrated several important concepts:

1. **BED file format:** Understanding the structure and coordinate system of BED files
2. **Loading genomic data:** Using `rtracklayer` to import BED files into R
3. **Basic exploration:** Counting features, examining distributions, and summarizing data
4. **Coordinate manipulation:** Accessing starts, ends, and performing coordinate arithmetic
5. **Range operations:** Shifting, resizing, and creating flanking regions
6. **Advanced analysis:** Finding overlaps and performing grouped operations

26.11.1. Common Use Cases

- **Peak calling analysis:** Examining ChIP-seq peaks, ATAC-seq peaks, etc.
- **Annotation overlap:** Finding genes or regulatory elements near peaks
- **Comparative analysis:** Comparing peak sets between conditions or samples
- **Motif analysis:** Creating sequences around peak centers for motif discovery
- **Visualization:** Preparing data for genome browser tracks or custom plots

26.11.2. Best Practices

1. Always check coordinate systems (0-based vs 1-based)
2. Verify chromosome naming conventions match your reference genome
3. Consider peak width distributions when setting analysis parameters
4. Use appropriate genome builds for all analyses
5. Document coordinate transformations and filtering steps

This foundation in genomic ranges and BED file manipulation will serve as a basis for more advanced genomic analyses in R.

27. Genomic ranges and features

Almost everything you study in genomics lives at a *place* on the genome. A gene sits between two coordinates on a chromosome. An exon is a stretch within that gene. A ChIP-seq peak marks where a protein bound. A SNP is a single position. As soon as you have two such sets of places, the most interesting biological questions become questions about *location*: Which peaks fall inside a promoter? What is the nearest gene to this variant? Where do these two experiments agree?

In this chapter you'll learn to represent those places as **genomic ranges** and to ask how they relate to one another. The workhorse is Bioconductor's [GenomicRanges](#) package (Lawrence, Huber, Pagès, Aboyoun, Carlson, Gentleman, Martin T. Morgan, et al. 2013a), which gives you a compact object to hold ranges plus any data attached to them, and a vocabulary of fast operations — overlap, nearest-neighbor, flank, reduce — that scale to millions of regions. We'll build small example objects by hand so you can see exactly what each operation does, then finish with real gene models pulled from a genome annotation.

i A note on coordinate systems

Different tools count positions differently. The UCSC Genome Browser uses 0-based, half-open coordinates internally, while Ensembl and Bioconductor use 1-based, inclusive coordinates (position 1 is the first base, and both endpoints are included). **GRanges** objects are always 1-based and inclusive, which is one fewer thing to track once you stay inside Bioconductor.

27.1. What you'll learn

- Build a **GRanges** object and read its two-part display (coordinates on the left, meta-data on the right).
- Subset and extract parts of a **GRanges** object with accessors like `seqnames()`, `ranges()`, `strand()`, and `mcols()`.
- Apply intra-range operations (`flank()`, `shift()`, `resize()`) and inter-range operations (`reduce()`, `disjoin()`, `gaps()`, `coverage()`).

- Find relationships between two sets of ranges with `findOverlaps()`, `subsetByOverlaps()`, `nearest()`, and `distanceToNearest()`.
- Group ranges into a `GRangesList` (for example, the exons of a transcript).
- Pull gene, transcript, and exon models out of a `TxDb` annotation package.

27.2. Bioconductor and GenomicRanges

```
library(GenomicRanges)
```

The Bioconductor `GenomicRanges` package is a comprehensive toolkit designed to handle and manipulate genomic intervals and variables systematized on these intervals (Lawrence, Huber, Pagès, Aboyoun, Carlson, Gentleman, Martin T. Morgan, et al. 2013a). Developed by Bioconductor, this package simplifies the complexity of managing genomic data, facilitating the efficient exploration, manipulation, and visualization of such data. `GenomicRanges` aids in dealing with the challenges of genomic data, including its massive size, intricate relationships, and high dimensionality.

The `GenomicRanges` package in Bioconductor covers a wide range of use cases related to the management and analysis of genomic data. Here are some key examples:

- **Genomic Feature Manipulation** The `GenomicRanges` and `GRanges` classes can be used to represent and manipulate various genomic features such as genes, transcripts, exons, or single-nucleotide polymorphisms (SNPs). Users can query, subset, resize, shift, or sort these features based on their genomic coordinates.
- **Genomic Interval Operations** The `GenomicRanges` package provides functions for performing operations on genomic intervals, such as finding overlaps, nearest neighbors, or disjoint intervals. These operations are fundamental to many types of genomic data analyses, such as identifying genes that overlap with ChIP-seq peaks, or finding variants that are in close proximity to each other.
- **Alignments and Coverage** The `GAlignments` and `GAlignmentPairs` classes can be used to represent alignments of sequencing reads to a reference genome, such as those produced by a read aligner. Users can then compute coverage of these alignments over genomic ranges of interest, which is a common task in RNA-seq or ChIP-seq analysis.
- **Annotation and Metadata Handling** The metadata column of a `GRanges` object can be used to store various types of annotation data associated with genomic ranges, such as gene names, gene biotypes, or experimental scores. This makes

27. Genomic ranges and features

it easy to perform analyses that integrate genomic coordinates with other types of biological information.

- **Genome Arithmetic** The GenomicRanges package supports a version of “genome arithmetic”, which includes set operations (union, intersection, set difference) as well as other operations (like coverage, complement, or reduction) that are adapted to the specificities of genomic data.
- **Efficient Data Handling** The CompressedGRangesList class provides a space-efficient way to represent a large list of GRanges objects, which is particularly useful when working with large genomic datasets, such as whole-genome sequencing data.

The GenomicRanges package in Bioconductor uses the S4 class system (see Table 27.1), which is a part of the methods package in R. The S4 system is a more rigorous and formal approach to object-oriented programming in R, providing enhanced capabilities for object design and function dispatch.

Table 27.1.: Classes within the GenomicRanges package. Each class has a slightly different use case.

Class Name	Description	Potential Use
GRanges	Represents a collection of genomic ranges and associated variables.	ChIPSeq peaks, CpG islands, etc.
GRangesList	Represents a list of GenomicRanges objects.	transcript models (exons, introns)
RangesList	Represents a list of Ranges objects.	
IRanges	Represents a collection of integer ranges.	used mainly to <i>build</i> GRanges, etc.
GPos	Represents genomic positions.	SNPs or other single-nucleotide locations
GAlignments	Represents alignments against a reference genome.	Sequence read locations from a BAM file
GAlignment-Pairs	Represents pairs of alignments, typically representing a single fragment of DNA.	Paired-end sequence alignments

In the context of the GenomicRanges package, the S4 class system allows for the creation of complex, structured data objects that can effectively encapsulate genomic intervals and

27. Genomic ranges and features

associated data. This system enables the package to handle the complexity and intricacy of genomic data.

For example, the `GenomicRanges` class in the package is an S4 class that combines several basic data types into a composite object. It includes slots for sequence names (`seqnames`), ranges (start and end positions), strand information, and metadata. Each slot in the S4 class corresponds to a specific component of the genomic data, and methods (see Table 27.2 and Table 27.3) can be defined to interact with these slots in a structured and predictable way.

Table 27.2.: Methods for accessing, manipulating single objects

Method	Description
length	Returns the number of ranges in the <code>GRanges</code> object.
seqnames	Retrieves the sequence names of the ranges.
ranges	Retrieves the start and end positions of the ranges.
strand	Retrieves the strand information of the ranges.
elementMetadata	Retrieves the metadata associated with the ranges.
seqlevels	Returns the levels of the factor that the <code>seqnames</code> slot is derived from.
seqinfo	Retrieves the <code>Seqinfo</code> (sequence information) object associated with the <code>GRanges</code> object.
start, end, width	Retrieve or set the start or end positions, or the width of the ranges.
resize	Resizes the ranges.
subset, [, [[, \$	Subset or extract elements from the <code>GRanges</code> object.
sort	Sorts the <code>GRanges</code> object.
shift	Shifts the ranges by a certain number of base pairs.

The S4 class system also supports inheritance, which allows for the creation of specialized subclasses that share certain characteristics with a parent class but have additional features or behaviors.

The S4 system's formalism and rigor make it well-suited to the complexities of bioinformatics and genomic data analysis. It allows for the creation of robust, reliable code that can handle complex data structures and operations, making it a key part of the `GenomicRanges` package and other Bioconductor packages.

27. Genomic ranges and features

Table 27.3.: Methods for comparing and combining multiple GenomicRanges-class objects

Method	Description
findOverlaps	Finds overlaps between different sets of ranges.
countOverlaps	Counts overlaps between different sets of ranges.
subsetByOverlaps	Subsets the ranges based on overlaps.
distanceToNearest	Computes the distance to the nearest range in another set of ranges.

To get going, we can construct a **GRanges** object by hand as an example.

The **GRanges** class represents a collection of genomic ranges that each have a single start and end location on the genome. It can be used to store the location of genomic features such as contiguous binding sites, transcripts, and exons. These objects can be created by using the **GRanges** constructor function. The following code just creates a **GRanges** object from scratch.

```
gr <- GRanges(
  seqnames = Rle(c("chr1", "chr2", "chr1", "chr3"), c(1, 3, 2, 4)),
  ranges = IRanges(101:110, end = 111:120, names = head(letters, 10)),
  strand = Rle(strand(c("-", "+", "*", "+", "-")), c(1, 2, 2, 3, 2)),
  score = 1:10,
  GC = seq(1, 0, length=10))
gr
```

GRanges object with 10 ranges and 2 metadata columns:

	seqnames	ranges	strand	score	GC
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>
a	chr1	101-111	-	1	1.000000
b	chr2	102-112	+	2	0.888889
c	chr2	103-113	+	3	0.777778
d	chr2	104-114	*	4	0.666667
e	chr1	105-115	*	5	0.555556
f	chr1	106-116	+	6	0.444444
g	chr3	107-117	+	7	0.333333
h	chr3	108-118	+	8	0.222222
i	chr3	109-119	-	9	0.111111
j	chr3	110-120	-	10	0.000000

27. Genomic ranges and features

```
seqinfo: 3 sequences from an unspecified genome; no seqlengths
```

This creates a **GRanges** object with 10 genomic ranges. The output of the **GRanges** `show()` method separates the information into a left and right hand region that are separated by `|` symbols (see Figure 27.1). The genomic coordinates (seqnames, ranges, and strand) are located on the left-hand side and the metadata columns are located on the right. For this example, the metadata is comprised of score and GC information, but almost anything can be stored in the metadata portion of a **GRanges** object.

i Reading a **GRanges**: the `|` is the dividing line

Think of a **GRanges** as a spreadsheet with a heavy vertical rule drawn down the middle. **Everything to the left of the `|` answers “where?”** — the chromosome (**seqnames**), the start and end (**ranges**), and the **strand**. These columns are mandatory and have their own special accessors. **Everything to the right of the `|` answers “what do we know about it?”** — the metadata: scores, GC content, gene names, p-values, whatever you attach. These are optional and you reach them all at once with `mcols()`. Keeping that line in mind makes every later operation easier: range operations rearrange the *left* side; your annotations ride along on the *right*.

The components of the genomic coordinates within a **GRanges** object can be extracted using the `seqnames`, `ranges`, and `strand` accessor functions.

```
seqnames(gr)
```

```
factor-Rle of length 10 with 4 runs
  Lengths:    1    3    2    4
  Values  : chr1 chr2 chr1 chr3
Levels(3): chr1 chr2 chr3
```

```
ranges(gr)
```

```
IRanges object with 10 ranges and 0 metadata columns:
      start      end      width
  <integer> <integer> <integer>
a         101       111        11
b         102       112        11
```

27. Genomic ranges and features

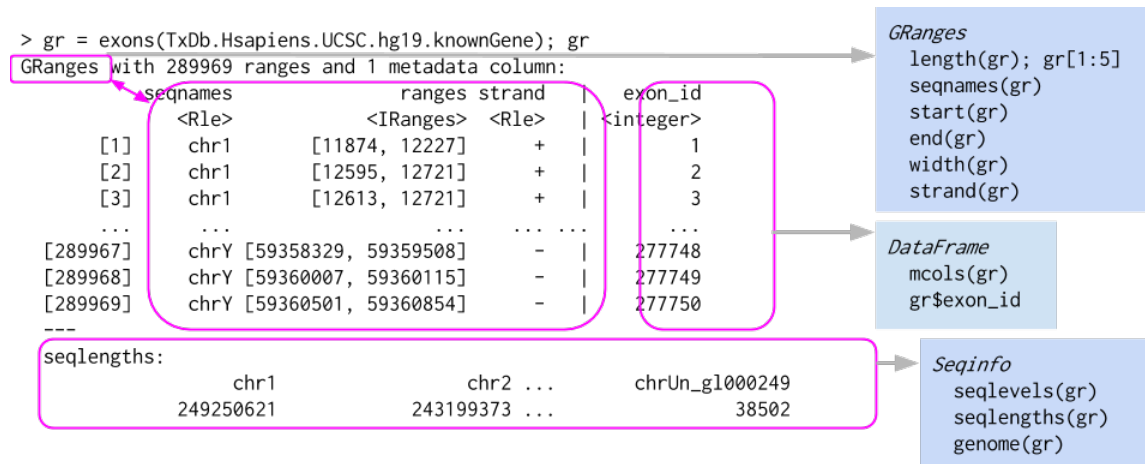


Figure 27.1.: The structure of a **GRanges** object, which behaves a bit like a vector of ranges, although the analogy is not perfect. A **GRanges** object is composed of the “Ranges” part the lefthand box, the “metadata” columns (the righthand box), and a “seqinfo” part that describes the names and lengths of associated sequences. Only the “Ranges” part is required. The figure also shows a few of the “accessors” and approaches to subsetting a **GRanges** object.

```

c      103      113      11
d      104      114      11
e      105      115      11
f      106      116      11
g      107      117      11
h      108      118      11
i      109      119      11
j      110      120      11
  
```

```
strand(gr)
```

```

factor-Rle of length 10 with 5 runs
Lengths: 1 2 2 3 2
Values : - + * + -
Levels(3): + - *
  
```

Note that the **GRanges** object has information to the “left” side of the | that has special accessors. The information to the right side of the |, when it is present, is the metadata and is accessed using `mcols()`, for “metadata columns”.

27. Genomic ranges and features

```
class(mcols(gr))
```

```
[1] "DFrame"  
attr(,"package")  
[1] "S4Vectors"
```

```
mcols(gr)
```

DataFrame with 10 rows and 2 columns

	score	GC
	<integer>	<numeric>
a	1	1.000000
b	2	0.888889
c	3	0.777778
d	4	0.666667
e	5	0.555556
f	6	0.444444
g	7	0.333333
h	8	0.222222
i	9	0.111111
j	10	0.000000

Since the class of `mcols(gr)` is `DFrame`, we can use our `DataFrame` approaches to work with the data.

```
mcols(gr)$score
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

We can even assign a new column. Here we add an `AT` column derived from `GC` (since the fractions sum to 1). Notice that the new column appears on the right of the `|`, alongside `score` and `GC`.

```
mcols(gr)$AT <- 1 - mcols(gr)$GC  
gr
```

27. Genomic ranges and features

GRanges object with 10 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
a	chr1	101-111	-	1	1.000000	0.000000
b	chr2	102-112	+	2	0.888889	0.111111
c	chr2	103-113	+	3	0.777778	0.222222
d	chr2	104-114	*	4	0.666667	0.333333
e	chr1	105-115	*	5	0.555556	0.444444
f	chr1	106-116	+	6	0.444444	0.555556
g	chr3	107-117	+	7	0.333333	0.666667
h	chr3	108-118	+	8	0.222222	0.777778
i	chr3	109-119	-	9	0.111111	0.888889
j	chr3	110-120	-	10	0.000000	1.000000

seqinfo: 3 sequences from an unspecified genome; no seqlengths

Another common way to create a `GRanges` object is to start with a `data.frame`, perhaps created by hand like below or read in using `read.csv` or `read.table`. We can convert from a `data.frame`, when columns are named appropriately, to a `GRanges` object.

```
df_regions <- data.frame(chromosome = rep("chr1", 10),
                        start = seq(1000, 10000, 1000),
                        end = seq(1100, 10100, 1000))
as(df_regions, 'GRanges') # note that names have to match with GRanges slots
```

GRanges object with 10 ranges and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
[1]	chr1	1000-1100	*
[2]	chr1	2000-2100	*
[3]	chr1	3000-3100	*
[4]	chr1	4000-4100	*
[5]	chr1	5000-5100	*
[6]	chr1	6000-6100	*
[7]	chr1	7000-7100	*
[8]	chr1	8000-8100	*
[9]	chr1	9000-9100	*
[10]	chr1	10000-10100	*

seqinfo: 1 sequence from an unspecified genome; no seqlengths

27. Genomic ranges and features

```
## fix column name
colnames(df_regions)[1] <- 'seqnames'
gr2 <- as(df_regions, 'GRanges')
gr2
```

GRanges object with 10 ranges and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
[1]	chr1	1000-1100	*
[2]	chr1	2000-2100	*
[3]	chr1	3000-3100	*
[4]	chr1	4000-4100	*
[5]	chr1	5000-5100	*
[6]	chr1	6000-6100	*
[7]	chr1	7000-7100	*
[8]	chr1	8000-8100	*
[9]	chr1	9000-9100	*
[10]	chr1	10000-10100	*

seqinfo: 1 sequence from an unspecified genome; no seqlengths

The first `as()` call worked even though the chromosome column was named `chromosome` — `as()` recognized `start` and `end` and treated the rest as metadata. After we rename that column to `seqnames`, the conversion places it correctly on the *left* of the `|` as the sequence name.

`GRanges` have one-dimensional-like behavior. For instance, we can check the `length` and even give `GRanges` names.

```
names(gr)
```

```
[1] "a" "b" "c" "d" "e" "f" "g" "h" "i" "j"
```

```
length(gr)
```

```
[1] 10
```

27.2.1. Subsetting GRanges objects

While `GRanges` objects look a bit like a `data.frame`, they can be thought of as vectors with associated ranges. Subsetting, then, works very similarly to vectors. To subset a `GRanges` object to include only second and third regions:

```
gr[2:3]
```

GRanges object with 2 ranges and 3 metadata columns:

	seqnames	ranges	strand		score	GC	AT
	<Rle>	<IRanges>	<Rle>		<integer>	<numeric>	<numeric>
b	chr2	102-112	+		2	0.888889	0.111111
c	chr2	103-113	+		3	0.777778	0.222222

seqinfo: 3 sequences from an unspecified genome; no seqlengths

That said, if the `GRanges` object has metadata columns, we can also treat it like a two-dimensional object kind of like a `data.frame`. Note that the information to the left of the `|` is not like a `data.frame`, so we *cannot* do something like `gr$seqnames`. Here is an example of subsetting with the subset of one metadata column.

```
gr[2:3, "GC"]
```

GRanges object with 2 ranges and 1 metadata column:

	seqnames	ranges	strand		GC
	<Rle>	<IRanges>	<Rle>		<numeric>
b	chr2	102-112	+		0.888889
c	chr2	103-113	+		0.777778

seqinfo: 3 sequences from an unspecified genome; no seqlengths

The usual `head()` and `tail()` also work just fine.

```
head(gr, n=2)
```

GRanges object with 2 ranges and 3 metadata columns:

	seqnames	ranges	strand		score	GC	AT
	<Rle>	<IRanges>	<Rle>		<integer>	<numeric>	<numeric>

27. Genomic ranges and features

```
a      chr1  101-111    - |          1  1.000000  0.000000
b      chr2  102-112    + |          2  0.888889  0.111111
-----
seqinfo: 3 sequences from an unspecified genome; no seqlengths
```

```
tail(gr,n=2)
```

GRanges object with 2 ranges and 3 metadata columns:

```
      seqnames      ranges strand |      score      GC      AT
      <Rle> <IRanges>  <Rle> | <integer> <numeric> <numeric>
i      chr3    109-119      - |         9  0.111111  0.888889
j      chr3    110-120      - |        10  0.000000  1.000000
-----
seqinfo: 3 sequences from an unspecified genome; no seqlengths
```

27.2.2. Interval operations on one GRanges object

27.2.2.1. Intra-range methods

The methods described in this section work *one-region-at-a-time* and are, therefore, called “intra-region” methods. Methods that work across all regions are described below in the [Inter-range methods](#) section.

The `GRanges` class has accessors for the “ranges” part of the data. For example:

```
## Make a smaller GRanges subset
g <- gr[1:3]
start(g) # to get start locations
```

```
[1] 101 102 103
```

```
end(g) # to get end locations
```

```
[1] 111 112 113
```

```
width(g) # to get the "widths" of each range
```

```
[1] 11 11 11
```

27. Genomic ranges and features

```
range(g) # to get the "range" for each sequence (min(start) through max(end))
```

GRanges object with 2 ranges and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
[1]	chr1	101-111	-
[2]	chr2	102-113	+

seqinfo: 3 sequences from an unspecified genome; no seqlengths

The GRanges class also has many methods for manipulating the ranges. The methods can be classified as intra-range methods, inter-range methods, and between-range methods. Intra-range methods operate on each element of a GRanges object independent of the other ranges in the object. For example, the flank method can be used to recover regions flanking the set of ranges represented by the GRanges object. So to get a GRanges object containing the ranges that include the 10 bases *upstream* of the ranges:

```
flank(g, 10)
```

GRanges object with 3 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
a	chr1	112-121	-	1	1.000000	0.000000
b	chr2	92-101	+	2	0.888889	0.111111
c	chr2	93-102	+	3	0.777778	0.222222

seqinfo: 3 sequences from an unspecified genome; no seqlengths

Note how flank pays attention to “strand”. To get the flanking regions *downstream* of the ranges, we can do:

```
flank(g, 10, start=FALSE)
```

GRanges object with 3 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
a	chr1	91-100	-	1	1.000000	0.000000
b	chr2	113-122	+	2	0.888889	0.111111

27. Genomic ranges and features

```
c      chr2   114-123      + |           3  0.777778  0.222222
-----
seqinfo: 3 sequences from an unspecified genome; no seqlengths
```

⚠ Strand changes what “upstream” means

For `flank()`, `resize()`, `precede()`, and `follow()`, “upstream” and “downstream” are interpreted *relative to the strand*. On the + strand, upstream is toward lower coordinates; on the - strand it is toward higher coordinates. A * (unstranded) range is treated as +. If your ranges have the wrong strand — or you forgot to set it — these operations will silently flank the wrong side. When a result looks backwards, check `strand()` first.

Other examples of intra-range methods include `resize` and `shift`. The `shift` method will move the ranges by a specific number of base pairs, and the `resize` method will extend the ranges by a specified width.

```
shift(g, 5)
```

GRanges object with 3 ranges and 3 metadata columns:

```
      seqnames      ranges strand |      score      GC      AT
      <Rle> <IRanges> <Rle> | <integer> <numeric> <numeric>
a      chr1    106-116      - |          1  1.000000  0.000000
b      chr2    107-117      + |          2  0.888889  0.111111
c      chr2    108-118      + |          3  0.777778  0.222222
-----
seqinfo: 3 sequences from an unspecified genome; no seqlengths
```

```
resize(g, 30)
```

GRanges object with 3 ranges and 3 metadata columns:

```
      seqnames      ranges strand |      score      GC      AT
      <Rle> <IRanges> <Rle> | <integer> <numeric> <numeric>
a      chr1     82-111      - |          1  1.000000  0.000000
b      chr2    102-131      + |          2  0.888889  0.111111
c      chr2    103-132      + |          3  0.777778  0.222222
-----
seqinfo: 3 sequences from an unspecified genome; no seqlengths
```

The [GenomicRanges](#) help page `?intra-range-methods` summarizes these methods.

27.2.2.2. Inter-range methods

Inter-range methods involve comparisons between ranges in a single `GRanges` object. For instance, the `reduce` method will align the ranges and merge overlapping ranges to produce a simplified set.

```
reduce(g)
```

`GRanges` object with 2 ranges and 0 metadata columns:

```

      seqnames      ranges strand
      <Rle> <IRanges> <Rle>
[1]      chr1    101-111      -
[2]      chr2    102-113      +
-----

```

```
seqinfo: 3 sequences from an unspecified genome; no seqlengths
```

The `reduce` method could, for example, be used to collapse individual overlapping coding exons into a single set of coding regions.

Sometimes one is interested in the gaps or the qualities of the gaps between the ranges represented by your `GRanges` object. The `gaps` method provides this information:

```
gaps(g)
```

`GRanges` object with 2 ranges and 0 metadata columns:

```

      seqnames      ranges strand
      <Rle> <IRanges> <Rle>
[1]      chr1      1-100      -
[2]      chr2      1-101      +
-----

```

```
seqinfo: 3 sequences from an unspecified genome; no seqlengths
```

 `gaps()` needs to know how long each chromosome is

We never told this object how long the chromosomes are, so Bioconductor assumes each chromosome ends at the largest coordinate it has seen. That means the final gap (from the last range to the true end of the chromosome) is missing. In real work you set the chromosome lengths with `seqlengths()` so that `gaps()`, `coverage()`, and friends know the full extent of each sequence. We can ignore that detail for these toy

examples.

The `disjoin` method represents a `GRanges` object as a collection of non-overlapping ranges:

```
disjoin(g)
```

`GRanges` object with 4 ranges and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
[1]	chr1	101-111	-
[2]	chr2	102	+
[3]	chr2	103-112	+
[4]	chr2	113	+

seqinfo: 3 sequences from an unspecified genome; no seqlengths

The `coverage` method quantifies the degree of overlap for all the ranges in a `GRanges` object.

```
coverage(g)
```

RleList of length 3

\$chr1

integer-Rle of length 111 with 2 runs

Lengths: 100 11

Values : 0 1

\$chr2

integer-Rle of length 113 with 4 runs

Lengths: 101 1 10 1

Values : 0 1 2 1

\$chr3

integer-Rle of length 0 with 0 runs

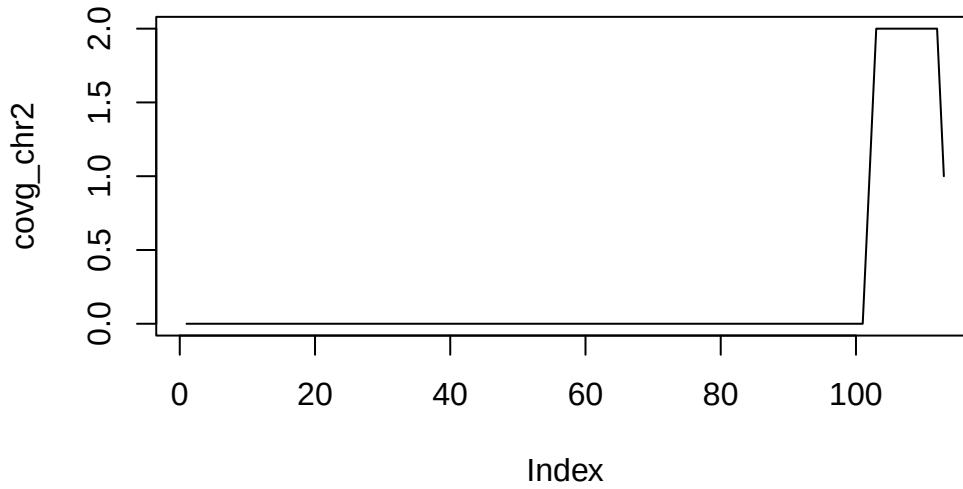
Lengths:

Values :

The coverage is summarized as a `list` of coverages, one for each chromosome. The `Rle` class is used to store the values. Sometimes, one must convert these values to `numeric` using `as.numeric`. In many cases, this will happen automatically, though.

27. Genomic ranges and features

```
covg <- coverage(g)
covg_chr2 <- covg[['chr2']]
plot(covg_chr2, type='l')
```



The plot shows how many ranges cover each base along `chr2`: the line steps up to 2 where two ranges overlap and drops back to 0 in the gaps. This is exactly the kind of per-base signal you compute from aligned sequencing reads in RNA-seq or ChIP-seq.

See the [GenomicRanges](#) help page `? "intra-range-methods"` for more details.

27.2.3. Set operations for GRanges objects

Between-range methods calculate relationships between different `GRanges` objects. Of central importance are `findOverlaps` and related operations; these are discussed below. Additional operations treat `GRanges` as mathematical sets of coordinates; `union(g, g2)` is the union of the coordinates in `g` and `g2`. Here are examples for calculating the union, the intersect and the asymmetric difference (using `setdiff`).

```
g2 <- head(gr, n=2)
GenomicRanges::union(g, g2)
```

27. Genomic ranges and features

GRanges object with 2 ranges and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
[1]	chr1	101-111	-
[2]	chr2	102-113	+

seqinfo: 3 sequences from an unspecified genome; no seqlengths

```
GenomicRanges::intersect(g, g2)
```

GRanges object with 2 ranges and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
[1]	chr1	101-111	-
[2]	chr2	102-112	+

seqinfo: 3 sequences from an unspecified genome; no seqlengths

```
GenomicRanges::setdiff(g, g2)
```

GRanges object with 1 range and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
[1]	chr2	113	+

seqinfo: 3 sequences from an unspecified genome; no seqlengths

There is extensive additional help available or by looking at the vignettes in at the [GenomicRanges](#) pages.

```
?GRanges
```

There are also many possible `methods` that work with `GRanges` objects. To see a complete list (long), try:

```
methods(class="GRanges")
```

27.3. GRangesList

Some important genomic features, such as spliced transcripts that are comprised of exons, are inherently compound structures. Such a feature makes much more sense when expressed as a compound object such as a **GRangesList**. If we think of each transcript as a set of exons, each transcript would be summarized as a **GRanges** object. However, if we have multiple transcripts, we want to keep them separate, with each transcript holding its own exons. A **GRangesList** is a list of **GRanges** objects. Continuing with the transcripts idea, a **GRangesList** can contain all the transcripts and their exons; each transcript is one element in the list.

```
> grl = exonsBy(TxDb.Hsapiens.UCSC.hg19.knownGene, "tx", use.names=TRUE); grl
GRangesList of length 82960:
$uc001aaa.3
GRanges with 3 ranges and 3 metadata columns:
      seqnames      ranges strand | exon_id exon_name exon_rank
      <Rle>        <IRanges> <Rle> | <integer> <character> <integer>
[1]   chr1 [11874, 12227]     + |         1      <NA>         1
[2]   chr1 [12613, 12721]     + |         3      <NA>         2
[3]   chr1 [13221, 14409]     + |         5      <NA>         3

$uc010nxq.1
GRanges with 3 ranges and 3 metadata columns:
      seqnames      ranges strand | exon_id exon_name exon_rank
      <Rle>        <IRanges> <Rle> | <integer> <character> <integer>
[1]   chr1 [11874, 12227]     + |         1      <NA>         1
[2]   chr1 [12595, 12721]     + |         2      <NA>         2
[3]   chr1 [13403, 14409]     + |         6      <NA>         3

$uc010nxr.1
GRanges with 3 ranges and 3 metadata columns:
      seqnames      ranges strand | exon_id exon_name exon_rank
      <Rle>        <IRanges> <Rle> | <integer> <character> <integer>
[1]   chr1 [11874, 12227]     + |         1      <NA>         1
[2]   chr1 [12646, 12697]     + |         4      <NA>         2
[3]   chr1 [13221, 14409]     + |         5      <NA>         3

...
<82957 more elements>
---
seqinfo: 93 sequences (1 circular) from hg19 genome
```

GRangesList
(list of GRanges)
length(grl)
grl[1:3]
shift(grl, 1)
range(grl)

GRanges
grl[[2]]
grl[["uc010nxq.1"]]

Two kinds of fun!
introns =
 psetdiff(range(grl), grl)

grr = unlist(grl)
transform grr, then...
grl = relist(grr, grl)

↑ 'flesh'
↑ 'skeleton'

Figure 27.2.: The structure of a **GRangesList**, which is a **list** of **GRanges** objects. While the analogy is not perfect, a **GRangesList** behaves a bit like a list. Each element in the **GRangesList** is a **GRanges** object. A common use case for a **GRangesList** is to store a list of transcripts, each of which have exons as the regions in the **GRanges**.

Whenever genomic features consist of multiple ranges that are grouped by a parent feature, they can be represented as a **GRangesList** object. Consider the simple example of the two

27. Genomic ranges and features

transcript `GRangesList` below created using the `GRangesList` constructor.

```
gr1 <- GRanges(  
  seqnames = "chr1",  
  ranges = IRanges(103, 106),  
  strand = "+",  
  score = 5L, GC = 0.45)  
gr2 <- GRanges(  
  seqnames = c("chr1", "chr1"),  
  ranges = IRanges(c(107, 113), width = 3),  
  strand = c("+", "-"),  
  score = 3:4, GC = c(0.3, 0.5))
```

The `gr1` and `gr2` are each `GRanges` objects. We can combine them into a “named” `GRangesList` like so:

```
grl <- GRangesList("txA" = gr1, "txB" = gr2)  
grl
```

`GRangesList` object of length 2:

`$txA`

`GRanges` object with 1 range and 2 metadata columns:

	seqnames	ranges	strand	score	GC
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>
[1]	chr1	103-106	+	5	0.45

seqinfo: 1 sequence from an unspecified genome; no seqlengths

`$txB`

`GRanges` object with 2 ranges and 2 metadata columns:

	seqnames	ranges	strand	score	GC
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>
[1]	chr1	107-109	+	3	0.3
[2]	chr1	113-115	-	4	0.5

seqinfo: 1 sequence from an unspecified genome; no seqlengths

The `show` method for a `GRangesList` object displays it as a named list of `GRanges` objects, where the names of this list are considered to be the names of the grouping feature. In the

example above, the groups of individual exon ranges are represented as separate **GRanges** objects which are further organized into a list structure where each element name is a transcript name. Many other combinations of grouped and labeled **GRanges** objects are possible of course, but this example is a common arrangement.

In some cases, **GRangesLists** behave quite similarly to **GRanges** objects.

27.3.1. Basic *GRangesList* accessors

Just as with **GRanges** object, the components of the genomic coordinates within a **GRangesList** object can be extracted using simple accessor methods. Not surprisingly, the **GRangesList** objects have many of the same accessors as **GRanges** objects. The difference is that many of these methods return a list since the input is now essentially a list of **GRanges** objects. Here are a few examples:

```
seqnames(grl)
```

```
RleList of length 2
$txA
factor-Rle of length 1 with 1 run
  Lengths:    1
  Values  : chr1
Levels(1): chr1

$txB
factor-Rle of length 2 with 1 run
  Lengths:    2
  Values  : chr1
Levels(1): chr1
```

```
ranges(grl)
```

```
IRangesList object of length 2:
$txA
IRanges object with 1 range and 0 metadata columns:
      start      end      width
  <integer> <integer> <integer>
[1]      103      106         4
```

27. Genomic ranges and features

```
$txB
IRanges object with 2 ranges and 0 metadata columns:
      start      end      width
  <integer> <integer> <integer>
[1]      107      109         3
[2]      113      115         3
```

```
strand(grl)
```

```
RleList of length 2
$txA
factor-Rle of length 1 with 1 run
  Lengths: 1
  Values  : +
Levels(3): + - *

$txB
factor-Rle of length 2 with 2 runs
  Lengths: 1 1
  Values  : + -
Levels(3): + - *
```

The length and names methods will return the length or names of the list and the seqlengths method will return the set of sequence lengths.

```
length(grl)
```

```
[1] 2
```

```
names(grl)
```

```
[1] "txA" "txB"
```

```
seqlengths(grl)
```

```
chr1
NA
```

27.4. Relationships between region sets

One of the more powerful approaches to genomic data integration is to ask about the relationship between sets of genomic ranges. The key features of this process are to look at overlaps and distances to the nearest feature. These functionalities, combined with the operations like `flank` and `resize`, for instance, allow pretty useful analyses with relatively little code. In general, these operations are *very* fast, even on thousands to millions of regions.

27.4.1. Overlaps

The `findOverlaps` method in the `GenomicRanges` package is a very useful function that allows users to identify overlaps between two sets of genomic ranges.

Here's how it works:

- **Inputs** The function requires two `GRanges` objects, referred to as query and subject.
- **Processing** The function then compares every range in the query object with every range in the subject object, looking for overlaps. An overlap is defined as any instance where the range in the query object intersects with a range in the subject object.
- **Output** The function returns a `Hits` (see `?Hits`) object, which is a compact representation of the matrix of overlaps. Each entry in the `Hits` object corresponds to a pair of overlapping ranges, with the query index and the subject index.

Here is an example of how you might use the `findOverlaps` function:

```
# Create two GRanges objects
gr1 <- gr[1:4]
gr2 <- gr[3:8]
gr1
```

`GRanges` object with 4 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
a	chr1	101-111	-	1	1.000000	0.000000
b	chr2	102-112	+	2	0.888889	0.111111
c	chr2	103-113	+	3	0.777778	0.222222
d	chr2	104-114	*	4	0.666667	0.333333

27. Genomic ranges and features

seqinfo: 3 sequences from an unspecified genome; no seqlengths

```
gr2
```

GRanges object with 6 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
c	chr2	103-113	+	3	0.777778	0.222222
d	chr2	104-114	*	4	0.666667	0.333333
e	chr1	105-115	*	5	0.555556	0.444444
f	chr1	106-116	+	6	0.444444	0.555556
g	chr3	107-117	+	7	0.333333	0.666667
h	chr3	108-118	+	8	0.222222	0.777778

seqinfo: 3 sequences from an unspecified genome; no seqlengths

```
# Find overlaps
overlaps <- findOverlaps(query = gr1, subject = gr2)
overlaps
```

Hits object with 7 hits and 0 metadata columns:

	queryHits	subjectHits
	<integer>	<integer>
[1]	1	3
[2]	2	1
[3]	2	2
[4]	3	1
[5]	3	2
[6]	4	1
[7]	4	2

queryLength: 4 / subjectLength: 6

In the resulting overlaps object, each row corresponds to an overlapping pair of ranges, with the first column giving the index of the range in gr1 and the second column giving the index of the overlapping range in gr2.

27. Genomic ranges and features

If you are interested in only the `queryHits` or the `subjectHits`, there are accessors for those (ie., `queryHits(overlaps)`). To get the actual ranges that overlap, you can use the `subjectHits` or `queryHits` as an index into the original `GRanges` object.

Spend some time looking at these results. Note how the strand comes into play when determining overlaps.

```
gr1[queryHits(overlaps)]
```

GRanges object with 7 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
a	chr1	101-111	-	1	1.000000	0.000000
b	chr2	102-112	+	2	0.888889	0.111111
b	chr2	102-112	+	2	0.888889	0.111111
c	chr2	103-113	+	3	0.777778	0.222222
c	chr2	103-113	+	3	0.777778	0.222222
d	chr2	104-114	*	4	0.666667	0.333333
d	chr2	104-114	*	4	0.666667	0.333333

seqinfo: 3 sequences from an unspecified genome; no seqlengths

```
gr2[subjectHits(overlaps)]
```

GRanges object with 7 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
e	chr1	105-115	*	5	0.555556	0.444444
c	chr2	103-113	+	3	0.777778	0.222222
d	chr2	104-114	*	4	0.666667	0.333333
c	chr2	103-113	+	3	0.777778	0.222222
d	chr2	104-114	*	4	0.666667	0.333333
c	chr2	103-113	+	3	0.777778	0.222222
d	chr2	104-114	*	4	0.666667	0.333333

seqinfo: 3 sequences from an unspecified genome; no seqlengths

As you might expect, the `countOverlaps` method counts the regions in the second `GRanges` that overlap with those that overlap with each element of the first.

27. Genomic ranges and features

```
countOverlaps(gr1, gr2)
```

```
a b c d
1 2 2 2
```

The `subsetByOverlaps` method simply subsets the query `GRanges` object to include *only* those that overlap the subject.

```
subsetByOverlaps(gr1, gr2)
```

GRanges object with 4 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
a	chr1	101-111	-	1	1.000000	0.000000
b	chr2	102-112	+	2	0.888889	0.111111
c	chr2	103-113	+	3	0.777778	0.222222
d	chr2	104-114	*	4	0.666667	0.333333

seqinfo: 3 sequences from an unspecified genome; no seqlengths

In some cases, you may be interested in only one hit per query range. The `select` parameter handles this: with `select="first"`, `findOverlaps()` returns a plain integer vector with one entry per query range (the index of its first overlapping subject, or `NA` if there is none) instead of a full `Hits` object.

```
findOverlaps(gr1, gr2, select="first")
```

```
[1] 3 1 1 1
```

The `%over%` logical operator allows us to do similar things to `findOverlaps` and `subsetByOverlaps`. It returns a `TRUE/FALSE` for each range, which is handy for subsetting.

```
gr2 %over% gr1
```

```
[1] TRUE TRUE TRUE FALSE FALSE FALSE
```

```
gr1[gr1 %over% gr2]
```

GRanges object with 4 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
a	chr1	101-111	-	1	1.000000	0.000000
b	chr2	102-112	+	2	0.888889	0.111111
c	chr2	103-113	+	3	0.777778	0.222222
d	chr2	104-114	*	4	0.666667	0.333333

seqinfo: 3 sequences from an unspecified genome; no seqlengths

27.4.2. Nearest feature

There are a number of useful methods that find the nearest feature (region) in a second set for each element in the first set.

We can review our two GRanges toy objects:

```
g
```

GRanges object with 3 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
a	chr1	101-111	-	1	1.000000	0.000000
b	chr2	102-112	+	2	0.888889	0.111111
c	chr2	103-113	+	3	0.777778	0.222222

seqinfo: 3 sequences from an unspecified genome; no seqlengths

```
gr
```

GRanges object with 10 ranges and 3 metadata columns:

	seqnames	ranges	strand	score	GC	AT
	<Rle>	<IRanges>	<Rle>	<integer>	<numeric>	<numeric>
a	chr1	101-111	-	1	1.000000	0.000000
b	chr2	102-112	+	2	0.888889	0.111111
c	chr2	103-113	+	3	0.777778	0.222222

27. Genomic ranges and features

d	chr2	104-114	*	4	0.666667	0.333333
e	chr1	105-115	*	5	0.555556	0.444444
f	chr1	106-116	+	6	0.444444	0.555556
g	chr3	107-117	+	7	0.333333	0.666667
h	chr3	108-118	+	8	0.222222	0.777778
i	chr3	109-119	-	9	0.111111	0.888889
j	chr3	110-120	-	10	0.000000	1.000000

seqinfo: 3 sequences from an unspecified genome; no seqlengths

- nearest: Performs conventional nearest neighbor finding. Returns an integer vector containing the index of the nearest neighbor range in subject for each range in x. If there is no nearest neighbor NA is returned. For details of the algorithm see the man page in the IRanges package (?nearest).
- precede: For each range in x, precede returns the index of the range in subject that is directly preceded by the range in x. Overlapping ranges are excluded. NA is returned when there are no qualifying ranges in subject.
- follow: The opposite of precede, follow returns the index of the range in subject that is directly followed by the range in x. Overlapping ranges are excluded. NA is returned when there are no qualifying ranges in subject.

Orientation and strand for precede and follow: Orientation is 5' to 3', consistent with the direction of translation. Because positional numbering along a chromosome is from left to right and transcription takes place from 5' to 3', precede and follow can appear to have 'opposite' behavior on the + and - strand. Using positions 5 and 6 as an example, 5 precedes 6 on the + strand but follows 6 on the - strand.

The table below outlines the orientation when ranges on different strands are compared. In general, a feature on * is considered to belong to both strands. The single exception is when both x and subject are * in which case both are treated as +.

	x		subject		orientation
	-----	+	-----	+	-----
a)	+		+		--->
b)	+		-		NA
c)	+		*		--->
d)	-		+		NA
e)	-		-		<---
f)	-		*		<---
g)	*		+		--->

27. Genomic ranges and features

h) * | - | <---
i) * | * | ---> (the only situation where * arbitrarily means +)

```
res <- nearest(g, gr)
res
```

```
[1] 5 4 4
```

Each element of `res` is the index *into* `gr` of the nearest range to the corresponding range in `g`. So `gr[res]` would give you the actual nearest ranges.

While `nearest` and friends give the index of the nearest feature, the distance to the nearest is sometimes also useful to have. The `distanceToNearest` method calculates the nearest feature as well as reporting the distance.

```
res <- distanceToNearest(g, gr)
res
```

Hits object with 3 hits and 1 metadata column:

	queryHits	subjectHits		distance
	<integer>	<integer>		<integer>
[1]	1	5		0
[2]	2	4		0
[3]	3	4		0

queryLength: 3 / subjectLength: 10

This returns a `Hits` object with an extra `distance` column. A distance of 0 means the two ranges overlap or touch; larger values are the gap in base pairs between them.

27.5. Gene models

So far every range has been one we typed by hand. In practice you usually pull ranges from an annotation. A `TxDb` (“transcript database”) package provides a convenient interface to gene models from a variety of sources. The `TxDb.Hsapiens.UCSC.hg38.knownGene` package provides access to the UCSC `knownGene` gene model for the **hg38** build of the human genome. Everything it returns is a `GRanges` (or `GRangesList`), so all the operations from this chapter apply directly.

27. Genomic ranges and features

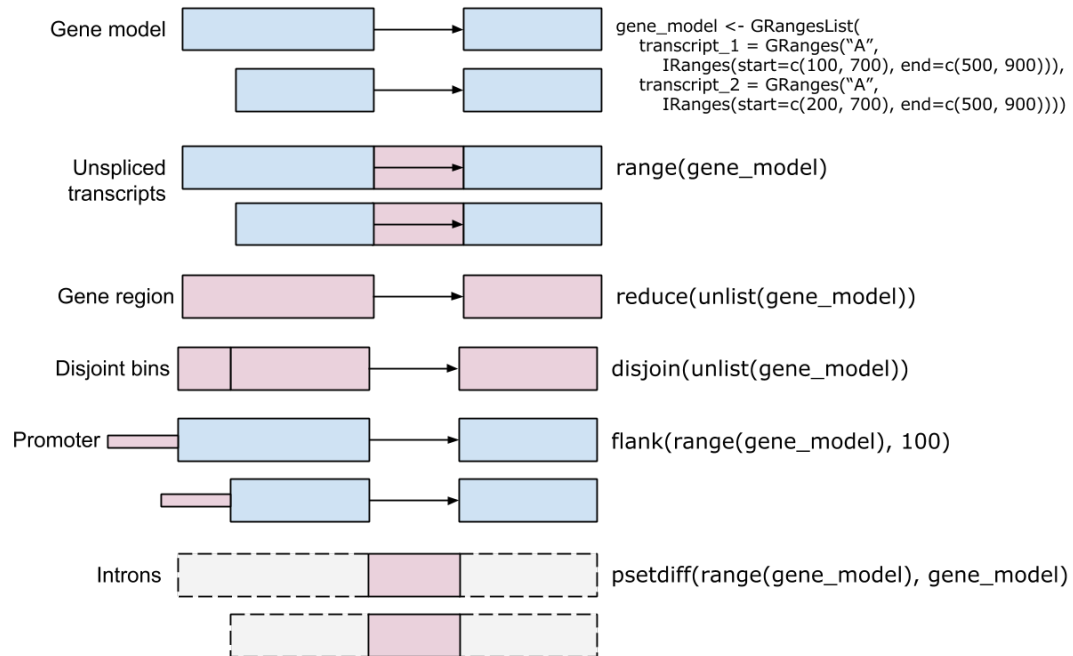


Figure 27.3.: A graphical representation of range operations demonstrated on a gene model.

27. Genomic ranges and features

```
library(TxDb.Hsapiens.UCSC.hg38.knownGene)
txdb <- TxDb.Hsapiens.UCSC.hg38.knownGene
```

The `transcripts` function returns a `GRanges` object with the transcripts for all genes in the database.

```
tx <- transcripts(txdb)
tx
```

`GRanges` object with 412044 ranges and 2 metadata columns:

	seqnames	ranges	strand	tx_id
	<Rle>	<IRanges>	<Rle>	<integer>
[1]	chr1	11121-14413	+	1
[2]	chr1	11125-14405	+	2
[3]	chr1	11410-14413	+	3
[4]	chr1	11411-14413	+	4
[5]	chr1	11426-14409	+	5
...	...	...	...	...
[412040]	chrX_MU273397v1_alt	239036-260095	-	412040
[412041]	chrX_MU273397v1_alt	272358-282686	-	412041
[412042]	chrX_MU273397v1_alt	314193-316302	-	412042
[412043]	chrX_MU273397v1_alt	314813-315236	-	412043
[412044]	chrX_MU273397v1_alt	324527-324923	-	412044

	tx_name
	<character>
[1]	ENST00000832824.1
[2]	ENST00000832825.1
[3]	ENST00000832826.1
[4]	ENST00000832827.1
[5]	ENST00000832828.1
...	...
[412040]	ENST00000710260.1
[412041]	ENST00000710028.1
[412042]	ENST00000710030.1
[412043]	ENST00000710216.1
[412044]	ENST00000710031.1

seqinfo: 711 sequences (1 circular) from hg38 genome

The `exons` function returns a `GRanges` object with the exons.

27. Genomic ranges and features

```
ex <- exons(txdb)
ex
```

GRanges object with 966596 ranges and 1 metadata column:

	seqnames	ranges	strand	exon_id
	<Rle>	<IRanges>	<Rle>	<integer>
[1]	chr1	11121-11211	+	1
[2]	chr1	11125-11211	+	2
[3]	chr1	11410-11671	+	3
[4]	chr1	11411-11671	+	4
[5]	chr1	11426-11671	+	5
...	...	...	...	...
[966592]	chrX_MU273397v1_alt	314193-314248	-	966592
[966593]	chrX_MU273397v1_alt	314813-315236	-	966593
[966594]	chrX_MU273397v1_alt	315258-315407	-	966594
[966595]	chrX_MU273397v1_alt	316254-316302	-	966595
[966596]	chrX_MU273397v1_alt	324527-324923	-	966596

seqinfo: 711 sequences (1 circular) from hg38 genome

The `genes` function returns a GRanges object with one range per gene.

```
gn <- genes(txdb)
gn
```

GRanges object with 35356 ranges and 1 metadata column:

	seqnames	ranges	strand	gene_id
	<Rle>	<IRanges>	<Rle>	<character>
1	chr19	58345178-58362751	-	1
10	chr8	18386273-18401218	+	10
100	chr20	44584896-44652252	-	100
1000	chr18	27932879-28177946	-	1000
100008586	chrX	49551278-49568218	+	100008586
...	...	...	...	...
9990	chr15	34229784-34338060	-	9990
9991	chr9	112215071-112333653	-	9991
9992	chr21	34180729-34371381	+	9992
9993	chr22	19036282-19122454	-	9993
9997	chr22	50523568-50526461	-	9997

```
-----
seqinfo: 711 sequences (1 circular) from hg38 genome
```

Notice the scale: these are real **GRanges** objects with tens of thousands of ranges, yet they print and subset instantly. That is the payoff of the compact representation — the same `findOverlaps()`, `nearest()`, and `subsetByOverlaps()` calls you practiced on the toy **gr** work unchanged here. For example, `subsetByOverlaps(gn, peaks)` would give you the genes hit by a set of ChIP-seq **peaks**, and `nearest(peaks, tx)` would assign each peak to its closest transcript.

💡 Grouping exons by transcript

`exons(txdb)` gives you a flat **GRanges** of every exon. When you instead want the exons *grouped by the transcript they belong to* — a **GRangesList**, one element per transcript — use `exonsBy(txdb, by = "tx")`. That is the natural object for counting reads per transcript and is exactly the **GRangesList** structure we built by hand earlier.

27.6. Summary

You now have a working vocabulary for genomic locations.

- A **GRanges** object holds a set of ranges. Coordinates (`seqnames`, `ranges`, `strand`) sit to the left of the `|`; optional metadata reached with `mcols()` sits to the right. You can build one from scratch or coerce a `data.frame` with `as(df, "GRanges")`.
- **Intra-range** operations (`flank()`, `shift()`, `resize()`) reshape each range on its own and respect strand. **Inter-range** operations (`reduce()`, `disjoin()`, `gaps()`, `coverage()`) summarize a set of ranges together.
- **Between-set** operations answer the integration questions: `findOverlaps()` and `subsetByOverlaps()` for overlap, `nearest()` and `distanceToNearest()` for proximity, plus `union()/intersect()/setdiff()` for set arithmetic.
- A **GRangesList** groups ranges by a parent feature, such as exons within a transcript.
- A **TxDb** package (here [TxDb.Hsapiens.UCSC.hg38.knownGene](#)) hands you real gene, transcript, and exon models as **GRanges**, so every operation above applies to genome-scale annotation with no new syntax.

27.7. Exercises

These reuse the objects built earlier in the chapter: the 10-range `gr`, its 3-range subset `g`, and the `grl` `GRangesList`. Run the chapter's chunks first so those objects exist.

- 1. Read the structure.** Without running any code, which of these belong to the *left* of the `|` in a `GRanges` (the coordinates) and which to the *right* (the metadata): `strand`, `score`, `seqnames`, `GC`, `width`?

i Solution

Left of the `|` (coordinates): `seqnames`, `strand`, and `width` (part of `ranges`). Right of the `|` (metadata): `score` and `GC`. The coordinate columns have dedicated accessors; the metadata columns are reached together with `mcols()`.

- 2. Promoters by hand.** A common task is to grab the region just upstream of a feature's start — a rough promoter. Use `flank()` to get the 50 bases upstream of each range in `g`, then confirm with `width()` that each result is 50 bases wide.

i Solution

```
prom <- flank(g, 50)
prom
```

GRanges object with 3 ranges and 3 metadata columns:

	seqnames	ranges	strand		score	GC	AT
	<Rle>	<IRanges>	<Rle>		<integer>	<numeric>	<numeric>
a	chr1	112-161	-		1	1.000000	0.000000
b	chr2	52-101	+		2	0.888889	0.111111
c	chr2	53-102	+		3	0.777778	0.222222

seqinfo: 3 sequences from an unspecified genome; no seqlengths

```
width(prom)
```

```
[1] 50 50 50
```

`flank()` defaults to the upstream side (`start = TRUE`), measured relative to strand. Every flank is exactly 50 bases wide.

3. Collapse overlaps. Use `reduce()` on `gr` to merge overlapping or adjacent ranges into a minimal set, then report how many ranges remain with `length()`.

i Solution

```
reduced <- reduce(gr)
reduced
```

GRanges object with 7 ranges and 0 metadata columns:

	seqnames	ranges	strand
	<Rle>	<IRanges>	<Rle>
[1]	chr1	106-116	+
[2]	chr1	101-111	-
[3]	chr1	105-115	*
[4]	chr2	102-113	+
[5]	chr2	104-114	*
[6]	chr3	107-118	+
[7]	chr3	109-120	-

seqinfo: 3 sequences from an unspecified genome; no seqlengths

```
length(reduced)
```

```
[1] 7
```

`reduce()` works per chromosome and per strand, merging ranges that overlap or abut into single ranges — the standard way to collapse overlapping exons into non-redundant regions.

4. Overlap counting. Build `q <- gr[1:4]` and `s <- gr[3:8]`, then use `countOverlaps(q, s)` to find how many ranges in `s` each range in `q` overlaps. Which range in `q` has the most overlaps?

i Solution

```
q <- gr[1:4]
s <- gr[3:8]
counts <- countOverlaps(q, s)
counts
```


27. Genomic ranges and features

```
a b c d  
1 2 2 2
```

```
which.max(counts)
```

```
b  
2
```

`countOverlaps()` returns one integer per range in the query. Remember that strand participates in the overlap test, so a + query range will not overlap a - subject range unless you set `ignore.strand = TRUE`.

5. Nearest with distance. Use `distanceToNearest(g, gr)` and pull out the `distance` metadata column with `mcols(...)$distance`. What does a distance of 0 tell you?

i Solution

```
hits <- distanceToNearest(g, gr)  
mcols(hits)$distance
```

```
[1] 0 0 0
```

A distance of 0 means the nearest range overlaps or directly touches the query range — here every range in `g` is also present in `gr`, so each finds itself at distance 0.

28. Ranges Exercises

In the following exercises, we will use the **GenomicRanges** package to explore range operations. We will use the **AnnotationHub** package to load DNase hypersensitivity data from the ENCODE project. In practice, the ENCODE project published datasets like these as bed files. AnnotationHub has packaged these into GRanges objects that we can load and use directly. However, if you have a bed file of your own (peak calls, enhancer regions, etc.), you can load them into GRanges objects using `rtracklayer::import`.

28.1. Exercise 1

In this exercise, we will use DNase hypersensitivity data to practice working with a GRanges object.

- Use the **AnnotationHub** package to find the `goldenpath/hg19/encodeDCC/wgEncodeUwDnase/wgEncodeUwDnaseK562PkRep1.narrowPeak.g` GRanges object. Load that into R as the variable `dnase`.

```
library(AnnotationHub)
ah = AnnotationHub()
query(ah, "goldenpath/hg19/encodeDCC/wgEncodeUwDnase/wgEncodeUwDnaseK562PkRep1.narrowPeak.g")
# the thing above should have only one record, so we can
# just grab it
dnase = query(ah, "goldenpath/hg19/encodeDCC/wgEncodeUwDnase/wgEncodeUwDnaseK562PkRep1.narrowPeak.g")
```

- What type of object is `dnase`?

```
dnase
class(dnase)
```

- What metadata is stored in `dnase`?

```
mcols(dnase)
```

- How many peaks are on each chromosome?

```
library(GenomicFeatures)
table(seqnames(dnase))
```

- What are the mean, min, max, and median widths of the peaks?

```
summary(width(dnase))
```

- What are the sequences that were used in the analysis? Do the names have “chr” or not? Experiment with changing the `seqlevelsStyle` to adjust the sequence names.

```
seqlevels(dnase)
seqlevelsStyle(dnase)
seqlevelsStyle(dnase) = 'ensembl'
seqlevelsStyle(dnase)
seqlevels(dnase)
```

- What is the total amount of “landscape” covered by the peaks? Assume that the peaks do not overlap. What portion of the genome does this represent?

```
sum(width(dnase))
sum(seqlengths(dnase))
sum(width(dnase))/sum(seqlengths(dnase))
```

28.2. Exercise 2

In this exercise, we are going to load the second DNase hypersensitivity replicate to investigate overlaps with the first replicate.

- Use the AnnotationHub to find the second replicate, `goldenpath/hg19/encodeDCC/wgEncodeUwDnase/wgEncodeUwDnaseK562PkRep2.narrowPeak.g`. Load that as `dnase2`.

```
query(ah, "goldenpath/hg19/encodeDCC/wgEncodeUwDnase/wgEncodeUwDnaseK562PkRep2.narrowPeak.g")
# the thing above should have only one record, so we can
# just grab it
dnase2 = query(ah, "goldenpath/hg19/encodeDCC/wgEncodeUwDnase/wgEncodeUwDnaseK562PkRep2.narrowPeak.g")
```

- How many peaks are there in `dnase` and `dnase2`? Are there are similar number?

```
length(dnase)
length(dnase2)
```

- What are the peak sizes for `dnase2`?

```
summary(width(dnase2))
```

- What proportion of the genome does `dnase2` cover?

```
sum(width(dnase))/sum(seqlengths(dnase))
```

- Count the number of peaks from `dnase` that overlap with `dnase2`.

```
sum(dnase %over% dnase2)
```

- Assume that your peak-caller was “too specific” and that you want to expand your peaks by 50 bp on each end (so make them 100 bp larger). Use a combination of `resize` (and pay attention to the `fix` argument) and `width` to do this expansion to `dnase` and call the new `GRanges` object “`dnase_wide`”.

```
w = width(dnase)
dnase_wide = resize(dnase, width=w+100, fix='center') #make a copy
width(dnase_wide)
```

28.3. Exercise 3

In this exercise, we are going to look at the overlap of DNase sites relative to genes. To get started, install and load the `TxDb.Hsapiens.UCSC.hg19.knownGene` txdb object.

```
BiocManager::install("TxDb.Hsapiens.UCSC.hg19.knownGene")
library("TxDb.Hsapiens.UCSC.hg19.knownGene")
kg = TxDb.Hsapiens.UCSC.hg19.knownGene
```

- Load the transcripts from the `knownGene` txdb into a variable. What is the class of this object?

28. Ranges Exercises

```
library("TxDb.Hsapiens.UCSC.hg19.knownGene")
kg = TxDb.Hsapiens.UCSC.hg19.knownGene
gx = genes(kg)
class(gx)
length(gx)
```

- Read about the `flank` method for GRanges objects. How could you use that to get the “promoter” regions of the transcripts? Let’s assume that the promoter region is 2kb upstream of the gene.

```
flank(gx,2000)
```

- Instead of using `flank`, could you do the same thing with the TxDb object? (See `?promoters`).

```
proms = promoters(kg)
```

- Do any of the regions in the promoters overlap with each other?

```
summary(countOverlaps(proms))
```

- To find overlap of our DNase sites with promoters, let’s collapse overlapping “promoters” to just keep the contiguous regions by using `reduce`.

```
# reduce takes all overlapping regions and collapses them
# into a single region that spans all of the overlapping regions
prom_regions = reduce(proms)

# now we can check for overlaps
summary(countOverlaps(prom_regions))
```

- Count the number of DNase sites that overlap with our promoter regions.

```
sum(dnase %over% prom_regions)
# if you notice no overlap, check the seqlevels
# and seqlevelsStyle
seqlevelsStyle(dnase) = "UCSC"
sum(dnase %over% prom_regions)
sum(dnase2 %over% prom_regions)
```

- Is this surprising? If we were to assume that the promoter and dnase regions are “independent” of each other, what number of overlaps would we expect?

```
prop_proms = sum(width(prom_regions))/sum(seqlengths(prom_regions))
prop_dnase = sum(width(dnase))/sum(seqlengths(prom_regions))
# Iff the dnase and promoter regions are
# not related, then we would expect this number
# of DNase overlaps with promoters.
prop_proms * prop_dnase * length(dnase)
```

28.4. Exercise 4

We’ll be using data from histone modification ChIP-seq experiments in human cells to illustrate the concepts of genomic ranges and features. The data consists of genomic intervals representing regions of the genome where specific histone modifications are enriched. These intervals are typically identified using ChIP-seq, a technique that maps protein-DNA interactions across the genome.

The ChIP-seq data is stored in a BED file format, which is a tab-delimited text file format commonly used to represent genomic intervals. Each line in the BED file corresponds to a genomic interval and contains information about the chromosome, start and end positions, and strand orientation of the interval. Additional columns may include metadata such as the signal strength or significance of the interval.

The AnnotationHub package in Bioconductor provides access to a wide range of genomic datasets, including ChIP-seq data. We can use this package to retrieve the ChIP-seq data for histone modifications in human cells and convert it into a GenomicRanges object for further analysis.

<https://www.encodeproject.org/chip-seq/histone/>

Let’s start by loading the AnnotationHub package and retrieving the ChIP-seq data for histone modifications in human cells. You can read more about the AnnotationHub package and how to use it in the Bioconductor documentation.

```
library(AnnotationHub)
ah <- AnnotationHub()
```

There are multiple ways to search the AnnotationHub database. We’ve done that for you and here are the `GRanges` objects for each of four histone marks, and one histone mark replicate.

28. Ranges Exercises

```
h3k4me1 <- ah[['AH25832']]
h3k4me3 <- ah[['AH25833']]
h3k9ac <- ah[['AH25834']]
h3k27me3 <- ah[['AH25835']]
h3k4me3_2 <- ah[['AH27284']]
```

Each of these variables now represents the peak calls after a chip-seq experiment pulling down the histone mark of interest. In the encode project these records were `bed` files. The `bed` files have been converted to `GRanges` objects to allow computation within R.

```
# Grab cpg islands as well
cpg = query(ah, c('cpg', 'UCSC', 'hg19'))[[1]]
```

Let's say that we don't know the behavior of the histone methylation marks with respect to CpG islands. We could ask the question, "What is the overlap of the histone peaks with CpG islands?"

```
sum(h3k4me1 %over% cpg)
```

We might want to actually count the number of bases of overlap between the methyl mark and CpG islands.

```
# The intersection of two peak sets results in the
# overlapping regions as a new set of regions
# The width of each peak is the number of overlapping bases
# And the sum of the widths is the total bases overlapping
sum(width(intersect(h3k4me1, cpg)))
```

But some methyl marks are known to have very broad signals, meaning that there is a higher chance of overlapping CpG islands just because there are more methylated bases. We can adjust for this by "normalizing" for all possible bases covered by either set of peaks, using `union`. We might think of this as a sort of "enrichment score" of one set in another set.

```
sum(width(union(h3k4me1, cpg)))
# and now "normalize"
sum(width(intersect(h3k4me1, cpg)))/sum(width(union(h3k4me1, cpg)))
```

Let's write a small function to calculate our little enrichment score.

28. Ranges Exercises

```
range_enrichment_score <- function(r1, r2) {  
  i = sum(width(intersect(r1, r2)))  
  u = sum(width(union(r1,r2)))  
  return(i/u)  
}
```

And give it a try:

```
range_enrichment_score(h3k4me1, cpg)
```


29. Genomic ranges and ATAC-Seq

R / *Bioconductor* packages used

- [*Rsamtools*](#)
- [*GenomicRanges*](#)
- [*GenomicFeatures*](#)
- [*GenomicAlignments*](#)
- [*rtracklayer*](#)
- [*heatmaps*](#)

29.1. Background

Chromatin accessibility assays measure the extent to which DNA is open and accessible. Such assays now use high throughput sequencing as a quantitative readout. DNase assays, first using microarrays (Crawford, Davis, et al. 2006) and then DNase-Seq (Crawford, Holt, et al. 2006), requires a larger amount of DNA and is labor-intensive and has been largely supplanted by ATAC-Seq (Buenrostro et al. 2013).

The Assay for Transposase Accessible Chromatin with high-throughput sequencing (ATAC-seq) method maps chromatin accessibility genome-wide. This method quantifies DNA accessibility with a hyperactive Tn5 transposase that cuts and inserts sequencing adapters into regions of chromatin that are accessible. High throughput sequencing of fragments produced by the process map to regions of increased accessibility, transcription factor binding sites, and nucleosome positioning. The method is both fast and sensitive and can be used as a replacement for DNase *and* MNase.

An early review of chromatin accessibility assays (Tsompana and Buck 2014) compares the use cases, pros and cons, and expected signals from each of the most common approaches (Figure @ref(fig:chromatinAssays)).

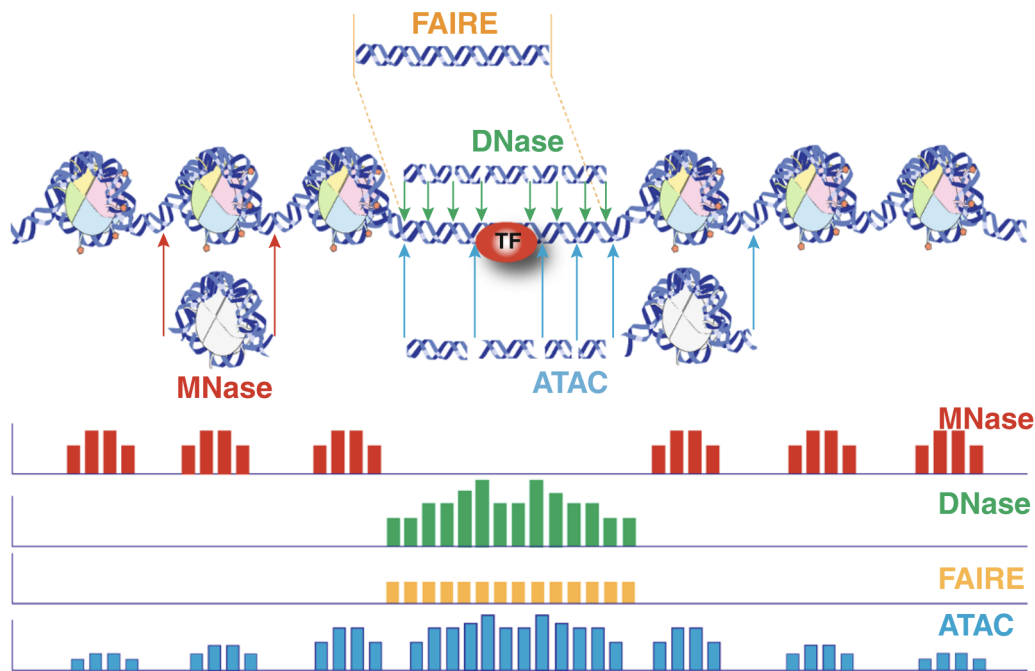


Figure 29.1.: Chromatin accessibility methods, compared. Representative DNA fragments generated by each assay are shown, with end locations within chromatin defined by colored arrows. Bar diagrams represent data signal obtained from each assay across the entire region. The footprint created by a transcription factor (TF) is shown for ATAC-seq and DNase-seq experiments.

The first manuscript describing ATAC-Seq protocol and findings outlined how ATAC-Seq data “line up” with other datatypes such as ChIP-seq and DNase-seq (Figure @ref(fig:green-leaf)). They also highlight how fragment length correlates with specific genomic regions and characteristics (Buenrostro et al. 2013, fig. 3).

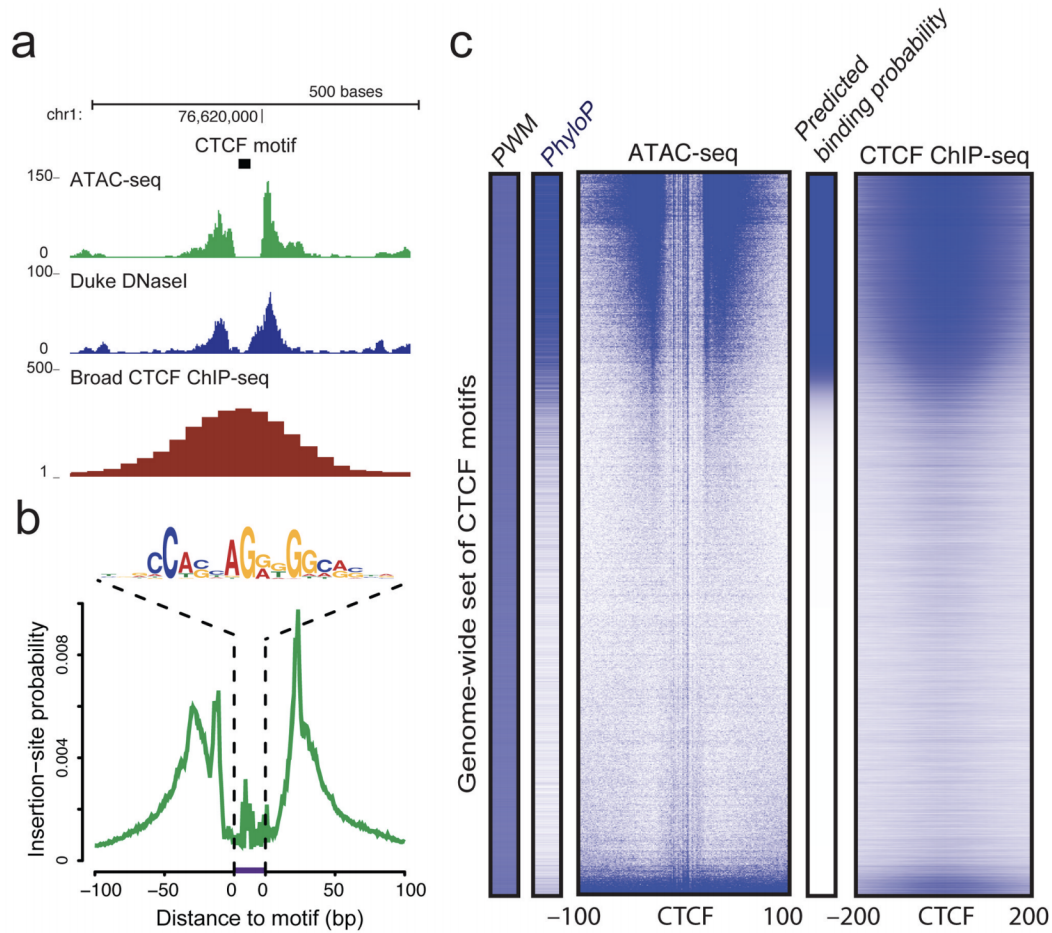


Figure 29.2.: Multimodal chromatin comparisons. From (Buenrostro et al. 2013), Figure 4. (a) CTCF footprints observed in ATAC-seq and DNase-seq data, at a specific locus on chr1. (b) Aggregate ATAC-seq footprint for CTCF (motif shown) generated over binding sites within the genome (c) CTCF predicted binding probability inferred from ATAC-seq data, position weight matrix (PWM) scores for the CTCF motif, and evolutionary conservation (PhyloP). Right-most column is the CTCF ChIP-seq data (ENCODE) for this GM12878 cell line, demonstrating high concordance with predicted binding probability.

Buenrostro et al. provide a detailed protocol for performing ATAC-Seq and quality control of results (Buenrostro et al. 2015). Updated and modified protocols that improve on signal-to-noise and reduce input DNA requirements have been described.

29.2. Informatics overview

ATAC-Seq protocols typically utilize paired-end sequencing protocols. The reads are aligned to the respective *genome* using **bowtie2**, **BWA**, or other short-read aligner. The result, after appropriate manipulation, often using **samtools**, results in a BAM file. Among other details, the BAM format includes columns for:

```
knitr::include_graphics('imgs/bam_shot.png')
```

NCI-02069526-ML:extdata sdavis2\$ samtools view Sorted_ATAC_21_22.bam head -10							
SRR891269.3150012	99	chr21	9411377	40	50M	=	9411438 111 CTCTGTTCCCTTGCTGACCTC
TTTGTCTATCCTTTTGCTGAGAGGTC			CCCCFFDFHHHHHJJJIIJJJJJJIGJIIFHIJJJJJIJJJJJJJJGH				HI:i:1 NH:i:1 NM:i
SRR891269.6181501	99	chr21	9411377	40	50M	=	9411438 111 CTCTGTTCCCTTGCTGACCTC
TTTGTCTATCCTTTTGCTGAGAGGTC			CCCCFFFFHHHHHJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJFIIJJHH				HI:i:1 NH:i:1 NM:i
SRR891269.13803406	73	chr21	9411423	0	46M4S	*	0 0 GGTCTGCTTAACCTTCCTTT
AGGTAGCTCGATTTTATGCTAAATCT			?81B;D>DFF<+C@E+ZAAC<33<CHB<9<+9B#####				HI:i:1 NH:i:1 NM:i
SRR891269.3150012	147	chr21	9411438	20	50M	=	9411377 -111 CTTTTAGTCAGGTAGCTCCA
ATGCTAAGCTTCTTAGTTGCTCACCT			JJJJJJJJJIIIIJJJJIGJIIJJIIJJIIJJIIHGJGJHHHHHFFED?@BB				HI:i:1 NH:i:1 NM:i
SRR891269.6181501	147	chr21	9411438	20	50M	=	9411377 -111 CTTTTAGTCAGGTAGCTCCA
ATGCTAAGCTTCTTAGTTGCTCACCT			JJJIGJHHHHHFFDDCCC				HI:i:1 NH:i:1 NM:i
SRR891269.52466482	81	chr21	9411533	0	8S42M	=	9411534 59 AAGAGACAGAAATTGCATTG
TACCGGCCCTTTATCAAGCCCTGGCC			JJIJJJJJJJJJJJJGIGGHCJIIHGDJIIJJJJJJJJHHHHHFFDD7FCCC				HI:i:1 NH:i:1 NM:i
SRR891269.52466482	161	chr21	9411534	0	41M9S	=	9411533 -59 AAATTGCATTGTTTCTACCG
TTTATCAAGCCCTGGCCCTGTCTCTT			@BCFFFFFFHHHHHJJJJJJIIJJJJJJIIIGIGIJJJIIJJJJIIHJJJJ				HI:i:1 NH:i:1 NM:i
SRR891269.8220653	163	chr21	9411566	13	50M	=	9411639 123 GCCTTGGCCACCATTGATAGT
AATTCCAATTGTTGTCTATGCAGGCC			CCCCFFFFHHHHHJJJIIJJJJIIJJJJIIJJJJJJGIIIIJJJJJJJJJE				HI:i:1 NH:i:1 NM:i
SRR891269.8220653	83	chr21	9411639	20	50M	=	9411566 -123 GCTACCATTTTCTTCTTAGC
TGCTCAGCAAATGTATCCAAATGAAA			EJJFHHHHFFFFFCCC				HI:i:1 NH:i:1 NM:i

Figure 29.3.: A BAM file in text form. The output of `samtools view` is the text format of the BAM file (called SAM format). Bioconductor and many other tools use BAM files for input. Note that BAM files also often include an index `.bai` file that enables random access into the file; one can read just a genomic region without having to read the entire file.

- sequence name (`chr1`)
- start position (integer)
- a *CIGAR* string that describes the alignment in a compact form
- the sequence to which the pair aligns
- the position to which the pair aligns
- a bit flag field that describes multiple characteristics of the alignment
- the sequence and quality string of the read
- additional tags that tend to be aligner-specific

Duplicate fragments (those with the *same* start and end position of other reads) are marked and likely discarded. Reads that fail to align “properly” are also often excluded from analysis. It is worth noting that most software packages allow simple “marking” of such reads and that there is usually no need to create a special BAM file before proceeding with downstream work.

After alignment and BAM processing, the workflow can switch to *Bioconductor*.

29.3. Working with sequencing data in Bioconductor

The *Bioconductor* project includes several infrastructure packages for dealing with ranges (sequence name, start, end, +/- strand) on sequences (Lawrence, Huber, Pagès, Aboyoun, Carlson, Gentleman, Martin T. Morgan, et al. 2013b) as well as capabilities with working with Fastq files directly (Morgan et al. 2016).

Table 29.1.: Commonly used Bioconductor and their high-level use cases.

Package	Use cases
<i>Rsamtools</i>	low level access to FASTQ, VCF, SAM, BAM, BCF formats
<i>GenomicRanges</i>	Container and methods for handling genomic regions
<i>GenomicFeatures</i>	Work with transcript databases, gff, gtf and BED formats
<i>GenomicAlignments</i>	Reader for BAM format
<i>rtracklayer</i>	import and export multiple UCSC file formats including BigWig and Bed

As noted in the previous section, the output of an ATAC-Seq experiment is a BAM file. As paired-end sequencing is a commonly-applied approach for ATAC-Seq, the `readGAlignmentPairs` function is the appropriate method to use.

30. Data import and quality control

```
library(GenomicAlignments)
```

Reading a paired-end BAM file looks a bit complicated, but the following code will:

1. Read the included BAM file.
2. Include read pairs only (`isPaired = TRUE`)
3. Include properly paired reads (`isProperPair = TRUE`)
4. Include reads with mapping quality ≥ 1
5. Add a couple of additional fields, `mapq` (mapping quality) and `isize` (insert size) to the default fields.

```
greenleaf <- readGAlignmentPairs(  
  "https://github.com/seandavi/RBiocBook/raw/main/atac-seq/extdata/Sorted_ATAC_21_22.bam"  
  param = ScanBamParam(  
    mapqFilter = 1,  
    flag = scanBamFlag(  
      isPaired = TRUE,  
      isProperPair = TRUE  
    ),  
    what = c("mapq", "isize")  
  )  
)
```

Exercise: What is the class of `greenleaf`? *Exercise:* Use the `GenomicAlignments::first()` accessor to get the first read of the pair as a `GAlignments` object. Save the result as a variable called `gl_first_read`. Use the `mcols` accessor to find the “metadata columns” of `gl_first_read`. *Exercise:* How many read pairs map to each chromosome?

We can make plot of the number of reads mapping to each chromosome.

30. Data import and quality control

```
library(ggplot2)
library(dplyr)
chromCounts <- table(seqnames(greenleaf)) %>%
  data.frame() %>%
  dplyr::rename(chromosome = Var1, count = Freq)
```

To keep things small, the example BAM file includes only chromosomes 21 and 22.

```
ggplot(chromCounts, aes(x = chromosome, y = count)) +
  geom_bar(stat = "identity") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

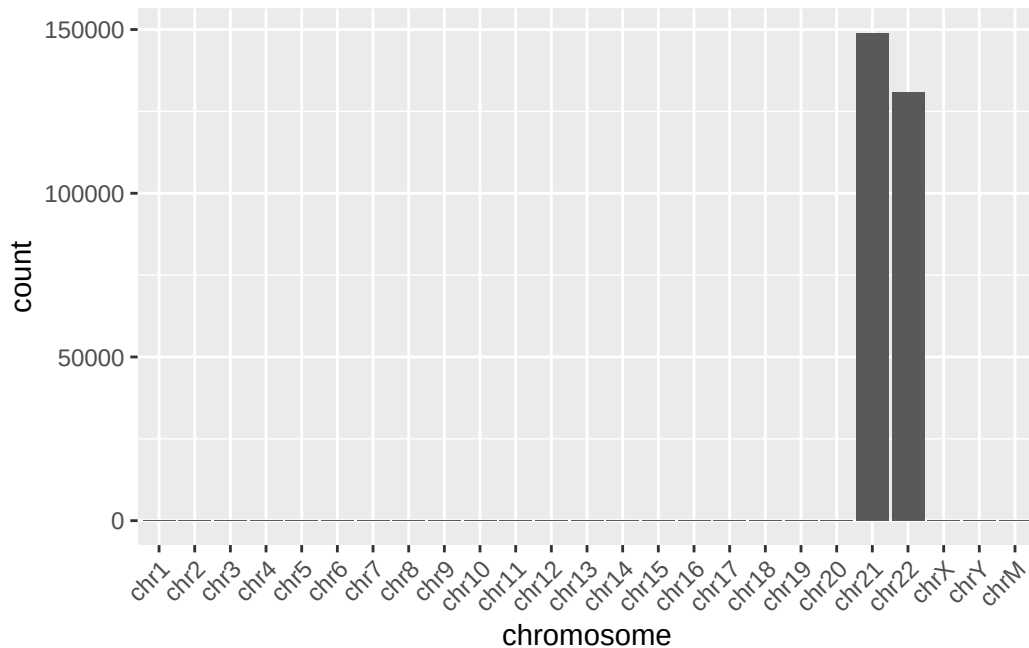


Figure 30.1.: Reads per chromosome. In our example data, we are using only chromosomes 21 and 22.

Normalizing by the chromosome length can yield the reads per megabase which should crudely be similar across all chromosomes.

```
chromCounts <- chromCounts %>%
  dplyr::mutate(readsPerMb = (count / (seqlengths(greenleaf) / 1e6)))
```

30. Data import and quality control

And show a plot. For two chromosomes, this is a little underwhelming.

```
ggplot(chromCounts, aes(x = chromosome, y = readsPerMb)) +  
  geom_bar(stat = "identity") +  
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +  
  theme_bw()
```

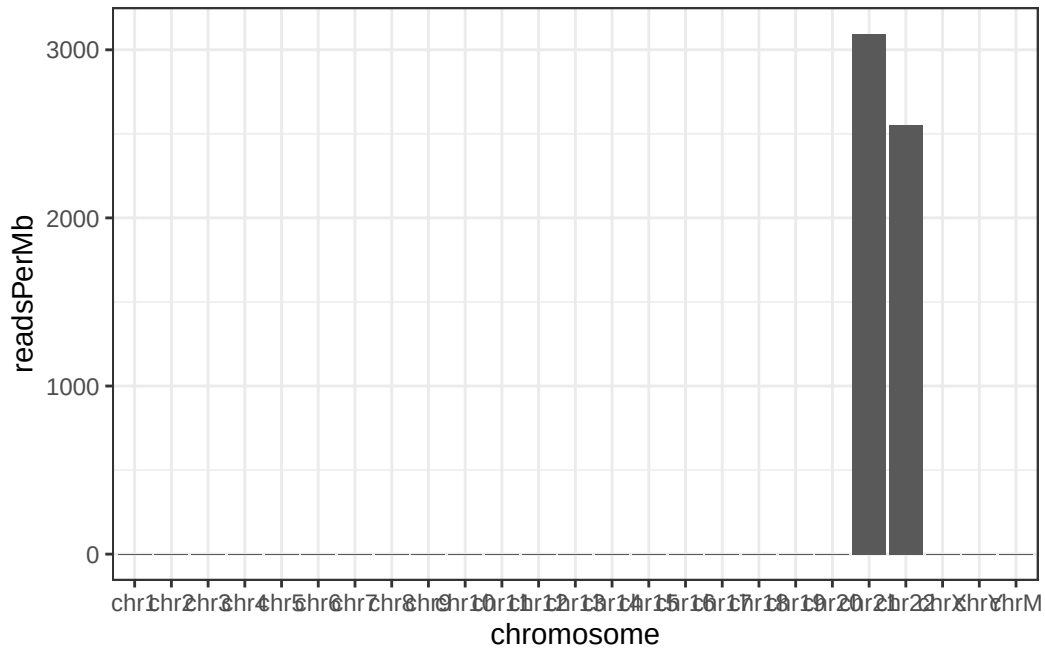


Figure 30.2.: Read counts normalized by chromosome length. This is not a particularly important plot, but it can be useful to see the relative contribution of each chromosome given its length.

30.1. Coverage

The `coverage` method for genomic ranges calculates, for each base, the number of overlapping features. In the case of a BAM file from ATAC-Seq converted to a `GAlignmentPairs` object, the coverage gives us an idea of the extent to which reads pile up to form peaks.

```
cvg <- coverage(greenleaf)  
class(cvg)
```


30. Data import and quality control

```
[1] "SimpleRleList"
attr(,"package")
[1] "IRanges"
```

The coverage is returned as a `SimpleRleList` object. Using `names` can get us the names of the elements of the list.

```
names(cvg)
```

```
[1] "chr1" "chr2" "chr3" "chr4" "chr5" "chr6" "chr7" "chr8" "chr9"
[10] "chr10" "chr11" "chr12" "chr13" "chr14" "chr15" "chr16" "chr17" "chr18"
[19] "chr19" "chr20" "chr21" "chr22" "chrX" "chrY" "chrM"
```

There is a name for each chromosome. Looking at the `chr21` entry:

```
cvg$chr21
```

```
integer-Rle of length 48129895 with 397462 runs
Lengths: 9411376      50      11      50 ...      36      14      28 10806
Values :      0      2      0      2 ...      1      2      1      0
```

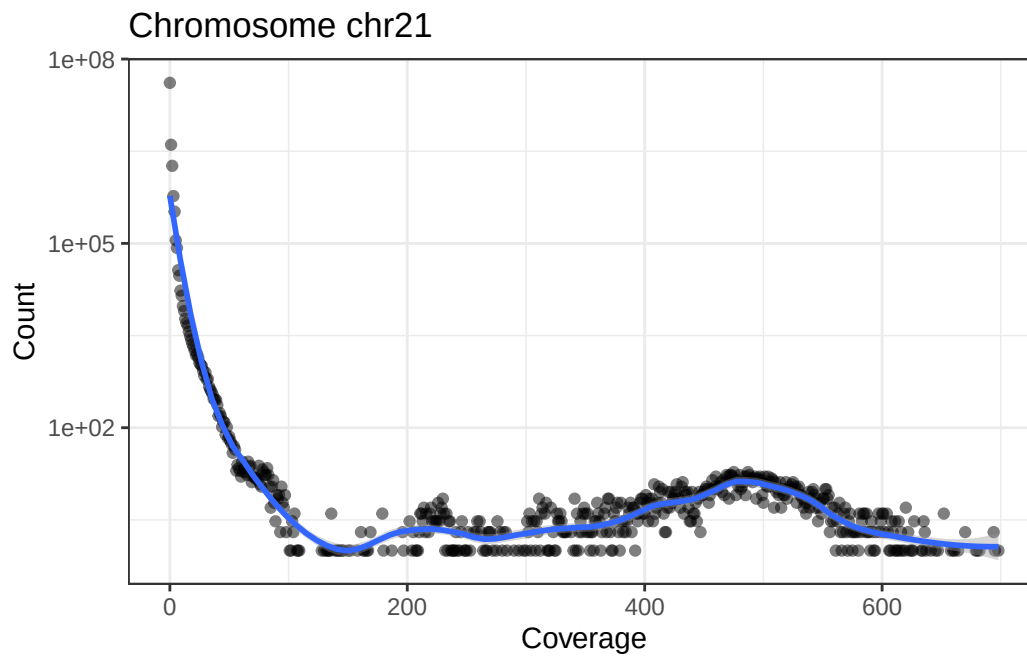
we see that each chromosome is represented as an `Rle`, short for run-length-encoding. Simply put, since along the chromosome there are many repeated values, we can recode the long vector as a set of (length: value) pairs. For example, if the first 9,410,000 base pairs have 0 coverage, we encode that as (9,410,000: 0). Doing that across the chromosome can very significantly reduce the memory use for genomic coverage.

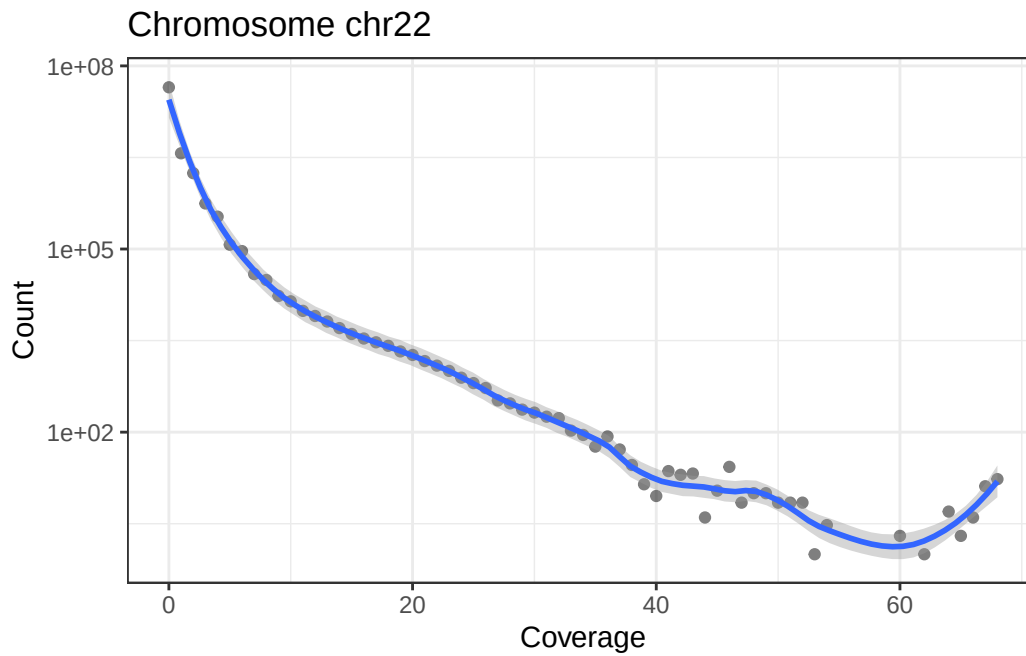
The following little function, `plotCvgHistByChrom` can plot a histogram of the coverage for a chromosome.

```
plotCvgHistByChrom <- function(cvg, chromosome) {
  library(ggplot2)
  cvgcounts <- as.data.frame(table(cvg[[chromosome]]))
  cvgcounts[, 1] <- as.numeric(as.character(cvgcounts[, 1]))
  colnames(cvgcounts) <- c("Coverage", "Count")
  ggplot(cvgcounts, aes(x = Coverage, y = Count)) +
    ggtitle(paste("Chromosome", chromosome)) +
    geom_point(alpha = 0.5) +
    geom_smooth(span = 0.2) +
```

30. Data import and quality control

```
scale_y_log10() +  
theme_bw()  
}  
for (i in c("chr21", "chr22")) {  
  print(plotCvgHistByChrom(cvg, i))  
}
```





30.2. Fragment Lengths

The first ATAC-Seq manuscript (Buenrostro et al. 2013) highlighted the relationship between fragment length and nucleosomes (see Figure @ref{fig:flgreenleaf}).

```
knitr::include_graphics("https://cdn.ncbi.nlm.nih.gov/pmc/blobs/0ad9/3959825/fde39a9fb288/n")
```

30. Data import and quality control

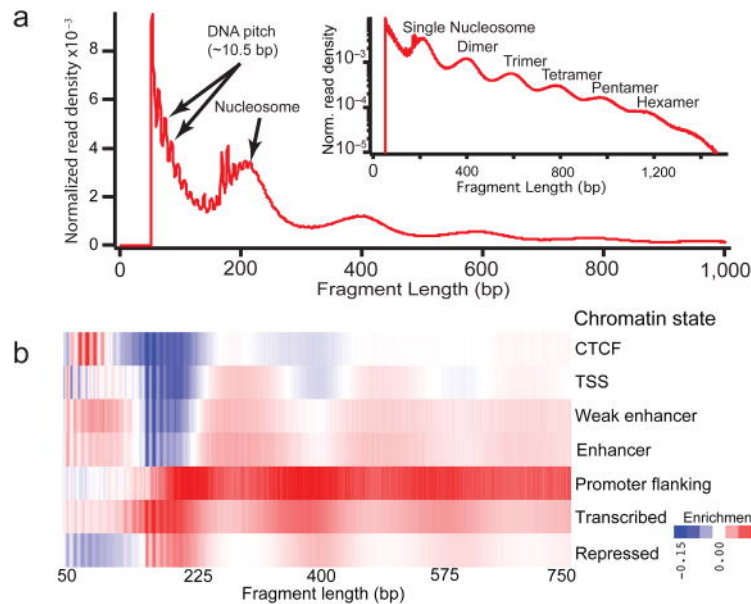


Figure 30.3.: Relationship between fragment length and nucleosome number.

Remember that we loaded the example BAM file with insert sizes (`isize`). We can use that “column” to examine the fragment lengths (another name for insert size). Also, note that the insert size for the `first` read and the `second` are the same (absolute value). Here, we will use `first`.

```
GenomicAlignments::first(greenleaf)
mcols(GenomicAlignments::first(greenleaf))
class(mcols(GenomicAlignments::first(greenleaf)))
head(mcols(GenomicAlignments::first(greenleaf))$isize)
fraglengths <- abs(mcols(GenomicAlignments::first(greenleaf))$isize)
```

We can plot the fragment length density (histogram) using the `density` function.

```
plot(density(fraglengths, bw = 0.05), xlim = c(0, 1000))
```

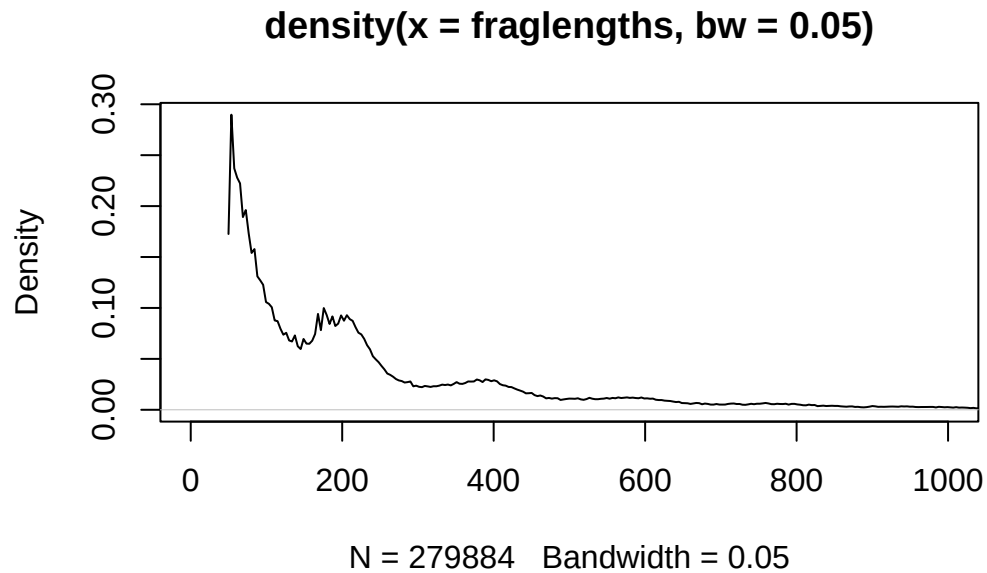
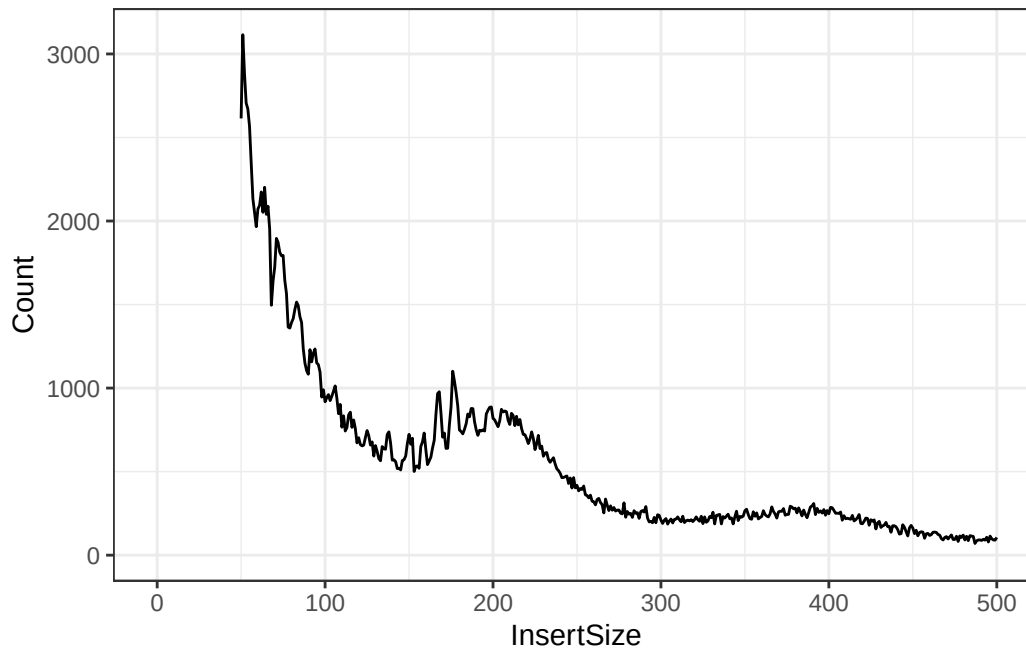


Figure 30.4.: Fragment length histogram.

And for fun, the ggplot2 version:

```
library(dplyr)
library(ggplot2)
fragLenPlot <- table(fraglengths) %>%
  data.frame() |>
  rename(
    InsertSize = fraglengths,
    Count = Freq
  ) |>
  mutate(
    InsertSize = as.numeric(as.vector(InsertSize)),
    Count = as.numeric(as.vector(Count))
  ) |>
  ggplot(aes(x = InsertSize, y = Count)) +
  geom_line()
print(fragLenPlot + theme_bw() + lims(x = c(-1, 500)))
```

30. Data import and quality control



Knowing that the nucleosome-free regions will have insert sizes shorter than one nucleosome, we can isolate the read pairs that have that characteristic.

```
gl_nf <- greenleaf[mcols(GenomicAlignments::first(greenleaf))$isize < 147]
```

And the mononucleosome reads will be between 147 and 294 base pairs for insert size/fragment length.

```
gl_mn <- greenleaf[mcols(GenomicAlignments::first(greenleaf))$isize > 147 &  
  mcols(GenomicAlignments::first(greenleaf))$isize < 294]
```

Finally, we expect nucleosome-free reads to be enriched near the TSS while mononucleosome reads should not be. We will use the [heatmaps](#) package to take a look at these two sets of reads with respect to the tss of the human genome.

```
library(TxDb.Hsapiens.UCSC.hg19.knownGene)  
proms <- promoters(TxDb.Hsapiens.UCSC.hg19.knownGene, 250, 250)  
seqs <- c("chr21", "chr22")  
seqlevels(proms, pruning.mode = "coarse") <- seqs # only chromosome 21 and 22
```

Take a look at the [heatmaps](#) package vignette to learn more about the heatmaps package capabilities.

30. Data import and quality control

```
library(heatmaps)
gl_nf_hm <- CoverageHeatmap(proms, coverage(gl_nf), coords = c(-250, 250))
label(gl_nf_hm) <- "NucFree"
scale(gl_nf_hm) <- c(0, 10)

gl_mn_hm <- CoverageHeatmap(proms, coverage(gl_mn), coords = c(-250, 250))
label(gl_mn_hm) <- "MonoNuc"
scale(gl_mn_hm) <- c(0, 10)
plotHeatmapMeta(list(gl_nf_hm, gl_mn_hm))
```

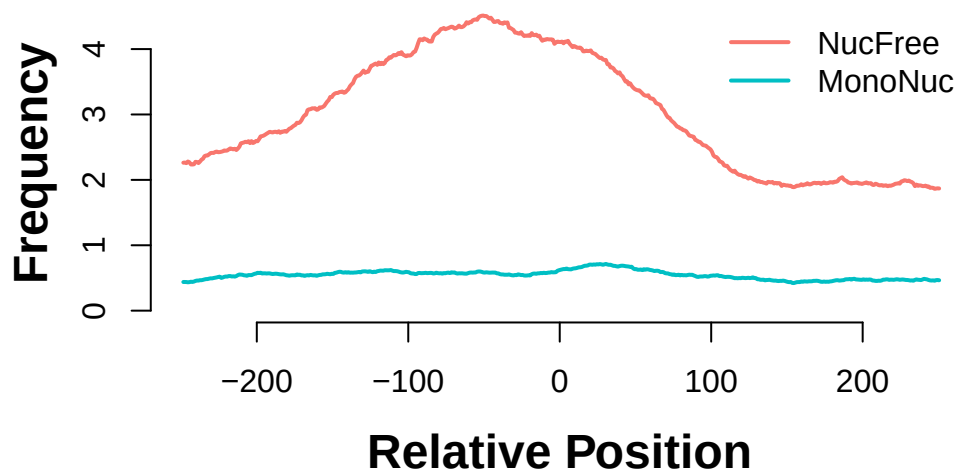


Figure 30.5.: Enrichment of nucleosome-free reads and depletion of mononucleosome reads around the TSS.

```
plotHeatmapList(list(gl_mn_hm, gl_nf_hm))
```

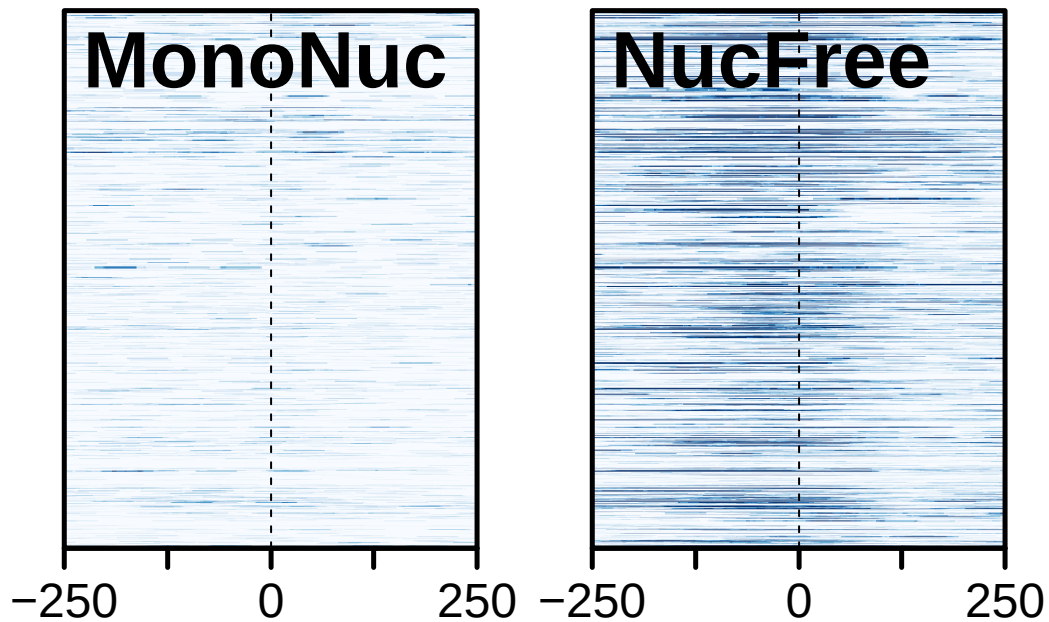


Figure 30.6.: Comparison of signals at TSS. Mononucleosome data on the left, nucleosome-free on the right. Both plots are scaled to the same range.

30.3. Viewing data in IGV

Install IGV from [here](#).

We export the greenleaf data as a BigWig file.

```
library(rtracklayer)
export.bw(coverage(greenleaf), "greenleaf.bw")
```

- *Exercise:* In IGV, choose `hg19`. Then, load the `greenleaf.bw` file and explore chromosomes 21 and 22.
- *Exercise:* Export the nucleosome-free portion of the data as a BigWig file and examine that in IGV. Where do you expect to see the strongest signals?

30.4. Additional work

For those working extensively on ATAC-Seq, there is a great workflow/tutorial available from Thomas Carroll:

MACS2

- https://rockefelleruniversity.github.io/RU_ATAC_Workshop.html

Feel free to work through it. In addition to the work above, there is also the *ATACseqQC* package vignette that offers more than just QC. At least a couple more packages are available in *Bioconductor*.

MACS2

The MACS2 package is a commonly-used package for calling peaks. Installation and other details are available¹.

```
pip install macs2
```

¹<https://github.com/taoliu/MACS>

31. Single Cell RNA-seq Analysis

31.1. Introduction

Single-cell RNA sequencing (scRNA-seq) is a powerful technique that allows us to measure gene expression in individual cells rather than bulk tissue samples. This capability has revolutionized our understanding of cellular heterogeneity, developmental processes, and disease mechanisms. Unlike traditional bulk RNA-seq, which provides an average expression profile across all cells in a sample, scRNA-seq reveals the unique molecular signatures of individual cells, enabling identification of distinct cell types, states, and trajectories.

This tutorial will walk through a typical scRNA-seq analysis workflow using R and the Bioconductor ecosystem. We'll cover data loading, quality control, normalization, dimensionality reduction, clustering, and visualization - the fundamental steps that transform raw sequencing data into biological insights.

31.2. Setup and Data Loading

31.2.1. Loading Essential Libraries

```
# Load essential libraries for single-cell RNA-seq analysis
library(scRNAseq)
library(scran)
library(scater)
```

We start by loading three essential R packages for single-cell analysis:

- **scRNAseq**: Provides access to curated single-cell RNA-seq datasets, making it easy to practice analysis techniques on real data. At the time of writing, it includes more than 50 datasets covering various tissues, organisms, and experimental conditions. See Risso and Cole (2024)

- **scran**: Contains methods for normalization, feature selection, and clustering specifically designed for single-cell data. See Lun et al. (2016).
- **scater**: Offers tools for quality control, visualization, and exploratory data analysis. See McCarthy et al. (2017).

Single-cell data has unique characteristics (sparsity, high dimensionality, technical noise) that require specialized tools. These packages form the backbone of the Bioconductor single-cell analysis ecosystem and provide methods that account for the specific challenges of single-cell data.

31.2.2. Exploring Available Datasets

We are going to use the `scRNAseq` package to access a curated collection of single-cell datasets. This package provides a convenient way to load and explore publicly available datasets that have been processed and annotated.

```
datasets <- scRNAseq::surveyDatasets()
```

The `surveyDatasets()` function queries the `scRNAseq` package database to show all available datasets. This creates a simple table of publicly available single-cell datasets that have been processed and made easily accessible.

Having access to well-curated datasets is useful for learning and benchmarking. These datasets come from published studies and represent different tissues, conditions, and experimental designs, allowing us to practice on real biological data without having to process raw sequencing files.

DataFrame with 10 rows and 15 columns

	name	version	path	object
	<character>	<character>	<character>	<character>
1	aztekin-tail-2019	2023-12-14	NA	single_cell_experiment
2	splicing-demonstrati..	2023-12-20	NA	single_cell_experiment
3	marques-brain-2016	2023-12-19	NA	single_cell_experiment
4	grun-bone_marrow-2016	2023-12-14	NA	single_cell_experiment
5	giladi-hsc-2018	2023-12-21	crispr	single_cell_experiment
6	giladi-hsc-2018	2023-12-21	rna	single_cell_experiment
7	macosko-retina-2015	2023-12-19	NA	single_cell_experiment
8	messmer-esc-2019	2023-12-19	NA	single_cell_experiment
9	ernst-spermatogenesi..	2023-12-21	cellranger	single_cell_experiment
10	ernst-spermatogenesi..	2023-12-21	emptydrops	single_cell_experiment

31. Single Cell RNA-seq Analysis

	title	description	taxonomy_id	genome	rows
	<character>	<character>	<List>	<List>	<integer>
1	Identification of a ..	Identification of a ..	8355	Xenla9.1	31535
2	[reprocessed, subset..	[reprocessed, subset..	10090	GRCm38	54448
3	Oligodendrocyte hete..	Oligodendrocyte hete..	10090	GRCm38	23556
4	De Novo Prediction o..	De Novo Prediction o..	10090	GRCm38	23536
5	Single-cell characte..	Single-cell characte..	10090	MGSCv37	30
6	Single-cell characte..	Single-cell characte..	10090	MGSCv37	27389
7	Highly Parallel Geno..	Highly Parallel Geno..	10090	GRCm38	24658
8	Transcriptional Hete..	Transcriptional Hete..	9606	GRCh38	58302
9	Staged developmental..	Staged developmental..	10090	GRCm38	33226
10	Staged developmental..	Staged developmental..	10090	GRCm38	32105
	columns	assays	column_annotations		
	<integer>	<List>	<List>		
1	13199	counts	sample,DevelopmentalStage,DaysPostAmputation,...		
2	2325	spliced,unspliced	celltype		
3	5069	counts	source_name,age,inferred cell type,...		
4	1915	counts	sample,protocol		
5	24070	counts	amplification.batch		
6	81408	counts	Amp_batch_ID,Seq_batch_ID,tier,...		
7	49300	counts	cluster		
8	1344	counts	Source Name,phenotype,single cell quality,...		
9	53510	counts	Sample,Barcode,Library,...		
10	68937	counts	Sample,Barcode,Library,...		
	reduced_dimensions	alternative_experiments			
	<List>	<List>			
1	UMAP				
2					
3					
4					
5					
6					
7					
8		ERCC			
9					
10					
	sources				
	<SplitDataFrameList>				
1	ArrayExpress:E-MTAB-7716:NA,PubMed:31097661:NA				
2	GEO:GSE109033:NA,GEO:GSM2928341:NA,SRA:SRR6459157:NA				
3	GEO:GSE75330:NA,PubMed:27284195:NA				

31. Single Cell RNA-seq Analysis

```
4                                GEO:GSE76983:NA,PubMed:27345837:NA
5                                PubMed:29915358:NA,GEO:GSE11349:NA
6                                PubMed:29915358:NA,GEO:GSE92575:NA
7  GEO:GSE63472:NA,PubMed:26000488:NA,URL:http://mccarrolllab...:2024-02-23
8                                PubMed:30673604:NA,ArrayExpress:E-MTAB-6819:NA
9                                ArrayExpress:E-MTAB-6946:NA,PubMed:30890697:NA
10                               ArrayExpress:E-MTAB-6946:NA,PubMed:30890697:NA
```

Each row represents a different study. The ‘columns’ field shows the number of cells, while ‘rows’ shows the number of genes. This helps you choose datasets appropriate for your computational resources and analysis goals. This little lightweight database is not the richest way to explore datasets, but it is a good starting point. In this tutorial, we will use the `ZeiselBrainData` dataset, which contains single-cell RNA-seq data from mouse brain tissue (Zeisel et al. (2015)).

Abstract: Zeisel Brain Dataset

The mammalian cerebral cortex supports cognitive functions such as sensorimotor integration, memory, and social behaviors. Normal brain function relies on a diverse set of differentiated cell types, including neurons, glia, and vasculature. Here, we have used large-scale single-cell RNA sequencing (RNA-seq) to classify cells in the mouse somatosensory cortex and hippocampal CA1 region. We found 47 molecularly distinct subclasses, comprising all known major cell types in the cortex. We identified numerous marker genes, which allowed alignment with known cell types, morphology, and location. We found a layer I interneuron expressing *Pax6* and a distinct postmitotic oligodendrocyte subclass marked by *Itpr2*. Across the diversity of cortical cell types, transcription factors formed a complex, layered regulatory code, suggesting a mechanism for the maintenance of adult cell type identity.

```
sce <- scRNAseq::ZeiselBrainData()
```

What is the `sce` object? It is a `SingleCellExperiment` object, which is a data structure specifically designed for single-cell RNA-seq data. As with other Bioconductor data structures, it is built on top of the `SummarizedExperiment` class (see Figure 31.1), which allows us to store not only the count matrix but also metadata about cells and features (genes).

Printing the `sce` object gives us a summary of the dataset:

```
sce
```

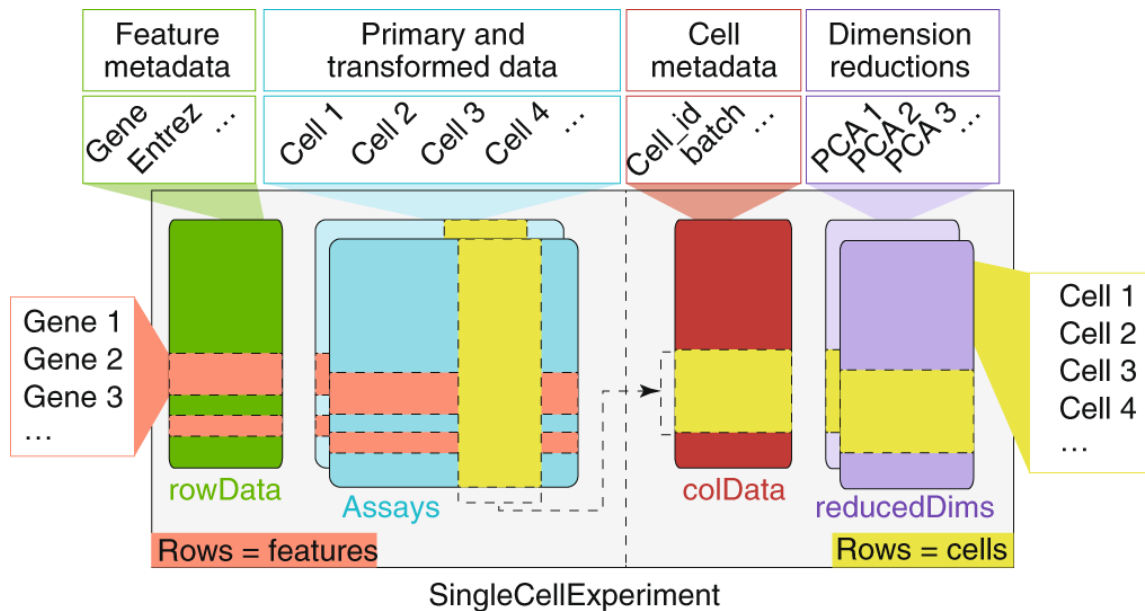


Figure 31.1.: The SingleCellExperiment object.

```

class: SingleCellExperiment
dim: 20006 3005
metadata(0):
assays(1): counts
rownames(20006): Tspan12 Tshz1 ... mt-Rnr1 mt-Nd41
rowData names(1): featureType
colnames(3005): 1772071015_C02 1772071017_G12 ... 1772066098_A12
               1772058148_F03
colData names(9): tissue group # ... level1class level2class
reducedDimNames(0):
mainExpName: gene
altExpNames(2): repeat ERCC

```

The `SingleCellExperiment` object contains:

- **Assays:** The main data matrix (counts) and any additional matrices (e.g., normalized counts).
- **Row metadata:** Information about genes (e.g., gene names, IDs).
- **Column metadata:** Information about cells (e.g., cell barcodes, tissue type, experimental conditions).

31. Single Cell RNA-seq Analysis

It **can** also contain additional information such as dimensionality reductions (e.g., PCA, t-SNE), clustering results, and quality control metrics.

In fact, we are going to add some of these additional information in the next steps. First, we add a new assay with the log-normalized counts, which is a common preprocessing step in single-cell RNA-seq analysis.

```
sce <- logNormCounts(sce)
```

The `logNormCounts()` function performs log-normalization of the count data. Raw count data from scRNA-seq experiments contains technical variation due to differences in sequencing depth between cells. Some cells might have been sequenced more deeply than others, leading to higher total counts that don't reflect true biological differences. Log-normalization addresses this by:

1. **Size factor normalization:** Scaling each cell's counts by a size factor that accounts for differences in sequencing depth. There are several methods to calculate size factors, but the most common is the median of ratios method, which divides each cell's counts by the median count for each gene across all cells. This helps to adjust for differences in library size.
2. **Log transformation:** Taking $\log(\text{count} + 1)$ to stabilize variance and make the data more normally distributed

For each gene g in cell i , the log-normalized expression becomes:

$$\log_2(\text{count}_{g_i} \times \text{sizefactor}_i + 1)$$

This log transformation is useful because the next steps in the downstream analyses (like clustering and dimensionality reduction) assume that the data is approximately normally distributed (or at least bell-shaped). Log-normalization helps achieve this by reducing the impact of outliers and making the data more suitable for PCA and visualization.

31.3. Dimensionality reduction

Single-cell datasets are extremely high-dimensional - typically containing 15,000-30,000 genes per cell. This creates several challenges: computational complexity, noise dominance, and visualization difficulties. Dimensionality reduction techniques help us identify the most informative patterns in the data while reducing noise and computational burden.

31.3.1. Feature Selection: Identifying Highly Variable Genes

Before applying dimensionality reduction, we need to identify the most informative genes. Highly variable genes (HVGs) are those that show significant variation across cells, indicating they may be biologically relevant. These genes are often more informative for downstream analyses like clustering and visualization.

Not all genes are equally informative for distinguishing cell types. Genes that show little variation across cells (like housekeeping genes) don't help us identify distinct cell populations. Highly variable genes are more likely to:

- Reflect biological differences between cell types
- Capture developmental or activation states
- Represent responses to environmental conditions

```
library(scran)
top_var_genes <- getTopHVGs(sce, n=2000)
```

The `getTopHVGs()` function identifies the top 2000 highly variable genes based on their variance across cells. Choosing a subset of highly variable genes reduces the dimensionality of the dataset while retaining the most informative features for downstream analyses.

The algorithm models the relationship between gene expression mean and variance, then identifies genes that show more variation than expected based on their expression level. This accounts for the fact that highly expressed genes naturally show more variance.

```
# Visualizing the highly variable genes
library(scran)
dec <- modelGeneVar(sce)

# Visualizing the fit:
fit.mv <- metadata(dec)
plot(fit.mv$mean, fit.mv$var, xlab="Mean of log-expression",
     ylab="Variance of log-expression", ylim=c(0, 4))
curve(fit.mv$trend(x), col="dodgerblue", add=TRUE, lwd=2)
points(
  fit.mv$mean[top_var_genes], fit.mv$var[top_var_genes],
  col="red", pch=16, cex=0.5
)
```

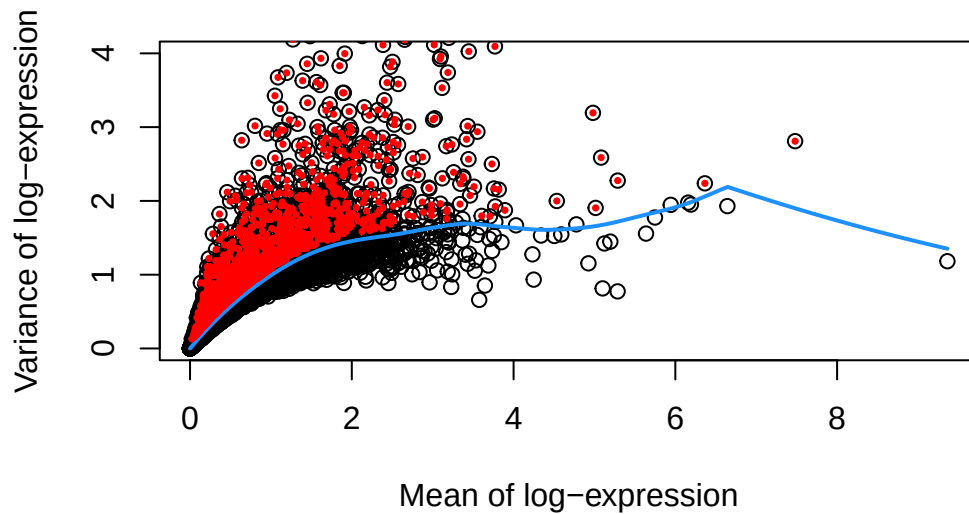



Figure 31.2.: A plot of the mean vs variance of log-expression for all the genes in the dataset, with the highly variable genes highlighted in red. Here, the x-axis represents the mean log-expression of each gene, and the y-axis represents the variance of log-expression. The blue curve represents the trend of variance as a function of mean expression, while the red points represent the highly variable genes.

Figure 31.2 shows the mean vs. variance of log-expression for all genes in the dataset, with highly variable genes highlighted in red. The blue curve represents the trend of variance as a function of mean expression, while the red points represent the highly variable genes.

We next proceed to calculating PCA using the subset of highly variable genes. PCA is a linear dimensionality reduction technique that finds the directions of maximum variance in the data. It's computationally efficient and provides a good foundation for further analysis. The first few principal components capture the most important patterns of variation in the dataset.

💡 Why PCA first?

PCA is a widely *linear* dimensionality reduction technique that:

- Reduces dimensionality while preserving variance
- Helps visualize high-dimensional data in 2D or 3D
- Serves as a preprocessing step for other methods like t-SNE or UMAP
- Is computationally efficient and easy to interpret

31. Single Cell RNA-seq Analysis

```
# Perform PCA on the highly variable genes
set.seed(100) # See below.
sce <- fixedPCA(sce, subset.row=top_var_genes)

reducedDimNames(sce)
```

```
[1] "PCA"
```

Note that we assign the output of `fixedPCA()` to the `sce` object. This function performs PCA on the highly variable genes we identified earlier, reducing the dimensionality of the dataset while retaining the most informative features. Because dimensionality reduction is a common step in single-cell RNA-seq analysis, we can store the PCA results in the `reducedDims` slot of the `SingleCellExperiment` object. This allows us to easily access and visualize the PCA coordinates later.

31.3.2. Non-linear Dimensionality Reduction with t-SNE

After PCA, we can apply non-linear dimensionality reduction techniques like t-SNE (t-distributed Stochastic Neighbor Embedding) to visualize the data in 2D or 3D. t-SNE is particularly useful for revealing local structure and separating distinct cell populations.

31.4. Why t-SNE after PCA

This is a two-step approach: 1. **PCA first:** Reduces noise and computational burden while preserving global structure 2. **t-SNE second:** Reveals local neighborhood structure and separates distinct cell populations

```
library(scater)
sce <- runTSNE(sce, dimred="PCA")
```

The `runTSNE()` function applies t-SNE to the PCA results, creating a 2D representation of the data. The `dimred="PCA"` argument specifies that we want to use the PCA coordinates as input for t-SNE.

```
nn.clusters <- clusterCells(sce, use.dimred = "TSNE")
table(nn.clusters)
```

```
nn.clusters
  1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20
152 126 190 286 131 191  63  87 113 328  88  97 155  45 138  40  71  42  61  92
 21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
 54  64  35  62  58  32  27  32  20  16  12  20  28  15  18  16
```

The `clusterCells()` function uses graph-based clustering to identify groups of similar cells. Graph-based clustering was [popularized by the Seurat package](#). It builds a nearest-neighbor graph and then finds communities within this graph. The resulting clusters often correspond to distinct cell types or states. The `use.dimred = "TSNE"` argument specifies that we want to use the t-SNE coordinates for clustering. This is an important choice because t-SNE captures local structure and can reveal distinct cell populations that may not be apparent in the original high-dimensional space. There are other clustering methods available, such as k-means or hierarchical clustering, but graph-based clustering is particularly effective for single-cell data due to its ability to handle complex, non-linear relationships.

31.4.1. Visualizing Cell Types and Clusters

In this particular dataset, we have pre-defined cell type annotations that we can use to color the t-SNE plot. The `level1class` and `level2class` columns in the metadata contain these annotations. We can use these annotations to color the t-SNE plot and visualize how well the clustering aligns with known cell types.

We can cluster the cells based on their t-SNE coordinates and visualize the results. The `plotReducedDim()` function from the `scater` package allows us to create a scatter plot of the t-SNE coordinates, coloring the points by their cluster assignments or cell type annotations.

```
library(scater)
colLabels(sce) <- nn.clusters
plotReducedDim(sce, "TSNE", colour_by="level1class")
```

31. Single Cell RNA-seq Analysis

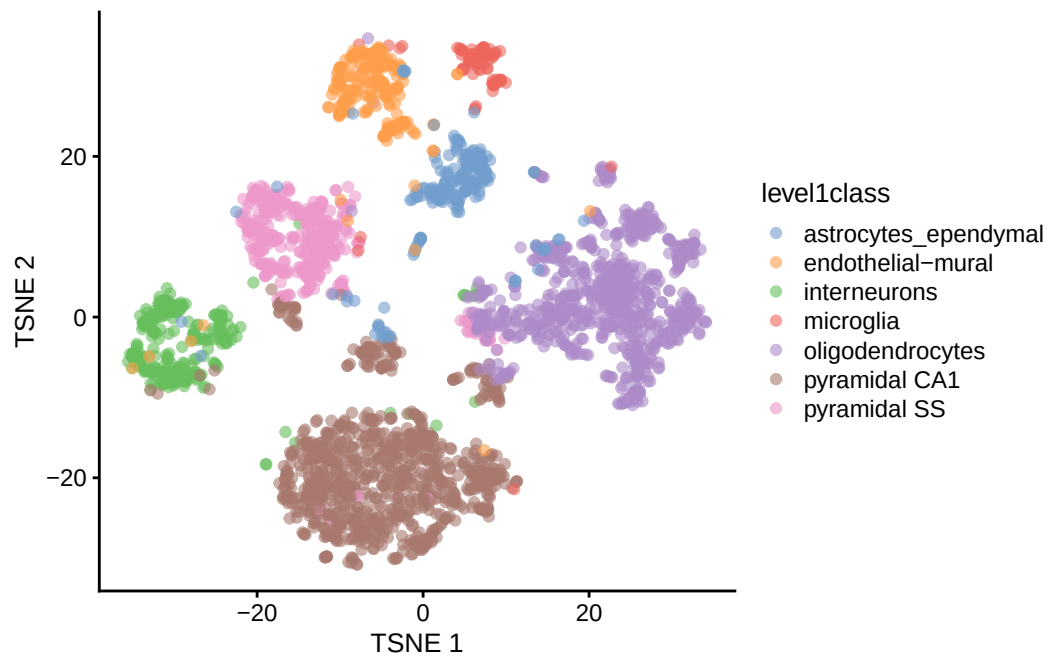


Figure 31.3.

We can also visualize the second level of cell type annotations.

```
plotReducedDim(sce, "TSNE", colour_by="level2class")
```

31. Single Cell RNA-seq Analysis

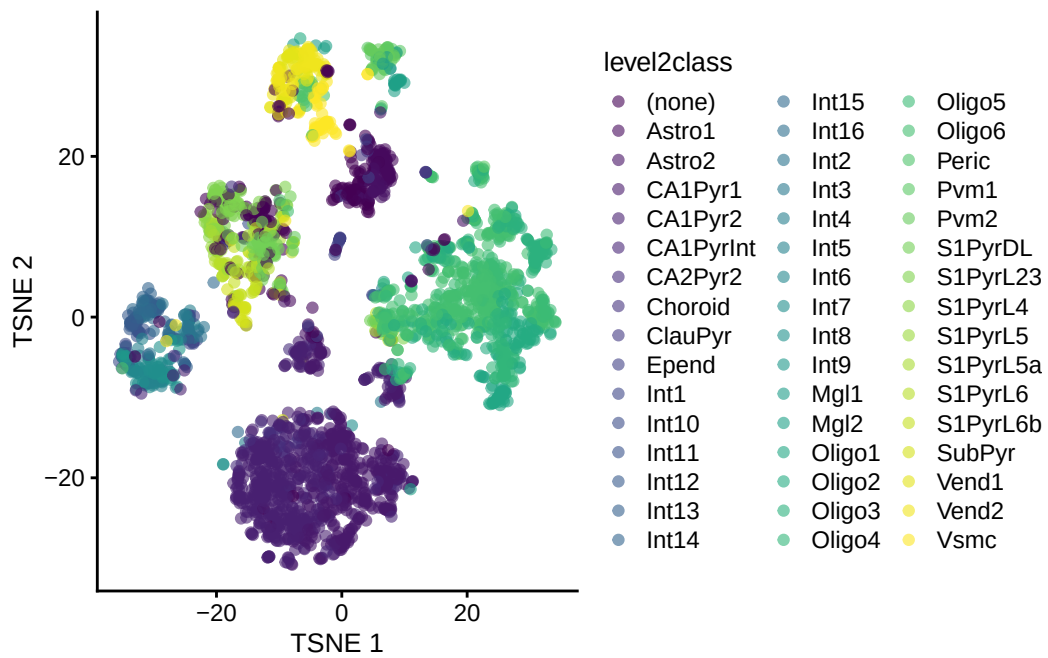


Figure 31.4.

And add additional metadata such as tissue type to the plot. The `shape_by` argument allows us to differentiate points by another categorical variable, such as tissue type.

```
plotReducedDim(sce, "TSNE", colour_by="level1class", shape_by="tissue", )
```

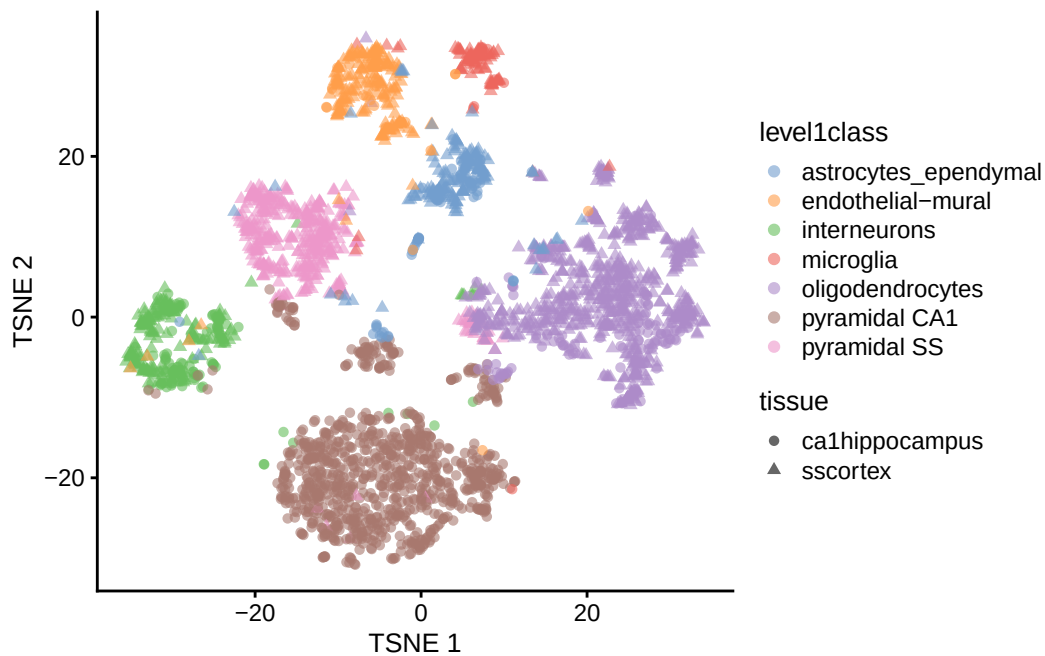


Figure 31.5.

31.5. Finding Marker Genes

Marker genes are genes that are differentially expressed in a specific cell type or cluster compared to others. Identifying marker genes helps us understand the molecular characteristics of each cell type and can provide insights into their functions.

To find marker genes, we can use the `findMarkers()` function from the `scran` package. This function performs pairwise differential expression analysis between clusters or cell types. We can specify the clusters we want to compare and the method for calculating differential expression.

```
markers <- findMarkers(sce,
  groups=as.numeric(as.factor(colData(sce)$level1class)), pval.type="all",
  direction="up", subset.row=top_var_genes, test="t")
```

The `findMarkers()` function identifies genes that are significantly upregulated in each cluster compared to all other clusters. The `pval.type="all"` argument specifies that we want to calculate p-values for all comparisons, and the `direction="up"` argument indicates that we are interested in genes that are upregulated in the specified clusters.

31.6. Interactive Visualization with iSEE

The [Interactive SummarizedExperiment Explorer \(iSEE\)](#) is a powerful tool for exploring single-cell data interactively. It allows us to visualize and interact with the data in real-time, making it easier to explore complex datasets and identify patterns.

A key component of iSEE is the `SingleCellExperiment` object, which serves as the data container for single-cell RNA-seq data. iSEE provides a user-friendly interface for exploring the data.

To use iSEE, we first need to install the iSEE package and then launch the iSEE app.

```
# Install iSEE package if not already installed
iSEE::iSEE(sce)
```

The `iSEE()` function launches the iSEE app, which provides an interactive interface for exploring the `SingleCellExperiment` object.

You may want to play with a few genes to see how they are expressed across the different clusters. You can also explore the metadata associated with the cells, such as tissue type, cell type annotations, and quality control metrics. The set of “marker genes” we identified earlier can also be visualized in iSEE, allowing us to see how these genes are expressed across different clusters and cell types.

For cluster 1, the marker genes are:

```
cat(paste(top_marker_genes_for_all_clusters[[1]], '\n'))
```

```
Aqp4
Cldn10
Clu
Aldoc
Ntsr2
Mt2
Fabp7
Gja1
Gpr3711
Prdx6
```

See the

31.7. Bonus: Working with 10x Genomics Data

In the previous sections, we used a curated dataset from the `scRNAseq` package. However, many real-world single-cell RNA-seq datasets come from 10x Genomics platforms, which produce data in specific formats. In this section, we'll cover a hypothetical workflow for loading and processing raw 10x Genomics data using R.

31.7.1. Understanding 10x Genomics Output Formats

10x Genomics Cell Ranger produces several output formats:

1. **Matrix Market format** (.mtx files): Sparse matrix format with separate files for the matrix, gene names, and cell barcodes
2. **HDF5 format** (.h5 files): A single compressed file containing all the data
3. **Filtered vs. unfiltered**: Cell Ranger provides both versions - filtered contains only cells that pass basic quality filters

31.7.2. Loading 10x Data from Matrix Market Files

```
# Loading from standard 10x output directory
# This assumes you have a directory with:
# - matrix.mtx.gz (or matrix.mtx)
# - features.tsv.gz (or genes.tsv.gz for older versions)
# - barcodes.tsv.gz

library(DropletUtils)
library(SingleCellExperiment)

# Method 1: Using DropletUtils (recommended)
sce_10x <- read10xCounts("path/to/10x/output/", col.names = TRUE)

# Method 2: Using Seurat (if you prefer the Seurat ecosystem)
# library(Seurat)
# seurat_obj <- Read10X("path/to/10x/output/")
# sce_10x <- as.SingleCellExperiment(CreateSeuratObject(seurat_obj))
```

What we're doing: The `read10xCounts()` function from `DropletUtils` reads the three files that comprise 10x output and creates a `SingleCellExperiment` object. The `col.names = TRUE` parameter ensures that cell barcodes are used as column names.

Why `DropletUtils`: This package is specifically designed for handling droplet-based single-cell data (like 10x Genomics). It includes specialized functions for:

- Reading 10x formats efficiently
- Distinguishing empty droplets from cells
- Handling ambient RNA contamination
- Quality control specific to droplet-based methods

31.7.3. Loading from HDF5 Files

```
# For HDF5 format files
library(rhdf5)

# Method 1: Using DropletUtils
sce_h5 <- read10xCounts("path/to/sample.h5", type = "HDF5")

# Method 2: Direct HDF5 reading (more control)
h5_file <- "path/to/sample.h5"
gene_info <- h5read(h5_file, "matrix/features")
barcodes <- h5read(h5_file, "matrix/barcodes")
matrix_data <- h5read(h5_file, "matrix/data")
# Additional processing needed to reconstruct sparse matrix
```

What we're doing: HDF5 files are more convenient as they contain all information in a single file. The `type = "HDF5"` parameter tells the function to expect this format instead of separate matrix files.

When to use H5 files:

- **Convenience:** Single file is easier to manage and transfer
- **Speed:** Often faster to read than multiple compressed files
- **Storage:** More efficient storage, especially for large datasets

31.7.4. Essential Quality Control After Loading Raw Data

```
# Add mitochondrial gene information
library(scuttle)
is.mito <- grepl("^MT-", rownames(sce_10x)) # Human mitochondrial genes
# For mouse data, use: is.mito <- grepl("^mt-", rownames(sce_10x))

# Calculate per-cell QC metrics
sce_10x <- addPerCellQC(sce_10x, subsets = list(Mito = is.mito))

# Calculate per-gene QC metrics
sce_10x <- addPerFeatureQC(sce_10x)

# View the QC metrics
colnames(colData(sce_10x))
```

What we're doing: We're calculating quality control metrics that are essential for raw 10x data:

- **Mitochondrial genes:** High mitochondrial gene expression often indicates dying cells
- **Per-cell metrics:** Total counts, number of detected genes, mitochondrial percentage
- **Per-gene metrics:** How many cells express each gene

Why QC is crucial for raw data: Unlike curated datasets, raw 10x data contains:

- **Empty droplets:** Droplets that captured ambient RNA but no cells
- **Dying cells:** Cells with compromised membranes
- **Doublets:** Droplets containing multiple cells
- **Low-quality cells:** Cells with very few detected genes

31.7.5. Filtering Low-Quality Cells and Genes

```
# Visualize QC metrics
library(scater)
plotColData(sce_10x, x = "sum", y = "detected", colour_by = "subsets_Mito_percent")

# Set filtering thresholds (these are examples - adjust based on your data)
```

```
# Filter cells
high_mito <- sce_10x$subsets_Mito_percent > 20 # Remove cells with >20% mitochondrial read
low_lib_size <- sce_10x$sum < 1000 # Remove cells with <1000 total counts
low_n_features <- sce_10x$detected < 500 # Remove cells with <500 detected genes

# Filter genes (remove genes expressed in very few cells)
low_expression <- rowSums(counts(sce_10x) > 0) < 10 # Expressed in <10 cells

# Apply filters
sce_filtered <- sce_10x[!low_expression, !(high_mito | low_lib_size | low_n_features)]

# Check how many cells and genes remain
dim(sce_10x) # Before filtering
dim(sce_filtered) # After filtering
```

What we're doing: We're applying filters to remove low-quality cells and rarely expressed genes. The specific thresholds should be adjusted based on your dataset characteristics.

Understanding the filters:

- **Mitochondrial percentage:** Cells with high mitochondrial gene expression are likely dying
- **Library size:** Total number of UMIs per cell - very low values suggest poor capture
- **Number of features:** How many different genes are detected - very low values suggest poor quality
- **Gene expression frequency:** Genes expressed in very few cells are often noise

Critical considerations:

- **Thresholds are dataset-dependent:** What works for one experiment may not work for another
- **Biological vs. technical variation:** Some cell types naturally have different RNA content
- **Visualization first:** Always plot your QC metrics before setting thresholds

31.8. Further Reading and Resources

For more in-depth information on single-cell RNA-seq analysis, consider the following resources:

31. Single Cell RNA-seq Analysis

- The [Orchestrating Single-Cell RNA-seq Analysis](#) book provides a comprehensive guide to single-cell RNA-seq analysis using Bioconductor.
- The [Seurat](#) package documentation offers extensive tutorials and vignettes for single-cell analysis in the Seurat ecosystem.

References

- Bourgon, Richard, Robert Gentleman, and Wolfgang Huber. 2010. “Independent Filtering Increases Detection Power for High-Throughput Experiments.” *Proceedings of the National Academy of Sciences* 107 (21): 9546–51. <https://doi.org/10.1073/pnas.0914005107>.
- Brouwer-Visser, Jurriaan, Wei-Yi Cheng, Anna Bauer-Mehren, et al. 2018. “Regulatory T-Cell Genes Drive Altered Immune Microenvironment in Adult Solid Cancers and Allow for Immune Contextual Patient Subtyping.” *Cancer Epidemiology, Biomarkers & Prevention* 27 (1): 103–12. <https://doi.org/10.1158/1055-9965.EPI-17-0461>.
- Buenrostro, Jason D, Paul G Giresi, Lisa C Zaba, Howard Y Chang, and William J Greenleaf. 2013. “Transposition of Native Chromatin for Fast and Sensitive Epigenomic Profiling of Open Chromatin, DNA-binding Proteins and Nucleosome Position.” *Nature Methods* 10 (12): 1213–18. <https://doi.org/10.1038/nmeth.2688>.
- Buenrostro, Jason D, Beijing Wu, Howard Y Chang, and William J Greenleaf. 2015. “ATAC-seq: A Method for Assaying Chromatin Accessibility Genome-Wide.” *Current Protocols in Molecular Biology* / Edited by Frederick M. Ausubel ... [Et Al.] 109 (January): 21.29.1–9. <https://doi.org/10.1002/0471142727.mb2129s109>.
- Center, Pew Research. 2016. “Lifelong Learning and Technology.” In *Pew Research Center: Internet, Science & Tech.* <https://www.pewresearch.org/internet/2016/03/22/lifelong-learning-and-technology/>.
- Crawford, Gregory E, Sean Davis, Peter C Scacheri, et al. 2006. “DNase-chip: A High-Resolution Method to Identify DNase I Hypersensitive Sites Using Tiled Microarrays.” *Nature Methods* 3 (7): 503–9. <http://www.ncbi.nlm.nih.gov/pubmed/16791207?dopt=AbstractPlus>.
- Crawford, Gregory E, Ingeborg E Holt, James Whittle, et al. 2006. “Genome-Wide Mapping of DNase Hypersensitive Sites Using Massively Parallel Signature Sequencing (MPSS).” *Genome Research* 16 (1): 123–31. <http://www.ncbi.nlm.nih.gov/pubmed/16344561?dopt=AbstractPlus>.

References

- DeRisi, J. L., V. R. Iyer, and P. O. Brown. 1997. “Exploring the Metabolic and Genetic Control of Gene Expression on a Genomic Scale.” *Science (New York, N.Y.)* 278 (5338): 680–86. <https://doi.org/10.1126/science.278.5338.680>.
- Greener, Joe G., Shaun M. Kandathil, Lewis Moffat, and David T. Jones. 2022. “A Guide to Machine Learning for Biologists.” *Nature Reviews Molecular Cell Biology* 23 (1): 40–55. <https://doi.org/10.1038/s41580-021-00407-0>.
- Knowles, Malcolm S., Elwood F. Holton, and Richard A. Swanson. 2005. *The Adult Learner: The Definitive Classic in Adult Education and Human Resource Development*. 6th ed. Elsevier.
- Lawrence, Michael, Wolfgang Huber, Hervé Pagès, Patrick Aboyoun, Marc Carlson, Robert Gentleman, Martin T. Morgan, et al. 2013a. “Software for Computing and Annotating Genomic Ranges.” *PLoS Computational Biology* 9 (8): e1003118. <https://doi.org/10.1371/journal.pcbi.1003118>.
- Lawrence, Michael, Wolfgang Huber, Hervé Pagès, Patrick Aboyoun, Marc Carlson, Robert Gentleman, Martin T Morgan, et al. 2013b. “Software for Computing and Annotating Genomic Ranges.” *PLoS Computational Biology* 9 (8): e1003118. <https://doi.org/10.1371/journal.pcbi.1003118>.
- Libbrecht, Maxwell W., and William Stafford Noble. 2015. “Machine Learning Applications in Genetics and Genomics.” *Nature Reviews Genetics* 16 (6): 321–32. <https://doi.org/10.1038/nrg3920>.
- Lun, Aaron T. L., Davis J. McCarthy, and John C. Marioni. 2016. “A Step-by-Step Workflow for Low-Level Analysis of Single-Cell RNA-Seq Data with Bioconductor.” *F1000Res*. 5: 2122. <https://doi.org/10.12688/f1000research.9501.2>.
- McCarthy, Davis J., Kieran R. Campbell, Aaron T. L. Lun, and Quin F. Willis. 2017. “Scater: Pre-Processing, Quality Control, Normalisation and Visualisation of Single-Cell RNA-Seq Data in R.” *Bioinformatics* 33: 1179–86. <https://doi.org/10.1093/bioinformatics/btw777>.
- Morgan, Martin, Herve Pages, V Obenchain, and N Hayden. 2016. “Rsamtools: Binary Alignment (BAM), FASTA, Variant Call (BCF), and Tabix File Import.” *R Package Version 1* (0): 677–89.
- Risso, Davide, and Michael Cole. 2024. *scRNAseq: Collection of Public Single-Cell RNA-Seq Datasets*. <https://doi.org/10.18129/B9.bioc.scRNAseq>.

References

- Student. 1908. “The Probable Error of a Mean.” *Biometrika* 6 (1): 1–25. <https://doi.org/10.2307/2331554>.
- Tsompana, Maria, and Michael J Buck. 2014. “Chromatin Accessibility: A Window into the Genome.” *Epigenetics & Chromatin* 7 (1): 33. <https://doi.org/10.1186/1756-8935-7-33>.
- Wasserstein, Ronald L., and Nicole A. Lazar. 2016. “The ASA Statement on p -Values: Context, Process, and Purpose.” *The American Statistician* 70 (2): 129–33. <https://doi.org/10.1080/00031305.2016.1154108>.
- Zeisel, Amit, Ana B. Muñoz-Manchado, Simone Codeluppi, et al. 2015. “Cell Types in the Mouse Cortex and Hippocampus Revealed by Single-Cell RNA-seq.” *Science* 347 (6226): 1138–42. <https://doi.org/10.1126/science.aaa1934>.

A. Appendix

A.1. Data Sets

- [BRFSS subset](#)
- [ALL clinical data](#)
- [ALL expression data](#)

A.2. Swirl

The following is from the [swirl website](#).

The swirl R package makes it fun and easy to learn R programming and data science. If you are new to R, have no fear.

To get started, we need to install a new package into R.

```
install.packages('swirl')
```

Once installed, we want to load it into the R workspace so we can use it.

```
library('swirl')
```

Finally, to get going, start swirl and follow the instructions.

```
swirl()
```

B. Git and GitHub

Git is a version control system that allows you to track changes in your code and collaborate with others. GitHub is a web-based platform that hosts Git repositories, making it easy to share and collaborate on projects. Github is NOT the only place to host Git repositories, but it is the most popular and has a large community of users.

You can use git by itself locally for version control. However, if you want to collaborate with others, you will need to use a remote repository, such as GitHub. This allows you to share your code with others, track changes, and collaborate on projects.

Note

It can be confusing to understand the difference between Git and GitHub. In short, Git is the version control system that tracks changes in your code, while GitHub is a platform that hosts your Git repositories and provides additional features for collaboration.

B.1. install Git and GitHub CLI

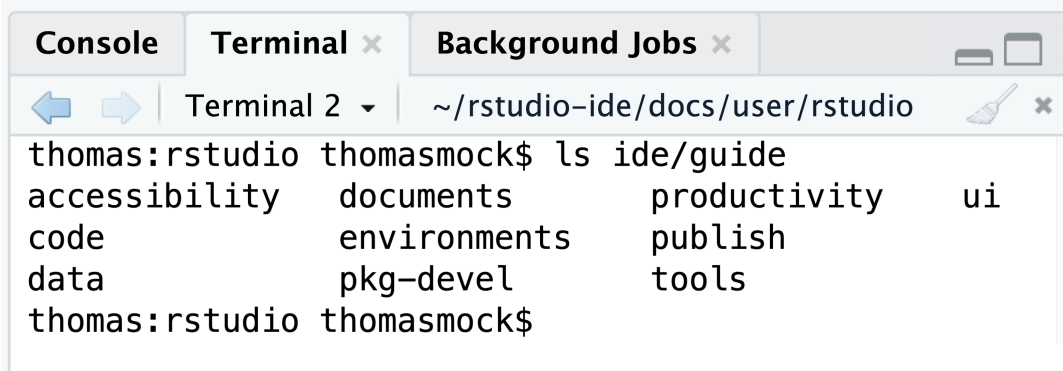
To use Git and GitHub, you need to have Git installed on your computer. You can download it from git-scm.com. After installation, you can check if Git is installed correctly by running the following command in your terminal:

```
git --version
```

We also need the `gh` command line tool to interact with GitHub. You can install it from cli.github.com. To install, go to [the releases page](#) and download the appropriate version for your operating system. For the Mac, it is the file named something like “Macos Universal” and the file will have a `.pkg` extension. You can install it by double-clicking the file after downloading it.

Using the RStudio Terminal

If you are using RStudio, you can use the built-in terminal to run Git commands. To open the terminal, go to the “Terminal” tab in the bottom pane of RStudio. This allows you to run Git commands directly from RStudio without needing to switch to a separate terminal application.



For more details, see the [RStudio terminal documentation](#).

B.2. Configure Git

After installing Git, you need to configure it with your name and email address. This information will be used to identify you as the author of the commits you make. Run the following commands in your terminal, replacing “Your Name” and “you@example.com” with your actual name and email address:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

B.3. Create a GitHub account

If you don’t already have a GitHub account, you can create one for free at github.com.

B.4. Login to GitHub CLI

After installing the GitHub CLI, you need to log in to your GitHub account. Run the following command in your terminal:

```
gh auth login
```

B.5. Introduction to Version Control with Git

Welcome to the world of version control! Think of Git as a “save” button for your entire project, but with the ability to go back to previous saves, see exactly what you changed, and even work on different versions of your project at the same time. It’s an essential tool for reproducible and collaborative research.

In this tutorial, we’ll learn the absolute basics of Git using the command line directly within RStudio.

B.5.1. Key Git Commands We’ll Learn Today:

- **git init:** Initializes a new Git repository in your project folder. This is the first step to start tracking your files.
 - **git add:** Tells Git which files you want to track changes for. You can think of this as putting your changes into a “staging area.”
 - **git commit:** Takes a snapshot of your staged changes. This is like creating a permanent save point with a descriptive message.
 - **git restore:** Discards changes in your working directory. It’s a way to undo modifications you haven’t committed yet.
 - **git branch:** Allows you to create separate timelines of your project. This is useful for developing new features without affecting your main work.
 - **git merge:** Combines the changes from one branch into another.
-

B.6. The Toy Example: An R Script

First, let's create a simple R script that we can use for our Git exercise. In RStudio, create a new R Script and save it as `data_analysis.R`.

```
# data_analysis.R

# Load necessary libraries
library(ggplot2)
library(dplyr)

# Create some sample data
data <- data.frame(
  x = 1:10,
  y = (1:10) ^ 2
)

# Initial data summary
summary(data)
```

B.7. Let's Get Started with Git!

Open the **Terminal** in RStudio (you can usually find it as a tab next to the Console). We'll be typing all our Git commands here.

B.7.1. Step 1: Initialize Your Git Repository

First, we need to tell Git to start tracking our project folder.

```
git init
```

You'll see a message like `Initialized empty Git repository in....` You might also notice a new `.git` folder in your project directory (it might be hidden). This is where Git stores all its tracking information. Your default branch is automatically named `main`.

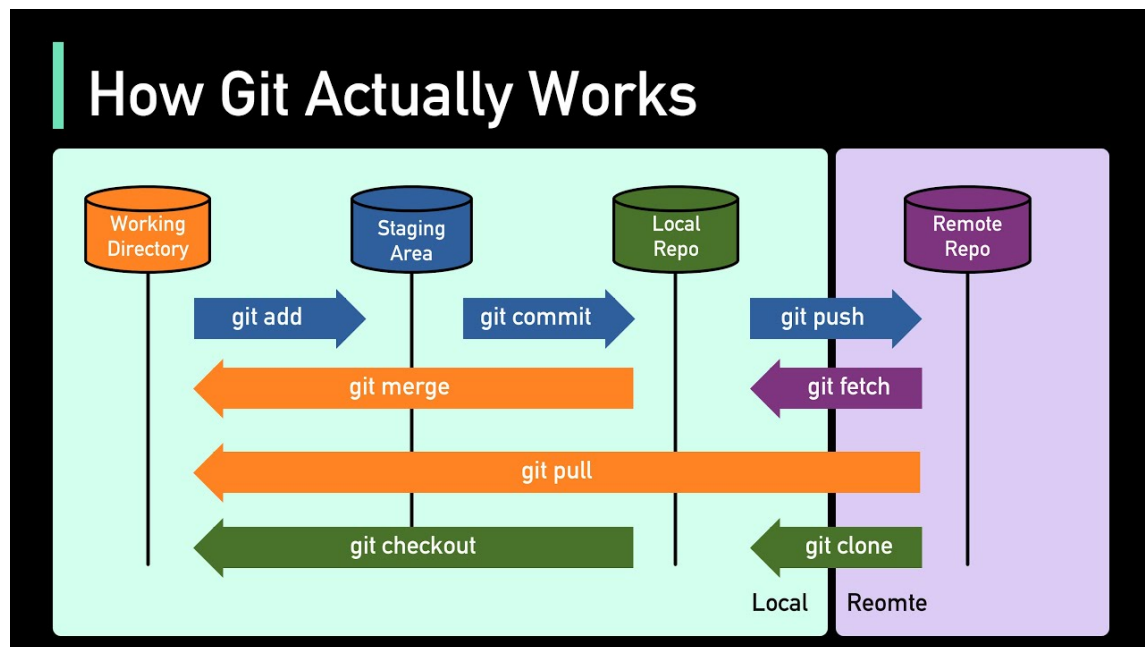


Figure B.1.: This is an overview of how git works along with the commands that make it tick. See [this video](#)

B.7.2. Step 2: Your First Commit

Now, let's add our `data_analysis.R` script to Git's tracking and make our first "commit."

1. Add the file to the staging area:

```
git add data_analysis.R
```

2. Commit the staged file with a message:

```
git commit -m "Initial commit: Add basic data script"
```

The `-m` flag lets you write your commit message directly in the command. Good commit messages are short but descriptive!

B.7.3. Step 3: Making and Undoing a Change

Let's modify our R script. Add a plotting section to the end of `data_analysis.R`.

```
# ... (keep the previous code)

# Create a plot
ggplot(data, aes(x = x, y = y)) +
  geom_point() +
  ggtitle("A Simple Scatter Plot")
```

Now, what if we decided we didn't want this change after all? We can use `git restore` to go back to our last committed version.

```
git restore data_analysis.R
```

If you look at your `data_analysis.R` file now, the plotting code will be gone!

B.7.4. Step 4: Branching Out

Branches are a powerful feature. Let's create a new branch to add our plot without messing up our `main` branch.

1. Create a new branch and switch to it:

```
git checkout -b add-plot
```

This is a shortcut for `git branch add-plot` and `git checkout add-plot`.

Now, re-add the plotting code to `data_analysis.R`.

```
# ... (keep the previous code)

# Create a plot
ggplot(data, aes(x = x, y = y)) +
  geom_point() +
  ggtitle("A Simple Scatter Plot")
```

Let's commit this change on our new `add-plot` branch.

```
git add data_analysis.R
git commit -m "feat: Add scatter plot"
```

B.7.5. Step 5: Seeing Branches in Action

Now for the magic of branches. Let's switch back to our `main` branch.

```
git checkout main
```

Now, open your `data_analysis.R` script in the RStudio editor. **The plotting code is gone!** That's because the change only exists on the `add-plot` branch. The `main` branch is exactly as we last left it.

Let's switch back to our feature branch.

```
git checkout add-plot
```

Check the `data_analysis.R` script again. **The plotting code is back!** This demonstrates how branches allow you to work on different versions of your project in isolation.

B.7.6. Step 6: Merging Your Work

Our plot is complete and we're happy with it. It's time to merge it back into our `main` branch to incorporate the new feature.

1. **Switch back to the main branch, which is our target for the merge:**

```
git checkout main
```

2. **Merge the `add-plot` branch into `main`:**

```
git merge add-plot
```

You'll see a message indicating that the merge happened. Now, your `main` branch has the updated `data_analysis.R` script with the plotting code!

C. Additional resources

- [Base R Cheat Sheet](#)
- [Modern Data Visualization with R](#)

C.1. AI

- [chatGPT](#)
- [Gemini](#)
- [Claude](#)
- [DeepSeek](#)
- [Perplexity](#)

D. AI Tools for Enhanced Learning and Productivity in Bioinformatics

The rise of sophisticated Artificial Intelligence (AI) tools, particularly Large Language Models (LLMs), has drastically altered day-to-day tasks for many. This chapter aims to guide you, as biologists venturing into these computational fields, on how to effectively and responsibly harness these AI tools. The focus will be on using them as powerful assistants to deepen your understanding of statistical concepts, enhance your ability to explore complex biological data with R, and ultimately boost your overall productivity and learning experience. Think of these tools not as a shortcut around learning, but as a versatile companion on your journey.

D.1. Introduction: AI as Your Bioinformatics Learning Companion

You’ve likely heard of tools like ChatGPT, Gemini, Claude, or GitHub Copilot. These are examples of LLMs and other AI-driven assistants that can process and generate human-like text, code, and even help with complex reasoning tasks. Their capabilities are particularly relevant when grappling with new programming languages like R, intricate statistical methods, or the challenge of deriving meaning from large biological datasets.

Table D.1.: Table AI Tools for Bioinformatics Learning

AI Tool	Description
ChatGPT	A conversational AI that can answer questions, explain concepts, and generate code snippets.
Gemini	Google’s AI that can assist with coding, data analysis, and more.
Claude	An AI assistant that can help with writing, coding, and problem-solving.

D. AI Tools for Enhanced Learning and Productivity in Bioinformatics

AI Tool	Description
GitHub Copilot	An AI-powered code completion tool that suggests code as you type, based on the context of your project.
Perplexity	An AI that can answer questions and provide explanations across various domains, including programming and data science.

Why should you consider integrating these AI tools into your learning workflow? The benefits are manifold. They can help demystify complex topics by offering alternative explanations, assist in drafting R code for specific analyses, aid in the often-frustrating process of debugging, summarize lengthy documents, and even help you brainstorm approaches for data exploration.

i A Supplement, Not a Replacement

It's crucial to remember that AI is here to *supplement* your learning and critical thinking, not replace it. Your growing expertise in biology, combined with a solid understanding of statistical principles, remains paramount. AI can help you get there faster and overcome hurdles, but the destination is your own deep understanding.

However, it's also important to approach these tools with realistic expectations. LLMs can sometimes provide inaccurate information—a phenomenon often called “hallucination”—or exhibit biases present in their training data.¹ Therefore, your domain knowledge and developing statistical intuition are your best guides in sifting the digital wheat from the chaff.

D.2. AI for Understanding Statistical Concepts and R Functions

One of the most immediate benefits of LLMs is their ability to act as a patient, tireless tutor for statistical concepts. If you find a particular statistical idea challenging, you can ask an LLM to explain it in various ways. For instance, you might prompt: “Explain the concept of a p-value as if you were talking to a biologist who is new to statistics,” or “Describe the key differences between Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE) when analyzing gene expression data.” The ability to request analogies or examples directly relevant to biological datasets can be incredibly helpful.

¹Always be prepared to critically evaluate and verify information from AI tools against trusted sources.

D. AI Tools for Enhanced Learning and Productivity in Bioinformatics

Similarly, when you're wrestling with the R language, AI tools can be invaluable. Perhaps you're unsure about the syntax for a `dplyr` verb or the arguments for a `ggplot2` function. You could ask, "How do I use the `dplyr::filter()` function in R to select rows from my dataframe where the 'gene_expression' column is greater than 100 and the 'treatment_group' column is 'Condition_A'?" or "Can you explain the main arguments of the `stats::t.test()` function in R and what its output signifies?" These tools can also help you discover R packages specifically designed for statistical tasks you have in mind.

When it comes to applying these concepts, AI can generate example R code. For common statistical tests like t-tests, ANOVAs, or chi-squared tests, you can request simple, reproducible examples, perhaps even with dummy data that you can then adapt.

Understand Before You Use

A word of caution is essential here: while AI can generate code, *you* are the scientist. Always strive to understand the code provided. Ensure it aligns with your research question, your data structure, and the assumptions of the statistical test you're performing. Blindly copying and pasting code without comprehension can lead to erroneous conclusions.

D.3. AI for Data Exploration and Visualization Strategies

Exploring a new biological dataset can sometimes feel like navigating uncharted territory. AI can act as a brainstorming partner in this process. You could describe your dataset—its variables, the type of data (e.g., RNA-seq counts, proteomics measurements, microbial abundances), and the underlying biological context—and then ask for suggestions on initial exploratory analyses. For example: "I have a dataset with normalized protein abundance levels for samples from cancerous and healthy tissues, along with patient metadata like age and disease stage. What are some initial plots I could make in R with `ggplot2` to explore potential patterns or differences?"

Speaking of `ggplot2` and other R visualization libraries, LLMs can be particularly adept at helping you craft the precise code for the plots you envision. You can request code for specific plot types, ask for help customizing aesthetics like colors, labels, and themes, or even troubleshoot why your plot isn't rendering as expected. "How can I add a title to my `ggplot`, change the x-axis label to 'Gene Length (bp)', and use a different color palette for my groups?" is a typical query an LLM can handle.

AI can also offer tentative interpretations of statistical outputs or visual patterns. However, this is where your scientific judgment is most critical.

AI Suggestions are Starting Points

While an LLM might suggest that a particular cluster in your PCA plot *could* represent a distinct biological condition, this is merely a hypothesis generated by the model. It is your responsibility to correlate this with your biological knowledge, experimental design, and further statistical validation. AI interpretations are starting points for your own deeper investigation, not final conclusions.

D.4. AI for Boosting Productivity and Troubleshooting

Beyond conceptual understanding and data exploration, AI tools can significantly enhance your day-to-day productivity. For instance, code generation and autocompletion features, whether in standalone LLMs or integrated into development environments (like GitHub Copilot, if available to you), can speed up the writing of boilerplate R code or complete lines of code you’ve started. This is especially useful for repetitive tasks, freeing you up to focus on the more analytical aspects of your work. Remember, the goal is to accelerate your coding, not to have entire complex analyses generated without your deep involvement and understanding.

One of the most common frustrations in programming is debugging. AI can be a powerful ally here. You can paste R error messages directly into an LLM and ask for explanations and potential fixes. If your code isn’t behaving as expected, describe its intended function and the problematic output, and the AI may be able to pinpoint the issue.²

Other productivity boosts include asking AI to summarize lengthy R package documentation or even sections of research articles (always being mindful of access permissions and copyright). Furthermore, if you have a piece of R code that works but feels clunky or inefficient, you can ask an AI for suggestions on how to refactor it to make it more readable, efficient, or aligned with common R programming idioms.

D.5. Best Practices for Prompt Engineering

The quality of the output you receive from an LLM is heavily dependent on the quality of your input—your “prompt.” Effective prompt engineering is a skill that improves with practice.

²Providing a minimal reproducible example (a small piece of code with sample data that replicates the error) greatly improves the chances of getting useful debugging help from an AI.

D. AI Tools for Enhanced Learning and Productivity in Bioinformatics

The cornerstone of a good prompt is **specificity and context**. The more information you provide about your goal, your data, the biological question at hand, the specific R package you're using, or the exact error message you're encountering, the more tailored and useful the AI's response will be. Consider the difference:

Vague Prompt	Specific & Contextualized Prompt
"How to plot data in R?"	"How can I create a scatter plot in R using <code>ggplot2</code> to show the relationship between 'gene_expression_log2fc' and 'mirna_expression_log2fc' from my <code>results_df</code> dataframe, with points colored by 'cancer_subtype'?"
"My R code isn't working."	"I'm getting the error 'Error: object 'x' not found' in R when I run this code: <code>plot(x, y)</code> . I defined <code>x</code> in a previous chunk. Why might this be happening and how can I fix it?"

Don't expect the perfect answer on your first attempt. **Iterate and refine your prompts**. If the initial response isn't quite what you need, try rephrasing your question, adding more details, or breaking down a complex request into smaller parts. You can also **specify the desired output format**, such as "Provide the R code for this," "Explain this concept in three simple bullet points," or "List three potential statistical approaches for this problem." Sometimes, asking for multiple perspectives or alternative solutions can also be enlightening. Finally, don't hesitate to experiment with different LLMs if you have access to more than one, as their strengths can vary.

D.6. Ethical Considerations and Limitations

While AI tools offer tremendous potential, it's imperative to use them responsibly and be acutely aware of their limitations.

Accuracy and "Hallucinations": This is perhaps the most critical point. LLMs are designed to generate plausible text, but this text is not always factually correct or logically sound. They can "hallucinate" answers, code, or citations that seem convincing but are entirely fabricated.

You **must** critically evaluate all information and code generated by AI. Cross-reference with trusted sources like official R documentation, textbooks, peer-reviewed literature, and your own developing expertise. Never blindly trust AI-generated content, especially for scientific analysis.

Bias in AI Models: AI models learn from the vast corpus of text and code they are trained on. This training data can contain societal biases, which may then be reflected in the AI's

D. AI Tools for Enhanced Learning and Productivity in Bioinformatics

responses. Be mindful of how this could subtly influence suggestions, interpretations, or even the tone of explanations.

Data Privacy and Confidentiality: This is a paramount concern in research.

Protect Your Sensitive Data

Under no circumstances should you input sensitive, unpublished, or personally identifiable data into public LLM interfaces. Always assume that any data you input could be stored or used by the AI provider. Familiarize yourself with the data usage policies of any AI tool you employ. For your coursework and research, focus on using AI for conceptual understanding, code related to non-sensitive example datasets, or publicly available data.

Over-Reliance and Skill Atrophy: The convenience of AI can be seductive. However, relying on it too heavily can hinder the development of your own foundational R programming and statistical reasoning skills. Use AI as a tool to *aid* and *accelerate* your learning, not to *replace* the hard work of understanding. The ultimate goal is for *you* to become proficient, not for the AI to perform tasks for you.

Reproducibility: If you incorporate AI-generated code or insights into your analyses, ensure you thoroughly understand them, can justify their use, and document them meticulously for reproducibility. It can be good practice to note the prompts you used if the interaction significantly shaped your approach, treating it almost like a consultation.

The table below summarizes some key limitations and suggested mitigations:

Limitation	Description	Mitigation Strategies
Accuracy Issues (Hallucinations)	Generates plausible but incorrect/fabricated information or code.	Verify with trusted sources; test code thoroughly; use critical thinking.
Bias	Reflects biases present in training data.	Be aware of potential biases; seek diverse perspectives; critically evaluate fairness of suggestions.
Data Privacy	Risk of exposing sensitive data if input into public tools.	Avoid inputting sensitive/unpublished data; use institutional/private instances if available and vetted; understand terms of service.

D. AI Tools for Enhanced Learning and Productivity in Bioinformatics

Limitation	Description	Mitigation Strategies
Lack of True Understanding	Manipulates patterns in data, doesn't "understand" concepts like humans do.	Don't attribute human-like reasoning; use it for specific tasks, not overarching scientific judgment.
Over-Reliance	Can hinder development of personal skills if used as a crutch.	Focus on learning principles; use AI to assist, not replace, your own effort; regularly practice without AI.
Reproducibility Challenges	AI responses can vary; difficult to document exact "reasoning" of the AI.	Document prompts used for significant code/ideas; thoroughly understand and vet any AI-generated code you use.

D.7. The Future: AI in Bioinformatics and Data Science

The field of AI is evolving at a breathtaking pace, and its integration into bioinformatics and data science is only set to deepen. We may see more sophisticated AI-powered tools for hypothesis generation, experimental design, drug discovery, and personalized medicine. As future biologists and data scientists, cultivating an adaptable mindset and committing to continuous learning will be key to navigating and leveraging these advancements effectively. Staying curious about new tools, while maintaining a strong foundation in scientific principles and ethics, will serve you well.

D.8. Practical Exercises and Activities

To help you get comfortable and skilled in using AI tools, we encourage you to try the following:

1. **Conceptual Clarification:** Choose a statistical concept from this course that you find somewhat challenging (e.g., "false discovery rate," "cross-validation," or "the assumptions of linear regression"). Use an LLM to ask for an explanation tailored to a biologist. Note down your prompt and the AI's response. Then, try to re-explain the concept in your own words to a classmate, drawing from (but not merely repeating) the AI's explanation.
2. **R Code Generation & Adaptation:** Imagine you have a data frame in R named `rna_seq_data` with columns `gene_id`, `sample_name`, `normalized_count`, and `treatment_group` (with levels "control" and "treated"). Ask an LLM to generate R code using

D. AI Tools for Enhanced Learning and Productivity in Bioinformatics

`dplyr` and `tidyr` to calculate the average `normalized_count` for each `gene_id` within each `treatment_group`, and then spread the data so that ‘control’ and ‘treated’ averages are in separate columns for each gene. Once you have the code, test it with some sample data (you can ask the AI to generate that too!) and ensure you understand each step.

3. **Debugging Practice:** Take a piece of R code. First, try to understand and debug it yourself. Then, paste the code and the error message into an LLM and ask for help. Compare the AI’s suggestions to your own troubleshooting. Did it help you find the solution faster? Did it explain the error well?
4. **Visualization Brainstorming:** Provide a clear description of a biological dataset, ideally one of your own (e.g., “A dataset containing measurements of 5 different cytokine levels in blood samples taken from patients at 3 time points post-vaccination, with patients also categorized by age group (young, middle, old).”). Use an LLM to brainstorm 3-4 different types of visualizations you could create using `ggplot2` to explore this data. For each visualization, describe what insights it might provide.

As you work through these exercises, make it a habit to document the prompts you use and critically assess the AI’s responses. Reflect on when the AI was most helpful, when its advice was off-base, and how you can refine your prompting strategy for better results in the future. This reflective practice is key to becoming a discerning and effective user of AI in your scientific endeavors.

E. Data Visualization with ggplot2

Start with this worked example to get a feel for the `ggplot2` package.

- <https://rkabacoff.github.io/datavis/IntroGGPLOT.html>

Then, for more detail, I refer you to this excellent [ggplot2 tutorial](#).

Finally, for more R graphics inspiration, see the [R Graph Gallery](#).

F. Matrix Exercises

F.1. Data preparation

For this set of exercises, we are going to rely on a dataset that comes with R. It gives the number of sunspots per month from 1749-1983. The dataset comes as a `ts` or time series data type which I convert to a matrix using the following code.

Just run the code as is and focus on the rest of the exercises.

```
data(sunspots)
sunspot_mat <- matrix(as.vector(sunspots),ncol=12,byrow = TRUE)
colnames(sunspot_mat) <- as.character(1:12)
rownames(sunspot_mat) <- as.character(1749:1983)
```

F.2. Exercises

- After the conversion above, what does `sunspot_mat` look like? Use functions to find the number of rows, the number of columns, the class, and some basic summary statistics.

```
ncol(sunspot_mat)
nrow(sunspot_mat)
dim(sunspot_mat)
summary(sunspot_mat)
head(sunspot_mat)
tail(sunspot_mat)
```

- Practice subsetting the matrix a bit by selecting:
 - The first 10 years (rows)
 - The month of July (7th column)
 - The value for July, 1979 using the rowname to do the selection.

F. Matrix Exercises

```
sunspot_mat[1:10,]  
sunspot_mat[,7]  
sunspot_mat['1979',7]
```

These next few exercises take advantage of the fact that calling a univariate statistical function (one that expects a vector) works for matrices by just making a vector of all the values in the matrix.

- What is the highest (max) number of sunspots recorded in these data?

```
max(sunspot_mat)
```

- And the minimum?

```
min(sunspot_mat)
```

- And the overall mean and median?

```
mean(sunspot_mat)  
median(sunspot_mat)
```

- Use the `hist()` function to look at the distribution of all the monthly sunspot data.

```
hist(sunspot_mat)
```

- Read about the `breaks` argument to `hist()` to try to increase the number of breaks in the histogram to increase the resolution slightly. Adjust your `hist()` and `breaks` to your liking.

```
hist(sunspot_mat, breaks=40)
```

Now, let's move on to summarizing the data a bit to learn about the pattern of sunspots varies by month or by year.

- Examine the dataset again. What do the columns represent? And the rows?

```
# just a quick glimpse of the data will give us a sense  
head(sunspot_mat)
```

- We'd like to look at the distribution of sunspots by month. How can we do that?

F. Matrix Exercises

```
# the mean of the columns is the mean number of sunspots per month.  
colMeans(sunspot_mat)  
  
# Another way to write the same thing:  
apply(sunspot_mat, 2, mean)
```

- Assign the month summary above to a variable and summarize it to get a sense of the spread over months.

```
monthmeans = colMeans(sunspot_mat)  
summary(monthmeans)
```

- Play the same game for years to get the per-year mean?

```
ymeans = rowMeans(sunspot_mat)  
summary(ymeans)
```

- Make a plot of the yearly means. Do you see a pattern?

```
plot(ymeans)  
# or make it clearer  
plot(ymeans, type='l')
```